

Java Full-Stack Interview Guide

In-Depth Answers with Internals & Follow-ups

Core Java · Java 8 Streams · Spring Boot · JPA/Hibernate · Security & JWT ·
Microservices · Kafka · SQL · Angular · JUnit · Project & HR

166 unique questions · 25 diagrams · 2 October 2026

Contents

A. Core Java

- A1. What is the use of Docker?
- A2. What are the main features introduced in Java 8, and why were they introduced...
- A3. What are the important features introduced in Java 18, and how are they used?
- A4. Can we override the main method in Java? Explain why or why not.
- A5. Explain the main OOP concepts in Java.
- A6. How do you achieve abstraction in Java? Write a class structure/program to d...
- A7. Why were functional interfaces introduced, what is one, and how do you defi...
- A8. If a functional interface already has one abstract method and another abstrac...
- A9. What is the use of default methods in interfaces? (incl. diamond problem res...
- A10. What is the use of static methods in interfaces?
- A11. Can we override private methods?
- A12. Can we override static methods? Can we override a static method in an inter...
- A13. What is method hiding?
- A14. What is serialization? How do you prevent a variable from being serialized?
- A15. Explain the volatile keyword.
- A16. What is garbage collection and how is it used?
- A17. Output of `System.out.println(Double.MIN_VALUE > 0.0d)` and why?
- A18. If you add a null to an empty Set, what is the size?
- A19. Can we call one constructor from another? How?
- A20. Explain composition and aggregation with examples, and how you'd use co...
- A21. What is polymorphism? Real project example.
- A22. Explain method overriding with a runtime polymorphism example (rules).
- A23. Class A has a public method; class B overrides it as private. Will it work? Why?
- A24. Difference between HashMap and Hashtable.
- A25. Internal working of HashMap.
- A26. How are HashMap collisions handled?
- A27. `hashCode()`, `equals()`, `toString()` in the context of HashMap.
- A28. What happens when custom objects are used as HashMap keys?
- A29. HashMap vs ConcurrentHashMap.
- A30. Java memory areas: Heap, Stack, Metaspace, PC register, native stack.
- A31. Explain the Java Collection Framework.
- A32. Concurrent collections and concurrent access in Java.
- A33. `wait()` vs `sleep()`.
- A34. The Callable interface (Future, ExecutorService, CompletableFuture).
- A35. `execute()` vs `submit()`.
- A36. How do you create a thread? Extending Thread vs implementing Runnable.
- A37. How did you connect Java with Python?
- A38. SOLID principles with a small Java example for each.
- A39. Monolithic vs microservice architecture.
- A40. Which declarations are correct and why?
- A41. Which array declarations are valid and why?

B. Java 8, Streams, Optional, Lambdas & Coding Problems

- B1. Explain Java 8 features
- B2. What is a lambda expression?
- B3. Functional interfaces with respect to lambdas
- B4. Method references
- B5. Stream API methods: intermediate vs terminal
- B6. `map()` vs `flatMap()`
- B7. Debugging streams
- B8. Optional: null-safe access, chaining with Stream, anti-patterns
- B9. Sequential vs parallel streams
- B10. Count duplicate elements in an array
- B11. Find even numbers from a list
- B12. Check occurrence of a character
- B13. Count first-character occurrences
- B14. Remove duplicates while maintaining order
- B15. Remove null and empty strings
- B16. Convert list of strings to uppercase
- B17. Join strings into CSV
- B18. Sum using `reduce()`
- B19. `List<String>` to `Map<String,Integer>` (string to length) and the duplicate key...
- B20. Group employees by department
- B21. Sort employees by salary with Comparator
- B22. Second-highest number in a list (stream)
- B23. Method taking `int[]` returning the second-largest DISTINCT element
- B24. Flatten a list of lists with `flatMap`
- B25. Count by category with `groupingBy` + counting
- B26. Sort a map by values (into a `LinkedHashMap`)
- B27. Filter by multiple conditions (Predicate composition)
- B28. `partitioningBy` even/odd
- B29. Duplicate keys in `Collectors.toMap()`
- B30. Caching with `ConcurrentHashMap.computeIfAbsent` (and the recursive-upd...
- B31. Custom collector joining strings in uppercase
- B32. Stream a large file line by line
- B33. Generic method using functional interfaces
- B34. CSV -> filter -> map -> collect pipeline
- B35. Longest string with `max(Comparator.comparingInt(String::length))`
- B36. "this is test" -> "THIS IS TEST" with streams
- B37. "Highest substring"
- B38. First non-repeating character
- B39. Remove duplicate letters: lexicographically smallest result (LeetCode 316)
- B40. Rotated sorted list: find a target with modified binary search
- B41. Merge Sort, Quick Sort, Bubble Sort, Linear Search, Binary Search

C. Spring Boot, Spring Framework, JPA/Hibernate & Testing

- C1. What is `@SpringBootApplication`?
- C2. What happens if `@EnableAutoConfiguration` (or equivalent) is not specified?
- C3. `@Configuration` vs `@Bean`; why configuration classes (proxyBeanMethods, C...
- C4. What is a Bean; how is a Bean created internally?
- C5. Bean life cycle
- C6. Bean scopes; `@Scope`
- C7. `@Lazy`
- C8. `@Component`, stereotypes, component scanning
- C9. Dependency injection and its types
- C10. Multiple beans of the same type
- C11. JPA vs Hibernate vs Spring Data JPA
- C12. `@GeneratedValue` strategies

C13. @RequestParam, @RequestBody, @PathVariable and handling request para...
C14. REST controllers
C15. Full skeleton for getEmployeeById
C16. PUT vs PATCH
C17. HTTP 204
C18. Actuator
C19. Front Controller pattern (DispatcherServlet flow)
C20. Exception handling
C21. @ConfigurationProperties
C22. Different environments (profiles, config precedence, Spring Cloud Config, se...
C23. SessionFactory: what it is and how to configure it
C24. Multiple data sources

D. Security, Microservices & System Design

D1. Authentication vs authorization in Java/Spring Boot
D2. Implementing authentication & authorization: classes, configuration, flow
D3. Handling exceptions in authentication & authorization
D4. Pre-authorization (method-level security)
D5. How JWT works
D6. Explain the authentication layer in your project
D7. Security layers for REST communication; practices in Spring Boot; securing int...
D8. What are microservices; key things to take care of
D9. Microservices architecture of your project
D10. How microservices communicate + tools
D11. Circuit breaker
D12. Retry mechanism

E. SQL, Angular/JavaScript, Project & HR

E1. Find the second-highest salary, and the name and salary of the employee(s) ...
E2. Find the employee(s) with the highest salary in each department
E3. List the name and salary of employees in each department
E4. Window functions
E5. ACID properties with a bank-transfer example, plus isolation levels and anom...
E6. Stored procedures: what, why, the hierarchy of database objects, parameters, ...
E7. Tables A and B have the same records. What does an INNER JOIN return?
E8. Fetch 20 records starting from the 5th row of users (first and last name), acros...
E9. INNER JOIN between two tables, compared with the other join types
E10. Highest receiver amount (from the transactions table)
E11. Explain Angular
E12. Your Angular work, roles and responsibilities, and introducing yourself as an ...

C25. Lazy vs eager loading
C26. One-to-Many across multiple tables
C27. Audit logs
C28. URL shortener design
C29. What does LOC mean?
C30. @Transactional
C31. @Cacheable, caching performance, and cache consistency across services
C32. JUnit 5 annotations and Mockito annotations
C33. Writing tests for Spring Boot apps
C34. Testing REST APIs
C35. Handling exceptions in test cases

D13. If Microservice B is down, what should A do?
D14. API Gateway
D15. Load Balancer & Application Load Balancer
D16. Distributed caching & distributed tracing
D17. Kafka & working flow
D18. One microservice you worked on
D19. Do microservices connect directly or via shared services?
D20. Complete API request flow through microservices (with operations and ann...
D21. API works internally but not for the client: investigation checklist
D22. Slow API: primary investigation
D23. Delay from a stored procedure / SQL: identify and fix
D24. Design patterns in microservices + code-level patterns

E13. Basic JavaScript concepts
E14. var, let, const and hoisting
E15. Tell me about yourself
E16. Roles and responsibilities
E17. Explain your project (e.g. a Trinity Mobility Smart City product)
E18. Architecture and codebase structure
E19. Technologies and tools used (including those apart from Java, Spring Boot a...
E20. Coding standards, best practices and precautions while coding
E21. Challenges faced
E22. Handling conflicts
E23. Integrating third-party data
E24. The microservices concept in your project, and how you handle microservice...
E25. "Given a list of strings, write..." (truncated question)

Detailed answers with internals, edge cases and likely follow-ups. For quick revision, see [Interview-Guide-Concise.md](#).

How to use this guide

The original question bank had ~330 questions, many repeated across the topic and company sections. They have been **deduplicated into ~165 unique questions** grouped into five sections:

Section	Topic
A	Core Java (OOP, collections, HashMap internals, multithreading, memory, tricky output questions)
B	Java 8+, Streams, Optional, Lambdas, and coding/DSA programs
C	Spring Boot, Spring Framework, JPA/Hibernate, caching, transactions, JUnit/testing
D	Security (JWT, Spring Security), Microservices, Kafka, System Design, troubleshooting
E	SQL, Angular/JavaScript, Project/Architecture and HR questions

Answers to "your project" and HR questions are **templates with [placeholders]** . Replace them with your real work before the interview.

Company question index

Each company question maps to its answer ID, e.g. [C15](#) is section C, question 15.

TCS

#	Question	Answer
1	Configure different environments in Spring Boot	C22
2	Skeleton for getEmployeeById	C15
3	What are REST controllers used for	C14
4	Occurrence of a character using Stream API	B12
5	Java 8 features	A2 B1
6	Stream API methods	B5
7	Functional interfaces w.r.t. lambda	B3
8	Java memory: Heap and Stack	A30
9	Collection Framework	A31
10	Search target in rotated list 4,5,6,7,0,1,2,3	B40
11	Merge/Quick/Bubble Sort, Linear/Binary Search	B41
12	What is a Bean	C4
13	Two beans with the same name	C10
14	No @EnableAutoConfiguration	C2
15	map() vs flatMap()	B6

Company 2

#	Question	Answer
1	Internal working of HashMap	A25
2	HashMap collisions	A26
3	hashCode>equals/toString in HashMap	A27
4	Stack and Heap	A30
5	Callable interface	A34
6	ACID	E5
7	Three beans of same type, use one	C10
8	Second-highest salary SQL	E1
9	Constructor calling another constructor	A19
10	Which declarations are correct (I/A/B/C)	A40
11	First-character count (guava, grapes...)	B13
12	HashMap Employee("Anshul") output	A28
13	Valid array declarations	A41
14	Composition & aggregation	A20
15	Java 8 features	A2
16	Method reference	B4
17	Default methods	A9
18	Intermediate operations	B5

#	Question	Answer
19	HTTP 204	C17
20	Microservices in your project	D9 , E24
21	HashMap vs Hashtable	A24
22	Authentication layer in your project	D6
23	How JWT works	D5
24	Garbage collection	A16
25	Method hiding	A13
26	SOLID	A38
27	@Lazy	C7
28–29	@Component, component scanning	C8
30	Exception handling in Spring Boot	C20
31	JPA	C11
32	Bean scope	C6
33–35	@RequestParam / @RequestBody / @PathVariable	C13
36	@GeneratedValue	C12
37	20 records from 5th row	E8
38	INNER JOIN query	E9

Hexaware

#	Question	Answer
1	Authentication vs authorization	D1
2	Exceptions in authn/authz	D3
3	INNER JOIN on identical tables	E7
4	Microservices, circuit breaker, communication	D8 , D11 , D10
5	Kafka and its flow	D17
6	Pre-authorization	D4
7	Stored procedures	E6
8	@Transactional	C30
9	Distributed caching & tracing	D16
10	Slow API investigation	D22
11	API Gateway	D14
12	API works internally, not for client	D21
13	One microservice you worked on	D18
14	Retry mechanism	D12
15	Microservice B is down	D13
16	Debugging a stream	B7
17	flatMap() example	B6
18	Architecture walkthrough	D9 , E18
19–20	Load Balancer / ALB	D15
21	Second-largest distinct element	B23
22	Composition in a real project	A20
23	Polymorphism real example	A21

#	Question	Answer
24	Overriding & runtime polymorphism	A22
25	Authn/authz implementation	D2
26–27	@Cacheable, caching & DB performance	C31
28	Microservices architecture implementation	D9
29	How two microservices communicate	D10
30	Cache consistency across instances	C31
31	Highest receiver amount SQL	E10
32	Complete API request flow	D20
33	Direct vs shared-service integration	D19

G2Healthcare

#	Question	Answer
1	OOP concepts	A5
2–3	Abstraction + program	A6
4	volatile	A15
5	Concurrent collections / access	A32
6–7	wait() vs sleep()	A33
8	Spring Boot scopes	C6
9–11	@ConfigurationProperties (custom)	C21
12	Types of DI	C9
13	Request parameters	C13
14	Exception handling	C20
15	Handling microservices architecture	D9 , E24
16	Design patterns in microservices	D24
17–18	Angular introduction, roles	E12
19–20	Project architecture, codebase structure	E18
21, 23	Technologies & tools	E19
22	Precautions / best practices	E20
24	Roles and responsibilities	E16
25	Tell me about yourself	E15
26	<i>(truncated in source)</i> "Given a list of strings, write..."	E25

A. Core Java

A1. What is the use of Docker?

Docker packages an application together with everything it needs to run (JRE, OS libraries, config, the app itself) into an **image**. An image is a standard, unchangeable, layered artifact. A **container** is a running instance of that image. Docker fixes the "works on my machine" problem: the same image runs the same way on a laptop, in CI, on a VM, or in Kubernetes.

How it works under the hood

- Containers are **not VMs**. They share the host kernel and are isolated using Linux kernel features:
 - Namespaces** isolate what a process can see: `pid`, `net`, `mnt`, `uts`, `ipc`, `user`.
 - cgroups** limit what a process can use: CPU, memory, I/O.
 - A **union filesystem** (overlay2) stacks read-only image layers and adds a thin writable layer per container.
- Each Dockerfile instruction (`FROM`, `COPY`, `RUN`) creates a cached layer. Put rarely-changing layers (dependencies) before often-changing ones (your jar) so rebuilds are fast.
- On Windows and macOS, Docker Desktop runs a lightweight Linux VM (WSL2 / HyperKit) because containers need a Linux kernel.

Aspect	Container	Virtual Machine
Isolation	Process level (shared kernel)	Hardware level (own kernel via hypervisor)
Startup	Milliseconds to seconds	Tens of seconds to minutes
Size	MBs (e.g. 80–250 MB for a JRE image)	GBs
Density	Hundreds per host	Tens per host
Security boundary	Weaker (kernel shared)	Stronger

Typical uses in a Spring Boot / microservices project

- Package each microservice (traffic-service, incident-service, etc.) as its own image.
- Local development with `docker compose`: Postgres, Kafka, Redis and Keycloak start with one command.
- CI/CD: build the image once, then promote the *same* image through dev → QA → prod. Only env vars and config change between environments.
- Orchestration: Kubernetes/OpenShift schedules containers, scales them, and restarts them when they fail.
- Integration tests with **Testcontainers**, which start a real Postgres/Kafka in Docker for each test run.

Multi-stage Dockerfile for a Spring Boot app

```
# ---- build stage ----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline          # cached unless pom changes
COPY src ./src
RUN mvn -q package -DskipTests

# ---- runtime stage ----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
USER app                                # don't run as root
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

```
# docker-compose.yml (local dev)
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    ports: ["5432:5432"]
  app:
    build: .
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/postgres
```

```
depends_on: [db]
ports: ["8080:8080"]
```

Gotchas

- **JVM memory in containers.** Older JVMs (before 8u191 / 10) read the *host's* RAM and ignored cgroup limits, so containers got OOM-killed. Modern JVMs are container-aware (`-XX:+UseContainerSupport` , on by default). Size the heap with `-XX:MaxRAMPercentage` instead of a hard-coded `-Xmx` .
- Containers are **ephemeral**. Anything written inside the container is lost when it is removed. Use **volumes** for persistent data.
- `ENTRYPOINT` ["java", ...] (exec form) makes Java PID 1, so it receives SIGTERM and Spring Boot shuts down gracefully. Shell form (`ENTRYPOINT java -jar`) wraps it in `sh` , which may not forward the signal.
- Don't bake secrets into images. Inject them through env vars or a secrets manager.
- Spring Boot can build images without a Dockerfile: `mvn spring-boot:build-image` (Cloud Native Buildpacks). It also supports **layered jars** , so dependencies sit in a separate cached layer.

Likely follow-ups:

- *Image vs container?* An image is the read-only template (like a class). A container is a running instance with a writable layer (like an object).
- *CMD vs ENTRYPOINT?* `ENTRYPOINT` is the fixed executable. `CMD` supplies default arguments, which `docker run` arguments override.
- *How do containers talk to each other?* Over a user-defined Docker network, using service names as DNS hostnames (e.g. `db:5432` in compose).
- *Docker vs Kubernetes?* Docker builds and runs containers. Kubernetes orchestrates many containers across nodes: scheduling, scaling, self-healing, service discovery.

A2. What are the main features introduced in Java 8, and why were they introduced?

Java 8 (March 2014, LTS) was the biggest language change since generics in Java 5. The main motive was **multi-core hardware**: Java needed a concise way to express *behaviour as data* (functional style) so that bulk operations on collections could be parallelised easily. Other languages (Scala, C#, JavaScript) already had lambdas, and Java code was verbose by comparison (anonymous inner classes everywhere).

Feature	What it is	Why it was introduced
Lambda expressions	<code>(a, b) -> a + b</code>	Pass behaviour concisely and drop anonymous-class boilerplate. They compile to <code>invokedynamic</code> + <code>LambdaMetafactory</code> , not to inner <code>.class</code> files.
Functional interfaces	One abstract method (<code>Function</code> , <code>Predicate</code> , <code>Supplier</code> , <code>Consumer</code> ...) in <code>java.util.function</code>	Give lambdas a <i>target type</i> without adding a function type to the type system.
Method references	<code>String::toUpperCase</code> , <code>System.out::println</code> , <code>ArrayList::new</code>	An even shorter lambda when you just delegate to an existing method.
Stream API	<code>list.stream().filter(...).map(...).collect(...)</code>	Declarative, lazy, composable bulk operations. <code>parallelStream()</code> gives easy parallelism (ForkJoinPool).
Default & static methods in interfaces	<code>default void forEach(...)</code>	Interface evolution : add <code>stream()</code> , <code>forEach()</code> , <code>removeIf()</code> to <code>Collection</code> without breaking every existing implementation.
Optional	<code>Optional<T></code>	Make "may be absent" explicit in return types and reduce NPEs.
java.time (JSR-310)	<code>LocalDate</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code> , <code>Instant</code> , <code>Duration</code>	Replace the mutable, non-thread-safe, confusing <code>Date</code> / <code>Calendar</code> / <code>SimpleDateFormat</code> . The new types are immutable and thread-safe.
CompletableFuture	Composable async pipelines	<code>Future</code> was blocking-only and could not be composed.
Metaspace replaces PermGen	Class metadata lives in native memory	Ends <code>OutOfMemoryError: PermGen space</code> and the PermGen tuning pain.
ConcurrentHashMap rewrite	CAS + bin-level locking; new <code>compute</code> , <code>merge</code> , <code>forEach</code>	Better scalability than segment locks.
HashMap treeification	Long buckets become red-black trees	Worst case drops from $O(n)$ to $O(\log n)$; defends against hash-collision DoS.
Others	<code>StringJoiner</code> , <code>String.join</code> , <code>Base64</code> , <code>Arrays.parallelSort</code> , <code>repeatable</code> & type annotations, Nashorn JS engine (removed in 15)	Small productivity wins.

```
import java.time.*;
import java.util.*;
import java.util.stream.*;

record Incident(String zone, int severity, LocalDateTime at) {}

public class Java8Demo {
    public static void main(String[] args) {
        List<Incident> incidents = List.of(
            new Incident("North", 3, LocalDateTime.now().minusHours(2)),
            new Incident("South", 5, LocalDateTime.now().minusMinutes(10)),
            new Incident("North", 4, LocalDateTime.now());

        // Before Java 8: loops + mutable maps. With streams:
        Map<String, Long> countByZone = incidents.stream()
            .filter(i -> i.severity() >= 3) // Predicate lambda
            .collect(Collectors.groupingBy(Incident::zone, // method reference
                Collectors.counting()));
        System.out.println(countByZone); // e.g. {South=1, North=2}

        Optional<Incident> worst = incidents.stream()
            .max(Comparator.comparingInt(Incident::severity)); // static interface method
        worst.map(Incident::zone).ifPresent(System.out::println); // South
    }
}
```

Gotchas

- Streams are **lazy**: nothing runs until a terminal operation. A stream can be consumed **only once**.
- `parallelStream()` uses the common `ForkJoinPool`. Blocking I/O inside it starves the whole JVM's pool. It is rarely a win for small collections.
- A lambda can capture only **effectively final** local variables.
- Don't use `Optional` for fields, method parameters, or collections. It was designed for return types.

Likely follow-ups:

- Intermediate vs terminal operations?* Intermediate ops (`map`, `filter`, `sorted`) are lazy and return a `Stream`. Terminal ops (`collect`, `forEach`, `reduce`, `count`) start the pipeline.
- map vs flatMap?* `map` is 1→1. `flatMap` is 1→many and flattens `Stream<Stream<T>>` into `Stream<T>`.
- Lambda vs anonymous class?* A lambda has no own `this` (it uses the enclosing instance), doesn't create a new scope for variable shadowing, and uses `invokedynamic` instead of a separate class file.
- findFirst vs findAny?* `findAny` may return any element and is faster in parallel streams. `findFirst` respects encounter order.

A3. What are the important features introduced in Java 18, and how are they used?

Java 18 (March 2022) is a **non-LTS** release (six months of support). In production, teams normally run **Java 17 LTS** (Sept 2021) or **Java 21 LTS** (Sept 2023). Java 18's features are still worth knowing because they carry into 21.

JEP	Feature	Status in 18
400	UTF-8 by default	Final
408	Simple Web Server (<code>jwebserver</code>)	Final
413	Code snippets in Javadoc (<code>@snippet</code>)	Final
416	Core reflection reimplemented with method handles	Final (internal)
417	Vector API	3rd incubator
418	Internet-address resolution SPI	Final
419	Foreign Function & Memory API	2nd incubator
420	Pattern matching for <code>switch</code>	2nd preview (final in 21)
421	Deprecate finalization for removal	Final (deprecation)

1. JEP 400 – UTF-8 by default

Before 18, the default charset (`Charset.defaultCharset()`, used by `FileReader`, `FileWriter`, `new String(bytes)`, `InputStreamReader` without a charset) depended on the OS locale. On Windows it was often `windows-1252`. A file written on Windows could therefore be garbled when read on Linux. From 18, the default is **UTF-8 everywhere**.

```
import java.io.*;
import java.nio.charset.Charset;

public class CharsetDemo {
    public static void main(String[] args) throws IOException {
        System.out.println(Charset.defaultCharset()); // UTF-8 on Java 18+
        System.out.println(System.getProperty("native.encoding")); // OS encoding, e.g. Cp1252
        try (var w = new FileWriter("out.txt")) { // now UTF-8 by default
            w.write("Bengaluru - ಬೆಂಗಳೂರು");
        }
    }
}
```

- To restore the old behaviour, use `-Dfile.encoding=COMPAT`.
- Best practice remains to pass the charset explicitly, e.g. `Files.readString(path, StandardCharsets.UTF_8)`.
- Console output (`System.out`) still uses the console's encoding. That is separate from the default charset.

2. JEP 408 – Simple Web Server

A minimal, static-file-only HTTP/1.1 server for prototyping, testing, and sharing files. It is **not for production**: no HTTPS, GET/HEAD only.

```
jwebserver # serves current dir on 127.0.0.1:8080
jwebserver -p 9000 -d /abs/path/to/site -o verbose
```

```
import com.sun.net.httpserver.SimpleFileServer;
import com.sun.net.httpserver.SimpleFileServer.OutputLevel;
import java.net.InetSocketAddress;
import java.nio.file.Path;

public class StaticServer {
    public static void main(String[] args) {
        var server = SimpleFileServer.createFileServer(
            new InetSocketAddress(8080),
            Path.of("C:/tmp/site").toAbsolutePath(), // must be absolute
            OutputLevel.VERBOSE);
        server.start();
    }
}
```

Use cases: serve an Angular `dist/` folder quickly, or mock a static JSON endpoint in tests.

3. JEP 413 – `@snippet` in Javadoc

This replaces the awkward `<pre>{@code ...}</pre>`. Snippets support highlighting, regions, links, and **external snippet files** that can be compiled and tested.

```
/**
 * Calculates a congestion score.
 * {@snippet :
 *   var svc = new TrafficService();
 *   int score = svc.score("MG Road"); // @highlight substring="score"
 * }
 */
public int score(String road) { return 0; }
```

4. JEP 418 – Internet-Address Resolution SPI

`InetAddress` used to be hard-wired to the OS resolver. Java 18 adds a service-provider interface (`java.net.spi.InetAddressResolverProvider`) so you can plug in your own resolver. Uses: custom DNS (DNS-over-HTTPS), Project Loom (non-blocking resolution), and **tests** that map hostnames to fixed IPs without editing `/etc/hosts`. Providers are registered via `ServiceLoader` (`META-INF/services/java.net.spi.InetAddressResolverProvider`).

5. JEP 421 – Deprecate finalization for removal

`Object.finalize()` is unpredictable (it may never run, or run late), slows GC, can resurrect objects, and causes security issues. It is now `@Deprecated(forRemoval = true)`. You can run with `--finalization=disabled` to test whether your app depends on it. Replacements: **try-with-resources** (`AutoCloseable`) and `java.lang.ref.Cleaner`.

```
import java.lang.ref.Cleaner;

public class NativeResource implements AutoCloseable {
    private static final Cleaner CLEANER = Cleaner.create();
    private final Cleaner.Cleanable cleanable;

    public NativeResource() {
```

```
// the state/action must NOT reference 'this' or it will never become unreachable
cleanable = CLEANER.register(this, () -> System.out.println("released"));
}
@Override public void close() { cleanable.clean(); } // deterministic path
}
```

6. Others

- **JEP 416**: reflection now uses method handles internally. This is mostly invisible to you, except for a slight change in stack traces and performance.
- **JEP 420** (preview): `switch` patterns such as `case Circle c -> ...` and guards (`when` is the final syntax from 21).
- **JEP 417 / 419**: incubators for SIMD vector math and for calling native code without JNI. FFM became final in Java 22.

Likely follow-ups:

- *Why would you not use Java 18 in production?* It's non-LTS, with only six months of updates. Use 17 or 21 LTS.
- *What major features are in Java 21 LTS?* Virtual threads (JEP 444), record patterns, pattern-matching `switch` (final), sequenced collections, and generational ZGC.
- *What breaks with JEP 400?* Code that relied implicitly on a non-UTF-8 platform charset, e.g. reading legacy windows-1252 files without specifying the charset.
- *Can `javax.servlet` serve a REST API?* No. It serves static files only, via GET/HEAD. You'd use `HttpServer` handlers or Spring Boot for that.

A4. Can we override the main method in Java? Explain why or why not.

No. `main` is `static`, and static methods belong to the class, not to an instance. Overriding is runtime dispatch based on the object's type, and static methods have no object to dispatch on. They are **bound at compile time** to the reference type.

What you *can* do:

Action	Possible?	Note
Override <code>main</code>	No	Static. A subclass declaring the same <code>static main</code> hides it (method hiding).
Overload <code>main</code>	Yes	e.g. <code>main(int x)</code> . The JVM only calls <code>public static void main(String[])</code> .
Hide <code>main</code> in a subclass	Yes	<code>java Child</code> runs Child's <code>main</code> . <code>java Parent</code> runs Parent's.
Make <code>main</code> <code>final</code>	Yes	Then a subclass can't even hide it.
Inherit <code>main</code>	Yes	If Child has no <code>main</code> , <code>java Child</code> runs Parent's inherited static <code>main</code> .

```
class Parent {
    public static void main(String[] args) { System.out.println("Parent main"); }
}
class Child extends Parent {
    // Not an override: this HIDES Parent.main. @Override here would be a compile error.
    public static void main(String[] args) { System.out.println("Child main"); }

    public static void main(int x) { System.out.println("overload " + x); } // overload, OK
}
```

- If Child declares `void main(String[] a)` (non-static) while Parent has `static main` → compile error: *"overriding method is static / instance method cannot override static method"*.
- Valid `main` variants: `final public static void main(String... args)`, `static public void main(String[] a)`, and `synchronized` / `strictfp` modifiers.
- **Newer Java**: JEP 445/463/477/495 (previews in Java 21–24), finalized in **Java 25 (JEP 512)**, allow *instance* `void main()` methods in simple programs. Even then, the launcher picks the method; it is not polymorphic overriding.

Likely follow-ups:

- *Can `main` be private?* It compiles, but the launcher fails with "Main method not found... please define as public static void main".
- *Why is `main` static?* So the JVM can call it without creating an object (it doesn't know which constructor to call).
- *What if `main` throws an exception?* The main thread dies, the default uncaught-exception handler prints the stack trace, and the JVM exits with a non-zero status (if no non-daemon threads remain).

A5. Explain the main OOP concepts in Java.

The Four Pillars of Object-Oriented Programming

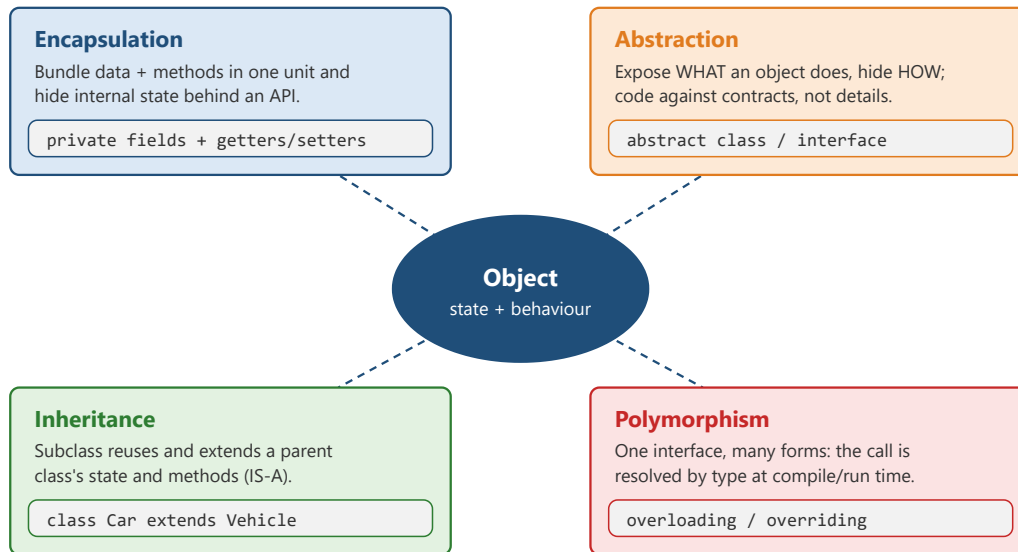


Figure 1: Four pillars of OOP

Java is built around **classes** (blueprints) and **objects** (instances with state + behaviour). The four pillars:

Pillar	Meaning	Java mechanism
Encapsulation	Hide internal state and expose controlled operations	<code>private</code> fields, getters/validated setters, immutability, packages, modules
Abstraction	Expose <i>what</i> an object does and hide <i>how</i>	<code>abstract</code> classes, interfaces
Inheritance	Reuse and specialise an existing type ("is-a")	<code>extends</code> (single class), <code>implements</code> (many interfaces)
Polymorphism	One interface, many forms	Overloading (compile time), overriding + dynamic dispatch (runtime)

```
// Abstraction: contract
interface Notifier { void send(String to, String msg); }

// Encapsulation: state is private, validated
final class Citizen {
    private final String phone;
    Citizen(String phone) {
        if (phone == null || !phone.matches("\\d{10}")) throw new IllegalArgumentException("bad phone");
        this.phone = phone;
    }
    String phone() { return phone; }
}

// Inheritance + Polymorphism
abstract class BaseNotifier implements Notifier {
    protected void log(String m) { System.out.println("[LOG] " + m); }
}
class SmsNotifier extends BaseNotifier {
    @Override public void send(String to, String msg) { log("SMS to " + to); }
}
class EmailNotifier extends BaseNotifier {
    @Override public void send(String to, String msg) { log("Email to " + to); }
}

public class OopDemo {
    public static void main(String[] args) {
        Notifier n = new SmsNotifier(); // reference type = abstraction
        n.send(new Citizen("9876543210").phone(), "Road closed"); // runtime dispatch
    }
}
```

Supporting concepts: **association / aggregation / composition** (has-a, see A20), **coupling and cohesion**, **message passing**, and *favour composition over inheritance* (Effective Java item 18).

Gotchas

- Java has **no multiple inheritance of state** (classes), only of type and behaviour (interfaces with default methods).
- Encapsulation is not "getters and setters for every field". A setter with no validation is barely better than a public field.
- Inheritance breaks encapsulation: a subclass depends on the superclass's implementation details (the fragile base class problem).

Likely follow-ups:

- *Abstraction vs encapsulation?* Abstraction is design-level (which operations to expose). Encapsulation is implementation-level (hiding data and enforcing invariants).
- *Why no multiple class inheritance?* To avoid the diamond problem with state and constructor ambiguity.
- *Is Java 100% object-oriented?* No. It has primitives (`int` , `boolean` ...) and static members.

A6. How do you achieve abstraction in Java? Write a class structure/program to demonstrate it.

Abstraction is achieved in two ways:

1. **Abstract classes** (0–100% abstraction). They can have state, constructors, concrete and abstract methods.
2. **Interfaces** (a pure contract, with default/static/private methods since Java 8/9). A class can implement many of them.

	Abstract class	Interface
Instance fields	Yes	No (only <code>public static final</code> constants)
Constructors	Yes (called via <code>super</code>)	No
Multiple inheritance	No (single <code>extends</code>)	Yes (<code>implements A, B</code>)
Method access	Any	<code>public</code> (abstract/default/static), <code>private</code> helpers (Java 9)
Use when	Related classes share state/code (is-a with common base)	Capability/contract across unrelated classes (can-do)
Sealed (17)	<code>sealed abstract class X permits ...</code>	<code>sealed interface X permits ...</code>

Demo: Smart City sensor devices (template method + interface)

```
import java.time.Instant;
import java.util.List;

// Contract (capability)
interface Calibratable {
    void calibrate();
}

// Partial abstraction with shared state + template method
abstract class SensorDevice {
    private final String deviceId;           // shared state
    protected SensorDevice(String deviceId) { this.deviceId = deviceId; }

    // Template method: fixed algorithm, steps supplied by subclasses
    public final Reading capture() {
        double raw = readRaw();              // abstract step
        double value = normalize(raw);       // hook (overridable)
        return new Reading(deviceId, type(), value, Instant.now());
    }

    protected abstract double readRaw();     // HOW is hidden in subclasses
    protected abstract String type();
    protected double normalize(double raw) { return raw; }

    public String getDeviceId() { return deviceId; }
}

record Reading(String deviceId, String type, double value, Instant at) {}

class AirQualitySensor extends SensorDevice implements Calibratable {
    AirQualitySensor(String id) { super(id); }
    @Override protected double readRaw() { return 182.4; } // e.g. read from serial port
    @Override protected String type() { return "AQI"; }
    @Override protected double normalize(double raw) { return Math.min(raw, 500); }
    @Override public void calibrate() { System.out.println("AQI sensor zeroed"); }
}

class TrafficCounter extends SensorDevice {
    TrafficCounter(String id) { super(id); }
    @Override protected double readRaw() { return 1240; } // vehicles/hour
}
```

```

@Override protected String type() { return "TRAFFIC"; }
}

public class AbstractionDemo {
    public static void main(String[] args) {
        List<SensorDevice> devices = List.of(new AirQualitySensor("AQ-01"), new TrafficCounter("TC-07"));
        for (SensorDevice d : devices) { // client knows only the abstraction
            System.out.println(d.capture());
            if (d instanceof Calibratable c) c.calibrate(); // pattern matching (Java 16+)
        }
        // new SensorDevice("x"); // compile error: SensorDevice is abstract; cannot be instantiated
    }
}

```

Gotchas

- An abstract class *can* have a constructor. It runs when a subclass is instantiated.
- `abstract` cannot be combined with `final`, `private` or `static` on a method (none of those can be overridden).
- A class with even one abstract method must be declared abstract. An abstract class may also have zero abstract methods (to prevent instantiation).
- Anonymous class instantiation is allowed: `new SensorDevice("x") { ... }`.

Likely follow-ups:

- *Abstract class or interface after Java 8?* Interface for the contract, abstract class when you need state or constructors. Often both: an interface plus an `AbstractXxx` skeleton (e.g. `List` + `AbstractList`).
- *Can an interface have a constructor?* No. It has no instance state to initialise.
- *Can an abstract class implement an interface without implementing its methods?* Yes. The first concrete subclass must implement them.

A7. Why were functional interfaces introduced, what is one, and how do you define one?

A **functional interface** is an interface with **exactly one abstract method** (a SAM, Single Abstract Method). It is the **target type** for lambda expressions and method references.

Why introduced (Java 8)

- Java's designers did not want to add *function types* (like `(int)->int`) to the type system, because that would complicate generics, erasure and the JVM. Instead, a lambda is converted to an instance of a SAM interface.
- **Backward compatibility:** existing single-method interfaces (`Runnable`, `Callable`, `Comparator`, `ActionListener`) automatically became lambda-friendly with no API change.
- They enable streams and functional-style APIs (`filter(Predicate)`, `map(Function)`).

Built-in interfaces (`java.util.function`, 43 total)

Interface	Method	Example
<code>Function<T,R></code>	<code>R apply(T)</code>	<code>String::length</code>
<code>BiFunction<T,U,R></code>	<code>R apply(T,U)</code>	<code>(a,b) -> a+b</code>
<code>Predicate<T></code>	<code>boolean test(T)</code>	<code>s -> s.isEmpty()</code>
<code>Consumer<T></code>	<code>void accept(T)</code>	<code>System.out::println</code>
<code>Supplier<T></code>	<code>T get()</code>	<code>ArrayList::new</code>
<code>UnaryOperator<T></code> / <code>BinaryOperator<T></code>	<code>T apply(T)</code> / <code>T apply(T,T)</code>	<code>String::trim</code> , <code>Integer::sum</code>
Primitive versions	<code>IntPredicate</code> , <code>ToIntFunction</code> , <code>IntBinaryOperator</code> ...	Avoid boxing

Defining one

```

@FunctionalInterface
interface FareCalculator {
    double calculate(double km, boolean peak); // the single abstract method

    default FareCalculator withSurcharge(double pct) { // defaults allowed
        return (km, peak) -> calculate(km, peak) * (1 + pct / 100);
    }
    static FareCalculator flat(double rate) { // statics allowed
        return (km, peak) -> km * rate;
    }
    boolean equals(Object o); // public Object methods do NOT count as abstract
}

```

```

    String toString();
}

public class FiDemo {
    public static void main(String[] args) {
        FareCalculator bus = (km, peak) -> peak ? km * 2.5 : km * 2.0;
        FareCalculator metro = FareCalculator.flat(3).withSurcharge(10);
        System.out.println(bus.calculate(10, true)); // 25.0
        System.out.println(metro.calculate(10, false)); // 33.0 (approx; floating point)
    }
}

```

Rules and gotchas

- `@FunctionalInterface` is **optional** but recommended. The compiler then *enforces* the single-abstract-method rule, and it documents intent.
- Default, static and private methods, plus abstract re-declarations of public `Object` methods (`equals` , `hashCode` , `toString`), don't count. That's why `Comparator` (which declares `equals`) is still functional.
- An interface that *inherits* one abstract method and declares nothing new is functional (e.g. `interface X extends Runnable {}`).
- Lambdas can't throw checked exceptions unless the SAM declares them. That's why `Callable.call()` throws `Exception` exists alongside `Runnable` .
- A lambda's type is decided by context. The same lambda `() -> 42` can be a `Supplier<Integer>` or a `Callable<Integer>` .

Likely follow-ups:

- *Is `@FunctionalInterface` mandatory for lambdas?* No. Any SAM interface works. The annotation only adds compile-time checking.
- *Can a functional interface extend another?* Yes, as long as the total number of abstract methods (after merging override-equivalent ones) stays one.
- *Why can a lambda only use effectively final locals?* It captures a *copy* of the value, so allowing mutation would cause confusing semantics and data races.

A8. If a functional interface already has one abstract method and another abstract method is added, what happens?

It depends on whether the interface is annotated and how it is used.

Scenario	Result
Annotated <code>@FunctionalInterface</code> + second abstract method	Compile error at the interface: <i>"Unexpected @FunctionalInterface annotation – X is not a functional interface: multiple non-overriding abstract methods found in interface X"</i>
Not annotated + second abstract method	The interface compiles fine (it's just a normal interface). Every lambda/method reference targeting it fails: <i>"incompatible types: X is not a functional interface"</i>
Second method is <code>default</code> / <code>static</code> / <code>private</code>	Still functional, no problem
Second method is an abstract re-declaration of a public <code>Object</code> method (<code>boolean equals(Object)</code>)	Still functional
Second method is override-equivalent to an inherited one (same signature)	Counts as one, still functional
Existing implementing classes	Must now implement the new method, otherwise compile error (unless they're abstract)

```

@FunctionalInterface
interface Validator {
    boolean validate(String s);
    // boolean validateAll(String... s); // uncommenting -> compile error on the interface
    default Validator and(Validator o) { return s -> validate(s) && o.validate(s); } // fine
}

interface Mapper { // no annotation
    int map(String s);
    int unmap(int x); // compiles...
}
// Mapper m = s -> s.length(); // ...but this line: "Mapper is not a functional interface"

```

Lesson: the annotation moves the failure from "every call site across the codebase" to "one place, at the definition". That's why library authors use it. To add behaviour safely, add a **default method** instead of an abstract one.

Likely follow-ups:

- *How do you add a method without breaking lambdas?* Make it `default` with a sensible implementation.

- Is `Comparator` functional even though it has two abstract-looking methods? Yes. `equals(Object)` is a public `Object` method and doesn't count.
- What if two parent functional interfaces have the same abstract method? The child is still functional, because the methods are override-equivalent and merge into one.

A9. What is the use of default methods in interfaces? (incl. diamond problem resolution)

A **default method** (Java 8) is an interface method with a body, marked `default`. Implementing classes inherit it and *may* override it.

Why

1. **Interface evolution / backward compatibility.** The JDK needed to add `forEach`, `stream`, `removeIf`, `splitterator` to `Collection / Iterable` without breaking millions of existing implementations.
2. **Optional behaviour / convenience methods:** `Comparator.reversed()`, `thenComparing()`, `Map.getOrDefault()`, `Map.computeIfAbsent()`.
3. Fewer "adapter" abstract classes (`WindowAdapter` -style).

```
interface Payment {
    void pay(double amount);
    default String currency() { return "INR"; } // convenience
    default void payWithReceipt(double amt) { // behaviour built on the abstract op
        pay(amt);
        System.out.println("Receipt: " + amt + " " + currency());
    }
}

class UpiPayment implements Payment {
    @Override public void pay(double amount) { System.out.println("UPI " + amount); }
}
```

Diamond problem: resolution rules (JLS 8.4.8, 9.4.1)

1. **Class wins.** A method from a superclass (concrete or abstract) takes priority over any interface default.
2. **More specific interface wins.** If `B` extends `A` and both define `hello()`, then `B.hello()` is chosen.
3. **Otherwise it's ambiguous.** The class **must override** and may pick one explicitly with `X.super.method()`.

```
interface Camera { default String start() { return "Camera on"; } }
interface Radar { default String start() { return "Radar on"; } }

// class TrafficUnit implements Camera, Radar {} // compile error:
// "types Camera and Radar are incompatible; class TrafficUnit inherits unrelated defaults for start()"

class TrafficUnit implements Camera, Radar {
    @Override public String start() {
        return Camera.super.start() + " " + Radar.super.start(); // explicit choice
    }
}

class Base { public String start() { return "Base"; } }
class Unit2 extends Base implements Camera { } // Rule 1: new Unit2().start() -> "Base"

interface SmartCamera extends Camera { default String start() { return "Smart camera on"; } }
class Unit3 implements Camera, SmartCamera { } // Rule 2: "Smart camera on"
```

Gotchas

- A default method **cannot override** `equals`, `hashCode` or `toString`. That's a compile error, because `Object`'s methods always win (rule 1).
- Default methods can't be `final` or `synchronized`, and they have no access to instance state (interfaces have no fields).
- `X.super.m()` works only for a *direct* super-interface.
- Unlike C++, there is no diamond problem with *state*, because interfaces carry behaviour only.

Likely follow-ups:

- *Abstract class vs interface with default methods?* Abstract classes hold state and constructors. Interfaces allow multiple inheritance but no state.
- *Can a sub-interface redeclare a default as abstract?* Yes. `interface X extends Camera { String start(); }` forces implementers to provide it again.
- *Java 9 addition?* `private` (and `private static`) interface methods, so defaults can share helper code.

A10. What is the use of static methods in interfaces?

Since Java 8, interfaces can have `static` methods with a body. They belong to the **interface itself** and are called as `InterfaceName.method()`.

Uses

- **Factory methods** that keep related utilities next to the type: `List.of()`, `Map.of()`, `Map.entry()`, `Comparator.comparing()`, `Comparator.naturalOrder()`, `Function.identity()`, `Stream.of()`, `Path.of()`.
- They replace companion utility classes (`Collections` for `Collection`, `Paths` for `Path`).
- Validation/helper logic that shouldn't be overridden.

```
import java.util.regex.Pattern;

interface VehicleNumber {
    String value();

    Pattern FORMAT = Pattern.compile("[A-Z]{2}\\d{2}[A-Z]{1,2}\\d{4}"); // implicitly public static final

    static boolean isValid(String s) { return s != null && FORMAT.matcher(s).matches(); }
    static VehicleNumber of(String s) {
        if (!isValid(s)) throw new IllegalArgumentException("Invalid: " + s);
        return () -> s; // VehicleNumber is a SAM, so a lambda works
    }
}

class Car implements VehicleNumber {
    public String value() { return "KA01AB1234"; }
    void check() {
        System.out.println(VehicleNumber.isValid(value())); // OK
        // System.out.println(isValid(value())); // compile error: not inherited by implementing class
    }
}
```

Key rules

- Static interface methods are **not inherited** by implementing classes or sub-interfaces. `Car.isValid(...)` and `carObj.isValid(...)` are compile errors. Static methods in *classes* are different: they are inherited and can be called via a subclass or an instance.
- This avoids ambiguity when a class implements two interfaces with the same static method.
- They are implicitly `public` unless declared `private static` (Java 9+). They can't be `abstract`, `default` or `final`.
- They can't be overridden. A same-signature static method in the implementing class is just an unrelated method.

Likely follow-ups:

- *Why not inherited, unlike class statics?* So that implementing multiple interfaces with same-named statics doesn't conflict, and so the call site always names the owner explicitly.
- *Can an interface have a `main` method?* Yes (`public static void main`), and you can run it with `java InterfaceName`.
- *Static vs default?* Static has no `this` and isn't polymorphic. Default is an instance method, inherited and overridable.

A11. Can we override private methods?

No. A `private` method is visible only within its declaring class (and its nest, since Java 11). It is **not inherited**, so a subclass cannot override it. If the subclass declares a method with the same signature, it is a **brand-new, unrelated method**.

Consequences:

- Calls to a private method are **statically bound**. Calling it from within the parent always invokes the parent's version, even if the runtime object is a subclass.
- `@Override` on the subclass method gives a compile error: *"method does not override or implement a method from a supertype"*.
- Since the subclass method is independent, it can have any access modifier, return type and exceptions.

```
class Account {
    private void audit() { System.out.println("Account.audit"); }
    public void process() { audit(); } // statically bound to Account.audit
}

class SavingsAccount extends Account {
    // @Override -> compile error
    public void audit() { System.out.println("SavingsAccount.audit"); } // new method
}

public class PrivateDemo {
    public static void main(String[] args) {
        Account a = new SavingsAccount();
        a.process(); // prints "Account.audit" - no polymorphism
        new SavingsAccount().audit(); // "SavingsAccount.audit"
    }
}
```

Compare: if `audit()` were `protected` / `public`, `a.process()` would print `SavingsAccount.audit` (dynamic dispatch). This is the classic trap question.

Edge cases

- **Nested classes:** a nested subclass can *access* the outer class's private method, but still cannot override it.
- Private methods are implicitly "effectively final". Adding `final` is legal but redundant.
- Reflection (`setAccessible(true)`) can *call* a private method, but never makes it overridable.

Likely follow-ups:

- *Can we override final methods?* No. Compile error "overridden method is final".
- *Can we override protected / package-private methods?* Protected yes. Package-private only from a subclass in the **same package**; in another package it's not inherited, so it's a new method.
- *Why are private methods statically bound?* Not inherited means no virtual dispatch is needed, so the compiler can resolve the exact target.

A12. Can we override static methods? Can we override a static method in an interface?

Static methods in classes: no override, only hiding. A static method is resolved at **compile time** using the **reference (declared) type**, not the runtime object. A subclass can declare a static method with the same signature, which **hides** the parent's (see A13).

```
class Logger {
    static String name() { return "Logger"; }
    String instanceName() { return "Logger"; }
}
class FileLogger extends Logger {
    static String name() { return "FileLogger"; } // hiding, not overriding
    @Override String instanceName() { return "FileLogger"; } // overriding
}
public class StaticDemo {
    public static void main(String[] args) {
        Logger l = new FileLogger();
        System.out.println(l.name()); // "Logger" -> reference type decides
        System.out.println(l.instanceName()); // "FileLogger" -> runtime type decides
    }
}
```

Compile-time rules for hiding

Parent	Child	Result
static	static (same signature)	Hiding. OK if access isn't weaker and return type is compatible
static	instance	Error: "instance method cannot override static method"
instance	static	Error: "static method cannot hide instance method"
static	@Override static	Error: "method does not override or implement a method from a supertype"
final static	static	Error: cannot hide a final method

Static methods in interfaces: also no. They're not even inherited (A10).

- An implementing class can declare a static (or even an instance) method with the same signature. It's simply **unrelated**: not overriding, not hiding.
- A sub-interface can likewise declare its own static method of the same name.
- So `InterfaceName.m()` always calls the interface's version, and `ClassName.m()` calls the class's.

```
interface Codec { static String id() { return "Codec"; } }
class JsonCodec implements Codec {
    static String id() { return "JsonCodec"; } // legal, unrelated to Codec.id()
}
// Codec.id() -> "Codec"; JsonCodec.id() -> "JsonCodec"
```

Likely follow-ups:

- *Why can't statics be overridden?* Overriding needs an object for dynamic dispatch (`invokevirtual`). Statics use `invokestatic`, resolved from the class.
- *Can we call a static method through a null reference?* Yes. `((Logger) null).name()` works (with a warning), because no object is dereferenced.
- *Can static methods be overloaded?* Yes. Overloading is compile-time and works fine for statics.

A13. What is method hiding?

Method hiding happens when a subclass declares a **static** method with the same signature as a static method in its superclass. The subclass method *hides* the superclass one. Which one runs depends **only on the compile-time (reference) type** of the expression, or the class name used.

```
class Parent {
    static void greet() { System.out.println("Parent.greet"); }
    void show()         { System.out.println("Parent.show"); }
}
class Child extends Parent {
    static void greet() { System.out.println("Child.greet"); } // hides
    @Override void show(){ System.out.println("Child.show"); } // overrides
}
public class HidingDemo {
    public static void main(String[] args) {
        Parent p = new Child();
        Child c = new Child();
        p.greet(); // Parent.greet (reference type Parent)
        c.greet(); // Child.greet
        Parent.greet(); // Parent.greet
        p.show(); // Child.show (runtime type Child)
        ((Parent) c).greet(); // Parent.greet - casting changes which static runs
    }
}
```

	Overriding	Hiding
Applies to	Instance methods	Static methods (also fields)
Binding	Runtime (dynamic dispatch)	Compile time (static binding)
Decided by	Object's actual type	Reference / class name
@Override allowed	Yes	No (compile error)
Access to hidden version	<code>super.m()</code>	<code>Parent.m()</code>

Field hiding is the same concept for fields: fields are **never polymorphic**.

```
class A { String name = "A"; }
class B extends A { String name = "B"; }
// A obj = new B(); obj.name -> "A"; ((B) obj).name -> "B"
```

Gotchas: hiding is confusing, and most style guides/IDEs warn against it. Always call static methods via the **class name**, never through an instance.

Likely follow-ups:

- *Is hiding a form of polymorphism?* No. It's static binding. There's no runtime decision.
- *Are rules for hiding like overriding?* Mostly. Access can't be weaker, the return type must be substitutable, and `throws` can't be broader.
- *Can an instance method hide a static one?* No, that's a compile error. Both must be static.

A14. What is serialization? How do you prevent a variable from being serialized?

Serialization converts an object graph into a byte stream (to store in a file, send over a network, or put in a cache/session). **Deserialization** rebuilds the object graph from the bytes. In Java you use `ObjectOutputStream.writeObject()` / `ObjectInputStream.readObject()`, and the class must implement the marker interface `java.io.Serializable`.

```
import java.io.*;

class User implements Serializable {
    private static final long serialVersionUID = 1L; // version stamp
    private String username;
    private transient String password; // NOT serialized
    private static String region = "KA"; // NOT serialized (class-level)
    private transient int loginCount = 5; // becomes 0 after deserialization

    User(String u, String p) { username = u; password = p; }
    @Override public String toString() {
        return username + "/" + password + "/" + loginCount + "/" + region;
    }
}

public class SerDemo {
```

```

    public static void main(String[] args) throws Exception {
        var bos = new ByteArrayOutputStream();
        try (var out = new ObjectOutputStream(bos)) { out.writeObject(new User("nagesh", "s3cret")); }
        try (var in = new ObjectInputStream(new ByteArrayInputStream(bos.toByteArray()))) {
            System.out.println(in.readObject()); // nagesh/null/0/KA
        }
    }
}

```

Ways to prevent a field from being serialized

Technique	How
<code>transient</code>	The field is skipped and gets its default value (<code>null</code> , <code>0</code> , <code>false</code>) on deserialization. The constructor and field initialisers do NOT run for a <code>Serializable</code> class, so <code>loginCount = 5</code> is lost.
<code>static</code>	Class state is not part of the object, so it's never written. The value you see after deserialization is the current JVM's static value.
<code>serialPersistentFields</code>	<code>private static final ObjectStreamField[]</code> explicitly lists the fields that <i>are</i> serialized.
Custom <code>writeObject</code> / <code>readObject</code>	<code>private void writeObject(ObjectOutputStream)</code> : call <code>defaultWriteObject()</code> and write/encrypt only what you want.
<code>Externalizable</code>	You control everything via <code>writeExternal</code> / <code>readExternal</code> . Needs a public no-arg constructor .
Don't implement <code>Serializable</code> on the field's type	Then serializing fails with <code>NotSerializableException</code> (you'd normally mark it <code>transient</code>).

serialVersionUID

- This is a version number checked during deserialization. If it doesn't match, you get `InvalidClassException` .
- If you don't declare one, the JVM computes it from the class structure. Adding even a method can change it and break old serialized data. **Always declare it.**
- Compatible changes (adding a field) work with the same UID: missing fields get default values.

Inheritance rules

- If the parent is **not** `Serializable`, its fields aren't saved. On deserialization, the parent's **no-arg constructor runs** (it must exist and be accessible, or you get `InvalidClassException`).
- If the parent is `Serializable`, all subclasses are too. To block serialization in a subclass, throw `NotSerializableException` from `writeObject` .

Singletons and records

- Deserialization creates a new instance and breaks singletons. Add `readResolve()` returning the singleton, or use an **enum singleton**.
- **Records** (Java 16+) deserialize through the canonical constructor, so validation runs. That's safer than for ordinary classes.

Security concerns (important)

- Java deserialization of untrusted data is a well-known **remote code execution** vector ("gadget chains" in Commons-Collections etc.; many CVEs).
- Mitigations:
 - Never deserialize untrusted input.
 - Use `ObjectInputFilter` (JEP 290, Java 9; context-specific filter factories in Java 17, JEP 415), e.g. `-Djdk.serialFilter=com.trinity.*;!*` .
 - Prefer **JSON (Jackson)/Protobuf/Avro** for APIs and messaging.
- Don't serialize secrets. Mark passwords and tokens `transient` or exclude them.

Likely follow-ups:

- *Serializable vs Externalizable?* `Serializable` is automatic, reflection-based, and needs no constructor. `Externalizable` is manual, can be faster and smaller, and needs a public no-arg constructor.
- *What does `transient final` do?* The field is set to its default after deserialization and can't be reassigned normally. You'd need `readResolve` or reflection.
- *Why is `Serializable` a marker interface?* It's an opt-in flag: the serialization machinery checks `instanceof Serializable` .
- *How does Spring/Kafka serialize?* Mostly JSON (Jackson) or Avro/Protobuf, not Java native serialization.

A15. Explain the volatile keyword.

`volatile` is a field modifier that makes **visibility** and **ordering** guarantees for that variable across threads. It does **not** provide mutual exclusion or atomicity for compound actions.

The problem it solves

Each core can keep values in registers or caches, and both the JIT and the CPU reorder instructions. Without synchronization, one thread may *never* see another thread's write:

```
public class StopFlag {
    private static volatile boolean running = true;    // remove volatile -> loop may never end

    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            long n = 0;
            while (running) { n++; }    // JIT could hoist 'running' into a register if not volatile
            System.out.println("stopped at " + n);
        });
        worker.start();
        Thread.sleep(1000);
        running = false;                // guaranteed to become visible to worker
        worker.join();
    }
}
```

Guarantees (Java Memory Model, JSR-133)

1. **Visibility.** A write to a volatile variable is immediately flushed so other threads see it. A read always gets the latest value from main memory.
2. **Happens-before.** A write to volatile `v` *happens-before* every subsequent read of `v`. Everything the writing thread did **before** the volatile write is visible to a thread **after** it reads that value. This is the "safe publication" piggyback.
3. **No reordering** across the volatile access. The compiler and CPU insert memory barriers (StoreStore/StoreLoad etc.).
4. Reads and writes of volatile `long` and `double` are **atomic**. Non-volatile 64-bit writes may tear on 32-bit JVMs.

Not atomic: classic trap

```
private volatile int count = 0;
void inc() { count++; }    // read-modify-write: 3 steps, two threads can both read 5 and write 6
```

Fix: `AtomicInteger.incrementAndGet()` (CAS), `LongAdder` (high contention), or `synchronized`.

	volatile	synchronized	Atomic*
Visibility	Yes	Yes	Yes
Atomic compound ops	No	Yes	Yes (CAS)
Blocking	No	Yes	No (lock-free)
Applies to	Fields	Methods/blocks	Specific variables

Double-checked locking (needs volatile)

```
public final class ConfigRegistry {
    private static volatile ConfigRegistry instance;    // volatile is REQUIRED
    private ConfigRegistry() { /* load config */ }

    public static ConfigRegistry getInstance() {
        ConfigRegistry local = instance;                // one volatile read on the fast path
        if (local == null) {
            synchronized (ConfigRegistry.class) {
                local = instance;
                if (local == null) {
                    instance = local = new ConfigRegistry();
                }
            }
        }
        return local;
    }
}
```

Why volatile? `instance = new ConfigRegistry()` is really three steps: (1) allocate memory, (2) run the constructor, (3) assign the reference. Without volatile, steps 2 and 3 can be **reordered**. Another thread could then see a non-null but **partially constructed** object. Simpler alternatives are the **holder-class idiom** (lazy, thread-safe class init) or an **enum** singleton.

Use volatile for: status flags, the one-writer-many-readers pattern, safely publishing immutable objects, and the DCL singleton.

Likely follow-ups:

- *Is volatile enough for a counter?* No. `count++` isn't atomic. Use `AtomicInteger` or `LongAdder`.
- *Can a local variable be volatile?* No. Locals are thread-confined (on the stack), so it's a compile error.
- *volatile vs synchronized?* `volatile` = visibility + ordering only. `synchronized` = visibility + mutual exclusion (atomicity), and it can block.
- *What is happens-before?* A JMM guarantee that the effects of action A are visible to action B, e.g. `unlock`→`lock`, `volatile write`→`read`, `Thread.start()` →the thread's first action, thread end→ `join()` returns.

A16. What is garbage collection and how is it used?

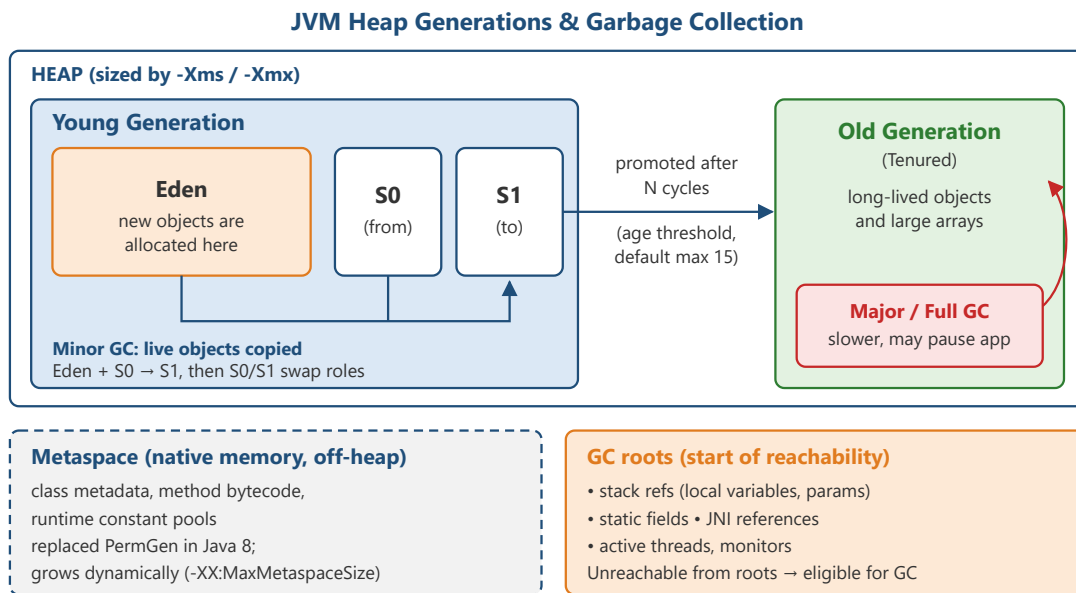


Figure 2: JVM heap generations and garbage collection

Garbage collection (GC) automatically reclaims heap memory occupied by objects that are **no longer reachable** from any **GC root**. You don't `free()` memory. The JVM finds dead objects and recycles their space.

Reachability and GC roots

An object is alive if there is a reference path to it from a root:

- Local variables and operand stacks of every thread's frames
- Static fields of loaded classes
- Active threads
- JNI references (native code)
- Monitors held by `synchronized`
- Classes loaded by the system/bootstrap class loader

Unreachable cycles (A→B with no root reference) **are** collected. Java uses tracing, not reference counting.

Generational hypothesis: most objects die young.

```
Heap
├── Young Generation
│   ├── Eden        <- new objects allocated here (TLABs: thread-local allocation buffers)
│   ├── Survivor S0  <- survivors copied back and forth; age incremented
│   └── Survivor S1
└── Old (Tenured) Gen <- promoted after surviving N minor GCs (MaxTenuringThreshold, default 15)
Metaspace (native memory) - class metadata, not part of the heap
```

- **Minor GC:** collects the Young gen. Fast, copying collector, stop-the-world (STW) but short.
- **Major/Full GC:** collects Old gen (or everything). Expensive and causes long pauses.

Phases (conceptual): **Mark** (trace from roots) → **Sweep** (free dead objects) → **Compact** (defragment), or **copy** survivors to a fresh space.

Collectors

Collector	Flag	Characteristics	Use for
Serial	<code>-XX:+UseSerialGC</code>	Single thread, STW	Small heaps, containers with 1 CPU
Parallel	<code>-XX:+UseParallelGC</code>	Multi-thread STW, max throughput	Batch jobs
G1 (default since Java 9)	<code>-XX:+UseG1GC</code>	Heap split into equal regions; collects the regions with the most garbage first; pause target <code>-XX:MaxGCPauseMillis=200</code>	General-purpose servers (Spring Boot default)
ZGC	<code>-XX:+UseZGC</code>	Concurrent, colored pointers + load barriers, sub-millisecond pauses , scales to TBs. Production since 15; generational ZGC in 21 (JEP 439), the default ZGC mode from 23	Low-latency, large heaps
Shenandoah	<code>-XX:+UseShenandoahGC</code>	Concurrent compaction (Red Hat)	Low latency
Epsilon	<code>-XX:+UseEpsilonGC</code>	No-op (never collects)	Performance testing
CMS	—	Deprecated in 9, removed in 14	—

How do you "use" it?

- Mostly you **don't** call it. Write code that doesn't hold unnecessary references.
- `System.gc()` / `Runtime.getRuntime().gc()` **is only a hint**. The JVM may ignore it (`-XX:+DisableExplicitGC`). It usually triggers a Full GC, which is bad in production.
- Tune with `-Xms` / `-Xmx` (heap size), `-XX:MaxRAMPercentage` (containers), the choice of collector, and pause goals.
- Observe with `-Xlog:gc*` (unified logging, Java 9+), `jstat -gc <pid>`, `jcmd <pid> GC.heap_info`, VisualVM, Java Flight Recorder/JMC, GCeasy, and Micrometer `jvm.gc.pause` metrics in Spring Boot Actuator.

Memory leaks still happen in Java

- Objects stay reachable unintentionally: static `Map` caches that only grow, unremoved listeners, `ThreadLocal` in thread pools (call `remove()`), unclosed resources, and inner classes holding a reference to their outer instance.
- Diagnose: take a heap dump (`jcmd <pid> GC.heap_dump file.hprof` or `-XX:+HeapDumpOnOutOfMemoryError`) and analyse it in Eclipse MAT (dominator tree, retained size).

Reference types: Strong (normal) → **Soft** (cleared before OOM, for caches) → **Weak** (cleared at next GC; `WeakHashMap`) → **Phantom** (post-mortem cleanup; used by `Cleaner`).

Likely follow-ups:

- Can you force GC?* No. `System.gc()` is only a request.
- Is `finalize()` reliable?* No. It's deprecated for removal (JEP 421). Use try-with-resources or `Cleaner`.
- What causes `OutOfMemoryError: Metaspace`?* Too many classes loaded or a classloader leak (e.g. repeated hot redeloys).
- Why G1 over Parallel?* Predictable pause times on large heaps, at the cost of a little throughput.

A17. Output of `System.out.println(Double.MIN_VALUE > 0.0d)` and why?

Output: `true`.

`Double.MIN_VALUE` is **not** the most negative double. It is the **smallest positive non-zero** value a `double` can represent: `4.9E-324` (2^{-1074}), a **subnormal (denormalized)** number. Since it's positive, it is greater than `0.0`.

```
public class MinValueDemo {
    public static void main(String[] args) {
        System.out.println(Double.MIN_VALUE);           // 4.9E-324
        System.out.println(Double.MIN_VALUE > 0.0d);    // true
        System.out.println(Double.MIN_NORMAL);          // 2.2250738585072014E-308 (smallest normal)
        System.out.println(-Double.MAX_VALUE);         // -1.7976931348623157E308 (most negative finite)
        System.out.println(Integer.MIN_VALUE);         // -2147483648 (integers: truly the minimum!)
        System.out.println(Double.MIN_VALUE / 2);       // 0.0 (underflow)
        System.out.println(Float.MIN_VALUE > 0);       // true (1.4E-45)
    }
}
```

Why the inconsistency? In IEEE-754 floating point, the sign is a separate bit. The value range is symmetric, so the "minimum magnitude" was the more useful constant. For integer types (two's complement), `MIN_VALUE` is the most negative value.

Related gotchas

- `0.0 == -0.0` → `true`, but `Double.compare(0.0, -0.0)` → `1` and `Double.valueOf(0.0).equals(-0.0)` → `false`.
- `Double.NaN == Double.NaN` → `false`. Use `Double.isNaN()`.
- `0.1 + 0.2 == 0.3` → `false`. Use `BigDecimal` for money.
- `Math.abs(Integer.MIN_VALUE)` → still negative (overflow).
- When finding a maximum, initialise with `Double.NEGATIVE_INFINITY` or `-Double.MAX_VALUE`, **not** `Double.MIN_VALUE`. That's a real bug source.

Likely follow-ups:

- *What is `1.0/0`?* Infinity. No exception for floating point. `1/0` with ints throws `ArithmeticException`.
- *Why use `BigDecimal` for currency?* Binary floating point can't represent 0.1 exactly. `BigDecimal` is decimal and exact. Construct it with `new BigDecimal("0.1")` OR `BigDecimal.valueOf(0.1)`, not `new BigDecimal(0.1)`.
- *What is a subnormal number?* A number below `MIN_NORMAL`, with reduced precision, filling the gap between 0 and the smallest normal value.

A18. If you add a null to an empty Set, what is the size?

It depends on the implementation.

Set	add(null) on empty set	Size
HashSet	Allowed (null hashes to 0, bucket 0)	1
LinkedHashSet	Allowed	1
TreeSet (natural ordering)	<code>NullPointerException</code> (since Java 7, <code>TreeMap.put</code> calls <code>compare(key, key)</code> even on an empty map)	—
<code>TreeSet(Comparator.nullsFirst(...))</code>	Allowed	1
<code>Set.of(null)</code> / <code>Set.copyOf</code> / <code>Collectors.toUnmodifiableSet</code>	<code>NullPointerException</code> (immutable sets reject nulls)	—
<code>Set.of().add(null)</code>	<code>UnsupportedOperationException</code> . Immutable, it fails before the null check	—
<code>CopyOnWriteArraySet</code>	Allowed	1
<code>ConcurrentHashMap.newKeySet()</code> / <code>ConcurrentSkipListSet</code>	<code>NullPointerException</code>	—
<code>EnumSet</code>	<code>NullPointerException</code>	—

```
import java.util.*;

public class NullSetDemo {
    public static void main(String[] args) {
        Set<String> hs = new HashSet<>();
        System.out.println(hs.add(null)); // true
        System.out.println(hs.add(null)); // false - duplicate, set semantics
        System.out.println(hs.size());    // 1
        System.out.println(hs.contains(null)); // true

        Set<String> ts = new TreeSet<>();
        try { ts.add(null); } catch (NullPointerException e) { System.out.println("TreeSet NPE"); }

        Set<String> tsNull = new TreeSet<>(Comparator.nullsFirst(Comparator.naturalOrder()));
        tsNull.add(null); tsNull.add("a");
        System.out.println(tsNull);        // [null, a]

        try { Set.of((String) null); } catch (NullPointerException e) { System.out.println("Set.of NPE"); }
    }
}
```

Why: `HashSet` is backed by a `HashMap` (elements are keys, values are a dummy `PRESENT` object), and `HashMap` explicitly supports one null key. `TreeSet` must *compare* elements, and `compareTo` on null throws. Java 9+ immutable collections deliberately ban nulls to avoid ambiguity.

Historical note: on Java 6, adding null as the **first** element of an empty `TreeSet` succeeded, and the next add threw NPE. Java 7 fixed this.

Likely follow-ups:

- *How many nulls can a `HashMap` hold?* One null key, any number of null values.
- *Why does `ConcurrentHashMap` reject null?* A `get()` returning null would be ambiguous ("absent" vs "mapped to null"), and you can't check atomically with `containsKey` in a concurrent setting.
- *What backs a `HashSet`?* A `HashMap<E, Object>` with a shared `PRESENT` dummy value.

A19. Can we call one constructor from another? How?

Yes, through **constructor chaining**:

- `this(...)` calls another constructor of the **same** class.
- `super(...)` calls a constructor of the **parent** class.

```
public class Camera {
    private final String id;
    private final String location;
    private final int resolution;

    public Camera(String id) {
        this(id, "UNASSIGNED");           // chain -> 2-arg
    }
    public Camera(String id, String location) {
        this(id, location, 1080);        // chain -> 3-arg
    }
    public Camera(String id, String location, int resolution) { // the "master" constructor
        super();                         // implicit if omitted (Object())
        if (resolution <= 0) throw new IllegalArgumentException("resolution");
        this.id = id; this.location = location; this.resolution = resolution;
    }
}

class PtzCamera extends Camera {
    private final int zoom;
    PtzCamera(String id, int zoom) {
        super(id, "Junction-12", 4000);   // must be first statement
        this.zoom = zoom;
    }
}
```

Rules

1. `this(...)` or `super(...)` must be the **first statement** in the constructor. You can't use both in one constructor.
2. If neither is written, the compiler inserts `super()`. If the parent has no no-arg constructor, that's a **compile error**.
3. **Recursive constructor invocation** ($A \rightarrow B \rightarrow A$) is a compile error.
4. You can't call a constructor like a normal method (`Camera("x")` is invalid). Only `this(...)`, `super(...)` or `new`.
5. The arguments to `this(...)` / `super(...)` can't reference instance fields or methods (the object isn't initialised yet). Static methods and parameters are fine: `this(normalize(id))` where `normalize` is static.
6. Instance initializer blocks run once, right after `super(...)` returns. They don't run again for a `this(...)` chain.

Newer Java: *Flexible Constructor Bodies* (JEP 447 preview in 22, previews in 23/24, **final in Java 25 via JEP 513**) allow statements such as validation **before** `super(...)` / `this(...)`, provided they don't touch `this`.

Alternatives to telescoping constructors: the **Builder pattern** (Lombok `@Builder`), static factory methods, and records with compact constructors.

Likely follow-ups:

- *Why must `super()` come first?* So the parent part of the object is initialised before the child uses it.
- *Can a constructor be private?* Yes, for singletons, factories, or utility classes.
- *Can constructors be inherited or overridden?* No. They aren't members. They can only be chained.

A20. Explain composition and aggregation with examples, and how you'd use composition in a real project.

Both are **"has-a"** associations. The difference is **ownership and lifecycle**.

	Association	Aggregation	Composition
Relationship	Uses / knows	Has-a (weak)	Part-of (strong)
Ownership	None	Shared	Exclusive
Lifecycle	Independent	Part can exist without the whole	Part dies with the whole
UML	Line	Hollow diamond ◇	Filled diamond ◆
Example	Doctor ↔ Patient	Department ◇— Employee	House ◆— Room, Order ◆— OrderLine

```
import java.util.*;

// AGGREGATION: Employees exist independently and are passed in; department doesn't own their lifecycle
class Employee { final String name; Employee(String n) { name = n; } }
class Department {
    private final List<Employee> members = new ArrayList<>();
    void add(Employee e) { members.add(e); } // reference to an externally created object
}

// COMPOSITION: Engine is created and owned by Car; it is never shared or exposed
final class Car {
    private final Engine engine; // created internally
    Car(int hp) { this.engine = new Engine(hp); }
    void start() { engine.ignite(); }
    private static final class Engine { // even its type is hidden
        private final int hp;
        Engine(int hp) { this.hp = hp; }
        void ignite() { System.out.println("Engine " + hp + "hp started"); }
    }
}
```

If you delete the `Department`, the employees still exist (aggregation). When a `Car` becomes unreachable, its `Engine` goes with it (composition).

Composition in a real project (favour composition over inheritance)

In a Smart City **Incident Management** service, inheritance like `SmsIncidentService` extends `IncidentService` explodes combinatorially (SMS + Email + escalation variants). Instead, **compose** behaviours behind interfaces and let Spring inject them:

```
import java.util.List;

interface NotificationChannel { void notify(Incident i); }
interface EscalationPolicy { boolean shouldEscalate(Incident i); }
interface IncidentRepository { Incident save(Incident i); }
record Incident(String id, String type, int severity) {}

// @Service in Spring - collaborators are composed via constructor injection
class IncidentService {
    private final IncidentRepository repo;
    private final List<NotificationChannel> channels; // SMS, Email, Push: plug and play
    private final EscalationPolicy escalation;

    IncidentService(IncidentRepository repo, List<NotificationChannel> channels, EscalationPolicy escalation) {
        this.repo = repo; this.channels = channels; this.escalation = escalation;
    }

    Incident raise(Incident i) {
        Incident saved = repo.save(i);
        if (escalation.shouldEscalate(saved)) channels.forEach(c -> c.notify(saved));
        return saved;
    }
}
```

Benefits:

- Swap behaviour at runtime or by config (Strategy pattern).
- Easy unit testing with mocks.
- No fragile base class.
- Respects SRP and OCP: a new channel is just a new bean.

JPA analogy: `@OneToMany(cascade = ALL, orphanRemoval = true)` on `Order` → `OrderLine` models **composition**. A `@ManyToOne` `Department` reference from `Employee` models **aggregation/association**.

Likely follow-ups:

- *Why favour composition over inheritance?* Lower coupling, runtime flexibility, no exposure to superclass internals, and it avoids deep hierarchies.
- *When is inheritance right?* A true is-a relationship with a stable base designed for extension (e.g. template method in an abstract class).
- *Composition vs delegation?* Delegation is the technique (forwarding calls to a composed object). Composition is the structural relationship.

A21. What is polymorphism? Real project example.

Polymorphism ("many forms") lets one reference type or method name represent different behaviour depending on the actual object or arguments.

Type	Also called	Mechanism	Resolved
Compile-time	Static, early binding	Method overloading (same name, different parameter lists), operator + for String	By the compiler, from the static types of the arguments
Runtime	Dynamic, late binding	Method overriding + upcasting	By the JVM (<code>invokevirtual</code> / <code>invokeinterface</code> , vtable/itable lookup), from the object's runtime class

Generics (*parametric polymorphism*) are a third flavour: `List<T>` works for any `T`.

```
// Compile-time polymorphism: overloading
class ReportExporter {
    String export(List<String> rows)           { return "CSV"; }
    String export(List<String> rows, String fmt) { return fmt; }
    String export(Map<String, Object> json)     { return "JSON"; }
}
```

Real project example: Smart City alert dispatch

```
import java.util.*;

sealed interface AlertHandler permits FireAlertHandler, TrafficAlertHandler, FloodAlertHandler {
    String type();
    void handle(Map<String, Object> payload);
}

final class FireAlertHandler implements AlertHandler {
    public String type() { return "FIRE"; }
    public void handle(Map<String, Object> p) { System.out.println("Dispatch fire tender to " + p.get("loc")); }
}

final class TrafficAlertHandler implements AlertHandler {
    public String type() { return "TRAFFIC"; }
    public void handle(Map<String, Object> p) { System.out.println("Reroute signals near " + p.get("loc")); }
}

final class FloodAlertHandler implements AlertHandler {
    public String type() { return "FLOOD"; }
    public void handle(Map<String, Object> p) { System.out.println("Open drains, alert ward " + p.get("ward")); }
}

class AlertRouter {
    private final Map<String, AlertHandler> handlers = new HashMap<>();
    AlertRouter(List<AlertHandler> all) { // in Spring: inject List<AlertHandler>
        all.forEach(h -> handlers.put(h.type(), h));
    }
    void route(String type, Map<String, Object> payload) {
        AlertHandler h = handlers.get(type);
        if (h == null) throw new IllegalArgumentException("Unknown alert " + type);
        h.handle(payload); // runtime polymorphism - no if/else chain
    }
}
```

Adding a new alert type means adding a new class, with **no change** to `AlertRouter` (Open/Closed Principle). Spring itself relies on polymorphism everywhere: `JpaRepository` proxies, `PasswordEncoder`, `HandlerInterceptor`.

Gotchas

- **Fields and static methods are not polymorphic.** They're resolved by reference type (A13).
- Overload resolution uses the **compile-time** argument types. `print(Object o)` vs `print(String s)` called with `Object x = "hi"` → `print(Object)`.
- Calling an overridable method from a constructor runs the subclass version **before** subclass fields are initialised, so you may see nulls or zeros.

Likely follow-ups:

- *Overloading vs overriding?* Overloading has the same name with different parameters (compile time). Overriding has the same signature in a subclass (runtime).
- *Can we overload by return type only?* No. That's a compile error, because the signature excludes the return type.
- *How does the JVM do dynamic dispatch?* Each class has a virtual method table. `invokevirtual` looks up the slot in the receiver's class. The JIT often inlines monomorphic call sites.

A22. Explain method overriding with a runtime polymorphism example (rules).

Overriding is when a subclass provides its own implementation of an **inherited instance method** with the **same signature** (name + parameter types). The JVM picks the implementation based on the **runtime object type**.

```
import java.io.*;

class Report {
    protected Object generate() throws IOException { return "generic"; }
    public void print() { System.out.println("Report: " + generate0()); }
    private Object generate0() { try { return generate(); } catch (IOException e) { return "err"; } }
}

class TrafficReport extends Report {
    @Override
    public String generate() // compiler checks this really overrides
                           // wider access (protected -> public): OK
                           throws FileNotFoundException { // narrower checked exception: OK
        return "traffic stats"; // covariant return: String is-a Object
    }
}

public class OverrideDemo {
    public static void main(String[] args) {
        Report r = new TrafficReport(); // upcast
        r.print(); // "Report: traffic stats" - runtime dispatch
    }
}
```

Rules

Rule	Detail
Signature	Same name and parameter types (after erasure for generics)
Return type	Same, or a covariant subtype for reference types (Java 5+). Primitives must match exactly
Access modifier	Same or wider (private < default < protected < public). Never narrower
Checked exceptions	Same, narrower (subclass), fewer or none. Never new or broader checked exceptions
Unchecked exceptions	Any. No restriction
Cannot override	final , static (hidden instead), private (not inherited), constructors
Package-private	Overridable only by subclasses in the same package
<code>super.method()</code>	Calls the parent's implementation
<code>@Override</code>	Optional but strongly recommended. It catches typos such as <code>equals(Employee e)</code> overloading instead of overriding
<code>synchronized</code> / <code>strictfp</code>	Not part of the signature. They can be added or removed freely

Why the access and exception rules? Because of the **Liskov Substitution Principle**: code written against `Report r` must still work when `r` is a `TrafficReport`. If the override were less accessible, or threw a checked exception the caller isn't prepared for, substitution would break.

Classic trap: overloading disguised as overriding

```
class Point {
    int x;
    public boolean equals(Point p) { return p.x == x; } // OVERLOAD, not override of equals(Object)
}
// HashSet / HashMap call equals(Object), so this method is never used. @Override would have caught it.
```

Likely follow-ups:

- Can an overriding method throw a `RuntimeException` the parent doesn't declare? Yes. Unchecked exceptions are unrestricted.
- What's a covariant return type? An overriding method returning a subtype of the original return type (e.g. `clone()` returning `MyClass` instead of `Object`).
- Can we override with a generic parameter change? Only if the erasures match. `void m(List<String>)` and `void m(List<Integer>)` clash with the error "name clash: same erasure".

A23. Class A has a public method; class B overrides it as private. Will it work? Why?

No. It's a compile-time error.

```
class A {
    public void show() { System.out.println("A"); }
}
class B extends A {
```

```
// private void show() { System.out.println("B"); }
// error: show() in B cannot override show() in A
//      attempting to assign weaker access privileges; was public
}
```

Why

- Overriding rule: the overriding method **cannot reduce visibility**. Java checks this because of **substitutability (LSP)**:

```
A ref = new B();
ref.show(); // the compiler allows this because A.show() is public...
            // ...so at runtime B's version MUST be callable from anywhere
```

- If B could make it private, a perfectly legal call through an A reference would hit a method the caller isn't allowed to access, which breaks the type system.
- Allowed directions: `protected` → `public` (widening) is fine, and `public` → `public` is fine. `public` → `protected/default/private` is an error.
- The same applies to interface methods. Implementing an interface method (implicitly public) requires `public` in the class. Omitting `public` gives "attempting to assign weaker access privileges".

Contrast with the reverse (A23 vs A11): if A's method is `private` and B declares a `public` method with the same signature, it compiles. It's **not** an override, just a new method (A11).

Likely follow-ups:

- Can B override it as `protected`? No, that's still weaker than public. Same error.
- What if the method is static in both? Hiding follows the same rule: you can't hide with weaker access.
- Why can overriding widen access? Widening never breaks existing callers. Anything callable before is still callable.

A24. Difference between HashMap and Hashtable.

Aspect	HashMap (Java 1.2)	Hashtable (Java 1.0, legacy)
Thread safety	Not synchronized	Every public method is <code>synchronized</code> (one lock on the whole table)
Performance	Fast	Slow under contention (all threads serialize)
Null key/values	One null key , any number of null values	No nulls (NPE for key or value)
Superclass	<code>AbstractMap</code>	<code>Dictionary</code> (obsolete abstract class)
Iteration	<code>Iterator</code> , fail-fast (<code>ConcurrentModificationException</code>)	<code>Enumeration</code> (<code>keys()</code> , <code>elements()</code>) is not fail-fast; its <code>Iterator</code> views are fail-fast
Default capacity / growth	16, doubles (power of two)	11, grows to $2n + 1$
Index calculation	$(n - 1) \& \text{hash}$ with hash spreading $h \wedge (h \ggg 16)$	$(\text{hash} \& 0x7FFFFFFF) \% \text{table.length}$
Collision handling	Linked list → red-black tree when the bucket has ≥ 8 entries (Java 8)	Linked list only
Ordering	None (use <code>LinkedHashMap</code> for insertion/access order)	None
Java 8 extras	<code>compute</code> , <code>merge</code> , <code>forEach</code> , <code>getOrDefault</code> (both have them via <code>Map</code> defaults, <code>Hashtable</code> overrides them synchronized)	Same API through <code>Map</code>
Recommendation	Single-threaded or confined use	Don't use . For concurrency use <code>ConcurrentHashMap</code>

```
import java.util.*;

public class MapCompare {
    public static void main(String[] args) {
        Map<String, String> hm = new HashMap<>();
        hm.put(null, "nullKey"); hm.put("k", null); // fine
        System.out.println(hm); // {null=nullKey, k=null}

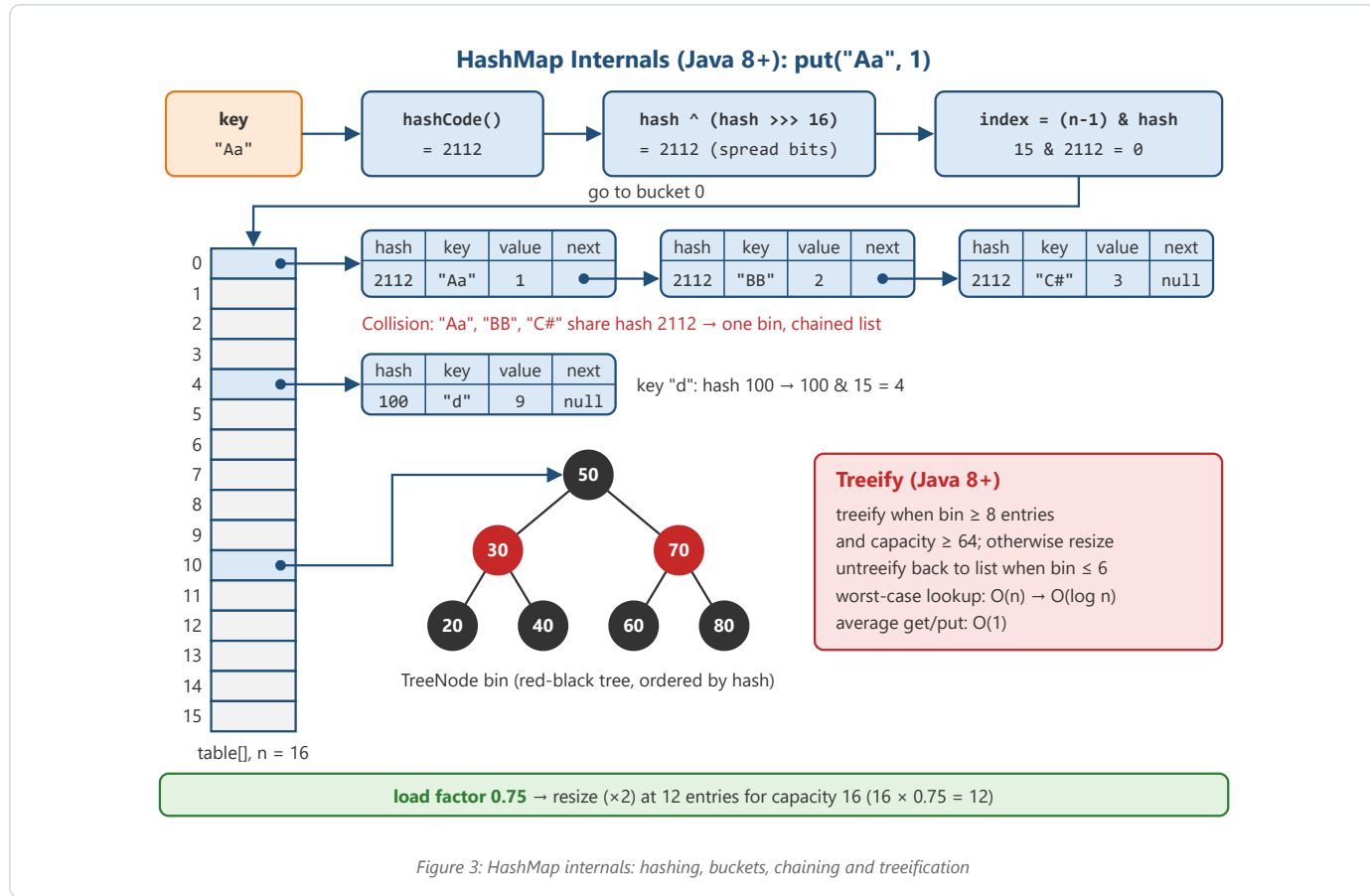
        Map<String, String> ht = new Hashtable<>();
        try { ht.put(null, "x"); } catch (NullPointerException e) { System.out.println("Hashtable: no null key"); }
        try { ht.put("k", null); } catch (NullPointerException e) { System.out.println("Hashtable: no null value"); }
    }
}
```

Gotcha: even `Hashtable`'s per-method synchronization doesn't make *compound* operations atomic. `if (!t.containsKey(k)) t.put(k, v)` is still a race. Use `ConcurrentHashMap.putIfAbsent / computeIfAbsent`.

Likely follow-ups:

- *HashMap vs Collections.synchronizedMap(new HashMap<>())* ? The latter wraps with a single mutex like `Hashtable` (and allows nulls). Iteration still needs manual `synchronized(map)`.
- *Why is Hashtable's index a modulo while HashMap's is a mask?* `HashMap` keeps the capacity a power of two, so `& (n-1)` equals `% n` and is cheaper.
- *Is Hashtable deprecated?* Not formally, but it's considered legacy.

A25. Internal working of HashMap.



Structure (Java 8+): an array `Node<K,V>[] table` of **buckets**. Each bucket is `null`, a linked list of `Node`s, or a red-black tree of `TreeNode`s.

```
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash; // cached spread hash
    final K key;
    V value;
    Node<K,V> next;
}
```

Key constants

Constant	Value	Meaning
DEFAULT_INITIAL_CAPACITY	16	Always a power of two (<code>tableSizeFor</code> rounds up)
DEFAULT_LOAD_FACTOR	0.75	Trade-off between space and collision probability
threshold	capacity × loadFactor	Resize when <code>size > threshold</code> (12 for the default)
TREEIFY_THRESHOLD	8	A bucket's list becomes a tree when it reaches 8 nodes...
MIN_TREEIFY_CAPACITY	64	...but only if <code>table.length >= 64</code> . Otherwise the table resizes instead

Constant	Value	Meaning
UNTREEIFY_THRESHOLD	6	A tree converts back to a list when it shrinks to ≤ 6 (during resize split)
MAXIMUM_CAPACITY	2^{30}	

put(key, value) step by step

1. Compute the hash with spreading:

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

XOR-ing the high 16 bits into the low 16 matters because the index uses only the low bits. Without it, hashCodes that differ only in their high bits (e.g. `Float` keys, or multiples of 2^{16}) would all collide.

2. Lazy init: the table is allocated on the first `put (resize())`, not in the constructor.

3. Find the bucket index: `index = (n - 1) & hash`. With `n = 16`, `n - 1 = 0b1111`, so it takes the low 4 bits. That equals `hash % n` for power-of-two `n`, but is faster and is never negative.

4. Empty bucket → place a new Node.

5. Non-empty bucket → walk the chain. For each node, check `p.hash == hash && (p.key == key || key.equals(p.key))`.

- Match → **replace the value** (the key object stays the same) and return the old value.
- No match → **append at the tail** (Java 8; Java 7 inserted at the head).
- If the chain length reaches `TREEIFY_THRESHOLD (8)` → `treeifyBin`: treeify if `table.length >= 64`, else resize.
- If the bucket is a `TreeNode` → `putTreeVal` ($O(\log n)$).

6. `modCount++` (drives fail-fast iterators). If `++size > threshold`, call `resize()`.

resize()

- Doubles the capacity (`newCap = oldCap << 1`) and the threshold.
- Rehash is cheap: since `n` doubles, each node either stays at index `i` or moves to `i + oldCap`, depending on a single bit: `(hash & oldCap) == 0`. The bucket is split into "lo" and "hi" lists, **preserving relative order**.
- Tree bins are split the same way. If a half has ≤ 6 nodes, it's untreeified.
- Cost: $O(n)$ when it happens, amortized $O(1)$ per put. **Pre-size** when you know the count: `new HashMap<>(expected / 0.75f + 1)`, or Java 19+ `HashMap.newHashMap(expected)`.

`get(key)`: compute the hash → index → check the first node → then traverse the list ($O(k)$) or the tree ($O(\log k)$), using `hash ==` then `equals`.

Tree bins: ordered by hash. On ties, by `compareTo` if keys are mutually `Comparable`, else `tieBreakOrder` (class name, then `System.identityHashCode`). Treeification is a **defence** against poor hash functions and hash-flooding attacks. With a good hash the probability of a bucket reaching 8 is ~ 0.00000006 (Poisson distribution, per the JDK source comments).

Complexity: average $O(1)$ for get/put/remove. Worst case $O(\log n)$ in Java 8+ ($O(n)$ in Java 7, or for non-Comparable keys with identical hashes).

Why Java 8 switched from head to tail insertion: in Java 7, concurrent resizes with head insertion could reverse a list and create a **cycle**, which made `get()` spin forever at 100% CPU. Tail insertion avoids the cycle, but `HashMap` is **still not thread-safe** (lost updates, corrupted size).

Likely follow-ups:

- Why is capacity a power of two?* So that `(n-1) & hash` replaces the slow `%`, and so the resize split works with a single bit.
- Why load factor 0.75?* An empirical balance. Higher means more collisions. Lower means more memory and more frequent resizes.
- Why treeify at 8 but untreeify at 6?* The gap (hysteresis) stops a bucket flapping between list and tree around one threshold.
- Is iteration order guaranteed?* No. It can change after a resize. Use `LinkedHashMap` or `TreeMap`.

A26. How are HashMap collisions handled?

A **collision** happens when two different keys land in the **same bucket index**. That happens either because their hashCodes are equal, or because `(n-1) & hash` maps different hashes to the same slot.

HashMap uses separate chaining:

1. Linked list in the bucket (Java 8 appends at the tail). Lookup walks the chain comparing `hash` first (a cheap int check), then `==`, then `equals()`.

- Treeification (Java 8)**: when a single bucket reaches **8** nodes and the table has ≥ 64 buckets, the list becomes a **red-black tree** (`TreeNode`), so lookups go from $O(k)$ to $O(\log k)$. If the table is smaller than 64, `HashMap` **resizes** instead, because spreading entries is the better fix when the table is small.
- Untreeify** back to a list when a bin drops to ≤ 6 nodes during a resize split.
- Resizing** reduces collisions overall by doubling the bucket count and spreading the entries.

```
import java.util.*;

public class CollisionDemo {
    record BadKey(int id) {
        @Override public int hashCode() { return 42; } // every key collides
    }
    public static void main(String[] args) {
        Map<BadKey, String> m = new HashMap<>();
        for (int i = 0; i < 100; i++) m.put(new BadKey(i), "v" + i);
        System.out.println(m.get(new BadKey(77))); // v77 - still correct, just slower
        // All 100 entries are in one bucket -> treeified once table >= 64.
        // BadKey isn't Comparable, so the tree falls back to tieBreakOrder: lookups may degrade toward O(n).
    }
}
```

"Aa" and "BB" have the same `String.hashCode()` (2112). That's a classic interview example of a real collision. Keys still stay distinct because `equals()` distinguishes them.

Other collision strategies (for comparison)

Strategy	Used by
Separate chaining (list/tree)	<code>HashMap</code> , <code>ConcurrentHashMap</code> , <code>Hashtable</code> (list only)
Open addressing, linear probing	<code>IdentityHashMap</code> , <code>ThreadLocal.ThreadLocalMap</code>
Open addressing, probing	<code>Set.of</code> / <code>Map.of</code> immutable implementations (<code>ImmutableCollections.SetN/MapN</code>)

Reducing collisions in your code

- Write a good `hashCode()` using all the significant fields (`Objects.hash(...)` , or records/Lombok generate it).
- Use immutable keys.
- Implement `Comparable` on key classes that are likely to collide, so that tree bins can order them.

Likely follow-ups:

- Is a collision the same as a duplicate key?* No. A duplicate is `equals()` -equal, and its value gets replaced. A collision is a different key in the same bucket, and both are kept.
- Why compare hash before equals?* An int comparison is cheap and filters out most non-matches before the potentially expensive `equals`.
- What is hash flooding?* An attacker sends many keys with identical hashes (e.g. crafted query params) to make the map degrade. Treeification limits the damage.

A27. hashCode(), equals(), toString() in the context of HashMap.

Roles

Method	Role in HashMap	Default (Object)
<code>hashCode()</code>	Chooses the bucket (after spreading and masking)	Identity-based int (not the memory address on modern JVMs; often a random or thread-local generated value stored in the object header)
<code>equals(Object)</code>	Finds the exact key within the bucket	Reference equality <code>==</code>
<code>toString()</code>	Not used for storage or lookup. Only for printing (<code>map.toString()</code> calls each key's and value's <code>toString()</code>) and logging	<code>ClassName@hexHashCode</code> , e.g. <code>Employee@1b6d3586</code>

The equals/hashCode contract (from `Object` Javadoc)

- Consistency**: repeated `hashCode()` calls return the same value while the fields used in `equals` don't change.
- $a.equals(b) \Rightarrow a.hashCode() == b.hashCode()$. This is mandatory.
- Equal hash codes do **not** imply `equals` (collisions are allowed).
- `equals` must be **reflexive, symmetric, transitive, consistent**, and `x.equals(null) == false` .

What breaks if you violate it

Override	Effect in HashMap
Neither	Identity semantics. Two "logically equal" objects are different keys
Only <code>equals</code>	Equal objects usually land in different buckets . Duplicates are stored and <code>get</code> with an equal-but-different object returns <code>null</code>
Only <code>hashCode</code>	Same bucket, but <code>equals</code> uses <code>==</code> , so they are still treated as different keys (duplicates stay in one bucket)
Both, consistently	Correct logical-key semantics

Correct implementation (Java 17)

```
import java.util.Objects;

public final class Employee {
    private final String name;
    public Employee(String name) { this.name = name; }
    public String getName() { return name; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Employee other)) return false; // pattern matching (16+)
        return Objects.equals(name, other.name);
    }
    @Override public int hashCode() { return Objects.hashCode(name); } // or Objects.hash(f1, f2...)
    @Override public String toString() { return "Employee[name=" + name + "]"; }
}
// Or simply: public record Employee(String name) {} -> equals/hashCode/toString generated
```

Gotchas

- Use the **same fields** in both `equals` and `hashCode`.
- `getClass() != o.getClass()` vs `instanceof`: `instanceof` allows subclasses but can break symmetry if a subclass adds fields. Make value classes `final`.
- JPA entities: don't base `hashCode` on a generated `@Id` that is null before persist. Use a business key, or a constant `hashCode` with id-based `equals`.
- Lombok `@Data` on JPA entities with bidirectional relations → `StackOverflowError` in `hashCode` / `toString`.
- Arrays as keys use identity `equals`. Wrap them, or use `Arrays.hashCode` / `Arrays.equals` in your own key class.

Likely follow-ups:

- Can two unequal objects have the same hashCode?* Yes. That's a collision, and it's legal.
- Why must equal objects have equal hash codes?* Otherwise `HashMap` looks in the wrong bucket and never calls `equals`.
- Does HashMap use toString?* No. It's only used in `map.toString()` and debugging output.

A28. What happens when custom objects are used as HashMap keys?

```
Employee e1 = new Employee("Anshul");
Employee e2 = new Employee("Ravi");
Employee e3 = new Employee("Anshul");
HashMap<Employee, Integer> empM = new HashMap<>();
empM.put(e1, 101);
empM.put(e2, 102);
empM.put(e3, 103);
```

Case 1: `equals` / `hashCode` NOT overridden

```
class Employee {
    private final String name;
    Employee(String name) { this.name = name; }
}
```

- `Object.hashCode()` is identity-based, so `e1`, `e2` and `e3` (three distinct objects) almost certainly get different hashes. `Object.equals` uses `==`, so even with a collision, `e1 != e3`.
- Result:** `empM.size()` → 3. Both "Anshul" objects are separate keys.
- `empM.get(e1)` → 101, `empM.get(e3)` → 103.
- `empM.get(new Employee("Anshul"))` → `null` (a new identity). This is the common bug: you can't look up by value.

- `System.out.println(empM)` → something like `{Employee@1b6d3586=101, Employee@4554617c=102, Employee@74a14482=103}` (order and hashes vary).

Case 2: equals / hashCode overridden on name (as in A27, or record `Employee(String name)`)

- `e1.hashCode() == e3.hashCode()` and `e1.equals(e3)` is `true`, so `e3` is a **duplicate key**.
- `put(e3, 103)` **replaces the value** of the existing entry and returns the old value `101`. **The key object stored remains `e1`**. `HashMap` doesn't replace the key.
- **Result:** `empM.size()` → `2`.
- `empM.get(e1)` → `103`, `empM.get(e3)` → `103`, `empM.get(new Employee("Anshul"))` → `103`.
- With the `toString()` override: `{Employee[name=Ravi]=102, Employee[name=Anshul]=103}` (iteration order isn't guaranteed).
- `empM.keySet().iterator()` returns the **`e1`** instance for Anshul. You can verify with `==`.

	Without overrides	With overrides on <code>name</code>
<code>size()</code>	3	2
<code>get(e1)</code>	101	103
<code>get(new Employee("Anshul"))</code>	null	103
Return of <code>put(e3,103)</code>	null	101

Partial overrides: only `equals` → most likely size 3 (different buckets, `equals` never called). Only `hashCode` → size 3 (same bucket, but `==` comparison).

Mutable-key pitfall

```
import java.util.*;

class MutableKey {
    String name;
    MutableKey(String n) { name = n; }
    @Override public boolean equals(Object o) { return o instanceof MutableKey m && Objects.equals(name, m.name); }
    @Override public int hashCode() { return Objects.hashCode(name); }
}

public class MutableKeyDemo {
    public static void main(String[] args) {
        Map<MutableKey, Integer> map = new HashMap<>();
        MutableKey k = new MutableKey("Anshul");
        map.put(k, 101);
        k.name = "Changed"; // hash changes, entry stays in old bucket
        System.out.println(map.get(k)); // null (looks in new bucket)
        System.out.println(map.get(new MutableKey("Anshul"))); // null (old bucket, but equals fails)
        System.out.println(map.size()); // 1 - entry is "lost" -> memory leak
        System.out.println(map.containsKey(k)); // false
    }
}
```

Rule: keys must be **immutable**, or at least the fields used in `equals` / `hashCode` must never change while the object is a key.

Records as keys (best practice, Java 16+)

```
record EmployeeKey(String name, String dept) {} // final, immutable fields, auto equals/hashCode/toString
Map<EmployeeKey, Integer> ids = new HashMap<>();
ids.put(new EmployeeKey("Anshul", "IT"), 101);
ids.get(new EmployeeKey("Anshul", "IT")); // 101
```

Caveat: a record is only *shallowly* immutable. A `List` component can still be mutated, so use `List.copyOf` in a compact constructor. Other good key types are `String`, `Integer` and enums (immutable, with cached hash in `String`'s case).

Likely follow-ups:

- Does `put` with an equal key replace the key? No. Only the value. The original key instance is kept.
- What's the cost of a poor `hashCode` (e.g. constant)? Every entry goes into one bucket: $O(\log n)$ if the key is `Comparable` (tree), else it approaches $O(n)$.
- Why is `String` a good key? It's immutable, caches its `hashCode`, and has a well-distributed hash.
- `TreeMap` with custom keys? It needs `Comparable` or a `Comparator`, and uses `compareTo`, not `equals` / `hashCode`. They should be consistent with each other.

A29. HashMap vs ConcurrentHashMap.

Aspect	HashMap	ConcurrentHashMap (Java 8+)
Thread safety	None. Concurrent writes cause lost updates or a corrupted size	Thread-safe, highly concurrent
Locking	—	CAS to insert into an empty bin. synchronized on the bin's head node for non-empty bins. Reads are lock-free (volatile reads of <code>Node.val / next</code>)
Java 7 design	—	Segments (default 16, a <code>ReentrantLock</code> each), so concurrency was limited to the segment count
Null keys/values	1 null key, many null values	Neither (NPE)
Iterators	Fail-fast (<code>ConcurrentModificationException</code>)	Weakly consistent : never throw CME, reflect some state at or after creation, may or may not show concurrent updates
<code>size()</code>	Exact field	Approximate under concurrency. Computed from <code>baseCount + CounterCell[]</code> (LongAdder-style striping). Use <code>mappingCount()</code> for a long
Atomic compound ops	None (<code>compute</code> is not atomic across threads)	<code>putIfAbsent</code> , <code>computeIfAbsent</code> , <code>compute</code> , <code>merge</code> , <code>replace(k, old, new)</code> are atomic
Resize	Single thread	Cooperative : multiple threads help transfer bins (<code>ForwardingNode</code> markers)
Treeify	Yes (8 / 64)	Yes (<code>TreeBin</code> with its own read-write lock)
Bulk ops	—	<code>forEach</code> , <code>reduce</code> , <code>search</code> with a parallelism threshold
Performance (single thread)	Faster	Slightly slower

Why no nulls? In a concurrent map, `get(k) == null` would be ambiguous: is the key absent, or mapped to null? You can't resolve it with a follow-up `containsKey(k)` because another thread may change the map in between. Doug Lea banned nulls to remove the ambiguity.

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.LongAdder;

public class ChmDemo {
    public static void main(String[] args) throws InterruptedException {
        ConcurrentHashMap<String, LongAdder> hits = new ConcurrentHashMap<>();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int i = 0; i < 10_000; i++) {
            String junction = "J" + (i % 5);
            pool.execute(() -> hits.computeIfAbsent(junction, k -> new LongAdder()).increment()); // atomic
        }
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println(hits); // each J0..J4 = 2000

        // WRONG (race) even on CHM - check-then-act across two calls:
        // if (!map.containsKey(k)) map.put(k, v);
        // RIGHT:
        // map.putIfAbsent(k, v); or map.merge(k, 1, Integer::sum);
    }
}
```

Gotchas

- Each CHM method is atomic, but a **sequence** of calls isn't. Use `compute*` / `merge`.
- Keep the function passed to `computeIfAbsent` **short**, and never modify the same map inside it. It holds the bin lock, and recursive updates can deadlock or throw `IllegalStateException("Recursive update")`.
- `ConcurrentHashMap.newKeySet()` gives a concurrent Set.

Likely follow-ups:

- Why not `Collections.synchronizedMap`?* It uses one global lock, so readers block writers and each other. CHM has non-blocking reads and per-bin write locks.
- What does "weakly consistent" mean?* The iterator won't throw CME. It traverses elements as they existed at some point and may reflect later changes.
- `ConcurrentHashMap` vs `ConcurrentSkipListMap`?* CHM is unordered with $O(1)$ average. The skip list map is sorted (a `NavigableMap`), $O(\log n)$, and lock-free.

A30. Java memory areas: Heap, Stack, Metaspace, PC register, native stack.

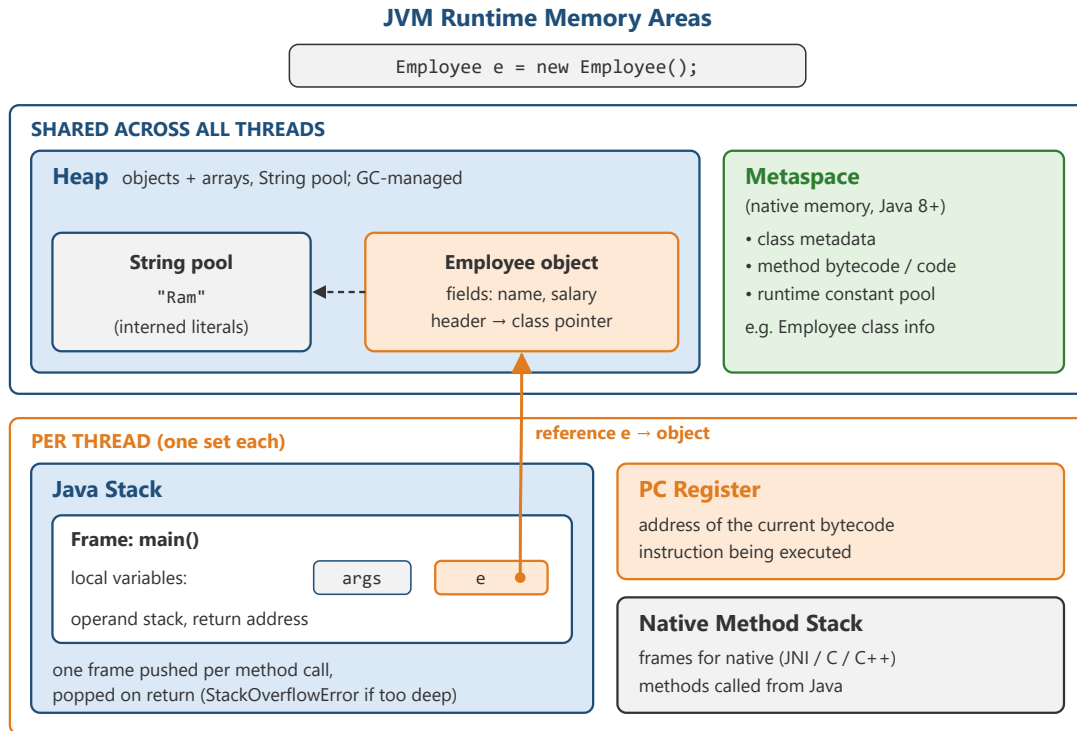


Figure 4: JVM runtime memory areas

The JVM spec defines **runtime data areas**. Some are shared by all threads; others are per thread.

		JVM Process
Shared (all threads)		HEAP (objects, arrays, String pool since Java 7) Young: Eden S0 S1 Old/Tenured METASPACE (native memory): class metadata, method bytecode, runtime constant pool (replaced PermGen) CODE CACHE: JIT-compiled native code
Per thread		JVM STACK: frames {local vars, operand stack, frame data / ref to constant pool, return address} PC REGISTER: address of current bytecode instruction NATIVE METHOD STACK: for JNI/native (C) calls

Area	Shared?	Stores	Size flags	Error when exhausted
Heap	Yes	All objects and arrays, instance fields, interned strings, static field <i>values</i> (they live with the <code>Class</code> object in the heap on HotSpot since 8)	<code>-Xms</code> , <code>-Xmx</code> , <code>-XX:MaxRAMPercentage</code>	<code>OutOfMemoryError</code> : Java heap space
Stack	Per thread	One frame per method call: primitives, object references , partial results	<code>-Xss</code> (e.g. 512k–1m)	<code>StackOverflowError</code> (deep or infinite recursion). OOM: unable to create native thread when there's no room for new stacks
Metaspace	Yes	Class structures, method metadata, constant pools, annotations	<code>-XX:MaxMetaspaceSize</code> (unbounded by default)	<code>OutOfMemoryError</code> : Metaspace
PC register	Per thread	Address of the current instruction (undefined for native methods)	—	None
Native method stack	Per thread	C frames for JNI calls	Platform	<code>StackOverflowError</code> / OOM
Code cache	Yes	JIT-compiled code	<code>-XX:ReservedCodeCacheSize</code>	"CodeCache is full. Compiler has been disabled"

Area	Shared?	Stores	Size flags	Error when exhausted
Direct memory	Yes	ByteBuffer.allocateDirect (Netty, NIO)	-XX:MaxDirectMemorySize	OOM: Direct buffer memory

```
public class MemoryDemo {
    static int counter = 0;           // class metadata in Metaspace; value stored with Class mirror on heap
    public static void main(String[] args) {    // frame for main pushed on main thread's stack
        int x = 10;                      // primitive in stack frame (local variable array)
        String s = new String("city");      // reference 's' on stack; object on heap; literal "city" in String pool (heap)
        int[] arr = new int[3];            // array object on heap
        compute(x);                       // new frame pushed; popped on return
    }
    static int compute(int n) { return n * 2; }
}
```

Key points

- **PermGen → Metaspace (Java 8):** PermGen was a fixed-size part of the heap and caused frequent OOM: PermGen space . Metaspace uses native memory, grows automatically, and its classes are freed when their classloader is GC'd.
- The **String pool** moved from PermGen to the heap in **Java 7**.
- **Escape analysis:** the JIT may scalar-replace objects that don't escape a method, so in effect they are never allocated on the heap. The JVM spec still says "objects live on the heap".
- Stack memory is thread-safe by nature. Heap objects shared between threads need synchronization.

Likely follow-ups:

- *Where are static variables stored?* The variable *metadata* is in Metaspace. On HotSpot 8+, the values live in the java.lang.Class mirror object on the heap.
 - *What causes StackOverflowError?* Unbounded recursion, or very deep call chains (e.g. a bidirectional JPA toString).
 - *Stack vs heap speed?* Stack allocation is a pointer bump with LIFO freeing, which is very fast. Heap allocation also uses pointer-bump TLABs, but needs GC to reclaim.
-

A31. Explain the Java Collection Framework.

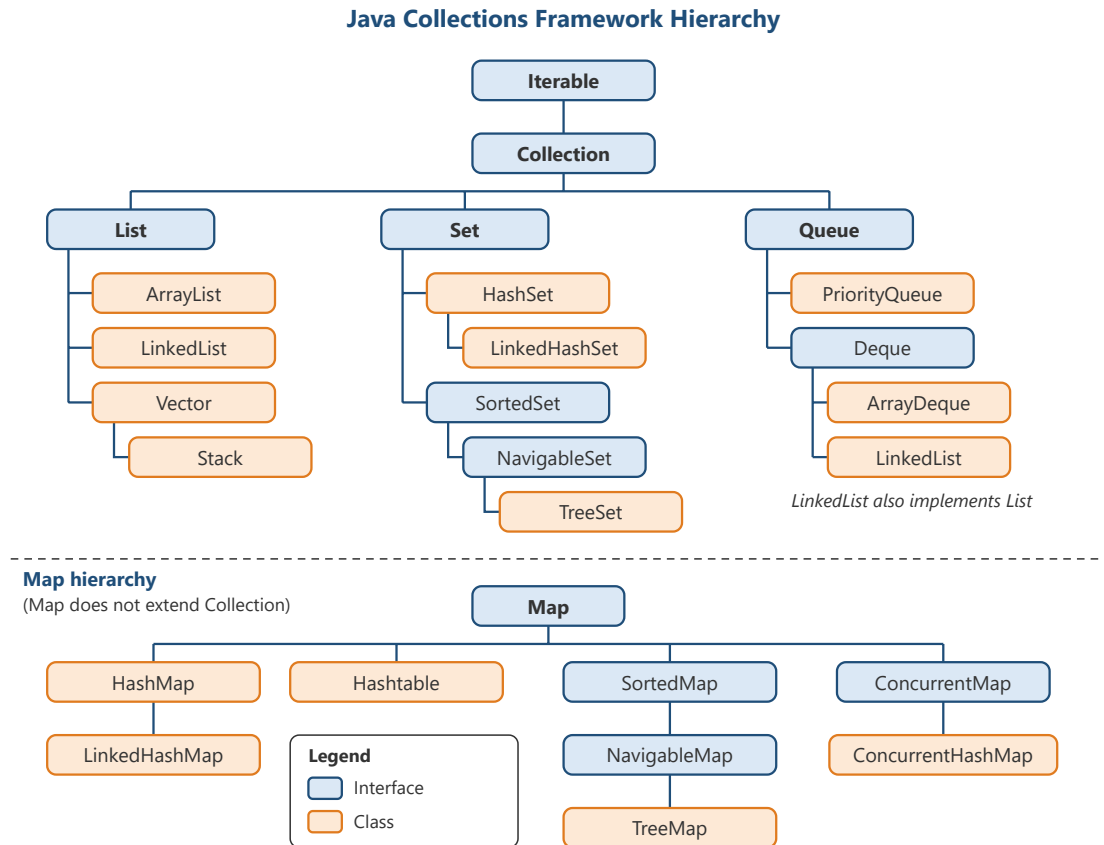
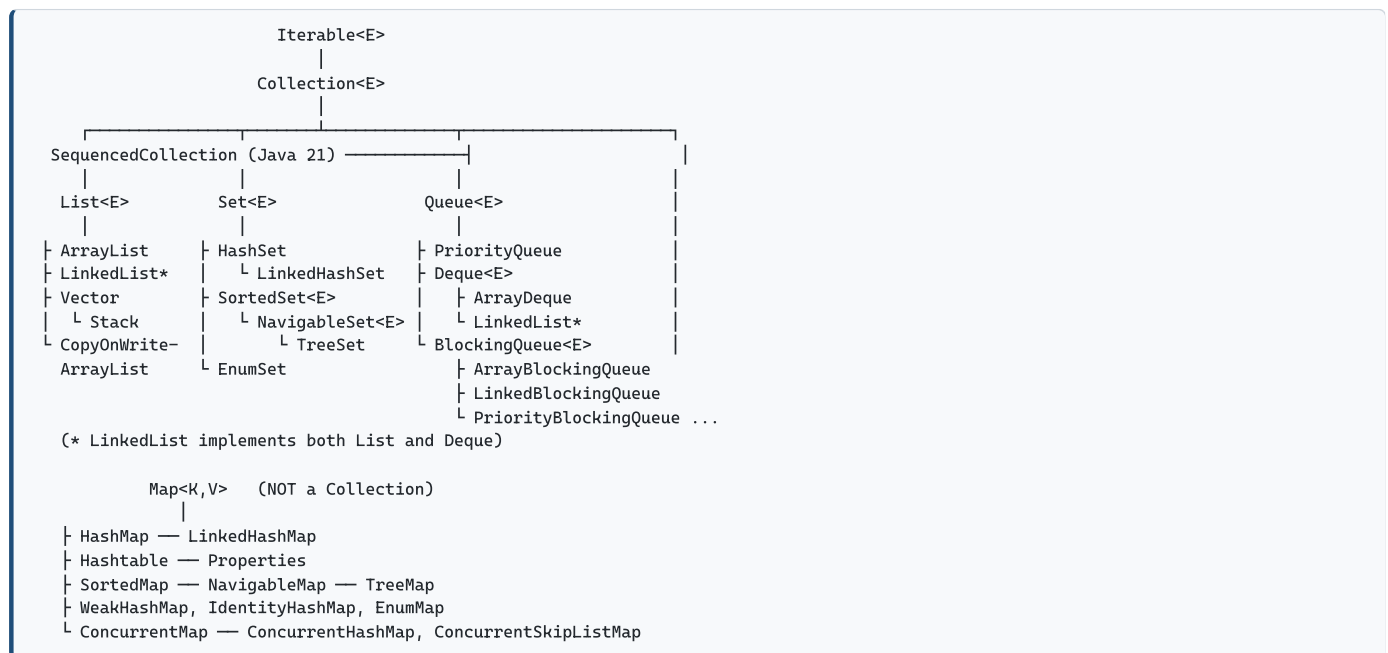


Figure 5: Java Collections Framework hierarchy

The **Java Collections Framework (JCF)**, from Java 1.2, is a unified architecture of **interfaces** (contracts), **implementations** (data structures) and **algorithms** (Collections, Arrays utilities) for storing and manipulating groups of objects.

Hierarchy



Choosing an implementation

Need	Use	Complexity
Index access, mostly appends	<code>ArrayList</code>	get O(1), add amortized O(1), insert in middle O(n)
Queue/stack	<code>ArrayDeque</code> (faster than <code>Stack</code> / <code>LinkedList</code>)	O(1) at both ends
Unique elements, no order	<code>HashSet</code>	O(1) avg
Unique, insertion order	<code>LinkedHashSet</code>	O(1) avg
Sorted / range queries	<code>TreeSet</code> / <code>TreeMap</code> (red-black tree)	O(log n)
Key-value lookup	<code>HashMap</code>	O(1) avg
LRU cache	<code>LinkedHashMap</code> (cap, 0.75f, true) + <code>removeEldestEntry</code>	O(1)
Priority processing	<code>PriorityQueue</code> (binary heap)	offer/poll O(log n), peek O(1)
Enum keys	<code>EnumMap</code> / <code>EnumSet</code> (array/bit-vector)	Very fast
Concurrency	See A32	

Key concepts

- **Comparable vs Comparator**: natural ordering (`compareTo`) vs external ordering (`compare`).
`Comparator.comparing(Emp::dept).thenComparing(Emp::name)`.
- **Fail-fast iterators** throw `ConcurrentModificationException` when a structural change happens outside the iterator (based on `modCount`). Remove safely with `iterator.remove()` or `list.removeIf(...)`.
- **Immutable vs unmodifiable**: `List.of()` (Java 9) is truly immutable and null-hostile. `Collections.unmodifiableList(x)` is a read-only **view**, so changes to `x` show through.
- `Arrays.asList` is a fixed-size view backed by the array: `set` works, `add` / `remove` throw `UnsupportedOperationException`.
- **Sequenced collections (Java 21)**: `getFirst()`, `getLast()`, `reversed()`, `addFirst()` on `List`, `Deque`, `LinkedHashSet`, `LinkedHashMap`.

Likely follow-ups:

- *ArrayList vs LinkedList?* `ArrayList` almost always wins (cache locality, O(1) random access). `LinkedList` only helps for heavy insert/remove through an iterator.
- *Collection vs Collections?* `Collection` is the root interface. `Collections` is a utility class (`sort`, `unmodifiableList`, `synchronizedMap`).
- *Why isn't Map a Collection?* It stores pairs, and `add(E)` doesn't fit. It offers `keySet()`, `values()` and `entrySet()` collection views instead.
- *ArrayList growth?* Default capacity 10 (allocated lazily on the first add), grows by 1.5x (`old + (old >> 1)`).

A32. Concurrent collections and concurrent access in Java.

Problem: ordinary collections (`ArrayList`, `HashMap`) aren't thread-safe. Concurrent modification can cause lost updates, corrupted internal state, `ConcurrentModificationException`, or infinite loops (Java 7 `HashMap`).

Three generations of solutions

Approach	Example	Characteristics
Legacy synchronized	<code>Vector</code> , <code>Hashtable</code> , <code>Stack</code>	Every method synchronized. Slow, compound ops still racy
Synchronized wrappers	<code>Collections.synchronizedList/Map/Set(...)</code>	One mutex. You must manually synchronize when iterating
<code>java.util.concurrent</code> (Java 5+)	<code>ConcurrentHashMap</code> , <code>CopyOnWriteArrayList</code> , <code>BlockingQueue</code> ...	Fine-grained locking or lock-free (CAS), weakly consistent iterators, atomic compound ops

```
List<String> sync = Collections.synchronizedList(new ArrayList<>());
synchronized (sync) {
    // REQUIRED, otherwise CME / inconsistent reads
    for (String s : sync) System.out.println(s);
}
```

java.util.concurrent collections

Class	Mechanism	Best for
<code>ConcurrentHashMap</code>	CAS + per-bin locks	Shared caches, counters (A29)
<code>ConcurrentSkipListMap</code> / <code>Set</code>	Lock-free skip list, sorted	Concurrent sorted data, leaderboards

Class	Mechanism	Best for
<code>CopyOnWriteArrayList</code> / <code>Set</code>	Every write copies the array. Snapshot iterators	Read-mostly lists: listeners, config, subscriber lists
<code>ConcurrentLinkedQueue</code> / <code>Deque</code>	Lock-free (Michael-Scott queue)	Non-blocking producer/consumer
<code>ArrayBlockingQueue</code>	Bounded array, single lock + 2 conditions	Bounded producer/consumer (back-pressure)
<code>LinkedBlockingQueue</code>	Optionally bounded, two locks (put/take)	Thread pool work queues (<code>newFixedThreadPool</code> uses an unbounded one, a memory risk)
<code>PriorityBlockingQueue</code>	Unbounded heap	Priority task processing
<code>DelayQueue</code>	Elements available after a delay	Scheduling, retries, expiring cache entries
<code>SynchronousQueue</code>	Zero capacity, direct hand-off	<code>newCachedThreadPool</code>
<code>LinkedTransferQueue</code>	<code>transfer()</code> waits for a consumer	High-throughput hand-off

Producer-consumer with BlockingQueue (sensor ingestion)

```
import java.util.concurrent.*;

public class SensorPipeline {
    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<String> queue = new ArrayBlockingQueue<>(100); // bounded = back-pressure

        Thread producer = new Thread(() -> {
            try {
                for (int i = 0; i < 10; i++) queue.put("reading-" + i); // blocks if full
                queue.put("EOF");
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        Thread consumer = new Thread(() -> {
            try {
                String r;
                while (!(r = queue.take()).equals("EOF")) System.out.println("processed " + r); // blocks if empty
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        producer.start(); consumer.start();
        producer.join(); consumer.join();
    }
}
```

Other concurrent-access tools: `synchronized`, `ReentrantLock` (tryLock, fairness, interruptible), `ReadWriteLock` / `StampedLock` (optimistic reads), `Atomic*` and `LongAdder`, `volatile`, `CountDownLatch`, `CyclicBarrier`, `Semaphore` (rate-limiting connections), `Phaser`, `ThreadLocal`, and immutability (the simplest form of thread safety).

Gotchas

- **Check-then-act** and **read-modify-write** sequences are races even on thread-safe collections.
- `CopyOnWriteArrayList` is terrible for write-heavy workloads (O(n) copy per write).
- `ConcurrentLinkedQueue.size()` is O(n) and approximate.
- In a microservices deployment, in-JVM concurrency doesn't protect against multiple **instances**. Use DB locks (`SELECT ... FOR UPDATE`, optimistic `@Version`), Redis locks (Redisson), or `ShedLock` for schedulers.

Likely follow-ups:

- *Fail-fast vs fail-safe iterators?* Fail-fast throws CME (`ArrayList`, `HashMap`). Fail-safe/weakly consistent iterators work on a snapshot or tolerate changes (`CopyOnWriteArrayList`, CHM).
- *put/take vs offer/poll on a BlockingQueue?* `put` / `take` block. `offer` / `poll` return immediately (false/null) or wait with a timeout.
- *How do you make a thread-safe Set?* `ConcurrentHashMap.newKeySet()`, `CopyOnWriteArraySet` (read-mostly), or `ConcurrentSkipListSet` (sorted).

A33. wait() vs sleep().

Aspect	<code>wait()</code>	<code>Thread.sleep()</code>
Defined in	<code>java.lang.Object</code> (instance method)	<code>java.lang.Thread</code> (static method)

Aspect	<code>wait()</code>	<code>Thread.sleep()</code>
Purpose	Inter-thread communication : wait for a condition signalled by another thread	Pause the current thread for a duration
Lock (monitor)	Releases the monitor of the object it's called on	Keeps all locks it holds
Must be in <code>synchronized</code>	Yes , on the same object, else <code>IllegalMonitorStateException</code>	No
Wakes up when	<code>notify()</code> / <code>notifyAll()</code> , a timeout (<code>wait(ms)</code>), interrupt, or spurious wakeup	The time elapses or it's interrupted
After waking	Must re-acquire the monitor before continuing	Continues immediately (and still holds its locks)
Thread state	<code>WAITING</code> / <code>TIMED_WAITING</code>	<code>TIMED_WAITING</code>
Exception	<code>InterruptedException</code>	<code>InterruptedException</code>

Correct wait/notify pattern: always loop on the condition

```
public class SignalBoard {
    private String message; // shared state guarded by 'this'

    public synchronized void publish(String msg) {
        message = msg;
        notifyAll(); // wake waiting threads (prefer notifyAll)
    }

    public synchronized String awaitMessage() throws InterruptedException {
        while (message == null) { // while, NOT if: guards against spurious wakeups
            wait(); // releases 'this' monitor while waiting
        }
        String m = message;
        message = null;
        return m;
    }

    public static void main(String[] args) throws Exception {
        SignalBoard board = new SignalBoard();
        Thread reader = new Thread(() -> {
            try { System.out.println("Got: " + board.awaitMessage()); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        reader.start();
        Thread.sleep(500); // sleep keeps no lock here; just pauses main
        board.publish("Signal J12 green");
        reader.join();
    }
}
```

Gotchas

- `sleep()` inside a `synchronized` block holds the lock the whole time. Other threads are blocked. That's a common performance bug.
- `notify()` wakes one arbitrary waiting thread and can cause missed signals or deadlock-like stalls if several conditions share one monitor. Prefer `notifyAll()`, or `Condition` objects from `ReentrantLock` (`await` / `signal`).
- On `InterruptedException`, restore the flag with `Thread.currentThread().interrupt()` rather than swallowing it.
- In modern code, prefer higher-level tools: `BlockingQueue`, `CountDownLatch`, `CompletableFuture`. `TimeUnit.SECONDS.sleep(1)` is more readable than `Thread.sleep(1000)`.

Likely follow-ups:

- Why is `wait()` in `Object` and not `Thread`?* Every object can be a monitor or lock, and a thread waits on a specific object's condition.
- What is a `spurious wakeup`?* A thread returns from `wait()` without a `notify` (allowed by the spec and the OS). That's why you check the condition in a `while` loop.
- `sleep(0)` / `yield`?* Both are scheduler hints. `yield()` suggests giving up the CPU, but there are no guarantees.

A34. The Callable interface (Future, ExecutorService, CompletableFuture).

`java.util.concurrent.Callable<V>` (Java 5) is a task that **returns a result** and **can throw checked exceptions**, unlike `Runnable`.

`@FunctionalInterface`

```
public interface Callable<V> { V call() throws Exception; }
```

	Runnable	Callable<V>
Method	void run()	V call() throws Exception
Returns value	No	Yes
Checked exceptions	Can't throw	Can throw
Usable with	Thread, execute(), submit()	submit(), invokeAll(), invokeAny() (not new Thread(...) directly; wrap it in a FutureTask)

With ExecutorService + Future

```
import java.util.*;
import java.util.concurrent.*;

public class CallableDemo {
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        Callable<Integer> trafficCount = () -> { Thread.sleep(300); return 1240; };
        Callable<Integer> aqi = () -> 182;

        Future<Integer> f = pool.submit(trafficCount); // returns immediately
        try {
            Integer v = f.get(2, TimeUnit.SECONDS); // blocks up to 2s
            System.out.println("traffic = " + v);
        } catch (ExecutionException e) { // exception thrown inside call()
            System.out.println("task failed: " + e.getCause());
        } catch (TimeoutException e) {
            f.cancel(true); // interrupt the running task
        }

        List<Future<Integer>> all = pool.invokeAll(List.of(trafficCount, aqi)); // wait for all
        for (Future<Integer> r : all) {
            try { System.out.println(r.get()); } catch (ExecutionException e) { e.printStackTrace(); }
        }
        pool.shutdown();
    }
}
```

Future API: get(), get(timeout, unit), isDone(), cancel(mayInterrupt), isCancelled(), plus resultNow() / exceptionNow() / state() in Java 19+. Limitations: get() **blocks**, you can't chain or combine results, you can't complete it manually, and there are no callbacks.

CompletableFuture (Java 8): non-blocking composition

```
import java.util.concurrent.*;

public class CfDemo {
    public static void main(String[] args) {
        ExecutorService io = Executors.newFixedThreadPool(4); // don't block the common pool with I/O

        CompletableFuture<Integer> traffic = CompletableFuture.supplyAsync(() -> 1240, io);
        CompletableFuture<Integer> aqi = CompletableFuture.supplyAsync(() -> 182, io);

        CompletableFuture<String> dashboard = traffic
            .thenCombine(aqi, (t, a) -> "Traffic=" + t + ", AQI=" + a) // combine two independent tasks
            .thenApply(String::toUpperCase) // transform
            .orTimeout(2, TimeUnit.SECONDS) // Java 9
            .exceptionally(ex -> "DATA UNAVAILABLE: " + ex.getMessage()); // fallback

        System.out.println(dashboard.join()); // join(): unchecked CompletionException instead of checked
        io.shutdown();
    }
}
```

Key methods: supplyAsync / runAsync, thenApply (map), thenCompose (flatMap, chains dependent async calls), thenCombine, allOf / anyOf, thenAccept, handle / whenComplete, exceptionally, completeOnTimeout. Without an executor argument, the async methods use ForkJoinPool.commonPool().

In Spring: @Async methods returning CompletableFuture<T>, which is useful for fanning out calls to several microservices in parallel.

Likely follow-ups:

- get() vs join()? get throws the checked ExecutionException / InterruptedException. join throws the unchecked CompletionException.

- *thenApply vs thenCompose?* `thenApply(T→U)` maps the result. `thenCompose(T→CompletableFuture<U>)` flattens nested futures.
- *How do you run a Callable on a raw Thread?* `new Thread(new FutureTask<>(callable)).start()`. `FutureTask` is both a `Runnable` and a `Future`.

A35. `execute()` vs `submit()`.

Aspect	<code>Executor.execute(Runnable)</code>	<code>ExecutorService.submit(...)</code>
Declared in	<code>Executor</code>	<code>ExecutorService</code>
Accepts	<code>Runnable</code> only	<code>Runnable</code> , <code>Runnable + result</code> , <code>Callable<T></code>
Returns	<code>void</code>	<code>Future<?></code> / <code>Future<T></code>
Exception in task	Propagates out of <code>run()</code> . The worker thread dies, the exception goes to the thread's <code>UncaughtExceptionHandler</code> (the default prints the stack trace to stderr), and the pool replaces the worker	Captured in the <code>Future</code> (<code>FutureTask</code> catches it). Nothing is printed. It only surfaces as <code>ExecutionException</code> when you call <code>future.get()</code>
Cancellation / result	Not possible	<code>future.cancel()</code> , <code>future.get()</code>
<code>afterExecute(r, t)</code> hook	<code>t</code> = the exception	<code>t</code> = null. You must inspect the <code>Future</code> yourself

```
import java.util.concurrent.*;

public class ExecVsSubmit {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newSingleThreadExecutor();

        pool.execute(() -> { throw new IllegalStateException("boom-execute"); });
        // -> stack trace printed: Exception in thread "pool-1-thread-1" java.lang.IllegalStateException: boom-execute

        Future<?> f = pool.submit(() -> { throw new IllegalStateException("boom-submit"); });
        // -> SILENT. Nothing printed.

        try { f.get(); }
        catch (ExecutionException e) { System.out.println("caught: " + e.getCause()); } // caught: java.lang.IllegalStateException:
boom-submit

        pool.shutdown();
    }
}
```

The production gotcha: "fire-and-forget" `submit()` calls whose `Future` is never checked **silently swallow exceptions**. Background jobs then fail with zero logs. Fixes:

- Use `execute()` for fire-and-forget tasks, with a configured `UncaughtExceptionHandler` via a custom `ThreadFactory`.
- Wrap the task body in try/catch and log inside it.
- Override `ThreadPoolExecutor.afterExecute` and unwrap the `Future`.
- Use `CompletableFuture.runAsync(...).exceptionally(...)` / `whenComplete(...)`.
- In Spring, configure an `AsyncUncaughtExceptionHandler` for `@Async` void methods.

`ScheduledExecutorService.scheduleAtFixedRate`: if one run throws, **all future runs are silently suppressed**. Always catch exceptions inside scheduled tasks.

Likely follow-ups:

- What does `submit(Runnable)` return from `get()`? `null` on success, or the given result for `submit(runnable, result)`.
- `shutdown()` vs `shutdownNow()`? `shutdown` stops accepting tasks and lets queued ones finish. `shutdownNow` interrupts running tasks and returns the queued ones.
- Why not `Executors.newFixedThreadPool` in production? Its unbounded `LinkedBlockingQueue` risks OOM. Build a `ThreadPoolExecutor` with a bounded queue and a rejection policy.

A36. How do you create a thread? Extending Thread vs implementing Runnable.

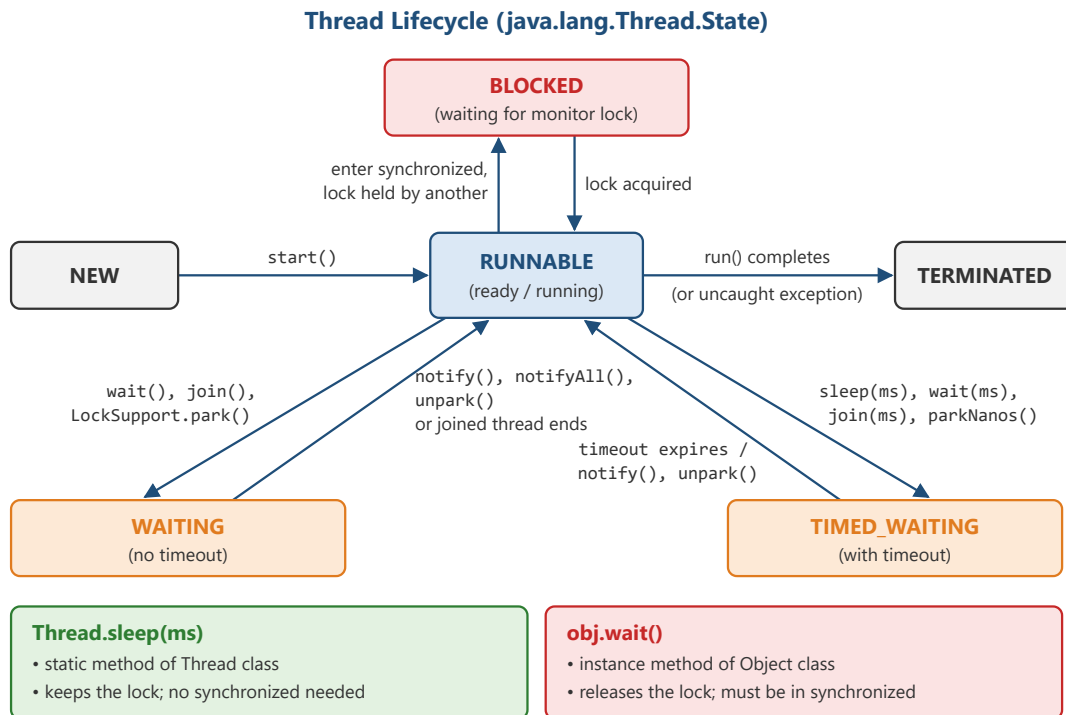


Figure 6: Java thread lifecycle states

Ways to create and run concurrent work

```
import java.util.concurrent.*;

public class ThreadWays {
    // 1. Extend Thread
    static class Worker extends Thread {
        @Override public void run() { System.out.println("1. " + getName()); }
    }

    public static void main(String[] args) throws Exception {
        new Worker().start();

        // 2. Implement Runnable (or lambda) and pass it to Thread
        Runnable task = () -> System.out.println("2. " + Thread.currentThread().getName());
        new Thread(task, "sensor-poller").start();

        // 3. Callable + FutureTask (result-bearing)
        FutureTask<Integer> ft = new FutureTask<>(() -> 42);
        new Thread(ft).start();
        System.out.println("3. " + ft.get());

        // 4. ExecutorService (preferred in real apps: pooling, lifecycle, futures)
        ExecutorService pool = Executors.newFixedThreadPool(2);
        pool.submit(() -> System.out.println("4. pooled"));
        pool.shutdown();

        // 5. CompletableFuture
        CompletableFuture.runAsync(() -> System.out.println("5. async")).join();
    }
}
```

Extending Thread vs implementing Runnable

	extends Thread	implements Runnable
Inheritance	Uses up your only superclass	Class can still extend another
Design	Mixes the <i>task</i> with the <i>execution mechanism</i>	Separates the task (what) from the thread (how). Follows SRP
Reuse	One Thread object = one run	The same Runnable can be shared by many threads or submitted to executors

	extends Thread	implements Runnable
Executor compatibility	Awkward	Works directly with <code>ExecutorService</code> , <code>CompletableFuture</code>
Lambdas	No	Yes (<code>Runnable</code> is functional)
When to use	Rarely. Only when customising Thread behaviour itself	Almost always

start() vs run(): `start()` creates a new OS/JVM thread, which then calls `run()`. Calling `run()` directly executes it **on the current thread**, like a normal method. Calling `start()` twice → `IllegalThreadStateException`.

Virtual threads (Java 21, JEP 444)

Lightweight threads managed by the JVM and mounted on a small pool of carrier (platform) threads. When a virtual thread blocks on I/O, it **unmounts**, so millions of concurrent blocking tasks become cheap. The thread-per-request style scales again without reactive code.

```
// Java 21+
import java.util.concurrent.*;
import java.util.stream.IntStream;

public class VirtualThreads {
    public static void main(String[] args) {
        Thread.ofVirtual().name("vt-1").start(() -> System.out.println("hello from " + Thread.currentThread()));
        Thread.startVirtualThread(() -> System.out.println("another"));

        try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) { // AutoCloseable since 19
            IntStream.range(0, 10_000).forEach(i -> ex.submit(() -> {
                Thread.sleep(100); // blocking is fine: VT unmounts
                return i;
            }));
        } // close() waits for all tasks
    }
}
```

- Spring Boot 3.2+: `spring.threads.virtual.enabled=true`.
- Don't **pool** virtual threads. Create one per task.
- Use a `Semaphore` to limit access to scarce resources (such as DB connections).
- **Pinning**: in Java 21, blocking inside `synchronized` pins the carrier thread. Prefer `ReentrantLock` there. JEP 491 (Java 24) removed most `synchronized` pinning.
- Virtual threads help **I/O-bound** work, not CPU-bound work.
- Be careful with `ThreadLocal` -heavy code (memory per thread). Scoped values are the future replacement.

Thread lifecycle: NEW → RUNNABLE → (BLOCKED | WAITING | TIMED_WAITING) → TERMINATED.

Likely follow-ups:

- *Daemon vs user thread?* The JVM exits when only daemon threads remain (`setDaemon(true)` before `start()`). GC threads are daemons.
- *How do you stop a thread?* Cooperatively: `interrupt()` plus checking `isInterrupted()`, or a volatile flag. `Thread.stop()` is deprecated, unsafe, and throws `UnsupportedOperationException` since Java 20.
- *Platform vs virtual threads?* A platform thread is a 1:1 OS thread (~1 MB stack, thousands max). A virtual thread is M:N, a few hundred bytes to KBs, and you can have millions.

A37. How did you connect Java with Python?

(A sample answer for a Smart City platform. Adapt it to your real experience.)

Main answer: Python as a separate microservice over REST (FastAPI)

"In our Smart City platform, the core services (incident management, traffic, dashboards) are Spring Boot. The data science team built Python models, for example traffic-congestion prediction and ANPR/video-analytics post-processing, using pandas, scikit-learn and PyTorch. Rather than embedding Python into the JVM, we deployed the models as a **FastAPI microservice** in its own Docker container, and the Java services call it over HTTP/JSON."

```
# predict_service.py (Python side)
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class TrafficInput(BaseModel):
    junction_id: str
```

```

hour: int
vehicle_count: int

@app.post("/predict/congestion")
def predict(inp: TrafficInput):
    score = min(1.0, inp.vehicle_count / 2000) # model.predict(...) in reality
    return {"junction_id": inp.junction_id, "congestion_score": score}
# run: uvicorn predict_service:app --host 0.0.0.0 --port 8000

```

```

// Java side (Spring Boot 3.2+, RestClient)
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

record TrafficInput(String junction_id, int hour, int vehicle_count) {}
record Prediction(String junction_id, double congestion_score) {}

@Service
class CongestionClient {
    private final RestClient client;
    CongestionClient(RestClient.Builder builder) {
        this.client = builder.baseUrl("http://ml-service:8000").build(); // K8s/compose service name
    }
    // annotate with Resilience4j @CircuitBreaker(name = "ml", fallbackMethod = "fallback") + @Retry
    Prediction predict(TrafficInput in) {
        return client.post().uri("/predict/congestion").body(in).retrieve().body(Prediction.class);
    }
    Prediction fallback(TrafficInput in, Throwable t) { return new Prediction(in.junction_id(), -1); }
}

```

What I'd emphasise:

- **Timeouts:** connect and read timeouts on the HTTP client.
- **Resilience4j:** circuit breaker, retry with backoff, bulkhead.
- **Contract:** an OpenAPI spec (FastAPI generates `/docs` automatically).
- **Auth:** a JWT/service token via the API gateway.
- **Observability:** correlation IDs, Micrometer/Prometheus metrics on latency.
- **Scaling:** the Python service scales independently (GPU nodes), with its own release cycle.

Alternatives and trade-offs

Approach	How	Pros	Cons	Use when
REST (FastAPI/Flask)	HTTP + JSON	Simple, language-agnostic, independent scaling and deployment, easy to debug	Network latency (ms), JSON overhead	Default for synchronous request/response
Kafka / RabbitMQ	Java publishes an event (<code>camera-frames</code> , <code>sensor-readings</code>). Python consumes and publishes results to another topic	Async, decoupled, buffers bursts, replayable	Eventual consistency, more infrastructure	Streaming sensor/video analytics, long-running jobs
gRPC	Protobuf contract, HTTP/2	Fast binary protocol, strong typing, streaming, code generation for both languages	Harder to debug, needs proto tooling	High-throughput internal calls
ProcessBuilder	Spawn <code>python script.py</code> , pass args/stdin, read stdout	No extra service, quick	Process startup per call (slow), fragile parsing, error handling, security (command injection), hard to scale	Batch scripts, one-off report generation
GraalPy (GraalVM Polyglot)	Run Python inside the JVM via <code>Context.create("python")</code>	In-process, no network	Limited native-extension support (NumPy etc. improving), GraalVM dependency	Lightweight embedded scripting
Jython	Python on the JVM	—	Python 2.7 only , effectively dead	Avoid
Py4J / JEP	Bridge to CPython	Access to native libs	Complex, coupled lifecycle	Niche
Shared DB / files	One writes, the other reads	Simple	Tight coupling via schema, polling	Batch ETL

ProcessBuilder example (if asked)

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.concurrent.TimeUnit;

public class PythonRunner {
    public static String run(String junction) throws IOException, InterruptedException {
        ProcessBuilder pb = new ProcessBuilder("python3", "predict.py", junction); // args list: no shell injection
        pb.redirectErrorStream(true);
        Process p = pb.start();
        String out;
        try (var r = new BufferedReader(new InputStreamReader(p.getInputStream(), StandardCharsets.UTF_8))) {
            out = r.lines().reduce("", (a, b) -> a + b);
        }
        if (!p.waitFor(30, TimeUnit.SECONDS)) { p.destroyForcibly(); throw new IllegalStateException("timeout"); }
        if (p.exitValue() != 0) throw new IllegalStateException("python failed: " + out);
        return out; // e.g. JSON parsed with Jackson
    }
}
```

Likely follow-ups:

- *How do you handle the Python service being slow or down?* Timeouts, a circuit breaker with fallback (cached/last-known value), retries with exponential backoff, and async via Kafka for non-critical paths.
- *Why not ProcessBuilder in production?* Per-call process overhead, no pooling, stdout parsing fragility, and difficulty scaling and monitoring.
- *How do you keep the contract in sync?* Use the OpenAPI spec from FastAPI to generate Java client DTOs (openapi-generator), or protobuf for gRPC.
- *Sync vs async integration?* Sync REST when the user waits for the answer. Kafka when the result can arrive later (e.g. analytics on camera feeds).

A38. SOLID principles with a small Java example for each.

S – Single Responsibility Principle: a class should have **one reason to change**.

```
// BAD: one class validates, persists, and emails
// GOOD: split responsibilities
record Complaint(String id, String text, String email) {}
class ComplaintValidator { void validate(Complaint c) { if (c.text().isBlank()) throw new IllegalArgumentException(); } }
class ComplaintRepository { void save(Complaint c) { /* JDBC/JPA */ } }
class ComplaintNotifier { void acknowledge(Complaint c) { /* send email */ } }
class ComplaintService {
    private final ComplaintValidator v = new ComplaintValidator();
    private final ComplaintRepository r = new ComplaintRepository();
    private final ComplaintNotifier n = new ComplaintNotifier();
    void register(Complaint c) { v.validate(c); r.save(c); n.acknowledge(c); } // orchestrates only
}
```

O – Open/Closed Principle: open for **extension**, closed for **modification**.

```
// BAD: if (type.equals("BUS")) ... else if ("METRO") ... edited for every new type
interface FarePolicy { double fare(double km); }
class BusFare implements FarePolicy { public double fare(double km) { return 10 + km * 1.5; } }
class MetroFare implements FarePolicy { public double fare(double km) { return 20 + km * 2.0; } }
// New mode (e.g. e-bike)? Add a class; FareCalculator is untouched.
class FareCalculator { double calc(FarePolicy p, double km) { return p.fare(km); } }
```

L – Liskov Substitution Principle: subtypes must be usable anywhere the base type is, **without breaking expectations**.

```
// BAD: class Square extends Rectangle - setWidth also changes height, breaking Rectangle's contract.
// BAD: class ReadOnlyList extends ArrayList { add() throws UnsupportedOperationException }
// GOOD: model capabilities separately
interface Shape { double area(); }
record Rect(double w, double h) implements Shape { public double area() { return w * h; } }
record Sq(double side) implements Shape { public double area() { return side * side; } }
```

Signs of a violation: overrides that throw `UnsupportedOperationException`, strengthen preconditions, or weaken postconditions, and `instanceof` checks in client code.

I – Interface Segregation Principle: clients shouldn't depend on methods they don't use. Prefer **small, focused interfaces**.

```
// BAD: interface Device { void readData(); void rotateCamera(); void changeSignal(); }
interface Readable { String readData(); }
interface Rotatable { void rotate(int degrees); }
class AirSensor implements Readable { public String readData() { return "AQI=182"; } }
```

```
class PtzCamera implements Readable, Rotatable {
    public String readData() { return "frame"; }
    public void rotate(int d) { System.out.println("rotate " + d); }
}
```

D – Dependency Inversion Principle: high-level modules depend on **abstractions**, not concrete classes. Both depend on interfaces. (Spring DI is the classic enabler.)

```
interface MessageSender { void send(String to, String body); }
class SmsSender implements MessageSender { public void send(String to, String b) { /* SMS gateway */ } }
class EmailSender implements MessageSender { public void send(String to, String b) { /* SMTP */ } }

class AlertService {
    // high-level policy
    private final MessageSender sender; // depends on abstraction
    AlertService(MessageSender sender) { this.sender = sender; } // injected (Spring @Service + constructor)
    void alert(String citizen) { sender.send(citizen, "Waterlogging reported near you"); }
}
// new AlertService(new SmsSender()); tests: new AlertService(mockSender)
```

Principle	One-liner	Smell when violated
SRP	One reason to change	God classes, 2000-line services
OCP	Extend, don't edit	Growing <code>if/else</code> / <code>switch</code> on types
LSP	Subtypes are substitutable	<code>UnsupportedOperationException</code> , <code>instanceof</code> checks
ISP	Small, specific interfaces	Empty or no-op method implementations
DIP	Depend on abstractions	<code>new ConcreteDao()</code> inside business logic; untestable code

Likely follow-ups:

- *DIP vs DI?* DIP is the design principle (depend on abstractions). DI is a technique (supply dependencies from outside). IoC containers like Spring automate DI.
- *Which design patterns support OCP?* Strategy, Template Method, Decorator, Observer.
- *Can you over-apply SOLID?* Yes. Too many tiny interfaces and indirection (YAGNI) hurt readability. Apply it where change is likely.

A39. Monolithic vs microservice architecture.

Monolith: the whole application (UI, business logic, data access for every module) is built, deployed and scaled as **one unit**, usually backed by one shared database. Example: a single Spring Boot WAR/JAR with `incident`, `traffic`, `billing` and `user` packages.

Microservices: the application is a set of **small, independently deployable services**, each owning one **business capability** (bounded context, DDD) and its **own data store**. They communicate over the network (REST/gRPC synchronously, Kafka/RabbitMQ asynchronously). Example: `incident-service`, `traffic-service`, `notification-service`, `identity-service`, behind an **API Gateway** (Spring Cloud Gateway), with service discovery (Eureka/K8s DNS), a config server, and centralized logging and tracing.

Aspect	Monolith	Microservices
Deployment	One artifact. Any change redeploys everything	Each service deployed independently
Scaling	Scale the whole app (vertical, or clone everything)	Scale hot services only (e.g. traffic ingestion ×10)
Database	Usually one shared DB, ACID transactions are easy	DB per service. Distributed transactions → Saga , outbox pattern, eventual consistency
Technology	One stack	Polyglot possible (Java + Python ML service)
Fault isolation	One memory leak can take down the whole app	Failures contained (with circuit breakers, bulkheads)
Development speed	Fast at first, slows as the codebase grows	Slower start, scales across many teams (Conway's law)
Testing	Simple end-to-end	Contract tests, integration environments, harder E2E
Communication	In-process method calls (ns)	Network calls (ms), with latency, partial failure, retries
Operations	Simple: one app, one log	Complex: K8s, CI/CD per service, distributed tracing (Zipkin/OTel), centralized logs (ELK), monitoring

Aspect	Monolith	Microservices
Consistency	Strong	Eventual (mostly)
Debugging	Stack traces in one process	Correlation IDs across services
Team ownership	Shared codebase, merge conflicts	Team per service, clear ownership

When to choose which

Choose a **monolith** (often a *modular monolith*) when:

- The team is small (fewer than ~10 developers), the domain is new or unclear, or you're building an MVP or startup product.
- Strong transactional consistency matters, or the ops/DevOps maturity is low.
- You don't have heavy differential-scaling needs.

Choose **microservices** when:

- Several teams need to deploy independently and the domain boundaries are well understood.
- Modules have very different scaling or availability requirements. In a Smart City platform, for example: IoT sensor ingestion vs reporting vs citizen portal.
- You need polyglot tech, and you have mature CI/CD, containers/Kubernetes and observability.

Migration path: Strangler Fig pattern. Put a gateway in front of the monolith and carve out one bounded context at a time (e.g. extract `notification-service` first, because it's loosely coupled).

Anti-pattern to mention: the **distributed monolith**. These are services that share a database or must be deployed together. You get all the costs of microservices with none of the benefits.

Likely follow-ups:

- *How do you handle transactions across services?* The Saga pattern (choreography via events, or orchestration), with compensating actions plus a transactional outbox for reliable publishing.
- *How do services find each other?* Service discovery: Eureka, Consul, or Kubernetes Services/DNS, with client- or server-side load balancing.
- *How do you prevent cascading failures?* Timeouts, circuit breakers (Resilience4j), bulkheads, retries with backoff, and fallbacks.
- *What is a modular monolith?* One deployable unit with strictly separated internal modules (e.g. Spring Modulith). It's the easiest to split later.

A40. Which declarations are correct and why?

```
interface I { }
class A implements I { }
class B extends A { }
class C extends B { }
```

Hierarchy: $I \leftarrow A \leftarrow B \leftarrow C$. So C is-a B, is-an A, is-an I. The reverse is not true.

```
A a = new I(); // (1)
I b = new B(); // (2)
C c = new B(); // (3)
C c2 = new A(); // (4)
B b = new C(); // (5)
```

#	Statement	Verdict	Reason
1	<code>A a = new I();</code>	Compile error	" <i>I is abstract; cannot be instantiated</i> ". Interfaces can't be instantiated. (Even <code>new I(){}</code> as an anonymous class would give type <code>I</code> , which can't be assigned to <code>A</code> without a cast. That cast would compile but throw <code>ClassCastException</code> , since the anonymous class isn't an <code>A</code> .)
2	<code>I b = new B();</code>	Valid	Upcasting: B extends A, which implements I, so B is-an I. Implicit widening reference conversion. Only <code>I</code> 's members are accessible via <code>b</code> .
3	<code>C c = new B();</code>	Compile error	" <i>incompatible types: B cannot be converted to C</i> ". That's downcasting without a cast: not every B is a C. With an explicit cast, <code>C c = (C) new B();</code> compiles (B and C are related) but throws <code>ClassCastException</code> at runtime, because the actual object is a B, not a C.
4	<code>C c2 = new A();</code>	Compile error	" <i>incompatible types: A cannot be converted to C</i> ". Same reason: an A is not a C. <code>(C) new A()</code> → compiles, then <code>ClassCastException</code> .

#	Statement	Verdict	Reason
5	<code>B b = new C();</code>	Type-wise valid (upcast C → B), but a compile error here	"variable <code>b</code> is already defined in method <code>main(String[])</code> ". Statement (2) already declared <code>b</code> (as <code>I</code>) in the same scope. Java doesn't allow redeclaring a local in the same scope, even with a different type. Renamed, e.g. <code>B b3 = new C();</code> , it's perfectly fine.

```

public class CastDemo {
    interface I { }
    static class A implements I { }
    static class B extends A { }
    static class C extends B { }

    public static void main(String[] args) {
        I b = new B();           // OK
        B b3 = new C();           // OK - upcast
        I anon = new I() { };     // OK - anonymous class implementing I (not "new I()")
        try {
            C c = (C) new B();     // compiles, fails at runtime
        } catch (ClassCastException e) {
            System.out.println(e.getMessage()); // class CastDemo$B cannot be cast to class CastDemo$I
        }
        if (b instanceof C c) { /* safe downcast via pattern matching (16+) */ } else {
            System.out.println("b is not a C");
        }
    }
}

```

Rules to state in the interview

- **Upcasting** (subtype → supertype) is implicit and always safe.
- **Downcasting** (supertype → subtype) needs an explicit cast. It compiles only when the types are related (could possibly succeed), and it's checked at runtime (`checkcast` bytecode) → `ClassCastException` if wrong.
- Casting between **unrelated classes** (e.g. `(String) new A()`) is a **compile error** ("incompatible types"). Casting to an **interface** from a non-final class compiles, because a subclass might implement it.
- Use `instanceof` (with pattern matching) before downcasting.

Likely follow-ups:

- What can you call on `I b = new B()`? Only members declared in `I` (and `Object`), even though the object is a `B`. Overridden methods still dispatch to `B`'s implementation.
- Why does `(C) new B()` compile at all? `B` and `C` are in the same hierarchy, so a `B` reference *could* point to a `C`. The compiler defers the check to runtime.
- Would `I i = (I) new String("x")` compile? No. `String` is `final` and doesn't implement `I`, so the compiler knows the cast can never succeed.

A41. Which array declarations are valid and why?

```

int[]    n1 = new int[0];
boolean[] n2 = new boolean[-200];
double[] n3 = new double[2241423798];
char[]   ch = new char[20];

```

Declaration	Compiles?	Runtime	Explanation
<code>new int[0]</code>	Yes	OK	A zero-length array is legal: <code>n1.length == 0</code> . Useful as an empty return value instead of <code>null</code> (Effective Java), and as <code>toArray(new String[0])</code> . Accessing <code>n1[0]</code> → <code>ArrayIndexOutOfBoundsException</code> .
<code>new boolean[-200]</code>	Yes	NegativeArraySizeException	The size expression only has to be of type <code>int</code> . The compiler does not check the sign, even for constants. The JVM checks at runtime in the <code>newarray</code> instruction.
<code>new double[2241423798]</code>	No	—	Compile error: <i>"integer number too large: 2241423798"</i> . The literal exceeds <code>Integer.MAX_VALUE</code> (2,147,483,647) and has no <code>L</code> suffix, so it isn't even a valid <code>int</code> literal. With <code>2241423798L</code> you'd get <i>"incompatible types: possible lossy conversion from long to int"</i> , because array sizes must be int (after unary promotion from <code>byte</code> / <code>short</code> / <code>char</code> ; <code>Long</code> is not allowed).
<code>new char[20]</code>	Yes	OK	20 chars, each defaulting to <code>'\u0000'</code> (the NUL character, numeric 0), not a space. Printing shows invisible characters.

Default values of array elements

Type	Default
byte, short, int, long	0 / 0L
float, double	0.0f / 0.0
char	'\u0000'
boolean	false
Any reference (String, Integer, objects)	null

Maximum array size

- Theoretically `Integer.MAX_VALUE` elements, because the index is an int.
- In practice, HotSpot reserves some header words, so the safe maximum is about `Integer.MAX_VALUE - 8` (the JDK's `ArrayList / ArraysSupport.SOFT_MAX_ARRAY_LENGTH` uses that limit).
- Requesting more → `OutOfMemoryError: Requested array size exceeds VM limit`. Requesting a big but legal size without enough heap → `OutOfMemoryError: Java heap space`. A `double[Integer.MAX_VALUE - 8]` needs ~16 GB.
- For more than 2^{31} elements, use multiple arrays, `long`-indexed off-heap memory (the FFM API `MemorySegment`, final in 22), or memory-mapped files.

```
import java.util.Arrays;

public class ArrayDemo {
    public static void main(String[] args) {
        int[] n1 = new int[0];
        System.out.println(n1.length);           // 0

        try { boolean[] n2 = new boolean[-200]; }
        catch (NegativeArraySizeException e) { System.out.println("NASE: " + e.getMessage()); } // NASE: -200

        // double[] n3 = new double[2241423798]; // compile error: integer number too large
        // double[] n4 = new double[2241423798L]; // compile error: possible lossy conversion from long to int

        char[] ch = new char[20];
        System.out.println((int) ch[0]);          // 0
        System.out.println(ch[0] == '\u0000');    // true
        System.out.println(Arrays.toString(new String[2])); // [null, null]

        int size = 'A';                          // char promoted to int -> legal size 65
        long[] fromChar = new long[size];
        System.out.println(fromChar.length);      // 65
    }
}
```

Other array declaration gotchas

- `int[] a = new int[]{1,2,3};` and `int[] a = {1,2,3};` are both valid. `int[] a = new int[3]{1,2,3};` is a compile error (you can't give both a size and an initializer).
- `int a[], b;` declares `a` as an array and `b` as a plain int. `int[] a, b;` declares both as arrays.
- Arrays are **covariant**: `Object[] o = new String[1]; o[0] = 1;` compiles but throws `ArrayStoreException`. Generics are invariant precisely to avoid this.
- `new int[3][]` (jagged) is valid. `new int[][3]` is a compile error.

Likely follow-ups:

- *Why does the negative size compile?* The JLS only requires the dimension expression to be of type int. The negative check is a runtime check in the JVM.
- *Can an array size be a `long` or a `char`?* `char / byte / short` are promoted to int, so they're allowed. `long` is a compile error without an explicit `(int)` cast.
- *Is an array an object?* Yes. It lives on the heap, has a `length` final field, extends `Object`, and implements `Cloneable` and `Serializable`.
- *Why is the max array size `MAX_VALUE - 8`?* Some VMs reserve header words in the array object, and asking for more risks the "exceeds VM limit" OOM.

B. Java 8, Streams, Optional, Lambdas & Coding Problems

B1. Explain Java 8 features

Java 8 (March 2014) was the largest language change since generics. It brought **functional programming** into Java and modernised several core APIs.

Feature	What it gives you	Key types / syntax
Lambda expressions	Pass behaviour as data; concise implementation of single-method interfaces	<code>(a, b) -> a + b</code>
Functional interfaces	The target type for a lambda: exactly one abstract method	<code>@FunctionalInterface</code> , <code>java.util.function.*</code>
Method references	Even shorter lambdas that only call an existing method	<code>String::length</code> , <code>System.out::println</code>
Stream API	Declarative, lazy, pipeline processing of collections, optionally parallel	<code>list.stream().filter(..).map(..).collect(..)</code>
Optional	Explicit "may be absent" return type in place of <code>null</code>	<code>Optional.ofNullable(x).map(..).orElse(..)</code>
Default & static interface methods	Add methods to interfaces without breaking existing implementations (that is how <code>Collection.stream()</code> was added)	<code>default void log() {..}</code> , <code>static Comparator<T> naturalOrder()</code>
java.time (JSR-310)	Immutable, thread-safe date/time API that replaces <code>Date</code> / <code>Calendar</code> / <code>SimpleDateFormat</code>	<code>LocalDate</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code> , <code>Instant</code> , <code>Duration</code> , <code>Period</code> , <code>DateTimeFormatter</code>
CompletableFuture	Composable asynchronous programming (non-blocking pipelines)	<code>supplyAsync</code> , <code>thenApply</code> , <code>thenCompose</code> , <code>thenCombine</code> , <code>exceptionally</code> , <code>allOf</code>
Nashorn JS engine	Replaced Rhino for running JavaScript on the JVM (<code>js</code>). Deprecated in 11, removed in 15	<code>ScriptEngineManager().getEngineByName("nashorn")</code>
Metaspace replaces PermGen	Class metadata moved from a fixed-size heap region to native memory, which grows automatically. Ends <code>OutOfMemoryError: PermGen space</code>	<code>-XX:MaxMetaspaceSize</code> , <code>-XX:MetaspaceSize</code>
StringJoiner / String.join	Build delimited strings with prefix/suffix	<code>new StringJoiner(", ", "[", "]")</code>
Others	<code>Map.getOrDefault</code> / <code>putIfAbsent</code> / <code>computeIfAbsent</code> / <code>merge</code> / <code>forEach</code> , <code>Iterable.forEach</code> , <code>Collection.removeIf</code> , <code>Arrays.parallelSort</code> , <code>Base64</code> , repeating & type annotations, <code>ConcurrentHashMap</code> rewrite (no segments, CAS + bin-level <code>synchronized</code>), <code>LongAdder</code> , <code>StampedLock</code> , parameter names via <code>-parameters</code>	

Small demo touching several features:

```
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.concurrent.CompletableFuture;
import java.util.stream.Collectors;

public class Java8Tour {
    interface Greeter {
        String name();
        default String greet() { return "Hello, " + name(); } // default method
        static Greeter of(String n) { return () -> n; } // static method + lambda
    }

    public static void main(String[] args) throws Exception {
        // Lambdas + streams + method references
        List<String> langs = List.of("java", "kotlin", "scala", "go");
        String joined = langs.stream()
            .filter(s -> s.length() > 2)
            .map(String::toUpperCase)
            .collect(Collectors.joining(", ", "[", "]"));
        System.out.println(joined); // [JAVA, KOTLIN, SCALA]

        // Optional
        Optional<String> first = langs.stream().filter(s -> s.startsWith("k")).findFirst();
        System.out.println(first.orElse("none")); // kotlin

        // Default/static interface methods
        System.out.println(Greeter.of("Nagesh").greet()); // Hello, Nagesh
    }
}
```

```

// java.time
LocalDate d = LocalDate.of(2024, 2, 28).plusDays(1);
System.out.println(d + " " + d.format(DateTimeFormatter.ofPattern("dd-MMM-yyyy"))); // 2024-02-29 29-Feb-2024
System.out.println(Duration.ofMinutes(135)); // PT2H15M

// CompletableFuture
CompletableFuture<Integer> f = CompletableFuture.supplyAsync(() -> 20)
    .thenApply(x -> x * 2)
    .thenCombine(CompletableFuture.supplyAsync(() -> 2), Integer::sum);
System.out.println(f.get()); // 42

// StringJoiner
StringJoiner sj = new StringJoiner("|", "<", ">");
sj.add("a").add("b");
System.out.println(sj); // <a|b>
}

```

Gotchas worth mentioning:

- A default method conflict (class implements two interfaces with the same default) is a compile error; you must override and may call `A.super.method()`.
- Class methods always beat interface defaults ("class wins" rule).
- Metaspace is **unbounded by default**: a classloader leak (e.g. redeploying in Tomcat) now eats native memory instead of failing fast, so set `MaxMetaspaceSize` in production.
- `java.time` types are immutable: `date.plusDays(1)` returns a new object, so calling it without assigning the result does nothing.

Likely follow-ups:

- Why were default methods introduced?* So that interfaces like `Collection` could get `stream()` / `forEach()` without breaking every existing implementation.
- Difference between `LocalDateTime` and `ZonedDateTime` / `Instant`?* `LocalDateTime` has no zone; `ZonedDateTime` adds a zone with DST rules; `Instant` is a UTC point on the timeline (use it for storage and timestamps).
- What replaced Nashorn?* Nothing in the JDK. Use GraalJS (GraalVM polyglot) or the standalone Nashorn library.
- What happens to PermGen flags on Java 8?* `-XX:PermSize` / `MaxPermSize` are ignored with a warning.

B2. What is a lambda expression?

A lambda is an **anonymous function**: a block of code with parameters and a body, but no name and no class of its own. It can be used anywhere a **functional interface** is expected. The compiler infers which interface from the context (the *target type*).

Syntax forms:

```

Runnable r = () -> System.out.println("no args"); // no params
Consumer<String> c = s -> System.out.println(s); // one param, parens optional
BinaryOperator<Integer> add = (a, b) -> a + b; // multiple params, expression body
Comparator<String> cmp = (String a, String b) -> a.compareTo(b); // explicit types
BiFunction<Integer, Integer, Integer> f = (var a, var b) -> a * b; // Java 11: var (allows annotations)
Function<Integer, Integer> sq = x -> { // block body: needs return
    int y = x * x;
    return y;
};

```

Effectively-final capture: A lambda can read local variables from the enclosing scope only if they are `final` or **effectively final** (never reassigned after initialisation).

```

int base = 10;
Function<Integer, Integer> addBase = x -> x + base; // OK
// base = 20; // compile error: variable used in lambda must be final or effectively final

```

Why? Locals live on the stack frame, which may be gone by the time the lambda runs (for example on another thread). Java **copies the value** into the lambda, and the effectively-final rule keeps that copy correct and avoids data races. Instance fields and static fields **can** be modified, because the lambda captures `this` rather than a copy of the field.

Workaround (usually a smell): `int[] counter = {0}; list.forEach(x -> counter[0]++);`. Prefer `count()`, `AtomicInteger`, or a reduction.

Lambda vs anonymous inner class:

Aspect	Lambda	Anonymous class
Target	Functional interface only	Any interface or abstract/concrete class, any number of methods
<code>this</code>	Refers to the enclosing instance	Refers to the anonymous class instance
Scope	Same scope as enclosing block: cannot redeclare a local name	New scope: can shadow enclosing variables
State	No fields of its own	Can have fields and initialisers
Compilation	No extra <code>.class</code> file; uses <code>invokedynamic</code>	Generates <code>Outer\$1.class</code>
Instance creation	Non-capturing lambdas are typically a cached singleton	New object on every evaluation

```

public class ThisDemo {
    String name = "outer";
    void run() {
        Runnable lambda = () -> System.out.println(this.name);    // prints "outer"
        Runnable anon = new Runnable() {
            String name = "inner";
            public void run() { System.out.println(this.name); }    // prints "inner"
        };
        lambda.run(); anon.run();
    }
    public static void main(String[] a) { new ThisDemo().run(); }
}

```

How lambdas work internally (invokedynamic):

1. `javac` moves the lambda body into a private synthetic method, e.g. `private static lambda$main$0(String)`. If it uses `this`, the method is an instance method.
2. At the creation site, `javac` emits an `invokedynamic` instruction whose bootstrap method is `LambdaMetafactory.metafactory`.
3. The first time the instruction runs, the bootstrap method creates (with a hidden class) an implementation of the functional interface that delegates to the synthetic method. It returns a `CallSite`, which is then linked permanently.
4. Later executions reuse the linked call site. Non-capturing lambdas return the same instance each time; capturing lambdas create a new instance holding the captured values.

Benefits: no class-file bloat, and the JVM can change the strategy later without recompiling. You can see this with `javap -c -p MyClass`.

Gotchas:

- Checked exceptions: `Function.apply` declares none, so a lambda that calls `Files.readString` must catch the exception or wrap it.
- Do not rely on lambda identity (`==`) or serialisation unless the target interface is `Serializable`.
- Overload ambiguity: `submit(() -> foo())` can match both `Runnable` and `Callable`. `Javac` picks `Callable` if `foo()` returns a value.

Likely follow-ups:

- *Can a lambda throw a checked exception?* Only if the functional interface's method declares it (for example `Callable.call()` throws `Exception`). Otherwise wrap it or write a custom `ThrowingFunction`.
- *Is a lambda an object?* Yes. At runtime it is an instance of a hidden class implementing the interface.
- *Can lambdas have generic type parameters?* No. Use a method reference or a generic interface method instead.
- *Why is "effectively final" a rule and not just "final"?* It is convenience: Java 8 removed the need to write the `final` keyword, but the semantics are the same.

B3. Functional interfaces with respect to lambdas

A **functional interface** has exactly **one abstract method** (SAM, single abstract method). It may have any number of `default` / `static` methods, and methods inherited from `Object` (`equals`, `toString`) do not count. `@FunctionalInterface` is optional, but it makes the compiler enforce the rule.

```

@FunctionalInterface
interface Validator<T> {
    boolean validate(T t);
    default Validator<T> and(Validator<T> other) { return t -> validate(t) && other.validate(t); }
    boolean equals(Object o);    // allowed: Object method
}

```

The `java.util.function` catalogue (43 interfaces):

Interface	Method	Signature	Typical use
Supplier<T>	get()	() -> T	Lazy values, factories, <code>orElseGet</code>
Consumer<T>	accept(T)	T -> void	<code>forEach</code> , side effects
BiConsumer<T,U>	accept(T,U)	(T,U) -> void	<code>Map.forEach</code>
Function<T,R>	apply(T)	T -> R	<code>map</code>
BiFunction<T,U,R>	apply(T,U)	(T,U) -> R	<code>Map.merge</code> , <code>replaceAll</code>
UnaryOperator<T>	apply(T)	T -> T	<code>List.replaceAll</code>
BinaryOperator<T>	apply(T,T)	(T,T) -> T	<code>reduce</code>
Predicate<T>	test(T)	T -> boolean	<code>filter</code> , <code>removeIf</code>
BiPredicate<T,U>	test(T,U)	(T,U) -> boolean	Two-arg checks

Primitive specialisations avoid boxing and unboxing (an `Integer` object per element):

- `IntPredicate`, `LongPredicate`, `DoublePredicate`
- `IntFunction<R>`, `IntToLongFunction`, `IntToDoubleFunction`, `ToIntFunction<T>`, `ToLongFunction<T>`, `ToDoubleFunction<T>`, `ToIntBiFunction<T,U>` ...
- `IntSupplier`, `BooleanSupplier`, `IntConsumer`, `ObjIntConsumer<T>`
- `IntUnaryOperator`, `IntBinaryOperator`, and the Long/Double variants

Older interfaces that are also functional: `Runnable`, `Callable<V>`, `Comparator<T>`, `ActionListener`, `FileFilter`.

Composition:

```
import java.util.function.*;

public class Composition {
    public static void main(String[] args) {
        Function<Integer,Integer> times2 = x -> x * 2;
        Function<Integer,Integer> plus3 = x -> x + 3;

        System.out.println(times2.andThen(plus3).apply(5)); // (5*2)+3 = 13 -> this first, then arg
        System.out.println(times2.compose(plus3).apply(5)); // (5+3)*2 = 16 -> arg first, then this
        System.out.println(Function.<Integer>identity().apply(7)); // 7

        Predicate<String> notEmpty = s -> !s.isEmpty();
        Predicate<String> shortStr = s -> s.length() < 5;
        System.out.println(notEmpty.and(shortStr).test("abc")); // true
        System.out.println(notEmpty.and(shortStr).negate().test("abc")); // false
        System.out.println(shortStr.or(s -> s.startsWith("x")).test("xylophone")); // true
        System.out.println(Predicate.not(String::isBlank).test(" ")); // false (Java 11)
        System.out.println(Predicate.isEqual("a").test("a")); // true

        Consumer<StringBuilder> a = sb -> sb.append("A");
        Consumer<StringBuilder> b = sb -> sb.append("B");
        StringBuilder sb = new StringBuilder();
        a.andThen(b).accept(sb);
        System.out.println(sb); // AB

        BinaryOperator<Integer> max = BinaryOperator.maxBy(Integer::compare);
        System.out.println(max.apply(3, 9)); // 9

        IntUnaryOperator inc = x -> x + 1;
        System.out.println(inc.andThen(x -> x * 10).applyAsInt(1)); // 20
    }
}
```

Gotchas:

- `andThen` vs `compose` is a common trick question. `f.andThen(g)` is `g(f(x))`; `f.compose(g)` is `f(g(x))`.
- `Predicate.and` short-circuits like `&&`.
- `Supplier` is not `Callable`: `Supplier.get()` cannot throw checked exceptions.
- Generic interfaces work only with reference types, so `Function<int,int>` is invalid. Use `IntUnaryOperator`.

Likely follow-ups:

- Is `Comparator` a functional interface even though it declares `equals`? Yes. `equals` is an `Object` method and does not count.
- Can a functional interface extend another? Yes, as long as the total number of abstract methods stays one.

- *Why so many primitive variants?* To avoid boxing cost. There are only int/long/double variants, which is why there is no `CharStream`.
- *Is `@FunctionalInterface` mandatory?* No. It is a compile-time check and documentation.

B4. Method references

A method reference is shorthand for a lambda whose body only calls an existing method. Syntax: `Target::method`.

#	Kind	Syntax	Lambda equivalent	Example
1	Static method	<code>ClassName::staticMethod</code>	<code>x -> ClassName.staticMethod(x)</code>	<code>Integer::parseInt</code>
2	Instance method of a particular (bound) object	<code>instance::method</code>	<code>x -> instance.method(x)</code>	<code>System.out::println, "Hi"::concat</code>
3	Instance method of an arbitrary object of a type (unbound)	<code>ClassName::instanceMethod</code>	<code>(obj, args) -> obj.method(args)</code>	<code>String::toUpperCase, String::compareToIgnoreCase</code>
4	Constructor	<code>ClassName::new / Type[]::new</code>	<code>args -> new ClassName(args)</code>	<code>ArrayList::new, int[]::new</code>

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;

public class MethodRefs {
    record Person(String name) {}

    static boolean isLong(String s) { return s.length() > 4; }

    public static void main(String[] args) {
        List<String> words = List.of("apple", "Kiwi", "banana", "fig");

        // 1. Static
        Function<String,Integer> parse = Integer::parseInt;
        System.out.println(parse.apply("42")); // 42
        System.out.println(words.stream().filter(MethodRefs::isLong).toList()); // [apple, banana]

        // 2. Bound instance (the receiver is fixed)
        String prefix = "Mr. ";
        Function<String,String> addPrefix = prefix::concat;
        System.out.println(addPrefix.apply("Bean")); // Mr. Bean
        words.forEach(System.out::println); // System.out is the bound object

        // 3. Unbound instance (the first parameter becomes the receiver)
        Function<String,String> upper = String::toUpperCase; // s -> s.toUpperCase()
        BiFunction<String,String,Boolean> eq = String::equalsIgnoreCase; // (a,b) -> a.equalsIgnoreCase(b)
        System.out.println(upper.apply("x") + " " + eq.apply("A", "a")); // X true
        List<String> sorted = new ArrayList<>(words);
        sorted.sort(String::compareToIgnoreCase);
        System.out.println(sorted); // [apple, banana, fig, Kiwi]

        // 4. Constructor
        Supplier<List<String>> listMaker = ArrayList::new;
        Function<String,Person> mk = Person::new;
        IntFunction<String[]> arr = String[]::new;
        System.out.println(listMaker.get() + " " + mk.apply("Ana")); // [] Person[name=Ana]
        String[] asArray = words.stream().toArray(String[]::new);
        System.out.println(asArray.length + " " + arr.apply(3).length); // 4 3
        Set<String> set = words.stream().collect(Collectors.toCollection(TreeSet::new));
        System.out.println(set); // [Kiwi, apple, banana, fig]
    }
}
```

Gotchas:

- **Bound receivers are evaluated eagerly.** With `obj::method`, `obj` is evaluated when the reference is created. If `obj` is null you get an NPE immediately, while the equivalent lambda would throw only when called.
- Ambiguity: `Integer::toString` can match both static `toString(int)` and instance `toString()` for `Function<Integer,String>`, which is a compile error.
- Which constructor `::new` picks depends on the functional interface's parameter list (`Supplier` gives no-arg, `Function` gives one-arg).

Likely follow-ups:

- When should you prefer a lambda over a method reference? When you need extra logic, argument reordering, or the reference would be ambiguous or less readable.
- How does `String::compareTo` fit `Comparator<String>`? It is unbound: the first argument is the receiver, the second is the parameter.
- Can you reference `super::method`? Yes, `super::toString` inside an instance context.

B5. Stream API methods: intermediate vs terminal

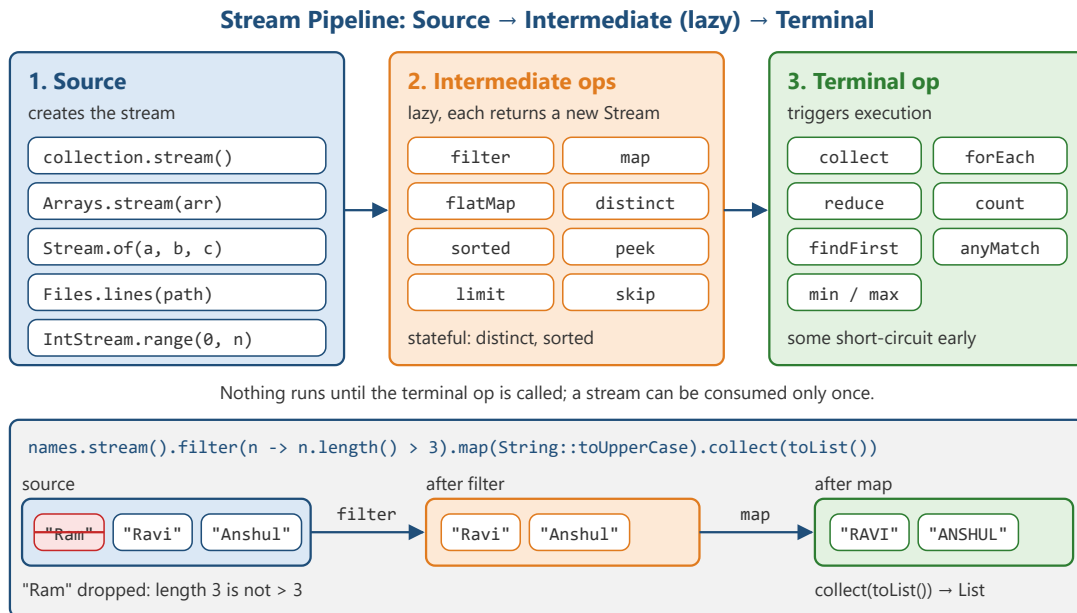


Figure 7: Java Stream pipeline

A stream pipeline = **source** -> zero or more **intermediate** operations -> exactly one **terminal** operation.

Intermediate operations return a new `Stream` and are **lazy**: nothing runs until a terminal operation is called.

Category	Methods
Filtering / slicing	<code>filter</code> , <code>distinct</code> , <code>limit</code> , <code>skip</code> *, <code>takeWhile</code> (9), <code>dropWhile</code> (9)
Mapping	<code>map</code> , <code>mapToInt/Long/Double/Obj</code> , <code>flatMap</code> , <code>flatMapToInt/...</code> , <code>mapMulti</code> (16), <code>boxed</code> , <code>asLongStream</code> , <code>asDoubleStream</code>
Ordering	<code>sorted()</code> , <code>sorted(Comparator)</code>
Debug	<code>peek</code>
Mode	<code>parallel</code> , <code>sequential</code> , <code>unordered</code> , <code>onClose</code>

* = **stateful** (must remember previously seen elements, or for `sorted` buffer everything). The others are **stateless**: each element is processed independently.

Terminal operations produce a result or a side effect and close the stream.

Category	Methods
Reduction	<code>reduce</code> (3 overloads), <code>collect</code> (2 overloads), <code>toList()</code> (16), <code>count</code> , <code>min</code> , <code>max</code>
Primitive-only	<code>sum</code> , <code>average</code> , <code>summaryStatistics</code>
Matching (short-circuit)	<code>anyMatch</code> , <code>allMatch</code> , <code>noneMatch</code>
Finding (short-circuit)	<code>findFirst</code> , <code>findAny</code>
Iteration	<code>forEach</code> , <code>forEachOrdered</code>
Conversion	<code>toArray()</code> , <code>toArray(IntFunction)</code> , <code>iterator</code> , <code>spliterator</code>

Short-circuiting operations can finish without consuming the whole (even infinite) source: `limit`, `takeWhile` (intermediate); `anyMatch`, `allMatch`, `noneMatch`, `findFirst`, `findAny` (terminal).

Laziness and vertical execution. Elements flow through the pipeline **one at a time** ("vertically"), not stage by stage:

```
import java.util.*;
import java.util.stream.*;

public class Laziness {
    public static void main(String[] args) {
        Stream<String> s = Stream.of("a", "bb", "ccc", "dddd")
            .filter(x -> { System.out.println("filter " + x); return x.length() > 1; })
            .map(x -> { System.out.println("map " + x); return x.toUpperCase(); });
        System.out.println("nothing printed yet"); // laziness
        System.out.println(s.findFirst().get());

        // Infinite stream made safe by short-circuiting
        System.out.println(Stream.iterate(1, x -> x * 2).limit(5).toList()); // [1, 2, 4, 8, 16]

        // Reuse is illegal
        Stream<Integer> once = Stream.of(1, 2, 3);
        once.count();
        try { once.count(); }
        catch (IllegalStateException e) { System.out.println(e.getMessage()); }
    }
}
```

Output:

```
nothing printed yet
filter a
filter bb
map bb
BB
[1, 2, 4, 8, 16]
stream has already been operated upon or closed
```

Notice that `ccc` and `dddd` were never touched, because `findFirst` short-circuited.

A stream cannot be reused. It is a one-shot view over the source. If you need it again, use a `Supplier<Stream<T>> sup = () -> list.stream();` and call `sup.get()` each time.

Other important facts:

- Streams do not store data and should not modify their source. Changing a non-concurrent source during the pipeline causes `ConcurrentModificationException` or undefined behaviour.
- Operation order matters for performance: put `filter` before `map` and before `sorted`.
- `sorted` is a barrier: it must consume all input before it emits anything, so it breaks the one-element-at-a-time flow.
- Streams are `AutoCloseable`. Close streams backed by I/O (`Files.lines()`); collection streams need no closing.
- `Stream.toList()` (16) returns an **unmodifiable** list. `Collectors.toList()` makes no guarantee, but in practice returns an `ArrayList`.

Likely follow-ups:

- What if there is no terminal operation?* Nothing executes at all.
 - Difference between `findFirst` and `findAny`?* `findAny` may return any element and is cheaper in parallel; `findFirst` respects encounter order.
 - `forEach` vs `forEachOrdered`?* In parallel, `forEach` does not guarantee order; `forEachOrdered` does, at a cost to performance.
 - `takeWhile` vs `filter`?* `takeWhile` stops at the first element that fails; `filter` checks every element.
-

B6. map() vs flatMap()

map() vs flatMap()

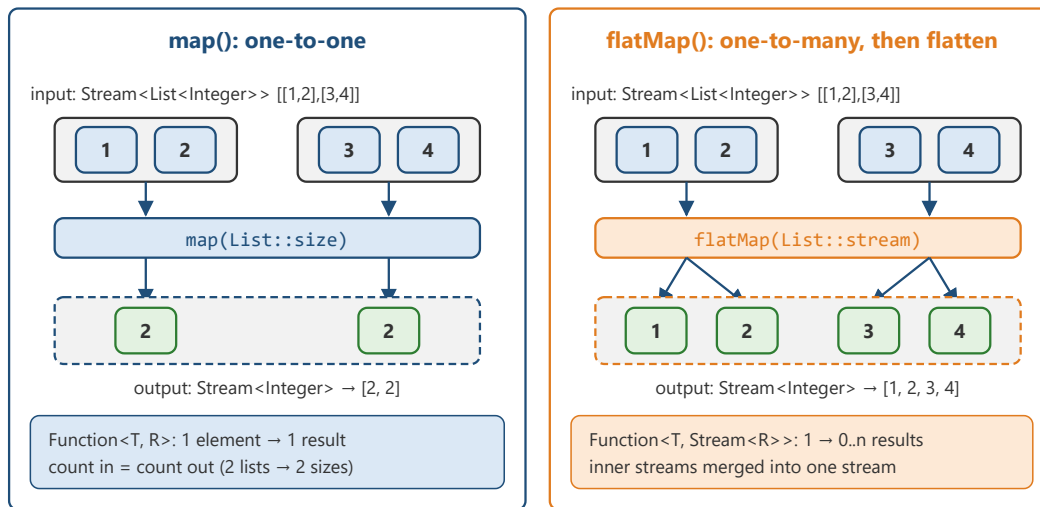


Figure 8: map vs flatMap in Java Streams

- `map(Function<T,R>)` is a **one-to-one** transformation: each input element produces exactly one output element. `Stream<T> -> Stream<R>`.
- `flatMap(Function<T, Stream<R>>)` is **one-to-many plus flatten**: each element becomes a stream of zero or more elements, and all those streams are concatenated into one. `Stream<T> -> Stream<R>`, not `Stream<Stream<R>>`.

If you use `map` with a function that returns a stream or list, you get nested structure: `Stream<List<String>>`. `flatMap` removes one level of nesting.

```
import java.util.*;
import java.util.stream.*;

public class MapVsFlatMap {
    record Order(String id, List<String> items) {}

    public static void main(String[] args) {
        List<String> sentences = List.of("hello world", "java streams rock");

        // map: one sentence -> one String[]
        List<String[]> arrays = sentences.stream().map(s -> s.split(" ")).toList();
        System.out.println(arrays.size()); // 2

        // flatMap: one sentence -> many words, flattened
        List<String> words = sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .toList();
        System.out.println(words); // [hello, world, java, streams, rock]

        // Real-world: all distinct items across orders
        List<Order> orders = List.of(
            new Order("o1", List.of("pen", "book")),
            new Order("o2", List.of("book", "bag")),
            new Order("o3", List.of())); // 0 elements is fine
        Set<String> items = orders.stream()
            .flatMap(o -> o.items().stream())
            .collect(Collectors.toCollection(TreeSet::new));
        System.out.println(items); // [bag, book, pen]

        // Distinct characters across words
        System.out.println(words.stream()
            .flatMapToInt(String::chars)
            .distinct()
            .mapToObj(c -> String.valueOf((char) c))
            .collect(Collectors.joining())); // helowrdjavstmck

        // Optional.flatMap avoids Optional<Optional<T>>
        Optional<String> name = Optional.of(" ana ");
        Optional<String> trimmed = name.flatMap(n -> n.isBlank() ? Optional.empty() : Optional.of(n.trim()));
        System.out.println(trimmed.get()); // ana
```

```
}  
}
```

mapMulti (Java 16) is an imperative alternative to `flatMap` when you produce only a few elements per input. It avoids creating a stream for each element:

```
Stream.of(1, 2, 3).<Integer>mapMulti((x, sink) -> { if (x % 2 == 1) { sink.accept(x); sink.accept(-x); } })  
    .toList(); // [1, -1, 3, -3]
```

Edge cases: if the `flatMap` function returns `null` instead of a stream, it is treated as an empty stream (documented behaviour), but returning `Stream.empty()` is clearer. A `null` element *inside* the list will pass through and can cause NPEs later.

Likely follow-ups:

- *Complexity of flatMap?* $O(\text{total number of inner elements})$. It adds the overhead of one stream per outer element.
- *Is flatMap lazy?* Yes. Since Java 10 it also short-circuits correctly inside inner streams (before that, `flatMap` + `findFirst` consumed the whole inner stream).
- *flatMap on Optional vs on Stream?* Same idea: `Optional.flatMap` flattens `Optional<Optional<T>>`, and `Stream.flatMap` flattens `Stream<Stream<T>>`.

B7. Debugging streams

Streams are hard to debug: a single line hides several stages, stack traces are full of `ReferencePipeline$...` frames, and evaluation is lazy and vertical. Techniques:

1. peek() for logging intermediate values

```
import java.util.*;  
import java.util.stream.*;  
  
public class PeekDemo {  
    public static void main(String[] args) {  
        List<Integer> result = Stream.of(5, 2, 8, 1, 9)  
            .peek(x -> System.out.println("source: " + x))  
            .filter(x -> x > 3)  
            .peek(x -> System.out.println("after filter: " + x))  
            .map(x -> x * 10)  
            .peek(x -> System.out.println("after map: " + x))  
            .toList();  
        System.out.println(result);  
    }  
}
```

Output (interleaved, showing vertical processing):

```
source: 5  
after filter: 5  
after map: 50  
source: 2  
source: 8  
after filter: 8  
after map: 80  
source: 1  
source: 9  
after filter: 9  
after map: 90  
[50, 80, 90]
```

peek caveats (frequently asked):

- **It is lazy.** Without a terminal operation, `peek` never runs.
- **Short-circuiting skips it.** With `findFirst` / `anyMatch` / `limit`, `peek` runs only for the elements actually pulled.
- **count() may skip the whole pipeline (Java 9+).** If the size is known from the source and no operation can change it, `count()` returns the size directly:

```
long n = List.of(1, 2, 3).stream().peek(System.out::println).count(); // prints NOTHING, n = 3  
long m = List.of(1, 2, 3).stream().filter(x -> true).peek(System.out::println).count(); // prints 1 2 3
```

- Its Javadoc says it exists **"mainly to support debugging"**. Do not use it for business side effects such as mutating elements or saving to a DB, because the stream implementation is free to elide it.

- In parallel streams, `peek` output is interleaved across threads.

2. Breakpoints inside lambdas. In IntelliJ/Eclipse, a breakpoint on a stream line offers "Line", "Lambda", or "All". Choose a lambda breakpoint to stop inside `x -> x * 10` for each element. Conditional breakpoints (`x == 8`) are very useful here.

3. IntelliJ Stream Trace (Java Stream Debugger). While paused on a line containing a stream chain, click the **"Trace Current Stream Chain"** button in the Debug tool window. It evaluates the pipeline and shows a tab for each stage, with arrows mapping input elements to output elements (Split mode or Flat mode). This is the quickest way to see where an element got filtered out.

4. Split the pipeline into named variables or methods.

```
Predicate<Employee> isSenior = e -> e.years() > 5;           // can be unit tested on its own
Function<Employee, String> toLabel = e -> e.name() + "(" + e.dept() + ")";
List<String> labels = emps.stream().filter(isSenior).map(toLabel).toList();
```

Extracting lambdas into methods (`.map(this::toLabel)`) also produces meaningful stack-trace frames.

5. Collect intermediate results temporarily: `var afterFilter = stream.filter(...).toList();` then inspect it, then continue streaming.

6. Exceptions: read the stack trace from the frame that names your lambda (`lambda$main$0`). Lambda methods are numbered by order of appearance in the class.

Likely follow-ups:

- *Why does `peek` print nothing in my code?* There is no terminal operation, or `count()` skipped the pipeline, or a short-circuit stopped early.
- *Can `peek` modify elements?* Technically it can mutate mutable objects, but that is an anti-pattern. Use `map`.
- *How do you debug a parallel stream?* Switch to `sequential()` temporarily, and log `Thread.currentThread().getName()` in `peek`.

B8. Optional: null-safe access, chaining with Stream, anti-patterns

`Optional<T>` is a container that holds either a non-null value or nothing. Its purpose is to be a **return type** that tells callers "this may be absent, handle it", so that NPE risk shows up in the method signature.

Creation: `Optional.of(v)` (NPE if `v` is null), `Optional.ofNullable(v)`, `Optional.empty()`.

API overview:

Method	Purpose
<code>isPresent()</code> , <code>isEmpty()</code> (11)	Checks (prefer the functional methods)
<code>ifPresent(Consumer)</code> , <code>ifPresentOrElse(Consumer, Runnable)</code> (9)	Side effects
<code>map</code> , <code>flatMap</code> , <code>filter</code>	Transform / chain
<code>or(Supplier<Optional>)</code> (9)	Fallback Optional
<code>orElse(T)</code>	Default value, always evaluated
<code>orElseGet(Supplier)</code>	Default value, lazily evaluated
<code>orElseThrow()</code> (10), <code>orElseThrow(Supplier)</code>	Throw if absent
<code>stream()</code> (9)	0- or 1-element stream

Null-safe navigation (replaces nested null checks):

```
import java.util.*;
import java.util.stream.*;

public class OptionalDemo {
    record City(String name) {}
    record Address(City city) {}
    record User(String id, Address address) {}

    static Map<String, User> db = Map.of(
        "u1", new User("u1", new Address(new City("Pune"))),
        "u2", new User("u2", null));

    static Optional<User> findUser(String id) { return Optional.ofNullable(db.get(id)); }

    static String cityOf(String id) {
        return findUser(id)
            .map(User::address)           // returns empty if address is null
            .map(Address::city)
    }
}
```

```

        .map(City::name)
        .map(String::toUpperCase)
        .orElse("UNKNOWN");
    }

    static String expensiveDefault() { System.out.println("computing default..."); return "X"; }

    public static void main(String[] args) {
        System.out.println(cityOf("u1")); // PUNE
        System.out.println(cityOf("u2")); // UNKNOWN
        System.out.println(cityOf("u9")); // UNKNOWN

        // orElse vs orElseGet
        Optional<String> present = Optional.of("A");
        present.orElse(expensiveDefault()); // prints "computing default..." anyway!
        present.orElseGet(OptionalDemo::expensiveDefault); // prints nothing

        // Chaining Optional with Stream: keep only the present values
        List<String> ids = List.of("u1", "u9", "u2");
        List<User> found = ids.stream()
            .map(OptionalDemo::findUser) // Stream<Optional<User>>
            .flatMap(Optional::stream) // Java 9+: drops the empties
            .toList();
        System.out.println(found.size()); // 2

        // Java 8 equivalent
        List<User> found8 = ids.stream().map(OptionalDemo::findUser)
            .filter(Optional::isPresent).map(Optional::get).collect(Collectors.toList());
        System.out.println(found8.size()); // 2

        // Stream terminal ops already return Optional
        Optional<String> longest = Stream.of("a", "abc", "ab").max(Comparator.comparingInt(String::length));
        longest.ifPresentOrElse(System.out::println, () -> System.out.println("empty")); // abc

        // Throw a domain exception
        try {
            findUser("u9").orElseThrow(() -> new NoSuchElementException("User u9 not found"));
        } catch (NoSuchElementException e) { System.out.println(e.getMessage()); }
    }
}

```

Anti-patterns:

Anti-pattern	Why it's bad	Do instead
<code>opt.get()</code> without a check	Throws <code>NoSuchElementException</code> , so you have just moved the NPE	<code>orElse</code> / <code>orElseThrow</code> / <code>map</code>
<pre>if (opt.isPresent()) { x = opt.get(); }</pre>	Verbose null check with extra steps	<code>opt.ifPresent(..)</code> / <code>opt.map(..).orElse(..)</code>
Optional fields	Not <code>Serializable</code> , extra object per field, JPA/Jackson friction	Nullable field + <code>Optional</code> getter: <code>Optional.ofNullable(field)</code>
Optional method parameters	Callers can still pass <code>null</code> ; awkward API	Overloads, or a nullable parameter with documentation
<code>Optional<List<T>></code>	Two kinds of "empty"	Return an empty collection
Returning <code>null</code> from an <code>Optional</code> -returning method	Defeats the whole purpose	Return <code>Optional.empty()</code>
<code>Optional.of(maybeNull)</code>	NPE immediately	<code>ofNullable</code>
<code>orElse(expensiveCall())</code>	The call is evaluated eagerly every time	<code>orElseGet</code>
<code>Optional<Integer></code> in hot paths	Boxing	<code>OptionalInt</code> / <code>OptionalLong</code> / <code>OptionalDouble</code>
Comparing <code>Optionals</code> with <code>==</code> or synchronising on them	They are value-based classes	<code>equals</code>

Likely follow-ups:

- *Why is `Optional` not `Serializable`?* It was designed only as a return type, not for fields or DTOs.
- `orElse` vs `orElseGet`? `orElse` evaluates its argument always; `orElseGet` calls the supplier only when the `Optional` is empty.
- *Does Jackson/JPA support `Optional`?* Jackson needs `jdk8-module` (included by default in Spring Boot). Spring Data repositories return `Optional<T>` from `findById`. JPA entity fields should not be `Optional`.
- *Is `Optional.get()` deprecated?* Not deprecated, but `orElseThrow()` (Java 10) was added as the clearer name for the same behaviour.

B9. Sequential vs parallel streams

- **Sequential** (default): one thread (the caller's) processes all elements in encounter order.
- **Parallel** (`list.parallelStream()` or `stream.parallel()`): the source is split into chunks with a **Splitter**, the chunks are processed on the **ForkJoinPool common pool**, and the partial results are combined.

Internals:

- **Common pool:** `ForkJoinPool.commonPool()` has `Runtime.availableProcessors() - 1` worker threads, and the **calling thread also takes part**. Configure it with `-Djava.util.concurrent.ForkJoinPool.common.parallelism=N`. The pool is **shared JVM-wide** (with `CompletableFuture.supplyAsync` without an executor, other parallel streams, and so on), so a blocking task in one place starves everything else.
- **Work stealing:** each worker has its own deque, and idle workers steal tasks from the tail of others' deques.
- **Splitter:** `trySplit()` divides the source. How well it splits decides whether parallel helps:
 - Excellent: `ArrayList`, arrays, `IntStream.range`, `HashMap` / `HashSet` (reasonably good)
 - Poor: `LinkedList`, `Stream.iterate`, `BufferedReader.lines()`, `Stream.generate` (sequential by nature)
- **Characteristics:** `SIZED`, `SUBSIZED`, `ORDERED`, `DISTINCT`, `SORTED`, `IMMUTABLE`, `CONCURRENT`, `NONNULL` let the framework optimise.

Custom pool trick (isolate from the common pool). This relies on an implementation detail: tasks forked from inside a `ForkJoinPool` run in that pool.

```
ForkJoinPool pool = new ForkJoinPool(4);
long result = pool.submit(() -> bigList.parallelStream().mapToLong(this::cpuHeavy).sum()).get();
pool.shutdown();
```

When parallel helps:

- Large N (Brian Goetz's "NQ model": $N \times Q$, where Q is the cost per element, should be above roughly 10,000)
- CPU-bound, independent, stateless operations
- A source that splits well; cheap combine (sum, min); few stateful operations
- Multiple cores available (do not do this inside a busy web server where each request thread already uses a core)

When it hurts: small collections, I/O or blocking calls (DB, HTTP), `LinkedList` / `iterate` sources, boxed streams, order-dependent operations (`limit`, `findFirst`, `sorted` on ordered streams), contention on shared state.

Thread-safety:

```
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;

public class ParallelDemo {
    public static void main(String[] args) {
        // WRONG: shared mutable ArrayList, a race condition
        List<Integer> unsafe = new ArrayList<>();
        try {
            IntStream.range(0, 10_000).parallel().forEach(unsafe::add);
        } catch (ArrayIndexOutOfBoundsException e) { System.out.println("crashed: " + e); }
        System.out.println("unsafe size (often < 10000): " + unsafe.size());

        // RIGHT: let the collector handle the merging
        List<Integer> safe = IntStream.range(0, 10_000).parallel().boxed().toList();
        System.out.println("safe size: " + safe.size()); // 10000, in order

        // Ordering
        IntStream.range(0, 5).parallel().forEach(i -> System.out.print(i + " ")); // e.g. 2 4 3 0 1
        System.out.println();
        IntStream.range(0, 5).parallel().forEachOrdered(i -> System.out.print(i + " ")); // 0 1 2 3 4
        System.out.println();

        // Thread names
        Set<String> threads = ConcurrentHashMap.newKeySet();
        IntStream.range(0, 1000).parallel().forEach(i -> threads.add(Thread.currentThread().getName()));
        System.out.println(threads); // [main, ForkJoinPool.commonPool-worker-1, ...]

        // Non-associative reduce gives a wrong answer in parallel
        int bad = IntStream.rangeClosed(1, 4).parallel().reduce(0, (a, b) -> a - b);
        System.out.println("subtraction reduce (unpredictable): " + bad);
    }
}
```

Ordering rules: parallel streams preserve encounter order for `toList`, `collect`, `findFirst`, `forEachOrdered`, `limit`, `skip`. That preservation costs buffering. If you do not care about order, use `.unordered()`, `findAny`, `forEach`, or `Collectors.toConcurrentMap/groupingByConcurrent`.

Requirements for correct parallel reductions: the accumulator and combiner must be **associative**, **non-interfering** (do not modify the source), and **stateless**. The identity must truly be an identity: `reduce(10, Integer::sum)` adds 10 once per chunk in parallel.

Likely follow-ups:

- *How many threads does a parallel stream use?* Common pool parallelism (cores - 1) plus the calling thread.
- *Is `parallelStream()` on a synchronised list thread-safe?* Reading is fine. The real issue is side effects in the lambdas, which you must avoid.
- *Can a parallel stream be slower?* Yes. Splitting and merging overhead plus thread hand-offs often make it slower for small or cheap workloads. Benchmark with JMH.
- *How do you turn a parallel stream back to sequential?* Call `.sequential()`. The last mode call wins for the **whole** pipeline; you cannot have mixed stages.

B10. Count duplicate elements in an array

Problem: given `int[] arr = {1, 2, 3, 2, 4, 1, 2, 5}`, report which elements appear more than once and how many times.

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.*;

public class CountDuplicates {

    // Stream version
    static Map<Integer, Long> duplicatesStream(int[] arr) {
        if (arr == null || arr.length == 0) return Map.of();
        return Arrays.stream(arr)
            .boxed()
            .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()))
            .entrySet().stream()
            .filter(e -> e.getValue() > 1)
            .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));
    }

    // Loop version with Map.merge
    static Map<Integer, Integer> duplicatesLoop(int[] arr) {
        Map<Integer, Integer> freq = new LinkedHashMap<>();
        if (arr == null) return freq;
        for (int x : arr) freq.merge(x, 1, Integer::sum);
        freq.values().removeIf(c -> c == 1);
        return freq;
    }

    // Only "how many distinct values are duplicated" using a Set
    static long countDuplicatedValues(int[] arr) {
        Set<Integer> seen = new HashSet<>(), dup = new HashSet<>();
        for (int x : arr) if (!seen.add(x)) dup.add(x);
        return dup.size();
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2, 4, 1, 2, 5};
        System.out.println(duplicatesStream(arr)); // {1=2, 2=3}
        System.out.println(duplicatesLoop(arr)); // {1=2, 2=3}
        System.out.println(countDuplicatedValues(arr)); // 2
        System.out.println(duplicatesStream(new int[0])); // {}
        System.out.println(duplicatesLoop(null)); // {}
    }
}
```

Complexity: $O(n)$ time, $O(k)$ space (k = distinct values). **Alternative:** sort the array and then count consecutive runs: $O(n \log n)$ time, $O(1)$ extra space (if you may mutate the array). For a small bounded range (for example 0..1000), use an `int[] count` array. **Edge cases:** null or empty input returns an empty map; all unique values return an empty map; negative numbers work with the map approach but not with a naive count array.

Likely follow-ups:

- *Why `LinkedHashMap::new`?* It keeps first-occurrence order. Plain `groupingBy` uses `HashMap`, so the order is unspecified.
- *How do you return the duplicates as a List?* Use `.map(Map.Entry::getKey).toList()` on the filtered entries.
- *Why `boxed()`?* `IntStream` has no `collect(Collector)`; grouping needs objects.

B11. Find even numbers from a list

```
import java.util.*;
import java.util.stream.*;

public class EvenNumbers {
    static List<Integer> evens(List<Integer> nums) {
        if (nums == null) return List.of();
        return nums.stream()
            .filter(Objects::nonNull)           // guard against null elements
            .filter(n -> n % 2 == 0)           // works for negatives: -4 % 2 == 0
            .toList();
    }

    static List<Integer> evensLoop(List<Integer> nums) {
        List<Integer> out = new ArrayList<>();
        for (Integer n : nums) if (n != null && (n & 1) == 0) out.add(n);
        return out;
    }

    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(1, 2, 3, 4, null, -6, 7, 8);
        System.out.println(evens(nums));           // [2, 4, -6, 8]
        System.out.println(evensLoop(nums));       // [2, 4, -6, 8]
        System.out.println(evens(List.of()));     // []

        // Primitive variants: no boxing
        int[] arr = {10, 15, 20, 25};
        System.out.println(Arrays.toString(IntStream.of(arr).filter(x -> x % 2 == 0).toArray())); // [10, 20]
        System.out.println(IntStream.rangeClosed(1, 10).filter(x -> x % 2 == 0).sum()); // 30
    }
}
```

Complexity: $O(n)$ time, $O(n)$ output space. **Gotcha:** testing oddness with `n % 2 == 1` fails for negative odd numbers, because `-3 % 2 == -1`. Use `n % 2 != 0` or `(n & 1) == 1`. **Edge cases:** null list, null elements, empty list.

Likely follow-ups:

- Remove the odd numbers in place? `list.removeIf(n -> n % 2 != 0)` on a mutable list.
- Get evens and odds together? `Collectors.partitioningBy(n -> n % 2 == 0)` (see B28).
- Is `toList()` mutable? No. Use `collect(Collectors.toCollection(ArrayList::new))` if you need to modify the result.

B12. Check occurrence of a character

(a) Count one character:

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.*;

public class CharOccurrence {
    static long countChar(String s, char target) {
        if (s == null || s.isEmpty()) return 0;
        return s.chars().filter(c -> c == target).count();
    }

    static int countCharLoop(String s, char target) {
        int count = 0;
        for (int i = 0; i < s.length(); i++) if (s.charAt(i) == target) count++;
        return count;
    }

    // Trick: length difference after removal
    static int countCharReplace(String s, char target) {
        return s.length() - s.replace(String.valueOf(target), "").length();
    }

    // (b) Frequency map of all characters (ignoring spaces), in first-occurrence order
    static Map<Character, Long> frequency(String s) {
        if (s == null) return Map.of();
        return s.chars()
            .filter(c -> !Character.isWhitespace(c))
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
    }
}
```

```
// Array-based frequency for lowercase a-z
static int[] frequencyArray(String s) {
    int[] f = new int[26];
    for (char c : s.toCharArray()) if (c >= 'a' && c <= 'z') f[c - 'a']++;
    return f;
}

public static void main(String[] args) {
    String s = "hello world";
    System.out.println(countChar(s, 'l')); // 3
    System.out.println(countCharLoop(s, 'o')); // 2
    System.out.println(countCharReplace(s, 'l')); // 3
    System.out.println(frequency(s)); // {h=1, e=1, l=3, o=2, w=1, r=1, d=1}
    System.out.println(frequencyArray(s)['l' - 'a']); // 3
    System.out.println(countChar("", 'x')); // 0

    // Case-insensitive
    System.out.println("Banana".toLowerCase().chars().filter(c -> c == 'b').count()); // 1
}
}
```

Complexity: $O(n)$ time. Space: $O(1)$ for counting one character, $O(k)$ for the map, $O(26)$ for the array. **Gotchas:** `chars()` returns an `IntStream` of UTF-16 code units, so `(char) c` is required for a `Character` key. For emoji or supplementary characters use `codePoints()`. Case sensitivity: normalise with `toLowerCase(Locale.ROOT)`.

Likely follow-ups:

- *Most frequent character?* `frequency(s).entrySet().stream().max(Map.Entry.comparingByValue()).map(Map.Entry::getKey)`.
- *Why `LinkedHashMap`?* Deterministic order for output and tests.
- *Count occurrences of a substring?* Loop with `indexOf(sub, from)`, or `s.split(Pattern.quote(sub), -1).length - 1`.

B13. Count first-character occurrences

Problem: `List.of("guava", "grapes", "apple", "orange") -> g-2, a-1, o-1` (in order of first appearance).

```
import java.util.*;
import java.util.stream.*;

public class FirstCharCount {
    static Map<Character, Long> countByFirstChar(List<String> words) {
        if (words == null) return Map.of();
        return words.stream()
            .filter(w -> w != null && !w.isEmpty()) // charAt(0) would throw on ""
            .collect(Collectors.groupingBy(
                w -> w.charAt(0), // classifier
                LinkedHashMap::new, // keep encounter order
                Collectors.counting())); // downstream
    }

    static Map<Character, Integer> countLoop(List<String> words) {
        Map<Character, Integer> m = new LinkedHashMap<>();
        for (String w : words) if (w != null && !w.isEmpty()) m.merge(w.charAt(0), 1, Integer::sum);
        return m;
    }

    public static void main(String[] args) {
        List<String> fruits = List.of("guava", "grapes", "apple", "orange");
        Map<Character, Long> result = countByFirstChar(fruits);
        System.out.println(result); // {g=2, a=1, o=1}
        result.forEach((k, v) -> System.out.println(k + "-" + v)); // g-2 / a-1 / o-1
        System.out.println(countLoop(fruits)); // {g=2, a=1, o=1}

        // Case-insensitive variant
        System.out.println(countByFirstChar(List.of("Guava", "grapes")).size()); // 2 (G and g differ)
        Map<Character, Long> ci = Stream.of("Guava", "grapes")
            .collect(Collectors.groupingBy(w -> Character.toLowerCase(w.charAt(0)),
                LinkedHashMap::new, Collectors.counting()));
        System.out.println(ci); // {g=2}
    }
}
```

Why `LinkedHashMap`? The 1-argument and 2-argument `groupingBy` use a `HashMap`. For `Character` keys a `HashMap` happens to iterate by hash (char code), giving `{a=1, g=2, o=1}`, which is *not* the order asked for. The 3-argument overload with a map factory gives you `LinkedHashMap` (insertion order) or `TreeMap` (sorted).

Complexity: $O(n)$ time, $O(k)$ space. **Edge cases:** empty strings (guarded), null list or elements, mixed case.

Likely follow-ups:

- *Get the words themselves grouped by first character?* `groupingBy(w -> w.charAt(0), LinkedHashMap::new, Collectors.toList())`.
- *Return an Integer count instead of Long?* `Collectors.summingInt(w -> 1)` or `collectingAndThen(counting(), Long::intValue)`.
- *Sorted by count descending?* Stream the entries and sort with `Map.Entry.comparingByValue(Comparator.reverseOrder())` (see B26).

B14. Remove duplicates while maintaining order

```
import java.util.*;
import java.util.stream.*;

public class RemoveDuplicates {
    // 1. distinct(): uses equals/hashCode and keeps the FIRST occurrence in encounter order
    static <T> List<T> dedupStream(List<T> list) {
        return list == null ? List.of() : list.stream().distinct().toList();
    }

    // 2. LinkedHashSet: classic, no streams
    static <T> List<T> dedupSet(List<T> list) {
        return new ArrayList<>(new LinkedHashSet<>(list));
    }

    // 3. Manual loop with a seen-set (lets you add custom logic)
    static <T> List<T> dedupLoop(List<T> list) {
        Set<T> seen = new HashSet<>();
        List<T> out = new ArrayList<>();
        for (T t : list) if (seen.add(t)) out.add(t);
        return out;
    }

    // 4. Deduplicate by a key (e.g. case-insensitive), keeping the first occurrence
    static List<String> dedupIgnoreCase(List<String> list) {
        Set<String> seen = new HashSet<>();
        return list.stream().filter(s -> seen.add(s.toLowerCase())).toList(); // stateful lambda: sequential only!
    }

    public static void main(String[] args) {
        List<Integer> nums = List.of(5, 3, 5, 1, 3, 7, 1);
        System.out.println(dedupStream(nums)); // [5, 3, 1, 7]
        System.out.println(dedupSet(nums));    // [5, 3, 1, 7]
        System.out.println(dedupLoop(nums));   // [5, 3, 1, 7]
        System.out.println(dedupIgnoreCase(List.of("Java", "java", "Go", "GO", "rust"))); // [Java, Go, rust]

        // Characters in a string
        String s = "programming";
        System.out.println(s.chars().distinct()
            .mapToObj(c -> String.valueOf((char) c)).collect(Collectors.joining())); // progamin
    }
}
```

Complexity: $O(n)$ time and $O(n)$ space for all approaches. A naive nested loop or `list.contains` check is $O(n^2)$. **Gotchas:**

- `distinct()` relies on correct `equals` / `hashCode`. Custom objects without them will not be deduplicated.
- In parallel streams on ordered sources, `distinct()` is expensive (it must buffer to keep the first occurrence). Use `.unordered().distinct()` if order does not matter.
- `HashSet` loses order and `TreeSet` sorts; only `LinkedHashSet` preserves insertion order.
- `List.of(...)` rejects null elements; `Arrays.asList` and `ArrayList` allow them, and `distinct()` handles null fine.

Likely follow-ups:

- *Deduplicate employees by id, keeping the last occurrence?* `toMap(Employee::id, e -> e, (a, b) -> b, LinkedHashMap::new).values()`.
- *Is `distinct()` stateful?* Yes. It keeps a set of seen elements.
- *Deduplicate an array in place without extra space?* Sort it, then use the two-pointer technique (this loses the original order).

B15. Remove null and empty strings

```
import java.util.*;
import java.util.function.Predicate;
import java.util.stream.*;
```

```

public class RemoveNullEmpty {
    static List<String> clean(List<String> input) {
        if (input == null) return List.of();
        return input.stream()
            .filter(Objects::nonNull)
            .filter(Predicate.not(String::isBlank)) // Java 11: removes "", " ", "\t"
            .toList();
    }

    // Java 8 compatible
    static List<String> clean8(List<String> input) {
        return input.stream()
            .filter(s -> s != null && !s.trim().isEmpty())
            .collect(Collectors.toList());
    }

    public static void main(String[] args) {
        List<String> data = Arrays.asList("java", null, "", " ", "spring", "\t", "boot");
        System.out.println(clean(data)); // [java, spring, boot]
        System.out.println(clean8(data)); // [java, spring, boot]

        // Only remove null and "" (keep whitespace-only strings)
        System.out.println(data.stream().filter(s -> s != null && !s.isEmpty()).count()); // 5

        // In place on a mutable list
        List<String> mutable = new ArrayList<>(data);
        mutable.removeIf(s -> s == null || s.isBlank());
        System.out.println(mutable); // [java, spring, boot]

        // Clean AND trim
        System.out.println(Stream.of(" a ", null, " b").filter(Objects::nonNull)
            .map(String::strip).filter(s -> !s.isEmpty()).toList()); // [a, b]
    }
}

```

Complexity: $O(n \times m)$ time where m is the average string length (`isBlank` scans the string), $O(n)$ space. **Gotchas:**

- Order of filters matters: `s.isEmpty()` before the null check throws NPE.
- `isEmpty()` checks only length; `isBlank()` (11) also treats whitespace-only strings as empty, and is Unicode-aware like `strip()`, unlike `trim()`.
- `List.of(...)` with a `null` element throws NPE at creation, which is why the example uses `Arrays.asList`.
- Apache Commons `StringUtils.isNotBlank` is a common alternative in pre-11 codebases.

Likely follow-ups:

- `trim()` vs `strip()`? `trim` removes characters $\leq U+0020$; `strip` removes all Unicode whitespace.
- Remove nulls from a Map's values? `map.values().removeIf(Objects::isNull)`.
- Why not `filter(s -> s != "")`? `!=` compares references. Always use `isEmpty()` or `equals`.

B16. Convert list of strings to uppercase

```

import java.util.*;
import java.util.stream.*;

public class ToUpper {
    static List<String> upper(List<String> in) {
        if (in == null) return List.of();
        return in.stream()
            .filter(Objects::nonNull)
            .map(s -> s.toUpperCase(Locale.ROOT)) // locale-safe
            .toList();
    }

    public static void main(String[] args) {
        List<String> names = List.of("java", "spring", "kafka");
        System.out.println(names.stream().map(String::toUpperCase).toList()); // [JAVA, SPRING, KAFKA]
        System.out.println(upper(Arrays.asList("a", null, "b"))); // [A, B]

        // In place (needs a mutable list)
        List<String> mutable = new ArrayList<>(names);
        mutable.replaceAll(String::toUpperCase);
        System.out.println(mutable); // [JAVA, SPRING, KAFKA]

        // Classic loop
    }
}

```

```

List<String> out = new ArrayList<>();
for (String s : names) out.add(s.toUpperCase());
System.out.println(out);

// The Turkish-locale gotcha
System.out.println("title".toUpperCase(new Locale("tr"))); // TITLE (dotted capital I)
System.out.println("title".toUpperCase(Locale.ROOT));        // TITLE
}
}

```

Complexity: $O(\text{total characters})$ time, $O(n)$ space for the new list. **Gotchas:** `toUpperCase()` with no argument uses the JVM default locale. In Turkish, `i` becomes `İ`, which breaks comparisons such as `"title".toUpperCase().equals("TITLE")`. Use `Locale.ROOT` for machine-readable strings (keys, enum names, headers). `List.of(...).replaceAll` throws `UnsupportedOperationException`.

Likely follow-ups:

- Capitalise only the first letter? `s.isEmpty() ? s : Character.toUpperCase(s.charAt(0)) + s.substring(1)`.
- Return a `Set` instead? `collect(Collectors.toSet())` or `toCollection(LinkedHashSet::new)` to keep order.
- Is `String::toUpperCase` a bound or unbound reference? Unbound (arbitrary object of type `String`).

B17. Join strings into CSV

```

import java.util.*;
import java.util.stream.*;

public class JoinCsv {
    public static void main(String[] args) {
        List<String> cols = List.of("id", "name", "salary");

        System.out.println(String.join(",", cols)); // id,name,salary
        System.out.println(cols.stream().collect(Collectors.joining(","))); // id,name,salary
        System.out.println(cols.stream().collect(Collectors.joining(", ", "[", "]"))); // [id, name, salary]
        System.out.println(cols.stream().collect(Collectors.joining("'", "'", "(" + ", " + ")"))); // ('id','name','salary') SQL IN

        // Non-strings must be mapped first
        List<Integer> ids = List.of(101, 102, 103);
        System.out.println(ids.stream().map(String::valueOf).collect(Collectors.joining("|"))); // 101|102|103

        // Empty input: the prefix and suffix are still printed
        System.out.println(Stream.<String>empty().collect(Collectors.joining(", ", "[", "]"))); // []

        // StringJoiner with a custom empty value
        StringJoiner sj = new StringJoiner(",", "{", "}");
        sj.setEmptyValue("EMPTY");
        System.out.println(sj); // EMPTY
        sj.add("x").add("y");
        System.out.println(sj); // {x,y}

        // Proper CSV escaping (values containing commas or quotes)
        List<String> row = List.of("Ana", "Pune, MH", "says \"hi\"");
        System.out.println(row.stream().map(JoinCsv::escape).collect(Collectors.joining(",")));
        // Ana,"Pune, MH","says ""hi""

    }

    static String escape(String v) {
        if (v == null) return "";
        boolean needsQuote = v.contains(",") || v.contains("\"") || v.contains("\n");
        String escaped = v.replace("\"", "\\\"");
        return needsQuote ? "\"" + escaped + "\"" : escaped;
    }
}

```

Internals: `Collectors.joining` uses a `StringBuilder` (via `StringJoiner`), so it is $O(\text{total length})$. Concatenating with `+` in a loop is $O(n^2)$ in the worst case. **Gotchas:** `joining` accepts only `CharSequence` elements, so map numbers first. `null` elements are appended as the text `"null"`; filter them out. For real CSV output use a library (OpenCSV, Apache Commons CSV) to handle escaping.

Likely follow-ups:

- `String.join` vs `Collectors.joining`? `String.join` works on an existing `Iterable` or array; `joining` works inside a stream pipeline and supports prefix and suffix.
- Is `joining` safe in parallel streams? Yes. The combiner merges `StringJoiner`s in order.
- Join a `Map` as `k=v&k2=v2`? `map.entrySet().stream().map(e -> e.getKey() + "=" + e.getValue()).collect(joining("&"))`.

B18. Sum using reduce()

`reduce` folds a stream into a single value by repeatedly applying a combining function. There are three overloads:

Overload	Returns	Use
<code>reduce(BinaryOperator<T> acc)</code>	<code>Optional<T></code>	No identity, so an empty stream gives an empty <code>Optional</code>
<code>reduce(T identity, BinaryOperator<T> acc)</code>	<code>T</code>	Identity is returned for an empty stream
<code>reduce(U identity, BiFunction<U,? super T,U> acc, BinaryOperator<U> combiner)</code>	<code>U</code>	Result type differs from the element type; the combiner merges partial results in parallel

```
import java.util.*;
import java.util.stream.*;

public class ReduceSum {
    public static void main(String[] args) {
        List<Integer> nums = List.of(1, 2, 3, 4, 5);

        int sum1 = nums.stream().reduce(0, Integer::sum); // 15
        int sum2 = nums.stream().reduce(0, (a, b) -> a + b); // 15
        Optional<Integer> sum3 = nums.stream().reduce(Integer::sum); // Optional[15]
        Optional<Integer> empty = Stream.<Integer>empty().reduce(Integer::sum); // Optional.empty
        int sum4 = nums.stream().mapToInt(Integer::intValue).sum(); // 15, no boxing (preferred)
        int sum5 = nums.stream().collect(Collectors.summingInt(Integer::intValue)); // 15

        // 3-arg form: the result type (int) differs from the element type (String)
        List<String> words = List.of("java", "is", "fun");
        int totalLen = words.parallelStream().reduce(0,
            (partial, w) -> partial + w.length(), // accumulator: (Integer, String) -> Integer
            Integer::sum); // combiner: merges partial sums
        System.out.println(totalLen); // 9

        int product = nums.stream().reduce(1, (a, b) -> a * b); // 120 (identity 1)
        int max = nums.stream().reduce(Integer.MIN_VALUE, Integer::max); // 5

        // Overflow-safe sum of large values
        long big = Stream.of(Integer.MAX_VALUE, 1).mapToLong(Integer::longValue).sum(); // 2147483648

        // WRONG identity: 10 is not the identity of +
        int wrong = nums.parallelStream().reduce(10, Integer::sum);
        System.out.println(sum1 + " " + sum2 + " " + sum3 + " " + empty + " " + sum4 + " " + sum5
            + " " + product + " " + max + " " + big + " " + wrong);
        // 15 15 Optional[15] Optional.empty 15 15 120 5 2147483648 65 (wrong: 15 + 10*chunks, varies by core count)
    }
}
```

Rules: the identity must satisfy `acc(identity, x) == x`; the accumulator must be **associative**; and in the 3-argument form, `combiner(u, acc(identity, t)) == acc(u, t)`.

reduce vs sum(): `IntStream.sum()` avoids boxing every element into an `Integer` and is easier to read. Use `reduce` for custom folds (product, max of a derived value, merging objects). For **mutable** accumulation (building a list or `StringBuilder`), use `collect`, not `reduce`. `reduce` should produce new immutable values each time.

Complexity: $O(n)$ time, $O(1)$ space.

Likely follow-ups:

- Why does the 3-argument `reduce` need a combiner? In parallel each chunk produces a partial `U`, and partial `U`s cannot be fed back into an accumulator that expects a `T`.
- `reduce` vs `collect`? `reduce` is immutable reduction (it combines values); `collect` is mutable reduction (it adds into a container).
- Average with `reduce`? Better to use `mapToInt(...).average()` or `Collectors.averagingInt / summaryStatistics()`.

B19. List to Map<String,Integer> (string to length) and the duplicate key issue

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.*;

public class ListToMap {
    public static void main(String[] args) {
        List<String> words = List.of("java", "spring", "kafka");
```

```

Map<String, Integer> m = words.stream()
    .collect(Collectors.toMap(Function.identity(), String::length));
System.out.println(m); // {spring=6, kafka=5, java=4} (HashMap: order unspecified)

// Ordered
Map<String, Integer> ordered = words.stream()
    .collect(Collectors.toMap(Function.identity(), String::length, (a, b) -> a, LinkedHashMap::new));
System.out.println(ordered); // {java=4, spring=6, kafka=5}

// Duplicate keys
List<String> dup = List.of("java", "go", "java");
try {
    dup.stream().collect(Collectors.toMap(Function.identity(), String::length));
} catch (IllegalStateException e) {
    System.out.println(e.getMessage()); // Duplicate key java (attempted merging values 4 and 4)
}

// Fix 1: merge function (keep the first)
System.out.println(dup.stream().collect(
    Collectors.toMap(Function.identity(), String::length, (oldV, newV) -> oldV))); // {java=4, go=2}

// Fix 2: distinct() first
System.out.println(dup.stream().distinct().collect(Collectors.toMap(w -> w, String::length)));

// Fix 3: inverse (length -> words) needs groupingBy, because many words share a length
Map<Integer, List<String>> byLen = Stream.of("java", "kafka", "ruby", "go")
    .collect(Collectors.groupingBy(String::length, TreeMap::new, Collectors.toList()));
System.out.println(byLen); // {2=[go], 4=[java, ruby], 5=[kafka]}

// Fix 4: count occurrences instead of failing
System.out.println(dup.stream().collect(Collectors.toMap(w -> w, w -> 1, Integer::sum))); // {java=2, go=1}
}

```

Complexity: O(n) time, O(n) space. **Gotchas:**

- Duplicate keys give `IllegalStateException` (Java 9+ messages include both values).
- **Null values give NPE**, because `toMap` uses `HashMap.merge`, which rejects null values. Workaround: `collect(HashMap::new, (map, s) -> map.put(s, f(s)), HashMap::putAll)`.
- Null keys are allowed by `HashMap`, but a null element would NPE on `String::length`.
- The map returned by the 2-argument and 3-argument `toMap` is a `HashMap`, but that is not guaranteed by the spec. Use the 4-argument form when you need a specific type.

Likely follow-ups:

- *Unmodifiable result?* `Collectors.toUnmodifiableMap(...)` (Java 10), which also rejects nulls.
- *Why does `toMap` throw while `groupingBy` doesn't?* `toMap` expects one value per key; `groupingBy` collects many values per key by design.
- *Thread-safe map from a parallel stream?* `Collectors.toConcurrentMap`.

B20. Group employees by department

```

import java.util.*;
import java.util.stream.*;

public class GroupEmployees {
    record Employee(String name, String dept, double salary, int age) {}

    public static void main(String[] args) {
        List<Employee> emps = List.of(
            new Employee("Asha", "IT", 90_000, 30),
            new Employee("Ravi", "HR", 50_000, 41),
            new Employee("John", "IT", 120_000, 35),
            new Employee("Meera", "Finance", 75_000, 28),
            new Employee("Kiran", "HR", 60_000, 25));

        // 1. Department -> List<Employee>
        Map<String, List<Employee>> byDept = emps.stream()
            .collect(Collectors.groupingBy(Employee::dept));
        System.out.println(byDept.keySet()); // [Finance, HR, IT] (HashMap order)

        // 2. Count per department (sorted keys with TreeMap)
        Map<String, Long> count = emps.stream()
            .collect(Collectors.groupingBy(Employee::dept, TreeMap::new, Collectors.counting()));
        System.out.println(count); // {Finance=1, HR=2, IT=2}
    }
}

```

```

// 3. Average salary per department
Map<String, Double> avg = emps.stream()
    .collect(Collectors.groupingBy(Employee::dept, TreeMap::new, Collectors.averagingDouble(Employee::salary)));
System.out.println(avg); // {Finance=75000.0, HR=55000.0, IT=105000.0}

// 4. Names only per department
Map<String, List<String>> names = emps.stream()
    .collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
        Collectors.mapping(Employee::name, Collectors.toList())));
System.out.println(names); // {Finance=[Meera], HR=[Ravi, Kiran], IT=[Asha, John]}

// 5. Highest-paid per department
Map<String, String> topPaid = emps.stream()
    .collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
        Collectors.collectingAndThen(
            Collectors.maxBy(Comparator.comparingDouble(Employee::salary)),
            opt -> opt.map(Employee::name).orElse(""))));
System.out.println(topPaid); // {Finance=Meera, HR=Kiran, IT=John}

// 6. Total salary and statistics
Map<String, DoubleSummaryStatistics> stats = emps.stream()
    .collect(Collectors.groupingBy(Employee::dept, Collectors.summarizingDouble(Employee::salary)));
System.out.println(stats.get("IT").getMax() + " " + stats.get("IT").getSum()); // 120000.0 210000.0

// 7. Multi-level grouping: dept -> (age >= 30?) -> names
Map<String, Map<Boolean, List<String>>> multi = emps.stream()
    .collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
        Collectors.partitioningBy(e -> e.age() >= 30,
            Collectors.mapping(Employee::name, Collectors.toList()))));
System.out.println(multi);
// {Finance={false=[Meera], true=[]}, HR={false=[Kiran], true=[Ravi]}, IT={false=[], true=[Asha, John]}}

// 8. Joined names
Map<String, String> joinedNames = emps.stream().collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
    Collectors.mapping(Employee::name, Collectors.joining("/"))));
System.out.println(joinedNames); // {Finance=Meera, HR=Ravi/Kiran, IT=Asha/John}
}

```

Key collectors to know: `groupingBy(classifier [, mapFactory] [, downstream])`, downstream options `counting`, `summingX`, `averagingX`, `summarizingX`, `mapping`, `filtering` (9), `flatMapMapping` (9), `maxBy / minBy`, `collectingAndThen`, `toSet`, `joining`, `reducing`, `teeing` (12).

Complexity: $O(n)$ time for grouping, $O(n)$ space. **Gotchas:** a null classifier result (an employee with `dept == null`) throws NPE ("element cannot be mapped to a null key"); map it with `e -> Objects.requireNonNullElse(e.dept(), "UNKNOWN")`. `maxBy` returns an `Optional`, hence the `collectingAndThen`. A department with no matching employees will not appear in a `groupingBy` result, but `partitioningBy` always has both keys.

Likely follow-ups:

- *Department with the highest average salary?* `avg.entrySet().stream().max(Map.Entry.comparingByValue()).map(Map.Entry::getKey)`.
- *filtering vs filter before grouping?* `filter` drops whole groups when they have no matches; `Collectors.filtering` keeps the key with an empty list.
- *Concurrent grouping?* `groupingByConcurrent` with a parallel unordered stream.

B21. Sort employees by salary with Comparator

```

import java.util.*;
import java.util.stream.*;

public class SortEmployees {
    record Employee(String name, String dept, double salary) {}

    public static void main(String[] args) {
        List<Employee> emps = List.of(
            new Employee("Asha", "IT", 90_000),
            new Employee("Ravi", "HR", 50_000),
            new Employee("John", "IT", 90_000),
            new Employee("Meera", "HR", 75_000));

        // Ascending by salary
        emps.stream().sorted(Comparator.comparingDouble(Employee::salary))
            .forEach(e -> System.out.print(e.name() + " ")); // Ravi Meera Asha John
        System.out.println();

        // Descending
        emps.stream().sorted(Comparator.comparingDouble(Employee::salary).reversed())
    }

```

```

        .forEach(e -> System.out.print(e.name() + " ")); // Asha John Meera Ravi (stable sort keeps Asha before John)
    System.out.println();

    // Salary desc, then name asc as a tie-breaker
    Comparator<Employee> bySalaryDescThenName = Comparator
        .comparingDouble(Employee::salary).reversed()
        .thenComparing(Employee::name);
    System.out.println(emps.stream().sorted(bySalaryDescThenName).map(Employee::name).toList());
    // [Asha, John, Meera, Ravi]

    // Dept asc, then salary desc (reverse ONLY the second key)
    Comparator<Employee> byDeptThenSalaryDesc = Comparator
        .comparing(Employee::dept)
        .thenComparing(Employee::salary, Comparator.reverseOrder());
    System.out.println(emps.stream().sorted(byDeptThenSalaryDesc).map(Employee::name).toList());
    // [Meera, Ravi, Asha, John]

    // Sort a mutable list in place
    List<Employee> mutable = new ArrayList<>(emps);
    mutable.sort(Comparator.comparing(Employee::name));
    System.out.println(mutable.get(0).name()); // Asha

    // Null-safe sorting when a key can be null
    List<String> withNull = Arrays.asList("b", null, "a");
    withNull.sort(Comparator.nullsLast(Comparator.naturalOrder()));
    System.out.println(withNull); // [a, b, null]

    // Old-school anonymous Comparator (pre-Java 8)
    Comparator<Employee> old = new Comparator<>() {
        @Override public int compare(Employee a, Employee b) { return Double.compare(a.salary(), b.salary()); }
    };
    System.out.println(emps.stream().min(old).get().name()); // Ravi
}
}

```

The reversed() gotcha: `reversed()` reverses **everything built so far**. `comparing(A).thenComparing(B).reversed()` reverses both A and B. To reverse only one key, pass `Comparator.reverseOrder()` as the key comparator, or call `.reversed()` right after that key. Also, with a lambda like `comparing(e -> e.salary()).reversed()`, type inference fails (it sees `Object`). Use a method reference or `Comparator.<Employee>comparingDouble(...)`.

Other details: `List.sort` and `Stream.sorted` are stable (TimSort), $O(n \log n)$. Never write comparators as `a.salary - b.salary`, which can overflow (int) or truncate (double). Use `Double.compare` / `Integer.compare`. Prefer `comparingDouble` / `comparingInt` to avoid boxing.

Likely follow-ups:

- *Comparable vs Comparator?* `Comparable` is the natural order inside the class (`compareTo`); `Comparator` is an external, pluggable order.
- *Top 3 earners?* `sorted(comparingDouble(Employee::salary).reversed()).limit(3)`.
- *Is sorted() stable?* Yes for ordered streams.

B22. Second-highest number in a list (stream)

```

import java.util.*;
import java.util.stream.*;

public class SecondHighest {
    static Optional<Integer> secondHighest(List<Integer> nums) {
        if (nums == null) return Optional.empty();
        return nums.stream()
            .filter(Objects::nonNull)
            .distinct() // important: [9, 9, 5] -> 5, not 9
            .sorted(Comparator.reverseOrder())
            .skip(1)
            .findFirst();
    }

    // Without a full sort: O(n)
    static Optional<Integer> secondHighestLinear(List<Integer> nums) {
        Integer first = null, second = null;
        for (Integer n : nums) {
            if (n == null || n.equals(first) || n.equals(second)) continue;
            if (first == null || n > first) { second = first; first = n; }
            else if (second == null || n > second) { second = n; }
        }
        return Optional.ofNullable(second);
    }
}

```

```

public static void main(String[] args) {
    System.out.println(secondHighest(List.of(10, 40, 30, 40, 20))); // Optional[30]
    System.out.println(secondHighest(List.of(7, 7, 7))); // Optional.empty
    System.out.println(secondHighest(List.of())); // Optional.empty
    System.out.println(secondHighestLinear(List.of(10, 40, 30, 40, 20))); // Optional[30]

    // Variant: second highest NOT distinct (the second element of the sorted list)
    System.out.println(Stream.of(40, 10, 40).sorted(Comparator.reverseOrder()).skip(1).findFirst()); // Optional[40]

    // Variant with a TreeSet
    TreeSet<Integer> ts = new TreeSet<>(List.of(10, 40, 30, 40, 20));
    System.out.println(ts.lower(ts.last())); // 30

    // Nth highest generalised
    int n = 3;
    System.out.println(Stream.of(10, 40, 30, 40, 20).distinct()
        .sorted(Comparator.reverseOrder()).skip(n - 1).findFirst().orElse(null)); // 20
}
}

```

Complexity: the stream with sort is $O(n \log n)$ time and $O(n)$ space. The linear scan is $O(n)$ time and $O(1)$ space. The `TreeSet` is $O(n \log n)$. **Edge cases:** empty list, a single element, all elements equal (no second distinct value, so return empty), nulls, negative numbers. Always clarify with the interviewer whether "second highest" means distinct.

Likely follow-ups:

- *Second highest salary per department?* Group by dept, then use `collectingAndThen(toList(), l -> l.stream().map(salary).distinct().sorted(reverseOrder()).skip(1).findFirst())`.
- *Kth largest efficiently?* A min-heap of size k (`PriorityQueue`) gives $O(n \log k)$; Quickselect gives average $O(n)$.
- *SQL equivalent?* `SELECT MAX(sal) FROM emp WHERE sal < (SELECT MAX(sal) FROM emp)`, or `DENSE_RANK()`.

B23. Method taking `int[]` returning the second-largest DISTINCT element

Requirements: single pass $O(n)$, handle duplicates (`[5, 5, 3]` gives 3), handle "no answer" (empty array, one element, all equal), and handle `Integer.MIN_VALUE` as a legitimate value.

```

import java.util.*;
import java.util.stream.*;

public class SecondLargestDistinct {

    /** Single pass,  $O(n)$  time,  $O(1)$  space. Returns OptionalInt.empty() when no answer exists. */
    public static OptionalInt secondLargest(int[] arr) {
        if (arr == null || arr.length < 2) return OptionalInt.empty();
        // long sentinels so Integer.MIN_VALUE can still be a real answer
        long first = Long.MIN_VALUE, second = Long.MIN_VALUE;
        for (int x : arr) {
            if (x > first) { // new maximum: the old max becomes second
                second = first;
                first = x;
            } else if (x < first && x > second) { // strictly less than max, so duplicates of max are skipped
                second = x;
            }
        }
        return second == Long.MIN_VALUE ? OptionalInt.empty() : OptionalInt.of((int) second);
    }

    /** Stream version:  $O(n \log n)$  because of sorting. */
    public static OptionalInt secondLargestStream(int[] arr) {
        if (arr == null) return OptionalInt.empty();
        return Arrays.stream(arr)
            .boxed()
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(1)
            .mapToInt(Integer::intValue)
            .findFirst();
    }

    /** Alternative contract: throw when there is no answer. */
    public static int secondLargestOrThrow(int[] arr) {
        return secondLargest(arr).orElseThrow(
            () -> new IllegalArgumentException("Array must contain at least two distinct values"));
    }

    public static void main(String[] args) {

```

```

int[][] tests = {
    {12, 35, 1, 10, 34, 1}, // 34
    {10, 10, 10},           // none
    {5},                     // none
    {},                     // none
    {-1, -5, -3},           // -3
    {Integer.MIN_VALUE, 0}, // MIN_VALUE
    {7, 7, 3, 3},           // 3
    {1, 2},                 // 1
};
for (int[] t : tests) {
    System.out.printf("%-30s loop=%s stream=%s\n",
        Arrays.toString(t), secondLargest(t), secondLargestStream(t));
}
System.out.println(secondLargest(null));
try { secondLargestOrThrow(new int[]{4, 4}); }
catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
}
}

```

Output:

```

[12, 35, 1, 10, 34, 1]    loop=OptionalInt[34] stream=OptionalInt[34]
[10, 10, 10]              loop=OptionalInt.empty stream=OptionalInt.empty
[5]                      loop=OptionalInt.empty stream=OptionalInt.empty
[]                       loop=OptionalInt.empty stream=OptionalInt.empty
[-1, -5, -3]             loop=OptionalInt[-3] stream=OptionalInt[-3]
[-2147483648, 0]         loop=OptionalInt[-2147483648] stream=OptionalInt[-2147483648]
[7, 7, 3, 3]             loop=OptionalInt[3] stream=OptionalInt[3]
[1, 2]                   loop=OptionalInt[1] stream=OptionalInt[1]
OptionalInt.empty
Array must contain at least two distinct values

```

Dry run on {12, 35, 1, 10, 34, 1}: 12 gives (12, -inf); 35 gives (35, 12); 1 changes nothing; 10 changes nothing; 34 gives (35, 34); 1 changes nothing. Result 34.

Why long sentinels? With `int` sentinels set to `Integer.MIN_VALUE`, the input `{MIN_VALUE, 0}` would give a second equal to the sentinel, and the method would wrongly report "no answer". Alternatives: an `Integer second = null`, or a `boolean found` flag.

Ways to signal "no answer": `OptionalInt` (best: explicit, no boxing), throw `IllegalArgumentException` / `NoSuchElementException`, or return a sentinel such as `-1` (only if the domain is non-negative, and say so explicitly).

Complexity: loop $O(n)$ time and $O(1)$ space; stream $O(n \log n)$ time and $O(n)$ space. A partly better stream alternative:

`IntStream.of(arr).distinct().boxed().sorted(...)`. Or a `TreeSet` limited to size 2.

Likely follow-ups:

- What if the duplicates should count ([5, 5, 3] gives 5)? Change `x < first && x > second` to `x > second` together with the `x > first` branch handling `>=`.
- Generalise to k ? A min-heap or `TreeSet` of size k gives $O(n \log k)$.
- Why not just sort? Sorting is $O(n \log n)$ and mutates the array (or needs a copy).

B24. Flatten a list of lists with flatMap

```

import java.util.*;
import java.util.stream.*;

public class Flatten {
    static <T> List<T> flatten(List<List<T>> nested) {
        if (nested == null) return List.of();
        return nested.stream()
            .filter(Objects::nonNull) // skip null inner lists
            .flatMap(List::stream)
            .toList();
    }

    public static void main(String[] args) {
        List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3), List.of(), List.of(4, 5, 6));
        System.out.println(flatten(nested)); // [1, 2, 3, 4, 5, 6]

        // With null inner lists (Arrays.asList allows null)
        System.out.println(flatten(Arrays.asList(List.of(1), null, List.of(2)))); // [1, 2]

        // Loop alternative
        List<Integer> out = new ArrayList<>();
    }
}

```

```

        nested.forEach(out::addAll);
        System.out.println(out); // [1, 2, 3, 4, 5, 6]

        // Three levels: flatMap twice
        List<List<List<String>>> deep = List.of(List.of(List.of("a", "b"), List.of("c")), List.of(List.of("d")));
        System.out.println(deep.stream().flatMap(List::stream).flatMap(List::stream).toList()); // [a, b, c, d]

        // 2D array to 1D
        int[][] grid = {{1, 2}, {3, 4}};
        System.out.println(Arrays.toString(Arrays.stream(grid).flatMapToInt(Arrays::stream).toArray())); // [1, 2, 3, 4]

        // Flatten and sum
        System.out.println(nested.stream().flatMap(List::stream).mapToInt(Integer::intValue).sum()); // 21
    }
}

```

Complexity: $O(N)$ total elements; $O(N)$ for the output list. **Edge cases:** null outer list, null inner lists (`List::stream` on null throws NPE), empty inner lists (they simply contribute nothing), null elements inside (they pass through).

Likely follow-ups:

- *Arbitrary-depth nesting* (`List<Object>`)? A recursive function: `o instanceof List<?> l ? l.stream().flatMap(this::flat) : Stream.of(o)`.
- *Flatten a `Map<String, List<String>>` into its values?* `map.values().stream().flatMap(List::stream)`.
- *`flatMap` vs `mapMulti`?* `mapMulti` avoids creating a stream per element; it is useful when each element yields only a few values.

B25. Count by category with groupingBy + counting

```

import java.util.*;
import java.util.stream.*;

public class CountByCategory {
    enum Category { ELECTRONICS, BOOKS, GROCERY }
    record Product(String name, Category category, double price) {}

    public static void main(String[] args) {
        List<Product> products = List.of(
            new Product("Laptop", Category.ELECTRONICS, 70000),
            new Product("Phone", Category.ELECTRONICS, 30000),
            new Product("Novel", Category.BOOKS, 400),
            new Product("Rice", Category.GROCERY, 90),
            new Product("Dal", Category.GROCERY, 120),
            new Product("Atlas", Category.BOOKS, 900));

        Map<Category, Long> counts = products.stream()
            .collect(Collectors.groupingBy(Product::category, Collectors.counting()));
        System.out.println(counts);

        // EnumMap: ordered by enum declaration, compact and fast
        Map<Category, Long> enumCounts = products.stream()
            .collect(Collectors.groupingBy(Product::category,
                () -> new EnumMap<>(Category.class), Collectors.counting()));
        System.out.println(enumCounts); // {ELECTRONICS=2, BOOKS=2, GROCERY=2}

        // Integer counts
        Map<Category, Integer> intCounts = products.stream()
            .collect(Collectors.groupingBy(Product::category, TreeMap::new, Collectors.summingInt(p -> 1)));
        System.out.println(intCounts); // {ELECTRONICS=2, BOOKS=2, GROCERY=2}

        // Count only products above 500 per category, keeping categories with zero
        Map<Category, Long> expensive = products.stream()
            .collect(Collectors.groupingBy(Product::category, () -> new EnumMap<>(Category.class),
                Collectors.filtering(p -> p.price() > 500, Collectors.counting())));
        System.out.println(expensive); // {ELECTRONICS=2, BOOKS=1, GROCERY=0}

        // Loop alternative
        Map<Category, Integer> m = new EnumMap<>(Category.class);
        for (Product p : products) m.merge(p.category(), 1, Integer::sum);
        System.out.println(m);

        // Most common category
        System.out.println(counts.entrySet().stream().max(Map.Entry.comparingByValue()).map(Map.Entry::getKey).orElseThrow());
    }
}

```

Complexity: $O(n)$ time, $O(k)$ space. **Gotchas:** `counting()` returns `Long`. The default map is a `HashMap`; for an enum key prefer `EnumMap`. Categories with no products will not appear at all; pre-fill from `Category.values()` if needed. A null category throws NPE.

Likely follow-ups:

- *Sum of prices per category?* `Collectors.summingDouble(Product::price)`.
- *Count per category and sub-category?* Nested `groupingBy(a, groupingBy(b, counting()))`.
- *Count and sum in one pass?* `Collectors.teeing(counting(), summingDouble(..), Result::new)`, or `summarizingDouble`.

B26. Sort a map by values (into a LinkedHashMap)

A `HashMap` has no order and a `TreeMap` sorts only by key, so to "sort by value" you stream the entries, sort them, and collect into a `LinkedHashMap`, which keeps insertion order.

```
import java.util.*;
import java.util.stream.*;

public class SortMapByValue {
    public static void main(String[] args) {
        Map<String, Integer> scores = new HashMap<>(Map.of("Asha", 82, "Ravi", 95, "John", 70, "Meera", 95));

        // Ascending by value
        LinkedHashMap<String, Integer> asc = scores.entrySet().stream()
            .sorted(Map.Entry.comparingByValue())
            .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));
        System.out.println(asc); // {John=70, Asha=82, Meera=95, Ravi=95} (tie order depends on HashMap iteration)

        // Descending by value, ties broken by key ascending (deterministic)
        LinkedHashMap<String, Integer> desc = scores.entrySet().stream()
            .sorted(Map.Entry.<String, Integer>comparingByValue(Comparator.reverseOrder())
                .thenComparing(Map.Entry.comparingByKey()))
            .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));
        System.out.println(desc); // {Meera=95, Ravi=95, Asha=82, John=70}

        // Top 2
        System.out.println(desc.entrySet().stream().limit(2).map(Map.Entry::getKey).toList()); // [Meera, Ravi]

        // Pre-Java 8 approach
        List<Map.Entry<String, Integer>> entries = new ArrayList<>(scores.entrySet());
        entries.sort(Map.Entry.comparingByValue());
        Map<String, Integer> manual = new LinkedHashMap<>();
        for (Map.Entry<String, Integer> e : entries) manual.put(e.getKey(), e.getValue());
        System.out.println(manual.values()); // [70, 82, 95, 95]
    }
}
```

Why the merge function `(a, b) -> a`? The 4-argument `toMap` is the only overload that takes a map supplier, so you must supply a merge function even though keys from a map are already unique.

Complexity: $O(n \log n)$ time, $O(n)$ space. **Gotchas:**

- Collecting into a plain `toMap` (a `HashMap`) throws the sorting away. This is the most common bug.
- `Map.Entry.<String, Integer>comparingByValue(..)` needs explicit type witnesses when it is chained, otherwise inference fails.
- A `TreeMap` with a value-based comparator is a trap: it breaks `get()` and silently drops entries with equal values.
- Null values throw NPE in `comparingByValue`.

Likely follow-ups:

- *Sort by key?* `new TreeMap<>(map)` or `comparingByKey()`.
- *Word-frequency top N?* `groupingBy(w -> w, counting())`, then sort the entries by value descending, then `limit(N)`.
- *Can you keep a map always sorted by value?* Not with standard maps. Use a `TreeMap` plus a reverse index, or sort on read.

B27. Filter by multiple conditions (Predicate composition)

```
import java.util.*;
import java.util.function.Predicate;
import java.util.stream.*;

public class MultiFilter {
    record Employee(String name, String dept, double salary, int age, boolean active) {}
}
```

```

public static void main(String[] args) {
    List<Employee> emps = List.of(
        new Employee("Asha", "IT", 90_000, 30, true),
        new Employee("Ravi", "HR", 50_000, 45, true),
        new Employee("John", "IT", 40_000, 25, false),
        new Employee("Meera", "IT", 120_000, 38, true));

    // 1. Inline &&
    System.out.println(emps.stream()
        .filter(e -> e.dept().equals("IT") && e.salary() > 50_000 && e.active())
        .map(Employee::name).toList()); // [Asha, Meera]

    // 2. Chained filters (same result, each one reads clearly)
    System.out.println(emps.stream()
        .filter(e -> "IT".equals(e.dept()))
        .filter(e -> e.salary() > 50_000)
        .filter(Employee::active)
        .map(Employee::name).toList()); // [Asha, Meera]

    // 3. Named, reusable predicates composed with and/or/negate
    Predicate<Employee> inIT = e -> "IT".equals(e.dept());
    Predicate<Employee> wellPaid = e -> e.salary() > 50_000;
    Predicate<Employee> senior = e -> e.age() >= 40;
    Predicate<Employee> active = Employee::active;

    Predicate<Employee> rule = inIT.and(wellPaid).and(active).or(senior);
    System.out.println(emps.stream().filter(rule).map(Employee::name).toList()); // [Asha, Ravi, Meera]
    System.out.println(emps.stream().filter(active.negate()).map(Employee::name).toList()); // [John]
    System.out.println(emps.stream().filter(Predicate.not(Employee::active)).count()); // 1

    // 4. Dynamic list of conditions (e.g. search filters from a UI)
    List<Predicate<Employee>> criteria = List.of(inIT, active);
    Predicate<Employee> all = criteria.stream().reduce(x -> true, Predicate::and);
    Predicate<Employee> any = criteria.stream().reduce(x -> false, Predicate::or);
    System.out.println(emps.stream().filter(all).map(Employee::name).toList()); // [Asha, Meera]
    System.out.println(emps.stream().filter(any).count()); // 4
}

```

Complexity: $O(n \times c)$, where c is the number of conditions. `and` / `or` short-circuit, so put the **cheapest or most selective** condition first. **Gotchas:** operator precedence with composition: `a.and(b).or(c)` means `(a && b) || c`. Use `"IT".equals(e.dept())` for null safety. Multiple `filter` calls are not slower in any meaningful way; they are fused into a single pass.

Likely follow-ups:

- Is `filter(a).filter(b)` two passes? No. Streams fuse the stages, and each element goes through both filters in one pass.
- How does Spring Data do the same thing? `Specification<T>` with `and` / `or`, or `Example/QueryDSL BooleanBuilder`, which pushes the filtering into SQL.
- Identity for `reduce(Predicate::and)`? `x -> true` (for `or` it is `x -> false`).

B28. partitioningBy even/odd

```

import java.util.*;
import java.util.stream.*;

public class PartitionEvenOdd {
    public static void main(String[] args) {
        List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7);

        Map<Boolean, List<Integer>> parts = nums.stream()
            .collect(Collectors.partitioningBy(n -> n % 2 == 0));
        System.out.println(parts); // {false=[1, 3, 5, 7], true=[2, 4, 6]}
        System.out.println("Even: " + parts.get(true) + ", Odd: " + parts.get(false));

        // With a downstream collector
        Map<Boolean, Long> counts = nums.stream()
            .collect(Collectors.partitioningBy(n -> n % 2 == 0, Collectors.counting()));
        System.out.println(counts); // {false=4, true=3}

        Map<Boolean, Integer> sums = nums.stream()
            .collect(Collectors.partitioningBy(n -> n % 2 == 0, Collectors.summingInt(Integer::intValue)));
        System.out.println(sums); // {false=16, true=12}

        // Both keys are ALWAYS present, even when empty
        System.out.println(Stream.of(1, 3).collect(Collectors.partitioningBy(n -> n % 2 == 0))); // {false=[1, 3], true=[]}
        System.out.println(Stream.of(1, 3).collect(Collectors.groupingBy(n -> n % 2 == 0))); // {false=[1, 3]}
    }
}

```

```

// Loop alternative
List<Integer> even = new ArrayList<>(), odd = new ArrayList<>();
for (int n : nums) (n % 2 == 0 ? even : odd).add(n);
System.out.println(even + " " + odd);    // [2, 4, 6] [1, 3, 5, 7]
}
}

```

partitioningBy vs groupingBy(boolean): `partitioningBy` takes a `Predicate`, returns a special two-entry map that **always** has both `true` and `false` keys, and is slightly more efficient. `groupingBy` only creates keys that occur.

Complexity: $O(n)$ time, $O(n)$ space. **Edge cases:** an empty list gives `{false=[], true=[]}`; a null element throws NPE on unboxing.

Likely follow-ups:

- *Partition into more than two groups?* Use `groupingBy` with a classifier returning an enum or string.
- *Passed and failed students' names?* `partitioningBy(s -> s.marks() >= 40, mapping(Student::name, toList()))`.
- *Is the returned map mutable?* It is not guaranteed; treat it as read-only.

B29. Duplicate keys in Collectors.toMap()

```

import java.util.*;
import java.util.function.Function;
import java.util.stream.*;

public class ToMapDuplicates {
    record Employee(int id, String name, String dept, double salary) {}

    public static void main(String[] args) {
        List<Employee> emps = List.of(
            new Employee(1, "Asha", "IT", 90_000),
            new Employee(2, "Ravi", "HR", 50_000),
            new Employee(3, "John", "IT", 70_000));

        // Problem: dept is not unique
        try {
            emps.stream().collect(Collectors.toMap(Employee::dept, Employee::name));
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage()); // Duplicate key IT (attempted merging values Asha and John)
        }

        // Merge function strategies
        Map<String, String> keepFirst = emps.stream()
            .collect(Collectors.toMap(Employee::dept, Employee::name, (a, b) -> a));
        Map<String, String> keepLast = emps.stream()
            .collect(Collectors.toMap(Employee::dept, Employee::name, (a, b) -> b));
        Map<String, String> concat = emps.stream()
            .collect(Collectors.toMap(Employee::dept, Employee::name, (a, b) -> a + "," + b));
        Map<String, Double> total = emps.stream()
            .collect(Collectors.toMap(Employee::dept, Employee::salary, Double::sum));
        Map<String, Employee> highestPaid = emps.stream()
            .collect(Collectors.toMap(Employee::dept, Function.identity(),
                (a, b) -> a.salary() >= b.salary() ? a : b));
        System.out.println(keepFirst + " " + keepLast + " " + concat + " " + total);
        System.out.println(highestPaid.get("IT").name()); // Asha

        // Map supplier (4-arg): choose the implementation
        TreeMap<String, String> sorted = emps.stream()
            .collect(Collectors.toMap(Employee::dept, Employee::name, (a, b) -> a, TreeMap::new));
        System.out.println(sorted); // {HR=Ravi, IT=Asha}

        // Unique key: id -> employee
        Map<Integer, Employee> byId = emps.stream().collect(Collectors.toMap(Employee::id, Function.identity()));
        System.out.println(byId.keySet()); // [1, 2, 3]

        // Null values give NPE even without duplicates
        try {
            Stream.of("a", "b").collect(Collectors.toMap(s -> s, s -> (String) null));
        } catch (NullPointerException e) {
            System.out.println("NPE: toMap rejects null values");
        }
    }
}

```

Overloads:

1. `toMap(keyMapper, valueMapper)` throws `IllegalStateException` on a duplicate key.
2. `toMap(keyMapper, valueMapper, mergeFunction)` resolves collisions with `BinaryOperator<V>`.
3. `toMap(keyMapper, valueMapper, mergeFunction, mapSupplier)` also chooses the map type (`LinkedHashMap`, `TreeMap`, `EnumMap` ...).

Also: `toUnmodifiableMap` (10), and `toConcurrentMap` for parallel streams.

Internals: it uses `Map.merge` (or `putIfAbsent` plus a check in newer JDKs). That is why null values give an NPE. **When to use `groupingBy` instead:** if you actually need *all* values per key, `groupingBy(Employee::dept, mapping(Employee::name, toList()))` is clearer than concatenating in a merge function.

Likely follow-ups:

- *Why did Java choose to throw instead of overwriting like `put`?* Silently losing data hides bugs, so fail-fast is the default.
- *How do you log duplicates?* `(a, b) -> { log.warn("dup {}", b); return a; }`.
- *Is the result thread-safe?* No. Use `toConcurrentMap` or wrap the result.

B30. Caching with `ConcurrentHashMap.computeIfAbsent` (and the recursive-update pitfall)

`computeIfAbsent(key, mappingFunction)` atomically does "if absent, compute and insert, then return the value". In `ConcurrentHashMap` (CHM) the function runs **at most once per key**, even under contention, because the bin is locked during computation. That makes it a one-line, thread-safe memoising cache.

```
import java.util.*;
import java.util.concurrent.*;
import java.util.function.Function;

public class ComputeIfAbsentCache {

    // Simple thread-safe memoiser
    static <K, V> Function<K, V> memoize(Function<K, V> fn) {
        Map<K, V> cache = new ConcurrentHashMap<>();
        return key -> cache.computeIfAbsent(key, fn);
    }

    static String slowLookup(String id) {
        System.out.println(" loading " + id + " on " + Thread.currentThread().getName());
        try { Thread.sleep(200); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        return "User-" + id;
    }

    // ---- The recursive-update pitfall ----
    static final Map<Integer, Long> FIB = new ConcurrentHashMap<>();

    static long fibBroken(int n) {
        if (n <= 1) return n;
        // Modifies the SAME map from inside the mapping function
        return FIB.computeIfAbsent(n, k -> fibBroken(k - 1) + fibBroken(k - 2));
    }

    // Fix: compute outside, then insert
    static final Map<Integer, Long> FIB_OK = new ConcurrentHashMap<>();
    static long fibSafe(int n) {
        if (n <= 1) return n;
        Long cached = FIB_OK.get(n);
        if (cached != null) return cached;
        long v = fibSafe(n - 1) + fibSafe(n - 2);
        FIB_OK.putIfAbsent(n, v);
        return v;
    }

    public static void main(String[] args) throws Exception {
        Function<String, String> cachedLookup = memoize(ComputeIfAbsentCache::slowLookup);
        ExecutorService pool = Executors.newFixedThreadPool(4);
        List<Future<String>> fs = new ArrayList<>();
        for (int i = 0; i < 4; i++) fs.add(pool.submit(() -> cachedLookup.apply("42")));
        for (Future<String> f : fs) System.out.println(f.get()); // "loading 42" printed ONCE, then User-42 x4
        pool.shutdown();

        try {
            System.out.println(fibBroken(20));
        } catch (IllegalStateException e) {
            System.out.println("Pitfall: " + e.getMessage()); // Recursive update
        }
        System.out.println(fibSafe(50)); // 12586269025
    }
}
```

```
}  
}
```

The recursive-update pitfall:

- The CHM Javadoc says: *"The mapping function must not attempt to update any other mappings of this map."*
- Inside `computeIfAbsent`, CHM puts a `ReservationNode` in the bin and holds its lock. If the function calls `computeIfAbsent` on the **same map** for a key that hashes to the **same bin** (or triggers a resize), it detects the re-entry.
- **Java 9+:** throws `IllegalStateException: Recursive update`. With small maps nearly every recursive call collides with something, so it fails reliably.
- **Java 8:** could **livelock / spin forever** (JDK-8062841), which is a nasty production hang.
- `HashMap.computeIfAbsent` with recursion throws `ConcurrentModificationException` (Java 9+) or silently corrupts the map.
- **Fixes:** compute outside the map and then `putIfAbsent` (as shown); or use a different structure (an array for DP); or use Caffeine's `LoadingCache` (still no recursion).

Other gotchas:

- Keep the mapping function **short and non-blocking**. It holds a bin lock, so a slow remote call blocks other writers to that bin (and readers of that key wait for it).
- A function returning `null` means "do not insert", and the next caller recomputes.
- If the function throws, nothing is inserted and the exception propagates.
- Java 8's CHM `computeIfAbsent` locked even when the key was present (JDK-8161372). The common workaround is `V v = map.get(k); return v != null ? v : map.computeIfAbsent(k, f);`. This was fixed in Java 9.
- CHM is an **unbounded** cache, with no eviction or TTL. In production use **Caffeine** (`Caffeine.newBuilder().maximumSize(10_000).expireAfterWrite(10, MINUTES).build(loader)`) or Spring's `@Cacheable`.
- `computeIfAbsent(k, x -> new ArrayList<>()).add(v)` is the idiomatic multimap, but the `ArrayList` itself is not thread-safe. Use `CopyOnWriteArrayList` or `ConcurrentHashMap.newKeySet()`.

Likely follow-ups:

- `putIfAbsent` vs `computeIfAbsent`? `putIfAbsent` needs the value already computed (eager); `computeIfAbsent` computes lazily and only once.
- *Is `get` blocked during computation?* `get` on a different key is not blocked; `get` on the same key returns null until the computation finishes (it does not wait). Only `computeIfAbsent` callers for that key wait.
- *Why not `synchronized` around a `HashMap`?* A global lock serialises everything; CHM locks per bin.

B31. Custom collector joining strings in uppercase

A `Collector<T, A, R>` has 4 functions plus characteristics:

Component	Type	Role	Here
<code>supplier()</code>	<code>Supplier<A></code>	Create a new mutable container	<code>new StringJoiner(", ")</code>
<code>accumulator()</code>	<code>BiConsumer<A,T></code>	Add one element	<code>sj.add(s.toUpperCase())</code>
<code>combiner()</code>	<code>BinaryOperator<A></code>	Merge two containers (parallel)	<code>StringJoiner::merge</code>
<code>finisher()</code>	<code>Function<A,R></code>	Final transformation	<code>StringJoiner::toString</code>
<code>characteristics()</code>	<code>Set<Characteristics></code>	<code>IDENTITY_FINISH</code> , <code>CONCURRENT</code> , <code>UNORDERED</code>	none (finisher is not identity, order matters)

```
import java.util.*;  
import java.util.function.*;  
import java.util.stream.*;  
  
public class UpperJoinCollector {  
  
    // Approach 1: implement the Collector interface  
    static class UpperCaseJoining implements Collector<String, StringJoiner, String> {  
        private final String delimiter;  
        UpperCaseJoining(String delimiter) { this.delimiter = delimiter; }  
  
        @Override public Supplier<StringJoiner> supplier() { return () -> new StringJoiner(delimiter); }  
        @Override public BiConsumer<StringJoiner, String> accumulator() {  
            return (sj, s) -> { if (s != null) sj.add(s.toUpperCase(Locale.ROOT)); };  
        }  
        @Override public BinaryOperator<StringJoiner> combiner() { return StringJoiner::merge; }  
        @Override public Function<StringJoiner, String> finisher() { return StringJoiner::toString; }  
        @Override public Set<Characteristics> characteristics() { return Collections.emptySet(); }  
    }  
}
```

```

}

// Approach 2: Collector.of factory
static Collector<String, StringJoiner, String> upperJoining(String delim, String prefix, String suffix) {
    return Collector.of(
        () -> new StringJoiner(delim, prefix, suffix),
        (sj, s) -> sj.add(s.toUpperCase(Locale.ROOT)),
        StringJoiner::merge,
        StringJoiner::toString);
}

// Approach 3: compose existing collectors (simplest in real code)
static Collector<String, ?, String> upperJoiningSimple(String delim) {
    return Collectors.mapping(s -> s.toUpperCase(Locale.ROOT), Collectors.joining(delim));
}

public static void main(String[] args) {
    List<String> words = List.of("java", "spring", "kafka", "docker");
    System.out.println(words.stream().collect(new UpperCaseJoining(", "))); // JAVA, SPRING, KAFKA, DOCKER
    System.out.println(words.stream().collect(upperJoining("|", "[", "]"))); // [JAVA|SPRING|KAFKA|DOCKER]
    System.out.println(words.stream().collect(upperJoiningSimple("-"))); // JAVA-SPRING-KAFKA-DOCKER

    // Parallel works because the combiner preserves order
    System.out.println(words.parallelStream().collect(new UpperCaseJoining(", "))); // JAVA, SPRING, KAFKA, DOCKER

    // Usable as a downstream collector too
    record Emp(String name, String dept) {}
    Map<String, String> byDept = Stream.of(new Emp("ana", "IT"), new Emp("raj", "IT"), new Emp("li", "HR"))
        .collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
            Collectors.mapping(Emp::name, upperJoining(", ", "", ""))));
    System.out.println(byDept); // {HR=LI, IT=ANA,RAJ}

    System.out.println "[" + Stream.<String>empty().collect(new UpperCaseJoining(", ") + "]"; // []
}
}

```

How collect uses it (sequential): A `a = supplier.get()`; for each `t`: `accumulator.accept(a, t)`; return `finisher.apply(a)`; . In parallel, each chunk gets its own container from the supplier, and `combiner` merges them **in encounter order** (unless the collector is `UNORDERED`).

Characteristics:

- `IDENTITY_FINISH`: the container is the result, so the finisher is skipped (for example `toList`). It does not apply here, since `StringJoiner` and `String` differ.
- `UNORDERED`: order of results does not matter (`toSet`).
- `CONCURRENT`: one shared container may be accumulated from many threads (requires a thread-safe container).

Complexity: $O(\text{total characters})$.

Likely follow-ups:

- *Why is `StringJoiner::merge` a valid combiner?* `merge(other)` appends the other's content (without its prefix and suffix) and returns `this`, which matches `BinaryOperator`.
- *When would you write a custom collector?* When no combination of `mapping` / `reducing` / `collectingAndThen` / `teeing` expresses the logic, for example top-N, batching into chunks, or custom statistics.
- *What happens if you wrongly mark it `IDENTITY_FINISH`?* The finisher is skipped and an unchecked cast happens, giving a `ClassCastException` at the call site.

B32. Stream a large file line by line

```

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;
import java.util.stream.*;

public class LargeFileStreaming {

    // 1. Files.lines: lazy, needs try-with-resources (it holds an open file handle)
    static long countErrors(Path file) throws IOException {
        try (Stream<String> lines = Files.lines(file, StandardCharsets.UTF_8)) {
            return lines.filter(l -> l.contains("ERROR")).count();
        }
    }

    // 2. BufferedReader.lines: same laziness, useful for InputStreams (classpath, HTTP, gzip)
}

```

```

static List<String> firstNErrors(InputStream in, int n) throws IOException {
    try (BufferedReader br = new BufferedReader(new InputStreamReader(in, StandardCharsets.UTF_8))) {
        return br.lines().filter(l -> l.contains("ERROR")).limit(n).toList(); // stops reading early
    }
}

// 3. Classic loop: full control, easy to handle per-line exceptions and to track line numbers
static Map<String, Long> levelCounts(Path file) throws IOException {
    Map<String, Long> counts = new TreeMap<>();
    try (BufferedReader br = Files.newBufferedReader(file)) {
        String line;
        while ((line = br.readLine()) != null) {
            String level = line.split(" ", 2)[0];
            counts.merge(level, 1L, Long::sum);
        }
    }
    return counts;
}

public static void main(String[] args) throws IOException {
    Path tmp = Files.createTempFile("app", ".log");
    try (BufferedWriter w = Files.newBufferedWriter(tmp)) {
        for (int i = 0; i < 100_000; i++) w.write((i % 10 == 0 ? "ERROR" : "INFO") + " line " + i + "\n");
    }
    System.out.println(countErrors(tmp)); // 10000
    System.out.println(firstNErrors(Files.newInputStream(tmp), 2)); // [ERROR line 0, ERROR line 10]
    System.out.println(levelCounts(tmp)); // {ERROR=10000, INFO=90000}
    Files.delete(tmp);
}

```

Memory characteristics:

- `Files.lines` / `BufferedReader.lines` read **lazily** through a buffer (8 KB by default), so memory is $O(\text{longest line})$ **as long as your terminal operation does not accumulate**. `count()`, `forEach`, filtering into a small result, and `limit` are all fine.
- `Files.readAllLines` / `Files.readString` load the **entire file** into memory, which can cause `OutOfMemoryError` on multi-GB files.
- `sorted()`, `distinct()`, or `collect(toList())` on all lines buffer everything. Avoid them on huge inputs, or use external sorting.

Gotchas:

- **Always close** the `Files.lines` stream (try-with-resources). It wraps an open file descriptor, and leaking it causes "Too many open files". This is one of the few streams you must close.
- Malformed input for the charset throws `UncheckedIOException` (wrapping `MalformedInputException`) **during** iteration, not at open time.
- `Files.lines` parallelises reasonably on JDK 9+ for UTF-8, US-ASCII, and ISO-8859-1 files (it memory-maps and splits on line breaks). `BufferedReader.lines` does not parallelise well.
- For gzip input: `new BufferedReader(new InputStreamReader(new GZIPInputStream(Files.newInputStream(p))))`.
- For huge files with processing that is heavy per line, consider batching plus an executor, Spring Batch, or memory-mapped `FileChannel`.

Likely follow-ups:

- *How do you get line numbers?* Use a `BufferedReader` loop with a counter, or `LineNumberReader`. Streams have no index.
- *Process a 10 GB file faster?* Split by byte ranges and run parallel workers, or use `Files.lines(...).parallel()` on JDK 9+. Benchmark, because disk I/O is usually the bottleneck.
- *Why does `Files.lines` throw `UncheckedIOException`?* Lambdas and stream methods cannot throw checked `IOException`.

B33. Generic method using functional interfaces

```

import java.util.*;
import java.util.function.*;
import java.util.stream.*;

public class GenericFunctional {

    // Filter + transform any list. PECS wildcards make it maximally reusable.
    public static <T, R> List<R> filterAndMap(Collection<? extends T> source,
                                           Predicate<? super T> filter,
                                           Function<? super T, ? extends R> mapper) {
        if (source == null || source.isEmpty()) return List.of();
        List<R> out = new ArrayList<>();
        for (T t : source) if (filter.test(t)) out.add(mapper.apply(t));
        return out;
    }
}

```

```

// Generic fold
public static <T, A> A fold(Iterable<T> items, A initial, BiFunction<A, ? super T, A> step) {
    A acc = initial;
    for (T t : items) acc = step.apply(acc, t);
    return acc;
}

// Retry helper: shows Supplier + exception handling
public static <T> T retry(Supplier<T> action, int attempts, Predicate<Exception> retryOn) {
    RuntimeException last = null;
    for (int i = 1; i <= attempts; i++) {
        try { return action.get(); }
        catch (RuntimeException e) {
            last = e;
            if (!retryOn.test(e)) throw e;
            System.out.println("attempt " + i + " failed: " + e.getMessage());
        }
    }
    throw last;
}

// Grouping helper with a bounded type parameter
public static <T, K extends Comparable<K>> SortedMap<K, List<T>> groupSorted(List<T> items, Function<T, K> key) {
    return items.stream().collect(Collectors.groupingBy(key, TreeMap::new, Collectors.toList()));
}

// Custom functional interface that allows checked exceptions
@FunctionalInterface
interface ThrowingFunction<T, R> { R apply(T t) throws Exception; }

static <T, R> Function<T, R> unchecked(ThrowingFunction<T, R> f) {
    return t -> {
        try { return f.apply(t); }
        catch (Exception e) { throw new RuntimeException(e); }
    };
}

public static void main(String[] args) {
    List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);
    List<String> labels = filterAndMap(nums, n -> n % 2 == 0, n -> "#" + n);
    System.out.println(labels); // [#2, #4, #6]

    // Wildcards in action: Predicate<Object> works for Integers
    Predicate<Object> notNull = Objects::nonNull;
    List<Number> asNumbers = filterAndMap(nums, notNull, n -> n * 1.5);
    System.out.println(asNumbers); // [1.5, 3.0, 4.5, 6.0, 7.5, 9.0]

    System.out.println(fold(nums, 0, Integer::sum)); // 21
    System.out.println(fold(List.of("a", "b"), new StringBuilder(), StringBuilder::append)); // ab

    int[] calls = {0};
    String r = retry(() -> { if (++calls[0] < 3) throw new IllegalStateException("flaky"); return "ok"; },
        5, e -> e instanceof IllegalStateException);
    System.out.println(r); // ok (after 2 failures)

    System.out.println(groupSorted(List.of("kiwi", "fig", "plum", "pear"), String::length)); // {3=[fig], 4=[kiwi, plum, pear]}

    System.out.println(Stream.of("1", "2").map(unchecked(Integer::parseInt)).toList()); // [1, 2]
}

```

Design notes:

- **PECS** ("Producer Extends, Consumer Super"): a `Predicate<? super T>` consumes `T`, so a `Predicate<Object>` can be reused for any type. A `Function<? super T, ? extends R>` consumes `T` and produces `R`.
- Type parameters are declared before the return type: `public static <T, R> ...`.
- Callers rarely need explicit witnesses (`GenericFunctional.<Integer,String>filterAndMap(...)`) because inference handles them.
- This is the **Strategy pattern** without classes. Spring's `JdbcTemplate.query(sql, RowMapper<T>)` and `TransactionTemplate.execute(TransactionCallback<T>)` work the same way.

Complexity: $O(n)$ for `filterAndMap` / `fold`.

Likely follow-ups:

- *Why `? super T` on the Predicate?* It accepts predicates written for supertypes (for example `Predicate<Object>`), which makes the API more flexible.
- *How do you handle checked exceptions in a lambda?* A custom functional interface plus a wrapper (as shown), or libraries such as Vavr's `CheckedFunction`.

- Can a generic method be overloaded only by different functional interfaces? It can, but lambdas then become ambiguous at call sites. Avoid it.

B34. CSV -> filter -> map -> collect pipeline

Task: read an employee CSV, skip the header, parse it into records, skip bad rows (logging them), keep active IT employees earning over 60k, and produce the total and names per department.

```
import java.io.*;
import java.nio.file.*;
import java.util.*;
import java.util.stream.*;

public class CsvPipeline {
    record Employee(int id, String name, String dept, double salary, boolean active) {}

    record ParseResult(Employee employee, String error) {
        static ParseResult ok(Employee e) { return new ParseResult(e, null); }
        static ParseResult fail(String err) { return new ParseResult(null, err); }
        boolean isOk() { return employee != null; }
    }

    static Optional<Employee> parse(String line) {
        try {
            String[] p = line.split(",", -1); // -1 keeps trailing empty fields
            if (p.length != 5) throw new IllegalArgumentException("expected 5 columns, got " + p.length);
            return Optional.of(new Employee(
                Integer.parseInt(p[0].trim()),
                p[1].trim(),
                p[2].trim(),
                Double.parseDouble(p[3].trim()),
                Boolean.parseBoolean(p[4].trim())));
        } catch (RuntimeException e) { // NumberFormatException, IllegalArgumentException
            System.err.println("Skipping bad row [" + line + "]: " + e.getMessage());
            return Optional.empty();
        }
    }

    static ParseResult parseWithResult(String line) {
        return parse(line).map(ParseResult::ok).orElseGet(() -> ParseResult.fail(line));
    }

    public static void main(String[] args) throws IOException {
        Path csv = Files.createTempFile("emps", ".csv");
        Files.writeString(csv, String.join("\n",
            "id,name,dept,salary,active",
            "1,Asha,IT,90000,true",
            "2,Ravi,HR,50000,true",
            "3,John,IT,abc,true", // bad salary
            "4,Meera,IT,120000,false",
            "", // blank line
            "5,Kiran,IT,65000,true",
            "6,Li,Finance,80000")); // missing column

        try (Stream<String> lines = Files.lines(csv)) {
            List<String> names = lines
                .skip(1) // header
                .filter(l -> !l.isBlank())
                .map(CsvPipeline::parse)
                .flatMap(Optional::stream) // drop unparseable rows
                .filter(Employee::active)
                .filter(e -> "IT".equals(e.dept()))
                .filter(e -> e.salary() > 60_000)
                .sorted(Comparator.comparingDouble(Employee::salary).reversed())
                .map(Employee::name)
                .toList();

            System.out.println(names); // [Asha, Kiran]
        }

        // Second pass: aggregate per department, and count errors explicitly
        try (Stream<String> lines = Files.lines(csv)) {
            Map<Boolean, List<ParseResult>> results = lines.skip(1).filter(l -> !l.isBlank())
                .map(CsvPipeline::parseWithResult)
                .collect(Collectors.partitioningBy(ParseResult::isOk));

            Map<String, Double> totalByDept = results.get(true).stream()
                .map(ParseResult::employee)
                .collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
                    Collectors.summingDouble(Employee::salary)));

            System.out.println(totalByDept); // {HR=50000.0, IT=275000.0}
            System.out.println("bad rows: " + results.get(false).size()); // bad rows: 2
        }
    }
}
```

(The "Skipping bad row" messages go to stderr.)

Design points to talk about:

- **Skip the header** with `skip(1)`. Alternatively, read the first line separately to map column names to indexes, which is resilient to column reordering.
- **Error handling inside lambdas**: a lambda cannot throw checked exceptions, and one bad row should not kill the whole job. Return `Optional` / Either / a result object, log, and continue. For strict mode, throw a custom unchecked exception that includes the line number.
- `split(",", -1)` keeps trailing empty columns. A naive `split(",")` drops them and **cannot handle quoted commas**. For real CSV use Apache Commons CSV, OpenCSV, or Jackson CSV.
- **Records** (Java 16) make immutable DTOs a one-liner, with `equals` / `hashCode` / `toString` included.
- Use **try-with-resources** for `Files.lines`.
- The stream reads lazily (one line at a time), but `sorted` buffers the filtered set.

Complexity: $O(n)$ to parse and filter, plus $O(m \log m)$ to sort the m matches.

Likely follow-ups:

- *How do you report the line number of a bad row?* Use a `BufferedReader` loop with a counter, or `IntStream.range` over `readAllLines` for small files.
- *Write results back as CSV?* `Files.write(path, list.stream().map(e -> e.id() + "," + e.name()).toList())`.
- *Large CSV into a DB?* Spring Batch (`FlatFileItemReader`, chunk-oriented processing with a skip policy) or JDBC batch inserts.

B35. Longest string with `max(Comparator.comparingInt(String::length))`

```
import java.util.*;
import java.util.stream.*;

public class LongestString {
    static Optional<String> longest(List<String> words) {
        if (words == null) return Optional.empty();
        return words.stream()
            .filter(Objects::nonNull)
            .max(Comparator.comparingInt(String::length));
    }

    static List<String> allLongest(List<String> words) {
        int maxLen = words.stream().mapToInt(String::length).max().orElse(0);
        return words.stream().filter(w -> w.length() == maxLen).toList();
    }

    public static void main(String[] args) {
        List<String> words = List.of("java", "microservices", "spring", "observability", "go");

        System.out.println(longest(words).orElse("none")); // microservices (13, first of the tie)
        System.out.println(allLongest(words)); // [microservices, observability]
        System.out.println(longest(List.of()).orElse("none")); // none

        // Shortest
        System.out.println(words.stream().min(Comparator.comparingInt(String::length)).get()); // go

        // Longest, with ties broken alphabetically (last alphabetically wins with max)
        System.out.println(words.stream()
            .max(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder())).get()); // observability

        // reduce equivalent
        System.out.println(words.stream().reduce((a, b) -> a.length() >= b.length() ? a : b).get()); // microservices

        // Longest word in a sentence
        String sentence = "Streams make Java collections expressive";
        System.out.println(Arrays.stream(sentence.split("\\s+"))
            .max(Comparator.comparingInt(String::length)).orElse("")); // collections

        // Loop
        String best = null;
        for (String w : words) if (best == null || w.length() > best.length()) best = w;
        System.out.println(best); // microservices
    }
}
```

```
}
}
```

Tie behaviour: `Stream.max` is implemented as `reduce(BinaryOperator.maxBy(cmp))`, and `maxBy` returns `cmp.compare(a, b) >= 0 ? a : b`. So for **equal** lengths it keeps the **first** encountered element (and `min` also keeps the first). Do not rely on this in parallel unless the stream is ordered, which it is for lists.

Complexity: $O(n)$ time, $O(1)$ space (`allLongest` takes two passes, still $O(n)$). **Gotchas:** `max` returns `Optional`, so calling `.get()` on an empty list throws. Use `comparingInt` rather than `comparing(String::length)` to avoid boxing. Null elements would NPE inside `String::length`.

Likely follow-ups:

- *Length of the longest word only?* `mapToInt(String::length).max()` gives an `OptionalInt`.
- *Longest per first letter?* `groupingBy(w -> w.charAt(0), maxBy(comparingInt(String::length)))`.
- *Top 3 longest?* `sorted(comparingInt(String::length).reversed()).limit(3)`.

B36. "this is test" -> "THIS IS TEST" with streams

```
import java.util.*;
import java.util.regex.Pattern;
import java.util.stream.*;

public class UpperWithStreams {
    // Word-wise: split, map, join (normalises runs of whitespace to a single space)
    static String upperWords(String s) {
        if (s == null || s.isBlank()) return s;
        return Arrays.stream(s.trim().split("\\s+"))
            .map(String::toUpperCase)
            .collect(Collectors.joining(" "));
    }

    // Word-wise, Pattern.splitAsStream (avoids the intermediate array)
    private static final Pattern WS = Pattern.compile("\\s+");
    static String upperWords2(String s) {
        return WS.splitAsStream(s).map(String::toUpperCase).collect(Collectors.joining(" "));
    }

    // Character-wise: preserves the exact spacing
    static String upperChars(String s) {
        if (s == null) return null;
        return s.chars()
            .map(Character::toUpperCase)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();
    }

    // Character-wise via mapToObj + joining (simpler to read, more allocation)
    static String upperChars2(String s) {
        return s.chars().mapToObj(c -> String.valueOf((char) Character.toUpperCase(c)))
            .collect(Collectors.joining());
    }

    // Bonus: Title Case ("This Is Test")
    static String titleCase(String s) {
        return Arrays.stream(s.trim().split("\\s+"))
            .map(w -> Character.toUpperCase(w.charAt(0)) + w.substring(1).toLowerCase())
            .collect(Collectors.joining(" "));
    }

    public static void main(String[] args) {
        String in = "this is test";
        System.out.println(upperWords(in));           // THIS IS TEST
        System.out.println(upperWords2(in));          // THIS IS TEST
        System.out.println(upperChars(in));           // THIS IS TEST
        System.out.println(upperChars2(in));          // THIS IS TEST
        System.out.println(titleCase(in));            // This Is Test
        System.out.println(in.toUpperCase());          // THIS IS TEST (the obvious non-stream answer)

        System.out.println "[" + upperWords(in) + "]"; // [THIS IS TEST] (spacing normalised)
        System.out.println "[" + upperChars(in) + "]"; // [ THIS IS TEST ] (spacing kept)

        // Reverse each word as well (a common twist)
        System.out.println(Arrays.stream(in.split(" "))
            .map(w -> new StringBuilder(w).reverse().toString().toUpperCase())
            .collect(Collectors.joining(" "))); // SIHT SI TSET
    }
}
```

```
}
}
```

Complexity: $O(n)$ for all versions. **Talking point:** in production code `s.toUpperCase(Locale.ROOT)` is the right answer. The stream versions exist to show `split / map / joining` and the 3-argument `collect` (supplier, accumulator, combiner) for building a string from an `IntStream`. **Edge cases:** null, empty, or blank input; multiple spaces (word-wise collapses them, char-wise keeps them); leading spaces (`split` on a string with a leading space yields an empty first token, which is why `trim()` comes first).

Likely follow-ups:

- Why `appendCodePoint` and not `append`? `IntStream` elements are ints. `append(int)` would add the digits ("84"), while `appendCodePoint` adds the character.
- Reverse word order? Collect to a list, then `Collections.reverse`, then `String.join(" ", list)`.
- Count the words? `WS.splitAsStream(s.trim()).count()`.

B37. "Highest substring"

This phrase is ambiguous, so clarify it with the interviewer. The two usual meanings are:

(a) Lexicographically largest substring

Key insight: the lexicographically largest substring is always a **suffix**. If substring `s[i..j]` is the best, extending it to `s[i..n-1]` only makes it larger, because a string is smaller than any string it is a proper prefix of. So the answer is the **maximum suffix**.

- Brute force: compare all n suffixes, $O(n^2)$ worst case (for example `"aaaa...a"`).
- **$O(n)$ two-pointer** (LeetCode 1163 "Last Substring in Lexicographical Order"): keep candidate start `i`, challenger `j`, and matched length `k`.
 - If `s[i+k] == s[j+k]`, increment `k`.
 - If `s[i+k] < s[j+k]`, the challenger wins. Every start from `i` to `i+k` is beaten as well, so `i = max(i+k+1, j)`, `j = i+1`, `k = 0`.
 - If `s[i+k] > s[j+k]`, the candidate wins. Every start from `j` to `j+k` is beaten, so `j = j+k+1`, `k = 0`.

Each step advances `i + j + k` by at least 1, so it is linear.

```
import java.util.*;
import java.util.stream.*;

public class HighestSubstring {

    /** (a) Lexicographically largest substring = max suffix.  $O(n)$  time,  $O(1)$  space. */
    static String lastSubstring(String s) {
        if (s == null || s.isEmpty()) return "";
        int n = s.length(), i = 0, j = 1, k = 0;
        while (j + k < n) {
            char a = s.charAt(i + k), b = s.charAt(j + k);
            if (a == b) {
                k++;
            } else if (a < b) { // challenger j is better
                i = Math.max(i + k + 1, j);
                j = i + 1;
                k = 0;
            } else { // candidate i is better
                j = j + k + 1;
                k = 0;
            }
        }
        return s.substring(i);
    }

    /** (a) Stream brute force,  $O(n^2)$ : all suffixes, take the max. */
    static String lastSubstringStream(String s) {
        return IntStream.range(0, s.length()).mapToObj(s::substring)
            .max(Comparator.naturalOrder()).orElse("");
    }

    /** (b) Longest substring without repeating characters. Sliding window,  $O(n)$ . */
    static String longestUniqueSubstring(String s) {
        if (s == null || s.isEmpty()) return "";
        int[] lastSeen = new int[128]; // ASCII; use Map<Character,Integer> for full Unicode
        Arrays.fill(lastSeen, -1);
        int left = 0, bestStart = 0, bestLen = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (lastSeen[c] >= left) left = lastSeen[c] + 1; // jump past the previous occurrence
            lastSeen[c] = right;
        }
        return s.substring(bestStart, bestStart + bestLen);
    }
}
```

```

        if (right - left + 1 > bestLen) { bestLen = right - left + 1; bestStart = left; }
    }
    return s.substring(bestStart, bestStart + bestLen);
}

/** (b) Variant using a HashMap (Unicode-safe for BMP characters). */
static int lengthOfLongestUnique(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int left = 0, best = 0;
    for (int r = 0; r < s.length(); r++) {
        Integer prev = last.put(s.charAt(r), r);
        if (prev != null && prev >= left) left = prev + 1;
        best = Math.max(best, r - left + 1);
    }
    return best;
}

public static void main(String[] args) {
    System.out.println(lastSubstring("abab"));           // bab
    System.out.println(lastSubstring("leetcode"));      // tcode
    System.out.println(lastSubstring("zazzb"));          // zzb
    System.out.println(lastSubstringStream("leetcode")); // tcode

    System.out.println(longestUniqueSubstring("abcabcbb")); // abc
    System.out.println(longestUniqueSubstring("bbbbbb"));  // b
    System.out.println(longestUniqueSubstring("pwwkew"));  // wke
    System.out.println(lengthOfLongestUnique("dvdf"));     // 3 (vdf)
    System.out.println("[ " + longestUniqueSubstring("") + " ]"); // []
}
}

```

Dry run (a) for "abab" : $i=0, j=1, k=0$: 'a' < 'b', so $i = \max(1, 1) = 1, j = 2$. Then compare $s[1]='b'$ with $s[2]='a'$: 'b' > 'a', so $j = 3$. Then $s[1]='b' == s[3]='b'$, so $k = 1$. Now $j + k = 4 = n$, stop. Answer: `s.substring(1)` = "bab".

Dry run (b) for "pwwkew" : window [p], [p,w], then 'w' repeats, so left jumps to 2 giving [w], then [w,k], [w,k,e] (length 3, best), then 'w' repeats, so left = 3, giving [k,e,w] (also length 3, not greater). Answer "wke".

Complexity: (a) $O(n)$ time and $O(1)$ space; the brute force is $O(n^2)$ comparisons plus $O(n^2)$ memory for the substrings. (b) $O(n)$ time, $O(\text{alphabet})$ space.

Why `lastSeen[c] >= left`? A previous occurrence *before* the window should not move `left` backwards. That is the classic bug: in "abba", without the check the final 'a' would move `left` back to 1.

Likely follow-ups:

- *Longest substring with at most K distinct characters?* The same sliding window with a count map; shrink while `map.size() > K`.
- *Longest palindromic substring?* Expand around centres, $O(n^2)$ time and $O(1)$ space (Manacher's algorithm is $O(n)$).
- *Lexicographically smallest substring?* Trivially the smallest single character. The interesting version is the smallest rotation (Booth's algorithm).

B38. First non-repeating character

```

import java.util.*;
import java.util.function.Function;
import java.util.stream.*;

public class FirstNonRepeating {

    // Stream + LinkedHashMap (keeps first-occurrence order)
    static Optional<Character> firstUniqueStream(String s) {
        if (s == null || s.isEmpty()) return Optional.empty();
        return s.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()))
            .entrySet().stream()
            .filter(e -> e.getValue() == 1L)
            .map(Map.Entry::getKey)
            .findFirst();
    }

    // O(n) two passes over a counting array: fastest, O(1) space (fixed 256 / 65536 slots)
    static int firstUniqueIndex(String s) {
        if (s == null) return -1;
        int[] count = new int[Character.MAX_VALUE + 1]; // or 26 for 'a'-'z', 256 for extended ASCII
        for (int i = 0; i < s.length(); i++) count[s.charAt(i)]++;
        for (int i = 0; i < s.length(); i++) if (count[s.charAt(i)] == 1) return i;
        return -1;
    }
}

```

```

}

// Single pass over the string, then a pass over at most 26 letters (very long strings, small alphabet)
static int firstUniqueIndexSinglePass(String s) {
    int[] firstIdx = new int[26];
    Arrays.fill(firstIdx, -1); // -1 = unseen, -2 = repeated
    for (int i = 0; i < s.length(); i++) {
        int c = s.charAt(i) - 'a';
        firstIdx[c] = firstIdx[c] == -1 ? i : -2;
    }
    int best = Integer.MAX_VALUE;
    for (int idx : firstIdx) if (idx >= 0) best = Math.min(best, idx);
    return best == Integer.MAX_VALUE ? -1 : best;
}

// The indexOf/lastIndexOf trick: O(n^2), but a short answer
static Optional<Character> firstUniqueTrick(String s) {
    return s.chars().filter(c -> s.indexOf(c) == s.lastIndexOf(c))
        .mapToObj(c -> (char) c).findFirst();
}

public static void main(String[] args) {
    System.out.println(firstUniqueStream("swiss")); // Optional[w]
    System.out.println(firstUniqueStream("leetcode")); // Optional[l]
    System.out.println(firstUniqueStream("aabb")); // Optional.empty
    System.out.println(firstUniqueIndex("loveleetcode")); // 2 ('v')
    System.out.println(firstUniqueIndexSinglePass("loveleetcode")); // 2
    System.out.println(firstUniqueTrick("aabbccdd")); // Optional[c]
    System.out.println(firstUniqueIndex("")); // -1

    // First REPEATING character (a common twist)
    Set<Character> seen = new HashSet<>();
    System.out.println("abcbad".chars().mapToObj(c -> (char) c).filter(c -> !seen.add(c)).findFirst()); // Optional[b]
}
}

```

Complexity:

Approach	Time	Space
LinkedHashMap stream	O(n)	O(k)
Count array, 2 passes	O(n)	O(1) (fixed alphabet)
firstIdx single pass	O(n + 26)	O(26)
indexOf/lastIndexOf	O(n^2)	O(1)

Gotchas: with a plain `HashMap` you lose order and might return the wrong character. Case sensitivity ("sS") needs normalising. Characters outside the BMP (emoji) need `codePoints()`. Comparing `Long` values with `== 1` works here because `1` is unboxed for comparison, but `Long == Long` would compare references.

Likely follow-ups:

- First unique character in a stream of characters (online)? Keep a count map plus a `LinkedHashSet` /queue of candidates, removing from the front while the front's count is above 1.
- Return the index instead of the char? Use the count-array version.
- Case-insensitive? Lowercase while counting, but return the original character at the found index.

B39. Remove duplicate letters: lexicographically smallest result (LeetCode 316)

Problem: remove duplicate letters so that every letter appears **exactly once**, and the result is the **lexicographically smallest** among all possible results. It must be a subsequence, so the relative order is kept. "bcabc" gives "abc"; "cbacdcbc" gives "acdb".

Why a monotonic stack (greedy)? We build the answer left to right and want small characters as early as possible. When we see character `c`, and the stack top is **greater** than `c` **and that top character appears again later**, we can safely pop it: putting `c` earlier makes the string smaller, and we can re-add the popped character later. If the top does not appear later, we must keep it (it is the last chance to include it). Characters already in the stack are skipped, because the earlier position is already optimal given the greedy choices. This keeps the stack as increasing as the "must keep" constraints allow, which gives the lexicographically smallest result.

```

import java.util.*;

public class RemoveDuplicateLetters {

```

```

public static String removeDuplicateLetters(String s) {
    if (s == null || s.isEmpty()) return "";
    int[] last = new int[26]; // last index of each letter
    for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;

    boolean[] inStack = new boolean[26];
    StringBuilder stack = new StringBuilder(); // StringBuilder used as a stack

    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (inStack[c - 'a']) continue; // already placed optimally
        while (stack.length() > 0) {
            char top = stack.charAt(stack.length() - 1);
            if (top > c && last[top - 'a'] > i) { // bigger AND can be re-added later
                stack.deleteCharAt(stack.length() - 1);
                inStack[top - 'a'] = false;
            } else break;
        }
        stack.append(c);
        inStack[c - 'a'] = true;
    }
    return stack.toString();
}

// Plain dedup (NOT lexicographically smallest), for contrast: keeps first occurrences
static String simpleDedup(String s) {
    return s.chars().distinct().collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append).toString();
}

public static void main(String[] args) {
    System.out.println(removeDuplicateLetters("bcabc")); // abc
    System.out.println(removeDuplicateLetters("cbacdcabc")); // acdb
    System.out.println(removeDuplicateLetters("aaaaa")); // a
    System.out.println(removeDuplicateLetters("abacb")); // abc
    System.out.println(simpleDedup("cbacdcabc")); // cbad (different!)
}

```

Dry run for "cbacdcabc" (last: a=2, b=6, c=7, d=4):

i	char	Action	Stack
0	c	push	c
1	b	top c > b, c appears later (7 > 1), so pop c; push b	b
2	a	top b > a, b appears later (6 > 2), so pop b; push a	a
3	c	a < c, push	ac
4	d	push	acd
5	c	already in stack, skip	acd
6	b	top d > b but d's last index is 4 < 6, so it cannot pop; push b	acdb
7	c	in stack, skip	acdb

Result: **acdb**.

Dry run for "bcabc" (last: a=2, b=3, c=4): b gives [b]; c gives [b,c]; at a, pop c (it appears at 4), pop b (it appears at 3), push a giving [a]; b gives [a,b]; c gives [a,b,c]. Result **abc**.

Complexity: O(n) time, because each character is pushed and popped at most once. O(1) extra space (26-letter arrays; the stack holds at most 26 characters).

Likely follow-ups:

- Same as LeetCode 1081 ("smallest subsequence of distinct characters")? Yes, it is the identical problem.
- Why skip characters already in the stack? Its existing position is already part of the smallest prefix; re-inserting it could only make the result the same or worse.
- Other monotonic stack problems? Next greater element, daily temperatures, largest rectangle in histogram, "remove K digits" (LC 402).

B40. Rotated sorted list: find a target with modified binary search

Search in Rotated Sorted Array (Binary Search)

nums = [4, 5, 6, 7, 0, 1, 2, 3], target = 1

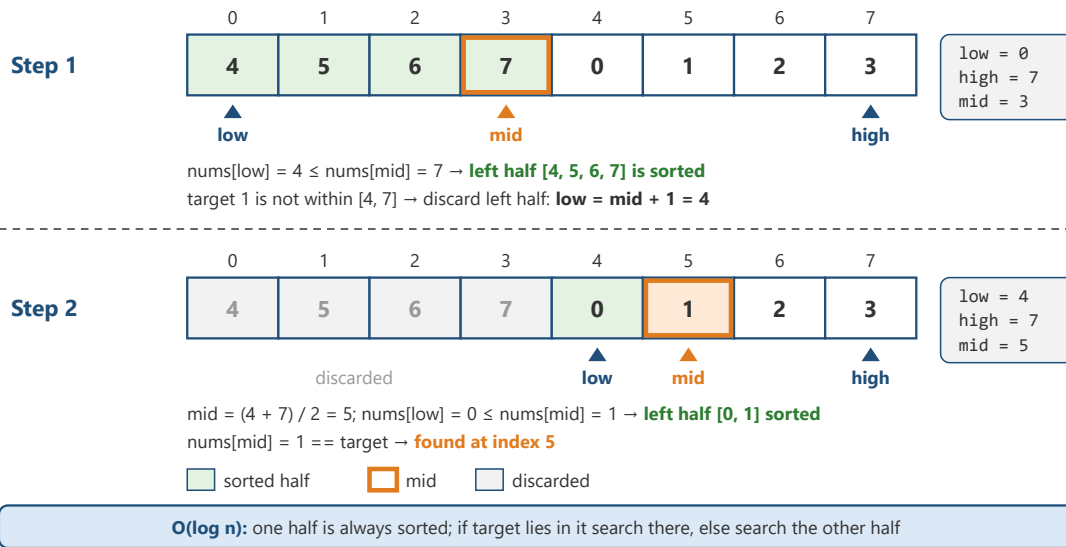


Figure 9: Binary search in a rotated sorted array

Problem: an ascending array rotated at an unknown pivot, for example [4,5,6,7,0,1,2,3]. Find the index of a target in O(log n). No duplicates.

Key insight: at any mid, at least one half is sorted. Check which half is sorted by comparing nums[lo] with nums[mid]. If the target lies inside the sorted half's range, search there; otherwise search the other half.

```
public class RotatedSearch {

    public static int search(int[] nums, int target) {
        if (nums == null || nums.length == 0) return -1;
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2; // avoids int overflow of (lo+hi)
            if (nums[mid] == target) return mid;

            if (nums[lo] <= nums[mid]) { // left half [lo..mid] is sorted
                if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else { // right half [mid..hi] is sorted
                if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        return -1;
    }

    // Bonus: index of the minimum element (= rotation count)
    public static int findPivot(int[] nums) {
        int lo = 0, hi = nums.length - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] > nums[hi]) lo = mid + 1; // the minimum is to the right of mid
            else hi = mid; // the minimum is at mid or to its left
        }
        return lo;
    }

    public static void main(String[] args) {
        int[] nums = {4, 5, 6, 7, 0, 1, 2, 3};
        System.out.println(search(nums, 1)); // 5
        System.out.println(search(nums, 6)); // 2
        System.out.println(search(nums, 3)); // 7
        System.out.println(search(nums, 9)); // -1
        System.out.println(search(new int[]{1}, 1)); // 0
        System.out.println(search(new int[] {}, 1)); // -1
        System.out.println(findPivot(nums)); // 4 (value 0)
    }
}
```

```
}
}
```

Dry run: target = 1 in [4,5,6,7,0,1,2,3] (indexes 0..7)

Step	lo	hi	mid	nums[mid]	Sorted half	Is target in the sorted range?	Move
1	0	7	3	7	left [4..7] (4 <= 7)	4 <= 1 < 7? No	lo = 4
2	4	7	5	1		nums[mid] == target	return 5

Dry run: target = 2 (a longer path): step 1 is the same as above, so lo = 4. Step 2: mid = 5 (value 1). nums[4]=0 <= 1, so the left half [0,1] is sorted; is 0 <= 2 < 1? No, so lo = 6. Step 3: mid = 6 (value 2), found, return 6.

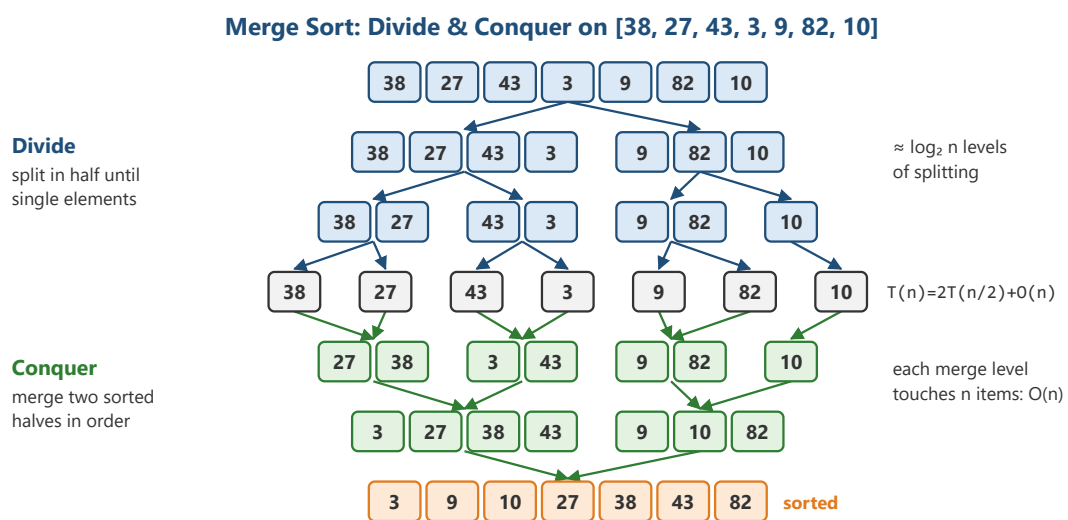
Complexity: O(log n) time, O(1) space. **Gotchas:**

- Use <= in `nums[lo] <= nums[mid]`. When `lo == mid` (a two-element window), the left half is a single element and counts as sorted. Using < breaks cases like [3,1] with target 1.
- Watch the strict and non-strict bounds: `target < nums[mid]`, because mid was already checked.
- With duplicates** (LC 81), `nums[lo] == nums[mid] == nums[hi]` makes the sorted half undecidable. Shrink with `lo++`; `hi--`; , which makes the worst case O(n).

Likely follow-ups:

- Find the rotation count?* It is the index of the minimum (`findPivot`), O(log n).
- Alternative approach?* Find the pivot, then run a normal binary search on the correct half. Still O(log n), with two passes.
- Why `lo + (hi - lo) / 2`?* `lo + hi` can overflow for huge arrays (a famous JDK bug in `Arrays.binarySearch`, fixed in 2006). `(lo + hi) >>> 1` also works.

B41. Merge Sort, Quick Sort, Bubble Sort, Linear Search, Binary Search



Algorithm	Time complexity	Stable?	Space / note
Bubble sort	O(n ²) (best O(n) if no swaps)	Yes	O(1), in-place
Merge sort	O(n log n) in all cases	Yes	O(n) extra space
Quick sort	avg O(n log n), worst O(n ²)	No	O(log n) stack, in-place
Linear search	O(n)	—	works on any array
Binary search	O(log n)	—	sorted array only

Figure 10: Merge sort divide and conquer, with sorting and searching complexities

Explanations

- Bubble Sort:** repeatedly walk the array and swap adjacent elements that are out of order. After pass k, the k largest elements are in place at the end. With an early-exit flag (no swaps in a pass means sorted), the best case is O(n). It is only for teaching or tiny or nearly-sorted inputs.

- **Merge Sort:** divide and conquer. Split in half, recursively sort each half, then **merge** two sorted halves in linear time. Always $O(n \log n)$, **stable**, needs $O(n)$ auxiliary space. Good for linked lists and external (disk) sorting. Java's `Collections.sort` / `Arrays.sort(Object[])` uses **TimSort**, a hybrid of merge sort and insertion sort.
- **Quick Sort:** pick a **pivot**, **partition** so that elements smaller than the pivot go left and larger go right, then recurse on both sides. In place, very cache-friendly, average $O(n \log n)$. The worst case is $O(n^2)$ when the pivot is always the smallest or largest (for example, sorted input with a first or last pivot). Mitigate with a random or median-of-three pivot. Not stable. Java's `Arrays.sort(int[])` uses **Dual-Pivot Quicksort**, safe for primitives because stability does not matter for them.
- **Linear Search:** check each element in turn. $O(n)$, works on unsorted data and any `equals`.
- **Binary Search:** on **sorted** data, compare with the middle and discard half each step. $O(\log n)$. Java provides `Arrays.binarySearch` / `Collections.binarySearch`, which return `-(insertionPoint) - 1` when the key is not found.

```
import java.util.*;

public class SortingAndSearching {

    // ----- Bubble Sort -----
    static void bubbleSort(int[] a) {
        int n = a.length;
        for (int pass = 0; pass < n - 1; pass++) {
            boolean swapped = false;
            for (int j = 0; j < n - 1 - pass; j++) { // the last 'pass' elements are already in place
                if (a[j] > a[j + 1]) {
                    int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                    swapped = true;
                }
            }
            if (!swapped) break; // already sorted: best case  $O(n)$ 
        }
    }

    // ----- Merge Sort -----
    static void mergeSort(int[] a) {
        if (a == null || a.length < 2) return;
        int[] tmp = new int[a.length]; // allocate the buffer once
        mergeSort(a, tmp, 0, a.length - 1);
    }

    private static void mergeSort(int[] a, int[] tmp, int lo, int hi) {
        if (lo >= hi) return;
        int mid = lo + (hi - lo) / 2;
        mergeSort(a, tmp, lo, mid);
        mergeSort(a, tmp, mid + 1, hi);
        if (a[mid] <= a[mid + 1]) return; // optimisation: already in order
        merge(a, tmp, lo, mid, hi);
    }

    private static void merge(int[] a, int[] tmp, int lo, int mid, int hi) {
        System.arraycopy(a, lo, tmp, lo, hi - lo + 1);
        int i = lo, j = mid + 1, k = lo;
        while (i <= mid && j <= hi) {
            a[k++] = (tmp[i] <= tmp[j]) ? tmp[i++] : tmp[j++]; // <= keeps it STABLE
        }
        while (i <= mid) a[k++] = tmp[i++];
        while (j <= hi) a[k++] = tmp[j++]; // usually already in place; kept for clarity
    }

    // ----- Quick Sort (Lomuto partition, random pivot) -----
    private static final Random RND = new Random();

    static void quickSort(int[] a) {
        if (a == null) return;
        quickSort(a, 0, a.length - 1);
    }

    private static void quickSort(int[] a, int lo, int hi) {
        while (lo < hi) {
            int p = partition(a, lo, hi);
            // Recurse into the smaller side and loop on the larger:  $O(\log n)$  stack depth
            if (p - lo < hi - p) { quickSort(a, lo, p - 1); lo = p + 1; }
            else { quickSort(a, p + 1, hi); hi = p - 1; }
        }
    }

    private static int partition(int[] a, int lo, int hi) {
        int r = lo + RND.nextInt(hi - lo + 1); // random pivot avoids  $O(n^2)$  on sorted input
        swap(a, r, hi);
        int pivot = a[hi];
        int i = lo; // a[lo..i-1] < pivot
        for (int j = lo; j < hi; j++) {

```

```

        if (a[j] < pivot) swap(a, i++, j);
    }
    swap(a, i, hi); // the pivot goes to its final position
    return i;
}

private static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }

// ----- Linear Search -----
static int linearSearch(int[] a, int key) {
    for (int i = 0; i < a.length; i++) if (a[i] == key) return i;
    return -1;
}

// ----- Binary Search (iterative) -----
static int binarySearch(int[] a, int key) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = (lo + hi) >> 1; // overflow-safe
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

// Binary Search (recursive)
static int binarySearchRec(int[] a, int key, int lo, int hi) {
    if (lo > hi) return -1;
    int mid = (lo + hi) >> 1;
    if (a[mid] == key) return mid;
    return a[mid] < key ? binarySearchRec(a, key, mid + 1, hi) : binarySearchRec(a, key, lo, mid - 1);
}

// Variant: first occurrence when duplicates exist (lower bound)
static int firstOccurrence(int[] a, int key) {
    int lo = 0, hi = a.length - 1, ans = -1;
    while (lo <= hi) {
        int mid = (lo + hi) >> 1;
        if (a[mid] >= key) { if (a[mid] == key) ans = mid; hi = mid - 1; }
        else lo = mid + 1;
    }
    return ans;
}

public static void main(String[] args) {
    int[] data = {38, 27, 43, 3, 9, 82, 10, 3};

    int[] b = data.clone(); bubbleSort(b);
    int[] m = data.clone(); mergeSort(m);
    int[] q = data.clone(); quickSort(q);
    System.out.println("Bubble: " + Arrays.toString(b)); // [3, 3, 9, 10, 27, 38, 43, 82]
    System.out.println("Merge : " + Arrays.toString(m)); // [3, 3, 9, 10, 27, 38, 43, 82]
    System.out.println("Quick : " + Arrays.toString(q)); // [3, 3, 9, 10, 27, 38, 43, 82]

    System.out.println("Linear search 82 in unsorted: " + linearSearch(data, 82)); // 5
    System.out.println("Binary search 27: " + binarySearch(m, 27)); // 4
    System.out.println("Binary search rec 43: " + binarySearchRec(m, 43, 0, m.length - 1)); // 6
    System.out.println("Binary search 100: " + binarySearch(m, 100)); // -1
    System.out.println("First occurrence of 3: " + firstOccurrence(m, 3)); // 0
    System.out.println("JDK binarySearch 11: " + Arrays.binarySearch(m, 11)); // -5 (insertion point 4)

    // Edge cases
    int[] empty = {}; quickSort(empty); mergeSort(empty); bubbleSort(empty);
    System.out.println(Arrays.toString(empty) + " " + binarySearch(empty, 1)); // [] -1

    // Large sanity check against Arrays.sort
    int[] big = new Random(42).ints(100_000, -1000, 1000).toArray();
    int[] expected = big.clone(); Arrays.sort(expected);
    int[] m2 = big.clone(); mergeSort(m2);
    int[] q2 = big.clone(); quickSort(q2);
    System.out.println(Arrays.equals(expected, m2) && Arrays.equals(expected, q2)); // true
}
}

```

Merge sort trace for [38, 27, 43, 3]: split into [38,27] and [43,3]; these become [27,38] and [3,43]; merging compares 27 vs 3 (take 3), 27 vs 43 (take 27), 38 vs 43 (take 38), then 43 is left. Result [3,27,38,43].

Quick sort partition (Lomuto, pivot 10) on [38,27,43,3,9,82,10]: elements < 10 (3, 9) are swapped to the front, giving [3,9,10,38,27,82,43]. The pivot 10 is at index 2, its final position. Recurse on [3,9] and [38,27,82,43].

Complexity table

Algorithm	Best	Average	Worst	Extra space	Stable	In-place	Notes
Bubble Sort	$O(n)$ (with swap flag)	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Only for tiny or nearly-sorted data
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No	Predictable; linked lists; external sort; TimSort basis
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ stack ($O(n)$ worst if naive)	No	Yes	Fastest in practice; randomise the pivot
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	-	-	Unsorted data, small n
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ iterative / $O(\log n)$ recursive	-	-	Requires sorted, random-access data

What the JDK uses:

- `Arrays.sort(int[]/long[]/double[]...)` : Dual-Pivot Quicksort (Yaroslavskiy), with insertion sort for small ranges. Not stable, but primitives do not need stability.
- `Arrays.sort(Object[])` , `List.sort` , `Collections.sort` : TimSort. Stable, $O(n)$ on already-sorted runs, and needs a consistent comparator, otherwise "Comparison method violates its general contract!".
- `Arrays.parallelSort` : parallel merge sort over the ForkJoin pool for large arrays.

Gotchas: binary search on unsorted data returns garbage rather than throwing. Quicksort with a fixed pivot on sorted input degrades to $O(n^2)$ and can cause a `StackOverflowError` on large n . Merge sort's `<=` (instead of `<`) is what makes it stable.

Likely follow-ups:

- Why does Java use different algorithms for primitives and objects?* Objects need stability (sorting by one key must keep a previous ordering), and TimSort is fast on real-world partially sorted data. Primitives cannot be told apart when equal, so the faster unstable quicksort is fine.
 - Which sort for a linked list?* Merge sort: $O(1)$ extra space with pointer manipulation, and no random access is needed.
 - What makes quicksort's worst case $O(n^2)$ and how do you avoid it?* Consistently bad pivots (already sorted input with a first or last pivot). Use a random or median-of-three pivot, or introsort (fall back to heapsort when recursion gets deep).
 - Other $O(n)$ sorts?* Counting sort, radix sort, and bucket sort, for integer keys in a bounded range. They are non-comparison sorts.
-

C. Spring Boot, Spring Framework, JPA/Hibernate & Testing

C1. What is @SpringBootApplication?

`@SpringBootApplication` is a **composed (meta-) annotation** that you put on the main class. It turns on the three things almost every Boot app needs:

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootApplication           // = @Configuration (marks the class as a config source; tests use it to find the root config)
@EnableAutoConfiguration         // = @AutoConfigurationPackage + @Import(AutoConfigurationImportSelector.class)
@ComponentScan(excludeFilters = {
    @Filter(type = FilterType.CUSTOM, classes = TypeExcludeFilter.class),
    @Filter(type = FilterType.CUSTOM, classes = AutoConfigurationExcludeFilter.class) })
public @interface SpringBootApplication { ... }
```

Meta-annotation	What it does
<code>@SpringBootApplication</code>	A specialised <code>@Configuration</code> . There should be only one; <code>@SpringBootTest</code> and test slices find it by walking up the packages from the test class.
<code>@EnableAutoConfiguration</code>	Imports <code>AutoConfigurationImportSelector</code> , which loads auto-configuration classes based on the classpath, existing beans and properties. <code>@AutoConfigurationPackage</code> records the main class's package (used by JPA entity scanning and Spring Data repository scanning).
<code>@ComponentScan</code>	Scans the main class's package and all sub-packages for <code>@Component</code> stereotypes. The exclude filters stop auto-config classes from being picked up twice through scanning.

Attributes you can set: `exclude`, `excludeName`, `scanBasePackages`, `scanBasePackageClasses`, `proxyBeanMethods`.

How auto-configuration is discovered (internals)

- `SpringApplication.run()` creates the `ApplicationContext` and calls `refresh()`.
- During `invokeBeanFactoryPostProcessors`, `ConfigurationClassPostProcessor` parses the main class and sees `@Import(AutoConfigurationImportSelector.class)`.
- `AutoConfigurationImportSelector` is a `DeferredImportSelector`, so it runs **after** all user `@Configuration` classes have been processed. That ordering is what lets `@ConditionalOnMissingBean` back off when you define your own bean.
- It loads candidate class names with `ImportCandidates.load(AutoConfiguration.class, classLoader)`, which reads every `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` file on the classpath (one fully qualified class name per line).
 - Boot 2.7 introduced this file. **Boot 3.0 dropped support for auto-config entries in `spring.factories`**.
- It removes exclusions (`exclude`, `excludeName`, `spring.autoconfigure.exclude`).
- Fast pre-filtering** with `AutoConfigurationImportFilter` implementations (`OnClassCondition`, `OnBeanCondition`, `OnWebApplicationCondition`). They read precomputed `META-INF/spring-autoconfigure-metadata.properties`, so classes whose required classes are missing are thrown out **without being loaded**.
- The survivors are sorted (`@AutoConfiguration(before/after)`, `@AutoConfigureOrder`) and parsed like normal configuration classes. Each `@Conditional*` on the class and its `@Bean` methods is evaluated.

The common `@Conditional*` annotations

Annotation	Matches when
<code>@ConditionalOnClass</code> / <code>@ConditionalOnMissingClass</code>	a class is / isn't on the classpath
<code>@ConditionalOnBean</code> / <code>@ConditionalOnMissingBean</code>	a bean of the type/name exists / doesn't exist (only reliable in auto-config, because of ordering)
<code>@ConditionalOnProperty(prefix, name, havingValue, matchIfMissing)</code>	a property has a value
<code>@ConditionalOnWebApplication(type = SERVLET/REACTIVE)</code> / <code>@ConditionalOnNotWebApplication</code>	the type of application
<code>@ConditionalOnResource</code>	a resource exists
<code>@ConditionalOnExpression</code>	a SpEL expression is true

Annotation	Matches when
<code>@ConditionalOnSingleCandidate</code>	exactly one candidate bean, or one <code>@Primary</code>
<code>@ConditionalOnJava</code> , <code>@ConditionalOnCloudPlatform</code> , <code>@ConditionalOnThreading</code> (3.2+, e.g. virtual threads)	environment checks

All of these build on Spring Framework's `@Conditional(Condition.class)`. Its `matches(ConditionContext, AnnotatedTypeMetadata)` method is called while bean definitions are being registered.

Writing your own auto-configuration (e.g. a shared Smart City "common-audit" starter)

```
package com.trinity.common.audit.autoconfigure;

import org.springframework.boot.autoconfigure.AutoConfiguration;
import org.springframework.boot.autoconfigure.condition.ConditionalOnClass;
import org.springframework.boot.autoconfigure.condition.ConditionalOnMissingBean;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.kafka.core.KafkaTemplate;

@AutoConfiguration
@ConditionalOnClass(KafkaTemplate.class)
@ConditionalOnProperty(prefix = "trinity.audit", name = "enabled", havingValue = "true", matchIfMissing = true)
@EnableConfigurationProperties(AuditProperties.class)
public class AuditAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean // the user's own AuditPublisher wins
    public AuditPublisher auditPublisher(KafkaTemplate<String, Object> template, AuditProperties props) {
        return new KafkaAuditPublisher(template, props.topic());
    }
}
```

src/main/resources/META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports:

```
com.trinity.common.audit.autoconfigure.AuditAutoConfiguration
```

Debugging

- Run with `--debug` or set `debug=true` to print the **Conditions Evaluation Report** (positive and negative matches).
- The Actuator `/actuator/conditions` endpoint shows the same report.
- To exclude something: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)` or `spring.autoconfigure.exclude=...`.

Gotchas

- If the main class is in the **default package** (no package), scanning covers the whole classpath. It is slow and can break the app. Always use a root package such as `com.trinity.citypulse`.
- Classes in sibling packages outside the main class's package are not scanned or found as entities. Use `scanBasePackages`, `@EntityScan` or `@EnableJpaRepositories`.
- `@ConditionalOnMissingBean` inside **your own** `@Configuration` (not an auto-config) is order-dependent and unreliable.

Likely follow-ups:

- Where were auto-configs listed before Boot 2.7? In `META-INF/spring.factories` under the `EnableAutoConfiguration` key. Boot 3 no longer reads auto-configs from there.
- How do you see why a bean wasn't auto-configured? Use `--debug` (the conditions report) or `/actuator/conditions`.
- Can you use the three annotations separately instead of `@SpringBootApplication`? Yes. It is just a composition, and splitting them lets you narrow the scan or replace `@EnableAutoConfiguration` with `@ImportAutoConfiguration`.
- What does `@AutoConfigurationPackage` do? It registers the base package so JPA `@Entity` scanning and Spring Data repositories know where to look.

C2. What happens if `@EnableAutoConfiguration` (or equivalent) is not specified?

Without `@EnableAutoConfiguration` (or `@SpringBootApplication`, or `@ImportAutoConfiguration`), `AutoConfigurationImportSelector` never runs, so **no auto-configuration class is loaded**. You still get a plain Spring `ApplicationContext` that contains only what you declare yourself.

What you lose:

Feature	Without auto-config
Embedded Tomcat	Not created.
DispatcherServlet, Jackson ObjectMapper, message converters	Not registered. @RestController s are scanned but never mapped.
DataSource, EntityManagerFactory, JpaTransactionManager, Spring Data repositories	None. Injecting a repository fails with NoSuchBeanDefinitionException .
Actuator, Security filter chain, Kafka/Redis templates, @ConfigurationProperties for starters	None.
Things that still work	SpringApplication itself: the Environment, application.yml loading, profiles, the banner, logging setup, ApplicationRunner / CommandLineRunner, and component scanning if @ComponentScan is present . Those features come from SpringApplication, not from auto-configuration.

The typical failure with spring-boot-starter-web on the classpath:

```
@Configuration
@ComponentScan
public class App {
    // no @EnableAutoConfiguration
    public static void main(String[] args) { SpringApplication.run(App.class, args); }
}
```

SpringApplication sees the Servlet API and Spring MVC on the classpath (WebApplicationContext.deduceFromClasspath() returns SERVLET). It therefore creates a ServletWebServerApplicationContext, but there is no ServletWebServerFactory bean, because ServletWebServerFactoryAutoConfiguration never ran:

```
ApplicationContextException: Unable to start web server;
nested exception is ... Unable to start ServletWebServerApplicationContext due to missing ServletWebServerFactory bean.
```

For a non-web app it simply starts with only your beans. @Autowired DataSource then fails.

Equivalents and partial options

- @ImportAutoConfiguration({DataSourceAutoConfiguration.class, JacksonAutoConfiguration.class}) imports only the auto-configs you list. Test slices like @WebMvcTest and @DataJpaTest work this way.
- You can configure everything manually (TomcatServletWebServerFactory, DispatcherServlet, DataSource ...). This is plain Spring Framework style.

Likely follow-ups:

- Why would you deliberately avoid full auto-config? Faster startup, a smaller memory footprint, tight control (libraries, CLI tools), or test slices.
- Is application.yml still read without it? Yes. Config loading is done by SpringApplication / ConfigDataEnvironmentPostProcessor, not by auto-config.
- How do you turn off just one auto-config? exclude = X.class or spring.autoconfigure.exclude.

C3. @Configuration vs @Bean; why configuration classes (proxyBeanMethods, CGLIB, lite mode)

- @Bean goes on a method. Its return value is registered as a bean, named after the method by default. You use it for **third-party classes you can't annotate** (RestClient, ObjectMapper, KafkaTemplate) or when creation needs logic.
- @Configuration goes on a class. It marks a source of bean definitions and is itself a @Component.

Full mode vs lite mode

```
@Configuration // proxyBeanMethods = true (default) -> FULL mode
public class AppConfig {

    @Bean
    public Clock clock() { return Clock.systemUTC(); }

    @Bean
    public TicketService ticketService() {
        return new TicketService(clock()); // intercepted: returns the SAME singleton Clock
    }

    @Bean
    public ReportService reportService() {
        return new ReportService(clock()); // same Clock instance again
    }
}
```

```

    }
}

```

Internals: `ConfigurationClassPostProcessor` (a `BeanFactoryPostProcessor`) runs `ConfigurationClassEnhancer`, which replaces the bean class with a **CGLIB subclass** (`AppConfig$$SpringCGLIB$$0`). Each `@Bean` method is overridden by a `BeanMethodInterceptor`. When `clock()` is called, the interceptor first checks the `BeanFactory`: if the bean already exists, it returns it; otherwise it calls `super.clock()`. This keeps **singleton semantics for inter-bean method calls**.

Lite mode applies to `@Configuration(proxyBeanMethods = false)`, or to `@Bean` methods inside a `@Component` or a plain class:

- There is no CGLIB subclass, so `clock()` is an ordinary Java call. **Each call creates a new `Clock`**, which silently breaks singleton expectations.
- The fix is to declare dependencies as **method parameters**. That works in both modes and is the recommended style:

```

@Configuration(proxyBeanMethods = false)
public class AppConfig {

    @Bean
    public Clock clock() { return Clock.systemUTC(); }

    @Bean
    public TicketService ticketService(Clock clock) { // container injects the singleton
        return new TicketService(clock);
    }
}

```

Why Boot's own auto-configs use `proxyBeanMethods = false` (`@AutoConfiguration` sets it): there is no CGLIB class generation, so startup is faster, memory use is lower, and the code works with **GraalVM native images and AOT**.

Rules and gotchas

- In full mode the `@Configuration` class **must not be `final`**, and `@Bean` methods **must not be `private` or `final`**, because CGLIB has to override them.
- `BeanFactoryPostProcessor` beans (e.g. `PropertySourcesPlaceholderConfigurer`) should be declared with **`static @Bean`** methods. Otherwise the config class is instantiated too early, and its `@Autowired` / `@Value` fields stay unprocessed (Spring logs a warning).
- `@Bean(initMethod = "...", destroyMethod = "...")`. For `@Bean`, the destroy method is **inferred**: a public `close()` or `shutdown()` method is called automatically. Use `destroyMethod = ""` to disable that.
- `@Bean` can be combined with `@Scope`, `@Lazy`, `@Primary`, `@Qualifier`, `@Profile`, `@Conditional*`, `@Order` and `@DependsOn`.
- `@Import(OtherConfig.class)` composes configuration classes.

Why use configuration classes at all? Wiring lives in one type-safe, refactorable, testable place: no XML, compile-time checking, and you can use conditionals and profiles. It also keeps business classes free of framework construction code.

Likely follow-ups:

- *What happens if you call a `@Bean` method from a `@Component` class?* A new instance is created (lite mode). Inject the bean instead.
- *Can `@Bean` methods be static?* Yes. That is required for `BeanFactoryPostProcessor`s. A static method is never intercepted by CGLIB.
- *`@Bean` vs `@Component`?* `@Component` is class-level and auto-detected by scanning. `@Bean` is method-level, explicit, and works for third-party types.

C4. What is a Bean; how is a Bean created internally?

A **bean** is an object whose **creation, wiring, lifecycle and scope are managed by the Spring IoC container**. Spring describes it with a `BeanDefinition` (the recipe), and the `BeanFactory` turns that recipe into an instance.

BeanDefinition: the recipe

It holds the bean class or factory method (`@Bean`), scope, lazy-init flag, primary flag, `dependsOn`, constructor arguments, property values, init and destroy method names, autowire candidate flag and role.

- Sources: `ClassPathBeanDefinitionScanner` (`@Component`), `ConfigurationClassBeanDefinitionReader` (`@Bean`, `@Import`), auto-configs, and XML.
- They are stored in a `BeanDefinitionRegistry`. In practice that is `DefaultListableBeanFactory`, which holds `Map<String, BeanDefinition>` `beanDefinitionMap`.

BeanFactory vs ApplicationContext

- `BeanFactory` is the basic container: `getBean`, lazy by default.
- `ApplicationContext` extends it and adds eager singleton pre-instantiation, automatic BPP/BFPP registration, events (`ApplicationEventPublisher`), i18n (`MessageSource`), `Environment` / profiles and resource loading.

- In Boot the concrete type is `AnnotationConfigServletWebServerApplicationContext` (web) or `AnnotationConfigApplicationContext`.

`AbstractApplicationContext.refresh()`: the big picture

1. `prepareRefresh()`, then `obtainFreshBeanFactory()`, then `prepareBeanFactory()`.
2. `invokeBeanFactoryPostProcessors()`: `ConfigurationClassPostProcessor` parses `@Configuration`, `@ComponentScan`, `@Import` and auto-configs and **registers BeanDefinitions**. `BeanFactoryPostProcessor`s can then modify the definitions (e.g. resolve `${...}` placeholders).
3. `registerBeanPostProcessors()`: instantiates every `BeanPostProcessor` early.
4. `initMessageSource()`, then `initApplicationEventMulticaster()`.
5. `onRefresh()`: **Boot creates and starts the embedded web server here.**
6. `registerListeners()`.
7. `finishBeanFactoryInitialization()` → `preInstantiateSingletons()`: creates every non-lazy singleton.
8. `finishRefresh()`: starts `SmartLifecycle` beans and publishes `ContextRefreshedEvent`. Boot later publishes `ApplicationStartedEvent` and `ApplicationReadyEvent`.

Creating one bean: `getBean()` → `doGetBean()` → `createBean()` → `doCreateBean()`

1. **Check the singleton caches** (`DefaultSingletonBeanRegistry`, the "three-level cache"):
 - `singletonObjects`: fully initialised beans
 - `earlySingletonObjects`: early references that are not yet fully initialised
 - `singletonFactories`: `ObjectFactory`s that can produce an early reference (possibly an AOP proxy)
2. Merge the bean definition (parent/child) and create the `dependsOn` beans first.
3. **resolveBeforeInstantiation**: an `InstantiationAwareBeanPostProcessor` may short-circuit and return a proxy.
4. **Instantiate** (`createBeanInstance`): pick a supplier, a factory method (`@Bean`) or a constructor (`autowireConstructor` resolves constructor arguments from the factory). The result is a raw object.
5. **MergedBeanDefinitionPostProcessor**: `AutowiredAnnotationBeanPostProcessor` and `CommonAnnotationBeanPostProcessor` scan the class for `@Autowired`, `@Value`, `@Inject`, `@Resource`, `@PostConstruct` and `@PreDestroy`, and cache that metadata.
6. **Expose an early reference**: `addSingletonFactory(beanName, () -> getEarlyBeanReference(...))`. This is what allows setter/field circular references.
7. **Populate properties** (`populateBean`): `InstantiationAwareBeanPostProcessor.postProcessProperties`. Here `AutowiredAnnotationBeanPostProcessor` injects `@Autowired` fields and setters and `@Value`s.
8. **Initialize** (`initializeBean`):
 - `invokeAwareMethods`: `BeanNameAware`, `BeanClassLoaderAware`, `BeanFactoryAware`
 - `BeanPostProcessor.postProcessBeforeInitialization`: `ApplicationContextAwareProcessor` handles `EnvironmentAware`, `ResourceLoaderAware`, `ApplicationEventPublisherAware`, `ApplicationContextAware`... and `CommonAnnotationBeanPostProcessor` calls `@PostConstruct`
 - `invokeInitMethods`: `InitializingBean.afterPropertiesSet()`, then the custom `initMethod`
 - `BeanPostProcessor.postProcessAfterInitialization`: **AbstractAutoProxyCreator** (`AnnotationAwareAspectJAutoProxyCreator`) wraps the bean in a **JDK dynamic proxy or CGLIB proxy** when an advisor applies (`@Transactional`, `@Async`, `@Cacheable`, `@Aspect`, method security). **The proxy is what gets registered and injected.**
9. `registerDisposableBeanIfNecessary` records destroy callbacks.
10. The bean is put into `singletonObjects`.

```
@Component
public class SignalController implements BeanNameAware, ApplicationContextAware, InitializingBean {
    private final SignalRepository repo; // 4. constructor injection
    @Value("${signal.max-phase:120}") private int max; // 7. populate

    public SignalController(SignalRepository repo) { this.repo = repo; }

    @Override public void setBeanName(String name) { } // 8a
    @Override public void setApplicationContext(ApplicationContext ctx) { } // 8b (via BPP)
    @PostConstruct void init() { } // 8b
    @Override public void afterPropertiesSet() { } // 8c
}
```

Proxy type: Spring Boot defaults to **CGLIB class proxies** (`spring.aop.proxy-target-class=true`), even when the bean implements an interface. With JDK proxies, injecting the concrete class would fail.

Likely follow-ups:

- *Why does a constructor circular dependency fail when a field one can work?* An early reference is exposed only **after** instantiation. With constructors, neither object can be instantiated first, so you get `BeanCurrentlyInCreationException`.

- *BeanPostProcessor vs BeanFactoryPostProcessor?* A BFPP modifies **definitions** before any bean exists. A BPP modifies **instances** around initialization.
- *When are AOP proxies created?* In `postProcessAfterInitialization`, or earlier through `getEarlyBeanReference` when a circular reference needs one.

C5. Bean life cycle

Spring Bean Lifecycle (Spring Boot 3 / Spring 6)

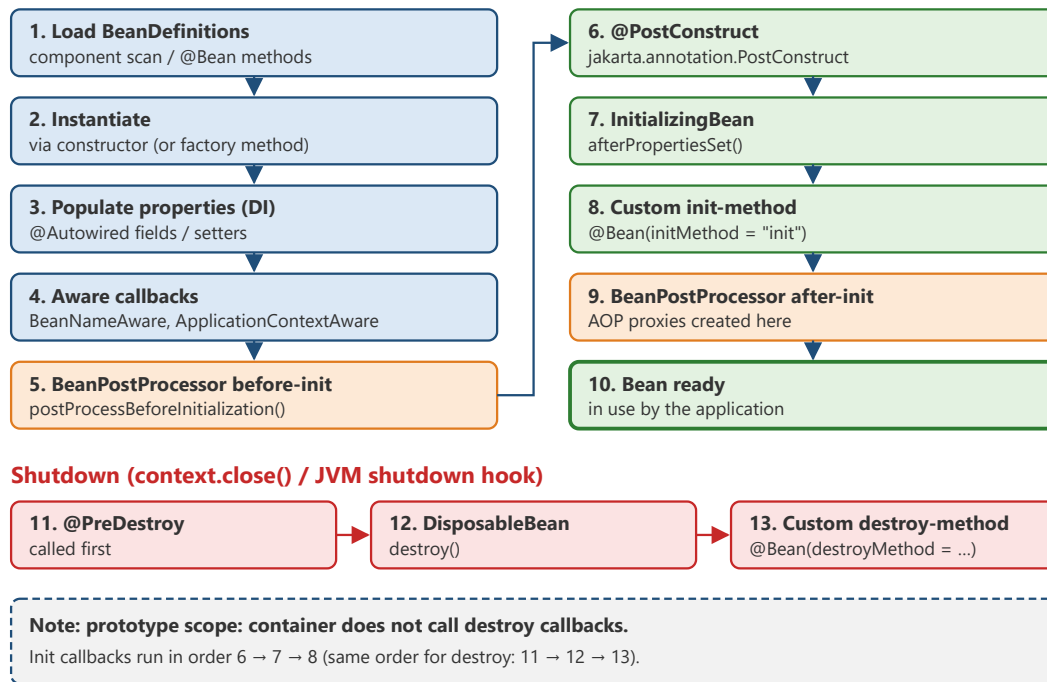


Figure 11: Spring Bean Lifecycle (Spring Boot 3 / Spring Framework 6)

Full sequence for a singleton:

1. Container starts → BeanDefinitions are loaded → `BeanFactoryPostProcessor`s run.
2. **Instantiation**: the constructor runs (constructor injection happens here).
3. **Populate properties**: field and setter injection, `@Value`.
4. **Aware callbacks**: `BeanNameAware.setBeanName` → `BeanClassLoaderAware` → `BeanFactoryAware` → (through BPP) `EnvironmentAware`, `EmbeddedValueResolverAware`, `ResourceLoaderAware`, `ApplicationEventPublisherAware`, `MessageSourceAware`, `ApplicationContextAware`.
5. `BeanPostProcessor.postProcessBeforeInitialization` (all BPPs).
6. `@PostConstruct`, which is run by `CommonAnnotationBeanPostProcessor` as part of step 5.
7. `InitializingBean.afterPropertiesSet()`.
8. **Custom init method**: `@Bean(initMethod = "start")`.
9. `BeanPostProcessor.postProcessAfterInitialization`. **AOP proxies are created here.**
10. After all singletons exist: `SmartInitializingSingleton.afterSingletonsInstantiated()` → `SmartLifecycle.start()` → `ContextRefreshedEvent` → (Boot) `ApplicationStartedEvent` → `CommandLineRunner / ApplicationRunner` → `ApplicationReadyEvent`.
11. **Bean in use.**
12. Shutdown (`context.close()` or the JVM shutdown hook that Boot registers): `ContextClosedEvent` → `SmartLifecycle.stop()` (graceful web server shutdown) → for each disposable bean, in **reverse dependency order**:
 - `@PreDestroy`
 - `DisposableBean.destroy()`
 - **custom destroy method** (`@Bean(destroyMethod=...)`, or the inferred `close()` / `shutdown()`)

```

package com.trinity.citypulse.sensor;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.beans.factory.DisposableBean;
  
```

```

import org.springframework.beans.factory.InitializingBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

public class SensorConnectionPool implements InitializingBean, DisposableBean {

    public SensorConnectionPool() { System.out.println("1. constructor"); }

    @PostConstruct void postConstruct() { System.out.println("2. @PostConstruct"); }
    @Override public void afterPropertiesSet() { System.out.println("3. afterPropertiesSet"); }
    public void customInit() { System.out.println("4. initMethod"); }

    @PreDestroy void preDestroy() { System.out.println("5. @PreDestroy"); }
    @Override public void destroy() { System.out.println("6. DisposableBean.destroy"); }
    public void customDestroy() { System.out.println("7. destroyMethod"); }
}

@Configuration(proxyBeanMethods = false)
class SensorConfig {
    @Bean(initMethod = "customInit", destroyMethod = "customDestroy")
    SensorConnectionPool sensorConnectionPool() { return new SensorConnectionPool(); }
}

```

Output order: 1 → 2 → 3 → 4 ... 5 → 6 → 7.

Which one should you use?

- Prefer `@PostConstruct` / `@PreDestroy` (jakarta.annotation.* in Boot 3). They don't couple your class to Spring.
- `InitializingBean` / `DisposableBean` couple you to Spring and are mostly used inside the framework.
- `initMethod` / `destroyMethod` suit third-party classes you can't annotate.

Gotchas

- **Prototype beans:** Spring runs the init callbacks but **never calls the destroy callbacks**. The client owns the instance.
- `@PostConstruct` runs **on the target object, before the AOP proxy exists**. Calling a `@Transactional` method from `@PostConstruct` therefore gets **no transaction**. Use `ApplicationReadyEvent` or `SmartInitializingSingleton` instead.
- Don't do heavy blocking work (such as calling remote services) in `@PostConstruct`. It delays startup and readiness. Use `ApplicationRunner` or an async listener.
- Destroy callbacks run only on a graceful shutdown (SIGTERM). They do not run on `kill -9`.

Likely follow-ups:

- *Where does BeanPostProcessor fit in?* It wraps initialization: before-init (which includes `@PostConstruct`), then `afterPropertiesSet/initMethod`, then after-init (proxies).
- *How do you run code once the app is fully started?* `@EventListener(ApplicationReadyEvent.class)` or `ApplicationRunner`.
- *Are destroy methods called for request-scoped beans?* Yes, at the end of the request or session. They are not called for prototypes.

C6. Bean scopes; @Scope

Scope	Instances	Notes
singleton (default)	One per <code>ApplicationContext</code> (not per JVM)	Must be stateless or thread-safe.
prototype	A new instance on every <code>getBean()</code> or injection	No destroy callbacks.
request	One per HTTP request	Web context only.
session	One per HTTP session	Web context only.
application	One per <code>ServletContext</code>	Like a singleton, but shared across contexts in the same servlet context.
websocket	One per WebSocket session	STOMP messaging (<code>@EnableWebSocketMessageBroker</code>).

```

@Component
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class ReportBuilder { private final List<String> lines = new ArrayList<>(); /* stateful */ }

@Component
@RequestScope // = @Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestContext { private String tenantId; /* getters/setters */ }

```

You can also write custom scopes (implement `org.springframework.beans.factory.config.Scope`, register with a `CustomScopeConfigurer`), e.g. a "tenant" scope. `@RefreshScope` in Spring Cloud is another example.

The prototype-into-singleton problem

A singleton is created **once**, so a prototype injected into it is also resolved **once**. Every call then sees the *same* "prototype" instance:

```
@Service
public class ReportService {
    private final ReportBuilder builder;    // injected once -> effectively singleton (BUG)
    public ReportService(ReportBuilder builder) { this.builder = builder; }
}
```

Fix 1: ObjectProvider<T> (recommended)

```
@Service
public class ReportService {
    private final ObjectProvider<ReportBuilder> builderProvider;
    public ReportService(ObjectProvider<ReportBuilder> builderProvider) { this.builderProvider = builderProvider; }

    public String generate() {
        ReportBuilder builder = builderProvider.getObject();    // new instance every call
        return builder.build();
    }
}
```

(`jakarta.inject.Provider<T>` works the same way. You can also inject `ApplicationContext` and call `getBean`, but that is the service-locator anti-pattern.)

Fix 2: scoped proxy

```
@Component
@Scope(value = ConfigurableBeanFactory.SCOPE_PROTOTYPE, proxyMode = ScopedProxyMode.TARGET_CLASS)
public class ReportBuilder { ... }
```

The singleton gets a CGLIB proxy, and **every method call** on the proxy fetches a **new** target. That can surprise you: `builder.add()`; `builder.build()`; uses *two different* instances. Scoped proxies make sense for request/session scope, but are dangerous for prototypes.

Fix 3: @Lookup method injection

```
@Service
public abstract class ReportService {
    public String generate() { return createBuilder().build(); }

    @Lookup
    protected abstract ReportBuilder createBuilder();    // CGLIB overrides it to call getBean()
}
```

(`@Lookup` doesn't work on beans created through `@Bean` factory methods.)

Request/session beans in singletons: they need a scoped proxy, which `@RequestScope` / `@SessionScope` set by default. Without one, startup fails with "Scope 'request' is not active for the current thread" because there is no request at startup time.

Gotchas

- A singleton holding mutable fields is shared across all request threads. That causes race conditions.
- Request scope doesn't propagate to `@Async` threads or `CompletableFuture`s.
- A prototype bean that depends on a singleton is fine. The problem only goes the other way.

Likely follow-ups:

- *Is a Spring singleton the same as the GoF Singleton?* No. It is one instance per container and bean id. You can define two beans of the same class.
- *Who destroys prototype beans?* You do. Spring doesn't track them.
- *How is request scope implemented?* `RequestContextHolder` holds request attributes in a `ThreadLocal` (set by `RequestContextFilter` / `DispatcherServlet`), and the scoped bean is stored as a request attribute.

C7. @Lazy

By default the `ApplicationContext` **eagerly** creates all singletons during `refresh()`. That gives fail-fast startup. `@Lazy` delays creation until first use.

1. On a bean definition

```

@Component
@Lazy
public class HeavyGisIndexLoader { ... } // created on first getBean/injection use

@Configuration
@Lazy
class ReportingConfig { ... } // applies to ALL @Bean methods in this class

```

Gotcha: if an **eager** singleton injects a lazy bean normally, the lazy bean is created right away anyway, because the eager bean needs it. So `@Lazy` on the class alone often has no effect.

2. On an injection point

```

@Service
public class AlertService {
    private final SmsGateway smsGateway;
    public AlertService(@Lazy SmsGateway smsGateway) { // gets a lazy-resolution PROXY
        this.smsGateway = smsGateway; // real bean resolved on first method call
    }
}

```

`ContextAnnotationAutowireCandidateResolver` builds a proxy whose methods call `beanFactory.doResolveDependency(...)` when first invoked. This is also a common way to **break circular dependencies**.

3. Global lazy initialization

```

spring:
  main:
    lazy-initialization: true

```

Every bean becomes lazy. Opt specific beans out with `@Lazy(false)`.

Trade-offs

Pros	Cons
Faster startup (dev, tests, serverless)	Misconfiguration surfaces at runtime on the first request, not at startup
Lower startup memory	First-request latency spikes (bad for SLA-bound APIs)
Breaks cycles	Health and readiness can report UP while beans are actually broken
Avoids creating beans you never use	Hides real design problems

Production guidance: keep eager init in production, especially with K8s readiness probes, where you want a failure to stop a bad pod from going live. Use lazy init selectively for heavy optional components, or in local/dev profiles.

Likely follow-ups:

- *Does `@Lazy` work on prototypes?* It isn't needed. Prototypes are always created on demand.
- *How do you lazily get a bean without a proxy?* Use `ObjectProvider<T>.getIfAvailable() / getObject()`.
- *Does `@Lazy` on `@Autowired` fields work?* Yes. It injects a lazy proxy, just like the constructor parameter case.

C8. @Component, stereotypes, component scanning

`@Component` marks a class for **auto-detection** by classpath scanning. The **stereotypes** are `@Component` specialisations that also carry meaning:

Annotation	Layer	Extra behaviour
<code>@Component</code>	generic	none
<code>@Service</code>	business logic	none (semantic only; a natural pointcut target)
<code>@Repository</code>	persistence	Exception translation: <code>PersistenceExceptionTranslationPostProcessor</code> proxies the bean and turns vendor exceptions (<code>SQLException</code> , Hibernate exceptions) into Spring's <code>DataAccessException</code> hierarchy
<code>@Controller</code>	MVC web	detected by <code>RequestMappingHandlerMapping</code>
<code>@RestController</code>	REST	<code>@Controller</code> + <code>@ResponseBody</code>
<code>@Configuration</code>	config	<code>@Component</code> plus CGLIB enhancement (C3)


```

private final NotificationClient notificationClient;

// Single constructor -> @Autowired optional (since Spring 4.3)
public ChallanService(VehicleRepository vehicleRepository, NotificationClient notificationClient) {
    this.vehicleRepository = vehicleRepository;
    this.notificationClient = notificationClient;
}
}

```

(With Lombok, `@RequiredArgsConstructor` on `final` fields generates this constructor.)

2. Setter injection (optional dependencies)

```

@Autowired(required = false)
public void setMetricsPublisher(MetricsPublisher metricsPublisher) { this.metricsPublisher = metricsPublisher; }

```

3. Field injection (discouraged)

```

@Autowired private VehicleRepository vehicleRepository;

```

Why constructor injection is preferred

Concern	Constructor	Field
Immutability (<code>final</code>)	yes	no
Object is never half-built	guaranteed	not guaranteed: can be used with nulls
Unit test without Spring	<code>new ChallanService(mockRepo, mockClient)</code>	needs reflection or <code>@InjectMocks</code>
Hidden dependencies	explicit; a long parameter list signals an SRP violation	hidden; easy to keep adding
Circular dependency	detected at startup (fail-fast)	silently allowed (if enabled)
Framework coupling	none (plain Java)	needs <code>@Autowired</code> /reflection

Other injection styles: `@Value` for config values, `@Resource` (JSR-250, **by name** first), `@Inject` (JSR-330, by type), method/ `@Bean` parameter injection, and `ObjectProvider<T>` for optional or lazy dependencies.

How resolution works: `AutowiredAnnotationBeanPostProcessor` → `DefaultListableBeanFactory.resolveDependency()` → `doResolveDependency()`. It finds candidates **by type**. If there are several, it narrows by `@Qualifier`, then `@Primary`, then `@Priority`, then **parameter or field name** (C10). `Optional<T>`, `ObjectProvider<T>`, `List<T>` and `Map<String,T>` are handled specially.

Circular dependencies

$A \rightarrow B \rightarrow A$.

- **Constructor cycle:** never resolvable. It fails with `BeanCurrentlyInCreationException: Requested bean is currently in creation: Is there an unresolvable circular reference?`
- **Field/setter cycle:** technically resolvable through the early-reference cache (C4). However, **Spring Boot 2.6+ forbids it by default**. Startup fails with "The dependencies of some of the beans in the application context form a cycle", and `spring.main.allow-circular-references=true` re-enables it (not recommended).

Fixes, best first:

1. **Redesign:** pull the shared logic into a third bean C that both A and B depend on.
2. Use **events** (`ApplicationEventPublisher`) to decouple the callback direction.
3. Use **@Lazy** on one injection point: a proxy breaks the cycle at construction time.
4. Use `ObjectProvider<B>` and resolve when needed.
5. Use setter injection plus `allow-circular-references` (last resort).

Likely follow-ups:

- *Why does `@InjectMocks` work with field injection?* Mockito uses reflection. That's exactly why field injection hides dependencies.
- *How do you inject an optional dependency?* `ObjectProvider<T>.getIfAvailable()`, `Optional<T>`, or `@Autowired(required = false)`.
- *@Autowired vs @Resource vs @Inject?* `@Autowired` / `@Inject` resolve by type then qualifier. `@Resource` resolves by name first. Only `@Autowired` has `required`.

C10. Multiple beans of the same type

```
public interface NotificationSender { void send(String to, String msg); }

@Component("smsSender") public class SmsSender implements NotificationSender { ... }
@Component("emailSender") public class EmailSender implements NotificationSender { ... }
@Component("pushSender") public class PushSender implements NotificationSender { ... }
```

Injecting `NotificationSender` alone fails: `NoUniqueBeanDefinitionException: expected single matching bean but found 3: smsSender, emailSender, pushSender`.

How Spring narrows candidates (`DefaultListableBeanFactory.determineAutowireCandidate`):

1. Match by type, then filter by `@Qualifier` if one is present at the injection point.
2. If still more than one: the `@Primary` bean.
3. Otherwise the highest `@Priority` (Jakarta).
4. Otherwise the bean whose **name equals the field or parameter name**. This "fallback qualifier" depends on the `-parameters` compiler flag for constructor parameters; the Boot Maven/Gradle plugins set it.
5. Otherwise `NoUniqueBeanDefinitionException`.

(Spring 6.2 adds `@Fallback`, the inverse of `@Primary`: such a bean is used only if no other candidate remains.)

1. `@Qualifier`: pick one explicitly

```
@Service
public class AlertService {
    private final NotificationSender sender;
    public AlertService(@Qualifier("smsSender") NotificationSender sender) { this.sender = sender; }
}
```

2. `@Primary`: the default choice

```
@Component("emailSender")
@Primary
public class EmailSender implements NotificationSender { ... }
```

3. Injection by name: the parameter name matches the bean name.

```
public AlertService(NotificationSender pushSender) { ... } // resolves pushSender
```

This is fragile (refactoring the name breaks it) and `@Primary` overrides it. Prefer `@Qualifier`.

4. Inject all of them: `List` or `Map`

```
@Service
public class BroadcastService {
    private final List<NotificationSender> senders; // ordered by @Order / Ordered
    private final Map<String, NotificationSender> sendersByName; // key = bean name

    public BroadcastService(List<NotificationSender> senders, Map<String, NotificationSender> sendersByName) {
        this.senders = senders;
        this.sendersByName = sendersByName;
    }

    public void broadcast(String to, String msg) { senders.forEach(s -> s.send(to, msg)); }

    public void sendVia(String channel, String to, String msg) { // strategy pattern
        NotificationSender s = sendersByName.get(channel + "Sender");
        if (s == null) throw new IllegalArgumentException("Unknown channel " + channel);
        s.send(to, msg);
    }
}
```

"There are three beans and I want only one": use `@Qualifier` at the injection point, or mark one `@Primary`. If you don't want the other two at all, don't create them: `@ConditionalOnProperty(name="notify.channel", havingValue="sms")`, `@Profile`, or a scan exclude filter.

"@Primary exists but I need a different one": `@Qualifier` takes precedence over `@Primary`, so `@Qualifier("smsSender")` gets SMS even though Email is primary. You can also define custom qualifier annotations:

```
@Target({ElementType.FIELD, ElementType.PARAMETER, ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Qualifier
```

```
public @interface Sms { }

@Component @Sms public class SmsSender implements NotificationSender { ... }

public AlertService(@Sms NotificationSender sender) { ... }
```

"Two beans with the same name"

- **Spring Boot 2.1+** disables bean definition overriding by default. Two `@Bean` methods (or a `@Bean` plus a scanned component) with the same name fail with: `BeanDefinitionOverrideException: Invalid bean definition with name 'dataSource' defined in class path resource [...]: Cannot register bean definition ... since there is already [...] bound.`
- Re-enable with:

```
spring:
  main:
    allow-bean-definition-overriding: true # last one registered wins - hides bugs, avoid
```

- Plain Spring Framework (without Boot) allows overriding by default and logs it.
- Two **scanned** classes with the same default name fail with `ConflictingBeanDefinitionException`. The fix is the same: give them explicit unique names.
- In tests, prefer `@MockitoBean` / `@TestConfiguration` with `@Primary` over overriding.

Likely follow-ups:

- *How do you order List injection?* `@Order(1)` or `Ordered` on the beans. Note that `@Order` does **not** affect single-bean selection.
- *What does ObjectProvider give you here?* `orderedStream()`, `getIfUnique()`, and lazy, optional resolution.
- *@Primary on two beans?* `NoUniqueBeanDefinitionException` again ("more than one 'primary' bean found").

C11. JPA vs Hibernate vs Spring Data JPA

	JPA (Jakarta Persistence)	Hibernate	Spring Data JPA
What	Specification (interfaces and annotations), <code>jakarta.persistence.*</code>	Implementation of JPA (the ORM provider), plus extra native features	Abstraction on top of JPA : repositories and query derivation
Key types	<code>EntityManager</code> , <code>EntityManagerFactory</code> , <code>@Entity</code> , JPQL, Criteria API	<code>Session</code> , <code>SessionFactory</code> , HQL, <code>@BatchSize</code> , <code>@Formula</code> , filters, Envers, 2nd-level cache	<code>JpaRepository</code> , <code>CrudRepository</code> , <code>@Query</code> , <code>Specification</code> , <code>Pageable</code> , projections, auditing
Can be swapped?	Yes: EclipseLink, OpenJPA	—	It uses whatever JPA provider is present (Boot defaults to Hibernate 6.x)

The layers: **Your code** → **Spring Data repository proxy** (`SimpleJpaRepository`) → **JPA** `EntityManager` → **Hibernate** `Session` → **JDBC** → **DB**.

Spring Data creates a **JDK proxy** for each repository interface. A call like `findByVehicleNumberAndStatus` is parsed by `PartTree` into JPQL at startup, so a bad method name fails fast. CRUD methods delegate to `SimpleJpaRepository`, which is `@Transactional(readOnly = true)` at class level, with `@Transactional` on its write methods.

Persistence context

A **persistence context** is the set of managed entity instances tied to an `EntityManager`. In Spring it is **scoped to the transaction** by default. Spring injects a shared, thread-safe `EntityManager` **proxy** that routes to the transaction-bound one.

- **First-level cache / identity map**: `find(Employee.class, 1L)` twice in one transaction returns the **same instance** with one SQL query.
- **Dirty checking**: at flush, Hibernate compares managed entities against their loaded snapshot and issues `UPDATE`s automatically. You don't need `save()` for managed entities.
- **Write-behind**: SQL is queued and flushed at commit, before a query that touches affected tables (`FlushModeType.AUTO`), or on an explicit `flush()`.

Entity states

```

new Employee()                                persist()/save()
[TRANSIENT/NEW] -----> [MANAGED]
                        | ^
                        | | merge() (returns a NEW managed copy)
tx commit / clear() / close() | |
                        v |
                        [DETACHED]
[MANAGED] -- remove() --> [REMOVED] -- flush/commit --> DELETE (then effectively transient)
```

- **Transient**: has no id and isn't tracked.

- **Managed:** in the persistence context; changes are auto-flushed.
- **Detached:** has an id but the persistence context is closed, so changes are **not** tracked. A typical example is an entity returned from a service after the transaction ended.
- **Removed:** scheduled for deletion.

`persist()` makes the *same* instance managed. `merge()` copies state onto a managed instance and **returns that instance**; the argument stays detached. Spring Data's `save()` calls `persist` if `entityInformation.isNew(entity)` (the id is null, or `@Version` is null, or `Persistable.isNew()`), and `merge` otherwise.

Gotcha: if you assign ids manually (e.g. a UUID set in the constructor), `save()` thinks the entity isn't new. It calls `merge`, which costs an extra `SELECT`. Implement `Persistable<ID>` or use `@Version` to fix that.

Likely follow-ups:

- *Is EntityManager thread-safe?* The real one isn't. Spring's injected shared proxy is, because it binds a real one per transaction/thread.
- *What is the 2nd-level cache?* A Hibernate `SessionFactory`-level cache shared across sessions (Ehcache/JCache/Infinispan). It is opt-in, using `@Cacheable` plus `@org.hibernate.annotations.Cache`.
- *Do you need `repo.save()` after modifying a managed entity?* No. Dirty checking flushes it at commit.

C12. @GeneratedValue strategies

Strategy	How the id is produced	Batch inserts	DBs
IDENTITY	DB auto-increment column; the id is known only after the INSERT	Disabled. Hibernate must run each INSERT immediately to learn the id, so <code>hibernate.jdbc.batch_size</code> is ignored for inserts	MySQL, SQL Server, Postgres (<code>serial</code> /identity)
SEQUENCE	DB sequence; <code>allocationSize</code> (default 50) with the pooled optimizer gives 50 ids per DB round trip	Works	Postgres, Oracle, SQL Server 2012+, H2
TABLE	A separate table emulates a sequence, using row locks	Works, but slow and contended	Any (portable)
AUTO (default)	The provider chooses. Hibernate 6: <code>SequenceStyleGenerator</code> with a sequence per entity (<code><entity>_SEQ</code> , allocation 50). On DBs without sequences (older MySQL), it falls back to a table	Depends	—
UUID (JPA 3.1 / Hibernate 6.2+)	A UUID generated in Java	Works (no DB round trip)	Any

```
@Entity
@Table(name = "challan")
public class Challan {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "challan_seq")
    @SequenceGenerator(name = "challan_seq", sequenceName = "challan_seq", allocationSize = 50)
    private Long id;
    // DB: CREATE SEQUENCE challan_seq START WITH 1 INCREMENT BY 50;  <- INCREMENT must match allocationSize
}

@Entity
public class Device {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)    // or @UuidGenerator(style = UuidGenerator.Style.TIME)
    private UUID id;
}

@Entity
public class Ward {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)    // MySQL: AUTO_INCREMENT
    private Long id;
}
```

Batching config:

```
spring:
  jpa:
    properties:
      hibernate:
```

```
jdbc.batch_size: 50
order_inserts: true
order_updates: true
```

(On MySQL, also add `rewriteBatchedStatements=true` to the JDBC URL.)

Gotchas

- If the sequence's `INCREMENT BY` doesn't match `allocationSize`, you get duplicate-key errors, or Hibernate 6 fails schema validation. Keep them aligned (Flyway: `INCREMENT BY 50`).
- Gaps in ids are normal: allocated blocks are lost on restart. **Never** use ids as business-visible sequential numbers such as challan numbers. Use a separate generator for those.
- Random UUIDv4 primary keys cause **B-tree index fragmentation** (random inserts). Prefer time-ordered UUIDv7 or `Style.TIME`, or a Long PK with a UUID public id.
- On MySQL, prefer `IDENTITY` over the `TABLE` strategy, and accept that inserts aren't batched (or use a UUID).

Likely follow-ups:

- *Why does `IDENTITY` kill batching?* The id is needed to put the entity in the persistence context, and it is only known after each `INSERT` executes.
- *What does `allocationSize = 50` mean?* One sequence call reserves 50 ids, which Hibernate hands out in memory.
- *Which would you pick for Postgres?* `SEQUENCE` with a pooled allocation size.

C13. @RequestParam, @RequestBody, @PathVariable and handling request parameters

Annotation	Source	Example
@PathVariable	URI template segment	/api/v1/junctions/{id}
@RequestParam	Query string or form field (<code>application/x-www-form-urlencoded</code> , <code>multipart</code>)	?status=OPEN&page=0
@RequestBody	Request body, deserialised by an <code>HttpMessageConverter</code> (JSON through Jackson)	POST/PUT/PATCH payloads
@RequestHeader	HTTP header	X-Tenant-Id
@CookieValue	Cookie	SESSION
@ModelAttribute	Query and form params bound onto an object (setters or constructor)	search filters
@RequestPart	A multipart part (file + JSON)	uploads
@MatrixVariable	;key=value path parameters (disabled by default)	rare

```
package com.trinity.citypulse.incident;

import jakarta.validation.Valid;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import org.springframework.http.ResponseEntity;
import org.springframework.util.MultiValueMap;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;
import java.util.Map;
import java.util.Optional;

@RestController
@RequestMapping("/api/v1/incidents")
public class IncidentController {

    private final IncidentService service;
    public IncidentController(IncidentService service) { this.service = service; }

    // /api/v1/incidents/42
    @GetMapping("/{id}")
    public IncidentDto get(@PathVariable Long id) { return service.get(id); }

    // /api/v1/incidents?status=OPEN&page=1&size=20&ward=W12
    @GetMapping
    public List<IncidentDto> search(
        @RequestParam(required = false) IncidentStatus status,           // enum converted; null if missing
        @RequestParam(defaultValue = "0") @Min(0) int page,              // defaultValue implies required=false
        @RequestParam(defaultValue = "20") @Min(1) @Max(100) int size,
        @RequestParam Optional<String> ward,                             // Optional = not required
        @RequestParam(name = "tag", required = false) List<String> tags, // ?tag=a&tag=b
        @RequestHeader(name = "X-Tenant-Id") String tenantId) { }
```

```

        return service.search(tenantId, status, ward.orElse(null), tags, page, size);
    }

    // all query params at once
    @GetMapping("/raw")
    public Map<String, String> raw(@RequestParam Map<String, String> all,
                                   @RequestParam MultiValueMap<String, String> multi) { return all; }

    // bind query params onto an object (no annotation needed for non-simple types: @ModelAttribute implied)
    @GetMapping("/filter")
    public List<IncidentDto> filter(@Valid @ModelAttribute IncidentFilter filter) { return service.filter(filter); }

    @PostMapping
    public ResponseEntity<IncidentDto> create(@Valid @RequestBody CreateIncidentRequest req) {
        IncidentDto created = service.create(req);
        return ResponseEntity.created(java.net.URI.create("/api/v1/incidents/" + created.id())).body(created);
    }

    @PostMapping(path =("/{id}/evidence", consumes = "multipart/form-data")
    public void upload(@PathVariable Long id,
                      @RequestPart("file") MultipartFile file,
                      @RequestPart("meta") @Valid EvidenceMeta meta) { service.attach(id, file, meta); }
}

record IncidentFilter(IncidentStatus status, String ward, @Min(0) Integer minSeverity) { }

record CreateIncidentRequest(
    @jakarta.validation.constraints.NotBlank String title,
    @jakarta.validation.constraints.NotNull IncidentStatus status,
    @jakarta.validation.constraints.Size(max = 500) String description) { }

```

Internals: `RequestMappingHandlerAdapter` holds a list of `HandlerMethodArgumentResolver`s: `RequestParamMethodArgumentResolver`, `PathVariableMethodArgumentResolver`, `RequestResponseBodyMethodProcessor` (for `@RequestBody`, using `HttpMessageConverter`s chosen by `Content-Type`), `ServletModelAttributeMethodProcessor`, and others. Simple types are converted by the `ConversionService` (`String` $\rightarrow$ `Long`, enum, `LocalDate` with `@DateTimeFormat`).

Errors you'll see (all 400 unless noted)

Situation	Exception
Required <code>@RequestParam</code> missing	<code>MissingServletRequestParameterException</code>
?page=abc	<code>MethodArgumentTypeMismatchException</code>
Missing header	<code>MissingRequestHeaderException</code>
Malformed JSON body	<code>HttpMessageNotReadableException</code>
<code>@Valid @RequestBody</code> fails	<code>MethodArgumentNotValidException</code>
Constraints directly on parameters (<code>@Min</code> on <code>@RequestParam</code>)	Spring 6.1+ built-in method validation $\rightarrow$ <code>HandlerMethodValidationException</code> (previously <code>ConstraintViolationException</code> with <code>@Validated</code> on the class)
Wrong <code>Content-Type</code>	<code>HttpMediaTypeNotSupportedException</code> $\rightarrow$ 415

Gotchas

- Since Spring 6.1, parameter names come only from `-parameters` compilation (the `LocalVariableTableParameterNameDiscoverer` was removed). Without the flag, `@RequestParam` with no name fails. Boot's parent POM sets the flag; otherwise use `@RequestParam("status")` explicitly.
- `@RequestBody` can be read **only once** (it's a stream). Filters that log bodies must use `ContentCachingRequestWrapper`.
- Don't use `@RequestBody` on GET. Use query params or `@ModelAttribute`.
- Never bind request bodies straight to JPA entities (mass-assignment risk). Use DTOs or records.

Likely follow-ups:

- `@RequestParam` vs `@PathVariable`? A path variable identifies a resource (`/vehicles/{id}`). Query params filter, sort or page.
- How do you make a path variable optional? Map two paths (`{"/x", "/x/{id}"}`) with `@PathVariable(required = false)` or `Optional`.
- How do you validate a `@RequestParam`? Put constraint annotations on it. Spring 6.1+ validates them automatically (or add `@Validated` on the class for older versions).

C14. REST controllers

`@Controller` vs `@RestController`

- `@Controller`: return values are **view names** resolved by a `ViewResolver` (Thymeleaf/JSP). Add `@ResponseBody` per method to write the return value to the body.
- `@RestController` = `@Controller` + `@ResponseBody` on every method. The return value goes through `HttpMessageConverter`'s (Jackson for JSON).

```
@RestController
@RequestMapping(path = "/api/v1/vehicles", produces = MediaType.APPLICATION_JSON_VALUE)
public class VehicleController {

    private final VehicleService service;
    public VehicleController(VehicleService service) { this.service = service; }

    @GetMapping("/{id}")
    public ResponseEntity<VehicleDto> get(@PathVariable Long id) {
        return ResponseEntity.ok()
            .cacheControl(CacheControl.maxAge(Duration.ofSeconds(30)))
            .eTag("\"" + service.version(id) + "\"")
            .body(service.get(id));
    }

    @PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<VehicleDto> create(@Valid @RequestBody CreateVehicleRequest req,
        UriComponentsBuilder uriBuilder) {
        VehicleDto dto = service.create(req);
        URI location = uriBuilder.path("/api/v1/vehicles/{id}").buildAndExpand(dto.id()).toUri();
        return ResponseEntity.created(location).body(dto); // 201 + Location
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT) // alternative to ResponseEntity
    public void delete(@PathVariable Long id) { service.delete(id); }
}
```

`ResponseEntity<T>` gives full control over the status, headers and body: `ok()`, `created(uri)`, `accepted()`, `noContent()`, `notFound()`, `badRequest()`, `status(HttpStatus.CONFLICT)`. Returning a plain DTO gives 200 (or the `@ResponseStatus` value).

Content negotiation

The `ContentNegotiationManager` determines the requested media types:

1. the **Accept header** (default strategy);
2. optionally a query parameter (`?format=xml`) via `spring.mvc.contentnegotiation.favor-parameter=true`;
3. path extensions (`.json`). This is disabled by default since Spring 5.3 and removed from suffix pattern matching in 6.

Then `AbstractMessageConverterMethodProcessor` intersects the requested types with the `produces` types and the types the registered converters can write, and picks the best match. If nothing matches, the client gets **406 Not Acceptable** (`HttpMediaTypeNotAcceptableException`). On the request side, `consumes` mismatches give **415**.

To support XML, add `jackson-dataformat-xml`. Boot then auto-registers `MappingJackson2XmlHttpMessageConverter`, and `Accept: application/xml` works.

```
@GetMapping(value =("/{id}", produces = { MediaType.APPLICATION_JSON_VALUE, MediaType.APPLICATION_XML_VALUE })
public VehicleDto getNegotiated(@PathVariable Long id) { return service.get(id); }
```

REST design checklist: plural nouns (`/vehicles`), HTTP verbs for actions, proper status codes, versioning (`/api/v1` or a header; Spring 7 adds first-class API versioning), pagination (`Pageable` with `page / size / sort`), idempotency keys for POST, HATEOAS only if needed, OpenAPI docs via `springdoc`.

Likely follow-ups:

- *What happens if a `@RestController` method returns null?* An empty body with 200. Prefer 404 or 204 explicitly.
- *How does Jackson get configured?* `JacksonAutoConfiguration` provides an `ObjectMapper` (customise it with `spring.jackson.*` or a `Jackson2ObjectMapperBuilderCustomizer`).
- *`ResponseEntity` vs `@ResponseStatus`?* `ResponseEntity` is dynamic per request (headers, status). `@ResponseStatus` is static.

C15. Full skeleton for `getEmployeeById`

The layers: **Controller (HTTP)** → **Service (business + transactions)** → **Repository (data)** → **Entity**. A **DTO** crosses the HTTP boundary, and a **global handler** maps exceptions to responses.

Entity

```

package com.trinity.hr.employee;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDate;

@Entity
@Table(name = "employee", uniqueConstraints = @UniqueConstraint(name = "uk_employee_email", columnNames = "email"))
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "employee_seq")
    @SequenceGenerator(name = "employee_seq", sequenceName = "employee_seq", allocationSize = 50)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, length = 150)
    private String email;

    @Column(length = 50)
    private String department;

    @Column(precision = 12, scale = 2)
    private BigDecimal salary;

    private LocalDate joiningDate;

    @Version
    private Long version;

    protected Employee() { } // JPA needs a no-arg constructor

    public Employee(String name, String email, String department, BigDecimal salary, LocalDate joiningDate) {
        this.name = name; this.email = email; this.department = department;
        this.salary = salary; this.joiningDate = joiningDate;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getDepartment() { return department; }
    public BigDecimal getSalary() { return salary; }
    public LocalDate getJoiningDate() { return joiningDate; }
    public Long getVersion() { return version; }
    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }
    public void setDepartment(String department) { this.department = department; }
    public void setSalary(BigDecimal salary) { this.salary = salary; }
}

```

Repository

```

package com.trinity.hr.employee;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    Optional<Employee> findByEmail(String email);
}

```

DTO (record) and mapper

```

package com.trinity.hr.employee;

import java.time.LocalDate;

public record EmployeeDto(Long id, String name, String email, String department, LocalDate joiningDate) {
    // salary intentionally NOT exposed
    static EmployeeDto from(Employee e) {
        return new EmployeeDto(e.getId(), e.getName(), e.getEmail(), e.getDepartment(), e.getJoiningDate());
    }
}

```

Custom exception

```

package com.trinity.hr.common;

public class ResourceNotFoundException extends RuntimeException {
    private final String resource;
    private final Object id;

    public ResourceNotFoundException(String resource, Object id) {
        super("%s not found with id %s".formatted(resource, id));
        this.resource = resource;
        this.id = id;
    }

    public String getResource() { return resource; }
    public Object getId() { return id; }
}

```

Service

```

package com.trinity.hr.employee;

import com.trinity.hr.common.ResourceNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional(readOnly = true)
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) { this.repository = repository; }

    public EmployeeDto getEmployeeById(Long id) {
        return repository.findById(id)
            .map(EmployeeDto::from)
            .orElseThrow(() -> new ResourceNotFoundException("Employee", id));
    }
}

```

Controller

```

package com.trinity.hr.employee;

import jakarta.validation.constraints.Positive;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) { this.service = service; }

    @GetMapping("/{id}")
    public ResponseEntity<EmployeeDto> getEmployeeById(@PathVariable @Positive Long id) {
        return ResponseEntity.ok(service.getEmployeeById(id));
    }
}

```

Global exception handler (RFC 9457/7807 ProblemDetail)

```

package com.trinity.hr.common;

import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.net.URI;
import java.time.Instant;

@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);
}

```

```

@ExceptionHandler(ResourceNotFoundException.class)
public ProblemDetail handleNotFound(ResourceNotFoundException ex, HttpServletRequest req) {
    ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    pd.setTitle("Resource Not Found");
    pd.setType(URI.create("https://api.trinity.com/errors/not-found"));
    pd.setInstance(URI.create(req.getRequestURI()));
    pd.setProperty("resource", ex.getResource());
    pd.setProperty("timestamp", Instant.now());
    return pd;
}

@ExceptionHandler(Exception.class)
public ProblemDetail handleGeneric(Exception ex) {
    log.error("Unhandled error", ex);
    return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error");
}
}

```

A response for `GET /api/v1/employees/99` (Content-Type: `application/problem+json`):

```

{
  "type": "https://api.trinity.com/errors/not-found",
  "title": "Resource Not Found",
  "status": 404,
  "detail": "Employee not found with id 99",
  "instance": "/api/v1/employees/99",
  "resource": "Employee",
  "timestamp": "2026-10-02T06:15:30Z"
}

```

Design points to say aloud

- Return a DTO, not the entity. That avoids lazy-loading serialisation errors, leaking fields (salary), and coupling the API to the schema.
- `findById` returns `Optional`, so there is no null handling. Throwing from the service keeps the controller thin.
- The read-only transaction lets Hibernate skip dirty-checking snapshots.
- Avoid `getReferenceById` here: it returns a lazy proxy and fails late with `EntityNotFoundException`.

Likely follow-ups:

- *Why not return `Optional` from the controller?* HTTP semantics belong in one place, and the advice handles 404 consistently.
- *Where do you map entity to DTO at scale?* MapStruct (compile-time, no reflection).
- *How would you test this?* `@WebMvcTest` with a mocked service for 200/404, `@DataJpaTest` for the repository, and a Mockito unit test for the service (C33–C35).

C16. PUT vs PATCH

	PUT	PATCH
Semantics	Replace the whole resource with the payload (RFC 9110)	Partially modify the resource (RFC 5789)
Missing fields	Treated as "set to default/null"	Left unchanged
Idempotent?	Yes: N identical requests produce the same state	Not guaranteed. Merge patch with absolute values is effectively idempotent; <code>{"op": "add"}</code> on arrays or "increment" ops are not
Safe?	No	No
Create if absent	Allowed (client-chosen id) → 201	Usually not
Formats	Full representation	<code>application/merge-patch+json</code> (RFC 7396), <code>application/json-patch+json</code> (RFC 6902), or an ad-hoc partial DTO

PUT implementation

```

@PutMapping("/{id}")
@Transactional
public EmployeeDto replace(@PathVariable Long id, @Valid @RequestBody EmployeeUpdateRequest req) {
    Employee e = repository.findById(id).orElseThrow() -> new ResourceNotFoundException("Employee", id);
    e.setName(req.name());
    e.setEmail(req.email());
    e.setDepartment(req.department()); // null in request => null in DB (full replacement)
    e.setSalary(req.salary());
}

```

```

    return EmployeeDto.from(e); // dirty checking flushes
}

```

(In real code, move the `@Transactional` logic into the service.)

PATCH option 1: nullable DTO ("null means don't change")

```

public record EmployeePatchRequest(String name, @Email String email, String department) { }

@Transactional
public EmployeeDto patch(Long id, EmployeePatchRequest req) {
    Employee e = repository.findById(id).orElseThrow(() -> new ResourceNotFoundException("Employee", id));
    if (req.name() != null) e.setName(req.name());
    if (req.email() != null) e.setEmail(req.email());
    if (req.department() != null) e.setDepartment(req.department());
    return EmployeeDto.from(e);
}

```

The problem: it **can't express "set this field to null"**. Fixes are `JsonNullable<T>` (openapitools `jackson-databind-nullable`), `Optional<T>` fields (absent = null field, explicit null = `Optional.empty()`), or the next option.

PATCH option 2: JSON Merge Patch (RFC 7396) with Jackson Rules: fields present overwrite, `null` removes or clears the field, absent fields are untouched, and arrays are replaced wholesale.

```

package com.trinity.hr.employee;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Validator;
import org.springframework.web.bind.annotation.*;

import java.io.IOException;

@RestController
@RequestMapping("/api/v1/employees")
public class EmployeePatchController {

    private final EmployeeService service;
    private final ObjectMapper objectMapper;
    private final Validator validator;

    public EmployeePatchController(EmployeeService service, ObjectMapper objectMapper, Validator validator) {
        this.service = service;
        this.objectMapper = objectMapper;
        this.validator = validator;
    }

    @PatchMapping(path =("/{id}", consumes = "application/merge-patch+json")
    public EmployeeDto mergePatch(@PathVariable Long id, @RequestBody JsonNode patch) throws IOException {
        EmployeeUpdateRequest current = service.getForUpdate(id); // current state as DTO
        EmployeeUpdateRequest patched = objectMapper.readerForUpdating(current) // apply only present fields
            .readValue(patch);
        var violations = validator.validate(patched); // validate the RESULT
        if (!violations.isEmpty()) throw new ConstraintViolationException(violations);
        return service.replace(id, patched);
    }
}

```

Note: `readerForUpdating` works on mutable objects (POJOs with setters). Records are immutable, so with record DTOs, merge at the `JsonNode` level instead: `ObjectNode base = objectMapper.valueToTree(current); merge(base, patch); objectMapper.treeToValue(base, Dto.class)`.

JSON Patch (RFC 6902) is an array of operations:

```

[ { "op": "replace", "path": "/department", "value": "Traffic" },
  { "op": "remove", "path": "/phone" } ]

```

Libraries: `com.github.java-json-tools:json-patch`, Or `jakarta.json.JsonPatch`.

Concurrency: both PUT and PATCH should use **optimistic locking**. The client sends `If-Match: "<version>"` (ETag), and the server returns **412 Precondition Failed** on a mismatch, or 409 when `@Version` throws `ObjectOptimisticLockingFailureException`.

Likely follow-ups:

- *Is PATCH ever idempotent?* It can be, for example merge patch with absolute values, but the spec doesn't require it.
- *Why validate after patching?* Each field may be valid on its own while the merged object isn't. Validate the final state.
- *Which status should PUT/PATCH return?* 200 with the updated body, or 204 with no body. PUT that creates returns 201.

C17. HTTP 204

204 No Content means the request **succeeded** and the server is **intentionally sending no body**. The response must not have a body. Headers (e.g. ETag) may still be present.

When to use it

- DELETE /vehicles/42 succeeded.
- PUT/PATCH succeeded and the client doesn't need the updated representation.
- Fire-and-forget actions that completed synchronously (POST /devices/7/reboot when done inline).
- CORS preflight OPTIONS responses (often 204).

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}

// or
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete2(@PathVariable Long id) { service.delete(id); }
```

Compared with other codes

Code	Meaning	Body	Typical use
200 OK	Success	Yes	GET; PUT/PATCH returning the updated entity
201 Created	A resource was created	Usually yes, plus a Location header	POST create
202 Accepted	Accepted for async processing, not done yet	Often a status URL	Bulk import, report generation through Kafka
204 No Content	Success with no body	No	DELETE, update without echo
404 Not Found	The resource doesn't exist	Error body (ProblemDetail)	GET/DELETE of an unknown id

Debated cases

- **An empty search result** (GET /incidents?ward=W99): return **200 with []** (or an empty page), not 204. The collection resource exists; it just has no items. Clients can iterate without special-casing.
- **DELETE of a missing id**: either 404 (strict), or 204 (idempotent: "the end state is the same"). Pick one and document it.

Likely follow-ups:

- *Can a 204 have a body?* No. Clients and proxies ignore or strip it, and some HTTP clients misbehave.
- *204 vs 202 for async?* 202 means "accepted, not done". 204 means "done, nothing to return".
- *What does a browser do with 204 on a form submit?* It stays on the current page.

C18. Actuator

spring-boot-starter-actuator adds production-ready endpoints for **health, metrics, info, and diagnostics**.

Endpoint	Purpose
/actuator/health	UP/DOWN, aggregated from HealthIndicator s (db, diskSpace, redis, kafka...)
/actuator/health/liveness , /readiness	K8s probes (auto-enabled on K8s, or via management.endpoint.health.probes.enabled=true)
/actuator/info	Build, git and custom info
/actuator/metrics , /metrics/{name}	Micrometer meters
/actuator/prometheus	Prometheus scrape format (needs micrometer-registry-prometheus)
/actuator/loggers	View or change log levels at runtime (POST)
/actuator/env , /configprops	Properties (values are sanitised/masked)
/actuator/beans , /conditions , /mappings	Context introspection
/actuator/threaddump , /heapdump	Diagnostics (sensitive!)

Endpoint	Purpose
/actuator/httpexchanges	Recent requests (needs an <code>HttpExchangeRepository</code> bean)
/actuator/scheduledtasks, /caches, /flyway, /liquibase	Feature-specific
/actuator/shutdown	Graceful shutdown. Disabled by default

Exposure: by default, **only health is exposed over HTTP**. All endpoints are available over JMX if JMX is enabled.

```
management:
  server:
    port: 8081 # separate port, not exposed through the public ingress
  endpoints:
    web:
      base-path: /actuator
      exposure:
        include: health,info,prometheus,metrics,loggers
        exclude: env,beans
    endpoint:
      health:
        show-details: when-authorized # never | when-authorized | always
      probes:
        enabled: true
      group:
        readiness:
          include: readinessState,db,redis
  info:
    env:
      enabled: true # info.* properties are hidden by default since Boot 2.6
    git:
      mode: simple
  metrics:
    tags:
      application: ${spring.application.name}
info:
  app:
    name: citypulse-incident-service
    team: smart-city
```

(Boot 3.4+ uses `management.endpoint.<id>.access: read-only|unrestricted|none` instead of the older `.enabled` property for enabling and restricting endpoints.)

Securing it

```
import org.springframework.boot.actuate.autoconfigure.security.servlet.EndpointRequest;
import org.springframework.boot.actuate.health.HealthEndpoint;
import org.springframework.boot.actuate.info.InfoEndpoint;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class ActuatorSecurityConfig {

    @Bean
    @Order(1)
    SecurityFilterChain actuatorChain(HttpSecurity http) throws Exception {
        http.securityMatcher(EndpointRequest.toAnyEndpoint())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(EndpointRequest.to(HealthEndpoint.class, InfoEndpoint.class)).permitAll()
                .anyRequest().hasRole("OPS"))
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

Other defences: a separate management port plus network policy, and never exposing `heapdump` / `env` publicly (they leak secrets).

Custom health indicator

```
package com.trinity.citypulse.health;

import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
```

```
import org.springframework.stereotype.Component;

@Component("cctvGateway") // shows as "cctvGateway" in /health
public class CctvGatewayHealthIndicator implements HealthIndicator {

    private final CctvGatewayClient client;
    public CctvGatewayHealthIndicator(CctvGatewayClient client) { this.client = client; }

    @Override
    public Health health() {
        try {
            long latencyMs = client.ping();
            return latencyMs < 500
                ? Health.up().withDetail("latencyMs", latencyMs).build()
                : Health.status("DEGRADED").withDetail("latencyMs", latencyMs).build();
        } catch (Exception e) {
            return Health.down(e).withDetail("gateway", "unreachable").build();
        }
    }
}
```

Don't put external dependencies in the liveness group. If they are there, a downstream outage makes K8s restart every pod (a cascading failure). Readiness is the right place for them.

Micrometer and Prometheus

```
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import org.springframework.stereotype.Service;

@Service
public class ChallanMetricsService {
    private final Counter challansIssued;
    private final Timer anprLatency;

    public ChallanMetricsService(MeterRegistry registry) {
        this.challansIssued = Counter.builder("challan.issued").tag("source", "anpr")
            .description("Challans issued").register(registry);
        this.anprLatency = Timer.builder("anpr.recognition.latency").publishPercentiles(0.95, 0.99)
            .register(registry);
    }

    public void issue(Runnable work) {
        anprLatency.record(work);
        challansIssued.increment();
    }
}
```

Boot auto-instruments `http.server.requests`, JVM, HikariCP, Tomcat, cache, Kafka and logback metrics. Prometheus scrapes `/actuator/prometheus`, and Grafana dashboards and alerts sit on top. `@Timed` needs a `TimedAspect` bean, and `@Observed` (Micrometer Observation API) also gives **tracing** (Micrometer Tracing with OTel/Zipkin).

Likely follow-ups:

- How do you change a log level in prod without restarting? `POST /actuator/loggers/com.trinity {"configuredLevel":"DEBUG"}`.
- Liveness vs readiness? Liveness means "restart me if I'm broken". Readiness means "don't route traffic to me yet".
- How do you add custom info? `info.*` properties (with `management.info.env.enabled`), or an `InfoContributor` bean.

C19. Front Controller pattern (DispatcherServlet flow)

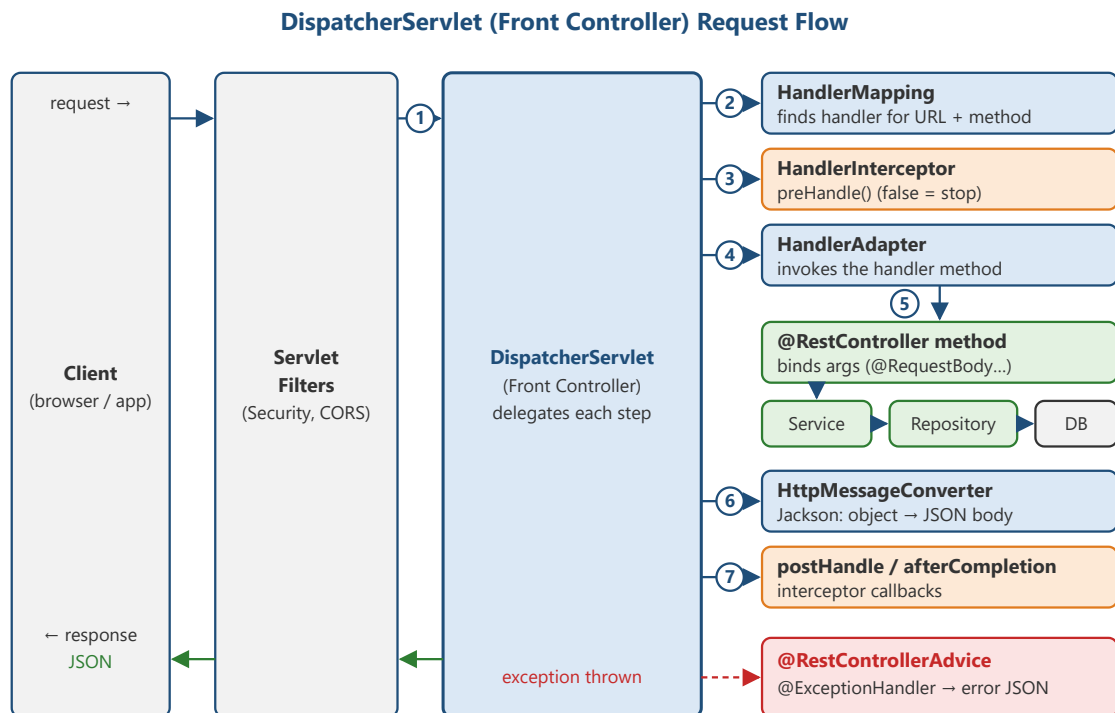


Figure 12: Spring MVC DispatcherServlet (Front Controller) request flow

Front Controller means a **single entry point** handles every request and delegates to specific handlers. Cross-cutting concerns (routing, security, conversion, errors) live in one place. In Spring MVC that entry point is **DispatcherServlet**, which Boot registers at `/` through `DispatcherServletAutoConfiguration`.

The flow

```
Client
-> Servlet container (Tomcat) -> Filter chain (Security, CORS, logging, ...)
-> DispatcherServlet.doService() -> doDispatch()
  1. checkMultipart()
  2. getHandler(): iterate HandlerMappings
     RequestMappingHandlerMapping (@RequestMapping), RouterFunctionMapping, SimpleUrlHandlerMapping (static)
     -> HandlerExecutionChain (HandlerMethod + HandlerInterceptors) [none -> 404 NoHandlerFoundException /
NoResourceFoundException]
  3. getHandlerAdapter(): RequestMappingHandlerAdapter
  4. interceptors.preHandle() (false -> stop)
  5. ha.handle():
     - HandlerMethodArgumentResolvers build arguments
       (@PathVariable, @RequestParam, @RequestBody -> HttpMessageConverter read, validation)
     - invoke the controller method (via the AOP proxy if any)
     - HandlerMethodReturnValueHandlers:
       @ResponseBody/ResponseEntity -> RequestResponseBodyMethodProcessor -> content negotiation
       -> HttpMessageConverter.write() (Jackson) => response committed, no view
       String/ModelAndView -> view name
  6. interceptors.postHandle() (not called if an exception occurred; too late to change a @ResponseBody response)
  7. processDispatchResult():
     exception? -> HandlerExceptionResolvers:
      ExceptionHandlerExceptionHandlerResolver (@ExceptionHandler / @ControllerAdvice)
      ResponseStatusExceptionHandlerResolver (@ResponseStatus, ResponseStatusExceptionHandler)
       DefaultHandlerExceptionHandlerResolver (Spring MVC exceptions -> 4xx/5xx)
     view? -> ViewResolver (ThymeleafViewResolver/InternalResourceViewResolver) -> View.render(model)
  8. interceptors.afterCompletion() (always, even on exception)
<- Filters (post-processing) <- Client
```

DispatcherServlet's **strategy beans** are all auto-configured and replaceable: `HandlerMapping`, `HandlerAdapter`, `HandlerExceptionResolver`, `ViewResolver`, `LocaleResolver`, `MultipartResolver`, `FlashMapManager`.

Custom interceptor

```
package com.trinity.citypulse.web;
```

```

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.MDC;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import java.util.UUID;

class CorrelationIdInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
        String cid = req.getHeader("X-Correlation-Id");
        if (cid == null) cid = UUID.randomUUID().toString();
        MDC.put("correlationId", cid);
        res.setHeader("X-Correlation-Id", cid);
        req.setAttribute("startNanos", System.nanoTime());
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest req, HttpServletResponse res, Object handler, Exception ex) {
        MDC.remove("correlationId");
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new CorrelationIdInterceptor())
            .addPathPatterns("/api/**")
            .excludePathPatterns("/api/public/**");
    }
}

```

Filter vs interceptor

Filter (Servlet API)	HandlerInterceptor (Spring MVC)
Runs before DispatcherServlet and for every request, including static resources	Runs only for requests that map to a handler
Can wrap or replace the request and response	Knows the <code>HandlerMethod</code> (can read annotations)
Spring Security lives here	Auth checks per handler, logging, locale
Exceptions are not handled by <code>@ControllerAdvice</code>	Exceptions go through <code>HandlerExceptionResolvers</code>

Likely follow-ups:

- *Who converts the object to JSON?* An `HttpMessageConverter` (`MappingJackson2HttpMessageConverter`) chosen through content negotiation.
- *Can there be multiple DispatcherServlets?* Yes, each with its own child `WebApplicationContext` and mapping, but that's rarely needed in Boot.
- *What is WebFlux's equivalent?* `DispatcherHandler` (reactive, non-blocking).

C20. Exception handling

Options, from local to global

1. `try/catch` in the controller (avoid, because it scatters logic).
2. `@ExceptionHandler` methods **inside one controller**, which apply only to that controller.
3. `@RestControllerAdvice` (= `@ControllerAdvice` + `@ResponseBody`): global handlers. Scope them with `basePackages`, `assignableTypes` or `annotations`, and order several advices with `@Order`.
4. `@ResponseStatus` on a custom exception class: maps it to a status, with no body control.
5. `ResponseStatusException`: throw it with a status and reason directly (`throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Junction not found")`). Good for quick cases.
6. `ExceptionHandler` / `BasicExceptionHandler` at `/error`: the fallback for anything not handled, including filter errors.

Resolution internals: `ExceptionHandlerResolver` looks first at `@ExceptionHandler` methods in the **controller**, then in the advices. It picks the **closest match in the exception hierarchy** (`ExceptionHandlerComparator`). It also matches causes for wrapped exceptions.

ProblemDetail (RFC 7807, now RFC 9457): Spring 6 supports it natively. `ProblemDetail`, `ErrorResponse` and `ErrorResponseException` produce `application/problem+json`.

- `spring.mvc.problemdetails.enabled=true` makes Boot register a `ResponseEntityExceptionHandler` advice that renders **Spring MVC's built-in exceptions** (400/404/405/415...) as `ProblemDetail`.
- To customise, **extend `ResponseEntityExceptionHandler`** yourself:

```
package com.trinity.citypulse.web;

import jakarta.validation.ConstraintViolationException;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.*;
import org.springframework.lang.NonNull;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

import java.util.List;
import java.util.Map;

@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    // validation of @Valid @RequestBody -> 400 with field errors
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(@NonNull MethodArgumentNotValidException ex,
        @NonNull HttpHeaders headers, @NonNull HttpStatus status, @NonNull WebRequest request) {
        ProblemDetail pd = ex.getBody(); // already a ProblemDetail
        pd.setTitle("Validation failed");
        List<Map<String, Object>> errors = ex.getBindingResult().getFieldErrors().stream()
            .map(fe -> Map.<String, Object>.of(
                "field", fe.getField(),
                "message", String.valueOf(fe.getDefaultMessage()),
                "rejected", String.valueOf(fe.getRejectedValue())))
            .toList();
        pd.setProperty("errors", errors);
        return ResponseEntity.status(status).headers(headers).body(pd);
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    ProblemDetail notFound(ResourceNotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(ConstraintViolationException.class) // @Validated method-level (service layer)
    ProblemDetail constraint(ConstraintViolationException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Constraint violation");
        pd.setProperty("violations", ex.getConstraintViolations().stream()
            .map(v -> v.getPropertyPath() + ": " + v.getMessage()).toList());
        return pd;
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    ProblemDetail conflict(DataIntegrityViolationException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, "Duplicate or invalid reference");
    }

    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
    ProblemDetail optimistic(ObjectOptimisticLockingFailureException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, "Resource was modified by another user");
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail fallback(Exception ex) {
        logger.error("Unhandled", ex); // never leak stack traces to clients
        return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "Internal error");
    }
}
```

You can also make domain exceptions extend `ErrorResponseException`, so they carry their own `ProblemDetail`.

Validation errors summary

- `@Valid @RequestBody` → `MethodArgumentNotValidException`
- `@Valid @ModelAttribute` → `MethodArgumentNotValidException` (a `BindException` subtype)
- constraints on `@RequestParam` / `@PathVariable` (6.1+) → `HandlerMethodValidationException`
- `@Validated` service methods → `ConstraintViolationException`

Exceptions not caught by `@ControllerAdvice`

- Exceptions thrown in **Servlet filters**, including Spring Security's filter chain, happen **before** `DispatcherServlet`. The advice never sees them, and they go to the container's `/error` (`BasicExceptionHandler`).
- For Security: implement `AuthenticationEntryPoint` (401) and `AccessDeniedHandler` (403).
- For custom filters: catch the exception in the filter and either write the response directly, or delegate to the MVC resolvers:

```
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.web.servlet.HandlerExceptionResolver;

import java.io.IOException;

@Component
public class TenantFilter extends OncePerRequestFilter {
    private final HandlerExceptionResolver resolver;

    public TenantFilter(@Qualifier("handlerExceptionResolver") HandlerExceptionResolver resolver) {
        this.resolver = resolver;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        try {
            if (req.getHeader("X-Tenant-Id") == null) throw new MissingTenantException();
            chain.doFilter(req, res);
        } catch (MissingTenantException ex) {
            resolver.resolveException(req, res, null, ex); // routes to @RestControllerAdvice
        }
    }
}
```

- Other cases the advice doesn't cover: exceptions in `@Async` methods (use an `AsyncUncaughtExceptionHandler`), `@Scheduled` jobs, Kafka listeners (`DefaultErrorHandler` /DLT), and errors after the response is already committed.

Likely follow-ups:

- *Two handlers match: which wins?* The most specific exception type. Across advices, `@Order`, and controller-local handlers beat advices.
- *ResponseStatusException vs custom exception plus advice?* RSE is quick but couples the service to HTTP. Domain exceptions plus advice are cleaner.
- *How do you hide stack traces in /error?* `server.error.include-stacktrace=never` (the default) and `include-message`.

C21. @ConfigurationProperties

`@ConfigurationProperties` binds a **group of externalised properties** (a prefix) onto a **type-safe object**. Compared with scattered `@Value`s it gives relaxed binding, nested objects, lists and maps, type conversion (`Duration`, `DataSize`), validation and IDE metadata.

1. Record (immutable, constructor binding; preferred in Boot 3)

```
package com.trinity.citypulse.config;

import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.context.properties.bind.DefaultValue;
import org.springframework.boot.convert.DurationUnit;
import org.springframework.util.unit.DataSize;
import org.springframework.validation.annotation.Validated;

import java.time.Duration;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.Map;

@Validated
@ConfigurationProperties(prefix = "citypulse.anpr")
public record AnprProperties(
    @NotBlank String baseUrl,
    @DefaultValue("5s") Duration connectTimeout,
    @DurationUnit(ChronoUnit.SECONDS) @DefaultValue("30") Duration readTimeout,
```

```

        @DefaultValue("10MB") DataSize maxImageSize,
        @Min(1) @DefaultValue("3") int maxRetries,
        @NotEmpty List<String> cameraZones,
        Map<String, String> headers,
        @Valid @DefaultValue(Retry retry) {

    public record Retry(@DefaultValue("200ms") Duration backoff, @DefaultValue("2.0") double multiplier) { }
}

```

- Boot 3 uses **constructor binding automatically** when there is a single parameterised constructor. `@ConstructorBinding` is needed only when there are several constructors, and in Boot 3 it lives in `org.springframework.boot.context.properties.bind`.
- `@DefaultValue` on a nested record makes Boot create an instance with defaults even when no properties are set.

2. JavaBean style (mutable; setters required)

```

@Validated
@ConfigurationProperties(prefix = "citypulse.notify")
public class NotifyProperties {
    @NotBlank private String smsProvider = "msg91";
    private boolean enabled = true;
    // getters + setters (binding uses setters)
}

```

Registering them (one of these):

```

@SpringBootApplication
@ConfigurationPropertiesScan("com.trinity.citypulse.config")    // scans for @ConfigurationProperties types
public class CityPulseApplication { }

// or
@Configuration
@EnableConfigurationProperties({AnprProperties.class, NotifyProperties.class})
class PropsConfig { }

// or, for third-party classes, on a @Bean
@Bean
@ConfigurationProperties(prefix = "citypulse.pool")
HikariDataSource reportingDataSource() { return new HikariDataSource(); }

```

YAML

```

citypulse:
  anpr:
    base-url: https://anpr.internal.trinity.com
    connect-timeout: 3s
    read-timeout: 45          # seconds because of @DurationUnit
    max-image-size: 5MB
    max-retries: 5
    camera-zones: [north, south, central]
    headers:
      X-API-Version: "2"
    retry:
      backoff: 500ms
      multiplier: 1.5

```

Usage: inject it like any bean.

```

@Service
public class AnprClient {
    private final RestClient restClient;
    public AnprClient(RestClient.Builder builder, AnprProperties props) {
        this.restClient = builder.baseUrl(props.baseUrl()).build();
    }
}

```

Validation: with `@Validated` and `spring-boot-starter-validation`, invalid config **fails startup** with a clear `BindValidationException` report. This is fail-fast on bad deployments. Use `@Valid` on nested objects so they are validated too.

Relaxed binding: all of these bind to `baseUrl: citypulse.anpr.base-url` (recommended kebab-case in files), `citypulse.anpr.baseUrl`, `citypulse.anpr.base_url`, and the env var `CITYPULSE_ANPR_BASEURL` (uppercase, `.` → `_`, dashes removed). List index from an env var: `CITYPULSE_ANPR_CAMERA_ZONES_0=north`. The prefix itself must be kebab-case, lowercase.

Metadata processor (IDE auto-complete and docs):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
```

This generates `META-INF/spring-configuration-metadata.json` at compile time, using Javadoc as descriptions. Add `additional-spring-configuration-metadata.json` for hints.

@ConfigurationProperties vs @Value

	@ConfigurationProperties	@Value
Relaxed binding	full	limited (kebab-case only)
Grouping/nesting, lists, maps	yes	awkward
Validation	@Validated	no
SpEL	no	yes (#{...})
IDE metadata	yes	no

Likely follow-ups:

- *How do you reload properties at runtime?* Spring Cloud `@RefreshScope` plus `/actuator/refresh`. `@ConfigurationProperties` beans are rebound on an `EnvironmentChangeEvent`.
- *Why records?* Immutability and thread safety, and nobody can mutate config at runtime.
- *How do you test properties binding?* Use `ApplicationContextRunner().withPropertyValues(...)`, or `@SpringBootTest(properties=...)`.

C22. Different environments (profiles, config precedence, Spring Cloud Config, secrets)

Profiles are named sets of configuration and beans: `dev`, `qa`, `uat`, `prod`, or per city, e.g. `blr`, `pune`.

1. Profile-specific files

```
src/main/resources/
  application.yml          # common defaults
  application-dev.yml
  application-qa.yml
  application-prod.yml
```

Activate one with `--spring.profiles.active=prod`, the env var `SPRING_PROFILES_ACTIVE=prod`, or `-Dspring.profiles.active=prod`. Profile-specific files **override** `application.yml`. If several profiles are active, the last one wins for conflicting keys.

2. Multi-document YAML with `spring.config.activate.on-profile` (Boot 2.4+; replaces the deprecated `spring.profiles` key)

```
spring:
  application:
    name: citypulse-incident
  datasource:
    url: jdbc:postgresql://localhost:5432/citypulse
---
spring:
  config:
    activate:
      on-profile: prod
  datasource:
    url: jdbc:postgresql://prod-db.internal:5432/citypulse
    hikari:
      maximum-pool-size: 30
---
spring:
  config:
    activate:
      on-profile: "dev | qa"      # profile expressions: !, &, |
logging:
  level:
    com.trinity: DEBUG
```

3. Profile groups (Boot 2.4+): one name activates several profiles.

```
spring:
  profiles:
```

```
group:
  prod: [proddb, prodkafka, monitoring]
  dev: [h2, localkafka]
```

4. Profile-conditional beans

```
@Bean @Profile("!prod") SmsSender fakeSmsSender() { return new LoggingSmsSender(); }
@Bean @Profile("prod") SmsSender realSmsSender(Msg91Client c) { return new Msg91SmsSender(c); }
```

`spring.profiles.default` sets the profile used when none is active. Don't use `spring.profiles.active` inside a profile-specific document; Boot rejects that.

Externalized config precedence (later overrides earlier: lowest → highest)

1. Default properties (`SpringApplication.setDefaultProperties`)
2. `@PropertySource` on `@Configuration` classes
3. **Config data files**, in this order: a. `application.yml` inside the jar b. `application-{profile}.yml` inside the jar c. `application.yml` **outside** the jar (`./config/`, `./`) d. `application-{profile}.yml` outside the jar
4. `RandomValuePropertySource` (`${random.uuid}`)
5. **OS environment variables**
6. **Java system properties** (`-D`)
7. JNDI, then `ServletContext` and `ServletConfig` init params
8. `SPRING_APPLICATION_JSON`
9. **Command-line arguments** (`--server.port=9090`)
10. Test properties: `@SpringBootTest(properties=...)`, `@TestPropertySource`, `@DynamicPropertySource` (highest)

The rule of thumb: **command line > system props > env vars > external files > packaged files > defaults**. `/actuator/env` shows which property source each value came from.

Importing extra config:

```
spring:
  config:
    import:
      - optional:file:./secrets.yml
      - optional:configtree:/run/secrets/          # K8s secret volume: each file = one property
      - optional:configserver:http://config-server:8888
```

Spring Cloud Config

- A central **Config Server** (`@EnableConfigServer`) is backed by **Git**, Vault, JDBC or the filesystem. It serves `/application/{profile}/{label}`, e.g. `/citypulse-incident/prod/main`.
- Clients use `spring-cloud-starter-config` plus `spring.config.import=configserver:http://config-server:8888`, with `spring.application.name` and the profile selecting files.
- **Refresh**: `@RefreshScope` beans and `@ConfigurationProperties` get rebound after `POST /actuator/refresh`. **Spring Cloud Bus** (Kafka/RabbitMQ) broadcasts the refresh to all instances (`/actuator/busrefresh`).
- Encrypted values use `{cipher}...`, decrypted by the server (symmetric key or keystore).
- On Kubernetes, ConfigMaps (plus Spring Cloud Kubernetes or `configtree`) are a common alternative.

Secrets handling

- **Never commit secrets** to Git (`application-prod.yml` should contain no passwords). Scan with `git-secrets` or `gitleaks` in CI.
- Inject them at runtime:
 - **Env vars** from K8s Secrets: `SPRING_DATASOURCE_PASSWORD`
 - Mounted secret files with `configtree`:
 - **HashiCorp Vault** (`spring-cloud-starter-vault-config`, dynamic DB credentials), AWS Secrets Manager or Azure Key Vault
 - `Jasypt ENC(...)` for encrypted values in files (the key comes from an env var)
- Use placeholders with no default for secrets: `password: ${DB_PASSWORD}`. Startup then fails if the value is missing.
- Actuator `/env` masks keys like `password`, `secret` and `key` (`management.endpoint.env.show-values=never|when-authorized`, Boot 3).

Likely follow-ups:

- *How do you see which profile is active?* The startup log line ("The following 1 profile is active: prod"), `Environment.getActiveProfiles()`, or `/actuator/env`.
- *Env var vs application-prod.yml: which wins?* The env var.

- Should you build different jars per environment? No. Build once and configure per environment (12-factor). The same artifact is promoted through dev → qa → prod.

C23. SessionFactory: what it is and how to configure it

SessionFactory (Hibernate native API):

- It is a **heavyweight, thread-safe, immutable** factory, created **once per database** at startup. It holds the compiled mappings, metadata, the connection provider, the 2nd-level cache and the statistics.
- It produces **Session s**, which are lightweight, **not thread-safe**, and one per unit of work or transaction. Each **Session** wraps the persistence context (first-level cache).
- JPA equivalents: **SessionFactory** ↔ **EntityManagerFactory**, and **Session** ↔ **EntityManager**. Since Hibernate 5.2, **SessionFactory** extends **EntityManagerFactory** and **Session** extends **EntityManager**.

Plain Spring (no Boot): LocalSessionFactoryBean

```
package com.trinity.legacy.config;

import com.zaxxer.hikari.HikariDataSource;
import org.hibernate.SessionFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.hibernate5.HibernateTransactionManager;
import org.springframework.orm.hibernate5.LocalSessionFactoryBean;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;
import java.util.Properties;

@Configuration
@EnableTransactionManagement
public class HibernateConfig {

    @Bean
    public DataSource dataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:postgresql://localhost:5432/citypulse");
        ds.setUsername("app");
        ds.setPassword(System.getenv("DB_PASSWORD"));
        return ds;
    }

    @Bean
    public LocalSessionFactoryBean sessionFactory(DataSource dataSource) {
        LocalSessionFactoryBean sf = new LocalSessionFactoryBean();
        sf.setDataSource(dataSource);
        sf.setPackagesToScan("com.trinity.legacy.domain");
        Properties props = new Properties();
        props.put("hibernate.hbm2ddl.auto", "validate");
        props.put("hibernate.show_sql", "false");
        props.put("hibernate.jdbc.batch_size", "50");
        sf.setHibernateProperties(props);
        return sf; // FactoryBean -> exposes SessionFactory
    }

    @Bean
    public HibernateTransactionManager transactionManager(SessionFactory sessionFactory) {
        return new HibernateTransactionManager(sessionFactory);
    }
}
```

(The package `org.springframework.orm.hibernate5` is still the one Spring 6.x uses for Hibernate 5.2+ and 6.x native support. A DAO then uses `sessionFactory.getCurrentSession()` inside `@Transactional`.)

Spring Boot: there is **no SessionFactory bean by default**. `HibernateJpaAutoConfiguration` creates a `LocalContainerEntityManagerFactoryBean` (an `EntityManagerFactory`) and a `JpaTransactionManager`. Configure it through properties:

```
spring:
  jpa:
    hibernate:
      ddl-auto: validate          # none | validate | update | create | create-drop (use Flyway/Liquibase in prod)
    open-in-view: false
    properties:
      hibernate:
        jdbc.batch_size: 50
```

```
default_batch_fetch_size: 50
generate_statistics: false
```

When you need the native API, **unwrap** it:

```
@Service
public class BulkSensorIngestService {

    private final EntityManagerFactory emf;
    @PersistenceContext private EntityManager em;

    public BulkSensorIngestService(EntityManagerFactory emf) { this.emf = emf; }

    public SessionFactory sessionFactory() {
        return emf.unwrap(SessionFactory.class); // e.g. for Statistics, StatelessSession
    }

    @Transactional
    public void useNativeSession() {
        Session session = em.unwrap(Session.class); // the transaction-bound Session
        session.setJdbcBatchSize(100);
    }

    public void statelessInsert(List<SensorReading> readings) { // high-volume IoT ingest, no persistence context
        try (StatelessSession ss = sessionFactory().openStatelessSession()) {
            Transaction tx = ss.beginTransaction();
            readings.forEach(ss::insert);
            tx.commit();
        }
    }
}
```

Likely follow-ups:

- *getCurrentSession vs openSession?* `getCurrentSession` is bound to the current transaction and closed automatically. `openSession` is a new one that you must close.
- *Why is SessionFactory expensive?* It parses all mappings, builds metamodels and SQL, and initialises caches. Do it once.
- *What is a StatelessSession?* It has no first-level cache, dirty checking or cascades. It suits bulk inserts.

C24. Multiple data sources

Scenario: the **primary** DB (Postgres, citizen incidents) plus a **reporting** DB (a MySQL warehouse). Each needs its own **DataSource** → **EntityManagerFactory** → **TransactionManager**, and the **repositories and entities go in separate packages**.

```
com.trinity.citypulse
├─ primary/  entity/Incident.java    repository/IncidentRepository.java
└─ reporting/ entity/DailyStat.java  repository/DailyStatRepository.java
```

```
app:
  datasource:
    primary:
      url: jdbc:postgresql://pg:5432/citypulse
      username: app
      password: ${PRIMARY_DB_PASSWORD}
      driver-class-name: org.postgresql.Driver
      hikari:
        maximum-pool-size: 20
        pool-name: primary-pool
    reporting:
      url: jdbc:mysql://mysql:3306/reporting
      username: report
      password: ${REPORT_DB_PASSWORD}
      driver-class-name: com.mysql.cj.jdbc.Driver
      hikari:
        maximum-pool-size: 5
        pool-name: reporting-pool
        read-only: true
  spring:
    jpa:
      open-in-view: false
```

Primary config

```
package com.trinity.citypulse.config;
```

```

import com.zaxxer.hikari.HikariDataSource;
import jakarta.persistence.EntityManagerFactory;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.boot.autoconfigure.jdbc.DataSourceProperties;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.orm.jpa.EntityManagerFactoryBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;
import java.util.Map;

@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(
    basePackages = "com.trinity.citypulse.primary.repository",
    entityManagerFactoryRef = "primaryEntityManagerFactory",
    transactionManagerRef = "primaryTransactionManager")
public class PrimaryDataSourceConfig {

    @Bean
    @Primary
    @ConfigurationProperties("app.datasource.primary")
    public DataSourceProperties primaryDataSourceProperties() { return new DataSourceProperties(); }

    @Bean
    @Primary
    @ConfigurationProperties("app.datasource.primary.hikari") // binds pool settings onto HikariDataSource
    public HikariDataSource primaryDataSource(
        @Qualifier("primaryDataSourceProperties") DataSourceProperties props) {
        return props.initializeDataSourceBuilder().type(HikariDataSource.class).build();
    }

    @Bean
    @Primary
    public LocalContainerEntityManagerFactoryBean primaryEntityManagerFactory(
        EntityManagerFactoryBuilder builder, @Qualifier("primaryDataSource") DataSource dataSource) {
        return builder.dataSource(dataSource)
            .packages("com.trinity.citypulse.primary.entity")
            .persistenceUnit("primary")
            .properties(Map.of(
                "hibernate.hbm2ddl.auto", "validate",
                "hibernate.physical_naming_strategy",
                "org.hibernate.boot.model.naming.CamelCaseToUnderscoresNamingStrategy"))
            .build();
    }

    @Bean
    @Primary
    public PlatformTransactionManager primaryTransactionManager(
        @Qualifier("primaryEntityManagerFactory") EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}

```

Reporting config (same shape, no @Primary)

```

package com.trinity.citypulse.config;

import com.zaxxer.hikari.HikariDataSource;
import jakarta.persistence.EntityManagerFactory;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.boot.autoconfigure.jdbc.DataSourceProperties;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.orm.jpa.EntityManagerFactoryBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.transaction.PlatformTransactionManager;

import javax.sql.DataSource;
import java.util.Map;

@Configuration

```

```

@EnableJpaRepositories(
    basePackages = "com.trinity.citypulse.reporting.repository",
    entityManagerFactoryRef = "reportingEntityManagerFactory",
    transactionManagerRef = "reportingTransactionManager")
public class ReportingDataSourceConfig {

    @Bean
    @ConfigurationProperties("app.datasource.reporting")
    public DataSourceProperties reportingDataSourceProperties() { return new DataSourceProperties(); }

    @Bean
    @ConfigurationProperties("app.datasource.reporting.hikari")
    public HikariDataSource reportingDataSource(
        @Qualifier("reportingDataSourceProperties") DataSourceProperties props) {
        return props.initializeDataSourceBuilder().type(HikariDataSource.class).build();
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean reportingEntityManagerFactory(
        EntityManagerFactoryBuilder builder, @Qualifier("reportingDataSource") DataSource dataSource) {
        return builder.dataSource(dataSource)
            .packages("com.trinity.citypulse.reporting.entity")
            .persistenceUnit("reporting")
            .properties(Map.of("hibernate.hbm2ddl.auto", "none"))
            .build();
    }

    @Bean
    public PlatformTransactionManager reportingTransactionManager(
        @Qualifier("reportingEntityManagerFactory") EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}

```

Usage

```

@Service
public class DashboardService {
    private final IncidentRepository incidents; // primary
    private final DailyStatRepository stats; // reporting

    public DashboardService(IncidentRepository incidents, DailyStatRepository stats) {
        this.incidents = incidents; this.stats = stats;
    }

    @Transactional("primaryTransactionManager") // default anyway (@Primary)
    public void closeIncident(Long id) { incidents.findById(id).ifPresent(Incident::close); }

    @Transactional(transactionManager = "reportingTransactionManager", readOnly = true)
    public List<DailyStat> lastWeek() { return stats.findTop7ByOrderByDayDesc(); }
}

```

Gotchas

- `@Primary` is required on one set. Otherwise auto-configured consumers (`JdbcTemplate` , `Flyway` , `Actuator` health) hit `NoUniqueBeanDefinitionException` .
- Boot's `spring.jpa.*` naming strategy and `ddl-auto` are applied to the EMF built by `EntityManagerFactoryBuilder` only partially. Pass Hibernate settings explicitly in `properties(...)` , as above, for consistent naming.
- With `DataSourceProperties` , use `url` (it is converted to Hikari's `jdbcUrl`). If you bind `HikariDataSource` directly with `@ConfigurationProperties` , the property must be `jdbc-url` .
- **A `@Transactional` spanning both DBs is NOT atomic.** Each transaction manager handles only its own DB. Options: JTA/XA (Atomikos/Narayana, heavy), or more commonly an **outbox/saga with eventual consistency**.
- Flyway migrates only the primary `DataSource` unless you configure a second `Flyway` bean.
- For read/write splitting of the same schema (primary + replica), use `AbstractRoutingDataSource` with `readOnly` -based routing (wrapped in `LazyConnectionDataSourceProxy`) rather than two EMFs.

Likely follow-ups:

- *How does a repository know which EMF to use?* Through `entityManagerFactoryRef` in its package's `@EnableJpaRepositories` .
- *How do you inject an EntityManager for the second DB?* `@PersistenceContext(unitName = "reporting")` .
- *Multi-tenant DB per city?* Hibernate multi-tenancy (`MultiTenantConnectionProvider` + `CurrentTenantIdentifierResolver`) or a routing `DataSource`.

C25. Lazy vs eager loading

Defaults (JPA)

Association	Default fetch
@ManyToOne , @OneToOne	EAGER
@OneToMany , @ManyToMany	LAZY
@ElementCollection	LAZY
@Basic fields	EAGER (lazy basics need bytecode enhancement)

Best practice: make everything LAZY (@ManyToOne(fetch = FetchType.LAZY)) and fetch explicitly per use case. EAGER can't be turned off per query, and it causes hidden extra queries (eager @ManyToOne under JPQL triggers N+1).

How lazy works

- For to-one: Hibernate puts a **proxy** (a ByteBuddy subclass, Employee\$HibernateProxy\$...) holding only the id. The first call to a non-id getter initialises it with a SELECT.
- For collections: a PersistentBag / PersistentSet wrapper loads on first access (size() , iteration).
- Initialisation needs an **open Session/persistence context**.

LazyInitializationException

```
@Service
public class DeptService {
    @Transactional(readOnly = true)
    public Department get(Long id) { return repo.findById(id).orElseThrow(); }
}
// controller:
dept.getEmployees().size(); // tx closed -> LazyInitializationException: could not initialize proxy - no Session
// Also triggered by Jackson serialising the entity outside the transaction.
```

Fixes:

- fetch what you need inside the transaction (JOIN FETCH or an entity graph);
- map to a DTO inside the service;**
- hibernate.initialize(dept.getEmployees()) (crude).

Anti-fixes: making things EAGER, hibernate.enable_lazy_load_no_trans=true (opens a new session per lazy load), or relying on OSIV.

Open Session in View (spring.jpa.open-in-view)

- Boot sets it to **true by default** and logs a warning. OpenEntityManagerInViewInterceptor keeps the EntityManager open for the whole web request, so lazy loads in the controller or Jackson "work".
- The cost: **DB connections are held for the entire request**, including slow serialisation and remote calls, which can exhaust the pool under load. Queries also fire from the view layer (hidden N+1), and the transaction boundary is blurred.
- Set spring.jpa.open-in-view=false** in services and fetch deliberately.

The N+1 problem

```
List<Department> depts = deptRepo.findAll(); // 1 query
depts.forEach(d -> d.getEmployees().size()); // N queries (one per dept)
```

Solutions:

```
public interface DepartmentRepository extends JpaRepository<Department, Long> {

    // 1. JOIN FETCH (DISTINCT is implicit for entity results in Hibernate 6)
    @Query("select d from Department d left join fetch d.employees where d.city = :city")
    List<Department> findWithEmployees(@Param("city") String city);

    // 2. Entity graph
    @EntityGraph(attributePaths = {"employees", "head"})
    List<Department> findByCity(String city);

    // 3. DTO projection: no entities, no lazy problems
    @Query("""
        select new com.trinity.hr.dept.DeptSummary(d.id, d.name, count(e))
        from Department d left join d.employees e group by d.id, d.name
        """)
```

```
List<DeptSummary> summaries();
}
```

```
# 4. Batch fetching: N queries -> N/50 queries using IN (...)
spring.jpa.properties.hibernate.default_batch_fetch_size: 50
```

5. `@BatchSize(size = 50)` on the collection or entity class, or `@Fetch(FetchMode.SUBSELECT)`.

Pagination gotcha: `JOIN FETCH` of a collection combined with `Pageable` makes Hibernate warn **HHH90003004** ("firstResult/maxResults specified with collection fetch; applying in memory"). It loads every row and paginates in memory. The fix: page the ids first (`select d.id from Department d ...`), then fetch those ids with `JOIN FETCH`. Or use batch fetching.

Fetching two bags (two `List` collections) in one query throws `MultipleBagFetchException`. Use `Set`, or fetch them in separate queries (Hibernate dedupes them in the persistence context).

How to detect N+1: `spring.jpa.properties.hibernate.generate_statistics=true`, `logging.level.org.hibernate.SQL=DEBUG`, `datasource-proxy` or `p6spy`, and asserting query counts in tests (e.g. `hypersistence-utils` / `SQLStatementCountValidator`).

Likely follow-ups:

- *Is FetchType.LAZY guaranteed?* No. It is a hint. Lazy `@OneToOne` on the non-owning side needs bytecode enhancement or `@MapsId`.
- *Why is getReferenceById lazy?* It returns an uninitialised proxy without hitting the DB. That is handy for setting FKs.
- *JOIN FETCH vs EntityGraph?* They are equivalent SQL-wise. Entity graphs are declarative and reusable on derived queries.

C26. One-to-Many across multiple tables

Bidirectional Department (1) → Employee (N)

- The **owning side** is the side with the foreign key: `Employee.department` (`@ManyToOne` + `@JoinColumn`). Only the owning side's changes are written to the FK column.
- The **inverse side** is `Department.employees` (`@OneToMany(mappedBy = "department")`). It is a mirror for navigation.

Multi-level: Department → Employee → Address

```
package com.trinity.hr.domain;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

@Entity
@Table(name = "department")
public class Department {

    @Id @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Employee> employees = new ArrayList<>();

    protected Department() { }
    public Department(String name) { this.name = name; }

    // helper methods keep BOTH sides in sync (in-memory consistency + correct FK)
    public void addEmployee(Employee e) {
        employees.add(e);
        e.setDepartment(this);
    }
    public void removeEmployee(Employee e) {
        employees.remove(e);
        e.setDepartment(null); // with orphanRemoval -> DELETE employee
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public List<Employee> getEmployees() { return List.copyOf(employees); } // don't expose the mutable list
}

@Entity
@Table(name = "employee", indexes = @Index(name = "idx_employee_department", columnList = "department_id"))
class Employee {
```

```

@Id @GeneratedValue(strategy = GenerationType.SEQUENCE)
private Long id;

private String name;

@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "department_id", nullable = false, foreignKey = @ForeignKey(name = "fk_employee_department"))
private Department department; // OWNING side (has the FK)

@OneToMany(mappedBy = "employee", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Address> addresses = new ArrayList<>();

protected Employee() { }
public Employee(String name) { this.name = name; }

public void addAddress(Address a) { addresses.add(a); a.setEmployee(this); }
public void removeAddress(Address a) { addresses.remove(a); a.setEmployee(null); }

void setDepartment(Department d) { this.department = d; }
public Long getId() { return id; }
public Department getDepartment() { return department; }
public List<Address> getAddresses() { return addresses; }

// equals/hashCode on id with a constant hashCode: safe across transient -> managed transition
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee other)) return false;
    return id != null && Objects.equals(id, other.id);
}
@Override public int hashCode() { return getClass().hashCode(); }
}

@Entity
@Table(name = "address")
class Address {

    @Id @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;

    private String line1;
    private String city;
    private String pincode;

    @Enumerated(EnumType.STRING)
    private AddressType type; // HOME, OFFICE

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "employee_id", nullable = false)
    private Employee employee;

    protected Address() { }
    public Address(String line1, String city, String pincode, AddressType type) {
        this.line1 = line1; this.city = city; this.pincode = pincode; this.type = type;
    }
    void setEmployee(Employee e) { this.employee = e; }
}

enum AddressType { HOME, OFFICE }

```

Persisting the whole graph with one save (cascade):

```

@Transactional
public Long createDept() {
    Department d = new Department("Traffic Ops");
    Employee e = new Employee("Ravi");
    e.addAddress(new Address("12 MG Road", "Bengaluru", "560001", AddressType.HOME));
    d.addEmployee(e);
    return departmentRepository.save(d).getId(); // cascades to employees and addresses
}

```

Cascade and orphanRemoval

- `cascade = ALL` (PERSIST, MERGE, REMOVE, REFRESH, DETACH) belongs only on **parent** → **child (composition)**. **Never cascade from** `@ManyToOne` (removing an employee would delete the department) or on `@ManyToMany REMOVE`.
- `orphanRemoval = true` deletes a child **removed from the collection**. `CascadeType.REMOVE` deletes children only when the **parent** is deleted.
- Don't replace the collection (`setEmployees(new ArrayList<>())`) when `orphanRemoval` is on. Hibernate throws "A collection with `cascade=all-delete-orphan` was no longer referenced". Call `clear()` and then `addAll()` instead.

Unidirectional @OneToMany and join tables

```

@OneToMany // no mappedBy, no @JoinColumn -> Hibernate creates JOIN TABLE department_employees
private List<Employee> employees;

@OneToMany
@JoinColumn(name = "department_id") // unidirectional with FK in child: extra UPDATE statements to set FK
private List<Employee> employees;

@ManyToMany
@JoinTable(name = "employee_project",
    joinColumns = @JoinColumn(name = "employee_id"),
    inverseJoinColumns = @JoinColumn(name = "project_id"))
private Set<Project> projects = new HashSet<>();

```

A unidirectional `@OneToMany` without `@JoinColumn` produces an unexpected join table and inefficient SQL. **Prefer bidirectional, or plain `@ManyToOne` alone.** For a join table with extra columns (e.g. `assigned_on`), model it as an entity (`EmployeeProject`) with two `@ManyToOne` s.

Fetching multi-level data

```

@Query("select distinct d from Department d left join fetch d.employees where d.id = :id")
Optional<Department> findWithEmployees(Long id);
// then addresses via default_batch_fetch_size, or a 2nd query:
@Query("select distinct e from Employee e left join fetch e.addresses where e.department.id = :deptId")
List<Employee> findWithAddresses(Long deptId);

```

Fetching `employees` and `addresses` (two `List` bags) in one query gives `MultipleBagFetchException` and a Cartesian product. Use two queries.

Gotchas

- Lombok `@Data` / `@ToString` / `@EqualsAndHashCode` on bidirectional entities cause **StackOverflowError** (recursive `toString`/`hashCode`) and trigger lazy loads.
- Jackson on bidirectional entities recurses infinitely. Use DTOs, or `@JsonManagedReference` / `@JsonBackReference` (or `@JsonIgnore`).
- Index FK columns. Postgres doesn't index them automatically.

Likely follow-ups:

- *Who owns the relationship?* The side with `@JoinColumn` (the FK). `mappedBy` marks the inverse side.
- *Why helper methods?* So both sides stay consistent in memory. Setting only the inverse side writes no FK.
- *List vs Set for collections?* A `Set` avoids `MultipleBagFetchException` and duplicates, but needs correct `equals`/`hashCode`. A `List` keeps order (`@OrderColumn` / `@OrderBy`).

C27. Audit logs

There are four levels of auditing, from simple to enterprise.

1. Spring Data JPA auditing ("who and when" columns)

```

package com.trinity.citypulse.audit;

import jakarta.persistence.*;
import org.springframework.data.annotation.*;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import java.time.Instant;

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class Auditable {

    @CreatedBy @Column(updatable = false) protected String createdBy;
    @CreatedDate @Column(updatable = false) protected Instant createdAt;
    @LastModifiedBy protected String updatedBy;
    @LastModifiedDate protected Instant updatedAt;
}

```

```

package com.trinity.citypulse.audit;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.domain.AuditorAware;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;
import org.springframework.security.authentication.AnonymousAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;

```

```
import java.util.Optional;

@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorAware")
public class AuditConfig {

    @Bean
    public AuditorAware<String> auditorAware() {
        return () -> Optional.ofNullable(SecurityContextHolder.getContext().getAuthentication())
            .filter(Authentication::isAuthenticated)
            .filter(a -> !(a instanceof AnonymousAuthenticationToken))
            .map(Authentication::getName)           // JWT "sub"/preferred_username
            .or(() -> Optional.of("system"));       // schedulers, Kafka consumers
    }
}
```

`@Entity` public class Incident extends Auditable { ... }. The `AuditingEntityListener` fills the fields in `@PrePersist` / `@PreUpdate`. **Limit:** this tells you only the *latest* modifier. There is no history of values.

2. Hibernate Envers (full row history)

```
<dependency>
<groupId>org.hibernate.orm</groupId>
<artifactId>hibernate-envers</artifactId>
</dependency>
```

```
@Entity
@Audited                                     // creates incident_aud table: all columns + rev + revtype (0 add,1 mod,2 del)
public class Incident extends Auditable {
    @Id @GeneratedValue private Long id;
    private String status;
    @NotAudited private String transientNote;
}

@Entity
@RevisionEntity(UserRevisionListener.class)
@Table(name = "revinfo")
public class UserRevision extends DefaultRevisionEntity {    // add "who" to each revision
    private String username;
    public void setUsername(String u) { this.username = u; }
}

public class UserRevisionListener implements RevisionListener {
    @Override public void newRevision(Object rev) {
        var auth = SecurityContextHolder.getContext().getAuthentication();
        ((UserRevision) rev).setUsername(auth != null ? auth.getName() : "system");
    }
}
```

(Envers types are in `org.hibernate.envers.*`.) To query history, use

`AuditReaderFactory.get(em).createQuery().forRevisionsOfEntity(Incident.class, false, true)`, or Spring Data Envers'

`RevisionRepository<Incident, Long, Integer> (findRevisions(id))`. Envers writes in the **same transaction**, so it is consistent, but it adds write overhead.

3. AOP plus an async audit table (business-action audit: "who did what, through which API")

```
@Target(ElementType.METHOD) @Retention(RetentionPolicy.RUNTIME)
public @interface AuditAction { String value(); }           // e.g. "CHALLAN_CANCELLED"
```

```
package com.trinity.citypulse.audit;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.MDC;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;

import java.time.Instant;

@Aspect
@Component
public class AuditAspect {

    private final AuditWriter writer;
    private final ObjectMapper mapper;
```

```

public AuditAspect(AuditWriter writer, ObjectMapper mapper) { this.writer = writer; this.mapper = mapper; }

@Around("@annotation(audit)")
public Object around(ProceedingJoinPoint pjp, AuditAction audit) throws Throwable {
    // capture context ON the request thread (SecurityContext/MDC are ThreadLocal)
    var auth = SecurityContextHolder.getContext().getAuthentication();
    String user = auth != null ? auth.getName() : "system";
    String correlationId = MDC.get("correlationId");
    String args = safeJson(pjp.getArgs());
    long start = System.currentTimeMillis();
    try {
        Object result = pjp.proceed();
        writer.write(new AuditEvent(audit.value(), user, args, "SUCCESS", null,
            correlationId, Instant.now(), System.currentTimeMillis() - start));
        return result;
    } catch (Throwable ex) {
        writer.write(new AuditEvent(audit.value(), user, args, "FAILURE", ex.getMessage(),
            correlationId, Instant.now(), System.currentTimeMillis() - start));
        throw ex;
    }
}

private String safeJson(Object[] args) {
    try { return mapper.writeValueAsString(args); } catch (Exception e) { return "[unserializable]"; }
}
}

```

```

public record AuditEvent(String action, String username, String payload, String outcome, String error,
    String correlationId, Instant at, long durationMs) { }

@Component
public class AuditWriter {
    private final AuditLogRepository repo;
    public AuditWriter(AuditLogRepository repo) { this.repo = repo; }

    @Async("auditExecutor") // needs @EnableAsync + a bounded executor
    @Transactional(propagation = Propagation.REQUIRES_NEW) // audit survives business rollback
    public void write(AuditEvent e) { repo.save(AuditLog.from(e)); }
}

```

Points to mention:

- **Mask PII** in the payload (Aadhaar, phone numbers).
- Capture the user **before** going async. Otherwise use `DelegatingSecurityContextAsyncTaskExecutor`.
- Use a bounded queue and a rejection policy so audit can't take the app down.
- Make the audit table append-only (no UPDATE/DELETE grants) and partition it by month.

4. Kafka-based audit events (microservices / Smart City scale)

- Every service publishes `AuditEvent`s to an `audit-events` topic, keyed by entity id for ordering. A dedicated **audit service** consumes them into Elasticsearch/OpenSearch (search) or cold storage (S3/Parquet) for retention.
- **Consistency**: publish only after commit.
 - `@TransactionalEventListener(phase = AFTER_COMMIT)` sends to Kafka: simple, but the event is lost if the app crashes after commit.
 - Or use the **Transactional Outbox**: write an `outbox` row in the same DB transaction, and let a relay or **Debezium CDC** publish to Kafka. This gives at-least-once delivery, and the consumer must be idempotent (dedupe by event id).
- Benefits: decoupled, cross-service view, replayable, and no write latency on the business path.

```

@Component
public class AuditEventPublisher {
    private final KafkaTemplate<String, AuditEvent> kafka;
    public AuditEventPublisher(KafkaTemplate<String, AuditEvent> kafka) { this.kafka = kafka; }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void on(AuditEvent e) { kafka.send("audit-events", e.correlationId(), e); }
}
// in service: applicationEventPublisher.publishEvent(new AuditEvent(...));

```

Likely follow-ups:

- *JPA auditing vs Envers?* JPA auditing gives the latest who/when columns. Envers gives a full versioned history per row.
- *How is the user captured in Kafka consumers or schedulers?* There's no `SecurityContext`, so fall back to "system", or carry the user id in the message headers.
- *How do you make audit tamper-evident?* Append-only storage, a hash chain or signed records, WORM storage, and separate DB credentials.

C28. URL shortener design

Requirements

- Functional: shorten a long URL, redirect a short URL to the long one, optional custom alias, expiry, click analytics.
- Non-functional: **read-heavy** (about 100:1 redirect vs create), low-latency redirects (under 10 ms at p99 from cache), high availability, unguessable or short codes.
- Rough scale: 100M new URLs a month is about 40 writes/s, and about 4,000 redirects/s. Over 5 years that is about 6B rows at ~500 B each, roughly 3 TB.

API

```
POST /api/v1/urls {"longUrl": "...", "customAlias": "blr-traffic", "expiresAt": "2027-01-01T00:00:00Z"}
-> 201 {"shortUrl": "https://tmy.ly/aZ3kQ9", "code": "aZ3kQ9", "expiresAt": ...}
GET /{code} -> 302 Location: <longUrl> (404 if unknown, 410 Gone if expired)
GET /api/v1/urls/{code}/stats -> clicks, by day/country/referrer
DELETE /api/v1/urls/{code} -> 204
```

Code generation: option A, Base62 of a DB sequence or ID (recommended)

- Characters `[0-9a-zA-Z]`. $62^7 \approx 3.5$ trillion codes, so 7 characters is plenty.
- id $\rightarrow$ Base62 is a **bijection**, so there are **no collisions by construction**.
- Downside: sequential codes are **guessable/enumerable**. Fixes: start at a large offset, apply a reversible permutation or obfuscation of the id (Feistel cipher, Hashids/Sqids), or use a random 64-bit id.
- To scale writes: give each app instance a **range of ids** (e.g. reserve 10,000 at a time from the DB or Redis `INCRBY`), or use **Snowflake ids**. Then no DB round trip is needed per create.

```
public final class Base62 {
    private static final char[] ALPHABET =
        "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ".toCharArray();
    private Base62() {}

    public static String encode(long value) {
        if (value == 0) return "0";
        StringBuilder sb = new StringBuilder();
        while (value > 0) {
            sb.append(ALPHABET[(int) (value % 62)]);
            value /= 62;
        }
        return sb.reverse().toString();
    }

    public static long decode(String s) {
        long result = 0;
        for (char c : s.toCharArray()) {
            int d;
            if (c >= '0' && c <= '9') d = c - '0';
            else if (c >= 'a' && c <= 'z') d = c - 'a' + 10;
            else if (c >= 'A' && c <= 'Z') d = c - 'A' + 36;
            else throw new IllegalArgumentException("Invalid char " + c);
            result = result * 62 + d;
        }
        return result;
    }
}
```

Option B: hash the long URL (MD5/SHA-256/MurmurHash, Base62-encoded, first 7 characters)

- It is deterministic: the same URL gives the same code, so dedupe comes for free.
- Truncation causes collisions.** Seven Base62 characters carry only about 41.7 bits. By the birthday paradox, there is about a 50% chance of at least one collision after roughly 2 million URLs, and at billions of URLs collisions happen constantly. On collision, append a salt or counter and rehash, then retry.
- Every create needs a DB check.

Collision and duplicate handling

- A **unique index on short_code** is the final arbiter. Insert, and on `DataIntegrityViolationException` regenerate and retry (bounded, e.g. 3 times). Never "check then insert", which is a race.
- Same long URL submitted again:** keep a `long_url_hash` column (SHA-256) with a unique index (per user/tenant if needed), and **return the existing code** instead of creating a new one. Hash it because long URLs exceed index-length limits.
- Custom aliases** go through the same unique index. A conflict returns **409 Conflict**.

```

@Transactional
public ShortUrl shorten(String longUrl, String alias, Instant expiresAt) {
    String hash = sha256(normalize(longUrl));
    if (alias == null) {
        Optional<ShortUrl> existing = repo.findByLongUrlHash(hash);
        if (existing.isPresent()) return existing.get(); // idempotent create
    }
    long id = idGenerator.next(); // range-allocated
    String code = alias != null ? alias : Base62.encode(obfuscate(id));
    try {
        return repo.saveAndFlush(new ShortUrl(id, code, longUrl, hash, expiresAt));
    } catch (DataIntegrityViolationException e) {
        if (alias != null) throw new AliasTakenException(alias); // -> 409
        return repo.findByLongUrlHash(hash).orElseThrow(() -> e); // concurrent same-URL insert
    }
}

```

(In production, do the retry outside the failed transaction. After a constraint violation the persistence context is unusable, so the retry or lookup should run in a new transaction.)

Storage schema

```

CREATE TABLE short_url (
    id          BIGINT          PRIMARY KEY,
    short_code   VARCHAR(16)    NOT NULL,
    long_url     TEXT           NOT NULL,
    long_url_hash CHAR(64)      NOT NULL,
    owner_id     BIGINT,
    created_at   TIMESTAMPTZ    NOT NULL DEFAULT now(),
    expires_at   TIMESTAMPTZ,
    is_active    BOOLEAN        NOT NULL DEFAULT TRUE,
    CONSTRAINT uk_short_code UNIQUE (short_code),
    CONSTRAINT uk_long_hash  UNIQUE (long_url_hash)      -- or (owner_id, long_url_hash)
);
CREATE INDEX idx_short_url_expires ON short_url (expires_at) WHERE expires_at IS NOT NULL;

```

- It is a pure key-value access pattern, so you can also use Cassandra or DynamoDB at large scale (partition key = `short_code`). SQL is fine up to billions of rows with sharding by code.
- Redis cache:** `GET url:{code} → longUrl`, using cache-aside with a TTL equal to `min(expiry, 24h)`. Hot links follow a Zipf distribution, so an 80–90%+ hit ratio is realistic. Cache **negative lookups** ("NULL" for 1 minute) to stop penetration attacks with random codes, or put a Bloom filter of existing codes in front.

Redirect: 301 vs 302

	301 Moved Permanently	302 Found / 307 Temporary
Browser caches it	Yes, so later clicks never reach you	No, every click hits the server
Load	lower	higher
Analytics	lost after the first click	accurate
Changing the target or expiring	impossible for clients that cached it	works

Pick **302 (or 307)** when you need analytics, expiry or editable links, which covers most products. Pick 301 for pure, permanent redirects where you want minimum load and SEO link equity.

Expiry: check `expires_at` on read (return 410 Gone), set the Redis TTL to the expiry, and have a nightly batch job delete or archive expired rows. Recycling codes is risky, so avoid it.

Analytics: don't write to the DB on the redirect path. Publish a click event (`code`, timestamp, IP → geo, user agent, referrer) asynchronously to **Kafka**. Stream-process it (Kafka Streams/Flink) into aggregates in ClickHouse, Druid or a time-series store. Use a Redis `INCR` for a real-time counter.

Scale and architecture

```

Client -> CDN/edge (optionally caches 301s) -> LB -> stateless redirect service (N pods, HPA)
      -> Redis Cluster (cache) -> DB (primary + read replicas / sharded by code)
      -> Kafka (click events) -> analytics consumers -> OLAP store
Create path: API service -> ID range allocator -> DB (unique indexes) -> warm cache

```

- Split the read (redirect) and write (create) services, and scale them independently.
- Add rate limiting per API key/IP on create (Redis token bucket), malicious URL scanning (Safe Browsing API), and abuse protection.
- Multi-region: the cache is regional and the DB has global replication. Ids should be region-prefixed or Snowflake to avoid coordination.

Likely follow-ups:

- *Why not UUIDs as codes?* Too long (22+ characters in Base62). Short codes are the whole point.
- *How do you avoid a single point of failure for the counter?* Hand out id ranges per node, or use Snowflake (timestamp + node id + sequence).
- *How do you handle a viral link?* The Redis hot key plus a local Caffeine near-cache on each pod, and CDN caching for 301s.

C29. What does LOC mean?

It depends on the context, so clarify it in the interview ("Do you mean lines of code?").

1. **Lines of Code (software engineering):** a size metric for a codebase or a change.
 - Variants: **physical LOC** (all lines), **SLOC/NCLOC** (source lines excluding blanks and comments; SonarQube reports "Lines of Code" as NCLOC), and **logical LOC** (statements).
 - Uses: rough size and effort estimation (COCOMO), normalising defect density (bugs per KLOC), and tracking code growth or PR size (small PRs, roughly under 400 LOC, review better).
 - Limits: it **measures volume, not value or quality**. Fewer lines often means better code, and LOC varies by language and style. Never use it as a productivity metric. Better signals: cyclomatic or cognitive complexity, coverage, DORA metrics, and story or cycle-time throughput.
 - Tools: `cloc`, `tokei`, SonarQube, `git diff --stat`.
2. **Letter of Credit (banking/trade finance):** a bank's guarantee to pay a seller on behalf of a buyer once documents meeting the terms are presented (governed by UCP 600). Types include sight/usance, revocable/irrevocable, confirmed, standby LC (SBLC) and back-to-back. This matters if the interviewer's domain is BFSI. Messages travel over SWIFT (MT700 for issuance).
3. Others: **Line of Control** (geography), and **Locus of Control** (psychology).

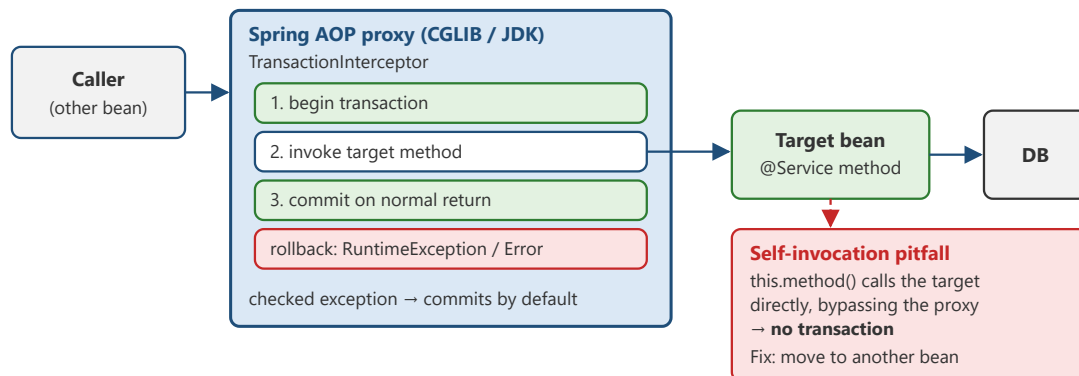
In a Smart City context it is almost always lines of code, e.g. "The incident service is about 25K LOC across 6 modules, with 80% coverage."

Likely follow-ups:

- *Is a higher LOC count good?* No. It means more to maintain. Value is delivered features at good quality.
- *What does SonarQube count as LOC?* Non-comment, non-blank lines (NCLOC).
- *KLOC?* Thousands of lines of code, e.g. defects per KLOC.

C30. @Transactional

@Transactional: How the Spring AOP Proxy Works



Propagation: @Transactional(propagation = ...)

Propagation	Behaviour
REQUIRED (default)	Join the existing transaction, or create a new one
REQUIRES_NEW	Suspend the outer transaction, always start a new one
NESTED	Savepoint inside the existing transaction; else like REQUIRED
SUPPORTS	Join if a transaction exists, else run non-transactionally
MANDATORY	Must run inside an existing transaction, else throw exception
NOT_SUPPORTED	Suspend any existing transaction, run non-transactionally
NEVER	Throw an exception if a transaction exists

Figure 13: @Transactional proxy mechanism and propagation types

How it works (proxy-based AOP)

- `@EnableTransactionManagement` (auto-enabled by Boot) registers `BeanFactoryTransactionAttributeSourceAdvisor` with `TransactionInterceptor`.
- At bean creation, the auto-proxy creator wraps `@Transactional` beans in a **CGLIB proxy** (C4).
- A call through the proxy goes to `TransactionInterceptor.invoke` → `TransactionAspectSupport.invokeWithinTransaction`:
 1. read the `TransactionAttribute` (propagation, isolation, timeout, readOnly, rollback rules);
 2. `PlatformTransactionManager.getTransaction(def)` . `JpaTransactionManager` begins the JPA transaction, takes a JDBC connection, and binds the `EntityManagerHolder` / `ConnectionHolder` to the thread through `TransactionSynchronizationManager (ThreadLocal)`;
 3. invoke the target method;
 4. on success, `commit` (flush, then commit). On an exception, check `rollbackOn(ex)` and **roll back**, or commit if the rule says so;
 5. clean up and unbind.

Propagation

Propagation	Existing tx present	No tx present
REQUIRED (default)	join it	create a new one
REQUIRES_NEW	suspend it and create an independent new one (a new connection!)	create a new one
NESTED	savepoint inside the existing tx (rolls back to the savepoint only). Supported with JDBC <code>DataSourceTransactionManager</code> ; limited with JPA	create a new one
SUPPORTS	join it	run non-transactionally
NOT_SUPPORTED	suspend it, run non-transactionally	run non-transactionally
MANDATORY	join it	throw <code>IllegalTransactionStateException</code>
NEVER	throw	run non-transactionally

The use case for `REQUIRES_NEW` is audit or failure logging that must persist even when the outer transaction rolls back. The cost: it holds 2 connections per thread, so under load you risk pool exhaustion or deadlock.

Isolation levels

Level	Dirty read	Non-repeatable read	Phantom
READ_UNCOMMITTED	possible	possible	possible
READ_COMMITTED (Postgres/Oracle/SQL Server default)	no	possible	possible
REPEATABLE_READ (MySQL InnoDB default)	no	no	possible (InnoDB prevents most with next-key locks)
SERIALIZABLE	no	no	no

`Isolation.DEFAULT` uses the DB default. For counters and inventory (e.g. parking slot booking), prefer **optimistic locking** (`@Version`) or `SELECT ... FOR UPDATE ( @Lock(PESSIMISTIC_WRITE) )` over raising the isolation level globally. Note: `JpaTransactionManager` supports a custom isolation level only through `HibernateJpaDialect`.

Rollback rules

- By default it **rolls back on unchecked exceptions** (`RuntimeException`) and `Error`, and **commits on checked exceptions** (an EJB heritage).
- `@Transactional(rollbackFor = Exception.class)` OR `noRollbackFor = BusinessException.class`.
- Catching the exception inside the method means **no rollback**, unless you rethrow or call `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`.
- **UnexpectedRollbackException**: an inner `REQUIRED` method throws, so the shared transaction is marked rollback-only. The outer method catches the exception and tries to commit, and Spring throws "Transaction silently rolled back because it has been marked as rollback-only". Fix it with `REQUIRES_NEW` for the inner call, by not catching, or by redesigning.

readOnly = true

- Hibernate sets the session to `FlushMode.MANUAL` and read-only, so **no dirty-checking snapshots** are kept (less memory and CPU).
- It calls `Connection.setReadOnly(true)` as a hint to the driver or DB, which can route to a replica with a routing `DataSource`.
- It does **not** prevent writes in all DBs. Native queries may still write.

The self-invocation pitfall

```

@Service
public class ChallanService {
    public void processBatch(List<Long> ids) {
        ids.forEach(this::processOne); // 'this' = target, NOT the proxy -> no transaction!
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void processOne(Long id) { ... }
}

```

Fixes:

1. **Move processOne to another bean** (the cleanest fix).
2. Self-injection: `@Lazy @Autowired private ChallanService self;` then `self.processOne(id)`.
3. **TransactionTemplate** (programmatic):

```

private final TransactionTemplate txTemplate; // inject PlatformTransactionManager, then new TransactionTemplate(tm)
public void processBatch(List<Long> ids) {
    ids.forEach(id -> txTemplate.executeWithoutResult(status -> processOneInternal(id)));
}

```

4. AspectJ weaving mode (`@EnableTransactionManagement(mode = AdviceMode.ASPECTJ)`), which has no proxy limitations but needs build or load-time weaving.

Method visibility

- **Private methods are never transactional.** The proxy can't override them, and Spring silently ignores the annotation.
- Since **Spring 6.0**, `protected` and package-private methods work with CGLIB class proxies. Before 6.0, only `public` methods worked.
- `final` methods can't be overridden either, so no transaction. The same goes for `static` methods.

Other gotchas

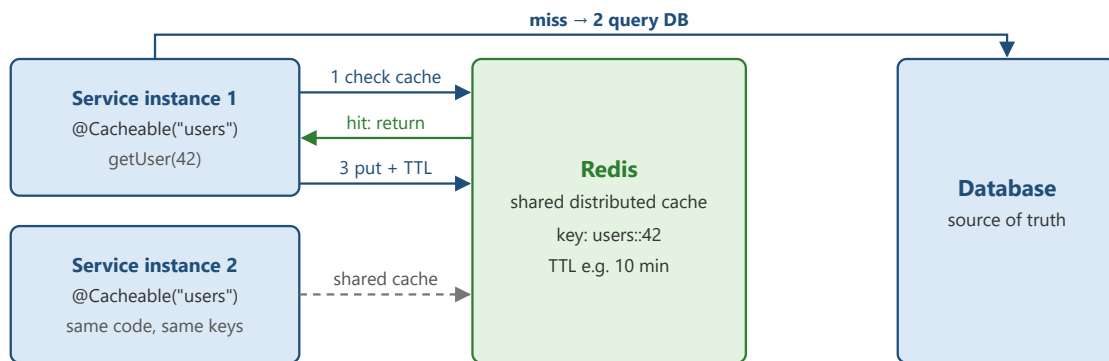
- `@Transactional` on a `@PostConstruct` method has no effect (C5).
- `@Async` plus `@Transactional`: the transaction is on the async thread. `ThreadLocal` transactions don't cross threads.
- A long transaction holding a DB connection while calling remote APIs: do the HTTP calls outside the transaction.
- `timeout = 5` (seconds) sets the query timeout.
- In tests, `@Transactional` on a test class **rolls back after each test** by default.

Likely follow-ups:

- *Class-level vs method-level @Transactional?* The method-level annotation overrides the class-level one. Class level applies to all methods the proxy can intercept.
- *Does @Transactional work on interfaces?* Yes, since Spring 5, also with class proxies. Annotating the concrete class is still recommended.
- *How do you run code after a commit?* `@TransactionalEventListener(phase = AFTER_COMMIT)`, or `TransactionSynchronizationManager.registerSynchronization`.

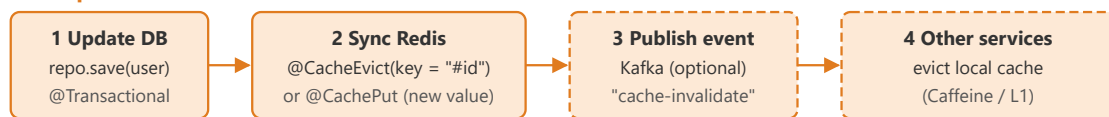
C31. @Cacheable, caching performance, and cache consistency across services

Cache-aside with @Cacheable across multiple instances



Read path: 1 check cache → hit: return; miss → 2 query DB → 3 put in cache with TTL, then return

Write path



Prefer @CacheEvict over @CachePut on writes: avoids racing a stale value into the cache.
Evict after the DB commit; TTL bounds staleness if an eviction is ever lost.

Figure 14: Cache-aside with @Cacheable across multiple service instances

Spring Cache abstraction: @EnableCaching plus a CacheManager (Caffeine, Redis, JCache/Ehcache, Hazelcast). It is proxy-based like @Transactional, so it has the **same self-invocation and private-method pitfalls**.

```
@Service
public class JunctionService {

    @Cacheable(cacheNames = "junction", key = "#id", unless = "#result == null")
    public JunctionDto get(Long id) { return mapper.toDto(repo.findById(id).orElseThrow()); }

    @Cacheable(cacheNames = "junctionsByWard", key = "#wardId", sync = true) // sync: one loader per key per JVM
    public List<JunctionDto> byWard(String wardId) { ... }

    @CachePut(cacheNames = "junction", key = "#result.id()") // always executes, updates cache
    public JunctionDto update(Long id, UpdateJunction cmd) { ... }

    @Caching(evict = {
        @CacheEvict(cacheNames = "junction", key = "#id"),
        @CacheEvict(cacheNames = "junctionsByWard", allEntries = true) })
    public void delete(Long id) { ... }
}
```

Attributes:

- **key** (SpEL; the default is SimpleKeyGenerator over all args), keyGenerator
- **condition** (evaluated before the call), **unless** (evaluated after, on #result)
- **sync**
- **beforeInvocation** on evict (evict even if the method throws)

Local cache: Caffeine (single instance, or data that is OK to diverge)

```
spring:
  cache:
    type: caffeine
    cache-names: junction,junctionsByWard
  caffeine:
    spec: maximumSize=10000,expireAfterWrite=10m,recordStats
```

Distributed cache: Redis, with per-cache TTL and JSON serialisation

```
package com.trinity.citypulse.config;
```

```

import org.springframework.boot.autoconfigure.cache.RedisCacheManagerBuilderCustomizer;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializationContext.SerializationPair;

import java.time.Duration;

@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public RedisCacheConfiguration defaultCacheConfig() {
        return RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues()
            .prefixCacheNameWith("citypulse:v1:") // versioned key prefix
            .serializeValuesWith(SerializationPair.fromSerializer(new GenericJackson2JsonRedisSerializer()));
    }

    @Bean
    public RedisCacheManagerBuilderCustomizer perCacheTtl(RedisCacheConfiguration base) {
        return builder -> builder
            .withCacheConfiguration("junction", base.entryTtl(Duration.ofMinutes(30)))
            .withCacheConfiguration("liveSignalState", base.entryTtl(Duration.ofSeconds(15)))
            .transactionAware(); // put/evict only after the tx commits
    }
}

```

Avoiding DB performance problems with a cache, and the classic failure modes

Problem	What happens	Mitigation
Stampede / thundering herd / dog-piling	A hot key expires, and 1,000 concurrent requests all hit the DB	<code>sync = true</code> (per JVM); a distributed lock (Redisson) so one rebuilds while others wait or serve stale data; early probabilistic refresh ; Caffeine <code>refreshAfterWrite</code>
Penetration	Requests for non-existent keys always miss and hit the DB (attacks, bad ids)	Cache nulls briefly; a Bloom filter of valid ids; input validation and rate limits
Avalanche	Many keys expire at the same moment, or Redis goes down, and load spikes	TTL jitter (<code>ttl + random(0..60s)</code>); a multi-level cache (Caffeine + Redis); circuit breaker or fallback; Redis HA (Sentinel/Cluster)
Hot key	One key overloads one Redis shard	A local near-cache; replicate the key under suffixes
Big values	Slow serialisation, network saturation	Cache DTOs not entities, compress, paginate
Sizing	Unbounded caches lead to OOM	<code>maximumSize / maximumWeight</code> , Redis <code>maxmemory + allkeys-lru / lfu</code> , and monitoring hit ratio and evictions via Actuator <code>cache.*</code> metrics

What to cache: read-mostly reference data (wards, junction metadata, device configs), expensive aggregations, and remote API responses. Don't cache highly volatile or per-user sensitive data without careful keys and TTLs. **Never cache JPA entities** (lazy proxies, detached state).

Cache consistency across multiple instances and microservices

The problem: with a **local** cache, instance A updates the DB and evicts its own cache, but instances B and C keep serving stale data until their TTL expires.

Strategies:

1. **A centralised distributed cache (Redis)**: every instance reads and evicts the same entry, so instance divergence goes away.
2. **Cache-aside with evict-on-write**: write to the DB, **then delete** the cache key (`@CacheEvict`). Don't update the cache: concurrent writers can leave a stale value. Do it **after commit** (`transactionAware()` or `@TransactionalEventListener(AFTER_COMMIT)`). Otherwise a reader can re-populate the old value between the eviction and the commit.
 - Remaining race: a reader misses, loads the old value, the writer commits and evicts, then the reader writes the old value. This is rare. Mitigations are a **delayed double delete** (evict again after ~500 ms) and short TTLs.
3. **Write-through / write-behind**: the cache layer writes to the DB synchronously (through) or asynchronously (behind, which risks data loss). It suits cache-first systems (Hazelcast/Redis Gears) and is less common with Spring's annotations.
4. **Two-level (Caffeine + Redis) with broadcast invalidation**: on write, publish `{cache, key}` on **Redis Pub/Sub** or a **Kafka** topic. Every instance subscribes and evicts its local L1 entry.
5. **Cross-microservice ownership**: service X caches data owned by service Y. Y publishes domain events (`JunctionUpdated`) to Kafka, and X's consumer evicts or refreshes its cache. Even better is **CDC (Debezium)** from Y's DB, which catches every change including manual SQL fixes.

- Versioning:** keys include a version (`junction:{id}:v{version}`) or a namespace version (`citypulse:v2:`). Bumping it invalidates everything at once, which helps with schema changes in DTOs. You can also compare `@Version` before serving.
- Short TTL as a safety net:** even with explicit invalidation, always set a TTL so missed events self-heal.

Here is an example of Kafka-driven eviction:

```
@Component
public class JunctionCacheInvalidator {
    private final CacheManager cacheManager;
    public JunctionCacheInvalidator(CacheManager cacheManager) { this.cacheManager = cacheManager; }

    @KafkaListener(topics = "junction-events", groupId = "#{T(java.util.UUID).randomUUID().toString()}") // every instance gets it
    public void onEvent(JunctionChangedEvent event) {
        var cache = cacheManager.getCache("junction");
        if (cache != null) cache.evict(event.junctionId());
    }
}
```

(A unique group id per instance gives broadcast semantics. An alternative is a static group per pod name.)

Likely follow-ups:

- Why delete instead of update on write?* Concurrent updates can write values in the wrong order. A delete plus a lazy reload converges.
- Is sync = true enough for stampede protection?* Only within one JVM. Across pods you need a distributed lock or stale-while-revalidate.
- How do you monitor the cache?* Actuator `/actuator/caches`, and the Micrometer metrics `cache.gets{result=hit|miss}`, `cache.evictions`, plus Redis INFO.

C32. JUnit 5 annotations and Mockito annotations

JUnit 5 = **Platform** (launcher) + **Jupiter** (the new programming and extension model) + **Vintage** (runs JUnit 3/4). Imports come from `org.junit.jupiter.api.*`.

Core and lifecycle

Annotation	Purpose
<code>@Test</code>	A test method (no <code>public</code> needed)
<code>@BeforeEach</code> / <code>@AfterEach</code>	Before and after each test (JUnit 4: <code>@Before</code> / <code>@After</code>)
<code>@BeforeAll</code> / <code>@AfterAll</code>	Once per class. Must be static unless <code>@TestInstance(PER_CLASS)</code> (JUnit 4: <code>@BeforeClass</code>)
<code>@TestInstance(Lifecycle.PER_CLASS)</code>	One instance for all tests (the default is PER_METHOD, a new instance per test)
<code>@DisplayName("...")</code> , <code>@DisplayNameGeneration(ReplaceUnderscores.class)</code>	Readable names
<code>@Nested</code>	Inner (non-static) class for grouping, e.g. "when junction is offline"
<code>@Tag("integration")</code>	Filter in Maven Surefire/Failsafe (<code>groups</code>)
<code>@Disabled("reason")</code>	Skip
<code>@Timeout(value = 2, unit = SECONDS)</code>	Fail if too slow
<code>@RepeatedTest(5)</code>	Repeat
<code>@TestFactory</code>	Dynamic tests (returns a <code>Stream<DynamicTest></code>)
<code>@TestTemplate</code>	Template invoked by providers (the base of parameterized and repeated tests)
<code>@TestMethodOrder(OrderAnnotation.class)</code> + <code>@Order(1)</code>	Ordering (avoid order dependence)
<code>@TestClassOrder</code>	Order nested classes
<code>@ExtendWith(X.class)</code>	Register an extension (<code>MockitoExtension</code> , <code>SpringExtension</code>)
<code>@RegisterExtension</code>	A programmatic extension instance on a field (e.g. <code>WireMock</code>)
<code>@TempDir</code>	Inject a temp directory (<code>Path</code>)
<code>@AutoClose</code> (5.11+)	Close a field after the test

Annotation	Purpose
<code>@EnabledOnOs</code> , <code>@DisabledOnJre</code> , <code>@EnabledIfSystemProperty</code> , <code>@EnabledIfEnvironmentVariable</code> , <code>@EnabledIf("method")</code>	Conditional execution

Execution order for one test: constructor → `@BeforeAll` (once) → `@BeforeEach` → `@Test` → `@AfterEach` → ... → `@AfterAll`. Inside `@Nested` classes, the outer `@BeforeEach` runs before the inner one.

Parameterized tests (junit-jupiter-params)

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;
import java.util.stream.Stream;
import static org.assertj.core.api.Assertions.assertThat;

class FineCalculatorTest {

    private final FineCalculator calc = new FineCalculator();

    @ParameterizedTest(name = "{0} km/h over -> Rs {1}")
    @CsvSource({ "5, 0", "15, 1000", "35, 2000", "60, 4000" })
    void speedingFine(int kmhOver, int expected) {
        assertThat(calc.speeding(kmhOver)).isEqualTo(expected);
    }

    @ParameterizedTest
    @ValueSource(strings = { "KA01AB1234", "MH12DE4567" })
    void validPlates(String plate) { assertThat(PlateValidator.isValid(plate)).isTrue(); }

    @ParameterizedTest
    @NullAndEmptySource
    @ValueSource(strings = { " ", "123" })
    void invalidPlates(String plate) { assertThat(PlateValidator.isValid(plate)).isFalse(); }

    @ParameterizedTest
    @EnumSource(value = ViolationType.class, names = { "RED_LIGHT", "WRONG_WAY" })
    void severeViolations(ViolationType type) { assertThat(type.isSevere()).isTrue(); }

    @ParameterizedTest
    @MethodSource("fineCases")
    void fines(ViolationType type, int repeat, int expected) {
        assertThat(calc.fine(type, repeat)).isEqualTo(expected);
    }

    static Stream<Arguments> fineCases() {
        return Stream.of(Arguments.of(ViolationType.RED_LIGHT, 1, 1000),
            Arguments.of(ViolationType.RED_LIGHT, 2, 2000));
    }
}
```

Other sources: `@NullSource`, `@EmptySource`, `@CsvFileSource(resources = "/fines.csv", numLinesToSkip = 1)`, `@ArgumentsSource(MyProvider.class)` and `@FieldSource` (5.11+). Arguments can be aggregated with `ArgumentsAccessor` or `@AggregateWith`.

Assertions: `assertEquals`, `assertTrue`, `assertNull`, `assertThrows`, `assertDoesNotThrow`, `assertTimeout`, and **`assertAll`** (grouped; reports every failure). Most teams use **`AssertJ`** (`assertThat(...)`, fluent, ships with `spring-boot-starter-test`). `Assumptions.assumeTrue(...)` skips a test when an assumption doesn't hold.

Mockito annotations (org.mockito.*, with @ExtendWith(MockitoExtension.class))

Annotation	Purpose
<code>@Mock</code>	A mock: all methods return defaults (null/0/empty) unless stubbed
<code>@Spy</code>	Wraps a real object; real methods run unless stubbed (use <code>doReturn().when(spy)</code> for stubbing)
<code>@InjectMocks</code>	Creates the class under test and injects <code>@Mock</code> / <code>@Spy</code> (constructor → setter → field)
<code>@Captor</code>	<code>ArgumentCaptor<T></code> to inspect arguments passed to mocks
<code>@MockitoSettings(strictness = Strictness.LENIENT)</code>	Relax strict stubs (the default <code>STRICT_STUBS</code> fails on unused stubs with <code>UnnecessaryStubbingException</code>)

Key APIs: `when(...).thenReturn/thenThrow/thenAnswer`, `doThrow/doNothing/doReturn().when(mock).method()` (for void methods and spies), `verify(mock, times(1)/never()/atLeastOnce()).method(...)`, `verifyNoMoreInteractions`, `InOrder`, argument matchers `any()`, `eq()`, `argThat()` (if you use one matcher, use matchers for every argument), `mockStatic` (inline mock maker, the default since Mockito 5), and BDD style `given(...).willReturn(...)` / `then(mock).should()`.

Likely follow-ups:

- *@BeforeAll must be static, why?* With PER_METHOD lifecycle there is no instance yet. Use `@TestInstance(PER_CLASS)` to make it non-static.
- *@Mock vs @Spy?* A mock is entirely fake. A spy is a real object with selective stubbing.
- *JUnit 4 to 5 differences?* `@Before` → `@BeforeEach`, `@RunWith` → `@ExtendWith`, `@Ignore` → `@Disabled`, `@Test(expected=...)` → `assertThrows`, and the new modular architecture with nested and parameterized tests.

C33. Writing tests for Spring Boot apps

The test pyramid

/ E2E / UI (few)	\	Postman/Newman, Selenium/Cypress against deployed env
/ Integration (some)	\	@SpringBootTest + Testcontainers, contract tests
/ Slice tests	\	@WebMvcTest, @DataJpaTest, @JsonTest, @RestClientTest
/ Unit tests (many, fast)	\	JUnit 5 + Mockito, no Spring context

Aim for fast feedback: most logic gets plain unit tests, and Spring contexts load only where wiring, serialisation or SQL matters. Spring's **context cache** reuses an identical configuration across test classes. Every distinct `@MockitoBean` set or property set creates a new context, which slows the suite.

1. Service unit test (Mockito, no Spring): the full example

```
package com.trinity.hr.employee;

import com.trinity.hr.common.ResourceNotFoundException;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Captor;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.BDDMockito.given;
import static org.mockito.BDDMockito.then;
import static org.mockito.Mockito.never;

@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {

    @Mock EmployeeRepository repository;
    @InjectMocks EmployeeService service;
    @Captor ArgumentCaptor<Employee> employeeCaptor;

    @Nested
    @DisplayName("getEmployeeById")
    class GetById {

        @Test
        void returnsDto_whenEmployeeExists() {
            Employee e = new Employee("Asha", "asha@trinity.com", "Traffic", new BigDecimal("90000"),
                LocalDate.of(2022, 1, 10));
            given(repository.findById(1L)).willReturn(Optional.of(e));

            EmployeeDto dto = service.getEmployeeById(1L);

            assertThat(dto.name()).isEqualTo("Asha");
            assertThat(dto.department()).isEqualTo("Traffic");
            then(repository).should().findById(1L);
        }

        @Test
        void throwsNotFound_whenMissing() {
            given(repository.findById(99L)).willReturn(Optional.empty());

            assertThatThrownBy(() -> service.getEmployeeById(99L))
                .assertInstanceOf(ResourceNotFoundException.class)
                .hasMessage("Employee not found with id 99");
        }
    }
}
```

```

        then(repository).should(never()).save(any());
    }
}
}

```

(When the service has a `create` method, `employeeCaptor` verifies what was saved:

```
then(repository).should().save(employeeCaptor.capture()); assertEquals(employeeCaptor.getValue().getEmail(), ...)
```

2. Web slice: `@WebMvcTest`. This loads only the MVC infrastructure (controllers, advice, converters, filters, `WebMvcConfigurer`). Services are mocked. See C34 for the full MockMvc example.

```

@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {
    @Autowired MockMvc mockMvc;
    @MockitoBean EmployeeService service; // Boot 3.4+/Spring 6.2:
    org.springframework.test.context.bean.override.mockito.MockitoBean
}

```

- **`@MockitoBean` / `@MockitoSpyBean`** (Spring Framework 6.2, Boot 3.4+) replace Boot's **`@MockBean` / `@SpyBean`** (`org.springframework.boot.test.mock.mockito`), which are **deprecated in 3.4** and removed in Boot 4.
- With Spring Security on the classpath, `@WebMvcTest` applies security. Use `@WithMockUser` or `.with(jwt())`, or exclude it.

3. JPA slice: `@DataJpaTest`. This loads entities, repositories, `EntityManager`, `TestEntityManager` and Flyway/Liquibase. **Each test is transactional and rolled back**. By default it replaces the `DataSource` with an **embedded** DB (H2). To test against the real engine, use Testcontainers:

```

package com.trinity.hr.employee;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import java.math.BigDecimal;
import java.time.LocalDate;

import static org.assertj.core.api.Assertions.assertThat;

@DataJpaTest
@Testcontainers
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE) // keep the container DB
class EmployeeRepositoryTest {

    @Container
    @ServiceConnection // Boot 3.1+: wires spring.datasource.* automatically
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired EmployeeRepository repository;
    @Autowired TestEntityManager em;

    @Test
    void findByEmail_returnsEmployee() {
        em.persistAndFlush(new Employee("Ravi", "ravi@trinity.com", "Ops", new BigDecimal("50000"),
            LocalDate.now()));
        em.clear(); // force a real SELECT, not the 1st-level cache

        assertThat(repository.findByEmail("ravi@trinity.com"))
            .get().extracting(Employee::getName).isEqualTo("Ravi");
    }
}

```

(Testcontainers 2.x moves the class to `org.testcontainers.postgresql.PostgreSQLContainer`, which is non-generic. Before Boot 3.1, use `@DynamicPropertySource` to register `spring.datasource.url` and the other properties.)

4. Full integration: `@SpringBootTest`. This loads the whole context. Use `webEnvironment = RANDOM_PORT` for a real server, or `MOCK` (the default) with `@AutoConfigureMockMvc`. Combine it with Testcontainers for Postgres, Kafka (`KafkaContainer`) and Redis, with WireMock for external HTTP, and with Awaitility for async assertions.

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Testcontainers
class IncidentFlowIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16-alpine");
}

```

```

@Container @ServiceConnection static KafkaContainer kafka =
    new KafkaContainer(DockerImageName.parse("apache/kafka:3.8.0")); // org.testcontainers.kafka.KafkaContainer
@Autowired TestRestTemplate rest;
// POST incident -> assert DB row -> assert Kafka event consumed (Awaitility)
}

```

Tip: put the containers in a shared abstract base class, or a `@TestConfiguration` with `@Bean @ServiceConnection` methods, so the context is reused. Spring Boot 3.1+ also supports `TestApplication` with `SpringApplication.from(App::main).with(TestcontainersConfig.class)` for local dev.

Other slices: `@JsonTest` (Jackson mapping), `@RestClientTest` (RestTemplate/RestClient with `MockRestServiceServer`), `@WebFluxTest`, `@DataRedisTest`, `@DataMongoTest`, `@JdbcTest`. Use `ApplicationContextRunner` for auto-config and conditional tests.

Likely follow-ups:

- *Why not use H2 for repository tests?* Dialect differences (JSONB, sequences, native queries) hide bugs. Testcontainers matches production.
- *Why is my test suite slow?* Too many unique contexts (different `@MockitoBean` sets or properties). Standardise base classes and prefer slices or unit tests.
- *Unit vs integration test in Maven?* Surefire runs `*Test`, Failsafe runs `*IT` in the `integration-test / verify` phase.

C34. Testing REST APIs

1. MockMvc (server-side, no real HTTP, fast): the full example

```

package com.trinity.hr.employee;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.trinity.hr.common.GlobalExceptionHandler;
import com.trinity.hr.common.ResourceNotFoundException;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.time.LocalDate;
import java.util.Map;

import static org.hamcrest.Matchers.is;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.BDDMockito.given;
import static org.mockito.Mockito.verifyNoInteractions;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultHandlers.print;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(EmployeeController.class)
@Import(GlobalExceptionHandler.class) // advice is picked up by the slice anyway; explicit for clarity
class EmployeeControllerTest {

    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;
    @MockitoBean EmployeeService service;

    @Test
    void getById_returns200AndBody() throws Exception {
        given(service.getEmployeeById(1L)).willReturn(
            new EmployeeDto(1L, "Asha", "asha@trinity.com", "Traffic", LocalDate.of(2022, 1, 10)));

        mockMvc.perform(get("/api/v1/employees/{id}", 1L).accept(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
            .andExpect(jsonPath("$.id", is(1)))
            .andExpect(jsonPath("$.name").value("Asha"))
            .andExpect(jsonPath("$.joiningDate").value("2022-01-10"))
            .andExpect(jsonPath("$.salary").doesNotExist());
    }

    @Test
    void getById_returns404ProblemDetail_whenMissing() throws Exception {
        given(service.getEmployeeById(99L)).willThrow(new ResourceNotFoundException("Employee", 99L));

        mockMvc.perform(get("/api/v1/employees/99"))
            .andExpect(status().isNotFound())
    }
}

```

```

        .andExpect(content().contentType(MediaType.APPLICATION_PROBLEM_JSON))
        .andExpect(jsonPath("$.title").value("Resource Not Found"))
        .andExpect(jsonPath("$.detail").value("Employee not found with id 99"));
    }

    @Test
    void create_returns400_whenBodyInvalid() throws Exception {
        String body = objectMapper.writeValueAsString(Map.of("name", "", "email", "not-an-email"));

        mockMvc.perform(post("/api/v1/employees")
            .contentType(MediaType.APPLICATION_JSON)
            .content(body))
            .andExpect(status().isBadRequest());

        verifyNoInteractions(service);
    }

    @Test
    void update_returns200() throws Exception {
        given(service.update(eq(1L), any()))willReturn(
            new EmployeeDto(1L, "Asha K", "asha@trinity.com", "Traffic", LocalDate.of(2022, 1, 10)));

        mockMvc.perform(put("/api/v1/employees/1")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"name": "Asha K", "email": "asha@trinity.com", "department": "Traffic"}
            """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Asha K"));
    }
}

```

(The `create` and `update` tests assume the controller also exposes POST and PUT mapped to the service, with a validated request record.) For secured APIs, add `spring-security-test`, then use `@WithMockUser(roles = "ADMIN")`, `.with(jwt().authorities(...))` and `.with(csrf())`.

Spring Framework 6.2 also adds **MockMvcTester**, which uses AssertJ-style assertions:

```
assertThat(mvc.get().uri("/api/v1/employees/1")).hasStatusOk().bodyJson()...
```

2. WebClient (fluent; for WebFlux, or for MVC against a running server)

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureWebTestClient
class EmployeeApiWebClientIT {
    @Autowired WebClient client; // needs spring-webflux on the test classpath for MVC apps

    @Test
    void getEmployee() {
        client.get().uri("/api/v1/employees/{id}", 1)
            .exchange()
            .expectStatus().isOk()
            .expectBody()
            .jsonPath("$.name").isEqualTo("Asha");
    }
}

```

You can also bind it to MockMvc with `MockMvcWebTestClient.bindTo(mockMvc)`. `TestRestTemplate` is the classic blocking option with `RANDOM_PORT`.

3. RestAssured (BDD DSL, black-box against a running app or environment)

```

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.equalTo;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class EmployeeRestAssuredIT {
    @LocalServerPort int port;

    @Test
    void getEmployee() {
        given().port(port).accept("application/json")
            .when().get("/api/v1/employees/{id}", 1)
            .then().statusCode(200).body("name", equalTo("Asha"));
    }
}

```

(`spring-mock-mvc` RestAssured module: `RestAssuredMockMvc` for the MockMvc mode.)

4. Postman / Newman (manual, exploratory and environment-level regression)

- Collections with environments (dev/qa/uat variables), pre-request scripts (fetch a token from Keycloak), and test scripts: `pm.test("200", () => pm.response.to.have.status(200)); pm.expect(pm.response.json().name).to.eql("Asha");`
- CI: `newman run citypulse.postman_collection.json -e qa.postman_environment.json --reporters cli,junit,htmlextra` in a Jenkins or GitLab stage after deployment (smoke tests).

Also mention: contract testing (Spring Cloud Contract, Pact) between microservices, and API docs/schema validation through springdoc OpenAPI.

Likely follow-ups:

- *MockMvc vs a real HTTP client?* MockMvc has no network or servlet container, so it is faster but misses server-level behaviour (real filters/ports, HTTP/2, compression). Use RANDOM_PORT tests for those.
- *How do you test file upload?* `multipart("/api/...").file(new MockMultipartFile("file", "a.jpg", "image/jpeg", bytes))`.
- *How do you mock an external API the service calls?* WireMock (`@EnableWireMock` or `WireMockExtension`) or `MockRestServiceServer`.

C35. Handling exceptions in test cases

1. JUnit 5 `assertThrows`. It returns the exception so you can assert on its details.

```
import static org.junit.jupiter.api.Assertions.*;

@Test
void throwsNotFound() {
    given(repository.findById(99L)).willReturn(Optional.empty());

    ResourceNotFoundException ex = assertThrows(ResourceNotFoundException.class,
        () -> service.getEmployeeById(99L));

    assertEquals("Employee not found with id 99", ex.getMessage());
    assertEquals("Employee", ex.getResource());
    assertEquals(99L, ex.getId());
}
```

- `assertThrows` accepts **subclasses** too. Use `assertThrowsExactly` (5.8+) for an exact type match.
- `assertDoesNotThrow(() -> ...)` asserts the happy path explicitly.
- JUnit 4's `@Test(expected = X.class)` and `ExpectedException` rule are gone. `expected` couldn't check messages and passed if *any* line threw.

2. AssertJ (more fluent, and checks causes and fields)

```
import static org.assertj.core.api.Assertions.*;

assertThatThrownBy(() -> service.getEmployeeById(99L))
    .isInstanceOf(ResourceNotFoundException.class)
    .hasMessageContaining("99")
    .hasFieldOrPropertyWithValue("resource", "Employee")
    .hasNoCause();

assertThatExceptionOfType(IllegalArgumentException.class)
    .isThrownBy(() -> Base62.decode("ab$"))
    .withMessage("Invalid char $");

Throwable thrown = catchThrowable(() -> service.process(badInput)); // given/when/then style
assertThat(thrown).isInstanceOf(ValidationException.class).hasRootCauseInstanceOf(DateTimeParseException.class);

assertThatNoException().isThrownBy(() -> service.getEmployeeById(1L));
```

3. Mockito: make dependencies throw

```
// non-void methods
given(repository.findById(1L)).willThrow(new DataAccessResourceFailureException("DB down"));
when(client.ping()).thenThrow(new ResourceAccessException("timeout"));

// void methods -> doThrow form
doThrow(new IllegalStateException("Kafka unavailable")).when(publisher).publish(any());
willThrow(new IllegalStateException("Kafka unavailable")).given(publisher).publish(any()); // BDD

// first call fails, second succeeds -> test retry logic
when(smsGateway.send(any(), any()))
    .thenThrow(new ResourceAccessException("timeout"))
    .thenReturn("MSG-123");

assertThat(alertService.notifyCitizen("9999999999", "Signal down")).isEqualTo("MSG-123");
verify(smsGateway, times(2)).send(any(), any());
```

Also test **the code's reaction** to exceptions: is it translated, does it roll back, is a fallback used, and is an error metric incremented?

```

@Test
void wrapsDbErrorInServiceException() {
    given(repository.findById(1L)).willThrow(new DataAccessResourceFailureException("DB down"));

    assertThatThrownBy(() -> service.getEmployeeById(1L))
        .isInstanceOf(ServiceUnavailableException.class)
        .hasCauseInstanceOf(DataAccessResourceFailureException.class);
}

```

4. MockMvc for advice-mapped exceptions (verifies the HTTP contract of the @RestControllerAdvice)

```

@Test
void notFoundIsMappedTo404() throws Exception {
    given(service.getEmployeeById(99L)).willThrow(new ResourceNotFoundException("Employee", 99L));

    mockMvc.perform(get("/api/v1/employees/99"))
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.detail").value("Employee not found with id 99"))
        .andExpect(result -> assertThat(result.getResolvedException())
            .isInstanceOf(ResourceNotFoundException.class));
}

@Test
void typeMismatchIs400() throws Exception {
    mockMvc.perform(get("/api/v1/employees/abc"))
        .andExpect(status().isBadRequest()); // MethodArgumentTypeMismatchException
}

```

`MvcResult.getResolvedException()` lets you assert which exception the resolver handled.

5. Integration-level checks

- `@DataJpaTest` with duplicate data: `assertThatThrownBy(() -> repo.saveAndFlush(dup)).isInstanceOf(DataIntegrityViolationException.class)`. You need `saveAndFlush`, because the constraint only fires on flush.
- Transaction rollback: call the service in a non-transactional test, force an exception, then assert the DB row count is unchanged.
- Async or Kafka error paths: use `Awaitility` to assert that the message landed in the DLT.

Gotchas

- Put **only the throwing call** inside the lambda. If the setup is inside it, a setup failure can satisfy the assertion and give a false pass.
- With Mockito strict stubs, an unused `willThrow` stub fails the test (`UnnecessaryStubbingException`). That's good, because it flags dead setup.
- Don't catch and swallow exceptions in tests (`try { ... } catch (Exception e) { }`). Use `fail()` if you must use try/catch.
- Checked exceptions: test methods can declare `throws Exception`. That is better than wrapping.

Likely follow-ups:

- *assertThrows vs assertThatThrownBy?* Both work. `assertThrows` returns a typed exception, while `AssertJ` chains assertions fluently (message, cause, fields).
- *How do you test that a void method throws?* `assertThrows(X.class, () -> service.voidMethod())`. To make a *mock's* void method throw, use `doThrow(...).when(mock).voidMethod()`.
- *How do you verify nothing was saved after an exception?* `verify(repository, never()).save(any())`, or `verifyNoMoreInteractions`.

D. Security, Microservices & System Design

D1. Authentication vs authorization in Java/Spring Boot

Authentication (AuthN) answers "Who are you?". It checks an identity claim against something you can verify: a password, a signed token, a certificate, or an OTP. **Authorization (AuthZ)** answers "What are you allowed to do?". It runs *after* authentication and compares the authenticated principal's roles, authorities, scopes, or attributes against the rules for a resource.

Aspect	Authentication	Authorization
Question	Who are you?	What can you do?
Happens	First	After authentication succeeds
Input	Credentials (username/password, JWT, client cert, API key)	Principal + authorities + resource + action
Failure HTTP status	401 Unauthorized (really "unauthenticated")	403 Forbidden
Spring component	<code>AuthenticationManager</code> , <code>AuthenticationProvider</code> , <code>UserDetailsService</code> , <code>JwtDecoder</code>	<code>AuthorizationManager</code> , <code>AuthorizationFilter</code> , <code>@PreAuthorize</code> , <code>hasRole()</code>
Spring exception	<code>AuthenticationException</code> (<code>BadCredentialsException</code> , <code>InsufficientAuthenticationException</code>)	<code>AccessDeniedException</code> (<code>AuthorizationDeniedException</code> in 6.3+)
Result stored in	<code>SecurityContextHolder</code> → <code>Authentication</code> object	Decision only (granted/denied); nothing stored

How Spring models it internally

```
Authentication object
┌ principal : UserDetails / Jwt / username      ← WHO (AuthN)
├ credentials : password (erased after auth) / token
├ authorities : [ROLE_ADMIN, incident:write, SCOPE_read] ← used by AuthZ
├ authenticated: true
└ details : remote IP, session id
stored in SecurityContext
stored in SecurityContextHolder (ThreadLocal by default)
```

- **AuthN flow:** a filter pulls credentials out of the request and builds an *unauthenticated* `Authentication` token, such as `UsernamePasswordAuthenticationToken(username, password)`. The filter hands it to `AuthenticationManager.authenticate()`. On success it gets back an *authenticated* token with authorities, and that token goes into `SecurityContextHolder`.
- **AuthZ flow:** `AuthorizationFilter` (the last filter in the chain) asks an `AuthorizationManager<RequestAuthorizationContext>` whether the current `Authentication` meets the URL rule. Later, method-security interceptors do the same check for `@PreAuthorize` and similar annotations.

Smart City example: a traffic operator logs into the command-centre UI with a username and password, and that is authentication. Their JWT carries `ROLE_OPERATOR`. They can `GET /api/incidents` but get a 403 on `DELETE /api/incidents/{id}`, which needs `ROLE_ADMIN`. That second check is authorization.

Likely follow-ups:

- *Why is 401 called "Unauthorized" if it means unauthenticated?* It's a naming mistake in the HTTP spec. 401 means "send valid credentials" and should come with a `WWW-Authenticate` header. 403 means "I know who you are, and the answer is no".
- *Where is the Authentication stored between requests?* In a stateful app, `HttpSessionSecurityContextRepository` keeps it in the HTTP session. In a stateless JWT app nothing is stored: every request re-authenticates from the token.
- *Is OAuth2 authentication or authorization?* OAuth2 is a *delegated authorization* framework. OpenID Connect (OIDC) adds authentication on top of it with the ID token.
- *Can ThreadLocal context break with async code?* Yes. Use `DelegatingSecurityContextExecutor` or `SecurityContextHolder.MODE_INHERITABLETHREADLOCAL`, or pass the context explicitly.

D2. Implementing authentication & authorization: classes, configuration, flow

Spring Security 6: JWT authentication flow (Spring Boot 3)

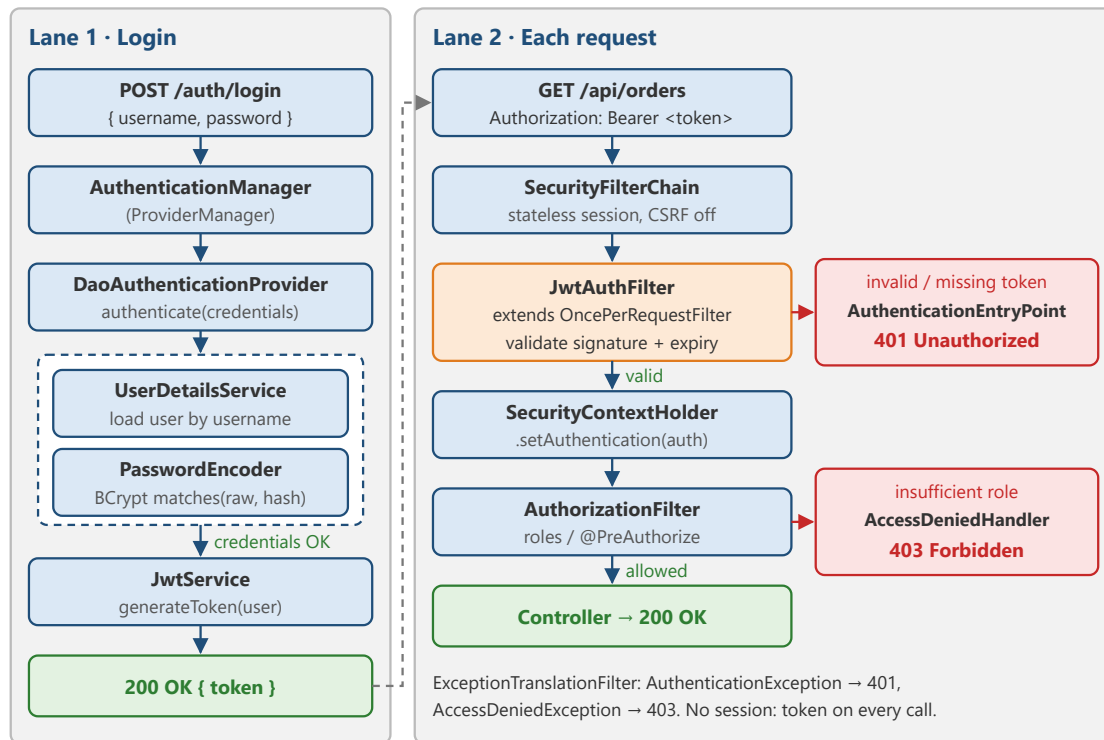


Figure 15: Spring Security 6 JWT flow: login lane and per-request lane

2.1 Big picture: how a request reaches Spring Security



- **DelegatingFilterProxy** is a servlet `Filter` that Spring Boot registers with Tomcat. Tomcat knows nothing about Spring beans, so this proxy looks up the bean named `springSecurityFilterChain` lazily and delegates to it.

- **FilterChainProxy** is that bean. It holds a `List<SecurityFilterChain>` and runs the first chain that matches. You can have several chains, for example one for `/actuator/**` with basic auth and one for `/api/**` with JWT. It also applies the `HttpFirewall` (rejects malformed URLs) and clears the `SecurityContext` after the request.
- **SecurityFilterChain** is what you define with `HttpSecurity`.

2.2 How authentication works inside: AuthenticationManager → ProviderManager → DaoAuthenticationProvider

```

AuthController.login()
| authenticationManager.authenticate(
|   UsernamePasswordAuthenticationToken.unauthenticated(user, pwd))
▼
ProviderManager (implements AuthenticationManager)
| loops over List<AuthenticationProvider>; calls the first one whose supports(tokenClass) is true
| if none succeed → delegates to parent AuthenticationManager (if any)
▼
DaoAuthenticationProvider (supports UsernamePasswordAuthenticationToken)
| 1. userDetailsService.loadUserByUsername(username) → UserDetails (or UsernameNotFoundException)
| 2. preAuthenticationChecks: locked? disabled? expired?
| 3. passwordEncoder.matches(rawPwd, userDetails.getPassword()) → else BadCredentialsException
| 4. postAuthenticationChecks: credentials expired?
| 5. returns UsernamePasswordAuthenticationToken.authenticated(userDetails, null, authorities)
▼
ProviderManager erases credentials, publishes AuthenticationSuccessEvent

```

Timing-attack defence: when the user isn't found, `DaoAuthenticationProvider` still runs a dummy BCrypt check. Responses then take the same time whether or not the username exists, and `UsernameNotFoundException` is hidden as `BadCredentialsException` (`hideUserNotFoundExceptions=true`).

2.3 The JWT flow we will build

```

(1) LOGIN
Angular → POST /api/auth/login {username,password} → AuthController
| authenticationManager.authenticate()
|   → ProviderManager → DaoAuthenticationProvider
|     → UserDetailsService (DB) + BCrypt
|     jwtService.generateToken(userDetails)
Angular ← 200 {accessToken, refreshToken}

(2) EVERY SUBSEQUENT REQUEST
Angular → GET /api/incidents Authorization: Bearer eyJ... → JwtAuthFilter
| extract token → jwtService.parse() (signature, exp)
| build Authentication(username, authorities)
| SecurityContextHolder.getContext().setAuthentication(...)
▼
AuthorizationFilter: hasRole("OPERATOR")? → Controller

```

2.4 Dependencies (Maven)

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<!-- jjwt 0.12.x -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>

```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

```
# application.yml
app:
  jwt:
    secret: ${JWT_SECRET}           # Base64 of >= 256-bit random key; injected from Vault/K8s secret
    access-token-ttl: 15m
    refresh-token-ttl: 7d
    issuer: smartcity-auth
```

2.5 Domain: User entity and UserDetailsService

```
@Entity
@Table(name = "app_user")
public class AppUser {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String passwordHash;           // BCrypt hash, never plain text

    private boolean enabled = true;

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "app_user_role", joinColumns = @JoinColumn(name = "user_id"))
    @Column(name = "role")
    private Set<String> roles = new HashSet<>(); // e.g. "OPERATOR", "ADMIN"

    // getters / setters omitted for brevity
}
```

```
public interface AppUserRepository extends JpaRepository<AppUser, Long> {
    Optional<AppUser> findByUsername(String username);
}
```

```
@Service
@RequiredArgsConstructor
public class DbUserDetailsService implements UserDetailsService {

    private final AppUserRepository repo;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        AppUser u = repo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));
        return User.withUsername(u.getUsername())
            .password(u.getPasswordHash())
            .disabled(!u.isEnabled())
            .roles(u.getRoles().toArray(String[]::new)) // adds "ROLE_" prefix automatically
            .build();
    }
}
```

2.6 JwtService (jjwt 0.12 API)

```
@Service
public class JwtService {

    private final SecretKey key;
    private final Duration accessTtl;
    private final String issuer;

    public JwtService(@Value("${app.jwt.secret}") String base64Secret,
        @Value("${app.jwt.access-token-ttl}") Duration accessTtl,
        @Value("${app.jwt.issuer}") String issuer) {
        this.key = Keys.hmacShaKeyFor(Decoders.BASE64.decode(base64Secret)); // HS256 needs >= 32 bytes
        this.accessTtl = accessTtl;
        this.issuer = issuer;
    }
}
```

```

public String generateAccessToken(UserDetails user) {
    Instant now = Instant.now();
    List<String> roles = user.getAuthorities().stream()
        .map(GrantedAuthority::getAuthority)
        .toList();
    return Jwts.builder()
        .id(UUID.randomUUID().toString()) // jti, needed if you ever want to revoke a token
        .issuer(issuer)
        .subject(user.getUsername())
        .claim("roles", roles)
        .issuedAt(Date.from(now))
        .expiration(Date.from(now.plus(accessTtl)))
        .signWith(key) // algorithm inferred from key → HS256
        .compact();
}

/** Verifies signature + exp + iss. Throws JwtException (ExpiredJwtException, SignatureException...) on failure. */
public Claims parseAndValidate(String token) {
    return Jwts.parser()
        .verifyWith(key)
        .requireIssuer(issuer)
        .clockSkewSeconds(30)
        .build()
        .parseSignedClaims(token) // rejects unsigned (alg=none) tokens
        .getPayload();
}

public String extractUsername(Claims claims) {
    return claims.getSubject();
}

@SuppressWarnings("unchecked")
public List<String> extractRoles(Claims claims) {
    return claims.get("roles", List.class);
}
}

```

2.7 JwtAuthFilter extends OncePerRequestFilter

`OncePerRequestFilter` makes sure the filter runs **once per request**, even when the request is forwarded or dispatched again for errors or async work.

```

@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {

    private static final String BEARER = "Bearer ";
    private final JwtService jwtService;

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        return request.getServletPath().startsWith("/api/auth/"); // login/refresh don't need a token
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws ServletException, IOException {

        String header = request.getHeader(HttpHeaders.AUTHORIZATION);
        if (header == null || !header.startsWith(BEARER)) {
            chain.doFilter(request, response); // no token → stays anonymous → AuthorizationFilter decides
            return;
        }

        String token = header.substring(BEARER.length());
        try {
            Claims claims = jwtService.parseAndValidate(token);

            if (SecurityContextHolder.getContext().getAuthentication() == null) {
                List<SimpleGrantedAuthority> authorities = jwtService.extractRoles(claims).stream()
                    .map(SimpleGrantedAuthority::new)
                    .toList();

                // Stateless: we trust the signed claims and skip the DB lookup.
                // To pick up "user disabled" immediately, load UserDetails here (costs one DB/cache hit).
                UsernamePasswordAuthenticationToken auth =
                    UsernamePasswordAuthenticationToken.authenticated(
                        jwtService.extractUsername(claims), null, authorities);
                auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            }
        }
    }
}

```

```

        SecurityContext context = SecurityContextHolder.createEmptyContext();
        context.setAuthentication(auth);
        SecurityContextHolder.setContext(context);
    }
} catch (ExpiredJwtException ex) {
    request.setAttribute("auth_error", "TOKEN_EXPIRED"); // read by AuthenticationEntryPoint (see D3)
} catch (JwtException | IllegalArgumentException ex) {
    request.setAttribute("auth_error", "TOKEN_INVALID");
}
chain.doFilter(request, response);
}
}
}

```

Why `chain.doFilter` even on failure? The context stays unauthenticated. `AuthorizationFilter` then throws `AccessDeniedException` for protected URLs, `ExceptionTranslationFilter` sees an anonymous user, and calls the `AuthenticationEntryPoint`, which returns a clean 401. Error handling stays in one place.

Pitfall: when `JwtAuthFilter` is a `@Component`, Spring Boot *also* registers it as a normal servlet filter, so it runs twice: once outside the security chain and once inside it. Stop that with a `FilterRegistrationBean` that sets `setEnabled(false)` (below), or don't annotate the filter and create it with `new` in the config.

2.8 SecurityConfig (Spring Security 6 lambda DSL)

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity // enables @PreAuthorize/@PostAuthorize (see D4)
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final JsonAuthEntryPoint authEntryPoint; // D3
    private final JsonAccessDeniedHandler accessDeniedHandler; // D3

    @Bean
    SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")
            .csrf(AbstractHttpConfigurer::disable) // stateless + bearer header → no CSRF risk
            .cors(Customizer.withDefaults()) // uses CorsConfigurationSource bean
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/incidents/**").hasAnyRole("OPERATOR", "SUPERVISOR", "ADMIN")
                .requestMatchers(HttpMethod.POST, "/api/incidents/**").hasAnyRole("OPERATOR", "ADMIN")
                .requestMatchers(HttpMethod.DELETE, "/api/incidents/**").hasRole("ADMIN")
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/sensors/**").hasAuthority("sensor:read")
                .anyRequest().authenticated()
            )
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(authEntryPoint) // 401
                .accessDeniedHandler(accessDeniedHandler) // 403
            )
            .headers(h -> h
                .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'"))
                .frameOptions(HeadersConfigurer.FrameOptionsConfig::deny)
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12); // strength 12 ≈ 250 ms/hash: slow on purpose
        // or PasswordEncoderFactories.createDelegatingPasswordEncoder() → "{bcrypt}$2a$..." (allows algorithm migration)
    }

    /**
     * Exposes the AuthenticationManager Boot builds. With exactly one UserDetailsService
     * and one PasswordEncoder bean, Boot wires a DaoAuthenticationProvider for us.
     */
    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration cfg = new CorsConfiguration();
    }

```

```

        cfg.setAllowedOrigins(List.of("https://command-centre.smartcity.gov.in"));
        cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
        cfg.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Correlation-Id"));
        cfg.setMaxAge(3600L);
        UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
        src.registerCorsConfiguration("/api/**", cfg);
        return src;
    }

    /** Stop Boot from also registering the @Component filter in the plain servlet chain. */
    @Bean
    FilterRegistrationBean<JwtAuthFilter> jwtFilterRegistration(JwtAuthFilter filter) {
        FilterRegistrationBean<JwtAuthFilter> reg = new FilterRegistrationBean<>(filter);
        reg.setEnabled(false);
        return reg;
    }
}

```

2.9 AuthController: login endpoint

```

public record LoginRequest(@NotBlank String username, @NotBlank String password) {}
public record TokenResponse(String accessToken, String refreshToken, long expiresInSeconds) {}

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    private final RefreshTokenService refreshTokenService; // stores hashed refresh tokens in DB/Redis (see D5)

    @PostMapping("/login")
    public ResponseEntity<TokenResponse> login(@Valid @RequestBody LoginRequest req) {
        Authentication auth = authenticationManager.authenticate(
            UsernamePasswordAuthenticationToken.unauthenticated(req.username(), req.password()));
        // Throws BadCredentialsException / DisabledException → handled by @RestControllerAdvice (this is in the MVC layer)

        UserDetails user = (UserDetails) auth.getPrincipal();
        String access = jwtService.generateAccessToken(user);
        String refresh = refreshTokenService.issue(user.getUsername());
        return ResponseEntity.ok(new TokenResponse(access, refresh, 900));
    }
}

```

```

@RestControllerAdvice
public class AuthExceptionHandler {
    @ExceptionHandler(BadCredentialsException.class)
    public ProblemDetail badCredentials() {
        return ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, "Invalid username or password");
    }
    @ExceptionHandler(DisabledException.class)
    public ProblemDetail disabled() {
        return ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, "Account disabled");
    }
}

```

2.10 Production alternative: OAuth2 Resource Server (no custom filter)

When an external IdP issues the tokens (Keycloak, Azure AD, Okta), you don't write a filter or a `JwtService`. Spring's `BearerTokenAuthenticationFilter` with a `NimbusJwtDecoder` validates RS256 signatures against the IdP's JWKS endpoint.

```

spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://keycloak.smartcity.local/realms/smartcity # discovers jwks_uri, validates iss + exp

```

```

@Bean
SecurityFilterChain resourceServer(HttpSecurity http) throws Exception {
    http.csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a.anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(j -> j.jwtAuthenticationConverter(keycloakRolesConverter())));
    return http.build();
}

```

```

@Bean
Converter<Jwt, AbstractAuthenticationToken> keycloakRolesConverter() {
    return jwt -> {
        Map<String, Object> realmAccess = jwt.getClaimAsMap("realm_access");
        @SuppressWarnings("unchecked")
        Collection<String> roles = realmAccess == null ? List.of()
            : (Collection<String>) realmAccess.getOrDefault("roles", List.of());
        List<GrantedAuthority> authorities = roles.stream()
            .map(r -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" + r.toUpperCase()))
            .toList();
        return new JwtAuthenticationToken(jwt, authorities, jwt.getClaimAsString("preferred_username"));
    };
}

```

2.11 How authorization works in web apps

Layer 1: URL rules (`authorizeHttpRequests`), checked by `AuthorizationFilter` through a `RequestMatcherDelegatingAuthorizationManager`. Rules are checked **in order, first match wins**, so put specific rules before general ones and finish with `anyRequest()`.

Layer 2: method security (`@PreAuthorize`). This gives finer, business-level checks such as "an operator can only edit incidents in their own zone". See D4.

Layer 3: domain/data-level filtering. Queries scoped to the user, e.g. `WHERE zone_id IN (:userZones)`, or Hibernate filters and row-level security.

Roles vs authorities

	Role	Authority
What it is	Coarse-grained job function	Fine-grained permission
Example	<code>ROLE_ADMIN</code> , <code>ROLE_OPERATOR</code>	<code>incident:write</code> , <code>sensor:read</code> , <code>SCOPE_profile</code>
In code	<code>hasRole("ADMIN")</code> → checks for authority <code>"ROLE_ADMIN"</code>	<code>hasAuthority("incident:write")</code> → exact string match
Underneath	Also just a <code>GrantedAuthority</code> , with the <code>ROLE_</code> prefix convention	<code>GrantedAuthority</code>

Common pattern: users get **roles**, roles map to **permissions** (a role→permission table), and both go into the authorities. Code checks permissions, which keeps role changes out of the code. `RoleHierarchy` lets `ADMIN > SUPERVISOR > OPERATOR`, so admins inherit everything automatically:

```

@Bean
static RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.fromHierarchy("""
        ROLE_ADMIN > ROLE_SUPERVISOR
        ROLE_SUPERVISOR > ROLE_OPERATOR
        """);
}

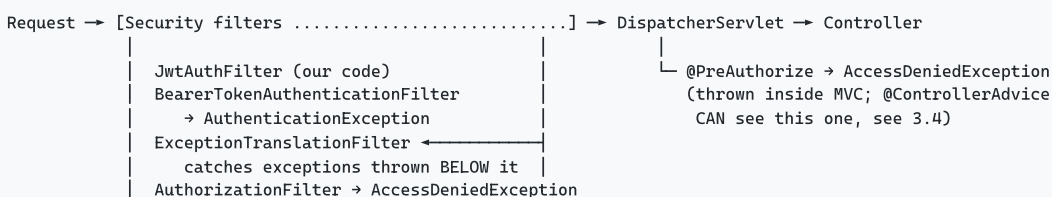
```

Likely follow-ups:

- *Why `addFilterBefore(UsernamePasswordAuthenticationFilter.class)`?* It's a well-known anchor in the chain. The authentication has to be in the context before `AuthorizationFilter` runs, and before form login would try anything.
- *Why `BCrypt` and not `SHA-256`?* `BCrypt` is salted and deliberately slow, with a tunable cost. `SHA-256` is fast, so it can be brute-forced on GPUs. `Argon2` and `PBKDF2` are also acceptable.
- *How do you support two auth mechanisms (JWT for UI, basic for actuator)?* Define two `SecurityFilterChain` beans with different `securityMatcher`s and `@Order`.
- *What replaced `WebSecurityConfigurerAdapter`?* Component-based config: `SecurityFilterChain` beans, `WebSecurityCustomizer`, and the `AuthenticationManager` from `AuthenticationConfiguration`.

D3. Handling exceptions in authentication & authorization

3.1 Where security exceptions come from



ExceptionHandler logic:

```
try { chain.doFilter() } // runs AuthorizationFilter and everything after
catch (AuthenticationException e) → sendStartAuthentication() → AuthenticationEntryPoint.commence() → 401
catch (AccessDeniedException e)
    if user is anonymous / remember-me → sendStartAuthentication() → AuthenticationEntryPoint → 401
    else → AccessDeniedHandler.handle() → 403
```

It only catches exceptions thrown **after** it in the chain. Anything thrown by a filter **before** it, our `JwtAuthFilter` included, never reaches it. That is why our filter doesn't throw. It records the reason and lets `AuthorizationFilter` reject the request.

3.2 Why `@ControllerAdvice` misses filter exceptions

`@ControllerAdvice` / `@ExceptionHandler` are run by the `HandlerExceptionResolver` that `DispatcherServlet` owns. Security filters run *before* the request reaches `DispatcherServlet`, in the servlet container's filter chain. An exception thrown in a filter climbs back up to Tomcat, which forwards to `/error` (Boot's `BasicErrorController`). You get a generic error body, and the advice never sees it.

```
Tomcat filter chain → DelegatingFilterProxy → FilterChainProxy → ... → DispatcherServlet → @ControllerAdvice
                    ▲ exception thrown here never reaches here ───────────
```

Options:

1. **Recommended:** a custom `AuthenticationEntryPoint` and `AccessDeniedHandler` that write JSON.
2. Inject `HandlerExceptionResolver` (qualifier `handlerExceptionResolver`) into the entry point or filter and call `resolver.resolveException(req, res, null, ex)`. That forwards the exception into your `@RestControllerAdvice`, so all error formatting lives in one place.

3.3 JSON 401 entry point and 403 handler

```
@Component
@RequiredArgsConstructor
public class JsonAuthEntryPoint implements AuthenticationEntryPoint {

    private final ObjectMapper mapper;

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
        AuthenticationException authException) throws IOException {
        String code = Optional.ofNullable((String) request.getAttribute("auth_error")).orElse("UNAUTHENTICATED");
        String detail = switch (code) {
            case "TOKEN_EXPIRED" -> "Access token expired. Use the refresh token to get a new one.";
            case "TOKEN_INVALID" -> "Access token is malformed or its signature is invalid.";
            default -> "Authentication is required to access this resource.";
        };

        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, detail);
        pd.setTitle("Unauthorized");
        pd.setInstance(Uri.create(request.getRequestURI()));
        pd.setProperty("code", code);
        pd.setProperty("timestamp", Instant.now().toString());
        pd.setProperty("correlationId", MDC.get("correlationId"));

        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        // RFC 6750: tells the client why the bearer token was rejected
        response.setHeader(HttpHeaders.WWW_AUTHENTICATE,
            "Bearer error=\"invalid_token\", error_description=\"" + code + "\"");
        response.setContentType(MediaType.APPLICATION_PROBLEM_JSON_VALUE);
        mapper.writeValue(response.getOutputStream(), pd);
    }
}
```

```
@Component
@RequiredArgsConstructor
@Slf4j
public class JsonAccessDeniedHandler implements AccessDeniedHandler {

    private final ObjectMapper mapper;

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
        AccessDeniedException ex) throws IOException {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        log.warn("Access denied user={} path={} method={}",
            auth != null ? auth.getName() : "unknown", request.getRequestURI(), request.getMethod());

        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.FORBIDDEN,
```

```

        "You do not have permission to perform this action.");
pd.setTitle("Forbidden");
pd.setInstance(URI.create(request.getRequestURI()));
pd.setProperty("code", "ACCESS_DENIED");

response.setStatus(HttpServletResponse.SC_FORBIDDEN);
response.setContentType(MediaType.APPLICATION_PROBLEM_JSON_VALUE);
mapper.writeValue(response.getOutputStream(), pd);
    }
}

```

They're wired in `SecurityConfig` with `.exceptionHandling(ex -> ex.authenticationEntryPoint(...).accessDeniedHandler(...))` (see D2).

With the OAuth2 resource server, `BearerTokenAuthenticationFilter` calls the configured entry point itself when a token is invalid. Set it with `.oauth2ResourceServer(o -> o.authenticationEntryPoint(entryPoint).jwt(...))`.

3.4 Method-security exceptions

`@PreAuthorize` throws `AccessDeniedException` (`AuthorizationDeniedException` in 6.3+) **inside** the controller or service call, so it travels through `DispatcherServlet`. Watch out:

- If your `@RestControllerAdvice` has a catch-all `@ExceptionHandler(Exception.class)`, it **swallows** `AccessDeniedException` and returns 500 instead of 403.
- Fix: add an explicit handler, or rethrow it so it reaches `ExceptionHandlerFilter`:

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(AccessDeniedException.class)
    public ProblemDetail accessDenied(AccessDeniedException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.FORBIDDEN, "Access denied");
    }

    @ExceptionHandler(AuthenticationException.class)
    public ProblemDetail authFailed(AuthenticationException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ProblemDetail generic(Exception ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error");
    }
}

```

3.5 Handling expired tokens end-to-end

```

Angular HttpInterceptor
├─ request with access token → API → 401 {code: TOKEN_EXPIRED}
├─ on 401 + TOKEN_EXPIRED: (single-flight, queue other requests)
│   └─ POST /api/auth/refresh {refreshToken}
│       └─ 200 new access + rotated refresh → retry original request(s)
│           └─ 401 (refresh expired/revoked) → clear tokens, redirect to /login
└─ on 403 → show "not permitted" toast (don't log the user out)

```

Backend rules: the expiry check comes from `ExpiredJwtException` (jjwt) or `JwtValidationException` with an `exp` failure (Nimbus). Allow a small clock skew (30–60 s). **Never** accept an expired token "just for refresh". The refresh token is a separate credential.

Likely follow-ups:

- *Why not throw from `JwtAuthFilter` directly?* It runs before `ExceptionHandlerFilter`, so the exception would go to Tomcat's `/error` and you'd lose control of the response. Either set an attribute and continue, or call the entry point yourself and `return`.
- *What does `ExceptionHandlerFilter` do for an anonymous user who gets `AccessDenied`?* It treats it as "not authenticated" and calls the `AuthenticationEntryPoint` (401), not the `AccessDeniedHandler`.
- *What should a 401 body never contain?* Stack traces, or whether the username or the password was wrong (that enables user enumeration).
- *How do you test these?* `MockMvc` with `@WithMockUser(roles="OPERATOR")` or `.with(jwt().authorities(...))`, asserting `status().isForbidden()` and the JSON fields.

D4. Pre-authorization (method-level security)

4.1 Enabling it

```
@Configuration
@EnableMethodSecurity(
    prePostEnabled = true,    // Spring Security 6; replaces @EnableGlobalMethodSecurity
                             // @PreAuthorize/@PostAuthorize/@PreFilter/@PostFilter (default true)
    securedEnabled = true,    // @Secured
    jsr250Enabled = true     // @RolesAllowed, @PermitAll, @DenyAll
)
public class MethodSecurityConfig { }
```

Internally, `@EnableMethodSecurity` registers `AuthorizationManagerBeforeMethodInterceptor` and `AuthorizationManagerAfterMethodInterceptor` as AOP advisors. Each annotated bean gets a **proxy**. On every call the interceptor evaluates the SpEL expression against a `MethodSecurityExpressionRoot` (`authentication`, `principal`, `hasRole`, ...) and the method arguments.

```
caller → [CGLIB/JDK proxy] → AuthorizationManagerBeforeMethodInterceptor
                             | evaluate @PreAuthorize SpEL
                             | false → throw AccessDeniedException (target never called)
                             ↓
                             target method executes
                             ↓
                             AuthorizationManagerAfterMethodInterceptor
                             | evaluate @PostAuthorize with returnObject
                             | false → AccessDeniedException (method already ran!)
                             ↓
                             return to caller
```

4.2 The annotations

Annotation	When it's checked	Supports SpEL	Example
<code>@PreAuthorize</code>	Before the method runs	Yes	<code>@PreAuthorize("hasRole('ADMIN')")</code>
<code>@PostAuthorize</code>	After it runs, can use <code>returnObject</code>	Yes	<code>@PostAuthorize("returnObject.reportedBy == authentication.name")</code>
<code>@PreFilter</code> / <code>@PostFilter</code>	Filters collection args or return values	Yes	<code>@PostFilter("filterObject.zoneId == principal.zoneId")</code>
<code>@Secured</code>	Before	No, role list only	<code>@Secured({"ROLE_ADMIN", "ROLE_SUPERVISOR"})</code>
<code>@RolesAllowed</code> (JSR-250)	Before	No	<code>@RolesAllowed("ADMIN")</code> (prefix added automatically)

4.3 SpEL with method parameters and custom bean checks

```
@Service
@RequiredArgsConstructor
public class IncidentService {

    private final IncidentRepository repo;

    // Role check
    @PreAuthorize("hasRole('ADMIN')")
    public void delete(Long id) { repo.deleteById(id); }

    // Combine role + param (#zoneId refers to the parameter; needs -parameters compiler flag, which Boot enables)
    @PreAuthorize("hasRole('SUPERVISOR') and #zoneId == authentication.principal.claims['zoneId']")
    public List<IncidentDto> listByZone(String zoneId) { /* ... */ return List.of(); }

    // Custom bean: @beanName.method(...) keeps complex rules in testable Java code
    @PreAuthorize("hasRole('ADMIN') or @incidentSecurity.canEdit(#id, authentication)")
    public IncidentDto update(Long id, UpdateIncidentRequest req) { /* ... */ return null; }

    // Post-check on returned object
    @PostAuthorize("hasRole('ADMIN') or returnObject.assignedTo == authentication.name")
    public IncidentDto get(Long id) { /* ... */ return null; }

    // Filter results to the caller's zone
    @PostFilter("hasRole('ADMIN') or filterObject.zoneId == @zoneResolver.zoneOf(authentication)")
    public List<IncidentDto> listAll() { /* ... */ return new ArrayList<>(); } // must be a mutable collection
}
```

```

@Component("incidentSecurity")
@RequiredArgsConstructor
public class IncidentSecurity {

    private final IncidentRepository repo;

    public boolean canEdit(Long incidentId, Authentication auth) {
        return repo.findById(incidentId)
            .map(i -> i.getAssignedTo().equals(auth.getName()) && !"CLOSED".equals(i.getStatus()))
            .orElse(false);
    }
}

```

Meta-annotations keep the expressions in one place:

```

@Target({ElementType.METHOD, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@PreAuthorize("hasAnyRole('SUPERVISOR', 'ADMIN')")
public @interface IsSupervisorOrAdmin { }

```

4.4 Proxy limitations (frequently asked)

1. **Self-invocation:** `this.delete(id)` from inside the same class skips the proxy, so **no security check runs**. Fixes: move the method to another bean, inject the bean's own proxy (`@Lazy` self-injection), or use AspectJ weaving (`@EnableMethodSecurity(mode = AdviceMode.ASPECTJ)`).
2. **Visibility:** proxies only intercept `public` methods (CGLIB can technically handle protected/package methods, but don't rely on it). `private` and `final` methods are never intercepted.
3. **Only Spring beans:** objects you create with `new` aren't proxied.
4. **@PostAuthorize runs after side effects:** if the method changed the database, the change has already happened, unless a surrounding `@Transactional` rolls it back. Don't put it on mutating methods.
5. **Order with @Transactional:** both are AOP advisors. Method security runs first by default (`AuthorizationInterceptorsOrder`), so a denied call never opens a transaction.
6. **Interface vs class:** annotations on interface methods work, but annotate one place consistently.

4.5 Testing

```

@SpringBootTest
class IncidentServiceSecurityTest {
    @Autowired IncidentService service;

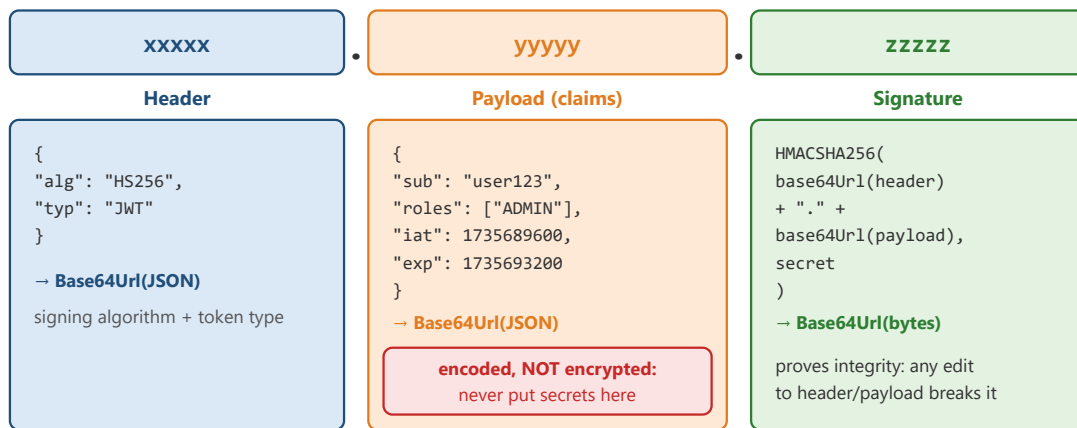
    @Test
    @WithMockUser(roles = "OPERATOR")
    void operatorCannotDelete() {
        assertThrows(AccessDeniedException.class, () -> service.delete(1L));
    }
}

```

Likely follow-ups:

- *URL security or method security?* Both. URL rules give coarse protection at the edge. Method security puts the business rules next to the logic and also protects non-HTTP entry points such as Kafka listeners and schedulers.
- *What's the difference between `hasRole('ADMIN')` and `hasAuthority('ADMIN')`?* `hasRole` adds the `ROLE_` prefix (it checks `ROLE_ADMIN`). `hasAuthority` checks the exact string.
- *Does `@PreAuthorize` work on a `@KafkaListener` method?* Only if there's an `Authentication` in the context. Listener threads have none, so you have to set one up yourself (e.g. a system principal).
- *Is `@PreAuthorize` slower than URL security?* Barely: one SpEL evaluation per call, and the parsed expressions are cached. A custom bean check that hits the DB costs a query, so cache it if it's hot.

D5. How JWT works



```
graph LR; A["1 Split on dots  
→ 3 Base64Url parts"] --> B["2 Recompute sig  
HMAC(h.p, secret key)"]; B --> C["3 Compare  
constant-time equals"]; C --> D["4 Check claims  
exp / iss / aud"];
```

HS256: shared secret (≥ 256 -bit). RS256 / ES256: private key signs, public key verifies.

5.1 Structure

A JWT (RFC 7519) is three Base64URL-encoded parts joined by dots: `header.payload.signature`.

```
eyJhbGciOiJSUzI1NiIsImtpZCI6Im9mcmVudCJlLCJleHAiOjE3...} . Qm9ndXNTaWduYXR1cmU...
|-----|-----|-----|-----|
|   HEADER   |             PAYLOAD (claims)              |           SIGNATURE            |
```

HEADER {"alg":"RS256","typ":"JWT","kid":"k1"} ← kid picks the key from the JWKS

PAYLOAD {"iss":"https://keycloak/realms/smartsiccity",
 "sub":"op1", "aud":"incident-api",
 "exp":1767225600, "iat":1767224700, "nbf":1767224700, "jti":"7f3c...",
 "roles":["OPERATOR"], "zoneId":"Z-07"}

SIGNATURE = sign(base64url(header) + "." + base64url(payload), key)

- **Encoded, not encrypted.** Anyone can Base64-decode the payload, so never put passwords, Aadhaar numbers, or PII in it. If the content must be confidential, use **JWE** (encrypted JWT).
- **Registered claims:** `iss` (issuer), `sub` (subject), `aud` (audience), `exp` (expiry), `nbfi` (not before), `iat` (issued at), `jti` (unique ID).
Private/custom claims: `roles`, `zoneId`, `tenant`.

5.2 HS256 vs RS256 (and ES256)

	HS256 (HMAC + SHA-256)	RS256 (RSA + SHA-256) / ES256 (ECDSA)
Key type	One shared secret	Private key signs, public key verifies
Who can verify	Anyone holding the secret, who can therefore also <i>forge</i> tokens	Anyone with the public key; only the issuer can sign
Distribution	The secret has to be copied to every service, which is risky	Public keys published at a JWKS endpoint (<code>.well-known/jwks.json</code>)
Key rotation	Hard: every service must update together	Easy: publish a new <code>kid</code> , keep the old one for a while
Speed	Faster	Slower (RSA); ES256 is faster and gives smaller signatures
Use when	A single monolith signs and verifies its own tokens	Microservices , external IdP (Keycloak, Auth0, Azure AD)

5.3 Validation steps (what a resource server must do)

1. Split into 3 parts; Base64URL-decode header & payload → malformed? reject
2. Check header.alg is in an ALLOW-LIST (e.g. only RS256) → never trust "alg" blindly; reject "none"
3. Pick the key by kid from the cached JWKS (refetch on unknown kid) → unknown? reject
4. Verify signature over header.payload → mismatch? reject

```

5. exp > now - skew ; nbf <= now + skew           → expired / not yet valid
6. iss == expected issuer                         → token from another IdP? reject
7. aud contains this API                          → token meant for another service? reject
8. (optional) jti not in revocation list; token type == "access"
9. Map claims → GrantedAuthorities → SecurityContext

```

Spring's `NimbusJwtDecoder` built from `issuer-uri` covers steps 1–6. Add audience validation like this:

```

@Bean
JwtDecoder jwtDecoder(@Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String issuer) {
    NimbusJwtDecoder decoder = JwtDecoders.fromIssuerLocation(issuer);
    OAuth2TokenValidator<Jwt> audience = new JwtClaimValidator<List<String>>(<
        JwtClaimNames.AUD, aud -> aud != null && aud.contains("incident-api"));
    decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(<
        JwtValidators.createDefaultWithIssuer(issuer), audience));
    return decoder;
}

```

5.4 Expiry, refresh tokens and rotation

Access tokens are **short-lived** (5–15 min) because a stateless token can't easily be revoked. A **refresh token** is long-lived (hours to days), opaque or a JWT, stored **server-side** (hashed), and used only at `/auth/refresh`.

```

Client → login → Auth: issue AT(15m) + RT1(7d) [store hash(RT1), family=F1]
... AT expires ...
Client - /refresh RT1 → Auth: RT1 valid & unused?
                        yes → mark RT1 used, issue AT' + RT2 (same family F1) ← ROTATION
Attacker - /refresh RT1 (stolen, replayed) → Auth: RT1 already used!
                        → REUSE DETECTED → revoke whole family F1 → user must log in again

```

```

@Service
@RequiredArgsConstructor
public class RefreshTokenService {
    private final StringRedisTemplate redis;
    private static final Duration TTL = Duration.ofDays(7);

    public String issue(String username) {
        String token = Base64.getUrlEncoder().withoutPadding()
            .encodeToString(KeyGenerators.secureRandom(32).generateKey());
        redis.opsForValue().set("rt:" + sha256(token), username, TTL);
        return token;
    }

    /** Rotates: the old token is deleted atomically; a second use returns empty. */
    public Optional<String> consume(String token) {
        String username = redis.opsForValue().getAndDelete("rt:" + sha256(token));
        return Optional.ofNullable(username);
    }

    private static String sha256(String s) {
        try {
            byte[] d = MessageDigest.getInstance("SHA-256").digest(s.getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(d);
        } catch (NoSuchAlgorithmException e) { throw new IllegalStateException(e); }
    }
}

```

(Full reuse detection needs a token *family* ID so you can revoke every descendant token. The sketch above only shows single-use rotation.)

5.5 Revocation / blacklisting

JWTs are valid until `exp` by design. Ways to revoke one earlier:

Technique	How it works	Cost
Short expiry + refresh	Revoke the refresh token; the access token dies within 15 min	Simplest; there's a window of exposure
<code>jti</code> deny-list in Redis	On logout or compromise, <code>SET deny:<jti> 1 EX <remaining ttl></code> ; the filter checks it	One Redis lookup per request
Token version per user	<code>tokenVersion</code> claim; bumping it in the DB invalidates every token the user holds	DB/cache lookup
Introspection (opaque tokens)	Resource server calls <code>/introspect</code> on the IdP	A network call per request (cache it)
Key rotation	Revokes <i>all</i> tokens signed with the old key	Emergency use only

5.6 Where the client stores tokens

Storage	XSS risk	CSRF risk	Notes
localStorage / sessionStorage	High: any injected script can read it	None	Easy with Angular interceptors, but only safe with a strict CSP
JS memory (a service variable)	Lower: gone on refresh, harder to steal	None	Pair it with a refresh token in an HttpOnly cookie
HttpOnly + Secure + SameSite cookie	JS can't read it	Needs SameSite=Strict/Lax or a CSRF token	Recommended for refresh tokens
BFF pattern	Tokens never reach the browser; only a session cookie does	CSRF protection on the BFF	The current OAuth guidance for SPAs

5.7 Common vulnerabilities

- 1. **alg: none** : an attacker strips the signature and sets `alg=none` . Defence: an algorithm allow-list. jjwt's `parseSignedClaims()` rejects unsigned tokens.
- 2. **Algorithm confusion (RS256→HS256)**: the attacker signs with HS256 using the server's *public* key as the HMAC secret. A library that picks the verification method from the header accepts it. Defence: fix the expected algorithm and key type.
- 3. **Weak HMAC secret**: brute-forced offline with hashcat. Use ≥256-bit random secrets.
- 4. **Not validating aud / iss** : a token issued for service A gets accepted by service B.
- 5. **kid / jku injection**: the header points to an attacker's key URL. Never fetch keys from URLs inside the header.
- 6. **Sensitive data in the payload** (it's only Base64).
- 7. **Long-lived access tokens with no revocation**.
- 8. **Tokens leaking** in URLs (query strings end up in logs), in logs generally, or in error messages. Mask the `Authorization` header in logs.

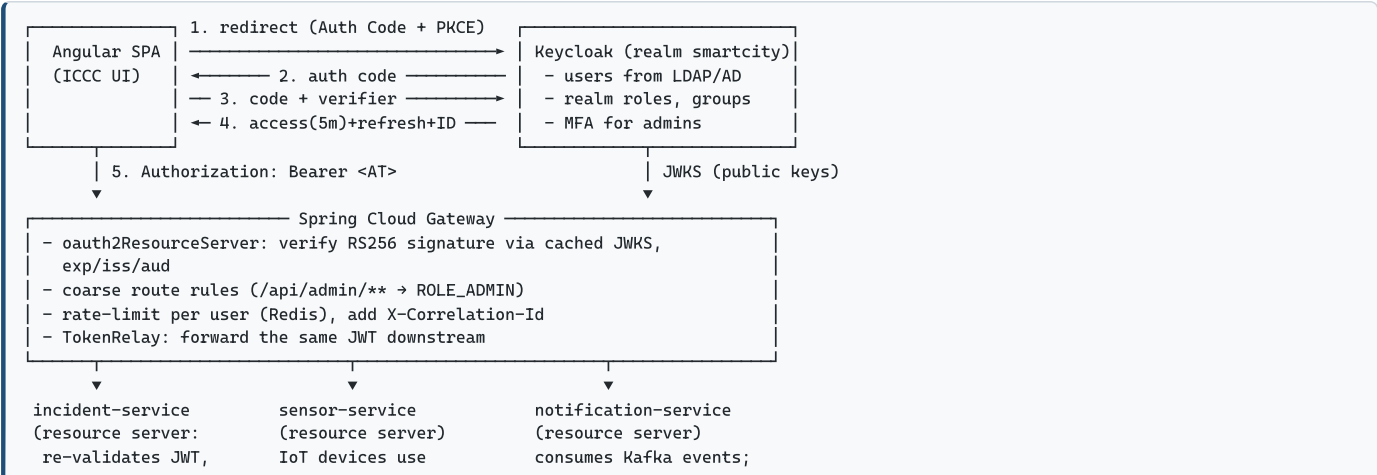
Likely follow-ups:

- *JWT vs session ID?* JWTs are self-contained and stateless, which scales across services with no session store, but they're hard to revoke and larger. Sessions are simple to revoke but need sticky sessions or a shared store (Spring Session + Redis).
- *What's an ID token vs an access token?* ID token (OIDC) = who logged in, meant for the *client*. Access token = permission to call an API, meant for the *resource server*. Never send the ID token to APIs.
- *What's PKCE?* Proof Key for Code Exchange: the SPA sends `code_challenge = SHA256(verifier)` on authorize and the `verifier` on token exchange, so a stolen auth code is useless. It's required for public clients.
- *How big should a JWT be?* Keep it under ~4–8 KB. Proxies and servers limit header size, so don't stuff in hundreds of permissions.

D6. Explain the authentication layer in your project

Template answer: adapt names, IdP, and numbers to your real project. Interviewers want to hear a clear flow, why each choice was made, and one problem you actually solved.

Context: "On our Smart City Integrated Command and Control Centre (ICCC) platform, the users are city operators, supervisors, field officers (mobile app), and admins. Services include Incident Management, IoT/Traffic Sensor Ingestion, Notification, Asset Management, and User/Auth. We centralised authentication in **Keycloak**, an OAuth2/OIDC provider. Earlier releases used our own Spring Boot `auth-service` issuing JWTs, and I'll explain both."



@PreAuthorize for zone-based rules)	client-credentials / mTLS, not user tokens	system calls use client-credentials tokens
-------------------------------------	--	--

Key points to say:

1. **Login:** Angular uses the *Authorization Code flow with PKCE* (via `angular-oauth2-oidc` / `keycloak-angular`). There's no password handling in our code, and MFA (OTP) is enforced for the ADMIN role.
2. **Token contents:** `sub`, `preferred_username`, `realm_access.roles` (OPERATOR, SUPERVISOR, ADMIN, FIELD_OFFICER), plus a custom `zoneId` mapper so we can scope data by city zone.
3. **Gateway:** validates the token once at the edge and rejects bad tokens before they use backend resources. It relays the token downstream (`TokenRelay` filter).
4. **Services:** each service is *also* a resource server and validates the JWT again (zero-trust; signature checks are cheap with the cached JWKS). An alternative some teams use is having the gateway pass `X-User-Id` / `X-Roles` headers. That's only safe if services can't be reached except through the gateway (network policy plus mTLS), so we preferred re-validation.
5. **RBAC:**

Role	Permissions
OPERATOR	View dashboards, create/update incidents in own zone, acknowledge alerts
SUPERVISOR	All operator rights + assign incidents across zones, close incidents, reports
FIELD_OFFICER	Mobile app: view assigned incidents, update status, upload photos
ADMIN	User/role management, device registration, configuration

URL rules sit at the gateway and in each service, and `@PreAuthorize("@zoneGuard.canAccess(#zoneId, authentication)")` covers the data-scoped rules. 6. **Service-to-service calls:** OAuth2 *client credentials* (each service has its own client in Keycloak, with scopes like `notification.send`). Feign attaches the token automatically through an `OAuth2AuthorizedClientManager` interceptor. 7. **Devices / IoT gateways:** they don't use user tokens. They use **mTLS** client certificates or per-device API keys at the ingestion endpoint (MQTT broker/HTTP). 8. **Session handling:** a 5-min access token, a refresh token rotated by Keycloak, and an idle timeout of 30 min (command-centre screens are shared). Logout calls Keycloak's end-session endpoint.

Problems solved (pick one you can defend):

- *Clock skew between K8s nodes* caused random 401s → set the validator clock skew to 60 s and enabled NTP sync.
- *JWKS fetch on every pod start hit Keycloak hard during scaling* → cached JWKS (Nimbus does this by default) and fixed the key-rotation runbook.
- *Big tokens:* fine-grained permissions as claims pushed headers over 8 KB → kept only roles in the token and resolved permissions from a Redis cache.
- *Migrating from custom JWT to Keycloak:* for one release the gateway accepted both issuers (`JwtIssuerAuthenticationManagerResolver`).

Likely follow-ups:

- *Why validate in each service if the gateway already did?* Defence in depth. If someone gets inside the cluster network they shouldn't be able to call services with forged headers.
- *How do Kafka consumers know who triggered an event?* We put `initiatedBy` and `correlationId` in the event payload or headers. Consumers don't re-authenticate users; they trust the broker, which uses SASL/mTLS plus ACLs.
- *How would you force-logout a compromised user?* Disable the user in Keycloak and revoke their sessions/refresh tokens. The access token expires in ≤5 min, and a `jwti` deny-list covers the gap if that window is too long.

D7. Security layers for REST communication; practices in Spring Boot; securing internal APIs

Think in **defence in depth**: each layer assumes the one in front of it can fail.

```

Internet
├── ① WAF / DDoS protection (Cloudflare, AWS WAF, F5) – OWASP rule sets, geo/IP blocks
├── ▼
├── ② TLS 1.2+/1.3 termination at LB (HSTS, strong ciphers, cert auto-renewal)
├── ▼
├── ③ API Gateway – authN (JWT/OAuth2), coarse authZ, rate limiting, CORS, request size limits,
│   └── IP allow-list for partner APIs, API keys for 3rd parties
├── ▼
├── ④ Service mesh / network – mTLS between pods (Istio/Linkerd), K8s NetworkPolicies
│   └── (only gateway → services), private subnets
├── ▼
├── ⑤ Service – re-validate token, method security, input validation, output encoding
├── ▼
└── ⑥ Data – least-privilege DB users, parameterized queries, encryption at rest, column

```

encryption for PII, audit tables
+ Cross-cutting: secrets in Vault, dependency scanning, logging with masking, monitoring/alerting

Layer / practice	What & why	Spring Boot implementation
TLS everywhere	Encrypts traffic in transit, stops MITM and token sniffing	<code>server.ssl.*</code> , or terminate at the LB/ingress; HSTS header; redirect HTTP→HTTPS
API Gateway	Single entry point, consistent auth and throttling	Spring Cloud Gateway + <code>oauth2ResourceServer</code>
OAuth2 / JWT for users	Standard, delegated, short-lived tokens	<code>spring-boot-starter-oauth2-resource-server</code>
OAuth2 client credentials (service → service)	Each service has its own identity and scopes, no shared passwords	<code>spring-boot-starter-oauth2-client</code> + <code>OAuth2AuthorizedClientManager</code>
mTLS	Both sides present certificates. Strong machine identity for internal or partner calls	Mesh sidecar, or <code>server.ssl.client-auth=need</code> + <code>truststore</code>
API keys	Simple identification for third parties (e.g. a weather data provider)	A gateway filter checking a hashed key; rotate keys; never the <i>only</i> control for sensitive data
Rate limiting / throttling	Stops brute force, abuse, DoS	Gateway <code>RequestRateLimiter</code> (Redis), Resilience4j <code>RateLimiter</code> , Bucket4j
CORS	The browser only lets allowed origins call the API	<code>CorsConfigurationSource</code> with an explicit origin list, never <code>*</code> together with credentials
CSRF	Only matters when the browser sends credentials automatically (cookies). Stateless bearer-header APIs can disable it	<code>csrf(AbstractHttpConfigurer::disable)</code> for bearer APIs; <code>CookieCsrfTokenRepository</code> if you use cookie auth
Input validation	Reject bad data early (size, format, ranges)	<code>@Valid</code> , <code>@NotBlank</code> , <code>@Size</code> , <code>@Pattern</code> , custom validators; <code>spring.servlet.multipart.max-file-size</code>
SQL injection prevention	Never concatenate user input into SQL	JPA/ <code>@Query</code> with <code>:params</code> , <code>JdbcTemplate</code> with <code>?</code> , Criteria API; validate sort fields against a whitelist
Output encoding / XSS	Never reflect raw input	Angular escapes by default; JSON content type; CSP header
Security headers	Browser-side hardening	Spring Security defaults: <code>X-Content-Type-Options</code> , <code>X-Frame-Options: DENY</code> , <code>Cache-Control</code> , HSTS; add CSP and <code>Referrer-Policy</code>
Secrets management	No passwords in Git or images	HashiCorp Vault (<code>spring-cloud-vault</code>), K8s Secrets + Sealed Secrets, AWS Secrets Manager; rotate regularly
Dependency scanning	Known CVEs (Log4Shell, Spring4Shell)	OWASP Dependency-Check, Snyk, Trivy for images, Dependabot; SonarQube SAST in CI
Least privilege	Limits the blast radius	DB user per service with only the grants it needs; K8s RBAC; non-root containers
Logging & masking	Audit trail without leaking secrets	Log auth events and access denials; mask <code>Authorization</code> , passwords, phone, Aadhaar via a Logback mask converter
Error handling	Don't leak internals	<code>ProblemDetail</code> , <code>server.error.include-stacktrace=never</code>
Actuator hardening	<code>/actuator/env</code> and <code>/heapdump</code> leak secrets	Expose only <code>health,info,prometheus</code> ; separate management port; secure with a role
Mass assignment	The client sets fields it shouldn't (e.g. <code>role</code>)	Use request DTOs, never bind entities directly
Idempotency & replay protection	Retries or replays don't duplicate actions	<code>Idempotency-Key</code> header; request timestamp + nonce for webhooks

OWASP API Security Top 10 (2023): know these names.

1. Broken Object Level Authorization (BOLA/IDOR): `GET /incidents/123` returns another zone's incident → check ownership with `@PreAuthorize` and data scoping.
2. Broken Authentication.
3. Broken Object Property Level Authorization (excessive data exposure / mass assignment).
4. Unrestricted Resource Consumption → rate limits, pagination caps, payload limits.
5. Broken Function Level Authorization → admin endpoints reachable by operators.
6. Unrestricted Access to Sensitive Business Flows.
7. Server-Side Request Forgery (SSRF) → validate URLs your server fetches.
8. Security Misconfiguration.

9. Improper Inventory Management (old, forgotten API versions).
10. Unsafe Consumption of third-party APIs.

Service-to-service with client credentials (Feign):

```
spring:
  security:
    oauth2:
      client:
        registration:
          incident-svc:
            provider: keycloak
            client-id: incident-service
            client-secret: ${INCIDENT_CLIENT_SECRET}
            authorization-grant-type: client_credentials
            scope: notification.send
        provider:
          keycloak:
            issuer-uri: https://keycloak.smartcity.local/realms/smartcity
```

```
@Configuration
public class FeignOAuth2Config {

    @Bean
    OAuth2AuthorizedClientManager authorizedClientManager(ClientRegistrationRepository regs,
                                                         OAuth2AuthorizedClientService service) {
        var manager = new AuthorizedClientServiceOAuth2AuthorizedClientManager(regs, service);
        manager.setAuthorizedClientProvider(OAuth2AuthorizedClientProviderBuilder.builder()
            .clientCredentials().build());
        return manager;
    }

    @Bean
    RequestInterceptor oauth2FeignInterceptor(OAuth2AuthorizedClientManager manager) {
        return template -> {
            OAuth2AuthorizeRequest req = OAuth2AuthorizeRequest
                .withClientRegistrationId("incident-svc").principal("incident-service").build();
            OAuth2AuthorizedClient client = manager.authorize(req); // cached until near expiry
            if (client == null) {
                throw new IllegalStateException("Could not obtain client-credentials token");
            }
            template.header(HttpHeaders.AUTHORIZATION, "Bearer " + client.getAccessToken().getTokenValue());
        };
    }
}
```

Securing internal APIs that are also exposed to clients:

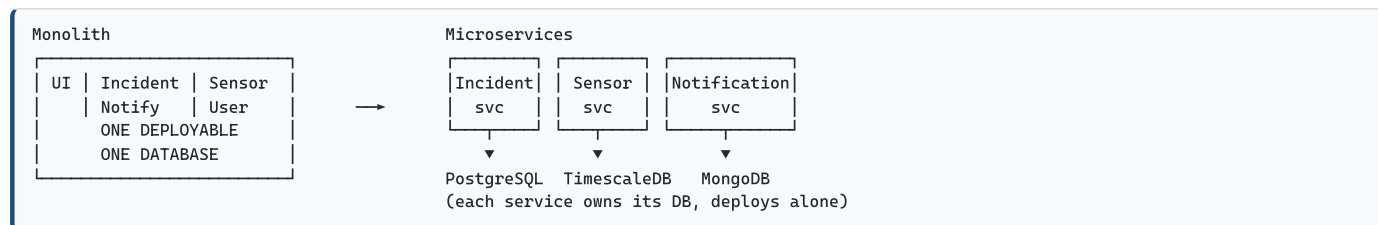
- Publish them **only through the gateway**, on a separate route prefix (/partner/v1/**), with their own auth (OAuth2 client credentials, or API key + IP allow-list), quotas, and an OpenAPI contract.
- Keep internal-only endpoints (/internal/**) off the gateway routes entirely, and block them at ingress and NetworkPolicy level.
- Return **DTOs designed for external use**, not internal models (no internal IDs, stack traces, or extra fields).
- Version the API, monitor usage per client, and give each client a revocable credential.

Likely follow-ups:

- *Why is disabling CSRF OK for JWT APIs?* CSRF works because browsers attach cookies automatically. A bearer token in the `Authorization` header is never attached automatically. Once you put the token in a cookie, you need CSRF protection again.
- *How do you keep the DB password out of application.yml?* Vault with `spring.config.import=vault://`, or K8s Secrets mounted as env vars; rotate with Vault's dynamic DB credentials.
- *How do you prevent IDOR?* Never trust IDs from the client alone. Check ownership or zone in the service layer, and scope queries with the caller's identity.
- *What is mTLS?* Both client and server present X.509 certificates and verify each other, which gives mutual authentication at the transport layer. A service mesh automates certificate issuance and rotation.

D8. What are microservices; key things to take care of

Definition: microservices are an architectural style where an application is a set of **small, independently deployable services**. Each one is built around a **business capability, owns its data**, runs in its own process, and talks to the others over lightweight protocols (HTTP/gRPC/messaging). A small team can build, deploy, and scale each service independently.



Benefits: independent deployment and scaling (sensor ingestion scales to 20 pods while the user service runs 2), fault isolation, technology freedom, smaller codebases, team autonomy. **Costs:** distributed-systems complexity (network failures, latency), data consistency, operational overhead, harder testing and debugging. *Don't start with microservices for a small team or an unclear domain.* A modular monolith is often better at first.

Key things to take care of:

#	Concern	What to do
1	Bounded contexts (DDD)	Split by business capability, not technical layer. "Incident", "Device/Sensor", "Notification", "Identity". A service that has to call 5 others for every request is split wrong
2	Database per service	No shared tables. Others get the data through APIs or events. That keeps services loosely coupled and lets each choose its store (polyglot persistence)
3	Communication style	Sync REST/gRPC for queries that need an immediate answer; async events (Kafka) for state changes and to cut coupling
4	Resilience	Timeouts on <i>every</i> remote call, circuit breakers, retries with backoff, bulkheads, fallbacks (D11–D13)
5	Observability	Centralised logs (ELK/Loki) with correlation IDs, metrics (Micrometer → Prometheus/Grafana), distributed tracing (OpenTelemetry → Jaeger/Zipkin), health checks
6	API versioning & contracts	/v1/ URIs or headers, backward-compatible changes, OpenAPI specs, consumer-driven contract tests (Spring Cloud Contract/Pact)
7	Idempotency	Retries and at-least-once messaging cause duplicates. Use idempotency keys, unique constraints, and a processed-event table
8	Distributed transactions	No 2PC. Use the Saga pattern (choreography or orchestration) with compensating actions, plus the Transactional Outbox for reliable event publishing
9	Security	Gateway auth, token propagation, service identity (client credentials/mTLS), secrets management (D7)
10	Service discovery & config	Eureka/Consul or K8s DNS; Spring Cloud Config or K8s ConfigMaps; externalised, per-environment config
11	CI/CD	A pipeline per service: build → unit/contract tests → image scan → deploy (blue-green/canary) → smoke tests; GitOps (ArgoCD)
12	Containers & orchestration	Docker images (small, non-root), Kubernetes for scaling, self-healing, rolling updates; resource requests and limits
13	Data consistency for reads	Eventual consistency, CQRS/read models, API composition at the gateway or BFF
14	Testing strategy	Unit → slice tests (@WebMvcTest , @DataJpaTest) → Testcontainers integration → contract → a few E2E
15	Team ownership	"You build it, you run it." Each team owns its services end to end (Conway's law)

Likely follow-ups:

- *How big should a microservice be?* Big enough to own one business capability, small enough for one team to understand and redeploy safely. Size by cohesion, not lines of code.
- *What's a distributed monolith?* Services that must be deployed together or share a DB, with chatty synchronous chains between them. It has all the costs of microservices and none of the benefits.
- *When would you not use microservices?* Small team, early product, unclear domain boundaries, no DevOps maturity.

D9. Microservices architecture of your project

Smart City Microservices — Reference Architecture

Template — adapt to your project

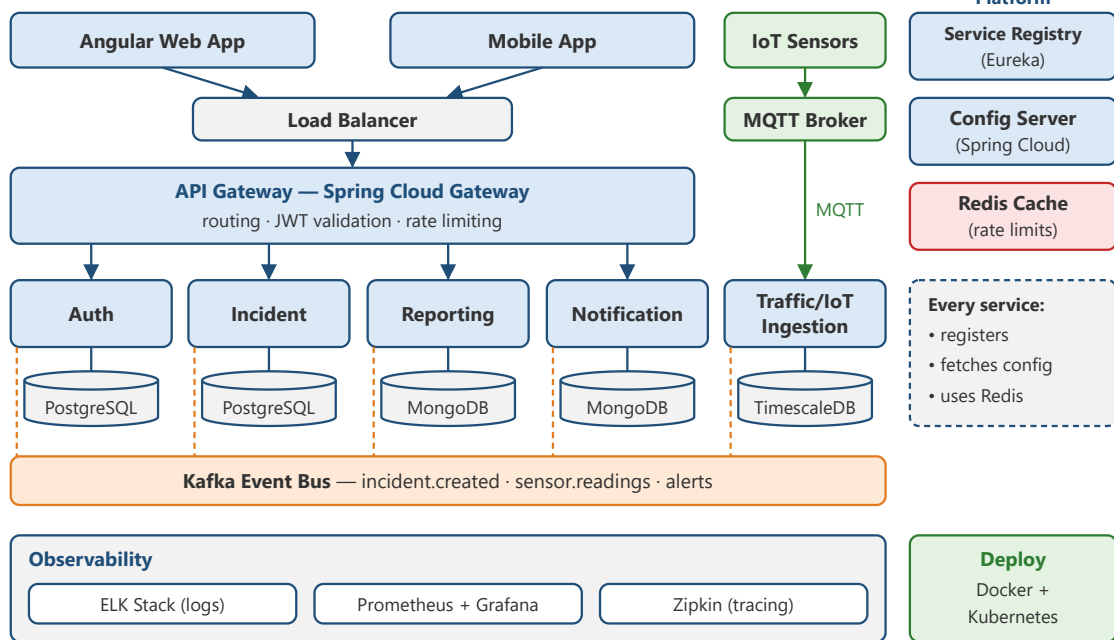
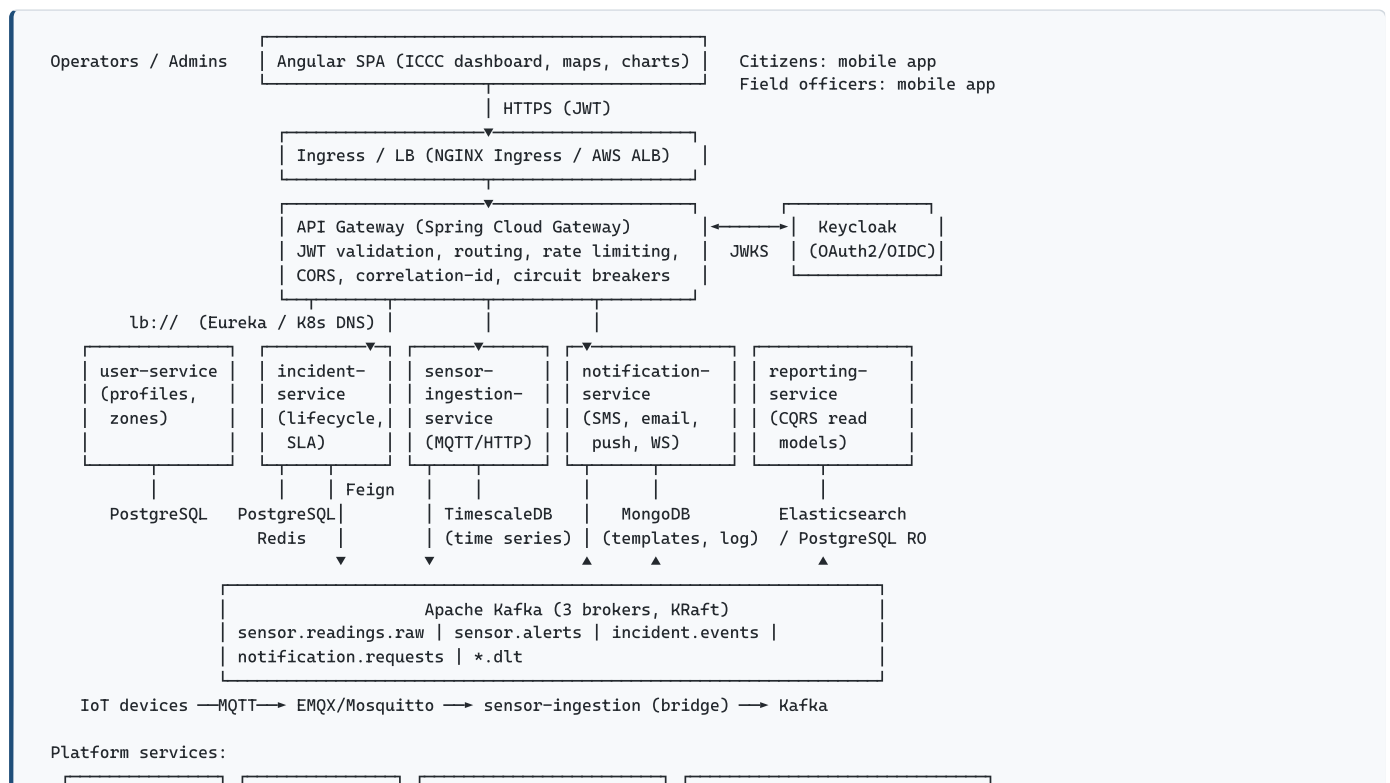


Figure 17: Smart City Microservices Reference Architecture

Template answer: replace the service names, counts, and tools with your real ones. Draw this on the whiteboard as you talk, moving left to right.

One-line pitch: "Our Smart City ICCC platform collects real-time data from traffic signals, ANPR cameras, environmental sensors, and citizen complaints, raises and manages incidents, and notifies field teams. It's built as about 10 Spring Boot microservices on Kubernetes, talking over REST and Kafka."



Config Server (Git-backed)	Eureka (or K8s Service DNS)	Vault (secrets)	Observability: ELK/EFK logs, Prometheus + Grafana, Zipkin/Jaeger traces
-------------------------------	--------------------------------	-----------------	---

CI/CD: GitLab CI/Jenkins → SonarQube → Docker build → Trivy scan → Harbor/ECR → Helm/ArgoCD → K8s

Talk through each piece (2–3 sentences each):

- 1. **Frontend:** an Angular 16+ SPA with lazy-loaded modules (dashboard, incidents, devices, admin), an OpenLayers/Leaflet map, and WebSocket (STOMP) for live alerts. An HTTP interceptor adds the token and handles 401s.
- 2. **Ingress + API Gateway:** NGINX Ingress terminates TLS. Spring Cloud Gateway handles routing (/api/incidents/** → lb://incident-service), JWT validation, Redis-backed rate limiting, a X-Correlation-Id filter, and a circuit breaker with fallbacks for non-critical widgets.
- 3. **Service discovery & config:** Eureka with Spring Cloud LoadBalancer, which we later moved to Kubernetes Services/DNS. Spring Cloud Config Server is backed by Git, one file per profile (dev/uat/prod), with secrets pulled from Vault.
- 4. **Core services:**
 - sensor-ingestion-service: receives about 5k msgs/sec from MQTT and HTTP device gateways, validates and normalises them, writes raw readings to Kafka sensor.readings.raw (keyed by deviceId), applies threshold rules, and publishes sensor.alerts.
 - incident-service: consumes alerts, auto-creates incidents with deduplication, and handles the lifecycle (NEW → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED) and SLA timers. It publishes incident.events through a transactional outbox.
 - notification-service: consumes incident.events and notification.requests, renders templates (MongoDB), and sends SMS (gateway API), email, FCM push, and WebSocket updates. Retry topics plus a DLT.
 - user-service: user profiles, zones, and shift rosters, synced with Keycloak.
 - reporting-service: builds read models (incidents per zone, MTTR) from events, CQRS-style, for the dashboards.
- 5. **Data:** a database per service: PostgreSQL (incidents, users), TimescaleDB/InfluxDB (sensor time series), MongoDB (notification templates and logs), Redis (cache, rate limits, distributed locks), Elasticsearch (search and reporting).
- 6. **Messaging:** Kafka with 3 brokers, replication factor 3, min.insync.replicas=2, idempotent producers, consumer groups per service. Partitions are sized for throughput (12 for sensor topics).
- 7. **Deployment:** Docker images (Eclipse Temurin 17/21 JRE, non-root, layered jars), Helm charts, Kubernetes with HPA on CPU and Kafka lag, readiness/liveness probes from Actuator, rolling updates.
- 8. **Observability:** JSON logs via Logback → Filebeat/Fluent Bit → Elasticsearch → Kibana; Micrometer → Prometheus → Grafana dashboards and Alertmanager (Slack/email); Micrometer Tracing → Zipkin.

Numbers worth having ready (make them real): ~10 services, ~3 environments, ~5k sensor events/sec at peak, ~2k concurrent operators, p95 API latency < 300 ms, 99.9% availability target.

Likely follow-ups:

- Why Kafka rather than calling incident-service from sensor-service directly? It decouples them and absorbs bursts. Sensor data keeps flowing while incident-service is down or redeploying, and several consumers (reporting, notification) can read the same stream.
- Why Eureka if you're on K8s? Legacy from the VM deployment. On K8s, the built-in Service DNS plus Spring Cloud Kubernetes (or plain http://incident-service) makes Eureka unnecessary.
- How do you deploy without downtime? Rolling updates with readiness probes, graceful shutdown (server.shutdown=graceful), backward-compatible DB migrations (Flyway, expand/contract), and a PodDisruptionBudget.

D10. How microservices communicate + tools

10.1 Two styles

SYNCHRONOUS (request/response) A → HTTP/gRPC → B A waits (blocks/awaits) for B's answer	ASYNCHRONOUS (event/message) A → publish → [Kafka topic] → B A continues immediately; B processes later
---	---

Criterion	Sync (REST/gRPC)	Async (Kafka/RabbitMQ)
Coupling	Temporal coupling: B must be up	Decoupled in time; the broker buffers
Latency to caller	Immediate response	Eventual; caller doesn't get B's result inline
Failure handling	Timeouts, retries, circuit breakers	Retries, DLT, replay; broker durability
Consistency	Easier read-your-writes	Eventual consistency
Scalability under bursts	B must scale with A	Queue smooths load; consumers scale independently

Criterion	Sync (REST/gRPC)	Async (Kafka/RabbitMQ)
Fan-out	A calls each consumer	One event, many consumers (pub/sub)
Debugging	Simpler call chain	Harder; needs tracing across the broker
Best for	Queries, validations that need an answer <i>now</i> ("is this user active?")	State-change notifications, long jobs, integration, high-volume streams

10.2 REST clients in Spring

Client	Style	Status
RestTemplate	Sync, blocking	Maintenance mode; still common in existing code
WebClient	Reactive, non-blocking	For WebFlux or high-concurrency fan-out
RestClient (Spring 6.1+)	Sync, fluent API	Recommended new sync client
OpenFeign	Declarative interface	Popular in Spring Cloud stacks
HTTP Interface (@HttpExchange)	Declarative, built into Spring 6	A Feign alternative with no extra dependency

RestClient with timeouts:

```
@Configuration
public class HttpClientsConfig {

    @Bean
    @LoadBalanced // only if using Spring Cloud LoadBalancer / Eureka
    RestClient.Builder loadBalancedBuilder() {
        return RestClient.builder();
    }

    @Bean
    RestClient userServiceClient(RestClient.Builder builder) {
        var factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(Duration.ofSeconds(2));
        factory.setReadTimeout(Duration.ofSeconds(3));
        return builder
            .baseUrl("http://user-service")
            .requestFactory(factory)
            .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
            .build();
    }
}

@Component
@RequiredArgsConstructor
public class UserClient {

    private final RestClient userServiceClient;

    public UserDto getUser(String id) {
        return userServiceClient.get()
            .uri("/api/users/{id}", id)
            .retrieve()
            .onStatus(HttpStatusCode::is4xxClientError, (req, res) -> {
                throw new UserNotFoundException(id);
            })
            .body(UserDto.class);
    }
}
```

OpenFeign:

```
@SpringBootApplication
@EnableFeignClients
public class IncidentServiceApplication { public static void main(String[] a) { SpringApplication.run(IncidentServiceApplication.class, a); } }

@FeignClient(name = "user-service", path = "/api/users", fallbackFactory = UserClientFallbackFactory.class)
public interface UserFeignClient {

    @GetMapping("/{id}")
    UserDto getUser(@PathVariable("id") String id);

    @GetMapping
    List<UserDto> findByZone(@RequestParam("zoneId") String zoneId);
}

@Component
```

```

@Slf4j
class UserClientFallbackFactory implements FallbackFactory<UserFeignClient> {
    @Override
    public UserFeignClient create(Throwable cause) {
        return new UserFeignClient() {
            public UserDto getUser(String id) {
                log.warn("user-service unavailable for {}: {}", id, cause.toString());
                return UserDto.unknown(id);
            }
            public List<UserDto> findByZone(String zoneId) { return List.of(); }
        };
    }
}

```

```

spring:
  cloud:
    openfeign:
      circuitbreaker:
        enabled: true          # wraps Feign calls with Resilience4j so the fallbacks are used
      client:
        config:
          default:
            connect-timeout: 2000
            read-timeout: 3000
            logger-level: basic

```

WebClient for parallel fan-out:

```

Mono<UserDto> user = webClient.get().uri("http://user-service/api/users/{id}", uid)
    .retrieve().bodyToMono(UserDto.class).timeout(Duration.ofSeconds(2));
Mono<DeviceDto> device = webClient.get().uri("http://sensor-service/api/devices/{id}", did)
    .retrieve().bodyToMono(DeviceDto.class).timeout(Duration.ofSeconds(2));
Mono<IncidentView> view = Mono.zip(user, device, IncidentView::new); // both calls run concurrently

```

10.3 Kafka (Spring Kafka)

```

spring:
  kafka:
    bootstrap-servers: kafka-0:9092,kafka-1:9092,kafka-2:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      properties:
        enable.idempotence: true
        linger.ms: 5
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.trusted.packages: "com.smartcity.events"
    listener:
      ack-mode: record

```

```

public record IncidentEvent(String eventId, String type, Long incidentId, String zoneId,
    String severity, Instant occurredAt) {}

@Service
@RequiredArgsConstructor
@Slf4j
public class IncidentEventPublisher {
    private final KafkaTemplate<String, IncidentEvent> kafkaTemplate;

    public void publish(IncidentEvent event) {
        // key = incidentId → every event for one incident goes to the same partition → ordered
        kafkaTemplate.send("incident.events", String.valueOf(event.incidentId()), event)
            .whenComplete((result, ex) -> {
                if (ex != null) {
                    log.error("Publish failed for {}", event.eventId(), ex);
                } else {
                    log.debug("Published {} to partition {} offset {}", event.eventId(),
                        result.getRecordMetadata().partition(), result.getRecordMetadata().offset());
                }
            });
    }
}

```

```

    }
}

@Component
@RequiredArgsConstructor
@Slf4j
public class IncidentEventListener {
    private final NotificationService notificationService;
    private final ProcessedEventRepository processed; // idempotency table

    @KafkaListener(topics = "incident.events", groupId = "notification-service", concurrency = "3")
    public void onEvent(IncidentEvent event,
                       @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                       @Header(KafkaHeaders.OFFSET) long offset) {
        if (processed.existsById(event.eventId())) {
            log.info("Duplicate event {} skipped", event.eventId());
            return;
        }
        notificationService.notifyForIncident(event);
        processed.save(new ProcessedEvent(event.eventId(), Instant.now()));
    }
}

```

10.4 gRPC

gRPC carries Protocol Buffers (binary, schema-first) over HTTP/2. It gives multiplexing, streaming (unary, server, client, bidirectional), generated clients, and fast serialisation. It suits internal high-throughput, low-latency calls (e.g. sensor-service → rules engine). Downsides: browsers can't call it directly (you need gRPC-Web), payloads aren't human-readable, and it needs HTTP/2-aware load balancing. Spring has the `spring-grpc` project (or `grpc-spring-boot-starter`).

```

syntax = "proto3";
service RuleEngine {
    rpc Evaluate (SensorReading) returns (RuleResult);
    rpc StreamReadings (stream SensorReading) returns (stream RuleResult);
}
message SensorReading { string device_id = 1; double value = 2; int64 ts = 3; }
message RuleResult { bool breached = 1; string rule_id = 2; }

```

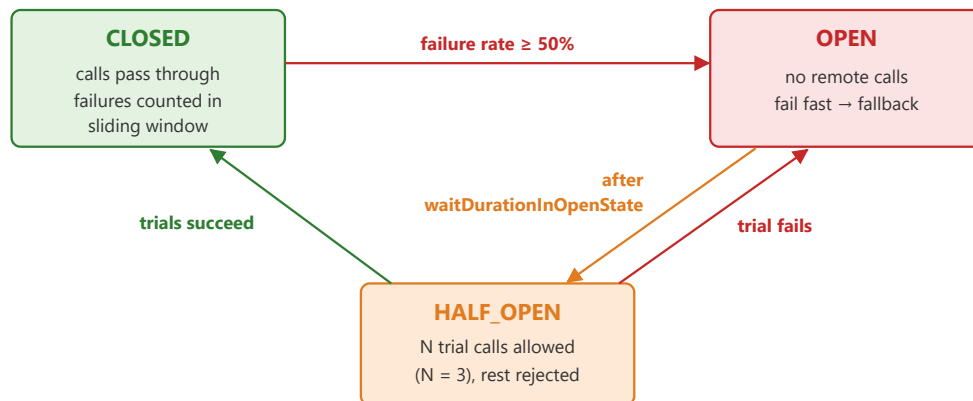
Other options: RabbitMQ (smart broker, routing, per-message ack, good for task queues), WebSockets/SSE (server → browser push), GraphQL (client-shaped queries at the BFF).

Likely follow-ups:

- *Feign vs RestClient?* Feign is declarative and integrates with Spring Cloud LoadBalancer and circuit breakers. RestClient is built in, with explicit control and no extra dependency. Both are fine; be consistent.
- *How do you propagate the JWT or correlation ID in Feign?* A `RequestInterceptor` that copies headers from the current request (or `TokenRelay` at the gateway). Micrometer Tracing propagates trace headers automatically.
- *How do you guarantee an event is published when the DB commit succeeds?* The Transactional Outbox (D24). Writing to the DB and then calling `kafkaTemplate.send` is a dual write, and either half can fail.
- *How does a consumer handle a poison message?* `ErrorHandlingDeserializer` plus a `DefaultErrorHandler` with a `DeadLetterPublishingRecoverer`, so the bad record goes to the DLT instead of blocking the partition.

D11. Circuit breaker

Circuit Breaker — State Machine (Resilience4j)



Resilience4j config (application.yml)

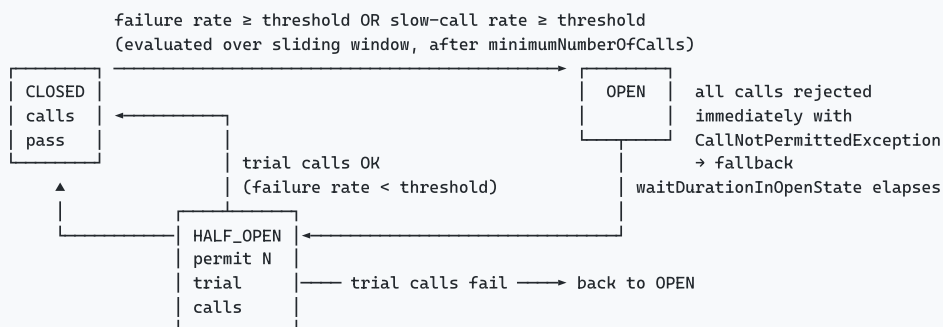
```
slidingWindowSize: 10          waitDurationInOpenState: 10s
failureRateThreshold: 50       permittedNumberOfCallsInHalfOpenState: 3
```

Figure 18: Circuit Breaker State Machine

11.1 Why

Without a breaker, when service B is slow, every call from A waits until timeout, so A's threads pile up and A goes down too: a **cascading failure**. A circuit breaker **fails fast** once B is clearly unhealthy, gives B time to recover, and probes it periodically.

11.2 State machine



Extra states: **DISABLED** (always allow), **FORCED_OPEN** (always reject), **METRICS_ONLY**

11.3 Sliding windows

- **COUNT_BASED**: the last *N* calls (a ring buffer). Example: size 10 with 5 failures = 50% failure rate.
- **TIME_BASED**: calls in the last *N seconds*, stored as *N* one-second buckets. Better when traffic is uneven.
- **minimumNumberOfCalls**: no decision is made until this many calls are recorded, so 1 failure out of 1 call doesn't open the circuit.
- **Slow calls**: calls longer than `slowCallDurationThreshold` count toward `slowCallRateThreshold`. The breaker can open on latency alone, before any errors appear.

11.4 Resilience4j configuration

```
<dependency>
<groupId>io.github.resilience4j</groupId>
<artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-aop</artifactId> <!-- needed for the annotations -->
</dependency>
```

```

resilience4j:
  circuitbreaker:
    configs:
      default:
        register-health-indicator: true
        sliding-window-type: COUNT_BASED
        sliding-window-size: 20
        minimum-number-of-calls: 10
        failure-rate-threshold: 50 # %
        slow-call-duration-threshold: 2s
        slow-call-rate-threshold: 60 # %
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 5
        automatic-transition-from-open-to-half-open-enabled: true
        record-exceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
        ignore-exceptions:
          - com.smartcity.common.exception.BusinessValidationException # 4xx are not B's fault
    instances:
      userService:
        base-config: default
      smsGateway:
        base-config: default
        sliding-window-type: TIME_BASED
        sliding-window-size: 60 # seconds
        wait-duration-in-open-state: 60s
  timelimiter:
    instances:
      userService:
        timeout-duration: 3s
        cancel-running-future: true

management:
  endpoints.web.exposure.include: health,circuitbreakers,circuitbreakerevents,prometheus
  health.circuitbreakers.enabled: true

```

```

@Service
@RequiredArgsConstructor
@Slf4j
public class AssigneeLookupService {

    private final UserClient userClient;
    private final CacheManager cacheManager;

    @CircuitBreaker(name = "userService", fallbackMethod = "getAssigneeFallback")
    public UserDto getAssignee(String userId) {
        UserDto user = userClient.getUser(userId);
        Objects.requireNonNull(cacheManager.getCache("users")).put(userId, user); // keep last-known-good copy
        return user;
    }

    /** Same signature + Throwable as the last param. A more specific exception type wins. */
    private UserDto getAssigneeFallback(String userId, CallNotPermittedException ex) {
        log.warn("Circuit OPEN for userService, serving cached user {}", userId);
        return cachedOrPlaceholder(userId);
    }

    private UserDto getAssigneeFallback(String userId, Throwable ex) {
        log.warn("userService call failed for {}: {}", userId, ex.toString());
        return cachedOrPlaceholder(userId);
    }

    private UserDto cachedOrPlaceholder(String userId) {
        Cache cache = cacheManager.getCache("users");
        UserDto cached = cache != null ? cache.get(userId, UserDto.class) : null;
        return cached != null ? cached : UserDto.unknown(userId);
    }
}

```

Monitor the breaker state and transitions in Grafana (`resilience4j_circuitbreaker_state`) and alert when a breaker opens.

Likely follow-ups:

- Which exceptions should count as failures? Infrastructure failures (timeouts, connection refused, 5xx). Not client errors (400, 404, validation): those don't mean B is unhealthy.

- *What should a fallback return?* Cached or stale data, a default, an empty list, or a "degraded" flag. Or queue the work for later. Never quietly return fake success for writes.
- *Gateway-level or service-level breaker?* Both have a place. The gateway's `CircuitBreaker` filter protects edge routes; service-level breakers protect specific dependencies with business-aware fallbacks.
- *What replaced Hystrix?* Resilience4j. Hystrix has been in maintenance mode since 2018.

D12. Retry mechanism

12.1 When to retry

Retry only **transient** failures: connection reset, timeouts, 503, 429 (honour `Retry-After`), Kafka `NotLeaderForPartition`. Don't retry permanent failures (400, 401, 403, 404, validation errors), because the result will never change.

Only retry idempotent operations, or make them idempotent first. GET, PUT, and DELETE are idempotent by HTTP semantics. A POST that creates an incident isn't. A retry after a timeout (where the first request actually succeeded) creates a **duplicate**. Fix it with an `Idempotency-Key` header that the server stores together with the result, or with a unique business key.

12.2 Backoff and jitter

Fixed:	--500ms-- --500ms-- --500ms--	→ synchronized clients hammer B together
Exponential:	-500- ---1000--- -----2000-----	→ gives B time to recover
+ Jitter:	each wait = random(0.5x .. 1.5x)	→ spreads retries; avoids the "thundering herd"

Retry storms multiply: with 3 layers that each retry 3 times, one user request becomes **27** calls to the bottom service. Retry at **one layer** (usually the one closest to the failing dependency), and cap the total time with timeouts.

12.3 Resilience4j @Retry

```
resilience4j:
  retry:
    instances:
      userService:
        max-attempts: 3                # including the first call
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2    # 500ms, 1000ms
        enable-randomized-wait: true        # combined with exponential → exponential random backoff
        randomized-wait-factor: 0.5
        retry-exceptions:
          - java.io.IOException
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException$ServiceUnavailable
        ignore-exceptions:
          - org.springframework.web.client.HttpClientErrorException
```

```
@Retry(name = "userService")
@CircuitBreaker(name = "userService", fallbackMethod = "fallback")
public UserDto getUser(String id) {
    return userClient.getUser(id);
}

private UserDto fallback(String id, Throwable t) {
    return UserDto.unknown(id);
}
```

12.4 Order: retry vs circuit breaker

Resilience4j's default aspect order (outermost first):

```
Retry ( CircuitBreaker ( RateLimiter ( Timelimiter ( Bulkhead ( yourMethod ) ) ) ) )
```

- **Retry outside the circuit breaker (the default, recommended):** every attempt is recorded by the breaker. Once the breaker opens, the next retry gets `CallNotPermittedException` straight away. Make that exception non-retryable (don't list it in `retry-exceptions`, or put it in `ignore-exceptions`) so you don't retry against an open circuit.
- The fallback is attached to the outermost decorator you put `fallbackMethod` on. Usually that's only the circuit breaker, or the retry if you want the fallback after every attempt has failed.
- Order can be changed with `resilience4j.retry.retry-aspect-order` and `resilience4j.circuitbreaker.circuit-breaker-aspect-order` (a higher value means higher priority, i.e. it runs outermost).

12.5 Spring Retry (alternative)

```
@Configuration
@EnableRetry
class RetryConfig { }

@Service
class SmsGatewayClient {
    @Retryable(retryFor = {ResourceAccessException.class},
        maxAttempts = 4,
        backoff = @Backoff(delay = 500, multiplier = 2, maxDelay = 5000, random = true))
    public void send(SmsRequest req) { /* call SMS provider */ }

    @Recover
    public void recover(ResourceAccessException ex, SmsRequest req) {
        // after all attempts: queue for later / mark FAILED
    }
}
```

12.6 Kafka consumer retries: non-blocking retry topics + DLT

A *blocking* retry (DefaultErrorHandler with FixedBackOff) stops the whole partition while it retries one bad record. `@RetryableTopic` moves the failed record to delay topics, so the main topic keeps flowing:

```
incident.events —fail→ incident.events-retry-0 (1s) —fail→ -retry-1 (2s) —fail→ -retry-2 (4s) —fail→ incident.events-dlt
                                                                |
                                                                @DltHandler: alert, persist, manual replay
```

```
@Component
@Slf4j
@RequiredArgsConstructor
public class NotificationListener {

    private final NotificationService service;

    @RetryableTopic(
        attempts = "4", // 1 original + 3 retries
        backoff = @Backoff(delay = 1000, multiplier = 2.0, maxDelay = 10000),
        dltTopicSuffix = "-dlt",
        exclude = {DeserializationException.class, IllegalArgumentException.class}, // straight to DLT
        autoCreateTopics = "true")
    @KafkaListener(topics = "incident.events", groupId = "notification-service")
    public void onIncident(IncidentEvent event) {
        service.notifyForIncident(event); // throws → goes to the next retry topic
    }

    @DltHandler
    public void onDlt(IncidentEvent event,
        @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
        @Header(KafkaHeaders.EXCEPTION_MESSAGE) String error) {
        log.error("DLT {} event={} error={}", topic, event.eventId(), error);
        // save to failed_events table + raise alert; ops can replay after fixing
    }
}
```

(`@Backoff` here is `org.springframework.retry.annotation.Backoff` in Spring Kafka 3.x.) Note that retry topics **break strict ordering** for the retried key. If ordering matters, use a blocking retry with a short backoff instead.

Likely follow-ups:

- *How many retries?* Usually 2–3 for sync calls, with the total bounded by the caller's timeout or SLA. More retries just add latency.
- *What is a retry budget?* Cap retries to a percentage of traffic (e.g. $\leq 10\%$) so retries can't overload a struggling service.
- *How do you make a POST idempotent?* The client sends an `Idempotency-Key` (UUID). The server stores key → response under a unique constraint and returns the stored response on a repeat.
- *What's in the DLT and how do you reprocess it?* The original record plus exception headers. A small admin tool or consumer republishes it to the main topic once the bug is fixed.

D13. If Microservice B is down, what should A do?

Principle: A must **degrade gracefully**, never hang, and never cascade the failure. Layer the techniques:

```
A — call → [Timeout] → [Retry (transient only)] → [Circuit Breaker] → [Bulkhead] → B
                                     |
                                     open / failed
```

▼
Fallback: cache / default / queue for later / partial response

#	Technique	What A does	Example in Smart City
1	Timeouts	Connect 1–2 s, read 2–5 s. Never wait forever (RestTemplate's default is <i>infinite</i>)	Incident detail page calls user-service with a 2 s timeout
2	Retry with backoff	1–2 retries for transient errors, idempotent calls only	GET user profile
3	Circuit breaker	Stop calling B once it's unhealthy, fail fast, probe in half-open	<code>userService</code> breaker opens at 50% failures
4	Fallback	Return a sensible degraded response	Show "Assignee: unavailable" while the incident still renders
5	Cached / stale data	Serve the last-known-good copy from Redis or a local cache (Caffeine)	Zone and user names change rarely, so serve them from cache
6	Async via queue	For <i>commands</i> , don't call B synchronously. Publish an event/message; B processes it when it's back	Incident created → <code>notification.requests</code> on Kafka; notification-service catches up after restarting
7	Transactional outbox	Save the business change and the outbox row in one DB transaction; a relay publishes when the broker or B is reachable	Incident status change always produces an event eventually
8	Bulkhead	Separate thread pools or semaphores per dependency, so slow B can't use up all of A's threads	SMS gateway gets max 20 concurrent calls; the dashboard APIs are unaffected
9	Graceful degradation / feature flags	Turn off non-critical features	Hide the "live camera feed" widget while video-service is down
10	Health checks & self-healing	K8s liveness restarts a hung B; readiness takes it out of the Service endpoints; multiple replicas, PodDisruptionBudget, anti-affinity	B pod OOM → restarted automatically; other replicas serve meanwhile
11	Alerting	Breaker opens, error rate rises, <code>up == 0</code> → Alertmanager → on-call	Grafana alert "user-service 5xx > 5% for 5m"
12	Clear error to the client	When there's no fallback, return 503 with <code>Retry-After</code> , not a raw 500 or a hang	<code>ProblemDetail</code> "Service temporarily unavailable"

Bulkhead config:

```
resilience4j:
  bulkhead:
    instances:
      smsGateway:
        max-concurrent-calls: 20
        max-wait-duration: 100ms      # wait this long for a permit, then BulkheadFullException → fallback
  thread-pool-bulkhead:
    instances:
      reportService:
        core-thread-pool-size: 4
        max-thread-pool-size: 8
        queue-capacity: 50
```

```
@Bulkhead(name = "smsGateway", fallbackMethod = "queueSms")
@CircuitBreaker(name = "smsGateway")
public void sendSms(SmsRequest req) { smsClient.send(req); }

private void queueSms(SmsRequest req, Throwable t) {
    kafkaTemplate.send("notification.sms.retry", req.phone(), req);    // retry later; never lose the message
}
```

What to say in an interview: "It depends on whether A *needs* B's answer right now. For **reads**, I use a timeout, a circuit breaker, and cached or default data, and show a partial response. For **writes/commands**, I move to async: publish to Kafka through an outbox so the work happens when B is back, and tell the user 'accepted' (HTTP 202). In both cases, monitoring and K8s self-healing bring B back quickly, and alerts tell the team."

Likely follow-ups:

- *What if B is critical, like payment or authentication?* Fail fast with a clear 503 and don't fake success. Use redundancy (multi-AZ replicas) and make sure it's caught by alerting. A queue is only acceptable if the business accepts a deferred result.
- *What's the difference between liveness and readiness?* Liveness failing → K8s restarts the container. Readiness failing → the pod is removed from load balancing but not restarted (e.g. while it warms up or loses its DB connection).
- *How do you stop A's Kafka consumer from hammering B while B is down?* Pause the listener container (`KafkaListenerEndpointRegistry...pause()`) when the breaker opens, and resume it when it closes, or rely on retry topics with backoff.

API Gateway — Single Entry Point (Spring Cloud Gateway)

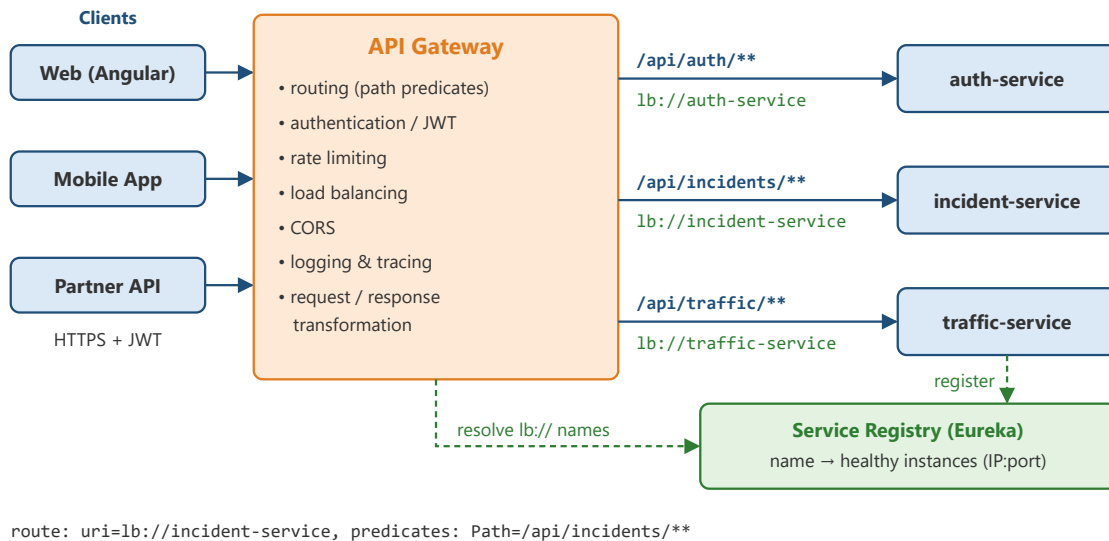
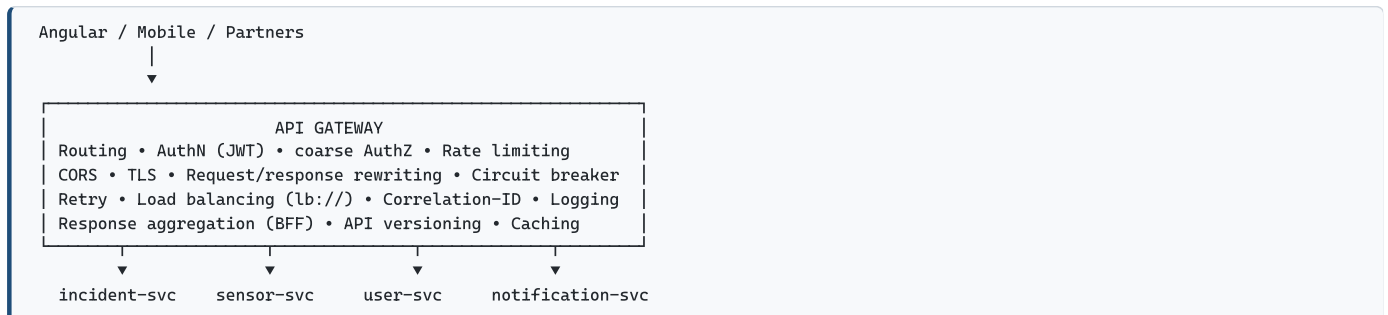


Figure 19: API Gateway Routing with Eureka

14.1 What it is

An API Gateway is the **single entry point** for all clients. It sits in front of the microservices and handles cross-cutting concerns, so the services don't each have to.



Responsibilities:

1. **Routing:** path, host, or header → service (with service discovery via `lb://`).
2. **Security:** token validation, rejecting unauthenticated traffic at the edge, CORS, IP allow-lists.
3. **Rate limiting / throttling:** per user, IP, or API key.
4. **Resilience:** timeouts, circuit breakers, retries, fallbacks per route.
5. **Transformation:** add, remove, or rewrite headers and paths, strip prefixes.
6. **Observability:** access logs, metrics per route, trace propagation, correlation IDs.
7. **Aggregation / BFF:** combine several backend calls into one client response.
8. **Hiding the internal topology:** clients never see service hosts or ports.

14.2 Spring Cloud Gateway: how it works

It's built on Spring WebFlux and Netty (reactive, non-blocking), so a few threads handle many concurrent connections. Spring Cloud 2025 also ships a WebMVC (servlet) variant. Core concepts:

- **Route** = ID + destination URI + **predicates** + **filters**
- **Predicate:** matches requests (Path, Method, Header, Host, Query, After/Before, Weight)
- **Filter:** changes the request or response, *pre* and *post* (GatewayFilter per route, GlobalFilter for all routes)

```
Request → Gateway Handler Mapping (match predicates → route)
      → Filter chain (pre: auth, rate limit, add header, ...)
      → Proxy to lb://incident-service
```

← Filter chain (post: add response headers, log latency)
← Response

14.3 Route configuration (YAML)

```
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      # NOTE: since Spring Cloud 2025.0 the preferred prefix is spring.cloud.gateway.server.webflux.*
      default-filters:
        - DedupeResponseHeader=Access-Control-Allow-Origin Access-Control-Allow-Credentials, RETAIN_UNIQUE
        - name: RequestRateLimiter
          args:
            redis-rate-limiter.replenishRate: 20          # tokens/sec per key
            redis-rate-limiter.burstCapacity: 40
            redis-rate-limiter.requestedTokens: 1
            key-resolver: "#{@userKeyResolver}"
      globalcors:
        cors-configurations:
          '[/*]':
            allowedOrigins: "https://command-centre.smartcity.gov.in"
            allowedMethods: [GET, POST, PUT, PATCH, DELETE, OPTIONS]
            allowedHeaders: [Authorization, Content-Type, X-Correlation-Id]
            maxAge: 3600
      routes:
        - id: incident-service
          uri: lb://incident-service
          predicates:
            - Path=/api/incidents/**
          filters:
            - TokenRelay=
            - name: CircuitBreaker
              args:
                name: incidentCB
                fallbackUri: forward:/fallback/incidents
            - name: Retry
              args:
                retries: 2
                methods: GET
                statuses: BAD_GATEWAY, SERVICE_UNAVAILABLE
                backoff:
                  firstBackoff: 100ms
                  maxBackoff: 1s
                  factor: 2

        - id: sensor-service
          uri: lb://sensor-ingestion-service
          predicates:
            - Path=/api/sensors/**
          filters:
            - RewritePath=/api/sensors/(?<segment>.*), /v1/sensors/${segment}

        - id: notification-service-v2-canary
          uri: lb://notification-service-v2
          predicates:
            - Path=/api/notifications/**
            - Weight=notifications, 10          # 10% canary
        - id: notification-service-v1
          uri: lb://notification-service
          predicates:
            - Path=/api/notifications/**
            - Weight=notifications, 90

      httpclient:
        connect-timeout: 2000
        response-timeout: 5s

  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://keycloak.smartcity.local/realms/smartcity
```

The `TokenRelay` filter needs the gateway to be an OAuth2 *client* (`spring-boot-starter-oauth2-client` , session-based login at the gateway, i.e. the BFF style). When the SPA already sends a bearer token, the `Authorization` header is forwarded unchanged by default, so you only need `oauth2ResourceServer` on the gateway.

14.4 Gateway security, key resolver and a custom global filter

```
@Configuration
@EnableWebFluxSecurity
public class GatewaySecurityConfig {

    @Bean
    SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity http) {
        return http
            .csrf(ServerHttpSecurity.CsrfSpec::disable)
            .authorizeExchange(ex -> ex
                .pathMatchers("/actuator/health/**", "/fallback/**").permitAll()
                .pathMatchers("/api/admin/**").hasRole("ADMIN")
                .anyExchange().authenticated())
            .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
            .build();
    }

    @Bean
    KeyResolver userKeyResolver() {
        return exchange -> exchange.getPrincipal()
            .map(Principal::getName)
            .defaultIfEmpty(Optional.ofNullable(exchange.getRequest().getRemoteAddress())
                .map(a -> a.getAddress().getHostAddress()).orElse("anonymous"));
    }
}
```

```
@Component
public class CorrelationIdGlobalFilter implements GlobalFilter, Ordered {

    public static final String HEADER = "X-Correlation-Id";

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String id = Optional.ofNullable(exchange.getRequest().getHeaders().getFirst(HEADER))
            .orElse(UUID.randomUUID().toString());
        ServerHttpRequest mutated = exchange.getRequest().mutate().header(HEADER, id).build();
        exchange.getResponse().getHeaders().add(HEADER, id);
        long start = System.nanoTime();
        return chain.filter(exchange.mutate().request(mutated).build())
            .doFinally(sig -> {
                long ms = (System.nanoTime() - start) / 1_000_000;
                // log: method, path, status, ms, correlation id
            });
    }

    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE; }
}
```

```
@RestController
class FallbackController {
    @RequestMapping("/fallback/incidents")
    Mono<ResponseEntity<ProblemDetail>> incidents() {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.SERVICE_UNAVAILABLE,
            "Incident service is temporarily unavailable. Please retry shortly.");
        return Mono.just(ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).body(pd));
    }
}
```

14.5 API Gateway vs Load Balancer

	Load Balancer (L4/L7)	API Gateway
Main job	Spread traffic over identical instances of one service	Route to many different services and apply API policies
Awareness	Connections and HTTP basics	API semantics: auth, users, quotas, versions
Features	Health checks, algorithms, TLS termination	Auth, rate limiting, transformation, aggregation, circuit breaking
Examples	AWS ALB/NLB, NGINX, HAProxy, F5	Spring Cloud Gateway, Kong, Apigee, AWS API Gateway
Typical layout	Internet → LB → Gateway (N replicas) → services	The gateway itself sits behind an LB and load-balances to services

Pitfalls: the gateway can become a **single point of failure** (run several replicas behind an LB) or a **bottleneck**, and putting business logic in it turns it into a "god gateway". Keep it thin.

Likely follow-ups:

- *Why is Spring Cloud Gateway reactive?* A gateway spends almost all its time waiting on network I/O. Non-blocking Netty handles thousands of connections with few threads. Never call blocking code in its filters.
- *How does `lb://` work?* `ReactiveLoadBalancerClientFilter` resolves the service name through `DiscoveryClient` (Eureka/K8s) and picks an instance (round-robin by default).
- *Gateway vs BFF?* A BFF is a gateway built for one client type (web vs mobile), with aggregation and response shapes tailored to that client.
- *How does rate limiting work?* A token bucket per key, held in Redis by a Lua script so it's atomic across gateway replicas. When the bucket is empty the client gets 429 with `X-RateLimit-*` headers.

D15. Load Balancer & Application Load Balancer

15.1 L4 vs L7

OSI: L7 Application (HTTP, gRPC) ← ALB / NGINX / Envoy: sees URL, headers, cookies
 L4 Transport (TCP/UDP) ← NLB / HAProxy TCP mode: sees IP:port only

	Layer 4 (Network LB)	Layer 7 (Application LB)
Decides by	IP, port, protocol	URL path, host, headers, cookies, HTTP method, query
TLS	Passes through (or terminates at TLS level)	Terminates TLS, inspects HTTP
Speed	Very fast, millions of req/s, static IPs, ultra-low latency	Slightly more overhead
Features	Simple forwarding, preserves client IP	Path/host routing, redirects, header rewrites, WAF integration, auth (OIDC), sticky cookies, HTTP/2, gRPC, WebSockets
AWS	NLB	ALB
Use	Non-HTTP (MQTT on 8883, databases), extreme throughput, static IP needs	Web/API traffic, microservice routing

Smart City usage: an **NLB** in front of the MQTT brokers (TCP 8883 from IoT devices) and an **ALB** (or NGINX Ingress) in front of the API gateway and Angular static content.

15.2 Algorithms

Algorithm	How	Good for
Round robin	Next server in turn	Similar servers, similar requests
Weighted round robin	More traffic to bigger servers or the stable version	Mixed instance sizes, canary
Least connections	Fewest active connections	Long-lived or uneven requests (WebSockets)
Least response time / "least outstanding requests" (ALB)	Fastest/least busy	Variable latency
IP hash / consistent hashing	Same client → same server	Cache affinity, sharded state
Random / power of two choices	Pick 2 at random, choose the less loaded	Large clusters, low overhead

15.3 Health checks

- **Active:** the LB polls `GET /actuator/health/readiness` every N seconds; after x failures the target is marked unhealthy and taken out of rotation, and after y successes it goes back in.
- **Passive:** watch real traffic (errors/timeouts) and eject bad hosts (Envoy outlier detection).
- Use the **readiness** endpoint (does it depend on DB/Kafka?) for LB health. Keep **liveness** cheap and free of dependencies.

```
management:
  endpoint:
    health:
      probes:
        enabled: true           # /actuator/health/liveness and /readiness
      group:
        readiness:
          include: readinessState, db, redis
```

15.4 Sticky sessions

The LB keeps sending a client to the same instance, based on a cookie (`AWSALB`) or the source IP. You need it for **stateful** apps that keep HTTP sessions in memory. Downsides: uneven load, sessions lost when an instance dies, harder scaling. **Prefer stateless services** (JWT) or an external

session store (Spring Session + Redis). Sticky sessions are still sometimes used for WebSocket/STOMP connections.

15.5 AWS ALB / NLB specifics

- **ALB:** listeners (HTTP/HTTPS) → **rules** (path `/api/*`, host `admin.city.gov`, header) → **target groups** (EC2, IPs, Lambda, ECS/EKS pods via the AWS Load Balancer Controller). Supports ACM certificates, WAF, OIDC/Cognito auth, slow start, deregistration delay (connection draining), and HTTP/2 and gRPC.
- **NLB:** TCP/UDP/TLS, static Elastic IPs, preserves the source IP, handles huge scale. Good for MQTT, or as the entry point for a PrivateLink service.
- **GWLB:** for inline security appliances (firewalls).
- **Cross-zone load balancing** and multi-AZ targets give high availability.

15.6 Server-side vs client-side load balancing

Server-side: A → [LB] → B1 | B2 | B3 (ALB, NGINX, K8s Service/kube-proxy)

Client-side: A (knows B1,B2,B3 from registry) → picks B2 directly (Spring Cloud LoadBalancer + Eureka, gRPC client LB, service-mesh sidecar)

```
@Bean
@LoadBalanced // RestClient resolves "http://user-service" through DiscoveryClient
RestClient.Builder restClientBuilder() { return RestClient.builder(); }
```

Client-side LB removes a network hop and the LB as a bottleneck, but every client needs the logic and a registry. In K8s, a `ClusterIP` Service gives L4 load balancing through kube-proxy. Since gRPC uses long-lived HTTP/2 connections, it needs L7 or client-side LB (a headless Service or a mesh).

Likely follow-ups:

- *What is connection draining?* When a target is deregistered (during deploys), the LB stops sending new requests but lets in-flight ones finish (ALB deregistration delay, default 300 s). Pair it with Spring's `server.shutdown=graceful`.
- *How does the backend get the real client IP behind an ALB?* From the `X-Forwarded-For` header. Set `server.forward-headers-strategy=framework` (or `native`) in Spring Boot.
- *Is the LB a single point of failure?* Managed LBs (ALB/NLB) are redundant across AZs. With self-hosted NGINX/HAProxy, run an active-passive pair with keepalived/a VIP.

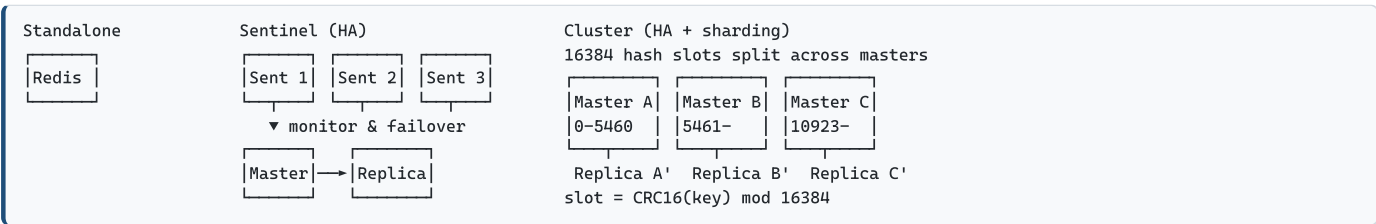
D16. Distributed caching & distributed tracing

Part A: Distributed caching

Why distributed rather than local? With 6 pods of incident-service, a local cache (Caffeine) gives 6 different copies: inconsistent, invalidated separately, and each pod warms up cold. A distributed cache (Redis/Hazelcast) is **shared** by all instances and survives pod restarts.

Two-level cache: L1 Caffeine in-process (microseconds, small, short TTL) + L2 Redis (sub-ms network hop, shared). This works well for very hot reference data such as zones and device types.

Redis topologies:



- **Sentinel:** one master with replicas; Sentinels detect failure and promote a replica. Good for HA when the data fits on one node.
- **Cluster:** shards data across masters by hash slot, each with replicas, so it scales writes and memory. Multi-key operations only work when the keys share a slot (use hash tags: `{incident:42}:details`, `{incident:42}:history`).

Caching patterns:

Pattern	Flow	Pros / Cons
Cache-aside (lazy loading)	App reads the cache → on a miss, reads the DB → writes the cache. On update: write the DB, evict the cache	Most common; only caches what's used. The first read is slow; there's a stale window
Read-through	The cache library loads from the DB on a miss	Simpler app code; needs cache-provider support
Write-through	Write the cache and the DB synchronously	Cache always fresh; slower writes

Pattern	Flow	Pros / Cons
Write-behind (write-back)	Write the cache; flush to the DB asynchronously	Fast writes; risk of data loss
Refresh-ahead	Refresh hot keys before their TTL runs out	No miss spikes; extra load

Common problems:

- **Cache stampede / dogpile:** a hot key expires and 1,000 requests hit the DB. Fixes: `@Cacheable(sync = true)` (per-JVM lock), a distributed lock, refresh-ahead, randomised TTLs.
- **Cache penetration:** requests for IDs that don't exist always miss. Fixes: cache null results briefly, a Bloom filter.
- **Cache avalanche:** many keys expire at once. Fix: TTL jitter.
- **Consistency:** evicting on update is better than updating the cache. For cross-service invalidation, publish events.
- **Hot keys and big values:** shard the key, compress, avoid values over 100 KB.

Spring Boot + Redis:

```
spring:
  data:
    redis:
      cluster:
        nodes: redis-0:6379,redis-1:6379,redis-2:6379
        password: ${REDIS_PASSWORD}
        timeout: 500ms
  cache:
    type: redis
```

```
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    RedisCacheManager cacheManager(RedisConnectionFactory cf) {
        RedisCacheConfiguration defaults = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues()
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer()));
        return RedisCacheManager.builder(cf)
            .cacheDefaults(defaults)
            .withCacheConfiguration("zones", defaults.entryTtl(Duration.ofHours(12)))
            .withCacheConfiguration("incidentSummary", defaults.entryTtl(Duration.ofSeconds(30)))
            .build();
    }
}

@Service
@RequiredArgsConstructor
public class ZoneService {
    private final ZoneRepository repo;

    @Cacheable(cacheNames = "zones", key = "#zoneId", sync = true)
    public ZoneDto get(String zoneId) { return repo.findById(zoneId).map(ZoneDto::from).orElseThrow(); }

    @CacheEvict(cacheNames = "zones", key = "#zone.id")
    @Transactional
    public ZoneDto update(ZoneDto zone) { /* save */ return zone; }
}
```

Remember that `@Cacheable` is proxy-based too: self-invocation bypasses the cache.

Part B: Distributed tracing

Problem: one user click → gateway → incident-service → user-service → Kafka → notification-service. When it's slow or fails, *where?* Logs from five services don't line up unless they share an ID.

Concepts:

- **Trace:** the whole journey of one request, identified by a `traceId` (128-bit).
- **Span:** one unit of work inside it (an HTTP server request, a DB query, a Kafka send) with a `spanId`, `parentSpanId`, start/end time, and tags.
- **Context propagation:** trace IDs travel in headers. The **W3C Trace Context** standard is `traceparent: 00-<traceId>-<spanId>-01` (plus `tracestate`). Older Zipkin **B3** headers are `X-B3-TraceId`, `X-B3-SpanId`, `X-B3-Sampled`, or the single `b3` header.

- **Sampling:** record 100% in dev and 5–10% in prod (or tail-based sampling in the OTEL Collector so all errors are kept).

```

traceId=4bf92f3577b34da6a3ce929d0e0e4736
├─ span A gateway GET /api/incidents/42 [0 — 480ms]
│   └─ span B incident-service GET /api/incidents/{id} [5 — 470ms]
│       └─ span C SELECT incident ... (JDBC) [10 — 40ms]
│           └─ span D user-service GET /api/users/op7 [45 — 430ms] ← the slow one
│               └─ span E SELECT user ... (JDBC) [50 — 420ms] ← missing index!
│                   └─ span F kafka send incident.events [440 — 450ms]
│                       └─ span G notification-service consume (async, linked)

```

Spring Boot 3 setup (Micrometer Tracing replaced Spring Cloud Sleuth):

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId> <!-- or micrometer-tracing-bridge-brave -->
</dependency>
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-zipkin</artifactId> <!-- or opentelemetry-exporter-otlp for Jaeger/Tempo/OTel Collector -->
</dependency>

```

```

management:
  tracing:
    sampling:
      probability: 0.1      # 10% in prod; 1.0 in dev
    propagation:
      type: w3c             # or b3
    zipkin:
      tracing:
        endpoint: http://zipkin:9411/api/v2/spans
    # OTLP alternative:
    # otlp:
    #   tracing:
    #     endpoint: http://otel-collector:4318/v1/traces

logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

spring:
  kafka:
    template:
      observation-enabled: true # propagate trace context into Kafka headers
    listener:
      observation-enabled: true

```

- Auto-instrumented: incoming HTTP (Spring MVC/WebFlux), outgoing `RestTemplate` / `RestClient` / `WebClient` **when built from the Spring-injected builder**, Feign (with `feign-micrometer`), Kafka (with observation enabled), JDBC (with `datasource-micrometer`), `@Observed` methods.
- **MDC:** Micrometer puts `traceId` and `spanId` into the SLF4J MDC, so every log line carries them. In Kibana, search `traceId:4bf92f...` and you get the logs from every service for that request.
- **Custom span:**

```

@Observed(name = "incident.assign", contextualName = "assign-incident")
public void assign(Long incidentId, String officerId) { /* ... */ }

```

(Needs an `ObservedAspect` bean.)

Tools: Zipkin (simple), Jaeger (CNCF), Grafana Tempo, Elastic APM, AWS X-Ray. **OpenTelemetry** is the vendor-neutral standard for traces, metrics, and logs, with the OTEL Collector in between to route and sample.

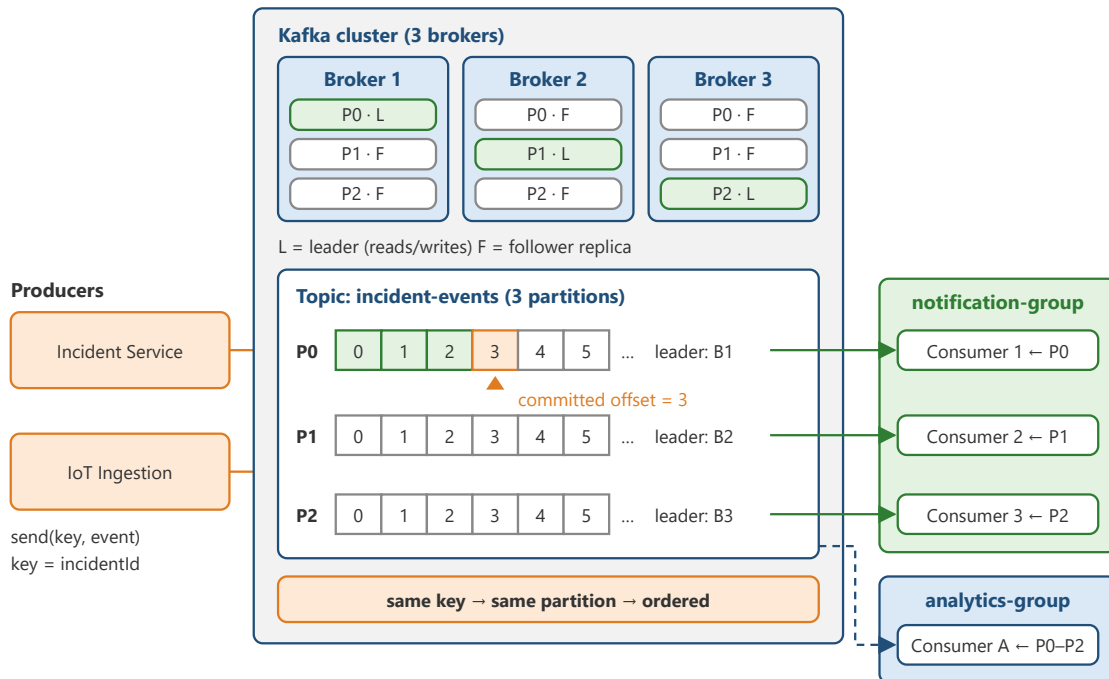
Likely follow-ups:

- *Correlation ID vs trace ID?* A correlation ID is a business or request ID you control, often shown to users or support. A trace ID comes from the tracing system. Many teams use the trace ID as the correlation ID, or log both.
- *How does trace context cross Kafka?* The producer puts `traceparent` in the record headers and the consumer continues the trace (a child span or a span link).
- *Why is my traceId missing in @Async threads?* The context is ThreadLocal. Use a context-propagating `TaskDecorator` (`ContextPropagatingTaskDecorator`) on the executor.

- *What do you cache, and what don't you?* Cache read-heavy data that changes rarely and tolerates brief staleness (zones, device metadata, user profiles). Don't cache rapidly changing or strongly consistent data (incident status during assignment, balances).

D17. Kafka & working flow

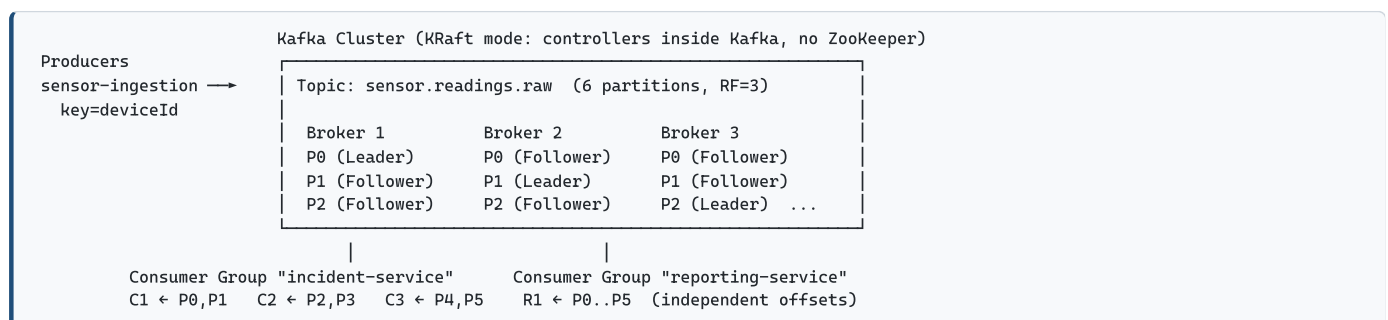
Kafka working flow — topic "incident-events", replication factor 3



Each group keeps its own committed offsets: analytics-group reads the same events independently.

Figure 20: Kafka working flow: producers, brokers, partitions, replicas and consumer groups

17.1 Core concepts



- **Broker:** a Kafka server that stores partitions and serves reads and writes.
- **Topic:** a named stream of records, e.g. `incident.events`.
- **Partition:** an ordered, append-only, immutable **log**. Partitions are the unit of **parallelism** and **ordering**. Each record has an **offset**, its position in the partition.
- **Segment:** each partition is stored as segment files on disk (`.log`, `.index`, `.timeindex`). Old segments get deleted or compacted.
- **Replication:** each partition has one **leader** (it handles all reads and writes by default) and followers that copy it. **ISR (In-Sync Replicas)** = replicas caught up within `replica.lag.time.max.ms`. If the leader dies, a new leader is elected from the ISR.
- **KRaft:** since Kafka 3.3 it's production-ready, and in Kafka 4.0 it's the **only** mode. Metadata lives in an internal Raft-replicated `__cluster_metadata` topic managed by controller nodes. ZooKeeper is gone, so it's simpler to operate, fails over faster, and supports millions of partitions.

17.2 Producer flow

send(record) → serializer → partitioner → RecordAccumulator (batch per partition)

→ Sender thread → leader broker → write to log → followers fetch → ack

- **Partitioning:** with a key, `murmur2(key) % numPartitions`, so **the same key always goes to the same partition**, which keeps ordering per key (e.g. all events for incident 42 stay in order). With no key, the sticky partitioner fills a batch for one partition before moving to the next.
- **Ordering:** guaranteed **only within a partition**. Adding partitions later changes the key→partition mapping, so plan partition counts up front.
- **acks :**
 - `acks=0` : fire and forget. Fastest, can lose data.
 - `acks=1` : the leader wrote it. Lost if the leader dies before followers copy it.
 - `acks=all (-1)` : all ISR replicas have it. Combine with `min.insync.replicas=2` and `RF=3`: the write survives one broker failure, and if the ISR shrinks below 2 the producer gets `NotEnoughReplicasException` instead of silently losing durability.
- **Idempotent producer** (`enable.idempotence=true`, the default since Kafka 3.0): the broker dedupes retries using producer ID + sequence number, so there are no duplicates from producer retries and ordering is kept with `max.in.flight.requests.per.connection ≤ 5`.
- **Throughput tuning:** `batch.size`, `linger.ms`, `compression.type=lz4/zstd`.

17.3 Consumer groups and rebalancing

- Consumers in the **same group** split the partitions: **each partition goes to exactly one consumer in the group**. Parallelism is capped at the partition count (with 6 partitions, a 7th consumer sits idle).
- **Different groups** each get **every** message (pub/sub). `incident-service` and `reporting-service` both read `sensor.alerts` on their own.
- A **group coordinator** (a broker) tracks membership through heartbeats (`session.timeout.ms`, `heartbeat.interval.ms`) and poll calls (`max.poll.interval.ms`). If processing takes longer than `max.poll.interval.ms`, the consumer is kicked out and a **rebalance** happens.
- **Rebalancing** reassigns partitions when consumers join or leave or partitions change.
 - Eager (old): everyone stops and gives up every partition, a "stop the world" pause.
 - **Cooperative incremental** (`CooperativeStickyAssignor`): only the moved partitions are revoked.
 - **Static membership** (`group.instance.id`): a pod restart within the session timeout doesn't trigger a rebalance.
 - **KIP-848 (Kafka 4.0)**: a new consumer rebalance protocol with server-side assignment.

17.4 Offsets and commit strategies

Committed offsets live in the internal `__consumer_offsets` topic, per (group, topic, partition). The committed value is the *next* offset to read.

Strategy	How	Risk
Auto-commit (<code>enable.auto.commit=true</code> , every 5 s)	Commits in the background during <code>poll()</code>	A crash after commit but before processing → loss ; a crash before commit → duplicates
Manual after processing (Spring <code>AckMode.RECORD / BATCH</code> , the Spring default is <code>BATCH</code>)	Commit once the records are processed	At-least-once: duplicates on a crash before commit, so make the consumer idempotent
<code>AckMode.MANUAL(_IMMEDIATE)</code>	Call <code>ack.acknowledge()</code> in code	Full control
Commit offsets in the DB transaction	Store offset with the result in the same DB	Effectively exactly-once into that DB

```
@KafkaListener(topics = "sensor.alerts", groupId = "incident-service",
    properties = {"max.poll.records=100"})
public void onAlert(ConsumerRecord<String, SensorAlert> rec, Acknowledgment ack) {
    incidentService.createFromAlert(rec.value()); // idempotent: unique(alert_id)
    ack.acknowledge(); // needs listener.ack-mode: manual
}
```

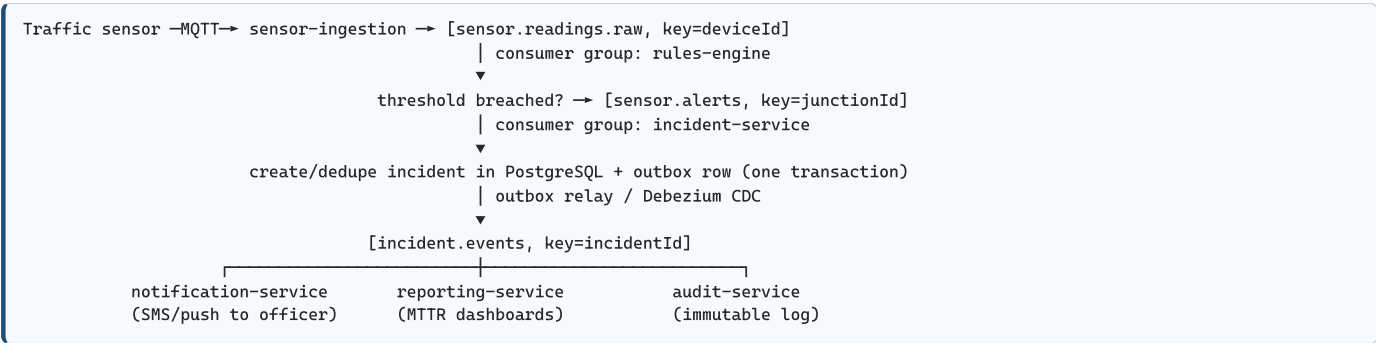
17.5 Delivery semantics

- **At-most-once:** commit before processing. Can lose messages.
- **At-least-once:** process, then commit. Can duplicate. **This is the usual choice, paired with idempotent consumers.**
- **Exactly-once (EOS):** idempotent producer + **transactions** (`transactional.id`), consumer `isolation.level=read_committed`. Gives atomic consume-transform-produce *within Kafka* (Kafka Streams `processing.guarantee=exactly_once_v2`). Side effects outside Kafka (DB, SMS) still need idempotency or an outbox.

17.6 Retention and compaction

- **Delete policy:** keep records for `retention.ms` (default 7 days) or `retention.bytes`, then delete whole segments. Consumers can **replay** anything inside the retention window by resetting offsets.
- **Compact policy** (`cleanup.policy=compact`): keep the **latest value per key** (a tombstone, i.e. a null value, deletes the key). Good for "current state" topics such as `device.status` or changelogs.
- Kafka is fast because of sequential disk writes, the OS page cache, zero-copy (`sendfile`), batching, and compression.

17.7 End-to-end Smart City flow



Likely follow-ups:

- *How many partitions should a topic have?* Enough for peak throughput divided by per-consumer throughput, plus room to grow (12 for `sensor.readings.raw`). Too many means more open files, longer elections, and slower rebalances.
- *What happens when a consumer is slow?* **Consumer lag** grows (watch it with Burrow, Kafka Exporter, or KEDA). Scale consumers up to the partition count, speed up processing, or batch. If `max.poll.interval.ms` is exceeded it gets kicked out and the partitions are rebalanced.
- *Kafka vs RabbitMQ?* Kafka is a distributed log: replay, very high throughput, ordering per partition, consumers pull. RabbitMQ is a smart broker: complex routing, per-message ack and TTL, push-based, messages removed once consumed. Kafka suits event streams; RabbitMQ suits task queues and RPC.
- *How do you handle schema changes?* Schema Registry with Avro or Protobuf and backward-compatible evolution rules (only add optional fields).

D18. One microservice you worked on

Template answer: Incident Management Service. Speak in the first person and give real numbers. Structure: *purpose* → *design* → *integration* → *hard problem* → *impact*. Adapt everything to your own service.

1. Purpose and domain. "The Incident Management Service owns the lifecycle of city incidents, such as signal failures, waterlogging, accidents, garbage overflow, and air-quality breaches. Incidents come from three sources: auto-generated from sensor alerts, raised manually by ICCC operators, and citizen complaints from the mobile app. It handles creation with deduplication, assignment to a field officer or department, SLA tracking and escalation, status updates with an audit history, and closure."

Lifecycle (state machine):



2. Tech stack: Java 17, Spring Boot 3.2, Spring Data JPA (Hibernate 6), PostgreSQL 15 with PostGIS (incident locations), Flyway, Redis, Spring Kafka, Resilience4j, OpenFeign, MapStruct, Spring Security as an OAuth2 resource server, Testcontainers, Docker/Helm on Kubernetes, 3–6 pods with HPA.

3. Main endpoints:

Method	Path	Role	Notes
POST	/api/v1/incidents	OPERATOR, CITIZEN_APP	Idempotency-Key header; validates category and location
GET	/api/v1/incidents?status=&zone=&severity=&from=&to=&page=&size=	OPERATOR+	Paginated, zone-scoped for operators; sort whitelist
GET	/api/v1/incidents/{id}	OPERATOR+	Includes timeline and assignee (Feign → user-service, cached)
PATCH	/api/v1/incidents/{id}/status	FIELD_OFFICER, OPERATOR	Checks the state-machine transition; optimistic locking
POST	/api/v1/incidents/{id}/assign	SUPERVISOR	Publishes IncidentAssigned
POST	/api/v1/incidents/{id}/attachments	FIELD_OFFICER	Pre-signed MinIO/S3 URL; the service never streams large files

Method	Path	Role	Notes
GET	/api/v1/incidents/nearby?lat=&lng=&radiusM=	OPERATOR	PostGIS ST_DWithin
GET	/api/v1/incidents/stats	SUPERVISOR	Cached 30 s

4. Database schema outline:

```
CREATE TABLE incident (  
  id BIGSERIAL PRIMARY KEY,  
  reference_no VARCHAR(20) UNIQUE NOT NULL,           -- INC-2025-000123  
  category_code VARCHAR(30) NOT NULL,                 -- SIGNAL_FAILURE, WATERLOGGING ...  
  severity VARCHAR(10) NOT NULL,                      -- P1..P4  
  status VARCHAR(20) NOT NULL,  
  zone_id VARCHAR(20) NOT NULL,  
  location GEOGRAPHY(POINT, 4326),  
  source VARCHAR(20) NOT NULL,                       -- SENSOR / OPERATOR / CITIZEN  
  source_ref VARCHAR(64),                             -- alertId / complaintId (dedupe)  
  assigned_to VARCHAR(64),  
  sla_due_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL,  
  version INT NOT NULL DEFAULT 0,                    -- @Version optimistic locking  
  CONSTRAINT uq_source UNIQUE (source, source_ref)  
);  
CREATE INDEX ix_incident_zone_status_created ON incident (zone_id, status, created_at DESC);  
CREATE INDEX ix_incident_location ON incident USING GIST (location);  
CREATE INDEX ix_incident_open_sla ON incident (sla_due_at) WHERE status NOT IN ('RESOLVED','CLOSED');  
  
CREATE TABLE incident_history (id BIGSERIAL PK, incident_id BIGINT REFERENCES incident, from_status, to_status,  
                                changed_by, remarks, changed_at);  
CREATE TABLE incident_attachment (id, incident_id, object_key, content_type, uploaded_by, uploaded_at);  
CREATE TABLE outbox_event (id UUID PK, aggregate_type, aggregate_id, event_type, payload JSONB,  
                             created_at, published_at NULL);  
CREATE TABLE idempotency_key (key VARCHAR(64) PK, response_hash, response_body JSONB, created_at);
```

5. Kafka events:

Direction	Topic	Event	Purpose
Consumes	sensor.alerts	SensorThresholdBreached	Auto-create incident (dedupe on source_ref and a 15-min window per junction)
Consumes	citizen.complaints	ComplaintSubmitted	Create a CITIZEN incident
Consumes	user.events	UserDeactivated	Unassign their open incidents
Produces	incident.events (key = incidentId)	IncidentCreated , IncidentAssigned , IncidentStatusChanged , IncidentSlaBreached , IncidentClosed	Notification, reporting, audit, citizen app updates

6. Caching: categories and SLA rules in Caffeine (reloaded on a config event), user/assignee lookups in Redis (10-min TTL), stats in Redis (30 s). Evicted on update.

7. Security: JWT resource server; URL rules plus @PreAuthorize("@zoneGuard.canAccess(#zoneId, authentication)"); field officers only see incidents assigned to them; audit history keeps who did what and when.

8. Challenges and how I solved them (pick 2–3 real ones):

Challenge	Root cause	Solution	Result
Duplicate incidents during sensor storms (one junction sent 200 alerts in 5 min)	At-least-once delivery + many alerts per fault	Unique (source, source_ref) + a time-window dedupe check using a Redis SET NX key dedupe:{junction}:{category} with a 15-min TTL; extra alerts are attached to the existing incident	~95% fewer duplicate incidents; operators stopped complaining about noise
Lost events: incident saved but notification never sent	Dual write: DB commit OK, then kafkaTemplate.send failed	Transactional Outbox: insert into outbox_event in the same transaction; a scheduled relay (@Scheduled + SELECT ... FOR UPDATE SKIP LOCKED) publishes and marks the rows	No lost events since; the relay is idempotent
Concurrent updates: two operators assigned the same incident	Last write wins	@Version optimistic locking → 409 Conflict with the latest state; the UI refreshes	Clean conflict handling

Challenge	Root cause	Solution	Result
Slow incident list API (2.5 s p95)	N+1 on history and attachments, OFFSET pagination on large tables, no composite index	DTO projection query, @EntityGraph , composite index (zone_id, status, created_at) , keyset pagination for infinite scroll	p95 dropped to ~180 ms
SLA escalation job ran on every pod	@Scheduled on 4 replicas	ShedLock (@SchedulerLock) with a JDBC lock provider	Exactly one escalation run per minute

9. Metrics and observability: custom Micrometer metrics incidents_created_total{source,category} , incident_sla_breaches_total , and a incident_open_gauge ; Grafana dashboard for MTTA/MTTR; alerts on Kafka consumer lag and SLA-breach spikes.

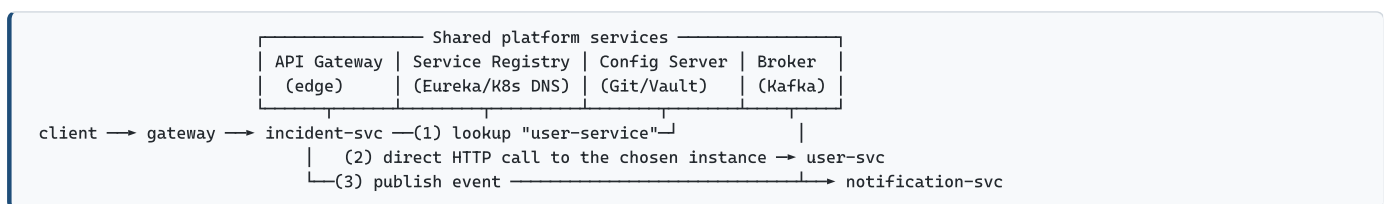
10. Testing: unit tests for the state machine (parameterised for every transition), @WebMvcTest for controllers and security, Testcontainers (PostgreSQL + Kafka) integration tests, contract tests with notification-service; about 80% coverage gated in SonarQube.

Likely follow-ups:

- *How did you implement the state machine?* An enum with the allowed transitions (Map<Status, Set<Status>>) checked in the domain method incident.transitionTo(newStatus, actor) . That was simpler than Spring Statemachine for our needs.
- *How does the outbox relay avoid publishing the same event twice from several pods?* FOR UPDATE SKIP LOCKED row locks plus idempotent consumers (they dedupe on eventId). An alternative is Debezium CDC reading the outbox table.
- *What would you improve?* Moving the outbox relay to Debezium, adding Schema Registry for event contracts, and moving reporting queries to a separate read model.

D19. Do microservices connect directly or via shared services?

Short answer: both. *Business calls* go service to service, synchronously over HTTP/gRPC or asynchronously through a broker. They rely on **shared infrastructure services** to find each other, get configuration, and stay decoupled. What services should **never** share is a **database**.



Shared component	Role	Pros	Cons / risks
API Gateway	Edge entry for <i>external</i> clients; usually not used for internal east-west calls	Central security, routing, throttling	SPOF/bottleneck if not replicated; avoid a "god gateway"
Service registry / discovery (Eureka, Consul, K8s DNS)	Services register themselves; callers look up healthy instances by name	No hard-coded hosts; elastic scaling	Another component to run (K8s DNS removes it); stale entries
Config Server (Spring Cloud Config, K8s ConfigMaps, Consul KV)	Externalised, versioned, per-environment config; @RefreshScope / Spring Cloud Bus for live refresh	One source of truth, audited in Git	Startup dependency (use optional:configserver: + fail-fast with retry)
Message broker (Kafka/RabbitMQ)	Async, decoupled integration and event streaming	Temporal decoupling, buffering, fan-out, replay	Eventual consistency, broker operations, harder debugging
Shared libraries (common DTOs, security auto-config, logging, error format)	Reuse cross-cutting code as a versioned internal starter (smartcity-common-starter)	Consistency, less boilerplate	Coupling: one shared domain library forces lock-step releases. Share <i>infrastructure</i> code, not <i>domain models</i> ; prefer contracts (OpenAPI/Avro) and generate clients
Service mesh (Istio/Linkerd)	A sidecar handles mTLS, retries, traffic splitting, telemetry	Resilience and security out of process, language-agnostic	Operational complexity, extra latency and resources
Shared database	Anti-pattern	Easy joins, quick to start	Tight coupling, schema changes break other services, no independent scaling: a distributed monolith

Direct vs mediated communication:

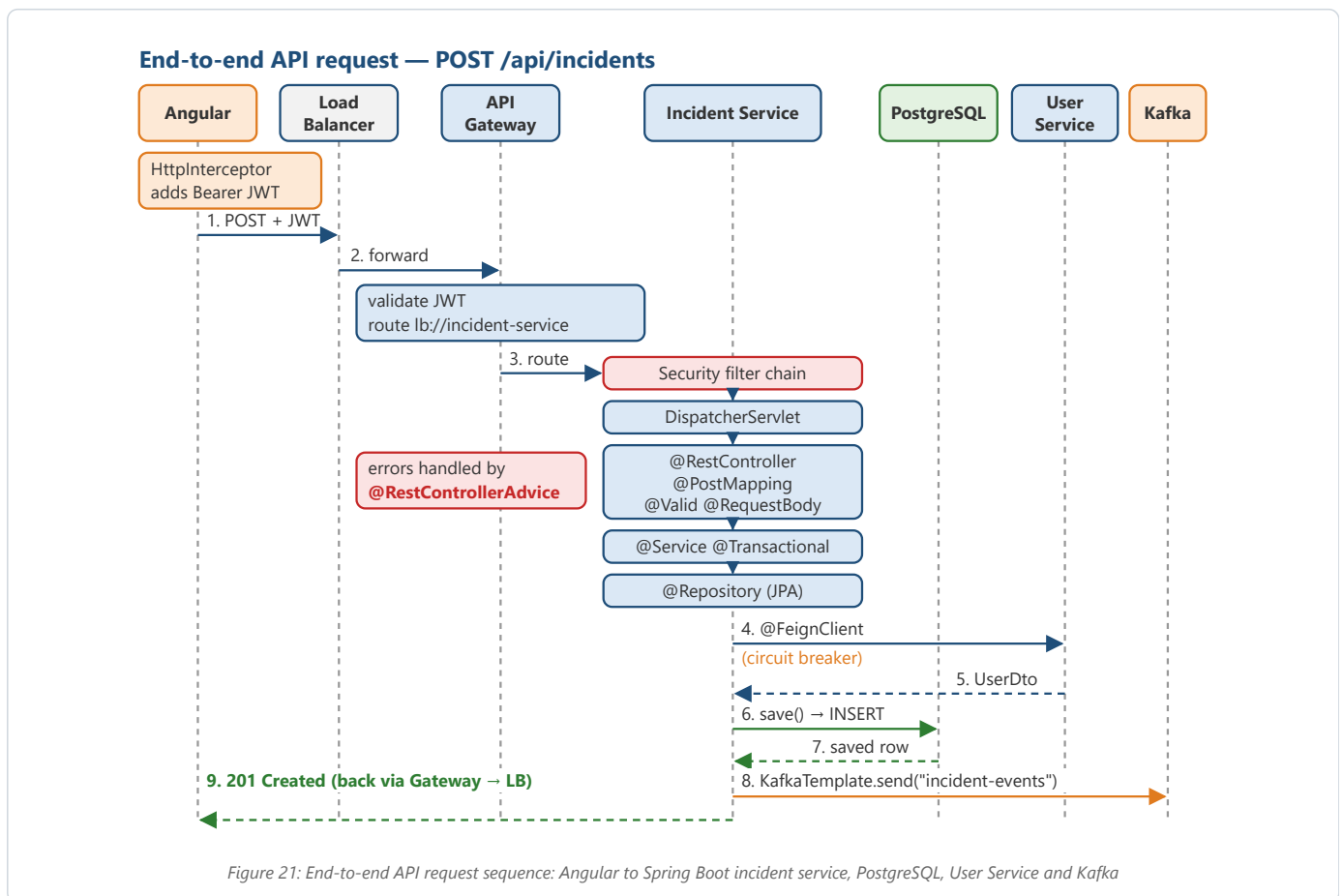
	Direct (A → B over HTTP via discovery)	Via broker (A → Kafka → B)
Coupling	A knows B's API and needs B up	A only knows the topic/event schema

	Direct (A → B over HTTP via discovery)	Via broker (A → Kafka → B)
Use	Queries that need an immediate answer	Events, commands that can be deferred, fan-out
Failure	Needs timeouts, circuit breakers	Broker absorbs B's downtime

Likely follow-ups:

- *Should internal calls go through the API gateway?* Generally no. It adds a hop and makes the gateway a bottleneck. Use discovery or K8s DNS directly, or a mesh. Some organisations put an *internal* gateway between domains.
- *Eureka vs Kubernetes service discovery?* On K8s, use the built-in Services/DNS: no extra server, and kube-proxy load-balances. Eureka is useful outside K8s or in hybrid VM setups.
- *How do you share DTOs without coupling?* Publish an OpenAPI or Avro contract, generate client code per consumer, and version the contract. Use a tolerant reader (ignore unknown fields).

D20. Complete API request flow through microservices (with operations and annotations)



Scenario: an operator creates an incident from Angular: `POST /api/v1/incidents`. Incident-service validates the reporter through user-service (Feign), saves to PostgreSQL, and publishes an event to Kafka, which notification-service consumes.

```

Browser → DNS → LB/Ingress (TLS) → API Gateway → incident-service → PostgreSQL
                                     |
                                     | Feign → user-service
                                     | Kafka → notification-service
  
```

Step by step:

#	Where	What happens	Key class / annotation
1	Angular	A component calls <code>incidentService.create(dto) → HttpClient.post('/api/v1/incidents', body)</code> . The <code>AuthInterceptor</code> adds <code>Authorization: Bearer <JWT></code> , <code>X-Correlation-Id</code> , and an <code>Idempotency-Key</code>	<code>HttpInterceptor</code>
2	Browser	Cross-origin → CORS preflight OPTIONS first (if the origin differs); DNS lookup; TCP + TLS handshake (HTTP/2)	—

#	Where	What happens	Key class / annotation
3	LB / Ingress	Terminates TLS (or passes it through), adds <code>X-Forwarded-For/Proto</code> , picks a healthy gateway pod	ALB / NGINX Ingress
4	API Gateway	<code>CorrelationIdGlobalFilter</code> → Spring Security WebFlux validates the JWT (signature via JWKS, exp, iss, aud) → <code>RequestRateLimiter</code> (Redis token bucket) → route predicate <code>Path=/api/incidents/**</code> matches → <code>CircuitBreaker</code> filter → <code>ReactiveLoadBalancerClientFilter</code> resolves <code>lb://incident-service</code> to a pod IP → proxied over Netty	<code>SecurityWebFilterChain</code> , <code>GlobalFilter</code> , route YAML
5	Tomcat (incident-service)	Connector accepts the request; a worker thread from the pool (<code>server.tomcat.threads.max</code>) is assigned (or a virtual thread with <code>spring.threads.virtual.enabled=true</code>)	—
6	Servlet filters	<code>ServerHttpObservationFilter</code> (starts a tracing span, MDC traceId) → <code>CharacterEncodingFilter</code> → <code>DelegatingFilterProxy</code> → <code>FilterChainProxy</code> → SecurityFilterChain : <code>BearerTokenAuthenticationFilter</code> decodes the JWT → <code>JwtAuthenticationToken</code> in <code>SecurityContextHolder</code> → <code>AuthorizationFilter</code> checks <code>hasAnyRole('OPERATOR', 'ADMIN')</code> for POST <code>/api/v1/incidents/**</code>	<code>SecurityFilterChain</code> , <code>OncePerRequestFilter</code>
7	DispatcherServlet	<code>HandlerMapping</code> (<code>RequestMappingHandlerMapping</code>) finds the controller method → <code>HandlerAdapter</code> → HandlerInterceptor.preHandle (e.g. tenant/zone resolution)	<code>DispatcherServlet</code> , <code>HandlerInterceptor</code>
8	Argument resolution	<code>HttpMessageConverter</code> (Jackson <code>MappingJackson2HttpMessageConverter</code>) deserialises JSON → <code>CreateIncidentRequest</code> record; @Valid triggers Bean Validation (<code>@NotBlank</code> , <code>@Size</code> , <code>@NotNull</code>) → on failure <code>MethodArgumentNotValidException</code> → 400	<code>@RequestBody</code> , <code>@Valid</code> , <code>@RequestHeader</code>
9	Controller	@RestController method: maps the request and calls the service; returns <code>ResponseEntity.created(uri).body(dto)</code>	<code>@RestController</code> , <code>@RequestMapping</code> , <code>@PostMapping</code>
10	Service (AOP proxy)	The proxy applies, in order: method security <code>@PreAuthorize("@zoneGuard.canAccess(#req.zoneId(), authentication)")</code> → @Transactional (<code>TransactionInterceptor</code> gets a connection from the HikariCP pool, <code>setAutoCommit(false)</code>) → optional <code>@Retry</code> / <code>@CircuitBreaker</code> on the Feign call	<code>@Service</code> , <code>@PreAuthorize</code> , <code>@Transactional</code>
11	Idempotency check	<code>idempotencyRepo.findById(key)</code> . If present, return the stored response	—
12	Feign call to user-service	<code>UserFeignClient.getUser(reporterId)</code> → <code>RequestInterceptor</code> adds the client-credentials token and correlation ID → Spring Cloud LoadBalancer picks an instance → HTTP call wrapped by Resilience4j CB/timeout. user-service runs its own steps 5–14 and returns <code>UserDto</code>	<code>@FeignClient</code> , <code>@CircuitBreaker</code>
13	Domain logic	Map DTO → entity (MapStruct), compute <code>slaDueAt</code> from SLA rules (Caffeine cache, <code>@Cacheable</code>), generate <code>referenceNo</code> , set status NEW	<code>@Cacheable</code> , <code>@Mapper</code>
14	Repository	<code>incidentRepository.save(incident)</code> → Spring Data proxy → <code>EntityManager.persist</code> → Hibernate assigns an ID (sequence). Then <code>outboxRepository.save(new OutboxEvent("IncidentCreated", payload))</code> in the same transaction	<code>@Repository</code> / <code>JpaRepository</code> , <code>@Entity</code> , <code>@Version</code>
15	Commit	The method returns → <code>TransactionInterceptor</code> commits → Hibernate flushes (INSERT incident, INSERT outbox_event, INSERT history), JDBC commit, connection back to the pool. On a <code>RuntimeException</code> → rollback	<code>@Transactional</code>
16	Event publish	The outbox relay (<code>@Scheduled</code> + <code>ShedLock</code>) reads unpublished rows → <code>kafkaTemplate.send("incident.events", incidentId, event)</code> (acks=all, idempotent) → marks them published. (A simpler variant: <code>@TransactionalEventListener(phase = AFTER_COMMIT)</code> sends to Kafka, but a crash before the send loses the event.)	<code>KafkaTemplate</code> , <code>@Scheduled</code> , <code>@SchedulerLock</code>
17	Response path	Controller returns <code>ResponseEntity</code> 201 with a <code>Location</code> header → <code>HandlerInterceptor.postHandle/afterCompletion</code> → Jackson serialises the <code>IncidentDto</code> → filters unwind (the security filter clears the context, the tracing span ends, the access log is written) → Tomcat writes the response	<code>ResponseEntity</code> , <code>HttpMessageConverter</code>
18	Errors (any step 8–15)	Exception → <code>DispatcherServlet</code> → <code>ExceptionHandlerExceptionResolver</code> → @RestControllerAdvice <code>@ExceptionHandler</code> → <code>ProblemDetail</code> JSON (400 validation, 404 not found, 409 optimistic lock / duplicate, 503 downstream unavailable). Security errors in filters → <code>AuthenticationEntryPoint</code> / <code>AccessDeniedHandler</code> (D3)	<code>@RestControllerAdvice</code> , <code>@ExceptionHandler</code> , <code>ProblemDetail</code>
19	Gateway → browser	Gateway post-filters add response headers and log latency → LB → browser. Angular gets 201, shows a toast, navigates to the incident page	—

#	Where	What happens	Key class / annotation
20	Async consumer	notification-service's <code>@KafkaListener(topics="incident.events")</code> receives the event (trace context continued from Kafka headers) → idempotency check on <code>eventId</code> → sends SMS/push, retried via <code>@RetryableTopic</code> , DLT on final failure → pushes a WebSocket update to dashboards (<code>SimpMessagingTemplate.convertAndSend("/topic/incidents", ...)</code>)	<code>@KafkaListener</code> , <code>@RetryableTopic</code> , <code>@DLTHandler</code>

Code skeleton for steps 8–15:

```

public record CreateIncidentRequest(
    @NotBlank @Size(max = 30) String categoryCode,
    @NotNull @Pattern(regexp = "P[1-4]") String severity,
    @NotBlank String zoneId,
    @NotNull @DecimalMin("-90") @DecimalMax("90") Double lat,
    @NotNull @DecimalMin("-180") @DecimalMax("180") Double lng,
    @Size(max = 2000) String description,
    @NotBlank String reporterId) {}

@RestController
@RequestMapping("/api/v1/incidents")
@RequiredArgsConstructor
public class IncidentController {

    private final IncidentService incidentService;

    @PostMapping
    public ResponseEntity<IncidentDto> create(@Valid @RequestBody CreateIncidentRequest req,
                                              @RequestHeader("Idempotency-Key") String idemKey) {
        IncidentDto dto = incidentService.create(req, idemKey);
        URI location = URI.create("/api/v1/incidents/" + dto.id());
        return ResponseEntity.created(location).body(dto);
    }
}

@Service
@RequiredArgsConstructor
public class IncidentService {

    private final IncidentRepository incidentRepository;
    private final OutboxRepository outboxRepository;
    private final UserFeignClient userClient;
    private final IncidentMapper mapper;
    private final SlaPolicy slaPolicy;
    private final ObjectMapper objectMapper;

    @PreAuthorize("@zoneGuard.canAccess(#req.zoneId(), authentication)")
    @Transactional
    public IncidentDto create(CreateIncidentRequest req, String idemKey) {
        UserDto reporter = userClient.getUser(req.reporterId()); // Feign + circuit breaker
        if (!reporter.active()) {
            throw new BusinessException("Reporter is inactive");
        }
        Incident incident = mapper.toEntity(req);
        incident.setStatus(IncidentStatus.NEW);
        incident.setSlaDueAt(slaPolicy.dueAt(req.categoryCode(), req.severity()));
        Incident saved = incidentRepository.save(incident);

        outboxRepository.save(OutboxEvent.of("Incident", saved.getId(), "IncidentCreated",
            toJson(mapper.toEvent(saved)))); // same TX as the insert
        return mapper.toDto(saved);
    }

    private String toJson(Object o) {
        try { return objectMapper.writeValueAsString(o); }
        catch (JsonProcessingException e) { throw new IllegalStateException(e); }
    }
}

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail validation(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Validation failed");
        pd.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
            .map(f -> Map.of("field", f.getField(), "message", String.valueOf(f.getDefaultMessage())))
            .toList());
        return pd;
    }

    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)

```

```

public ProblemDetail conflict() {
    return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, "Incident was modified by someone else");
}

@ExceptionHandler(CallNotPermittedException.class)
public ProblemDetail downstream() {
    return ProblemDetail.forStatusAndDetail(HttpStatus.SERVICE_UNAVAILABLE, "Dependent service unavailable");
}
}

```

Design note: a remote Feign call *inside* `@Transactional` holds a DB connection while it waits on the network. Under load this can drain the HikariCP pool. A better approach is to do the remote validation **before** opening the transaction (a separate non-transactional method or an orchestrating bean), then open a short transaction just for the DB writes.

Likely follow-ups:

- *Where does `@Transactional` start and end?* At the proxy boundary of the service method. It commits when the method returns normally, and rolls back on a `RuntimeException` / `Error` (checked exceptions don't trigger a rollback unless you set `rollbackFor`).
- *What does `spring.jpa.open-in-view` do and should it be on?* It keeps the Hibernate session open through view rendering, so lazy loading works in controllers but connections are held longer. Set it to `false` in REST APIs.
- *What's the difference between a filter and an interceptor?* Filters are servlet-level (before `DispatcherServlet`, they see raw requests; security lives here). Interceptors are Spring MVC-level (they know the handler method and run around the controller).

D21. API works internally but not for the client: investigation checklist

Mindset: "Works for us, fails for them" means *something in the client's path or context is different*. Narrow it down layer by layer, **from the outside in**, and confirm each step with evidence before moving on.

Client → Client's network/proxy → Internet/DNS → WAF/Firewall → LB (TLS) → Gateway → Service → DB/downstream
 ① ② ③ ④ ⑤⑥⑦ ⑧⑨⑩

Step	Check	How / commands	Typical finding
1. Reproduce with the client's exact request	Get the full request (URL, method, headers, body, time, client IP) and the exact error (status code, body, client log). Replay it with <code>curl -v</code> / Postman from <i>outside</i> your network	<code>curl -v -X POST https://api.city.gov/api/v1/incidents -H "Authorization: Bearer ..." -d @body.json</code>	Different URL (http vs https, missing /v1), different payload
2. Network / DNS / firewall / whitelisting	Does the client's DNS resolve the right IP? Can it reach the port? Is the client IP allow-listed at the WAF, security group, or partner firewall? Corporate proxy?	<code>nslookup api.city.gov</code> , <code>dig</code> , <code>traceroute</code> , <code>telnet host 443</code> / <code>nc -vz host 443</code> ; WAF/SG logs	Client IP not in allow-list; WAF rule blocking a payload pattern; split-horizon DNS pointing to an internal IP
3. Gateway route & load balancer	Is the route exposed on the <i>external</i> gateway/ingress, or only the internal one? Path rewrite correct? Do LB target-group health checks pass? Does it fail on only one AZ or pod?	Gateway access logs, <code>/actuator/gateway/routes</code> , ALB target health, ingress config	Route only on the internal ingress; 502/504 from an unhealthy target; LB idle timeout < API duration
4. TLS certificate	Expired cert, incomplete chain (missing intermediate), hostname mismatch, client trusts an old CA, TLS version or cipher mismatch, mTLS client cert missing or expired	<code>openssl s_client -connect api.city.gov:443 -servername api.city.gov -showcerts</code> ; SSL Labs	Missing intermediate cert (browsers cope, Java/curl clients fail); client on TLS 1.0
5. CORS (browser clients only)	Browser console: "blocked by CORS policy". Preflight <code>OPTIONS</code> returns 401/403, or the origin isn't allowed	DevTools → Network → preflight; check <code>Access-Control-Allow-Origin</code>	Client's origin (<code>https://partner.app</code>) missing from the allowed list; security blocks <code>OPTIONS</code> ; duplicate CORS headers from gateway + service
6. Authentication / authorization / scopes	401: token from a different issuer or environment, expired, wrong audience, clock skew. 403: missing role or scope; client registered without the scope	Decode the JWT (jwt.io, locally); compare <code>iss</code> , <code>aud</code> , <code>scope</code> , <code>exp</code> with config; security debug logs	Client uses a UAT token on prod; <code>aud</code> mismatch; scope <code>incident.write</code> not granted

Step	Check	How / commands	Typical finding
7. Rate limits / quotas	429s? A per-client quota smaller than for internal callers? WAF bot protection?	Gateway rate-limit metrics, <code>X-RateLimit-Remaining</code> headers	Partner's batch job hits 20 req/s and gets throttled
8. Payload / contract / version	Different API version, content-type (<code>text/plain</code> vs <code>application/json</code>), encoding, date formats, field names, enum values, required headers (<code>Idempotency-Key</code>), request size limit	Compare against OpenAPI; server logs of the 400/415 detail; <code>spring.servlet.multipart.max-file-size</code> , <code>nginx client_max_body_size</code> (413)	The client sends the old v1 schema; a 413 from nginx on large uploads; the timezone format differs
9. Environment-specific config	Does the client hit a different environment, region, or tenant? Feature flags, config-server profile, secrets differ? Is a downstream service only reachable internally?	Compare the config between envs; <code>/actuator/env</code> (secured); check which pod/version served it	Prod config missing the new route or flag; external DNS still points to the old cluster
10. Logs via correlation ID / trace	Ask the client for the <code>X-Correlation-Id</code> from the response (or the timestamp + client ID). Search Kibana and Zipkin: did the request reach the gateway? The service? Where did it stop?	Kibana <code>correlationId:"..."</code> ; Zipkin trace; gateway access logs	The request never reached the gateway (so a network problem), or the service threw a specific exception

Key principle: if there's **no log at all** at the gateway, the problem is *before* it (DNS, firewall, TLS, LB). If the gateway logged a 401/403/429, it's a *policy* problem. If the service logged an error, it's *app/data/config*.

Communicate: acknowledge to the client, share findings with evidence, fix (config change, allow-list, cert chain), confirm with the client, and add monitoring or a synthetic check (an external probe such as a Blackbox exporter) so it's caught earlier next time.

Likely follow-ups:

- *A Java client fails with `PKIX path building failed` but browsers work. Why?* The server isn't sending the intermediate certificate (browsers fetch or cache it, Java doesn't), or the client's truststore lacks the CA. Fix the chain on the server.
- *The client gets 504 only for large reports.* The LB, gateway, or client timeout is shorter than processing time. Make it async (202 + polling or a webhook), or paginate and stream.
- *Postman works but the browser doesn't.* Almost always CORS, or a cookie/SameSite issue. Postman doesn't enforce CORS.

D22. Slow API: primary investigation

Method: **measure** → **locate** → **diagnose** → **fix** → **verify**. Never guess. Profile or trace first.

1. MEASURE What's slow? Which endpoint, which percentile (p50 vs p99), since when, for whom, at what load? Is it every request or spikes? Correlated with deploys, traffic, batch jobs, time of day?
2. LOCATE Distributed trace / APM → which span takes the time: gateway, service code, DB, downstream, Kafka?
3. DIAGNOSE Drill into that layer (DB, downstream, JVM/app, network/payload)
4. FIX The smallest change for the biggest gain (index, cache, async, pool size)
5. VERIFY Load test (JMeter/Gatling/k6) before vs after; watch p95/p99 in Grafana after release

Step 1: measure (tools)

- **Grafana / Prometheus** using Actuator's `http_server_requests_seconds` (enable histograms with `management.metrics.distribution.percentiles-histogram.http.server.requests=true`). Look at the p95/p99 trend per `uri`, error rate, and throughput.
- Confirm whether it's the *server* or the *client/network*: compare the gateway's latency with the service's latency.

Step 2: locate with tracing. Zipkin/Jaeger/APM (Elastic APM, Dynatrace, New Relic, Glowroot) show the span waterfall. That tells you whether 80% of the time is in a SQL span, a Feign call, or the service itself (a gap with no child spans = CPU, locks, GC, or waiting on a pool).

Step 3: diagnose by layer

Layer	Symptoms	What to check	Tools
Database	Long JDBC spans, many SQL spans per request	Slow queries, N+1 (100 SELECTs per request), missing indexes, locks/blocking, large result sets, no pagination	<code>spring.jpa.properties.hibernate.generate_statistics=true</code> , <code>logging.level.org.hibernate.SQL=debug</code> , <code>p6spy/datasource-proxy</code> , <code>pg_stat_statements</code> , slow query log, <code>EXPLAIN ANALYZE</code> (D23)

Layer	Symptoms	What to check	Tools
Connection pool	Time spent <i>before</i> SQL runs	HikariCP <code>hikaricp_connections_pending > 0</code> , <code>hikaricp_connections_acquire_seconds</code> high; pool too small, or connections held during remote calls or long transactions	Actuator Hikari metrics, <code>leak-detection-threshold</code>
Downstream services	Long HTTP client spans	The downstream is slow itself, no timeouts, sequential calls that could run in parallel, retries piling up	Traces, CB metrics, client metrics <code>http_client_requests_seconds</code>
App: threads	Gaps in the trace, high latency under load only	Tomcat thread pool exhausted (<code>tomcat_threads_busy = max</code>), blocking calls in WebFlux event loops, <code>synchronized</code> hot spots, deadlocks	Thread dumps (<code>jstack <pid></code> , <code>jcmd <pid> Thread.print</code> , <code>/actuator/threaddump</code>); take 3 dumps 5 s apart and look for many threads <code>BLOCKED</code> on the same lock or <code>WAITING</code> on the pool
App: GC / memory	Periodic latency spikes, high CPU	Long GC pauses, heap near max, high allocation rate, memory leak	GC logs (<code>-Xlog:gc*</code>), <code>jvm_gc_pause_seconds</code> , JFR (<code>jcmd <pid> JFR.start duration=60s filename=rec.jfr</code> → JDK Mission Control), heap dump + Eclipse MAT
App: CPU / code	High CPU, gap spans	Inefficient algorithms ($O(n^2)$ loops), regex, heavy JSON serialisation of huge objects, excessive logging (DEBUG in prod), reflection-heavy mapping	JFR method profiling, async-profiler flame graphs
Network / payload	Large response sizes, slow on mobile	Responses in MBs (no pagination, entire entity graphs), no compression, chatty APIs (10 calls per screen)	Response size metric, <code>server.compression.enabled=true</code> , DevTools waterfall
Infrastructure	All services slow	CPU throttling from K8s limits (<code>container_cpu_cfs_throttled_seconds_total</code>), noisy neighbours, node pressure, DB server CPU/IO	Kubernetes metrics, node exporter
Cache	Slow after a deploy or at expiry times	Cold cache, stampede, low hit ratio	<code>cache_gets_total{result="miss"}</code>

Common fixes:

- $N+1$ → `JOIN FETCH` / `@EntityGraph` / `@BatchSize` / DTO projections
- Missing index → add a composite or covering index
- Big results → pagination (keyset for deep pages), projections, compression
- Sequential downstream calls → parallel (`CompletableFuture` / `WebClient.zip`), cache, or precompute with events
- Pool exhaustion → keep transactions short, don't make remote calls inside transactions, tune the pool (`maximum-pool-size` $\approx$ what the DB can handle, not "bigger is better")
- Heavy work in the request path → move it async (Kafka, `@Async`) and return 202
- GC → right-size the heap, cut allocations, use G1/ZGC
- High-concurrency I/O → virtual threads (Java 21, `spring.threads.virtual.enabled=true`)

Example story: "The incident dashboard API's p95 went from 300 ms to 3 s after a release. Zipkin showed 120 SELECT spans per request. Hibernate statistics confirmed an $N+1$ on `incident.history`, caused by a new field in the DTO mapping. I replaced it with a DTO projection query and `@EntityGraph`, and added a composite index on `(zone_id, status, created_at)`. p95 came back to 180 ms, which I verified with a Gatling run at 200 rps. I then added a Grafana alert on `p95 > 800 ms`."

Likely follow-ups:

- *Why look at p99 rather than the average?* Averages hide tail latency. A few very slow requests hurt users and pile up across fan-out calls.
- *How do you read a thread dump for a slowness issue?* Group threads by state and stack. Many `http-nio-*` threads `WAITING` in `HikariPool.getConnection` = pool starvation. Many `BLOCKED` on the same monitor = lock contention.
- *What is JFR's overhead?* Very low (~1–2%) with the default profile, so it's safe to run briefly in production.

D23. Delay from a stored procedure / SQL: identify and fix

23.1 Identify the slow statement

1. **Confirm the DB is where the time goes.** A trace span or JDBC timing (datasource-proxy/p6spy) shows the slow SQL or the `{call proc(?)}`.

2. DB-side tooling:

- PostgreSQL: `pg_stat_statements` (top queries by `total_exec_time`, `mean_exec_time`), `log_min_duration_statement = 500ms`, `auto_explain` (logs plans of slow queries, including the ones *inside* functions with `auto_explain.log_nested_statements=on`), `pg_stat_activity` (running and blocked queries), `pg_locks`.
- MySQL: slow query log (`long_query_time`), `performance_schema`, `SHOW PROCESSLIST`.
- SQL Server: Query Store, `sys.dm_exec_query_stats`, Extended Events, actual execution plans.
- Oracle: AWR/ASH reports, `V$SQL`, `DBMS_XPLAN.DISPLAY_CURSOR`.

3. **Inside a stored procedure:** find *which statement* is slow. Add timing logs (`RAISE NOTICE clock_timestamp()` in PL/pgSQL, `SET STATISTICS TIME ON` in T-SQL), use nested-statement plan logging, or run each statement on its own with representative parameters.

23.2 Read the execution plan

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT i.id, i.reference_no, i.status, i.created_at
FROM incident i
WHERE i.zone_id = 'Z-07' AND i.status = 'NEW'
ORDER BY i.created_at DESC
LIMIT 50;
```

BEFORE:

```
Limit (actual time=1830..1830 rows=50)
-> Sort (Sort Key: created_at DESC) Sort Method: top-N heapsort
-> Seq Scan on incident i (actual rows=4120 loops=1)
    Filter: (zone_id = 'Z-07' AND status = 'NEW')
    Rows Removed by Filter: 2895880      ← scanned 2.9M rows to find 4k
    Buffers: shared read=48210          ← heavy disk IO
Execution Time: 1832 ms
```

```
AFTER CREATE INDEX ix_incident_zone_status_created ON incident (zone_id, status, created_at DESC);
Limit (actual time=0.05..0.21 rows=50)
-> Index Scan using ix_incident_zone_status_created on incident i
    Index Cond: (zone_id = 'Z-07' AND status = 'NEW') ← no sort needed: index order matches
Execution Time: 0.3 ms
```

What to look for in a plan:

- Seq Scan / Table Scan / Full Table Scan** on large tables with selective filters → missing or unusable index.
- Estimated vs actual rows** far apart (estimate 10, actual 2M) → stale statistics, which lead to the wrong join type.
- Nested Loop** with a large outer side, **Hash Join** spilling to disk, **Sort** spilling (`external merge Disk`) → `work_mem`, indexes, query shape.
- Key Lookup / bookmark lookup** (SQL Server), many heap fetches → make it a covering index.
- High Buffers: shared read → IO-bound. `loops=N` multiplies the cost.

23.3 Fixes

Problem	Fix	Example
Missing index	B-tree on filter/join/order columns	<code>CREATE INDEX ON incident (assigned_to)</code>
Composite index column order	Equality columns first, then range/sort columns. The leftmost-prefix rule applies	<code>(zone_id, status, created_at)</code> serves <code>WHERE zone_id=? AND status=? ORDER BY created_at</code>
Covering index	Put every selected column in the index → index-only scan, no table lookup	PG: <code>CREATE INDEX ... (zone_id, status) INCLUDE (reference_no, severity);</code> SQL Server <code>INCLUDE</code>
Partial / filtered index	Index only the "hot" rows	<code>CREATE INDEX ... ON incident (sla_due_at) WHERE status NOT IN ('RESOLVED', 'CLOSED')</code>
Non-sargable predicates (the index can't be used)	Don't wrap indexed columns in functions or casts; avoid leading wildcards; avoid implicit type conversion; OR across columns → <code>UNION ALL</code>	✗ <code>WHERE DATE(created_at) = '2025-06-01'</code> → ✓ <code>WHERE created_at >= '2025-06-01' AND created_at < '2025-06-02'</code> ; ✗ <code>WHERE LOWER(email)=?</code> → functional index <code>ON (LOWER(email))</code> ; ✗ <code>LIKE '%abc'</code> → trigram (<code>pg_trgm</code>) / full-text index; ✗ <code>varchar_col = 123</code> (implicit cast)
Parameter sniffing (SQL Server, also Oracle bind peeking, PG generic plans)	The plan is compiled for the first parameter value (e.g. a tiny zone) and reused for a huge zone → a bad plan	<code>OPTION (RECOMPILE)</code> , <code>OPTIMIZE FOR UNKNOWN</code> , split procedures per data shape, local variables; PG: <code>plan_cache_mode = force_custom_plan</code>
Stale statistics	Wrong row estimates → wrong join or scan choice	PG <code>ANALYZE incident;</code> / tune autovacuum; SQL Server <code>UPDATE STATISTICS</code> ; Oracle <code>DBMS_STATS.GATHER_TABLE_STATS</code>

Problem	Fix	Example
Cursors / row-by-row loops in the procedure ("RBRAR")	Rewrite as set-based SQL: one UPDATE/INSERT...SELECT instead of a loop of 100k single-row statements	See below
Deep OFFSET pagination	OFFSET 100000 reads and throws away 100k rows	Keyset/seek pagination: WHERE (created_at, id) < (:lastCreated, :lastId) ORDER BY created_at DESC, id DESC LIMIT 50
Huge tables (sensor readings, billions of rows)	Partitioning by time (range) → partition pruning, cheap retention (drop the old partition)	PARTITION BY RANGE (reading_ts), monthly partitions, or TimescaleDB hypertables
Locking / blocking	Long transactions, lock escalation, missing FK index causing table locks	Short transactions, index the FK columns, SKIP LOCKED for queue tables, right isolation level
Repeated expensive aggregates	Caching / materialized views / summary tables refreshed by events	CREATE MATERIALIZED VIEW zone_daily_stats ...; REFRESH MATERIALIZED VIEW CONCURRENTLY
N+1 from the ORM	Batch fetching	@BatchSize(size = 50), hibernate.default_batch_fetch_size=50, JOIN FETCH, @EntityGraph
Bulk writes from Java	JDBC batching	hibernate.jdbc.batch_size=50, order_inserts=true, sequence-based IDs (IDENTITY disables insert batching), or JdbcTemplate.batchUpdate / COPY
SELECT * and wide rows	Select only what you need; DTO projections	SELECT new com..IncidentRow(i.id, i.status) FROM Incident i ...

Cursor → set-based example (escalate overdue incidents):

```
-- BEFORE (PL/pgSQL loop: 1 UPDATE + 1 INSERT per row, 40k rows → ~90 s)
FOR r IN SELECT id FROM incident WHERE status IN ('NEW', 'ASSIGNED') AND sla_due_at < now() LOOP
    UPDATE incident SET escalated = true WHERE id = r.id;
    INSERT INTO incident_history(incident_id, to_status, remarks, changed_at)
    VALUES (r.id, 'ESCALATED', 'SLA breach', now());
END LOOP;

-- AFTER (set-based with a data-modifying CTE → ~1 s)
WITH esc AS (
    UPDATE incident
    SET escalated = true
    WHERE status IN ('NEW', 'ASSIGNED') AND sla_due_at < now() AND escalated = false
    RETURNING id
)
INSERT INTO incident_history (incident_id, to_status, remarks, changed_at)
SELECT id, 'ESCALATED', 'SLA breach', now() FROM esc;
```

Calling a procedure from Spring with a timeout:

```
@Repository
public interface ReportRepository extends JpaRepository<Incident, Long> {
    @Procedure(procedureName = "generate_zone_report")
    void generateZoneReport(@Param("p_zone") String zone, @Param("p_from") LocalDate from);
}

// or JdbcTemplate/SimpleJdbcCall with a statement timeout so a slow proc can't hold threads forever
jdbcTemplate.setQueryTimeout(10); // seconds
```

Process: reproduce with production-like data volume (a small dev dataset hides the problem) → capture the actual plan → make one change at a time → compare plans and timings → check the write overhead of new indexes (each one slows INSERT/UPDATE) → deploy through Flyway (CREATE INDEX CONCURRENTLY in PG, so the table isn't locked) → monitor.

Likely follow-ups:

- *Why not just add indexes on every column?* Each index slows writes, uses storage and memory, and the planner may still ignore it. Index for real query patterns.
- *Why does a query run fast in SSMS/psql but slow from the app?* Different parameter values or types (parameter sniffing, implicit `nvarchar` conversion from JDBC `sendStringParametersAsUnicode`), different session settings, or the app fetches all rows.
- *When would you move logic out of stored procedures?* When business logic needs versioning, testing, and scaling with the app. Keep set-based, data-heavy operations in SQL.

D24. Design patterns in microservices + code-level patterns

Part A: Microservice architecture patterns

Pattern	Problem it solves	How	Where we use it (Smart City)
API Gateway	Clients shouldn't know about N services; cross-cutting concerns	One entry point: routing, auth, rate limits	Spring Cloud Gateway in front of all services (D14)
Service Discovery	Instances change dynamically	Registry (client-side) or platform DNS (server-side)	Eureka → K8s Services
Circuit Breaker	Cascading failures	Fail fast when a dependency is unhealthy	Resilience4j on user-service and SMS gateway calls (D11)
Bulkhead	One slow dependency uses up every thread	Isolated pools or semaphores per dependency	SMS gateway limited to 20 concurrent calls (D13)
Retry / Timeout	Transient faults	Bounded retries with backoff + jitter	Feign GETs, Kafka retry topics (D12)
Database per Service	Coupling through a shared schema	Each service owns its data, others use APIs/events	PostgreSQL per service, TimescaleDB for sensors
Saga	A business transaction across services with no 2PC	A sequence of local transactions + compensating actions	Incident → work order → crew dispatch
CQRS	Read and write models need different shapes and scaling	Separate command model from read model(s), synced by events	reporting-service builds dashboard read models from <code>incident.events</code>
Event Sourcing	Need a full audit and the ability to rebuild state	Store events as the source of truth; derive state by replaying them	Device state history / incident audit trail (often only partially used)
Transactional Outbox	Dual write: DB + broker not atomic	Write an outbox row in the same transaction; a relay or CDC (Debezium) publishes it	incident-service events (D18)
Strangler Fig	Migrating a monolith safely	Route features one at a time to new services behind a facade/gateway until the monolith is gone	The legacy ICCM monolith's complaint module was moved first
Sidecar	Cross-cutting infrastructure kept out of app code	A helper container in the same pod (Envoy proxy, log shipper, Vault agent)	Istio Envoy for mTLS; Fluent Bit sidecar
BFF (Backend for Frontend)	Web and mobile need different payloads	A dedicated gateway/aggregator per client type	Mobile field-officer BFF returning compact payloads
Externalized Configuration	Config per environment without rebuilding	Config Server / ConfigMaps / Vault	Spring Cloud Config
Health Check / Observability (log aggregation, distributed tracing, metrics)	Operating many services	Actuator, ELK, Prometheus, Zipkin	D16
Anti-Corruption Layer	Legacy or third-party models leaking into your domain	A translation layer/adaptor	Adaptor for different vendors' sensor payload formats

Saga: choreography vs orchestration

CHOREOGRAPHY (events, no central coordinator)

```
incident-svc → IncidentApproved → workorder-svc → WorkOrderCreated → crew-svc → CrewAssigned → incident-svc
                                                    | failure
workorder-svc ← CrewUnavailable → compensate: cancel work order
incident-svc ← WorkOrderCancelled → set incident ON_HOLD
```

+ loose coupling, simple for 2-4 steps - flow is implicit and hard to follow, cyclic deps risk

ORCHESTRATION (central saga orchestrator sends commands)

```
Dispatch Saga Orchestrator
┌ 1. CreateWorkOrder → workorder-svc (reply OK)
├ 2. AssignCrew → crew-svc (reply FAIL)
├ 3. compensate: CancelWorkOrder → workorder-svc
└ 4. MarkIncidentOnHold → incident-svc
```

+ explicit flow, easy to monitor and change, good for many steps - orchestrator is extra; risk of a "god service"
Tools: Camunda/Zeebe, Temporal, Axon, Eventuate Tram, or a hand-rolled state table + Kafka

Saga rules: every step must be **idempotent**, compensations must be **semantic undo** operations (cancel, refund), not DB rollbacks, and a saga gives no isolation, so watch out for dirty reads (use a "PENDING" state and semantic locks).

Outbox relay sketch:

```
@Component
@RequiredArgsConstructor
public class OutboxRelay {
    private final OutboxRepository outbox;
    private final KafkaTemplate<String, String> kafka;

    @Scheduled(fixedDelay = 1000)
    @SchedulerLock(name = "outboxRelay", lockAtMostFor = "PT30S") // ShedLock: one pod at a time
    @Transactional
    public void publish() {
        // repository query: SELECT ... WHERE published_at IS NULL ORDER BY created_at LIMIT 100 FOR UPDATE SKIP LOCKED
        for (OutboxEvent e : outbox.lockNextBatch(100)) {
            kafka.send("incident.events", e.getAggregateId(), e.getPayload()).join(); // wait for ack
            e.markPublished(Instant.now());
        }
    }
}
```

Part B: Code-level (GoF) patterns and where Spring uses them

Pattern	Intent	Where in Spring / our code
Singleton	One shared instance	Spring beans are singleton-scoped by default (one per container, not the classic static GoF singleton). Our stateless <code>@Service</code> s, so don't keep mutable request state in fields
Factory / Factory Method	Create objects without naming concrete classes	<code>BeanFactory</code> / <code>ApplicationContext</code> , <code>FactoryBean</code> , <code>@Bean</code> methods; our <code>NotificationSenderFactory</code> returns SMS/Email/Push senders by channel
Abstract Factory	Families of related objects	<code>ConnectionFactory</code> , <code>EntityManagerFactory</code>
Builder	Build complex objects step by step, immutably	Lombok <code>@Builder</code> , <code>RestClient.builder()</code> , <code>UriComponentsBuilder</code> , <code>Jwts.builder()</code> , <code>HttpSecurity DSL</code> , <code>ResponseEntity.ok()</code>
Strategy	Swap algorithms at runtime	<code>PasswordEncoder</code> , <code>AuthenticationProvider</code> , <code>Resilience4j IntervalFunction</code> ; our SLA calculation per incident category and notification channel strategies
Observer	Notify subscribers about events	<code>ApplicationEventPublisher</code> + <code>@EventListener</code> / <code>@TransactionalEventListener</code> ; Kafka pub/sub at the system level
Template Method	Fixed algorithm skeleton, steps varied by subclasses or callbacks	<code>JdbcTemplate</code> , <code>RestTemplate</code> , <code>KafkaTemplate</code> , <code>TransactionTemplate</code> , <code>OncePerRequestFilter.doFilterInternal</code> , <code>AbstractAuthenticationProcessingFilter</code>
Proxy	Control access by wrapping the target	Spring AOP : JDK dynamic or CGLIB proxies for <code>@Transactional</code> , <code>@Cacheable</code> , <code>@PreAuthorize</code> , <code>@Async</code> , <code>@Retry</code> ; Spring Data repository interfaces; Feign clients; <code>@Lazy</code>
Decorator	Add behaviour without changing the class	<code>Resilience4j Decorators.ofSupplier(...).withCircuitBreaker(...).withRetry(...)</code> , <code>BufferedReader</code> , <code>HttpServletRequestWrapper</code> (e.g. caching the request body)
Adapter	Convert one interface to another	<code>HandlerAdapter</code> in Spring MVC; our vendor-specific sensor payload adapters → a common <code>SensorReading</code>
Chain of Responsibility	Pass a request along handlers	Servlet <code>FilterChain</code> , Spring Security <code>SecurityFilterChain</code> , <code>HandlerInterceptor</code> s, gateway filters; our incident validation-rule chain
Front Controller	One handler for every request	<code>DispatcherServlet</code>
Dependency Injection / IoC	Invert object creation	The core of Spring
State	Behaviour depends on state	Incident lifecycle (<code>IncidentStatus</code> enum with allowed transitions)

Strategy + Factory example (notification channels):

```
public interface NotificationSender {
    Channel channel();
    void send(Notification n);
}

@Component class SmsSender implements NotificationSender {
    public Channel channel() { return Channel.SMS; }
    public void send(Notification n) { /* call SMS gateway */ }
}

@Component class EmailSender implements NotificationSender {
```

```

    public Channel channel() { return Channel.EMAIL; }
    public void send(Notification n) { /* JavaMailSender */ }
}
@Component class PushSender implements NotificationSender {
    public Channel channel() { return Channel.PUSH; }
    public void send(Notification n) { /* FCM */ }
}

@Component
public class NotificationSenderFactory {
    private final Map<Channel, NotificationSender> senders;

    public NotificationSenderFactory(List<NotificationSender> all) { // Spring injects every implementation
        this.senders = all.stream().collect(Collectors.toUnmodifiableMap(NotificationSender::channel, s -> s));
    }

    public NotificationSender get(Channel channel) {
        return Optional.ofNullable(senders.get(channel))
            .orElseThrow(() -> new IllegalArgumentException("No sender for " + channel));
    }
}
// Adding WhatsApp = one new @Component; nothing else changes (Open/Closed Principle)

```

Observer within a service (Spring events):

```

public record IncidentClosedEvent(Long incidentId, String closedBy) {}

@Service
@RequiredArgsConstructor
class IncidentCloser {
    private final ApplicationEventPublisher events;

    @Transactional
    public void close(Long id, String user) {
        // ... update status
        events.publishEvent(new IncidentClosedEvent(id, user));
    }
}

@Component
class IncidentClosedHandlers {
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT) // only fires if the TX committed
    public void updateStats(IncidentClosedEvent e) { /* refresh cached stats */ }
}

```

Likely follow-ups:

- *CQRS vs Event Sourcing?* They're independent. CQRS separates read and write models. Event sourcing stores state as events. They're often combined, but CQRS works fine with a normal DB plus events to build the read model.
- *How does Spring decide between a JDK proxy and CGLIB?* Spring Boot defaults to CGLIB (`proxyTargetClass=true`), which subclasses the class. A JDK proxy only implements interfaces. Either way, `final` methods and self-invocation aren't intercepted.
- *When would you choose orchestration over choreography?* When the flow has more than ~4 steps, needs clear visibility or timeouts, or changes often. Choreography suits simple, stable reactions between a few services.
- *Is a Spring singleton thread-safe?* Only if it's stateless or its shared state is thread-safe. Spring doesn't synchronise anything for you.

E. SQL, Angular/JavaScript, Project & HR

How to use this section: E1–E10 are SQL, E11–E14 are Angular/JavaScript, E15–E24 are project/HR (fill-in-the-blank **templates**: replace every [placeholder] with your real facts, never claim anything you did not do), and E25 covers the truncated Java Streams question.

Sample schema and data used in all SQL answers (E1–E10)

Every query below runs against this data. Some rows are there on purpose to test edge cases: **ties** (three people earn 120000), a **NULL salary** (Neha), an employee with **no department** (Arjun), and a **department with no employees** (Finance).

```
CREATE TABLE department (  
  id INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE employee (  
  id INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  salary DECIMAL(10,2), -- nullable on purpose  
  dept_id INT REFERENCES department(id) -- nullable on purpose  
);  
  
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  first_name VARCHAR(50),  
  last_name VARCHAR(50)  
);  
  
CREATE TABLE transactions (  
  id INT PRIMARY KEY,  
  sender_id INT REFERENCES users(id),  
  receiver_id INT REFERENCES users(id),  
  amount DECIMAL(12,2) NOT NULL  
);
```

department

id	name
1	Engineering
2	Operations
3	Analytics
4	Finance

employee

id	name	salary	dept_id
1	Asha	90000	1
2	Ravi	120000	1
3	Meena	120000	2
4	John	75000	2
5	Priya	95000	1
6	Karan	95000	3
7	Neha	NULL	3
8	Arjun	60000	NULL
9	Vikas	120000	1

Distinct non-NULL salaries, highest first: **120000, 95000, 90000, 75000, 60000**.

users: 30 rows with ids 1–30. The first five are used by the transaction queries:

id	first_name	last_name
1	Asha	Rao
2	Ravi	Kumar
3	Meena	Iyer
4	John	Dsouza
5	Priya	Shah
...	...	...

transactions

id	sender_id	receiver_id	amount
1	1	2	500
2	3	2	700
3	2	4	1500
4	1	4	200
5	4	3	1200
6	5	3	500

Total received: user 2 = 1200, user 3 = 1700, user 4 = 1700 (a **tie**). Users 1 and 5 received nothing.

NULL sorting by dialect (this changes several answers): with `ORDER BY salary DESC`, **MySQL and SQL Server** put NULLs **last**, while **PostgreSQL and Oracle** put NULLs **first**, because they treat NULL as larger than any value. To get the same result everywhere, write `WHERE salary IS NOT NULL`. PostgreSQL and Oracle also accept `NULLS LAST`.

E1. Find the second-highest salary, and the name and salary of the employee(s) who earn it

Expected answer: 95000, earned by **Priya** and **Karan**.

Approach 1: subquery with MAX (works in every dialect)

```
SELECT MAX(salary) AS second_highest
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

second_highest
95000

- The inner query returns 120000. The outer query returns the highest salary below that.
- Ties are handled correctly:** the three 120000 rows are all excluded together.
- NULLs are safe:** `MAX` ignores NULLs, and `NULL < 120000` is UNKNOWN, so that row is filtered out.
- No second-highest** (one distinct salary, or an empty table): returns **one row containing NULL**. That is often what the interviewer wants (it is the LeetCode #176 requirement).
- It only works for N = 2. For N = 3 you would have to nest it again, so it does not generalise.

Approach 2: DISTINCT + LIMIT/OFFSET (or OFFSET...FETCH)

```
-- MySQL / PostgreSQL
SELECT DISTINCT salary
FROM employee
WHERE salary IS NOT NULL           -- required in PostgreSQL, where NULL sorts first in DESC
ORDER BY salary DESC
LIMIT 1 OFFSET 1;                -- MySQL also accepts: LIMIT 1, 1

-- SQL Server 2012+ / Oracle 12c+ / PostgreSQL
SELECT DISTINCT salary
FROM employee
WHERE salary IS NOT NULL
ORDER BY salary DESC
```

```

OFFSET 1 ROWS FETCH NEXT 1 ROWS ONLY;

-- SQL Server, older style with TOP
SELECT MAX(salary) FROM employee
WHERE salary NOT IN (SELECT TOP 1 salary FROM employee ORDER BY salary DESC);

-- Oracle 11g and older, with ROWNUM
SELECT salary FROM (
  SELECT salary, ROWNUM rn FROM (
    SELECT DISTINCT salary FROM employee
    WHERE salary IS NOT NULL ORDER BY salary DESC)
  ) WHERE rn = 2;

```

Common mistake: leaving out DISTINCT.

```

SELECT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET 1;
-- returns 120000 (the second of three 120000 rows), which is WRONG

```

Getting NULL instead of an empty result: when no second salary exists, LIMIT/OFFSET returns **zero rows**. To return NULL, wrap it as a scalar subquery:

```

SELECT (SELECT DISTINCT salary FROM employee
  WHERE salary IS NOT NULL
  ORDER BY salary DESC LIMIT 1 OFFSET 1) AS second_highest;

```

Approach 3: DENSE_RANK (best for "Nth highest" and for getting the names)

```

SELECT name, salary
FROM (
  SELECT name, salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employee
  WHERE salary IS NOT NULL
) ranked
WHERE rnk = 2;

```

name	salary
Priya	95000
Karan	95000

Why DENSE_RANK and not RANK or ROW_NUMBER?

salary	ROW_NUMBER	RANK	DENSE_RANK
120000	1	1	1
120000	2	1	1
120000	3	1	1
95000	4	4	2
95000	5	4	2
90000	6	6	3

- `ROW_NUMBER() = 2` gives a 120000 row, which is wrong.
- `RANK() = 2` gives **no rows**, because RANK jumps from 1 to 4.
- `DENSE_RANK() = 2` gives the two 95000 rows, which is correct.

A CTE version reads more cleanly:

```

WITH ranked AS (
  SELECT e.*, DENSE_RANK() OVER (ORDER BY salary DESC) rnk
  FROM employee e WHERE salary IS NOT NULL
)
SELECT name, salary FROM ranked WHERE rnk = 2;

```

Approach 4: correlated subquery for the Nth highest (no window functions)

Logic: a salary is the Nth highest when exactly N-1 distinct salaries are greater than it.

```
-- N = 2
SELECT DISTINCT e1.salary
FROM employee e1
WHERE e1.salary IS NOT NULL      -- important, see the note below
AND 1 = (SELECT COUNT(DISTINCT e2.salary)
        FROM employee e2
        WHERE e2.salary > e1.salary);
```

- For 95000, the distinct salaries above it are {120000}, so the count is $1 = N - 1$. It matches.
- **NULL trap:** without `e1.salary IS NOT NULL`, the NULL row gives a count of 0 (`x > NULL` is never true), so for **N = 1** it would wrongly match alongside 120000.
- Cost is $O(n^2)$. That is fine on small tables but slow on large ones. Prefer DENSE_RANK.

To get the names of the Nth-highest earners with this method:

```
SELECT name, salary FROM employee
WHERE salary = (SELECT DISTINCT e1.salary FROM employee e1
               WHERE e1.salary IS NOT NULL
               AND 1 = (SELECT COUNT(DISTINCT e2.salary) FROM employee e2
                       WHERE e2.salary > e1.salary));
```

Getting the names with Approach 1

```
SELECT name, salary
FROM employee
WHERE salary = (SELECT MAX(salary) FROM employee
               WHERE salary < (SELECT MAX(salary) FROM employee));
```

Nth highest as a reusable MySQL function (classic follow-up)

```
DELIMITER $$
CREATE FUNCTION getNthHighestSalary(N INT) RETURNS DECIMAL(10,2)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE off INT DEFAULT N - 1;      -- LIMIT does not accept an expression such as N-1
    RETURN (SELECT DISTINCT salary FROM employee
            WHERE salary IS NOT NULL
            ORDER BY salary DESC LIMIT 1 OFFSET off);
END$$
DELIMITER ;
```

Approach summary

Approach	Handles ties	Generalises to N	Result when none exists	Portable
MAX with subquery	Yes	No	NULL	Yes
DISTINCT with LIMIT/OFFSET	Only with DISTINCT	Yes	Empty set	Syntax varies
DENSE_RANK	Yes	Yes	Empty set	All modern databases (MySQL 8+)
Correlated COUNT	Yes	Yes	Empty set	Yes, but $O(n^2)$

Likely follow-ups:

- *What if the table is empty?* MAX returns NULL, while LIMIT and DENSE_RANK return no rows. Wrap the query in a scalar subquery (or use `COALESCE`) if you need exactly one row.
- *Which is fastest on a large table?* With an index on `salary`, the MAX subquery and ORDER BY...LIMIT both use an index scan. DENSE_RANK needs a sort or window pass, so it is a little heavier, but it is the most flexible.
- *Second highest per department?* Add `PARTITION BY dept_id` to the DENSE_RANK.
- *Does MySQL 5.7 support window functions?* No, they arrived in MySQL 8.0. Use Approach 1 or 4 on 5.7.

E2. Find the employee(s) with the highest salary in each department

Expected result (Arjun has no department, so he is excluded once we join to `department`):

department	name	salary
Analytics	Karan	95000

department	name	salary
Engineering	Ravi	120000
Engineering	Vikas	120000
Operations	Meena	120000

Engineering has a **tie**, so a correct answer must return **both** Ravi and Vikas.

Approach 1: window function (preferred)

```
SELECT department, name, salary
FROM (
  SELECT d.name AS department, e.name, e.salary,
         RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rnk
  FROM employee e
  JOIN department d ON d.id = e.dept_id
  WHERE e.salary IS NOT NULL      -- otherwise Neha (NULL) ranks 1st in Analytics on PostgreSQL/Oracle
) t
WHERE rnk = 1
ORDER BY department, name;
```

- `RANK` or `DENSE_RANK` = 1 keeps ties. `ROW_NUMBER` = 1 picks **one arbitrary** row per department. Use it only when the requirement is exactly one row per group, and add a tie-breaker such as `ORDER BY salary DESC, id`.
- PostgreSQL alternative: `ORDER BY e.salary DESC NULLS LAST` instead of the WHERE filter.

Approach 2: join with the grouped MAX (works everywhere, including MySQL 5.7)

```
SELECT d.name AS department, e.name, e.salary
FROM employee e
JOIN (SELECT dept_id, MAX(salary) AS max_sal
      FROM employee
      GROUP BY dept_id) m
ON m.dept_id = e.dept_id
AND m.max_sal = e.salary
JOIN department d ON d.id = e.dept_id
ORDER BY department, e.name;
```

- The derived table is `{1:120000, 2:120000, 3:95000, NULL:60000}`. Joining back on both columns keeps every tied row.
- The `NULL` group (Arjun) is dropped because `NULL = NULL` is not true in a join.

Approach 3: correlated subquery

```
SELECT d.name AS department, e.name, e.salary
FROM employee e
JOIN department d ON d.id = e.dept_id
WHERE e.salary = (SELECT MAX(e2.salary)
                  FROM employee e2
                  WHERE e2.dept_id = e.dept_id);
```

This is easy to read. It runs the subquery once per row, though the optimizer often rewrites it, so an index on `(dept_id, salary)` makes it fast.

Approach 4: NOT EXISTS (no one in the same department earns more)

```
SELECT d.name, e.name, e.salary
FROM employee e
JOIN department d ON d.id = e.dept_id
WHERE e.salary IS NOT NULL      -- without this, Neha is returned (see trap)
  AND NOT EXISTS (SELECT 1 FROM employee e2
                  WHERE e2.dept_id = e.dept_id
                  AND e2.salary > e.salary);
```

Trap: for Neha, `e2.salary > NULL` is never true, so `NOT EXISTS` is TRUE and she would show up as a top earner. That is why the explicit NULL filter is needed.

Wrong answer interviewers like to catch

```
SELECT dept_id, name, MAX(salary) FROM employee GROUP BY dept_id;
```

`name` is neither grouped nor aggregated. PostgreSQL, SQL Server, Oracle and MySQL with `ONLY_FULL_GROUP_BY` (the default since 5.7) **reject** it. Older MySQL returned an **arbitrary** name that was not necessarily the top earner.

Variations

- **Include departments with no employees** (Finance shows up with NULLs): `FROM department d LEFT JOIN (...ranked...) t ON t.dept_id = d.id AND t.rnk = 1`.
- **Top 3 per department:** `WHERE rnk <= 3`. Use `DENSE_RANK` if you mean the top three *salary levels*.
- **Only the max value per department** (no names): `SELECT dept_id, MAX(salary) FROM employee GROUP BY dept_id`.

Likely follow-ups:

- *RANK vs ROW_NUMBER here?* `RANK` keeps ties. `ROW_NUMBER` forces exactly one row per department, which you might want for "pick one", and then you should add a tie-breaker.
- *What index would help?* A composite index on `(dept_id, salary DESC)` serves the grouping, the correlated subquery and the window partition sort.
- *How do you show the department average next to each person?* Use `AVG(salary) OVER (PARTITION BY dept_id)`. See E4.

E3. List the name and salary of employees in each department

Basic INNER JOIN (only employees that have a department)

```
SELECT d.name AS department, e.name AS employee, e.salary
FROM employee e
INNER JOIN department d ON d.id = e.dept_id
ORDER BY d.name, e.salary DESC, e.name;
```

department	employee	salary
Analytics	Karan	95000
Analytics	Neha	NULL
Engineering	Ravi	120000
Engineering	Vikas	120000
Engineering	Priya	95000
Engineering	Asha	90000
Operations	Meena	120000
Operations	John	75000

- Arjun is missing because his `dept_id` is `NULL`. Finance is missing because it has no employees.
- `e.name` is the tie-breaker, so Ravi and Vikas always come out in the same order. Without a full `ORDER BY`, row order is **not guaranteed**.

Include empty departments: LEFT JOIN from department

```
SELECT d.name AS department, e.name AS employee, e.salary
FROM department d
LEFT JOIN employee e ON e.dept_id = d.id
ORDER BY d.name, e.salary DESC;
```

This returns the same rows plus `Finance | NULL | NULL`.

Include employees without a department: LEFT JOIN from employee

```
SELECT COALESCE(d.name, 'Unassigned') AS department, e.name, e.salary
FROM employee e
LEFT JOIN department d ON d.id = e.dept_id;
```

This adds `Unassigned | Arjun | 60000`.

Both directions: FULL OUTER JOIN (MySQL has none, so emulate it with UNION)

```
-- PostgreSQL / SQL Server / Oracle
SELECT d.name, e.name, e.salary
FROM department d FULL OUTER JOIN employee e ON e.dept_id = d.id;

-- MySQL
SELECT d.name, e.name, e.salary FROM department d LEFT JOIN employee e ON e.dept_id = d.id
```

```
UNION
SELECT d.name, e.name, e.salary FROM department d RIGHT JOIN employee e ON e.dept_id = d.id;
```

One row per department with names aggregated

```
-- MySQL
SELECT d.name, GROUP_CONCAT(e.name ORDER BY e.name SEPARATOR ', ') AS employees
FROM department d LEFT JOIN employee e ON e.dept_id = d.id GROUP BY d.name;

-- PostgreSQL / SQL Server 2017+
SELECT d.name, STRING_AGG(e.name, ', ') AS employees ... -- PostgreSQL also allows ORDER BY inside STRING_AGG

-- Oracle
SELECT d.name, LISTAGG(e.name, ', ') WITHIN GROUP (ORDER BY e.name) ...
```

Count per department: the COUNT(*) gotcha

```
SELECT d.name, COUNT(*) AS wrong_cnt, COUNT(e.id) AS right_cnt, AVG(e.salary) AS avg_sal
FROM department d LEFT JOIN employee e ON e.dept_id = d.id
GROUP BY d.name;
```

name	wrong_cnt	right_cnt	avg_sal
Analytics	2	2	95000
Engineering	4	4	106250
Finance	1	0	NULL
Operations	2	2	97500

COUNT(*) counts the NULL-padded row that the LEFT JOIN creates for Finance. COUNT(e.id) counts only actual matches.

Likely follow-ups:

- *Why use a table alias?* To avoid ambiguous column names (name exists in both tables) and to keep the query short.
- *Is a WHERE condition on the right table of a LEFT JOIN safe?* No. WHERE e.salary > 50000 removes the NULL-padded rows, which turns the LEFT JOIN into an INNER JOIN. Put the condition in the ON clause instead.
- *How would you do this in JPA?* @ManyToOne Department dept on Employee, then JPQL SELECT d.name, e.name, e.salary FROM Employee e JOIN e.dept d, or a LEFT JOIN for optional departments.

E4. Window functions

Window Functions: ROW_NUMBER vs RANK vs DENSE_RANK

fn() OVER (ORDER BY salary DESC)

name	dept	salary	ROW_NUMBER	RANK	DENSE_RANK
A	IT	9000	1	1	1
B	IT	8000	2	2	2
C	HR	8000	3	2	2
D	HR	7000	4	4	3

tie

ROW_NUMBER()

always unique: 1, 2, 3, 4
ties ordered arbitrarily

RANK()

ties share a rank,
next rank skips: 1, 2, 2, 4

DENSE_RANK()

ties share a rank,
no gaps: 1, 2, 2, 3

PARTITION BY dept: ranking restarts in each partition

RANK() OVER (PARTITION BY dept ORDER BY salary DESC)

partition: dept = 'IT'

name	salary	rank
A	9000	1
B	8000	2

partition: dept = 'HR'

name	salary	rank
C	8000	1
D	7000	2

C restarts at rank 1 in HR; without PARTITION BY the whole result set is one window.

Figure 22: SQL window ranking functions: ROW_NUMBER, RANK, DENSE_RANK

What they are

A window function computes a value **across a set of rows related to the current row** without collapsing them. GROUP BY returns one row per group; a window function keeps every row and adds a computed column.

```
function_name(args) OVER (  
  [PARTITION BY col, ...]      -- split rows into groups (like GROUP BY, but rows are kept)  
  [ORDER BY col [ASC|DESC], ...] -- order inside each partition  
  [frame_clause]              -- which rows relative to the current row (ROWS/RANGE BETWEEN ...)  
)
```

Categories

Category	Functions
Ranking	ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE(n), PERCENT_RANK(), CUME_DIST()
Offset/value	LAG(col, n, default), LEAD(col, n, default), FIRST_VALUE(), LAST_VALUE(), NTH_VALUE()
Aggregate as window	SUM, AVG, COUNT, MIN, MAX with OVER(...)

Logical execution order: FROM → WHERE → GROUP BY → HAVING → window functions → SELECT → DISTINCT → ORDER BY → LIMIT. This is why **you cannot use a window function in WHERE**. Wrap it in a subquery or CTE and filter outside. (Some databases, such as Snowflake, BigQuery and Teradata, support QUALIFY for this; MySQL, PostgreSQL and SQL Server do not.)

Ranking comparison table

```
SELECT name, salary,  
  ROW_NUMBER() OVER (ORDER BY salary DESC, id) AS row_num,  
  RANK() OVER (ORDER BY salary DESC) AS rnk,  
  DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk,  
  NTILE(4) OVER (ORDER BY salary DESC, id) AS quartile,  
  ROUND(PERCENT_RANK() OVER (ORDER BY salary DESC), 2) AS pct_rank,  
  ROUND(CUME_DIST() OVER (ORDER BY salary DESC), 3) AS cume_dist  
FROM employee  
WHERE salary IS NOT NULL;
```

name	salary	row_num	rnk	dense_rnk	quartile	pct_rank	cume_dist
Ravi	120000	1	1	1	1	0.00	0.375

name	salary	row_num	rnk	dense_rnk	quartile	pct_rank	cume_dist
Meena	120000	2	1	1	1	0.00	0.375
Vikas	120000	3	1	1	2	0.00	0.375
Priya	95000	4	4	2	2	0.43	0.625
Karan	95000	5	4	2	3	0.43	0.625
Asha	90000	6	6	3	3	0.71	0.750
John	75000	7	7	4	4	0.86	0.875
Arjun	60000	8	8	5	4	1.00	1.000

Function	Ties	Gaps after ties	Typical use
ROW_NUMBER	Different numbers (arbitrary unless a tie-breaker is given)	n/a	Deduplication, pagination, "pick one per group"
RANK	Same number	Yes (1,1,1,4)	Competition ranking
DENSE_RANK	Same number	No (1,1,1,2)	Nth highest value
NTILE(n)	May split ties across buckets	n/a	Quartiles, percentiles, load-splitting
PERCENT_RANK	Same value	(rank-1)/(n-1)	Relative position from 0 to 1
CUME_DIST	Same value	rows ≤ current / n	"Top x% of earners"

NTILE: 8 rows into 4 buckets gives 2 per bucket. With 9 rows, the first bucket would get the extra row (3, 2, 2, 2).

PARTITION BY: aggregate without collapsing

```

SELECT name, dept_id, salary,
       AVG(salary) OVER (PARTITION BY dept_id)      AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept_id) AS diff_from_avg,
       COUNT(*)   OVER (PARTITION BY dept_id)      AS dept_headcount
FROM employee
WHERE dept_id IS NOT NULL;
```

Ravi gets dept_avg 106250 and diff 13750; Asha gets 106250 and –16250; Neha gets 95000 and NULL. That is impossible with plain GROUP BY unless you join back to an aggregated subquery.

LAG / LEAD, running total and moving average

```

SELECT id, amount,
       SUM(amount) OVER (ORDER BY id)           AS running_total,
       LAG(amount)  OVER (ORDER BY id)         AS prev_amount,
       LEAD(amount) OVER (ORDER BY id)         AS next_amount,
       amount - LAG(amount, 1, 0) OVER (ORDER BY id) AS change_vs_prev,
       ROUND(AVG(amount) OVER (ORDER BY id
                               ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3
FROM transactions;
```

id	amount	running_total	prev_amount	next_amount	change_vs_prev	moving_avg_3
1	500	500	NULL	700	500	500.00
2	700	1200	500	1500	200	600.00
3	1500	2700	700	200	800	900.00
4	200	2900	1500	1200	–1300	800.00
5	1200	4100	200	500	1000	966.67
6	500	4600	1200	NULL	–700	633.33

Running total **per receiver**: SUM(amount) OVER (PARTITION BY receiver_id ORDER BY id) .

Frames: ROWS vs RANGE (the most commonly misunderstood part)

```

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  -- physical rows
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- all rows with the same ORDER BY value (peers) included
ROWS BETWEEN 2 PRECEDING AND 1 FOLLOWING
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
```

- **Default frame:** with ORDER BY and no frame, it is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Without ORDER BY, the frame is the whole partition.
- **Consequence 1 (ties):** `SUM(salary) OVER (ORDER BY salary)` gives all three 120000 rows the **same** running total, because they are peers. Use `ROWS` (or add a unique tie-breaker) for a true row-by-row running total.
- **Consequence 2 (the LAST_VALUE trap):** `LAST_VALUE(name) OVER (PARTITION BY dept_id ORDER BY salary)` returns the *current* row, not the department's last row, because the default frame ends at the current row. Fix it with `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`, or use `FIRST_VALUE` with the opposite ORDER BY.

```
SELECT name, dept_id, salary,
       FIRST_VALUE(name) OVER w AS lowest_paid,
       LAST_VALUE(name) OVER (w ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS highest_paid
FROM employee
WHERE dept_id IS NOT NULL AND salary IS NOT NULL
WINDOW w AS (PARTITION BY dept_id ORDER BY salary); -- named window: MySQL 8, PostgreSQL
```

Practical uses

- **Deduplicate** (keep the latest reading per IoT sensor):

```
DELETE FROM sensor_reading WHERE id IN (
  SELECT id FROM (SELECT id, ROW_NUMBER() OVER (PARTITION BY sensor_id, reading_ts ORDER BY id DESC) rn
    FROM sensor_reading) x WHERE rn > 1);
```

- **Latest status per incident:** `ROW_NUMBER() OVER (PARTITION BY incident_id ORDER BY updated_at DESC) = 1`.
- **Gap detection** (sensor went silent): `reading_ts - LAG(reading_ts) OVER (PARTITION BY sensor_id ORDER BY reading_ts) > INTERVAL '5' MINUTE`.
- **Top N per group, month-over-month change, percent of total:** `amount * 100.0 / SUM(amount) OVER ()`.

Likely follow-ups:

- *Window function vs GROUP BY?* GROUP BY collapses rows into one per group. A window function keeps all rows and adds the aggregate alongside each one.
- *Can you filter on a window result in WHERE?* No, because windows are evaluated after WHERE. Use a subquery or CTE (or `QUALIFY` where supported).
- *Performance?* Each distinct PARTITION BY/ORDER BY combination needs a sort. An index matching `(partition cols, order cols)` can avoid it, and you should reuse the same window (a named `WINDOW`) where possible.
- *Which versions support them?* MySQL 8.0+, PostgreSQL 8.4+, SQL Server 2005+ (frames since 2012), and Oracle 8i+.

E5. ACID properties with a bank-transfer example, plus isolation levels and anomalies

Scenario: transfer ₹500 from account A (balance 1000) to account B (balance 200).

```
CREATE TABLE account (id INT PRIMARY KEY, holder VARCHAR(50),
                      balance DECIMAL(12,2) NOT NULL CHECK (balance >= 0));

START TRANSACTION; -- BEGIN in PostgreSQL / SQL Server
UPDATE account SET balance = balance - 500 WHERE id = 1; -- A
UPDATE account SET balance = balance + 500 WHERE id = 2; -- B
INSERT INTO transactions(sender_id, receiver_id, amount) VALUES (1, 2, 500);
COMMIT; -- or ROLLBACK on any error
```

Property	Meaning	In the transfer
Atomicity	All statements succeed, or none take effect	If the credit to B fails (for example the server crashes after the debit), the debit from A is rolled back . Money is never lost halfway. Implemented with the undo log / rollback segments.
Consistency	A transaction moves the database from one valid state to another, and all constraints hold	<code>CHECK (balance >= 0)</code> , foreign keys and the business rule "total money A+B stays 1200" hold before and after. If A had only 300, the CHECK fails and the transaction rolls back.
Isolation	Concurrent transactions do not see each other's intermediate state (to a degree set by the isolation level)	Another transaction reading A and B mid-transfer never sees A=500 and B=200 (₹500 "missing"). Implemented with locks and/or MVCC.
Durability	Once committed, the change survives crashes	After COMMIT returns, A=500 and B=700 survive a power failure. Implemented with the write-ahead / redo log flushed to disk.

In Spring Boot:

```
@Service
@RequiredArgsConstructor
public class TransferService {
    private final AccountRepository accounts;

    @Transactional(isolation = Isolation.READ_COMMITTED, rollbackFor = Exception.class)
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        // Lock rows in a consistent order (lower id first) to avoid deadlocks
        Account from = accounts.findByIdForUpdate(fromId); // SELECT ... FOR UPDATE
        Account to = accounts.findByIdForUpdate(toId);
        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException(fromId); // RuntimeException, so the transaction rolls back
        }
        from.debit(amount);
        to.credit(amount);
        // JPA dirty checking flushes both UPDATES at commit, atomically
    }
}

public interface AccountRepository extends JpaRepository<Account, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.id = :id")
    Account findByIdForUpdate(@Param("id") Long id);
}
```

Gotchas worth mentioning: by default Spring rolls back only on **unchecked** exceptions (use `rollbackFor`). `@Transactional` also does nothing on **self-invocation** (a call from inside the same class bypasses the proxy) or on private methods.

Read anomalies

Anomaly	What happens	Bank example
Dirty read	T2 reads data T1 has written but not committed	T1 debits A to 500 and has not committed. T2 reads 500 and approves a loan. T1 rolls back, so T2 acted on a value that never existed.
Non-repeatable read	T1 reads the same row twice and gets different values , because T2 updated and committed in between	A statement job reads A=1000, a transfer commits, and the job re-reads A=500 within the same transaction.
Phantom read	T1 runs the same range query twice and gets different rows , because T2 inserted or deleted matching rows	<code>SELECT COUNT(*) FROM transactions WHERE amount > 1000</code> returns 2, then T2 inserts a new large transfer, and it returns 3.
Lost update (bonus)	Two transactions read, modify and write the same row; one overwrites the other	Two deposits each read 1000, add 100 and write 1100. The result should be 1200. Prevent it with <code>SELECT ... FOR UPDATE</code> , an atomic <code>SET balance = balance + x</code> , or optimistic <code>@Version</code> .

Isolation levels (SQL standard)

Level	Dirty read	Non-repeatable read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible in the standard; mostly prevented in InnoDB and PostgreSQL
SERIALIZABLE	Prevented	Prevented	Prevented

Database defaults and implementation notes

Database	Default	Notes
MySQL InnoDB	REPEATABLE READ	MVCC snapshot reads plus next-key/gap locks for locking reads, which prevents most phantoms
PostgreSQL	READ COMMITTED	Its REPEATABLE READ is snapshot isolation (no phantoms). SERIALIZABLE is SSI and can abort transactions with serialization failures, so retry them. READ UNCOMMITTED behaves like READ COMMITTED.
SQL Server	READ COMMITTED (lock-based)	<code>READ_COMMITTED_SNAPSHOT</code> switches it to MVCC. There is also a <code>SNAPSHOT</code> level.
Oracle	READ COMMITTED	Supports only READ COMMITTED, SERIALIZABLE and READ ONLY

Higher isolation means fewer anomalies but more locking or aborts, so less concurrency. Most applications stay at READ COMMITTED and protect critical rows with explicit locking (`FOR UPDATE`) or optimistic locking (`@Version`):

```
@Entity
public class Account {
    @Id Long id;
    BigDecimal balance;
    @Version Long version;    // UPDATE ... WHERE id=? AND version=? → OptimisticLockException on a conflict
}
```

Likely follow-ups:

- *Optimistic vs pessimistic locking?* Optimistic uses a version column, detects conflicts at write time and retries; it suits low contention. Pessimistic uses `SELECT FOR UPDATE` and blocks other writers; it suits high-contention money movement.
- *How do you keep ACID across microservices?* You can't with a single database transaction. Use the Saga pattern (choreography or orchestration with compensating actions), the transactional outbox, and idempotent consumers. Avoid 2PC/XA in microservices.
- *What does `propagation = REQUIRES_NEW` do?* It suspends the current transaction and starts an independent one, for example for an audit log that must be saved even if the outer transaction rolls back.
- *How do deadlocks happen, and how do you avoid them?* Two transactions lock rows in opposite order. Lock in a consistent order, keep transactions short, and retry on deadlock errors.

E6. Stored procedures: what, why, the hierarchy of database objects, parameters, calling from Spring, pros and cons

What

A **stored procedure** is a named, precompiled block of SQL and procedural logic (variables, IF, loops, cursors, error handlers) stored **in the database** and executed with `CALL` (MySQL/PostgreSQL) or `EXEC` (SQL Server). It can take IN, OUT and INOUT parameters, run DML and DDL, manage transactions, and return result sets.

Why use them

- **Fewer network round-trips:** one call runs many statements, which helps with bulk or batch operations such as nightly aggregation of IoT readings.
- **Logic close to the data**, which suits heavy set-based processing.
- **Security:** grant `EXECUTE` on the procedure without granting table access.
- **Reuse** by several applications or report tools.
- **Plan caching** (in SQL Server and Oracle especially).

Hierarchy and comparison of database objects

Object	Returns	Usable in SELECT?	DML allowed?	Transaction control	Invoked by
View	Virtual table (a stored query)	Yes, like a table	Simple views are updatable	No	Querying it
Function (UDF)	A single value (scalar) or a table (table-valued)	Yes (<code>SELECT fn(x)</code>)	Generally no side effects (MySQL allows some; SQL Server forbids them)	No	Expressions
Stored procedure	0..n result sets and OUT params	No	Yes	Yes (COMMIT/ROLLBACK inside, in most databases)	<code>CALL / EXEC</code> from an app or another procedure
Trigger	Nothing	No	Yes	Runs inside the triggering transaction	Automatically on INSERT/UPDATE/DELETE (BEFORE/AFTER, row or statement level)
Package (Oracle)	Container	n/a	n/a	n/a	Groups related procedures and functions with shared state

Nesting (the "hierarchy" in practice):

- A procedure can call other procedures and functions. MySQL allows recursive calls only if `max_sp_recursion_depth > 0`; SQL Server limits nesting to **32 levels** (`@@NESTLEVEL`).
- A function can call other functions, but generally **cannot call a procedure** that modifies data (rules vary by database).
- A trigger can call procedures or functions and can fire other triggers (cascading triggers). This is a common source of hidden behaviour and debugging pain.

- A view can use functions.

MySQL example: fund transfer with IN/OUT parameters and error handling

```
DELIMITER $$

CREATE PROCEDURE transfer_funds(
  IN p_from INT,
  IN p_to INT,
  IN p_amount DECIMAL(12,2),
  OUT p_status VARCHAR(30),
  OUT p_new_bal DECIMAL(12,2)
)
BEGIN
  DECLARE v_balance DECIMAL(12,2);

  -- Any SQL error: roll back and report it
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SET p_status = 'ERROR';
  END;

  START TRANSACTION;

  SELECT balance INTO v_balance FROM account WHERE id = p_from FOR UPDATE;

  IF v_balance IS NULL THEN
    SET p_status = 'ACCOUNT_NOT_FOUND';
    ROLLBACK;
  ELSEIF v_balance < p_amount THEN
    SET p_status = 'INSUFFICIENT_FUNDS';
    ROLLBACK;
  ELSE
    UPDATE account SET balance = balance - p_amount WHERE id = p_from;
    UPDATE account SET balance = balance + p_amount WHERE id = p_to;
    INSERT INTO transactions(sender_id, receiver_id, amount) VALUES (p_from, p_to, p_amount);
    COMMIT;
    SET p_status = 'SUCCESS';
    SET p_new_bal = v_balance - p_amount;
  END IF;
END$$

DELIMITER ;

-- Call it
CALL transfer_funds(1, 2, 500, @status, @bal);
SELECT @status, @bal;
```

`DELIMITER` is a MySQL **client** command. It changes the statement terminator so the `;` characters inside the body do not end the `CREATE` statement.

A function and a trigger, for comparison:

```
DELIMITER $$

CREATE FUNCTION total_received(p_user INT) RETURNS DECIMAL(12,2)
DETERMINISTIC READS SQL DATA
BEGIN
  RETURN (SELECT COALESCE(SUM(amount), 0) FROM transactions WHERE receiver_id = p_user);
END$$

CREATE TRIGGER trg_audit_salary
AFTER UPDATE ON employee FOR EACH ROW
BEGIN
  IF NOT (OLD.salary <=> NEW.salary) THEN -- <=> is MySQL's NULL-safe equality
    INSERT INTO salary_audit(emp_id, old_salary, new_salary, changed_at)
    VALUES (OLD.id, OLD.salary, NEW.salary, NOW());
  END IF;
END$$

DELIMITER ;

SELECT id, first_name, total_received(id) FROM users; -- a function works inside SELECT
```

Equivalent syntax in other dialects (briefly): PostgreSQL uses `CREATE PROCEDURE ... LANGUAGE plpgsql AS $$ ... $$` (procedures since v11; earlier versions had functions only) and `CALL`. SQL Server uses `CREATE PROCEDURE dbo.transfer_funds @from INT, @status VARCHAR(30) OUTPUT AS BEGIN ... END` and `EXEC`. Oracle uses `CREATE OR REPLACE PROCEDURE ... (p_from IN NUMBER, p_status OUT VARCHAR2) IS BEGIN ... END;`

Calling from Spring

Option 1: Spring Data JPA @Procedure

```
@Entity
@NamedStoredProcedureQuery(
    name = "Account.transferFunds",
    procedureName = "transfer_funds",
    parameters = {
        @StoredProcedureParameter(mode = ParameterMode.IN, name = "p_from", type = Integer.class),
        @StoredProcedureParameter(mode = ParameterMode.IN, name = "p_to", type = Integer.class),
        @StoredProcedureParameter(mode = ParameterMode.IN, name = "p_amount", type = BigDecimal.class),
        @StoredProcedureParameter(mode = ParameterMode.OUT, name = "p_status", type = String.class),
        @StoredProcedureParameter(mode = ParameterMode.OUT, name = "p_new_bal", type = BigDecimal.class)
    }
})
public class Account { /* ... */ }

public interface AccountRepository extends JpaRepository<Account, Long> {

    // Multiple OUT params are returned as Map<String, Object>
    @Procedure(name = "Account.transferFunds")
    Map<String, Object> transferFunds(@Param("p_from") Integer from,
                                     @Param("p_to") Integer to,
                                     @Param("p_amount") BigDecimal amount);

    // Simple procedure with a single OUT param, without a named query
    @Procedure(procedureName = "count_active_sensors")
    Integer countActiveSensors(Integer zoneId);
}
```

Option 2: SimpleJdbcCall (JDBC, no entity needed, flexible)

```
@Repository
public class TransferDao {
    private final SimpleJdbcCall transferCall;

    public TransferDao(DataSource ds) {
        this.transferCall = new SimpleJdbcCall(ds).withProcedureName("transfer_funds");
        // Metadata lookup discovers the params. Or declare them explicitly:
        // .withoutProcedureColumnMetaDataAccess().declareParameters(new SqlParameter("p_from", Types.INTEGER), ...)
    }

    public TransferResult transfer(int from, int to, BigDecimal amount) {
        Map<String, Object> out = transferCall.execute(new MapSqlParameterSource()
            .addValue("p_from", from)
            .addValue("p_to", to)
            .addValue("p_amount", amount));
        return new TransferResult((String) out.get("p_status"), (BigDecimal) out.get("p_new_bal"));
    }
}
```

For functions use `new SimpleJdbcCall(ds).withFunctionName("total_received").executeFunction(BigDecimal.class, params)`. Other options are `EntityManager.createStoredProcedureQuery(...)` and plain `JdbcTemplate.call(...)`.

Pros and cons

Pros	Cons
Fewer round-trips, so faster bulk operations	Business logic split between Java and the database is harder to find, review and debug
Fine-grained security (EXECUTE only)	Vendor lock-in: PL/SQL, T-SQL and MySQL procedural SQL are not portable
Reuse across applications and BI tools	Harder to unit test, version (needs Flyway/Liquibase) and run in CI
Plan caching	Scales with the database, which is usually the hardest tier to scale out
Good for heavy set-based ETL and reporting	Hidden side effects (especially triggers), and weak IDE/refactoring support

Typical stance in a Spring Boot microservice: keep business logic in the service layer, and use procedures selectively for heavy batch/report jobs or legacy integration. Keep them in version control and deploy them with Flyway or Liquibase migrations.

Likely follow-ups:

- Procedure vs function in one line?* A function returns a value and can be used in SQL expressions. A procedure is CALLED, can have several OUT params or result sets, and can control transactions.
- Can a procedure return a result set?* Yes. A plain `SELECT` inside a MySQL procedure returns one; in Spring, read it with `SimpleJdbcCall.returningResultSet(...)`.

- *How do you prevent SQL injection in a procedure?* Use parameters. Avoid dynamic SQL built by string concatenation; if dynamic SQL is unavoidable, use `PREPARE ... USING` or `sp_executesql` with parameters.
- *Why are triggers risky?* They are invisible to application developers, cascade, slow down writes and complicate bulk loads. Use them sparingly, for example for auditing.

E7. Tables A and B have the same records. What does an INNER JOIN return?

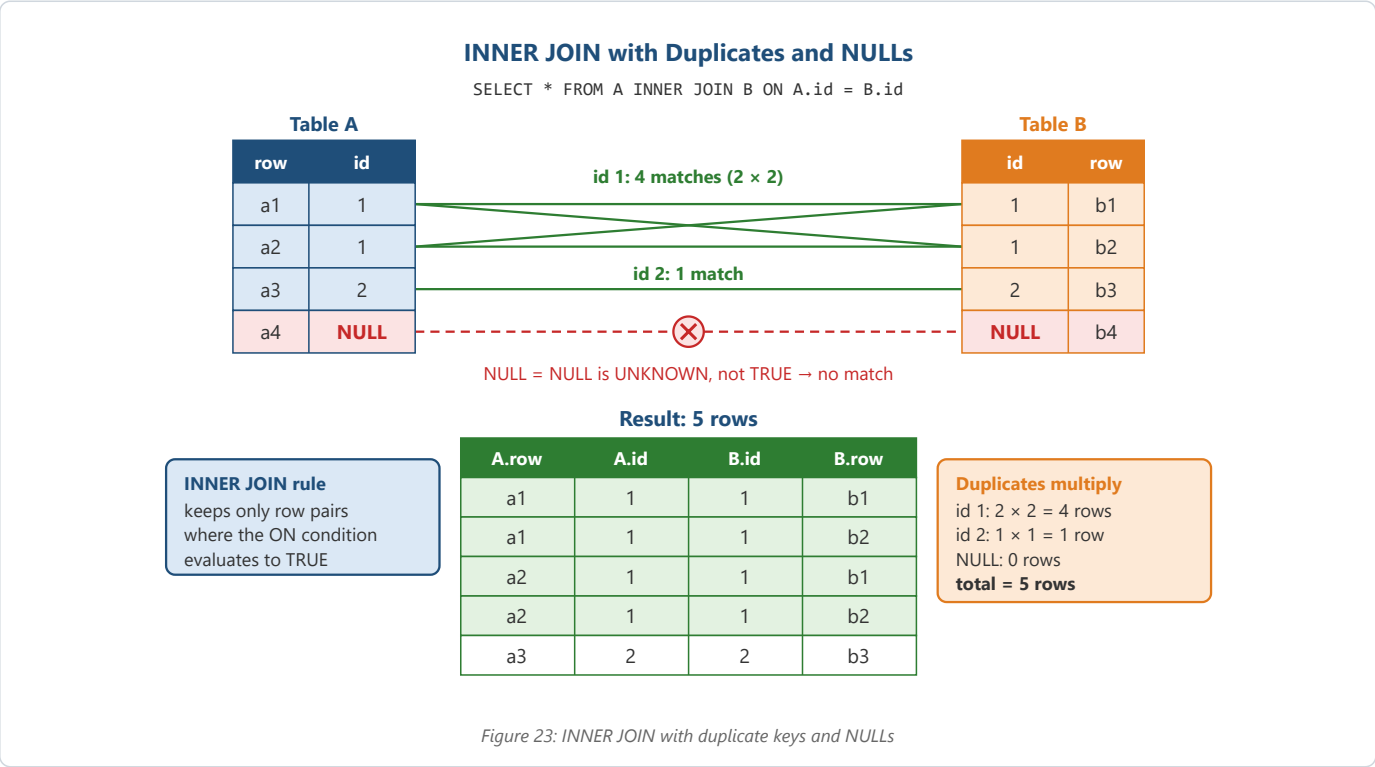


Figure 23: INNER JOIN with duplicate keys and NULLs

Short answer: an INNER JOIN returns **every pair of rows** (a from A, b from B) for which the join condition is TRUE. If both tables have the same **unique** values, you get **one row per value**, the same number of rows as each table. **Duplicates multiply**, and **NULLs never match**.

Case 1: same records, unique values

A.id: 1, 2, 3
B.id: 1, 2, 3

`SELECT A.id AS a_id, B.id AS b_id FROM A INNER JOIN B ON A.id = B.id;`

a_id	b_id
1	1
2	2
3	3

That is 3 rows. LEFT, RIGHT and FULL joins would also return exactly 3 rows here, because every row has a match.

Case 2: same records, with duplicates (the interview trick)

A.id: 1, 1, 2
B.id: 1, 1, 2

value	count in A	count in B	rows produced
1	2	2	2 × 2 = 4
2	1	1	1 × 1 = 1

Total: **5 rows**, not 3. Each A row with value 1 matches *both* B rows with value 1.

a_id	b_id
1	1
1	1
1	1
1	1
2	2

Rule: for each join key, output rows = (count in A) × (count in B).

Case 3: NULLs in both tables

A.id: 1, 2, NULL B.id: 1, 2, NULL

INNER JOIN returns **2 rows** (1 and 2). `NULL = NULL` evaluates to **UNKNOWN**, not TRUE, so the NULL rows never pair.

- To match NULLs on purpose: `ON A.id = B.id OR (A.id IS NULL AND B.id IS NULL)`, MySQL's `ON A.id <=> B.id`, or the standard `IS NOT DISTINCT FROM` (PostgreSQL, SQL Server 2022+).
- LEFT JOIN would return 3 rows, with `NULL | NULL` for A's NULL row (unmatched, so B's columns are NULL-padded). FULL JOIN would return 4 rows: the two matches, plus A's NULL and B's NULL as separate unmatched rows.

Mixed worked example

A.id: 1, 2, 2, 3, NULL B.id: 2, 2, 3, 4, NULL

key	A count	B count	INNER rows
1	1	0	0
2	2	2	4
3	1	1	1
4	0	1	0
NULL	1	1	0

INNER = **5**. LEFT = 5 + 2 unmatched A rows (1 and NULL) = **7**. RIGHT = 5 + 2 unmatched B rows (4 and NULL) = **7**. FULL = 5 + 2 + 2 = **9**.

Same records, but no join condition (or ON 1=1)

That is a **CROSS JOIN** (Cartesian product): $3 \times 3 = 9$ rows for Case 1.

Related set operations interviewers sometimes want instead

- `SELECT * FROM A INTERSECT SELECT * FROM B` returns the common rows with **duplicates removed** (Case 2 gives {1, 2}). In set operations, NULLs are treated as equal.
- `A EXCEPT B` (`MINUS` in Oracle) returns rows in A but not in B. For tables with identical records it returns nothing.
- `UNION` removes duplicates; `UNION ALL` keeps them.

Likely follow-ups:

- How do you avoid row multiplication? Join on a unique key, deduplicate first (`SELECT DISTINCT` or `ROW_NUMBER` in a CTE), or use `EXISTS` when you only need to filter.
- How do you check that two tables are identical? Run `(A EXCEPT B) UNION ALL (B EXCEPT A)` and confirm it is empty, and also compare `COUNT(*)` (and per-key counts) to catch differences in duplicates.
- Is there any difference between `JOIN` and `INNER JOIN`? No. `INNER` is optional.

E8. Fetch 20 records starting from the 5th row of users (first and last name), across dialects

"Starting from the 5th row" means **skip 4 rows and take 20**, which gives rows 5–24 in the chosen order (ids 5–24 if ordered by id).

Why ORDER BY is mandatory

Tables are **unordered sets**. Without ORDER BY, the database may return rows in any order (insertion order, index order, or whatever a parallel scan produces), and the "5th row" can change between executions. That causes duplicated or skipped rows across pages. Always order by a **unique**

column, or add a unique tie-breaker such as `ORDER BY last_name, id`.

MySQL

```
SELECT first_name, last_name
FROM users
ORDER BY id
LIMIT 20 OFFSET 4;

-- equivalent shorthand: LIMIT offset, count
SELECT first_name, last_name FROM users ORDER BY id LIMIT 4, 20;
```

PostgreSQL

```
SELECT first_name, last_name FROM users ORDER BY id LIMIT 20 OFFSET 4;
-- the SQL-standard form also works
SELECT first_name, last_name FROM users ORDER BY id OFFSET 4 ROWS FETCH NEXT 20 ROWS ONLY;
```

SQL Server 2012+

```
SELECT first_name, last_name
FROM users
ORDER BY id -- ORDER BY is REQUIRED for OFFSET/FETCH
OFFSET 4 ROWS FETCH NEXT 20 ROWS ONLY;
```

SQL Server 2008 and older (no OFFSET) use ROW_NUMBER:

```
SELECT first_name, last_name FROM (
    SELECT first_name, last_name, ROW_NUMBER() OVER (ORDER BY id) AS rn FROM users
) t
WHERE rn BETWEEN 5 AND 24;
```

Or TOP with NOT IN (works, but clumsy):

```
SELECT TOP 20 first_name, last_name FROM users
WHERE id NOT IN (SELECT TOP 4 id FROM users ORDER BY id)
ORDER BY id;
```

Oracle 12c+

```
SELECT first_name, last_name FROM users
ORDER BY id
OFFSET 4 ROWS FETCH NEXT 20 ROWS ONLY;
```

Oracle 11g and older use ROWNUM. ROWNUM is assigned **before** ORDER BY, so you need nested queries:

```
SELECT first_name, last_name FROM (
    SELECT a.*, ROWNUM rn FROM (
        SELECT first_name, last_name FROM users ORDER BY id
    ) a
    WHERE ROWNUM <= 24 -- upper bound in the middle layer (stops early)
)
WHERE rn >= 5; -- lower bound outside
```

`WHERE ROWNUM >= 5` on its own returns **nothing**. The first candidate row gets ROWNUM 1, fails the test and is discarded, so the next candidate also gets ROWNUM 1, and so on.

Dialect summary

Database	Syntax
MySQL	<code>LIMIT 20 OFFSET 4</code> or <code>LIMIT 4, 20</code>
PostgreSQL	<code>LIMIT 20 OFFSET 4</code> or <code>OFFSET 4 ROWS FETCH NEXT 20 ROWS ONLY</code>
SQL Server 2012+	<code>ORDER BY ... OFFSET 4 ROWS FETCH NEXT 20 ROWS ONLY</code>
SQL Server (old)	<code>TOP / ROW_NUMBER() BETWEEN 5 AND 24</code>
Oracle 12c+	<code>OFFSET 4 ROWS FETCH NEXT 20 ROWS ONLY</code>
Oracle (old)	Nested ROWNUM

General formula: page p (1-based) of size s uses OFFSET $(p-1) \cdot s$. "From row r " uses OFFSET $r-1$.

Performance: why OFFSET gets slow, and keyset (seek) pagination

OFFSET 1000000 still makes the database **read and discard** a million rows, so cost grows with page depth. Rows inserted or deleted between page loads also cause duplicates or skips.

Keyset pagination remembers the last key seen and seeks to it through the index:

```
-- first page
SELECT id, first_name, last_name FROM users ORDER BY id LIMIT 20;

-- next page: the client sends the last id it received (say 24)
SELECT id, first_name, last_name FROM users
WHERE id > 24
ORDER BY id
LIMIT 20;

-- multi-column sort (last_name, id): row-value comparison (MySQL, PostgreSQL)
SELECT id, first_name, last_name FROM users
WHERE (last_name, id) > (:lastName, :lastId)
ORDER BY last_name, id
LIMIT 20;
-- SQL Server / Oracle: WHERE last_name > :ln OR (last_name = :ln AND id > :lastId)
```

	OFFSET	Keyset
Deep page cost	$O(\text{offset} + n)$	$O(\log N + n)$ with an index
Jump to page 57	Easy	Not directly (next/previous only)
Stable under inserts	No	Yes
Typical use	Admin tables with page numbers	Infinite scroll, live alert feeds, APIs, exports

In Spring Data:

```
// Offset-based: page 0 = rows 1-20
Page<User> page = userRepo.findAll(PageRequest.of(0, 20, Sort.by("id")));

// "Start at row 5" is not page-aligned, so use a native query with LIMIT/OFFSET or OffsetScrollPosition

// Keyset: simple derived query
List<User> findTop20ByIdGreaterThanOrderByAsc(Long lastId);

// Keyset: Spring Data 3.1+ scrolling API
Window<User> findFirst20ByOrderByAsc(ScrollPosition position); // call with ScrollPosition.keyset()
```

Page triggers an extra COUNT(*) query. Use Slice when you do not need the total.

Likely follow-ups:

- *Why not use SELECT *?* It fetches unneeded columns, defeats covering indexes, and breaks when the schema changes. Select only first_name and last_name.
- *How does Angular's paginator map to this?* It sends page and size to Spring's Pageable, and the response contains content and totalElements.
- *Is ORDER BY by itself enough?* Only if the ordering is unique. Otherwise add a unique tie-breaker such as id.

E9. INNER JOIN between two tables, compared with the other join types

Using employee and department from the sample data:

```
SELECT e.name, d.name AS dept
FROM employee e
INNER JOIN department d ON e.dept_id = d.id;
```

This returns 8 rows: every employee whose dept_id matches a department. **Arjun** (NULL dept_id) and **Finance** (no employees) are excluded.

Older comma syntax (works, but avoid it, because forgetting the WHERE clause produces a Cartesian product):

```
SELECT e.name, d.name FROM employee e, department d WHERE e.dept_id = d.id;
```

USING / NATURAL: `JOIN ... USING (col)` needs the same column name in both tables. `NATURAL JOIN` joins on all same-named columns, which is dangerous here because both tables have `id` and `name`.

Join types compared (small example)

L (employee subset)		R (department subset)	
id	name dept_id	id	name
1	Asha 1	1	Engineering
7	Neha 3	2	Operations
8	Arjun NULL	3	Analytics

Join	Rule	Result (L.name / R.name)	Rows
INNER	Matches only	Asha/Engineering, Neha/Analytics	2
LEFT	All of L, plus matches or NULL	Asha/Engineering, Neha/Analytics, Arjun/NULL	3
RIGHT	All of R, plus matches or NULL	Asha/Engineering, NULL/Operations, Neha/Analytics	3
FULL OUTER	All of both	Asha/Engineering, Neha/Analytics, Arjun/NULL, NULL/Operations	4
CROSS	Every combination	3 × 3	9
SELF	A table joined to itself	see below	—

```
-- LEFT anti-join: employees without a department
SELECT e.name FROM employee e LEFT JOIN department d ON d.id = e.dept_id WHERE d.id IS NULL; -- Arjun

-- Departments without employees (often better written as NOT EXISTS)
SELECT d.name FROM department d
WHERE NOT EXISTS (SELECT 1 FROM employee e WHERE e.dept_id = d.id); -- Finance

-- CROSS JOIN: e.g. every sensor × every hour slot for a report grid
SELECT s.sensor_id, h.slot FROM sensor s CROSS JOIN hour_slot h;

-- SELF JOIN: needs a manager_id column, e.g. employee(id, name, manager_id)
SELECT e.name AS employee, m.name AS manager
FROM employee e LEFT JOIN employee m ON m.id = e.manager_id;

-- SELF JOIN: pairs in the same department (e1.id < e2.id avoids mirror duplicates and self-pairs)
SELECT e1.name, e2.name, e1.dept_id
FROM employee e1 JOIN employee e2 ON e1.dept_id = e2.dept_id AND e1.id < e2.id;
```

- `RIGHT JOIN` is just `LEFT JOIN` with the tables swapped. Most teams standardise on `LEFT` for readability.
- MySQL has **no FULL OUTER JOIN**. Emulate it with `LEFT JOIN ... UNION ... RIGHT JOIN` (see E3).

How databases execute joins (bonus depth)

Algorithm	Best when
Nested loop (with index lookup)	Small outer input, indexed inner join column
Hash join	Large unsorted inputs with equality conditions (PostgreSQL, SQL Server, Oracle; MySQL 8.0.18+)
Merge join	Both inputs already sorted on the join key

Index the **foreign-key column** (`employee.dept_id`). MySQL InnoDB creates that index automatically for FK constraints; PostgreSQL does **not**.

Likely follow-ups:

- *ON vs WHERE in a LEFT JOIN?* A condition in `ON` filters the right table *before* matching, so unmatched left rows are kept. A condition in `WHERE` filters *after* the join and can drop the NULL-padded rows.
- *Join vs subquery: which is faster?* Modern optimizers often produce the same plan. Use `EXISTS` for semi-joins and `JOIN` when you need columns from both tables, and check with `EXPLAIN`.
- *What is the JPA N+1 problem?* A lazy association is loaded once per row. Fix it with `JOIN FETCH`, `@EntityGraph`, or batch fetching.

E10. Highest receiver amount (from the transactions table)

The question is ambiguous, so **clarify it first**: "Do you mean the user who received the highest total, or the single largest transaction?" Interviewers like to see that. Both answers are below.

Interpretation A: the receiver with the highest total received

Totals: user 2 = 1200, user 3 = **1700**, user 4 = **1700**. There is a tie.

Naive LIMIT 1 (misses ties):

```
SELECT receiver_id, SUM(amount) AS total_received
FROM transactions
GROUP BY receiver_id
ORDER BY total_received DESC
LIMIT 1;           -- returns only 3 OR 4, chosen arbitrarily
```

SQL Server handles ties with `SELECT TOP 1 WITH TIES receiver_id, SUM(amount) ... ORDER BY SUM(amount) DESC`. Oracle 12c+ and PostgreSQL 13+ use `FETCH FIRST 1 ROWS WITH TIES`.

Tie-safe with RANK, including the name:

```
WITH totals AS (
  SELECT receiver_id, SUM(amount) AS total_received
  FROM transactions
  GROUP BY receiver_id
),
ranked AS (
  SELECT t.*, RANK() OVER (ORDER BY total_received DESC) AS rnk
  FROM totals t
)
SELECT u.id, u.first_name, u.last_name, r.total_received
FROM ranked r
JOIN users u ON u.id = r.receiver_id
WHERE r.rnk = 1;
```

id	first_name	last_name	total_received
3	Meena	Iyer	1700
4	John	Dsouza	1700

Tie-safe without window functions (HAVING = MAX of the sums):

```
SELECT u.first_name, u.last_name, SUM(t.amount) AS total_received
FROM transactions t
JOIN users u ON u.id = t.receiver_id
GROUP BY u.id, u.first_name, u.last_name
HAVING SUM(t.amount) = (
  SELECT MAX(s) FROM (SELECT SUM(amount) AS s FROM transactions GROUP BY receiver_id) x
);
```

The derived table must have an alias (`x`) in MySQL, PostgreSQL and SQL Server.

Interpretation B: the single highest transaction and its receiver

```
SELECT t.id, u.first_name, u.last_name, t.amount
FROM transactions t
JOIN users u ON u.id = t.receiver_id
WHERE t.amount = (SELECT MAX(amount) FROM transactions);
```

id	first_name	last_name	amount
3	John	Dsouza	1500

`WHERE amount = (SELECT MAX(...))` is automatically tie-safe.

Variations interviewers chain on

```
-- Net balance per user (received - sent), including users with no transactions
SELECT u.id, u.first_name,
       COALESCE(r.recv, 0) - COALESCE(s.sent, 0) AS net
FROM users u
LEFT JOIN (SELECT receiver_id, SUM(amount) recv FROM transactions GROUP BY receiver_id) r ON r.receiver_id = u.id
LEFT JOIN (SELECT sender_id, SUM(amount) sent FROM transactions GROUP BY sender_id) s ON s.sender_id = u.id;
-- Aggregate in subqueries BEFORE joining. Joining transactions twice and then summing multiplies rows (see E7).

-- Highest single amount received per receiver
SELECT receiver_id, MAX(amount) FROM transactions GROUP BY receiver_id;

-- Show sender and receiver names: join users twice with different aliases
```

```
SELECT t.id, s.first_name AS sender, r.first_name AS receiver, t.amount
FROM transactions t
JOIN users s ON s.id = t.sender_id
JOIN users r ON r.id = t.receiver_id;
```

Net result for the first five users: Asha -700, Ravi -300, Meena +1000, John +500, Priya -500. (Check: 0-700, 1200-1500, 1700-700, 1700-1200, 0-500. The values sum to 0, as they should.)

Edge cases:

- Users who received nothing do not appear in a GROUP BY on transactions. Start from `users LEFT JOIN ...` if they must appear with 0.
- A `receiver_id` that is NULL, or that has no matching user (orphan data), is dropped by the INNER JOIN.
- Use DECIMAL, not FLOAT, for money.

Likely follow-ups:

- *Top 3 receivers?* Use `DENSE_RANK() <= 3` on the totals, or `LIMIT 3` if ties don't matter.
- *Highest receiver per month?* Partition the RANK by month: `DATE_FORMAT(created_at, '%Y-%m')` in MySQL, `date_trunc('month', created_at)` in PostgreSQL.
- *Which index helps?* `(receiver_id, amount)`, which is a covering index for the GROUP BY plus SUM.

E11. Explain Angular

Angular architecture — component, DI, HTTP and routing

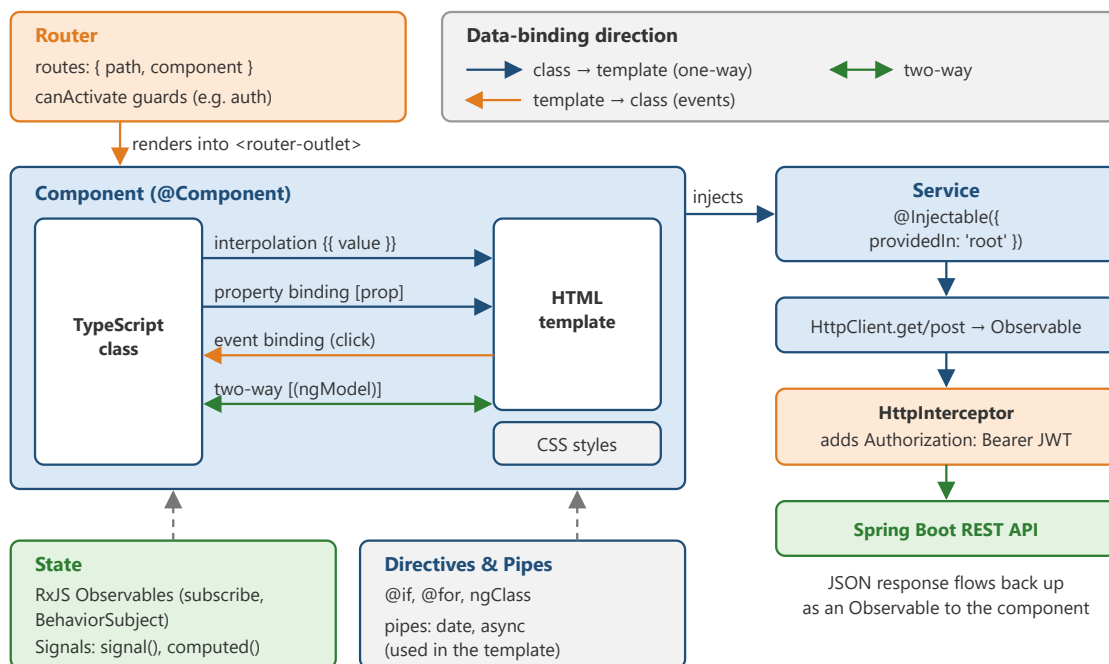
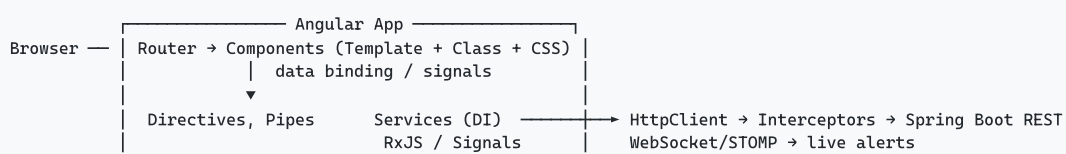


Figure 24: Angular architecture: component bindings, services, HttpClient, interceptor, router, state, directives and pipes

One-liner: Angular is Google's **TypeScript-based, component-driven framework** for building single-page applications. It ships "batteries included": a router, HttpClient, forms, dependency injection, testing tools and the CLI, all in one opinionated, consistent structure. That is why it is popular for large enterprise dashboards.

Big picture



1. Components and templates

A component is a TypeScript class, a template (HTML) and styles. Modern Angular (v14+, and the default since v17) uses **standalone components**, which declare their own `imports` with no `NgModule` needed.

```
import { Component, ChangeDetectionStrategy, input, output, signal, computed } from '@angular/core';
import { DatePipe } from '@angular/common';

export interface Incident { id: number; title: string; severity: 'LOW'|'MEDIUM'|'HIGH'; reportedAt: string; }

@Component({
  selector: 'app-incident-list',
  standalone: true, // default (and can be omitted) from Angular v19
  imports: [DatePipe],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <input placeholder="Filter" (input)="filter.set($any($event.target).value)" />

    @if (filtered().length) {
      <ul>
        @for (inc of filtered(); track inc.id) {
          <li [class.high]="inc.severity === 'HIGH'" (click)="selected.emit(inc)">
            {{ inc.title }}: {{ inc.reportedAt | date:'short' }}
          </li>
        } @empty {
          <li>No incidents</li>
        }
      </ul>
    } @else {
      <p>No incidents match "{{ filter() }}"</p>
    }
  `,
  styles: ['.high { color: crimson; font-weight: 600; }'],
})
export class IncidentListComponent {
  incidents = input.required<Incident[]>(); // signal input (v17.1+)
  selected = output<Incident>(); // output function (v17.3+)
  filter = signal('');
  filtered = computed(() =>
    this.incidents().filter(i => i.title.toLowerCase().includes(this.filter().toLowerCase())));
}
```

Classic (NgModule) equivalent, still common in existing codebases:

```
@Component({ selector: 'app-incident-list', templateUrl: './incident-list.component.html' })
export class IncidentListComponent {
  @Input() incidents: Incident[] = [];
  @Output() selected = new EventEmitter<Incident>();
}

@NgModule({
  declarations: [IncidentListComponent], // components/directives/pipes owned by this module
  imports: [CommonModule, FormsModule], // other modules needed by those templates
  exports: [IncidentListComponent], // what other modules may use
  providers: [IncidentService] // module-level DI (prefer providedIn: 'root')
})
export class IncidentModule {}
```

Classic template syntax: `<li *ngFor="let i of incidents; trackBy: trackById">`, `<p *ngIf="loading; else done">`, `[ngSwitch]`. The new control flow (`@if`, `@for`, `@switch`, `@defer`) is built in, needs no import, makes `track` mandatory, and is faster. Run `ng generate @angular/core:control-flow` to migrate.

2. Data binding (four types)

Type	Syntax	Direction	Example
Interpolation	{{ expr }}	Component → view	{{ sensor.name }}
Property binding	[prop]="expr"	Component → view	[disabled]="!form.valid", [class.active], [style.width.px]
Event binding	(event)="handler()"	View → component	(click)="ack(alert)"
Two-way	[(ngModel)]="x" or [(value)]	Both	Banana-in-a-box. With signals, use <code>model()</code> inputs (v17.2+)

Also: **attribute binding** `[attr.aria-label]` and **template reference variables** `#searchBox`.

3. Directives

- **Component:** a directive with a template.
- **Structural:** changes the DOM layout. Classic `*ngIf`, `*ngFor`, `*ngSwitch`; now mostly replaced by `@if` and `@for`.
- **Attribute:** changes the appearance or behaviour of an element. Built-ins are `ngClass` and `ngStyle`; you can write your own:

```
@Directive({ selector: '[appSeverity]', standalone: true })
export class SeverityDirective {
  appSeverity = input<'LOW'|'MEDIUM'|'HIGH'>();
  private el = inject(ElementRef);
  constructor() {
    effect(() => {
      const colors = { LOW: '#2e7d32', MEDIUM: '#f9a825', HIGH: '#c62828' };
      this.el.nativeElement.style.borderLeft = `4px solid ${colors[this.appSeverity() ?? 'LOW']}`;
    });
  }
}
// <div [appSeverity]="alert.severity">...</div>
```

Host directives (`hostDirectives`) let you compose behaviours onto a component.

4. Pipes

Pipes transform values in templates. Built-in pipes: `date`, `currency`, `decimal`, `percent`, `uppercase`, `json`, `async`, `keyvalue`, `slice`.

```
@Pipe({ name: 'timeAgo', standalone: true }) // pure by default: re-runs only when the input reference changes
export class TimeAgoPipe implements PipeTransform {
  transform(iso: string): string {
    const mins = Math.floor((Date.now() - new Date(iso).getTime()) / 60000);
    return mins < 1 ? 'just now' : mins < 60 ? `${mins} min ago` : `${Math.floor(mins / 60)} h ago`;
  }
}
```

The `async` pipe subscribes to an Observable or Promise, **unsubscribes automatically** on destroy, and marks the component for check under `OnPush`.

5. Services and dependency injection

A service is a class holding reusable logic or state (API calls, caching, WebSocket connections). Angular's hierarchical injector creates instances and passes them in.

```
@Injectable({ providedIn: 'root' }) // app-wide singleton, tree-shakable
export class IncidentService {
  private http = inject(HttpClient); // inject() function (v14+); constructor injection also works
  private base = '/api/incidents';

  list(params: { page: number; size: number }) {
    return this.http.get<Page<Incident>>(this.base, { params: { ...params } });
  }
  acknowledge(id: number) {
    return this.http.patch<void>(`${this.base}/${id}/ack`, {});
  }
}
```

- **Injector hierarchy:** environment/root injector, then route-level `providers`, then the element injector (component `providers: [...]` gives each component instance its own copy).
- **Tokens:** `InjectionToken<AppConfig>('APP_CONFIG')` with `{ provide: APP_CONFIG, useValue: {...} }`. Other provider types are `useClass`, `useFactory` and `useExisting`.
- **Resolution modifiers:** `inject(X, { optional: true, self: true, skipSelf: true, host: true })`.

6. Routing and guards

```
// app.routes.ts
export const routes: Routes = [
  { path: 'login', loadComponent: () => import('./auth/login.component').then(m => m.LoginComponent) },
  {
    path: '',
    canActivate: [AuthGuard],
    children: [
      { path: 'dashboard', loadComponent: () => import('./dashboard/dashboard.component').then(m => m.DashboardComponent) },
      { path: 'incidents', loadChildren: () => import('./incidents/incident.routes').then(m => m.INCIDENT_ROUTES) },
      { path: 'admin', canMatch: [roleGuard('ADMIN')], loadComponent: () => import('./admin/admin.component').then(m => m.AdminComponent) },
      { path: 'incidents/:id', loadComponent: () => import('./incidents/detail.component').then(m => m.DetailComponent),
        resolve: { incident: incidentResolver } },
    ]
  }
];
```

```

    ],
  },
  { path: '**', redirectTo: 'dashboard' }
];

// Functional guards (v15+; class-based guards are deprecated)
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  return auth.isLoggedIn() ? true : inject(Router).createUrlTree(['/login']);
};

export const roleGuard = (role: string): CanMatchFn => () => inject(AuthService).hasRole(role);

```

Guard	Purpose
canActivate / canActivateChild	Can the user enter this route or its children?
canMatch	Should this route definition match at all? (Also prevents lazy-loading the chunk for unauthorised users.)
canDeactivate	Can the user leave? (For example, unsaved form changes.)
resolve	Prefetch data before activation

Read route parameters with `inject(ActivatedRoute).paramMap`, or bind them directly to inputs using `withComponentInputBinding()`.

Security note: guards are a **UX** feature. The **real authorisation is enforced on the Spring Boot side** (Spring Security, `@PreAuthorize`).

7. HttpClient and interceptors

```

// Functional interceptor (v15+)
export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router); // call inject() synchronously, not inside the async callback
  const token = inject(AuthService).token();
  const authReq = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;
  return next(authReq).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) router.navigate(['/login']); // or try a refresh-token flow
      return throwError(() => err);
    })
  );
};

// main.ts, standalone bootstrap
bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withInterceptors([jwtInterceptor, loadingInterceptor, errorInterceptor])),
    provideAnimationsAsync(),
  ]
});

```

Classic version: a class implementing `HttpInterceptor`, registered with `{ provide: HTTP_INTERCEPTORS, useClass: JwtInterceptor, multi: true }` in `AppModule`. Interceptors run in the order they are provided for requests, and in reverse order for responses.

Typical interceptors: JWT/auth header, global error toast, loading spinner, retry with backoff, caching, correlation-ID header (`X-Correlation-Id`) for tracing across microservices.

8. RxJS

Angular's `HttpClient`, `Router` events and forms `valueChanges` all expose **Observables**.

Operator	Use
<code>map</code> , <code>filter</code> , <code>tap</code>	Transform, filter, side effects
<code>switchMap</code>	Cancel the previous inner request (search-as-you-type, route parameter changes)
<code>mergeMap</code>	Run inner requests in parallel
<code>concatMap</code>	Run inner requests in order, queued (ordered saves)
<code>exhaustMap</code>	Ignore new triggers while one is running (prevents double-submitting login)
<code>debounceTime</code> , <code>distinctUntilChanged</code>	Throttle user input
<code>combineLatest</code> , <code>forkJoin</code> , <code>withLatestFrom</code>	Combine streams (<code>forkJoin</code> is like <code>Promise.all</code>)
<code>catchError</code> , <code>retry({count, delay})</code>	Error handling

Operator	Use
<code>takeUntilDestroyed()</code>	Auto-unsubscribe (v16+)
<code>shareReplay(1)</code>	Cache and multicast

```
search = new FormControl('', { nullable: true });
results$ = this.search.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  filter(q => q.length >= 2),
  switchMap(q => this.api.searchSensors(q).pipe(catchError(() => of([]))))
);
```

Subjects: `Subject` (no initial value), `BehaviorSubject` (holds the current value; classic service state), `ReplaySubject(n)`. **Hot vs cold:** HTTP Observables are cold, and each subscribe triggers a new request.

9. Signals (Angular 16+, the modern reactivity primitive)

```
count = signal(0); // writable
double = computed(() => this.count() * 2); // derived, memoized, lazy
constructor() {
  effect(() => console.log('count changed', this.count())); // side effects (logging, localStorage, manual DOM)
}
inc() { this.count.update(c => c + 1); } // or .set(5)
```

- **Interop with RxJS:** `toSignal(obs$, { initialValue: [] })` and `toObservable(sig)` from `@angular/core/rxjs-interop`.
- **Newer APIs:** `input()`, `output()`, `model()`, `viewChild()`, `linkedSignal()`, and `resource()` / `httpResource()` for async data. Check which are stable in your Angular version.
- **Why signals:** fine-grained change tracking (Angular knows exactly which views depend on which signal), no manual subscriptions in templates, and they enable **zoneless** change detection.

10. Change detection

- **Default:** Zone.js patches async APIs (events, timers, XHR). After any of them, Angular checks the whole component tree from top to bottom.
- **OnPush:** a component is checked only when (a) an `@Input` reference changes, (b) an event fires inside it, (c) the `async` pipe emits or a signal it reads changes, or (d) `markForCheck()` is called. You must use **immutable updates**: `this.items = [...this.items, x]`, not `push`.
- **Zoneless:** `provideZonelessChangeDetection()` drops Zone.js and relies on signals, the `async` pipe and events. It became stable in v20 and is the default for new projects in recent versions.
- Tools: `ChangeDetectorRef.markForCheck()` / `detectChanges()`, plus `track` in `@for` to avoid re-rendering whole lists.
- A real-time dashboard receiving many WebSocket messages per second **must** use `OnPush` or signals, and often buffers or throttles updates (`bufferTime(500)`).

11. Lifecycle hooks (in order)

Hook	When / use
<code>constructor</code>	DI only; keep it light
<code>ngOnChanges(changes)</code>	Before <code>ngOnInit</code> and whenever an <code>@Input</code> reference changes
<code>ngOnInit</code>	Once, after the first inputs are set. Load data and set up subscriptions.
<code>ngDoCheck</code>	Every change-detection run (custom change detection; use rarely)
<code>ngAfterContentInit</code> / <code>ngAfterContentChecked</code>	Projected <code><ng-content></code> is ready or checked
<code>ngAfterViewInit</code> / <code>ngAfterViewChecked</code>	The component's view and children are ready. Access <code>@ViewChild</code> (for example, initialise a Leaflet map or chart here).
<code>ngOnDestroy</code>	Clean up: unsubscribe, close the WebSocket, clear intervals. <code>DestroyRef.onDestroy()</code> and <code>takeUntilDestroyed()</code> are the modern helpers.

Newer: `afterNextRender()` and `afterRender()` / `afterEveryRender()` are browser-only DOM hooks, which matter for SSR-safe third-party libraries.

12. Forms: reactive vs template-driven

	Template-driven	Reactive
Module/imports	<code>FormsModule</code>	<code>ReactiveFormsModule</code>
Model defined in	The template (<code>ngModel</code>)	The component class (<code>FormGroup</code> / <code>FormControl</code>)
Data flow	Asynchronous, mutable	Synchronous, explicit, immutable-friendly
Validation	Directives (<code>required</code> , <code>minlength</code>)	Functions (<code>Validators.required</code> , custom/async validators)
Dynamic fields	Awkward	<code>FormArray</code> makes it easy
Testing	Harder (needs the DOM)	Easy (pure class)
Best for	Simple forms (login, search)	Complex forms (incident creation with dynamic checklists)

```
// Reactive form with typed forms (v14+)
private fb = inject(NonNullableFormBuilder);
form = this.fb.group({
  title: ['', [Validators.required, Validators.maxLength(120)]],
  severity: ['MEDIUM' as 'LOW'|'MEDIUM'|'HIGH', Validators.required],
  location: this.fb.group({ lat: [0, Validators.required], lng: [0, Validators.required] }),
  tags: this.fb.array<string>([])
});
submit() {
  if (this.form.invalid) { this.form.markAllAsTouched(); return; }
  this.incidentService.create(this.form.getRawValue()).subscribe();
}
```

```
<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="title" />
  @if (form.controls.title.touched && form.controls.title.hasError('required')) {
    <small class="error">Title is required</small>
  }
  <button [disabled]="form.invalid">Save</button>
</form>
```

```
<!-- Template-driven -->
<form #f="ngForm" (ngSubmit)="login(f.value)">
  <input name="username" ngModel required />
  <input name="password" type="password" ngModel required minlength="8" />
  <button [disabled]="f.invalid">Login</button>
</form>
```

(Signal-based forms are arriving in newer versions as an experimental API. Mention that only if asked.)

Other features worth mentioning

- **@defer**: lazy-load part of a template (`@defer (on viewport) { <heavy-chart/> } @placeholder { ... }`).
- **SSR and hydration** (`@angular/ssr`).
- **The CLI**: `ng new` , `ng g c` , `ng build` (esbuild/Vite since v17), `ng test` , `ng update` .
- **Environments** and file replacement.
- **Content projection** with `<ng-content select="...">` .
- **ViewEncapsulation** (Emulated, None or ShadowDom).

Likely follow-ups:

- *Angular vs AngularJS?* AngularJS (1.x) is JavaScript with MVC, `$scope` and digest cycles, and is end-of-life. Angular (2+) is TypeScript, components, RxJS and AOT compilation, and a complete rewrite.
- *What is AOT?* Ahead-of-time compilation turns templates into JavaScript at build time, which gives faster startup, earlier template error detection, and smaller bundles. It is the default.
- *Observable vs Promise?* An Observable is lazy, can emit many values, can be cancelled (unsubscribed), and has operators. A Promise is eager, resolves once, and cannot be cancelled.
- *How do you share data between components?* Parent to child with `input()` / `@Input` , child to parent with `output()` / `@Output` , and between unrelated components with a shared service holding a signal or BehaviorSubject (or NgRx/SignalStore for large apps).

E12. Your Angular work, roles and responsibilities, and introducing yourself as an Angular developer

TEMPLATE. Replace every [placeholder] with your real facts. Delete any feature you did not personally build. Interviewers probe details, so only claim what you can explain line by line.

30-second intro (as an Angular / full-stack developer)

"I'm [Name], a full-stack developer with [X] years of experience, currently at Trinity Mobility, where I work on [product/module name, e.g. the Integrated Command & Control Center web console] for Smart City deployments. On the front end I work with Angular [version], using [standalone components / NgModules], RxJS, [signals], [Angular Material / PrimeNG / Bootstrap], and [Leaflet / OpenLayers / Google Maps] for map views. On the back end I build Spring Boot microservices that the UI consumes. Recently I [one concrete achievement, e.g. built the live alert feed over WebSocket, or cut dashboard load time from [X]s to [Y]s with lazy loading and OnPush]."

Roles and responsibilities (front-end focused; pick the ones that are true)

- Translated [UX/Figma/BA] requirements into **reusable Angular components** and feature modules (or standalone feature folders).
- Built **dashboards** for [traffic/incident/environment/loT sensor] monitoring with [chart library] and map layers.
- Integrated **REST APIs** from Spring Boot services with `HttpClient`, typed models and error handling.
- Implemented **authentication and authorisation on the UI**: login, a JWT interceptor, token refresh, route guards, and **role-based menus**.
- Implemented **real-time updates** over [WebSocket/STOMP/SockJS/SSE] for [alerts/vehicle positions/sensor readings].
- Built **reactive forms** with custom and async validators (for example [incident creation, SOP configuration, device registration]).
- Performance work**: lazy loading, OnPush/signals, `trackBy / track`, virtual scrolling (`cdk-virtual-scroll-viewport`), `@defer`, bundle-size budgets.
- Wrote **unit tests** with [Jasmine/Karma or Jest] (and [Cypress/Playwright] e2e) to [X]% coverage.
- Handled **Angular version upgrades** ([from vA to vB]) with `ng update`, and fixed breaking changes.
- Took part in **code reviews**, sprint planning, and production issue triage.

Example components a Smart City dashboard would have (talking points with code)

These are **illustrations** of typical components. Describe only the ones you actually built, in your own project's terms.

1. Map view (incidents and sensors on a map)

```
@Component({
  selector: 'app-city-map',
  standalone: true,
  template: '<div #mapEl class="map"></div>',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class CityMapComponent {
  incidents = input<Incident[]>([]);
  private mapEl = viewChild.required<ElementRef<HTMLDivElement>>('mapEl');
  private map?: L.Map;
  private layer = L.LayerGroup();

  constructor() {
    afterNextRender(() => { // DOM is ready (SSR-safe)
      this.map = L.map(this.mapEl().nativeElement).setView([12.97, 77.59], 12);
      L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(this.map);
      this.layer.addTo(this.map);
    });
    effect(() => { // re-draw markers when the input signal changes
      const list = this.incidents();
      this.layer.clearLayers();
      list.forEach(i => L.marker([i.lat, i.lng]).bindPopup(i.title).addTo(this.layer));
    });
    inject(DestroyRef).onDestroy(() => this.map?.remove());
  }
}
```

Talking points: marker clustering for thousands of devices, GeoJSON layers for zones/wards, a bounding-box query to fetch only the visible area, and running the map outside Angular's zone (`NgZone.runOutsideAngular`) to avoid excessive change detection.

2. Live alert feed over WebSocket (STOMP with Spring)

```
@Injectable({ providedIn: 'root' })
export class AlertStreamService {
```

```

private auth = inject(AuthService); // inject() must run in an injection context (field initialiser)
private client = new Client({ // @stomp/stompjs
  brokerURL: 'wss://[host]/ws',
  reconnectDelay: 5000,
  beforeConnect: () => { this.client.connectHeaders = { Authorization: `Bearer ${this.auth.token()}` } }; }
});
private _alerts = signal<Alert[]>([]);
readonly alerts = this._alerts.asReadonly();
readonly highCount = computed(() => this._alerts().filter(a => a.severity === 'HIGH').length);

connect() {
  this.client.onConnect = () => {
    this.client.subscribe('/topic/alerts', msg => {
      const alert: Alert = JSON.parse(msg.body);
      this._alerts.update(list => [alert, ...list].slice(0, 200)); // cap the list size
    });
    this.client.activate();
  }
}
disconnect() { this.client.deactivate(); }
}

```

Back-end side: Spring `@EnableWebSocketMessageBroker`, with `SimpMessagingTemplate.convertAndSend("/topic/alerts", alert)` fed by a Kafka consumer. Talking points: reconnect and backoff, deduplication by alert ID, throttling UI updates, audible or visual escalation for HIGH severity, and acknowledging alerts through REST.

3. Charts (KPI widgets)

These show incidents per hour, average response time, and sensor values over time, built with [ngx-charts / Chart.js via ng2-charts / ECharts / Highcharts]. Talking points: time-window selectors, server-side aggregation (do not send raw readings to the browser), and `@defer (on viewport)` for heavy charts.

4. Role-based menus

```

export const MENU: MenuItem[] = [
  { label: 'Dashboard', route: '/dashboard', roles: ['OPERATOR', 'SUPERVISOR', 'ADMIN'] },
  { label: 'Incidents', route: '/incidents', roles: ['OPERATOR', 'SUPERVISOR'] },
  { label: 'Reports', route: '/reports', roles: ['SUPERVISOR', 'ADMIN'] },
  { label: 'User Admin', route: '/admin', roles: ['ADMIN'] },
];

@Component({
  selector: 'app-side-nav', standalone: true, imports: [RouterLink, RouterLinkActive],
  template: `
    @for (item of visibleMenu(); track item.route) {
      <a [routerLink]="item.route" routerLinkActive="active">{{ item.label }}</a>
    }`
})
export class SideNavComponent {
  private auth = inject(AuthService);
  visibleMenu = computed(() => MENU.filter(m => m.roles.some(r => this.auth.roles().includes(r))));
}

```

Roles come from JWT claims. The same roles protect routes (`canMatch`) and, most importantly, the APIs (Spring Security).

5. JWT interceptor: see E11 §7. Mention attaching the token, handling 401 with a refresh token (queueing concurrent requests during the refresh), and logging out on refresh failure.

Other typical components: a paginated and filterable incident grid (server-side paging, see E8), an incident detail and workflow timeline, SOP checklists, a CCTV camera tile grid, a device health/status board, a notifications bell, and dark/light themes for control-room screens.

Delivery tips

- Lead with **business context**, then **the components**, then **one hard problem you solved**.
- Quantify: "[N] screens", "[N] reusable components", "load time from [X]s to [Y]s", "[N] alerts per minute rendered smoothly".
- Expect deep-dive follow-ups on anything you mention: OnPush, the interceptor order, how you unsubscribe, how token refresh works.

What interviewers look for: real ownership (not "we did"), understanding of *why* a choice was made, awareness of performance and security, and the ability to connect the UI to back-end APIs end to end.

Likely follow-ups:

- *How do you avoid memory leaks?* Use the `async` pipe or signals, `takeUntilDestroyed()`, close WebSockets and clear intervals in `ngOnDestroy` / `DestroyRef`.
- *How do you handle thousands of map markers?* Clustering, viewport-based fetching, canvas renderers, and updating outside the zone.
- *Where do you keep the JWT?* In memory or a short-lived store, with a refresh token in an HttpOnly cookie if the backend supports it. Avoid `localStorage` for long-lived tokens because of XSS risk.

- *How do you test a component?* `TestBed` with `provideHttpClientTesting()` / `HttpTestingController`, plus mocked services and harnesses for Material components.

E13. Basic JavaScript concepts

1. Data types

- **Primitives (7):** `string`, `number`, `bigint`, `boolean`, `undefined`, `symbol`, `null`. They are immutable and copied by value.
- **Object (reference) types:** objects, arrays, functions, `Date`, `Map`, `Set` and so on. They are copied **by reference**.

```
typeof 42          // "number"
typeof 10n         // "bigint"
typeof 'hi'        // "string"
typeof undefined  // "undefined"
typeof null       // "object" ← a historical bug
typeof []         // "object" → use Array.isArray([]) === true
typeof function(){} // "function"
typeof Symbol()   // "symbol"

let a = { x: 1 }; let b = a; b.x = 2;
console.log(a.x); // 2 (same reference)
let c = structuredClone(a); // deep copy; { ...a } is only a shallow copy
```

Falsy values: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. Everything else is truthy, including `[]`, `{}` and `'0'`.

2. Scope

- **Global, function and block scope.** `let` and `const` are block-scoped; `var` is function-scoped (see E14).
- **Lexical scope:** a function sees variables where it is *defined*, not where it is called.

```
const city = 'Bengaluru';
function outer() {
  const zone = 'North';
  function inner() { return `${city}-${zone}`; } // looks up the scope chain
  return inner();
}
outer(); // "Bengaluru-North"
```

3. Closures

A closure is a function bundled with references to its outer variables, which stay alive after the outer function returns.

```
function createCounter() {
  let count = 0; // private state
  return {
    inc: () => ++count,
    get: () => count
  };
}
const c1 = createCounter();
c1.inc(); c1.inc();
console.log(c1.get()); // 2
console.log(c1.count); // undefined (truly private)
```

Uses: data privacy and the module pattern, function factories, memoization, debounce, event handlers, and RxJS operators.

```
function debounce(fn, ms) {
  let t;
  return (...args) => { clearTimeout(t); t = setTimeout(() => fn(...args), ms); };
}
```

4. `this` binding (determined by how a function is called)

Call style	<code>this</code> is
<code>obj.method()</code>	<code>obj</code> (implicit binding)
<code>fn()</code> plain call	<code>undefined</code> in strict mode or modules; <code>globalThis</code> in sloppy mode
<code>fn.call(o, a)</code> / <code>fn.apply(o, [a])</code> / <code>fn.bind(o)</code>	<code>o</code> (explicit binding)
<code>new Fn()</code>	The newly created object

Call style	this is
Arrow function	Lexical: inherited from the enclosing scope; cannot be rebound

```
const sensor = {
  name: 'AQI-01',
  regular() { return this.name; },
  arrow: () => this?.name, // 'this' comes from module scope, so undefined
  delayed() { setTimeout(() => console.log(this.name), 0); } // the arrow keeps the outer 'this'
};
sensor.regular(); // "AQI-01"
sensor.arrow(); // undefined
const f = sensor.regular;
f(); // TypeError in strict mode (this is undefined); undefined in sloppy mode
f.call({ name: 'X' }); // "X"
sensor.delayed(); // "AQI-01"
```

This is why Angular templates and RxJS callbacks commonly use arrow functions: they keep the component's `this`.

5. Prototypes and inheritance

Every object has an internal `[[Prototype]]` link. Property lookup walks the **prototype chain**. `class` is syntactic sugar over prototypes.

```
function Device(id) { this.id = id; }
Device.prototype.describe = function () { return `Device ${this.id}`; };

const d = new Device(7);
d.describe(); // "Device 7" (found on Device.prototype)
Object.getPrototypeOf(d) === Device.prototype; // true
d.hasOwnProperty('describe'); // false

class Camera extends Device {
  constructor(id, url) { super(id); this.url = url; }
  describe() { return `${super.describe()} (camera)`; }
}
new Camera(9, 'rtsp://...').describe(); // "Device 9 (camera)"
```

6. Promises and async/await

```
function fetchSensor(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => (id > 0 ? resolve({ id, value: 42 }) : reject(new Error('bad id'))), 100);
  });
}

fetchSensor(1)
  .then(s => s.value)
  .then(v => console.log(v)) // 42
  .catch(e => console.error(e))
  .finally(() => console.log('done'));

async function load() {
  try {
    const [a, b] = await Promise.all([fetchSensor(1), fetchSensor(2)]); // parallel
    return a.value + b.value;
  } catch (e) {
    console.error(e.message);
  }
}
```

Combinator	Resolves when	Rejects when
<code>Promise.all</code>	All fulfil	Any rejects (fails fast)
<code>Promise.allSettled</code>	All settle	Never; returns <code>{status, value/reason}</code> for each
<code>Promise.race</code>	The first one settles	The first one settles with a rejection
<code>Promise.any</code>	The first one fulfils	All reject (<code>AggregateError</code>)

Pitfall: `await` inside a `for` loop runs requests **sequentially**. Use `Promise.all(items.map(fetch))` for parallel calls.

7. The event loop: microtasks vs macrotasks

JavaScript runs on a single thread. The event loop works like this:

1. Run the current synchronous code (the call stack).
2. **Drain all microtasks:** `Promise .then / catch / finally`, `await` continuations, `queueMicrotask`, `MutationObserver`.

3. Render (in browsers), if needed.

4. Take **one macrotask** (`setTimeout` , `setInterval` , I/O, UI events, `MessageChannel`), then go back to step 2.

```
console.log('1: sync');
setTimeout(() => console.log('2: timeout (macrotask)'), 0);
Promise.resolve().then(() => console.log('3: promise (microtask)'));
queueMicrotask(() => console.log('4: queueMicrotask'));
(async () => {
  console.log('5: async fn body runs sync');
  await null;
  console.log('6: after await (microtask)');
})();
console.log('7: sync');
```

Output:

```
1: sync
5: async fn body runs sync
7: sync
3: promise (microtask)
4: queueMicrotask
6: after await (microtask)
2: timeout (macrotask)
```

Why: the synchronous lines run first (1, 5, 7). Then all microtasks run in the order they were queued (3, 4, 6). Then the macrotask runs (2). An endless stream of microtasks can **starve** rendering and timers. Node.js also has `process.nextTick` , which runs before promise microtasks.

8. == vs ===

- `===` is strict equality: no type coercion. **Use it by default.**
- `==` is loose equality: it coerces types first.

```
0 == ''           // true   ('' → 0)
0 === ''          // false
'5' == 5          // true
null == undefined // true   (the one useful case: x == null checks both)
null === undefined // false
null == 0         // false
[] == false       // true   ([] → '' → 0, false → 0)
NaN == NaN        // false   → use Number.isNaN(x)
Object.is(NaN, NaN) // true;  Object.is(0, -0) → false
{} == {}          // false   (different references)
```

9. Spread, rest and destructuring

```
// Spread: expand
const base = { severity: 'LOW', status: 'OPEN' };
const incident = { ...base, id: 1, severity: 'HIGH' }; // later keys win → severity 'HIGH'
const all = [...[1, 2], ...[3, 4]];                  // [1,2,3,4]
Math.max(...[3, 9, 2]);                               // 9

// Rest: collect
function log(level, ...msgs) { console.log(level, msgs); }
log('INFO', 'a', 'b');                               // INFO ['a','b']
const { id, ...rest } = incident;                     // rest = {severity:'HIGH', status:'OPEN'}

// Destructuring with defaults and renaming
const { status: st = 'NEW', location: { lat = 0 } = {} } = incident; // st='OPEN', lat=0
const [first, , third = 'x'] = ['a', 'b'];             // first='a', third='x'
[a1, b1] = [b1, a1];                                  // swap

// Optional chaining and nullish coalescing
const zone = incident.location?.zone ?? 'Unknown';     // ?? replaces only null/undefined, not 0 or ''
```

Other quick ones: `map` / `filter` / `reduce` ; template literals; modules (`import` / `export`); `Map` / `Set` ; immutability with `Object.freeze` (shallow); `JSON.parse` / `JSON.stringify` .

Likely follow-ups:

- *What does `[1,2,3].map(parseInt)` return?* `[1, NaN, NaN]` , because `map` passes `(value, index)` and `parseInt` treats the index as the radix.
- *Shallow vs deep copy?* `Spread` and `Object.assign` copy only the top level. `structuredClone` copies deeply (but not functions).
- *Arrow vs regular function?* An arrow has no own `this` , `arguments` or `prototype` , and cannot be used with `new` .
- *What is event delegation?* One listener on a parent handles events from its children via bubbling and `event.target` .

E14. var, let, const and hoisting

JavaScript: var vs let vs const (Scope, Hoisting, TDZ)

var	let	const
Scope function (or global)	Scope block { }	Scope block { }
Hoisting hoisted + set to undefined	Hoisting hoisted; TDZ until declared	Hoisting hoisted; TDZ until declared
Redeclare (same scope) allowed	Redeclare (same scope) SyntaxError	Redeclare (same scope) SyntaxError
Reassign allowed	Reassign allowed	Reassign TypeError; must initialise
<pre>if (true) { var v = 1; } v; // 1 (leaks block)</pre>	<pre>if (true) { let l = 1; } l; // ReferenceError</pre>	<pre>const o = { n: 1 }; o.n = 2; // OK, mutable o = {}; // TypeError</pre>

Temporal Dead Zone (TDZ) inside a block



With var, reading x before its declaration gives undefined instead (no TDZ).

Figure 25: JavaScript var, let, const and hoisting

Comparison

	var	let	const
Scope	Function (or global)	Block { }	Block
Hoisted?	Yes, initialised to undefined	Yes, but uninitialised (TDZ)	Yes, uninitialised (TDZ)
Access before declaration	undefined	ReferenceError	ReferenceError
Redeclare in the same scope	Allowed	SyntaxError	SyntaxError
Reassign	Yes	Yes	No (TypeError)
Must initialise at declaration	No	No	Yes
Creates a property on <code>window</code> (global scope)	Yes	No	No

Hoisting

During the **creation phase** of each scope, JavaScript registers declarations before running code. The code is not physically moved.

```
console.log(a); // undefined (var is hoisted and initialised to undefined)
var a = 10;
console.log(a); // 10

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20;

console.log(c); // ReferenceError: Cannot access 'c' before initialization
const c = 30;

console.log(d); // ReferenceError: d is not defined (never declared, a different message)
```

Temporal Dead Zone (TDZ)

The TDZ is the period from the start of the block until the `let / const` declaration line runs.

```
let x = 'outer';
{
  // TDZ for the inner x starts here. It shadows the outer x even before its declaration.
  console.log(x); // ReferenceError (not 'outer')
  let x = 'inner';
}
```

```

}

typeof undeclaredVar; // "undefined" (safe)
typeof y; // ReferenceError: y is in its TDZ
let y = 1;

function f(p = q, q = 1) {} // f() → ReferenceError: q is accessed in its TDZ (default params are evaluated left to right)

```

Function scope vs block scope

```

function test() {
  if (true) {
    var v = 'var';
    let l = 'let';
    const k = 'const';
  }
  console.log(v); // "var" (var ignores the block)
  console.log(l); // ReferenceError: l is not defined
}

```

Redeclaration

```

var n = 1;
var n = 2; // fine, silently overwrites (a source of bugs)

let m = 1;
let m = 2; // SyntaxError: Identifier 'm' has already been declared

let s = 1;
{ let s = 2; } // fine: a different block, so this is shadowing, not redeclaration

```

const with objects and arrays

`const` makes the **binding** immutable, not the value.

```

const cfg = { refreshMs: 5000 };
cfg.refreshMs = 1000; // allowed (mutating the object)
cfg = {}; // TypeError: Assignment to constant variable.

const ids = [1, 2];
ids.push(3); // allowed → [1,2,3]

const frozen = Object.freeze({ a: 1, nested: { b: 2 } });
frozen.a = 99; // ignored (TypeError in strict mode)
frozen.nested.b = 99; // allowed: freeze is SHALLOW

```

The classic loop closure question

```

for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log('var:', i), 0);
}
// var: 3
// var: 3
// var: 3

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log('let:', j), 0);
}
// let: 0
// let: 1
// let: 2

```

Why: `var i` is **one** function-scoped variable shared by all callbacks. By the time the timeouts run (macrotasks, after the loop finishes), `i` is 3. `let j` creates a **new binding for each iteration**, so each closure captures its own value.

Pre-ES6 fixes, which interviewers sometimes ask for:

```

for (var i = 0; i < 3; i++) {
  (function (k) { setTimeout(() => console.log(k), 0); })(i); // IIFE captures a copy
}
for (var i = 0; i < 3; i++) {
  setTimeout(console.log, 0, i); // pass it as a setTimeout argument
}

```

Function declarations vs function expressions

```
greet(); // "hello": declarations are fully hoisted (name AND body)
function greet() { console.log('hello'); }

sayHi(); // TypeError: sayHi is not a function (var hoisted as undefined)
var sayHi = function () { console.log('hi'); };

sayBye(); // ReferenceError: Cannot access 'sayBye' before initialization (TDZ)
const sayBye = () => console.log('bye');

// Classes are hoisted like let (TDZ)
new Foo(); // ReferenceError
class Foo {}
```

Precedence quiz:

```
console.log(typeof foo); // "function"
var foo = 1;
function foo() {}
console.log(typeof foo); // "number"
```

Why: during creation, the function declaration initialises `foo`, and the `var foo` declaration is ignored because the name already exists. At runtime, `foo = 1` reassigns it.

Best practice

Use **const** by default, **let** when you must reassign, and **never var** in modern code. ESLint has the `no-var` and `prefer-const` rules for this, and they are commonly enabled in Angular projects.

Likely follow-ups:

- *Are let and const hoisted?* Yes, they are hoisted but not initialised. That is exactly what the TDZ is.
- *What does this print: `let a = 1; { console.log(a); let a = 2; }`?* A ReferenceError, because the inner `a` is in its TDZ and shadows the outer one.
- *Why does global var matter?* `var x` at the top level becomes `window.x` and can clash with other scripts. `let` and `const` live in a separate declarative scope.

Project and HR templates (E15–E24): everything in `[brackets]` is a placeholder. The Smart City wording describes the *kind* of product Trinity Mobility builds (an integrated command and control center, IoT sensor data, traffic and incident dashboards). It is **not** a statement of what *you* did. Keep only what is true for you, and use numbers you can defend. Interviewers routinely ask “how exactly?” about any claim.

E15. Tell me about yourself

Structure: Present → Past → Strengths/Proof → Future (about 90 seconds, roughly 220–250 words)

Part	Time	Content
Present	20 s	Current role, company, product domain, core stack
Past / growth	20 s	How you got here: education or early roles, how your responsibilities grew
Proof	35 s	2–3 concrete achievements with numbers (back end, front end, delivery)
Future / fit	15 s	Why this role and company, in one sentence

Full template

"Hi, I'm **[Name]**. I'm a Java full-stack developer with about **[X]** years of experience, currently working at **Trinity Mobility** on its **Smart City platform**, specifically **[module, e.g. the Integrated Command & Control Center / traffic management / incident management]**. The platform helps **[city operators / police / municipal departments]** monitor **[IoT sensors, traffic signals, CCTV, environmental sensors]** and respond to incidents in real time.

My core stack is **Java [8/11/17/21]**, **Spring Boot**, **Spring Data JPA**, **[MySQL/PostgreSQL]** on the back end and **Angular [version]** on the front end, with **[Kafka/RabbitMQ]**, **[Redis]**, **[Docker/Kubernetes]**, **[Jenkins/GitLab CI]** around it.

I started as **[e.g. a trainee / junior developer working on bug fixes and REST APIs]**, and over time I took ownership of **[e.g. the incident-management microservice end to end]**.

A few things I'm proud of:

- I **[built / redesigned]** [e.g. the real-time alert pipeline from sensor events to the operator dashboard over WebSocket], which [e.g. reduced alert latency from [X] s to [Y] s].
- I **[optimised]** [e.g. slow dashboard queries with proper indexing and pagination], cutting response time from [X] ms to [Y] ms.
- On the front end I built [e.g. the map-based incident view and role-based menus], and I [e.g. raised unit test coverage to [X]%).

I enjoy working on systems where reliability and performance matter. I'm now looking for **[a role with larger-scale distributed systems / more ownership of design / a product with [domain]]**, and that's why this position at **[Company]** interests me, particularly **[something specific from their job description or product]**."

Short variant (about 45 seconds, for "briefly introduce yourself")

"I'm [Name], a Java full-stack developer with [X] years at Trinity Mobility, building Spring Boot microservices and Angular dashboards for Smart City command-and-control products. My main work has been [module], where I [top achievement with a number]. I'm looking for [goal], which matches this role."

Tips

- **Don't recite your résumé** or your personal history (hometown, hobbies) unless asked. Keep it professional.
- **End with a hook.** Your last sentence should invite the interviewer to ask about your strongest project.
- **Tailor the proof points** to the job description. If they stress microservices, lead with that achievement.
- Practise out loud with a timer. Aim for about 90 seconds and avoid filler words.
- Say "I" for your own contributions and "we" for team outcomes. Be honest about which is which.
- Do not disclose confidential client names, city contracts or internal metrics if your NDA forbids it. Say "a large Indian city deployment" instead.

What interviewers look for: clarity, relevance to the role, evidence of ownership and growth, communication skill, and whether your story matches your résumé.

E16. Roles and responsibilities

Template (state your designation first, then group by area)

"As a **[Software Engineer / Senior Software Engineer]** in the **[team name]** team of about **[N]** people, my responsibilities are:"

Design and development (back end)

- Design and develop **RESTful microservices** with Spring Boot for **[module(s)]**, including entities, repositories, services, controllers, DTOs and validation.
- Write **efficient SQL and JPA queries**, design **schema changes** with **[Flyway/Liquibase]**, and tune indexes.
- Integrate with **[Kafka/RabbitMQ]** for event-driven flows (for example, sensor events and incident lifecycle events).
- Implement **security**: Spring Security with JWT/OAuth2 ([Keycloak/custom auth service]) and role-based access.
- Integrate **third-party and device data sources** ([IoT gateways, vendor APIs, GIS services]); see E23.

Front end

- Build **Angular** screens and reusable components, integrate APIs, and implement guards, interceptors and real-time views (see E12).

Quality

- Write **unit and integration tests** ([JUnit 5, Mockito, Testcontainers] / [Jasmine/Jest]), and fix **SonarQube** issues.
- Do **peer code reviews** through pull/merge requests.

Delivery and operations

- Take part in **Agile ceremonies**: sprint planning, estimation, stand-ups, retros. Break stories into tasks in **[Jira]**.
- Handle **CI/CD** pipelines ([Jenkins/GitLab CI/GitHub Actions]), **Dockerise** services, and support deployments to **[Kubernetes/VMs/on-prem city data center]**.
- Provide **production support / L3**: analyse logs in **[ELK/Grafana/Kibana]**, find root causes, deliver hotfixes. Take part in **[on-call rotation]** if applicable.

Collaboration and leadership (include only if true)

- Work with **BAs, QA, DevOps and field/integration teams**, and sometimes join **client demos or UAT** with **[city authority / client]** stakeholders.
- **Mentor** [N] juniors, onboard new joiners, and write technical documentation (Swagger/OpenAPI, Confluence).

Delivery tips

- Lead with the **3–4 most relevant** responsibilities. Do not list all fifteen.
- Attach **one example** to each: "...for example, I owned the [X] API used by [Y] screens."
- Make **progression** visible: "Initially I did ___; since [year] I own ___."
- Match the seniority you are claiming. A 5–6 year candidate should mention design decisions, reviews and mentoring.

What interviewers look for: the scope of your ownership, whether you are hands-on, your exposure to the full SDLC (design, code, test, deploy, support), and whether your responsibilities match your years of experience.

E17. Explain your project (e.g. a Trinity Mobility Smart City product)

Framework: Problem → Users → Solution/Modules → Architecture → Scale → My contribution → Impact → Learnings

TEMPLATE. Fill it in from your actual project. If details are confidential, generalise them ("a metro-city deployment", "thousands of devices").

1. Problem / context (1–2 sentences)

"Cities run many separate systems: traffic signals, CCTV, environmental sensors, emergency calls, utility SCADA and so on. Operators have no single view, so incident response is slow and manual. Our product, **[product name, e.g. the Integrated Command & Control Center platform]**, brings these feeds together into **one operational picture** with **workflows** for responding to events."

2. Users

"The main users are **[control-room operators, supervisors, department heads (police / traffic / municipal / disaster management), and city administrators]**. Some also use **[a mobile app for field officers]**."

3. Key modules (name the ones that exist in *your* product)

Module	What it does	My involvement
[Device/sensor integration]	Ingests data from [IoT sensors, ATCS, ANPR/RLVD cameras, GPS on vehicles]	[built / maintained / none]
[Event & alert engine]	Applies rules/thresholds to raw data, raises alerts, deduplicates and correlates	[]
[Incident management]	Creates incidents, assigns them, follows SOPs, escalates, tracks SLA, closes	[]
[GIS / map dashboard]	Live map of assets, incidents and vehicles, with layers	[]
[Analytics & reports]	KPIs, historical trends, scheduled reports	[]
[User / role management]	RBAC, departments, audit trail	[]
[Notifications]	SMS / email / push / [IVR]	[]

4. Architecture (summary; details in E18)

"It's a **microservices** architecture on **Spring Boot**, with an **Angular** front end. Devices and third-party systems push data through **[an integration layer / MQTT broker / REST adapters]** into **[Kafka]**. Services such as **[event-processing, incident, notification, user]** consume those events. Data is stored in **[PostgreSQL/MySQL]** for transactions, **[TimescaleDB/Cassandra/Elasticsearch/MongoDB]** for time-series and search data, and **[Redis]** for caching. The UI gets live updates over **[WebSocket/STOMP]**. Everything runs in **[Docker on Kubernetes / on-prem VMs in the city data center]**."

5. Scale (use only real numbers, or bands like "thousands")

"[N] cities/deployments, about [N] connected devices, about [N] events per [second/minute/day], [N] concurrent operator sessions, [N] microservices, and a team of [N]."

6. My contribution (the most important part, so spend the most time here)

"I mainly worked on [module]. Specifically:

- I designed and built [feature], which [what it does].
- I implemented [technical piece, e.g. a Kafka consumer with retry and dead-letter topic for sensor events].
- On the UI I built [screen/component].
- I fixed / optimised [issue], which resulted in [metric]."

7. Impact (placeholders; never invent numbers)

"This [reduced incident response time by [X]% / cut alert latency from [X] s to [Y] s / removed [N] hours of manual reporting per week / supported a [N]× increase in devices without extra hardware / improved the API p95 from [X] ms to [Y] ms]."

8. Learnings / what you would do differently (shows maturity)

"I learned [e.g. designing for idempotency when devices resend data, and the value of observability]. If I were starting again, I would [e.g. introduce contract tests between services earlier]."

Delivery tips

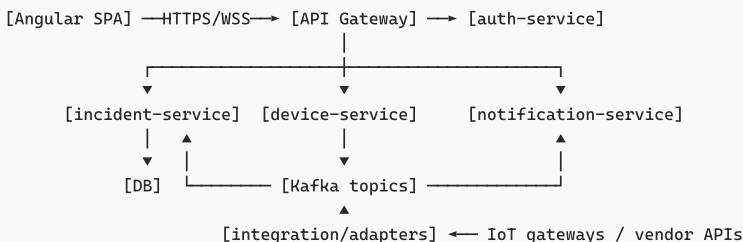
- Draw it. Offer: "Can I sketch the architecture?" Have a 5-box diagram ready: devices → integration → Kafka → services → DB/UI.
- Spend about **20%** on the product and **80%** on your part and your technical decisions.
- Prepare for deep dives on every box you draw: why Kafka, how you handle duplicates, how auth works, how you deploy.
- Use domain vocabulary correctly but explain it briefly (for example, "SOP: the standard operating procedure an operator follows for an incident type").
- Respect your NDA. Never share client secrets, IPs or proprietary algorithms.

What interviewers look for: whether you understand the *business* purpose, end-to-end system thinking, clear ownership, measurable impact, and the ability to explain complexity simply.

E18. Architecture and codebase structure

Template answer

"At a high level we follow [microservices / a modular monolith moving to microservices]. The **Angular** SPA talks to an **API gateway** ([**Spring Cloud Gateway** / **Nginx** / **Kong**]), which routes to the Spring Boot services. Services register in [**Eureka** / **Kubernetes service discovery**] and pull configuration from [**Spring Cloud Config** / **ConfigMaps**]. Synchronous calls use [**REST with OpenFeign/WebClient**]. Asynchronous flows use [**Kafka**]. Each service owns its **database/schema**. Cross-cutting pieces include [**Keycloak/auth-service**] for auth, [**ELK/EFK**] for logs, [**Prometheus + Grafana**] for metrics, and [**Zipkin/Jaeger/OpenTelemetry**] for tracing. Inside each service we use a **layered architecture**: controller → service → repository, with DTOs and mappers at the boundaries."



Typical Spring Boot service layout (layered, package-by-layer within a feature)

```
incident-service/
├─ pom.xml (or build.gradle)
├─ Dockerfile
├─ src/main/java/com/[company]/[product]/incident/
│   └─ IncidentServiceApplication.java
│   └─ config/                # SecurityConfig, KafkaConfig, OpenApiConfig, WebSocketConfig, CorsConfig
│   └─ controller/           # @RestController: IncidentController (thin; validation + mapping only)
│   └─ dto/                  # request/response records: CreateIncidentRequest, IncidentResponse
│   └─ mapper/               # MapStruct mappers: IncidentMapper
│   └─ service/              # interfaces + impl: IncidentService, IncidentServiceImpl (business logic, @Transactional)
```

```

| repository/          # Spring Data JPA: IncidentRepository, specifications/custom queries
| entity/ (or domain/) # @Entity classes: Incident, IncidentStatus (enum), AuditableEntity (base)
| messaging/          # Kafka producers/consumers: IncidentEventPublisher, SensorAlertListener
| client/             # Feign/WebClient clients to other services or third parties
| exception/          # custom exceptions + GlobalExceptionHandler (@RestControllerAdvice)
| security/           # JWT filter, permission evaluators
| scheduler/          # @Scheduled jobs (SLA breach checks, data pulls)
| util/               # small helpers/constants (keep minimal)
| src/main/resources/
| | application.yml    # + application-dev.yml / -qa.yml / -prod.yml (profiles)
| | db/migration/      # Flyway: V1__init.sql, V2__add_incident_priority.sql
| | logback-spring.xml
| | src/test/java/...  # unit tests (Mockito), slice tests (@WebMvcTest, @DataJpaTest), Testcontainers ITs

```

Alternatives to mention: **package-by-feature** (`incident/` , `assignment/` , `sop/` , each with its own controller, service and repository), or **hexagonal / ports & adapters** (`domain/` , `application/` , `adapters/in/web` , `adapters/out/persistence`). Say which one your project uses and why. Shared code (common DTOs, exception model, security utilities) usually lives in a **common library** JAR, which should be kept small to avoid coupling.

Typical Angular layout (feature-folder, standalone)

```

src/
| main.ts                # bootstrapApplication(AppComponent, appConfig)
| app/
| | app.config.ts        # provideRouter, provideHttpClient(withInterceptors(...))
| | app.routes.ts       # top-level lazy routes
| | core/               # singletons, loaded once
| | | auth/             # auth.service.ts, auth.guard.ts, role.guard.ts
| | | interceptors/     # jwt.interceptor.ts, error.interceptor.ts, loading.interceptor.ts
| | | services/         # websocket.service.ts, notification.service.ts
| | | layout/           # shell, header, side-nav (role-based menu)
| | | shared/           # reusable, stateless building blocks
| | | | components/     # data-table, confirm-dialog, map-widget, kpi-card
| | | | directives/     # has-role.directive.ts, severity.directive.ts
| | | | pipes/          # time-ago.pipe.ts
| | | | models/         # shared interfaces / enums
| | | features/         # one folder per business feature, lazy loaded
| | | | dashboard/
| | | | incidents/
| | | | | incident.routes.ts
| | | | | incident-list/ # .ts / .html / .scss / .spec.ts
| | | | | incident-detail/
| | | | | incident-form/
| | | | | data-access/   # incident.service.ts, incident.store.ts (signals/NgRx)
| | | | devices/
| | | | reports/
| | | | admin/
| environments/         # environment.ts, environment.prod.ts (API base URLs; no secrets!)
| assets/               # icons, map styles, i18n json

```

Classic NgModule apps follow the same idea with `CoreModule` (imported once), `SharedModule` , and lazy-loaded `FeatureModule` s using `loadChildren` .

Delivery tips

- Explain **why**: "Each service owns its data so teams can deploy independently", "Kafka decouples the device ingestion rate from processing", "a thin controller keeps business logic testable".
- Mention **conventions**: DTOs never expose entities, a global exception handler, consistent API error format, and versioned APIs (`/api/v1`).
- Be ready to discuss **trade-offs** and **pain points**: distributed transactions, shared libraries, and the cost of local development with many services.

What interviewers look for: whether you know the structure beyond your own folder, separation of concerns, the reasoning behind decisions, and familiarity with cross-cutting concerns (security, configuration, observability).

Likely follow-ups:

- Why use DTOs instead of returning entities?* To decouple the API contract from the schema, avoid lazy-loading and serialisation issues, prevent over-posting, and allow API versioning.
- Package-by-layer vs package-by-feature?* By-layer is simple for small services. By-feature scales better, keeps related code together, and allows package-private encapsulation.
- Where does validation happen?* Format and required checks use Bean Validation on DTOs at the controller. Business rules belong in the service layer. Integrity is enforced by DB constraints.
- How do the services share code?* Through a small versioned common library for DTOs and errors. Avoid sharing domain entities between services.

E19. Technologies and tools used (including those apart from Java, Spring Boot and Angular)

Pick only what you have actually used, and be ready to answer a basic question about each one. Group them; don't recite a flat list.

Template answer

"Apart from Java, Spring Boot and Angular, I've worked with **[messaging]** for event-driven communication, **[databases]**, **[caching]**, and on the DevOps side **[containers/CI]**. For observability we use **[tools]**, and for quality **[tools]**. I've also had exposure to **[cloud/other]**."

Category	Options (tick what is true)	One-line "how I used it" placeholder
Languages	Java [8/11/17/21], TypeScript, JavaScript, SQL, [Python], [Shell/Bash]	"[Used Python for a data-migration script]"
Back-end frameworks	Spring Boot, Spring MVC, Spring Data JPA/Hibernate, Spring Security, Spring Cloud (Gateway, Config, OpenFeign, Eureka), [Spring Batch], [WebFlux], Resilience4j, MapStruct, Lombok	
Front end	Angular, RxJS, [NgRx/SignalStore], [Angular Material/PrimeNG/Bootstrap], [Leaflet/OpenLayers/Google Maps], [ECharts/Chart.js/Highcharts], HTML5, SCSS	
Databases	[MySQL/PostgreSQL/Oracle/SQL Server], [PostGIS for geo], [MongoDB], [Cassandra/TimescaleDB/InfluxDB] for time series, [Elasticsearch] for search	
Caching	[Redis], [Caffeine/Ehcache], Spring Cache	
Messaging / streaming	[Apache Kafka], [RabbitMQ], [MQTT (Mosquitto/EMQX/HiveMQ)] for IoT, [ActiveMQ]	
Real time	WebSocket/STOMP, SockJS, [Server-Sent Events]	
Security	JWT, OAuth2/OIDC, [Keycloak], HTTPS/TLS, [Vault]	
Build / version control	Maven/Gradle, npm, Git ([GitLab/GitHub/Bitbucket]), branching strategy	
CI/CD	[Jenkins/GitLab CI/GitHub Actions], [ArgoCD], [Nexus/Artifactory]	
Containers / orchestration	Docker, docker-compose, [Kubernetes], [Helm], [OpenShift]	
Cloud (if any)	[AWS (EC2, S3, RDS, EKS) / Azure / GCP], or on-prem	
Observability	[ELK/EFK], [Prometheus + Grafana], [Zipkin/Jaeger/OpenTelemetry], Spring Boot Actuator, Micrometer	
Testing	JUnit 5, Mockito, [Testcontainers], [WireMock], Spring Boot Test, [Jasmine/Karma/Jest], [Cypress/Playwright], [Postman/Newman], [JMeter/Gatling] for load tests	
Code quality	SonarQube/SonarLint, Checkstyle/PMD/SpotBugs, ESLint, Prettier, [JaCoCo] coverage	
API docs	Swagger/OpenAPI (springdoc), Postman collections	
Project / collaboration	Jira, Confluence, [Teams/Slack], Figma (for reading designs)	
IDEs	IntelliJ IDEA / Eclipse / STS, VS Code, DBeaver/MySQL Workbench/pgAdmin	
OS / servers	Linux (log analysis, <code>grep</code> / <code>tail</code> / <code>top</code>), Nginx/Apache (reverse proxy), Tomcat (embedded)	

Delivery tips

- Separate **"worked hands-on"** from **"exposure / familiar"**. Being honest here builds trust.
- For each major tool, prepare **one usage sentence** plus **one reason why** ("Redis to cache device metadata, which cut DB calls by [X]%").
- If asked about something you haven't used, say so, then relate it: "Not RabbitMQ in production, but I've used Kafka, and the concepts of exchanges vs topics map like this..."

What interviewers look for: breadth that fits the role, depth in the core stack, honesty about levels of experience, and an understanding of why each tool exists.

E20. Coding standards, best practices and precautions while coding

Template opening

"Our team follows [a documented coding guideline / Google Java Style / an internal checklist], enforced through [SonarQube quality gates, Checkstyle, ESLint] in CI and through mandatory peer review. Personally, I focus on readability, testability, security and observability. Concretely:"

1. Naming and readability

- Classes are `PascalCase` nouns (`IncidentService`). Methods are `camelCase` verbs (`assignOperator`). Constants are `UPPER_SNAKE_CASE` . Packages are lowercase.
- Use meaningful names, not `data` , `temp` or `obj` . Booleans read as questions: `isActive` , `hasPermission` .
- Keep methods small with one level of abstraction. Avoid deep nesting by using guard clauses or early returns.
- Avoid magic numbers: `private static final Duration SLA_BREACH = Duration.ofMinutes(15);`
- Comments should explain **why**, not what. Use Javadoc for public APIs.

2. SOLID and design

Principle	Practical example
Single Responsibility	Controller handles only HTTP, service holds business logic, repository handles data access
Open/Closed	A <code>NotificationSender</code> interface with <code>SmsSender</code> and <code>EmailSender</code> implementations. Add a channel without editing existing code.
Liskov Substitution	Subclasses must not break the parent's contract (no <code>UnsupportedOperationException</code> overrides)
Interface Segregation	Small, focused interfaces, not one <code>GodService</code>
Dependency Inversion	Depend on interfaces; use constructor injection (<code>@RequiredArgsConstructor</code>), not field <code>@Autowired</code>

Also: **DRY** (without premature abstraction), **KISS**, **YAGNI**, composition over inheritance, and design patterns where they genuinely help (Strategy, Factory, Builder, Adapter for vendor integrations).

3. Exception handling

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<ApiError> notFound(ResourceNotFoundException ex, HttpServletRequest req) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiError("NOT_FOUND", ex.getMessage(), req.getRequestURI(), Instant.now()));
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<ApiError> validation(MethodArgumentNotValidException ex, HttpServletRequest req) { /* field errors */ }

    @ExceptionHandler(Exception.class)
    ResponseEntity<ApiError> generic(Exception ex, HttpServletRequest req) {
        log.error("Unhandled error at {}", req.getRequestURI(), ex); // log the stack trace server-side
        return ResponseEntity.internalServerError()
            .body(new ApiError("INTERNAL_ERROR", "Something went wrong", req.getRequestURI(), Instant.now())); // no stack trace to
the client
    }
}
```

- Never swallow exceptions (`catch (Exception e) {}`). Never use `e.printStackTrace()` .
- Throw **specific** custom exceptions, keep the cause (`new XException(msg, e)`), and use try-with-resources.
- Return a consistent error format with correct HTTP status codes (400, 401, 403, 404, 409, 422, 500). Spring 6 also offers `ProblemDetail` (RFC 7807/9457).

4. Logging

- Use SLF4J with **parameterised** messages: `log.info("Incident {} assigned to {}", id, userId)` , not string concatenation.
- Use the right level: `ERROR` for failures needing action, `WARN` for recoverable problems, `INFO` for business events, `DEBUG` for diagnostics (off in production).
- Never log secrets or PII**: passwords, tokens, full phone numbers or Aadhaar-like IDs. Mask them.
- Add a **correlation/trace ID** with MDC so requests can be followed across microservices. Use structured (JSON) logs for ELK.

5. Null safety

- Return `Optional<T>` from finders, and use `orElseThrow(() -> new ResourceNotFoundException(...))`. Don't use `Optional` for fields or parameters.
- Return **empty collections**, never null.
- Use `Objects.requireNonNull`, Bean Validation (`@NotNull`, `@NotBlank`, `@Valid`), and `"CONST".equals(var)` ordering.
- In TypeScript, enable `strict` mode and use optional chaining `?.` and `??`.

6. Security precautions (OWASP Top 10 mindset)

- **Never hardcode secrets** (passwords, API keys, JWT signing keys) in code or `application.yml` in Git. Use environment variables, **[Vault / Kubernetes Secrets / a cloud secret manager]**, and Jasypt if required. Add `.gitignore` rules and secret scanning (for example gitleaks).
- **SQL injection**: use parameterised queries, JPA parameters and `PreparedStatement`. Never concatenate user input into SQL.
- **Validate input** on the server (never trust the UI), and encode output (Angular escapes interpolation by default; avoid `bypassSecurityTrust*`).
- **AuthN/AuthZ**: method-level `@PreAuthorize`, and check ownership to prevent IDOR. Apply least privilege for DB users.
- Use HTTPS only, a CORS allow-list (not `*` with credentials), and rate limiting at the gateway.
- **Dependencies**: OWASP Dependency-Check, Snyk or Dependabot, and keep libraries patched.
- Do not expose entities directly (mass-assignment risk). Use DTOs.

7. Testing

- Write unit tests for business logic (JUnit 5 + Mockito), following **Arrange-Act-Assert** with one behaviour per test and descriptive names (`shouldEscalateWhenSLABreached`).
- Use slice tests (`@WebMvcTest`, `@DataJpaTest`) and integration tests with **Testcontainers** (a real Postgres or Kafka).
- Follow the coverage gate (**[e.g. 80% on new code]**), but focus on meaningful assertions, not just coverage numbers.
- Every bug fix gets a regression test first: reproduce, fix, verify.

8. Code reviews and static analysis

- Use small, focused PRs with a clear description and linked ticket. Self-review before requesting review.
- The reviewer checklist covers correctness, edge cases, naming, tests, security, performance (N+1, missing indexes, unbounded queries), logging and backward compatibility.
- Enforce a **SonarQube** quality gate in CI: no new blocker or critical issues, no new code smells above the threshold, and minimum coverage on new code. Use SonarLint in the IDE.
- Use formatting and lint tools (Checkstyle/Spotless, ESLint/Prettier) so style is automatic, not debated.

9. Git branching and commits

- Branching follows **[GitFlow (main, develop, feature/*, release/*, hotfix/*) / trunk-based with short-lived feature branches]**.
- Use meaningful commits, for example Conventional Commits: `feat(incident): add SLA escalation`, or `fix: handle null device location`.
- Never commit directly to `main`. Branch protection requires approvals and green CI. Rebase or squash according to team policy.
- Tag releases and keep a changelog.

10. Performance and database precautions

- Avoid N+1 queries (`JOIN FETCH`, `@EntityGraph`). Paginate all list APIs. Select only the columns you need.
- Index on filter and join columns, and check plans with `EXPLAIN`.
- Keep transactions short. Avoid remote calls inside `@Transactional`.
- Close resources, use timeouts on all HTTP clients, and use connection pool sizing (HikariCP).
- Use backward-compatible DB migrations (expand then contract) for zero-downtime deployments.

11. Angular-specific standards

- Follow the Angular style guide (one component per file, feature folders), use `OnPush` or signals, use the `async` pipe and `takeUntilDestroyed`, and use typed forms and strict TypeScript.
- Use `track` in `@for`, avoid function calls in templates (use `computed` or pure pipes), and avoid `any`.

Delivery tips

Pick **5–6 points** and give a **real example** for each ("In one review I caught a query without pagination that would have loaded [N] records"). Mention **automation**: standards that are enforced by tools are more credible than standards that depend on memory.

What interviewers look for: professional maturity, a security and quality mindset, team practices (reviews, CI gates), and concrete habits rather than buzzwords.

Likely follow-ups:

- *Why constructor injection over field injection?* It allows immutable (`final`) dependencies, makes dependencies explicit, works in tests without Spring, and fails fast on circular dependencies.
- *What does a SonarQube quality gate check?* Conditions on new code: bugs, vulnerabilities, security hotspots reviewed, code smells, coverage percentage and duplication. CI fails if the gate fails.
- *Checked vs unchecked exceptions?* Checked exceptions are for recoverable conditions the caller must handle. Unchecked exceptions are for programming errors or business violations. Spring rolls back on unchecked exceptions by default.
- *How do you store secrets in Kubernetes?* In Secrets, ideally synced from Vault or a cloud secret manager through an external secrets operator, mounted as environment variables or files, and never committed in manifests.

E21. Challenges faced

Use the STAR method (aim for about 2 minutes)

Step	Share	What to say
Situation	15%	Context: product, module, when, why it mattered
Task	10%	Your responsibility or goal
Action	50–60%	The steps you took: analysis, options considered, the decision, the implementation
Result	15–20%	A measurable outcome, plus what you learned or changed to prevent a repeat

Blank template:

"**S:** In [month/year], on [module] of [product], we faced [problem], which was affecting [users/SLA/business]. **T:** I was responsible for [diagnosing / fixing / delivering] [X] by [deadline]. **A:** First I [investigated with tools/data]. I found [root cause]. I considered [option A] and [option B], and chose [B] because [reason]. I implemented [specifics], tested it with [how], and rolled it out [how, safely]. **R:** [Metric before → after]. We also [added monitoring / a test / a runbook] so it doesn't recur. I learned [lesson]."

The three scenarios below are **EXAMPLES ONLY**. They are plausible situations for a Smart City full-stack developer, written to show the structure. **Do not narrate them as your own** unless they actually happened. Replace them with your real stories, borrowing only the structure and the technical vocabulary.

Example 1 (illustrative): performance issue on the incident dashboard

- **S:** "The operator dashboard's incident list API took [**~8 s**] at peak once a city crossed [**N lakh**] incident records. Operators complained that the screen froze during [**festival / heavy-rain**] events."
- **T:** "I was asked to bring the p95 under [**1 s**] without a schema redesign."
- **A:** "I enabled SQL logging and used `EXPLAIN`. I found (1) an **N+1** problem loading `assignedOfficer` and `location` lazily for every row, (2) a **missing composite index** on `(city_id, status, created_at)`, and (3) the API returning **all** records with client-side paging. I added `@EntityGraph / JOIN FETCH`, created the index through a Flyway migration (built concurrently in Postgres to avoid locking), switched to server-side **pagination** with a projection DTO, and cached reference data (incident types, zones) in **Redis**. On the Angular side I moved the list to **OnPush** with virtual scrolling."
- **R:** "p95 dropped from [**8 s**] to [**~400 ms**], DB CPU fell [**X**]%, and I added a Grafana alert on API latency. Lesson: always paginate and profile queries with production-sized data."

Example 2 (illustrative): production bug with duplicate alerts

- **S:** "After a release, operators saw **duplicate alerts** for the same sensor event, sometimes 3–4 copies, which caused confusion in the control room."
- **T:** "As the owner of the alert consumer, I had to find the root cause quickly and fix it without losing alerts."
- **A:** "I correlated logs by event ID in [**Kibana**] and saw the same Kafka offset processed several times. A slow downstream call had pushed processing past `max.poll.interval.ms`, so the consumer group **rebalanced** and the messages were redelivered (Kafka's at-least-once delivery). As an immediate mitigation, I raised the poll interval and reduced `max.poll.records`. For the permanent fix, I made the consumer **idempotent**, using a unique constraint on `(source_id, event_id)` with insert-if-absent, and moved the slow call to an async step. I added a regression test with **Testcontainers Kafka**."
- **R:** "Duplicates went to **zero**. We documented 'every consumer must be idempotent' in our review checklist. Lesson: design for at-least-once delivery from day one."

Example 3 (illustrative): integrating a third-party IoT data source

- **S:** "We had to onboard [a vendor's air-quality / parking / GPS] devices. The vendor API was poorly documented, rate-limited ([N] calls/min), returned inconsistent units and timestamps in local time without a zone, and was down intermittently."
- **T:** "Deliver a reliable integration in [2 sprints] that feeds the dashboard and alert engine."
- **A:** "I built an **adapter service**: a scheduled poller using `WebClient` with **timeouts, retries with exponential backoff, and a Resilience4j circuit breaker**, plus a token-bucket limiter to respect the rate limit. I stored the **raw payload** for audit and replay, then **normalised** it into our canonical schema (UTC timestamps, SI units, a validated range). Invalid records went to a **dead-letter table** with the reason. Normalised events were published to **Kafka**. I wrote **contract tests with WireMock** using recorded vendor responses, and agreed a field-mapping document with the vendor."
- **R:** "We went live on schedule and reached [99.x]% data completeness. Vendor outages no longer affected other services, and the same adapter pattern was reused for [N] more vendors. Lesson: treat external data as untrusted and build in observability for it."

Other challenge types to prepare: a tight deadline or changing requirements, learning a new technology quickly, a legacy code refactor, a difficult deployment or rollback, and working with an onsite or client team.

Delivery tips

- Prepare **3–4 stories** that can be adapted to many questions (challenge, failure, conflict, leadership, learning).
- Pick challenges that were **technical and impactful**, where **you** drove the solution.
- Be honest about mistakes. "I caused it, here is how I fixed it, and here is the process we changed" is a strong answer.
- Avoid blaming people. Focus on the system and process.

What interviewers look for: structured thinking, debugging methodology, ownership, technical depth, measurable outcomes and learning.

E22. Handling conflicts

Principles to mention

- Separate the **person** from the **problem**. Listen first and restate the other view.
- Use **data and shared goals** (customer impact, SLA, deadline), not opinions or seniority.
- Try to resolve it **1:1 first**. **Escalate** only with options and data, never as a complaint.
- **Disagree and commit** once a decision is made, and document the decision (for example in an ADR).

STAR template

"**S:** On [project], [colleague/role] and I disagreed about [topic]. **T:** We needed to [decide/deliver] by [deadline] without hurting [quality/relationship]. **A:** I set up a short discussion to understand their reasoning, which was [their concern]. I explained mine: [my concern]. We listed the criteria that mattered ([e.g. delivery time, performance, maintainability]). I [built a quick POC / gathered metrics / asked the architect for input]. We agreed on [decision / compromise]. **R:** [Outcome]. Our working relationship [improved]. I learned [lesson]."

The two scenarios below are **EXAMPLES** showing the structure. Use your own real situation.

Example A (illustrative): technical disagreement

- **S:** "A senior colleague wanted the new alert-notification feature to call the notification service **synchronously over REST**. I preferred publishing an event to **Kafka**."
- **T:** "We had to finalise the design in the sprint's first week."
- **A:** "I asked what worried him. It was mainly the extra complexity and harder debugging of async flows. I acknowledged that, then showed the risk of the synchronous approach: if the SMS gateway slows down, incident creation slows down too. I built a **one-day POC** and measured both under a simulated slow gateway. Sync incident creation went from [150 ms] to [3 s]; async stayed at [150 ms]. To address his concern, I proposed adding a **correlation ID**, a **dead-letter topic**, and a dashboard for consumer lag."
- **R:** "We went with Kafka plus the observability additions he asked for. The design held up during a later SMS-gateway outage with no impact on incident creation. Lesson: the other person's concern often improves the final design."

Example B (illustrative): team or process conflict

- **S:** "QA and development kept clashing over late-reported bugs. QA felt builds came too late; developers felt requirements kept changing."
- **T:** "As [the module owner / a developer on the team], I wanted to reduce friction before a major city go-live."
- **A:** "I talked to the QA lead one-on-one, then suggested three changes in the retro: (1) QA joins **story refinement** to add acceptance criteria up front, (2) we deploy to the QA environment **daily** from CI instead of at sprint end, and (3) we agree a **definition of done** that includes unit tests and a passing API test collection."

- **R:** "Late-sprint defects dropped by **[X]%**, and go-live went smoothly. Lesson: most people conflicts are really process gaps."

Other variants to prepare: disagreeing with your manager about an estimate, a teammate not delivering their part, and a client or stakeholder pushing scope late.

Delivery tips

- Never paint the other person as the villain. Show empathy and respect.
- Make sure the story ends in a **resolution**, not "I was right".
- Keep it professional. Avoid personal or emotional conflicts.

What interviewers look for: emotional maturity, communication, collaboration, a data-driven approach to decisions, and the ability to commit to a team decision.

E23. Integrating third-party data

Integration patterns

Pattern	When	Spring tools
REST pull (polling)	Vendor exposes an API; data changes periodically	<code>WebClient</code> / <code>RestClient</code> / <code>OpenFeign</code> + <code>@Scheduled</code> (+ <code>ShedLock</code> when running several instances)
Webhooks (push)	Vendor calls us on events	A <code>@RestController</code> endpoint with HMAC signature verification, returning 2xx fast and processing asynchronously
Message/stream ingestion	High-volume events, IoT	Kafka (<code>@KafkaListener</code>), MQTT (Spring Integration MQTT / Paho) → Kafka bridge
File / SFTP / batch	Daily dumps (CSV, XML, Excel)	Spring Batch, Spring Integration SFTP, <code>@Scheduled</code>
Database / CDC	Partner database or legacy system	Debezium CDC → Kafka, or read replicas / views
Real-time streams	Vendor WebSocket or SSE	Reactor Netty <code>WebSocketClient</code> / <code>WebClient</code> SSE

Pipeline (whatever the transport)

```
Source → Adapter (auth, rate limit, retry, circuit breaker)
      → Store RAW (audit/replay) → Validate → Map/Normalise (canonical model)
      → Deduplicate (idempotency key) → Persist normalised + publish event (Kafka)
      → Invalid → dead-letter (table/topic) + alert
      → Metrics: fetched/accepted/rejected counts, lag, last-success timestamp
```

Key concerns to discuss

- **Authentication:** API keys, OAuth2 client credentials, mutual TLS. Store secrets in a vault, never in code.
- **Mapping:** an anti-corruption layer that translates vendor fields into our **canonical domain model**. Use `MapStruct` or a dedicated mapper. Make field mappings configurable per vendor where possible.
- **Validation:** mandatory fields, value ranges (for example, AQI from 0 to 500), timestamps (convert to **UTC**, reject future-dated readings), units, and enums. Use Bean Validation plus custom rules.
- **Retries:** exponential backoff with jitter. Retry only **idempotent** calls and **transient** errors (timeouts, 5xx, 429). Do not retry 4xx validation errors.
- **Rate limits:** a client-side limiter (`Resilience4j RateLimiter`), respect `Retry-After`, batch requests, and use incremental fetches (`since=lastSyncTime`, cursor or ETag).
- **Circuit breaker and bulkhead:** stop hammering a failing vendor, and isolate thread pools so one vendor cannot take down the service.
- **Idempotency:** use a natural key (`vendor + externalId + timestamp`) with a unique constraint and upsert. Webhooks are redelivered and Kafka is at-least-once, so **duplicates will happen**.
- **Store raw and normalised:** raw JSON (in a JSONB column, object storage or a Kafka topic) for audit, debugging, and **reprocessing** when mapping logic changes. The normalised tables serve queries and the UI.
- **Ordering and late data:** use event time rather than processing time. Handle out-of-order data with last-write-wins on `event_ts`.
- **Schema evolution:** a tolerant reader (`@JsonIgnoreProperties(ignoreUnknown = true)`), version adapters, and contract tests.
- **Monitoring:** Micrometer counters (fetched, accepted, rejected), a "last successful sync" gauge, an alert when data is stale for more than N minutes, and a correlation ID in logs. Show data health on an ops dashboard.
- **Testing:** WireMock with recorded vendor responses, Testcontainers, and a sandbox vendor environment.

Code sketch

```
// 1) Client with timeouts, retry, circuit breaker and rate limiter
@Component
@RequiredArgsConstructor
@Slf4j
public class AqiVendorClient {
    private final WebClient vendorWebClient; // built with base URL, auth filter, connect/read timeouts

    @CircuitBreaker(name = "aqiVendor", fallbackMethod = "fallback")
    @RateLimiter(name = "aqiVendor")
    @Retry(name = "aqiVendor") // backoff configured in application.yml; retries on IOException/5xx/429
    public List<VendorReading> fetchSince(Instant since) {
        return vendorWebClient.get()
            .uri(uri -> uri.path("/v1/readings").queryParam("since", since.toString()).build())
            .retrieve()
            .bodyToFlux(VendorReading.class)
            .collectList()
            .block(Duration.ofSeconds(30)); // blocking is fine inside a scheduler thread
    }

    private List<VendorReading> fallback(Instant since, Throwable t) {
        log.warn("AQI vendor unavailable since={}, cause={}", since, t.toString());
        return List.of(); // skip this cycle; the watermark is not advanced
    }
}

// 2) Scheduled poller with a watermark (incremental sync), safe across instances
@Component
@RequiredArgsConstructor
public class AqiSyncJob {
    private final AqiVendorClient client;
    private final IngestionService ingestion;
    private final SyncStateRepository syncState;

    @Scheduled(fixedDelayString = "${integration.aqi.poll-ms:60000}")
    @SchedulerLock(name = "aqiSync", lockAtMostFor = "PT5M") // ShedLock: only one instance runs it
    public void sync() {
        Instant since = syncState.getWatermark("AQI").orElse(Instant.now().minus(Duration.ofHours(1)));
        List<VendorReading> batch = client.fetchSince(since);
        if (batch.isEmpty()) return;
        ingestion.ingest("AQI", batch);
        syncState.saveWatermark("AQI", batch.stream().map(VendorReading::observedAt)
            .max(Comparator.naturalOrder()).orElse(since));
    }
}

// 3) Ingestion: raw store → validate → map → idempotent upsert → publish
@Service
@RequiredArgsConstructor
public class IngestionService {
    private final RawPayloadRepository rawRepo;
    private final SensorReadingRepository readingRepo;
    private final DeadLetterRepository dlq;
    private final ReadingMapper mapper;
    private final Validator validator;
    private final KafkaTemplate<String, SensorReadingEvent> kafka;
    private final MeterRegistry metrics;

    @Transactional
    public void ingest(String source, List<VendorReading> batch) {
        for (VendorReading r : batch) {
            rawRepo.save(RawPayload.of(source, r.externalId(), toJson(r))); // keep raw for audit/replay

            SensorReading reading = mapper.toCanonical(r); // units → SI, time → UTC
            Set<ConstraintViolation<SensorReading>> errors = validator.validate(reading);
            if (!errors.isEmpty()) {
                dlq.save(DeadLetter.of(source, r.externalId(), errors.toString()));
                metrics.counter("ingest.rejected", "source", source).increment();
                continue;
            }
            // Idempotent: unique (source, external_id) → INSERT ... ON CONFLICT DO NOTHING / upsert
            boolean inserted = readingRepo.insertIfAbsent(reading) == 1;
            if (inserted) {
                metrics.counter("ingest.accepted", "source", source).increment();
                // For strict consistency use the transactional outbox pattern instead of a direct send
                kafka.send("sensor.readings.normalized", reading.sensorId(), SensorReadingEvent.from(reading));
            }
        }
    }
}
```

```
// Webhook variant: verify, store, acknowledge fast, process async
@PostMapping("/webhooks/parking")
public ResponseEntity<Void> parkingWebhook(@RequestHeader("X-Signature") String sig,
                                           @RequestBody String body) {
    if (!HmacVerifier.isValid(body, sig)) return ResponseEntity.status(401).build();
    inboundQueue.publish(body);           // e.g. to Kafka; processed by an idempotent consumer
    return ResponseEntity.accepted().build(); // 202 quickly, so the vendor does not time out and retry
}
```

```
resilience4j:
  retry:
    instances:
      aqiVendor: { maxAttempts: 3, waitDuration: 2s, enableExponentialBackoff: true, exponentialBackoffMultiplier: 2 }
  circuitbreaker:
    instances:
      aqiVendor: { slidingWindowSize: 20, failureRateThreshold: 50, waitDurationInOpenState: 60s }
  ratelimiter:
    instances:
      aqiVendor: { limitForPeriod: 50, limitRefreshPeriod: 1m, timeoutDuration: 5s }
```

Template answer for "Have you integrated third-party data?"

"Yes. In [project] we integrated [vendor/system: e.g. traffic signal controllers, GPS trackers, weather API, CCTV VMS, SMS gateway] using [pattern]. I built [adapter/poller/consumer] with [retries/circuit breaker/rate limiting]. We stored raw and normalised data, made processing idempotent with [key], and monitored [metrics]. The main challenge was [inconsistent data/rate limits/downtime], and I solved it by [approach]."

Likely follow-ups:

- *How do you handle vendor downtime?* A circuit breaker plus fallback, do not advance the watermark, catch up on recovery, and alert when data goes stale.
- *Polling vs webhooks?* Polling is simple and under our control, but adds latency and wastes calls. Webhooks are real time but need a public endpoint, signature verification, idempotency, and reconciliation polling as a safety net.
- *How do you reprocess data after a mapping bug?* Replay from the stored raw payloads (or the Kafka raw topic) through the fixed mapper, using idempotent upserts.
- *What about large files?* Spring Batch chunk processing (read, process, write), with skip/retry policies, restartable jobs, and streaming parsers rather than loading everything into memory.

E24. The microservices concept in your project, and how you handle microservices architecture

Cross-reference: the microservices patterns themselves (service discovery, API gateway, config server, circuit breaker, Saga, outbox, distributed tracing, inter-service communication) are covered in depth in **Section D**. This answer is the **project-story template** that ties them to your experience.

Template answer

"Our platform is split into about [N] microservices, organised by **business capability**: for example [auth/user-service, device-service, event/alert-service, incident-service, notification-service, gis-service, report-service, integration adapters]. Each is a **Spring Boot** application with **its own database/schema**, packaged as a **Docker** image and deployed on [Kubernetes/OpenShift/VMs] through [Jenkins/GitLab CI]."

"**Client entry**: The Angular app calls an **API gateway** ([Spring Cloud Gateway/Nginx/Kong]), which handles routing, JWT validation, CORS and rate limiting."

"**Service-to-service communication**: For synchronous queries we use [OpenFeign/WebClient] with **timeouts, retries and Resilience4j circuit breakers**. For events such as *SensorThresholdBreached* or *IncidentCreated* we use **Kafka**, so producers and consumers are decoupled and can scale independently."

"**Discovery and configuration**: [Eureka / Kubernetes DNS] for discovery, and [Spring Cloud Config / ConfigMaps + Secrets] for externalised config per environment."

"**Data consistency**: There are no distributed transactions. We use [Saga choreography with events / the transactional outbox] and **idempotent consumers**. For example, when an incident is created, the notification and assignment services react to events; if something fails, we [retry / use the DLQ / compensate]."

"**Observability:** a correlation/trace ID through **[Micrometer Tracing + Zipkin/Jaeger/OpenTelemetry]**, centralised logs in **[ELK]**, metrics in **[Prometheus/Grafana]**, Actuator health probes for Kubernetes liveness and readiness, and alerts on error rate, latency and Kafka consumer lag."

"**Security:** **[Keycloak/OAuth2]** issues tokens. The gateway validates them, and services also act as **resource servers** with role checks (`@PreAuthorize`). Service-to-service calls use **[client credentials / mTLS]**."

"**My part:** I own / owned **[service(s)]**. I **[designed its APIs and events, wrote the Kafka consumers, added circuit breakers, wrote contract tests, handled deployment configuration]**. One concrete example is **[story from E21]**."

How you "handle" microservices day to day (talking points)

- **Service boundaries:** DDD bounded contexts, with one team or service owning one capability and its data. Avoid a "distributed monolith" with chatty synchronous chains.
- **API contracts and versioning:** OpenAPI specs, `/v1` and `/v2` versions, backward-compatible changes, and consumer-driven contract tests ([Spring Cloud Contract/Pact]).
- **Resilience:** timeouts everywhere, retries with backoff, circuit breakers, bulkheads, fallbacks, and graceful degradation (the dashboard still loads if analytics is down).
- **Deployment:** independent pipelines, rolling or blue-green deployments, feature flags, and backward-compatible DB migrations.
- **Local development:** docker-compose for dependencies (Kafka, Postgres, Redis), and Testcontainers in tests.
- **Debugging across services:** start from the trace ID in the UI error or log, then look at the Zipkin trace to find the slow or failing service, then at that service's logs.
- **Scaling:** horizontal scaling of stateless services (Kubernetes HPA), Kafka partitions for consumer parallelism, and caching hot reads in Redis.

Honest variants

- *If your project is mostly a monolith:* "Our core is a **modular monolith**, with [N] separate services for [integration/notification]. I understand the trade-offs: we split out [X] because it needed independent scaling. Here is how I would decompose further..."
- *If someone else set up the infrastructure:* "The platform team owns Kubernetes and the gateway. I consume them: I write the Helm values, health checks and configuration for my service."

Delivery tips

- Draw the services and arrows. Mark sync and async calls differently.
- Mention **at least one real problem** (cascading failure, duplicate events, a debugging difficulty) and how you solved it. That proves hands-on experience.
- Don't claim patterns you didn't use. "We didn't need Sagas because..." is a fine answer.

What interviewers look for: a real understanding of *why* microservices (and their costs), communication and consistency strategies, resilience and observability practice, and clear personal ownership.

Likely follow-ups:

- *How do you handle a transaction spanning services?* A Saga (choreography or orchestration) with compensating actions, plus the outbox pattern for reliable event publishing. See Section D.
- *What happens if a downstream service is down?* The circuit breaker opens, a fallback is used (cached or default data, or queue for later), the user gets a degraded but working UI, and alerts fire.
- *How do you keep services from becoming too chatty?* Coarser APIs, event-carried state transfer (local read models), and BFF/aggregation at the gateway.
- *When would you NOT use microservices?* With a small team, an unclear domain, or an early-stage product. A modular monolith is cheaper to build and operate.

E25. "Given a list of strings, write..." (truncated question)

Note: the source question (G2Healthcare Q26) was **cut off** after "Given a list of strings, write...". **Candidate:** please supply the complete question text so it can be answered exactly. Meanwhile, here are full solutions to the three most likely completions, using Java Streams.

Sample input used throughout:

```
List<String> words = List.of("apple", "bat", "cat", "banana", "kiwi", "apple", "avocado");
```

Completion 1: group the strings by length

```
import java.util.*;
import java.util.stream.*;

Map<Integer, List<String>> byLength = words.stream()
    .collect(Collectors.groupingBy(String::length));
// HashMap: order is not guaranteed by contract

// Sorted by length, with duplicates removed and each group sorted
Map<Integer, Set<String>> byLengthSorted = words.stream()
    .collect(Collectors.groupingBy(
        String::length,
        TreeMap::new, // sorted keys
        Collectors.toCollection(TreeSet::new))); // distinct and sorted values

System.out.println(byLengthSorted);
```

Output:

```
{3=[bat, cat], 4=[kiwi], 5=[apple], 6=[banana], 7=[avocado]}
```

Variations:

```
// Count per length
Map<Integer, Long> countByLength = words.stream()
    .collect(Collectors.groupingBy(String::length, TreeMap::new, Collectors.counting()));
// {3=2, 4=1, 5=2, 6=1, 7=1}

// Partition into short (<5) and long (>=5)
Map<Boolean, List<String>> parts = words.stream()
    .collect(Collectors.partitioningBy(s -> s.length() >= 5));
// {false=[bat, cat, kiwi], true=[apple, banana, apple, avocado]}
```

Completion 2: filter strings starting with a given letter, then sort

```
// Starting with 'a', distinct, sorted alphabetically
List<String> aWords = words.stream()
    .filter(Objects::nonNull) // defensive: skip nulls
    .filter(s -> s.startsWith("a"))
    .distinct()
    .sorted()
    .collect(Collectors.toList()); // Java 16+: .toList() (unmodifiable)
System.out.println(aWords); // [apple, avocado]

// Case-insensitive, sorted by length then alphabetically, uppercase output
List<String> bWords = words.stream()
    .filter(s -> s != null && !s.isEmpty() && Character.toLowerCase(s.charAt(0)) == 'b')
    .sorted(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder()))
    .map(String::toUpperCase)
    .toList();
System.out.println(bWords); // [BAT, BANANA]

// Reusable method
static List<String> startingWith(List<String> list, String prefix) {
    return list.stream()
        .filter(s -> s != null && s.regionMatches(true, 0, prefix, 0, prefix.length()))
        .sorted(String.CASE_INSENSITIVE_ORDER)
        .toList();
}
```

Plain `sorted()` uses natural order, which is lexicographic and case-sensitive: uppercase letters sort before lowercase, so `"Zebra" < "apple"`. Use `String.CASE_INSENSITIVE_ORDER` when case should not matter.

Completion 3: frequency of each string

```
// Frequency map, preserving first-seen order
Map<String, Long> freq = words.stream()
    .collect(Collectors.groupingBy(
        Function.identity(),
        LinkedHashMap::new,
        Collectors.counting()));
System.out.println(freq);
// {apple=2, bat=1, cat=1, banana=1, kiwi=1, avocado=1}

// Only the duplicates
List<String> duplicates = freq.entrySet().stream()
```

```

        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .toList(); // [apple]

// Most frequent string
String mostFrequent = freq.entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse(null); // "apple"

// First non-repeating string (relies on LinkedHashMap encounter order)
String firstUnique = freq.entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null); // "bat"

// Sorted by frequency descending, then alphabetically
List<Map.Entry<String, Long>> ranked = freq.entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .thenComparing(Map.Entry.comparingByKey())
    .toList();
// [apple=2, avocado=1, banana=1, bat=1, cat=1, kiwi=1]

// Alternative without streams
Map<String, Integer> freq2 = new HashMap<>();
for (String w : words) freq2.merge(w, 1, Integer::sum);

```

Case-insensitive frequency: `groupingBy(s -> s.toLowerCase(Locale.ROOT), LinkedHashMap::new, counting())`. Character frequency inside a single string is another common variant:

```

Map<Character, Long> charFreq = "banana".chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
// {b=1, a=3, n=2}

```

Other possible completions (quick answers)

```

String longest = words.stream().max(Comparator.comparingInt(String::length)).orElse(""); // "avocado"
String joined = words.stream().distinct().collect(Collectors.joining(", ", "[", "]")); // [apple, bat, cat, banana, kiwi, avocado]
List<String> reversed = words.stream().map(s -> new StringBuilder(s).reverse().toString()).toList();
Map<Character, List<String>> byFirstLetter = words.stream()
    .collect(Collectors.groupingBy(s -> s.charAt(0), TreeMap::new, Collectors.toList()));
// {a=[apple, apple, avocado], b=[bat, banana], c=[cat], k=[kiwi]}
List<String> palindromes = Stream.of("level", "kiwi", "madam")
    .filter(s -> new StringBuilder(s).reverse().toString().equals(s)).toList(); // [level, madam]

```

Likely follow-ups:

- *Why does `groupingBy` need `TreeMap::new` or `LinkedHashMap::new`?* The default is a `HashMap`, which guarantees no order. Supply a map factory for sorted or insertion-ordered output.
- *`toList()` vs `Collectors.toList()`?* Java 16's `Stream.toList()` returns an **unmodifiable** list. `Collectors.toList()` returns a mutable `ArrayList` in practice, though the spec does not guarantee that.
- *How do you handle nulls in the list?* Add `filter(Objects::nonNull)` first. `groupingBy` throws an NPE if the classifier returns a null key.
- *Parallel stream here?* Not worth it for small lists. For large lists, use `groupingByConcurrent` and avoid order-dependent operations.